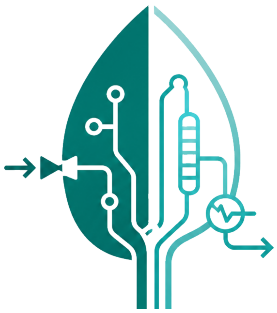

Choupo

The Open Source Chemical Process Simulator

Theory Guide

Version 2607

30 June 2026

Authors: Vítor Geraldes, Pedro Mendes, and Miguel Rodrigues.

Typeset in LaTeX.

Acknowledgment. Choupo was conceived and architected by Vítor Geraldes. Substantial parts of the initial implementation, documentation, and tutorial corpus were produced with assistance from Anthropic Claude Code using Claude Opus/Fable models. This guide is published under human curation, review, and editorial responsibility by Vítor Geraldes, Pedro Mendes, and Miguel Rodrigues.

License

Rights.

e. Effective Technological Measures means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.

f. Exceptions and Limitations means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.

g. License Elements means the license attributes listed in the name of a Creative Commons Public License. The License Elements of this Public License are Attribution and ShareAlike.

h. Licensed Material means the artistic or literary work, database, or other material to which the Licensor applied this Public License.

i. Licensed Rights means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensor has authority to license.

j. Licensor means the individual(s) or entity(ies) granting rights under this Public License.

k. Share means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.

l. Sui Generis Database Rights means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.

m. You means the individual or entity exercising the Licensed Rights under this Public License. Your has a corresponding meaning.

Section 2 -- Scope.

a. License grant.

1. Subject to the terms and conditions of this Public License, the Licensor hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:

a. reproduce and Share the Licensed Material, in whole or in part; and

b. produce, reproduce, and Share Adapted Material.

2. Exceptions and Limitations. For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.

3. Term. The term of this Public License is specified in Section 6(a).

4. Media and formats; technical modifications allowed. The Licensor authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensor waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a) (4) never produces Adapted Material.

5. Downstream recipients.

a. Offer from the Licensor -- Licensed Material. Every recipient of the Licensed Material automatically receives an offer from the Licensor to exercise the Licensed Rights under the terms and conditions of this Public License.

b. Additional offer from the Licensor -- Adapted Material. Every recipient of Adapted Material from You automatically receives an offer from the Licensor to exercise the Licensed Rights in the Adapted Material under the conditions of the Adapter's License You apply.

c. No downstream restrictions. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

6. No endorsement. Nothing in this Public License constitutes or
   may be construed as permission to assert or imply that You
   are, or that Your use of the Licensed Material is, connected
   with, or sponsored, endorsed, or granted official status by,
   the Licensor or others designated to receive attribution as
   provided in Section 3(a)(1)(A)(i).

b. Other rights.

1. Moral rights, such as the right of integrity, are not
   licensed under this Public License, nor are publicity,
   privacy, and/or other similar personality rights; however, to
   the extent possible, the Licensor waives and/or agrees not to
   assert any such rights held by the Licensor to the limited
   extent necessary to allow You to exercise the Licensed
   Rights, but not otherwise.

2. Patent and trademark rights are not licensed under this
   Public License.

3. To the extent possible, the Licensor waives any right to
   collect royalties from You for the exercise of the Licensed
   Rights, whether directly or through a collecting society
   under any voluntary or waivable statutory or compulsory
   licensing scheme. In all other cases the Licensor expressly
   reserves any right to collect such royalties.

Section 3 -- License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the
following conditions.

a. Attribution.

1. If You Share the Licensed Material (including in modified
   form), You must:

   a. retain the following if it is supplied by the Licensor
      with the Licensed Material:

      i. identification of the creator(s) of the Licensed
         Material and any others designated to receive
         attribution, in any reasonable manner requested by
         the Licensor (including by pseudonym if
         designated);

      ii. a copyright notice;

      iii. a notice that refers to this Public License;

      iv. a notice that refers to the disclaimer of
          warranties;

      v. a URI or hyperlink to the Licensed Material to the
         extent reasonably practicable;

   b. indicate if You modified the Licensed Material and
      retain an indication of any previous modifications; and

   c. indicate the Licensed Material is licensed under this
      Public License, and include the text of, or the URI or
      hyperlink to, this Public License.

2. You may satisfy the conditions in Section 3(a)(1) in any
   reasonable manner based on the medium, means, and context in
   which You Share the Licensed Material. For example, it may be
   reasonable to satisfy the conditions by providing a URI or
   hyperlink to a resource that includes the required
   information.

3. If requested by the Licensor, You must remove any of the
   information required by Section 3(a)(1)(A) to the extent
   reasonably practicable.

b. ShareAlike.

   In addition to the conditions in Section 3(a), if You Share
   Adapted Material You produce, the following conditions also apply.

1. The Adapter's License You apply must be a Creative Commons
   license with the same License Elements, this version or
   later, or a BY-SA Compatible License.

2. You must include the text of, or the URI or hyperlink to, the
   Adapter's License You apply. You may satisfy this condition
   in any reasonable manner based on the medium, means, and
   context in which You Share Adapted Material.

3. You may not offer or impose any additional or different terms
   or conditions on, or apply any Effective Technological
   Measures to, Adapted Material that restrict exercise of the
   rights granted under the Adapter's License You apply.

Section 4 -- Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that
apply to Your use of the Licensed Material:

Trademarks

Choupo is a trademark of TalentGround Lda.

Anthropic, Claude, and Claude Code are trademarks or registered trademarks of Anthropic, PBC.

OpenFOAM is a registered trademark of OpenCFD Ltd.

REFPROP and NIST WebBook are products or services of the National Institute of Standards and Technology.

PHREEQC is a software product of the U.S. Geological Survey.

Contents

Preface — how to read this manual

Every equation in this manual maps to lines of source code you can grep for, and to runnable cases in `tutorials/`. The goal is that nothing in the simulator is unaccounted for in writing. If you find a mismatch — a line of code without a justification here, or a derivation here that does not match what the simulator prints — that is a bug.

The structure is the obvious one. Foundations first — notation, equilibrium thermodynamics, K-values and Rachford-Rice. Then the families of models the simulator carries (vapour pressure, activity, EoS, heat capacity). Then the numerical machinery the models are solved with. Finally, the unit operations that compose them into a flowsheet.

Each chapter follows the same rhythm:

1. Motivation — why this object exists.
2. Hypotheses — what we assume, in plain language.
3. Derivation — the equations, step by step.
4. Worked example — a small numerical case carried by hand.
5. Runnable case — the tutorial that exercises the same model.

Read it linearly the first time. After that, treat it as a reference: the worked examples are written so they can be re-read as standalone exercises.

> **Key idea.** The simulator is a glass box. This manual is the lens.

A note on units

All equations in this manual are written in SI: pressure in Pa, molar flow in kmol/s, temperature in K, length in m, area in m^2, volume in m^3, energy in J, mass in kg, amount in kmol. Choupo's parser accepts named units in case files (`P 1 bar;`, `F 100 kmol/h;`) and converts to canonical SI internally, so the case files you read in the tutorials are written in the industrial-convention units the engineer expects while the equations on the page stay in the SI form they take in textbooks. Every dict-loaded scalar carries a five-int dimension tag $[M\,L\,T\,\Theta\,N]$ for mass, length, time, temperature and amount; a mismatch between a value's declared dimensions and the expected ones is caught with an explicit error before the solver runs. Throughout the manual the intermediate expressions and the numerical worked examples are SI; the final result is reported in whichever unit is most natural (kJ/kmol, bar, kmol/h,...) with the conversion written out.

Part I — Foundations

1 The sequential modular architecture

What you'll learn. The architectural credo Choupo follows, set out by Westerberg *et al.* in 1979 [68], codified by the CAPE-OPEN consortium in the late 1990s [71], and embodied in process flowsheeting ever since. Memorise five sentences and the rest of this manual reads as a footnote.

1. **The flowsheet is a directed graph.** Nodes are *unit operations*; edges are *streams* (material, energy, or information). Topology comes first; equations come second.
2. **Every unit operation is a black box with two signatures.** *Ports* for streams (inputs / outputs); *parameters* for hardware (area, volume, tray count, ...). Nothing else.
3. **Every stream that enters a unit must be fully specified.** The simulator computes the outputs; the engineer specifies the inputs (or seeds them and the outer driver iterates).
4. **Solve in topological order.** Each unit's `solve()` runs once its inputs are known. Recycles are broken by *tear streams* that the outer Wegstein iteration converges [54, 68].
5. **Design targets live on top of the simulation, not inside it.** An outer *DesignSpec* manipulates the engineer's free parameters until KPI constraints are satisfied. The simulator itself never solves the design problem; it solves the analysis problem.

1.1 Why this chapter exists

The honest reason: the project's first version of an evaporator unit op accepted a duty Q directly in its operation block, even though there was no heating-medium stream attached. Students reading the case asked, reasonably, "where does this Q come from?". The answer involved a long story about energy balances that the unit op was computing internally. After several rounds of confusion, we went back and read the textbooks. The architecture was already fully worked out forty years before Choupo existed; what we needed was discipline, not invention.

This chapter writes the discipline down so that no future author (or AI assistant) can reinvent it differently. Anything in the manual that contradicts these five rules is a bug.

1.2 Westerberg's classification of flowsheeting strategies

Following [68] Chapter 1, a process simulator can attack the global flowsheet equations in three ways:

1. **Sequential modular (SM).** Each unit operation is a self-contained *module* with its own internal `solve()`. The flowsheet executive (a) orders the units topologically, (b) calls each module in order, (c) loops Wegstein over tear streams to close recycles. Each module is self-contained; the executive only routes streams. This is the classic sequential-modular architecture, the one Choupo uses.
2. **Equation-oriented (EO).** Strip every module of its `solve`() and dump all the algebraic equations from every unit into one big system. Solve the system with a sparse Newton. Pros: faster on highly recycled flowsheets; can be wrapped in a non-linear programming solver without iteration overhead. Cons: unit-op authors lose locality (an error in one module infects the whole Jacobian); debug becomes harder.
3. **Simultaneous modular (SMM).** A hybrid: modules keep their internal solvers, but the executive linearises them at each outer iteration and solves a sparse system of the linearised modules. Used in optimisation-heavy applications.

For pedagogical and open-source purity reasons Choupo adopts SM throughout: students see one module's equations at a time, can breakpoint inside any `solve()`, and the recycle loop is the only place where multiple modules interact. EO is on the v1.0+ roadmap as an alternative back-end behind the same module interface.

1.3 Ports vs parameters — the CAPE-OPEN contract

The CAPE-OPEN Unit Operation Interface Specification [71] states the contract precisely:

> *A Unit Operation may have several **Ports** that allow it to be connected to other Unit Operations and to exchange material, energy or information with them. Unit operations also have sets of **Parameters**. These represent information that is not associated with the Ports and include design specifications and computed results.*

The credo translated:

	Port	Parameter
What it carries	A stream (material / energy / info)	A scalar (or list, or sub-dict)
In the dict	Under `inputs (...)` / `outputs (...)`	Under `operation { ...}`
Examples	`feed`, `liveSteam`, `Q_in`	area, V_R, refluxRatio, N_{stages}
Who writes the value	Upstream unit OR the engineer	The engineer (or outer DesignSpec)
Who reads it	The unit op via the port handle	The unit op via `operation.lookupScalar(..`
Physical meaning	The actual fluid / energy crossing the boundary	A property of the equipment itself

Three kinds of port that CAPE-OPEN sanctions:

- **Material port** — carries a fluid; the standard Choupo stream record (`ProcessStream`). Choupo supports this.
- **Energy port** — carries a heat duty Q in W. Used to model a heating utility *without* representing the carrier fluid explicitly (a so-called "energy stream"). Choupo does not yet expose this as a first-class type; cases that need a duty source pass it through a material stream + `state saturatedVapour`. Energy streams are on the roadmap.
- **Information port** — carries a scalar or vector of values for control signals, set-points, sensor readings. Used in dynamic flowsheets (the `choupoCtrl` Controller layer); the steady-state simulator (`choupoSolve`) ignores them.

1.4 What lives in `operation`: hardware and operating setpoints, both as parameters

A subtlety students invariably trip over: the `operation { ...}` block of a unit op mixes two physically different things, both classified as *parameters* (not ports) by the CAPE-OPEN contract.

Hardware parameters are intrinsic to the equipment: the heat-exchange surface area A, the overall heat-transfer coefficient U, the reactor volume V_R, the number of trays N, the tube length L. They describe *geometry* or material properties. The engineer chooses them at design time and they typically do not change during operation.

Operating setpoints are control choices: the operating pressure of an evaporator vessel (P_{op}), the jacket-fluid temperature of a CSTR (T_{jacket}), the reflux ratio of a column (R). They describe the *set-point of an external controller* (valve, vacuum pump, DCS loop). An operator can change them in real time without touching the equipment.

The dict treats them identically. Both appear under `operation { ...}` as scalar parameters; the dict parser does not distinguish them. This is deliberate — the unit op needs both to solve, and the engineer specifies both at design time, so there is no reason to fork the syntax.

The output streams inherit operating setpoints. An operating pressure declared as `operation { P 14 kPa; }` on a flash drum becomes the pressure of the vapour and liquid outputs: $P_{V,\,\text{out}} = P_{L,\,\text{out}} = P_{\text{op}}$ by mechanical equilibrium within the vessel. Students often read this as "the simulator computed the output pressure", which is technically correct but misleading — the simulator copied the operating-setpoint parameter to the output stream. The non-trivial computation is on the conjugate variable: in this example, the boiling temperature $T_{\text{boil}} = T_{\text{sat}}(P_{\text{op}}) + \Delta T_{\text{BPE}}$, found by inverting the saturation curve.

Where does P_{op} physically come from? From equipment *outside* the unit op model: a vacuum pump on the vapour line, a back-pressure valve, a flare header. The unit op does not model the controller — it takes P_{op} as given, the same way it takes A as given. In a multi-effect cascade where only the terminal P_N is set by external equipment, the intermediate P_i are *design unknowns*: the equal-area design problem is typically solved by a `DesignSpec` that manipulates them until the cascade closes (see Choupo's `evaporator03_co_current` tutorial for the worked case).

The conjugate-spec flavour. Most P-spec operations can be written as T-spec equivalently. A flash block typically accepts $\{T, P\}$, $\{T, V/F\}$, $\{P, V/F\}$, $\{P, Q\}$, $\{T, Q\}$ — any two of the five conjugate variables ($T, P, V/F, Q$, plus the duty constraint). Choupo evaporator defaults to P-spec for tutorial alignment with the textbook problems; a T-spec flavour is a one-line addition behind the same module interface.

> *Intuition.* The pedagogical heuristic from §1 sharpens: if you can imagine *walking out to the plant* and adjusting the setting with a wrench (resizing a pipe, replacing a tube bundle), it is a hardware parameter. If you can adjust it from the DCS in five minutes (changing a valve position, nudging a set-point), it is an operating setpoint. Both live in `operation`; the simulator treats them as data, not as unknowns.

1.5 Common practice

A common rule of thumb, in one sentence:

> *Inlet streams must be defined by the user as process variables, while outlet streams can be calculated.*

And it gives a two-tier hierarchy of heat-transfer unit ops that Choupo now mirrors:

Block	Ports	Parameters
Heater	1 material in + (optional) 1 energy in → 1 material out	T_{out} *or* Q
HeatX	2 material in (hot + cold) → 2 material out	area, U, geometry

The Heater is a deliberate *abstraction*: when the utility-side fluid is not pedagogically interesting, you provide a duty number or an energy stream and the block does the math. The HeatX is the full physical-equipment model. Choupo will ship both (`heater` and `heatExchanger`); the single-block hybrid is being split.

1.6 The unit-op interface in Choupo

In code (`src/unitOperations/UnitOperation.H`), the credo materialises as four virtual methods:

```
class UnitOperation {
public:
    // Streams in / streams out / parameters all arrive in `dict`.
    virtual int solve(const DictPtr& dict,
                      const ThermoPackage& thermo,
                      int verbosity) = 0;

    // Streams produced by this unit's last solve().  Order matches
    // the user's `outputs (...)` declaration in flowsheetDict.
    virtual std::vector<ProcessStream> producedStreams() const = 0;

    // Scalar results exposed by name (`A_needed`, `duty`, `economy`,
    //...) for the outer DesignSpec / SweepDriver / postProcessor.
    virtual std::map<std::string,scalar> kpis() const { return {}; }

    // Optional 1-D internal profile (axial sweep for PFR, stage
    // table for a column,...).  Read by the GUI's profile plot.
    virtual std::optional<UnitProfile> profile() const
                                          { return std::nullopt; }
};
```

Notice what is *absent*: there is no method for the unit op to write back into its inputs, no method for the unit op to demand a stream that was not declared, no method for the unit op to inject parameters into the outer dict. The interface is deliberately one-way: *streams in → solve → streams out + KPIs*. The credo enforced by the type system.

1.7 The flowsheet executive

The dispatcher (`src/unitOperations/flowsheet/Flowsheet.cpp`) runs the SM loop:

1. **Load streams.** `readSourceStream()` parses every top-level `streams { ...}` entry into a `ProcessStream`, applying flash-on-import to complete the thermodynamic state (§2).
2. **Topological order.** Read `units (...)` in the order written. For acyclic flowsheets (the DAG case) this order *is* the calculation order. For recycles, the user declares `tearStreams (...)` and the executive iterates Wegstein over the tear-stream guesses.
3. **Per-unit dispatch.** `runUnit()` builds the augmented unit dict (injects each input stream's $T, P, F, \mathbf{z}$ as sub-dicts; merges `solverDict` defaults; resolves named reactions), then calls `UnitOperation::New(type)->solve()`.
4. **Register outputs.** `producedStreams()` are bound to the names in `outputs (...)` and put into the stream registry, available to downstream units and to the result emitter.

1.8 The outer layer — DesignSpec, SweepDriver, OptimizationDriver

When a case has a `system/outerDict`, `main.cpp` instantiates an outer driver that *wraps the simulator functor*. The driver:

- declares its own *manipulated variables* (typically case-level `$variables`),
- declares its own *targets* (a list of either "KPI/stream-field equals a literal" or "two computed quantities are equal"),
- runs Newton-ND (or Nelder-Mead, or a sweep) over the manipulated variables, calling the simulator functor at each evaluation,
- ends with a final replay at the converged design point.

This outer layer — *Design Spec*, *Sensitivity*, *Optimization* — Choupo provides the same names. The pedagogical point: the simulator never solves a design problem; it solves an analysis problem (forward simulation of a given hardware). The design problem belongs to the outer driver.

1.9 Recycles and tear streams

Westerberg [68] Chapter 6 frames the recycle problem cleanly. In any SM flowsheet whose directed graph has a cycle, at least one stream on the cycle must be *torn* — that is, declared with an initial guess and updated each pass until the new value matches the old to a tolerance. Choupo exposes this directly in `flowsheetDict`:

```
tearStreams            (recycle );
recycleTol             1.0e-5;
recycleMaxIter         60;
recycleWegsteinQmin   -0.5;
recycleWegsteinQmax    0.0;
```

The Wegstein [54] accelerator is the default; $q_{\min} = q_{\max} = 0$ degenerates to plain direct substitution for cases where acceleration is unsafe (flash regime switches, strongly non-linear recycles).

> *Intuition.* Why does a flowsheet of hundreds of unit operations, each resting on its own assumptions, end up *consistent*? A student once put this very question to Prof. Luiz Alves (1920–1990) — professor of Chemical Engineering, Director of Instituto Superior Técnico and later head of its Chemical Engineering Department, and a director of CUF/Quimigal, then Portugal's largest chemical company. His answer, from a man who had actually built and run chemical plants: *"Don't worry — in the end it all checks out."* This is not faith. It checks out because a process engineer carries a physical sense of the *bounds* of every stream (a temperature in roughly 300–500 K, a vapour fraction in $[0, 1]$, a flow of the order of the feed) and of the operating *limits* of every unit. Numbers that wander outside those ranges are caught — by the engineer, and, in a glass-box simulator, by the bounds the author writes down and that the solver reports when they bind. The consistency is *earned*, by judgement made explicit — never conjured by a hidden crutch.

1.10 Choupo's compliance scorecard

Where the project stands today against the credo:

Rule	Status	Notes
1. Topology first, equations second	✓	`flowsheetDict.units (...)` is the DAG
2. Ports + parameters separation	✓	`inputs/outputs` vs `operation {...}`
3. Inlets fully specified	✓	dropped the demand-driven F shortcut
4. Topological solve + Wegstein	✓	Both DAG and recycle paths in place
5. Outer DesignSpec for targets	✓	`designSpec` driver
Material ports	✓	`ProcessStream`
Energy ports	×	on roadmap; today via state-tagged material
Information ports	partial	in `choupoCtrl` (`Controller` layer) only

The remaining gap (energy ports) is the only piece of the standard architecture not yet implemented. Everything else in this manual — thermodynamics, numerical methods, unit-op algorithms — is the *content* that fills the architecture.

> *Intuition.* If at any point the manual seems to describe a unit op doing something *outside* the streams-in / parameters / streams-out contract, treat that as a bug report. The architecture is the load-bearing pedagogical commitment: it is what makes the source code a glass box. Anything else is a leak.

2 What a process simulator predicts, and how

What you'll learn. Before any equation, a clear-eyed view of what a simulator like Choupo actually does: it takes a handful of measured numbers about pure components and binary mixtures, applies the three universal pillars of process engineering (conservation laws, thermodynamic laws, physical-chemistry models), and predicts the behaviour of a system that has never been built or measured. This chapter sets the scope, the power, and the limits that the rest of the manual will fill in.

2.1 The question

A chemical engineer is handed an assignment:

> *"You have a stream of 50/50 mol ethanol/water at 300 K and 1.01 bar leaving a fermenter, 100 kmol/h. The next step in the process needs this stream at 360 K, still liquid. Size the heater: tell me Q, the heat-exchange area, and the cooling-water cost."*

No one has ever measured this exact stream at 360 K — it is a *design* question, not a *reporting* one. The data sheet on your desk lists pure ethanol properties, pure water properties, a table of NRTL binary parameters fitted from VLE measurements at 1 atm, and the price of cooling water. How do you get from those numbers to a Q you can defend in a design review?

A process simulator answers the question. Internally it stacks three layers of reasoning, each grounded in a different kind of knowledge:

pillar	what it asserts	where it fails
conservation laws	mass, energy, momentum balance, always	never (an exact identity)
thermodynamic laws	equilibrium criteria, ΔG direction	never (an exact identity)
physical-chemistry models	how γ, P^{sat}, c_p, k depend on state	outside the calibration range

The first two pillars are **universal**: a balance is a balance, the second law is the second law, no engineer ever has to fit them. The third pillar is **empirical**: every model has a domain of validity bounded by the data it was fitted to, and pushing it past those bounds is where engineering judgement begins. The rest of this manual is in effect a tour of the third pillar — what each model means, what its parameters mean, where they come from, and when the simulator will quietly lie to you because they were not quite the right model for your problem.

2.2 Pillar 1 — conservation laws

The mass and energy balances are the simulator's only commandments that admit no exception. For a steady-state control volume around the heater of the opening question:

$$\dot{n}_{\mathrm{in}} = \dot{n}_{\mathrm{out}} \qquad \text{(mole balance, per component if no reaction)}, \tag{1}$$

$$\dot{n}_{\mathrm{in}}\, h_{\mathrm{in}} + \dot{Q} = \dot{n}_{\mathrm{out}}\, h_{\mathrm{out}} \qquad \text{(energy balance, no shaft work)}. \tag{2}$$

With one unknown ($\dot{Q}$) and one equation, the duty falls out *algebraically* once the inlet and outlet enthalpies are known. That is the value of pillar 1: it converts a design question into a property-evaluation question. In the balances are written by hand inside each unit operation (`src/unitOperations/...`); a future equation-oriented version will hand them all to a single sparse Newton solver (roadmap, post-v1).

Intuition. Balances are *exact* by construction. Every error a simulator ever commits has to come from the property models that feed them. That is why the next pillar's models, and the third pillar's parameter fits, deserve so much of this manual's attention.

2.3 Pillar 2 — thermodynamic laws

The first law of thermodynamics is the energy balance dressed up; the second law gives us *direction* and *equilibrium*. The simulator uses it in three concrete shapes:

- **Phase-equilibrium criterion.** At a flash, two phases coexist when the chemical potential of every component is equal across them: $\mu_i^{\mathrm{L}} = \mu_i^{\mathrm{V}}$. Rewritten in fugacities and divided through by RT, this becomes $\gamma_i x_i P_i^{\mathrm{sat}} \varphi_i^{\mathrm{sat}} = y_i \varphi_i P$ — the γ-φ formulation we will derive piece by piece in Part III. The shape is universal; the specific functions γ_i, φ_i are pillar-3 models.
- **Equilibrium constant of a reaction.** $K_{\mathrm{eq}}(T) = \exp(-\Delta G_r(T)/RT)$ comes from the second law's minimum-Gibbs criterion; we use it to bound conversions even when kinetics are slow. (Not yet consumed by the reactor models, which are kinetic only.)

- **Phase-stability test.** A trial flash that satisfies $\mu^{\mathrm{L}} = \mu^{\mathrm{V}}$ may still be unstable against splitting into three phases. Michelsen's tangent-plane distance criterion screens for that; ships the detector but not yet the full LL flash (§9 of CLAUDE.md).

> *Intuition.* Pillars 1 and 2 together tell you *where the system wants to sit* but say nothing about *how the parameters look in practice.* A spreadsheet that knew only these two pillars could not answer the opening question — it would need to be told, for every T on the path from 300 K to 360 K, what the enthalpy is. That information lives in pillar 3.

2.4 Pillar 3 — physical-chemistry models

Each property the simulator needs (vapour pressure, latent heat, heat capacity, activity coefficient, fugacity coefficient, viscosity, ...) is represented by a *model* — a closed-form functional dependence on T, P, composition — with two or three fitted parameters. Antoine for $P^{\mathrm{sat}}(T)$; Watson for $\Delta h^{\mathrm{vap}}(T)$; NRTL or Wilson for γ_i in a non-ideal liquid; ideal-gas $\varphi_i = 1$ for the vapour (currently); Arrhenius for reaction rate constants. Each model is a *compression* of experimental data: it remembers the shape of the data with few enough numbers that the simulator carries them cheaply, and forgets the rest.

That compression is where the power comes from. Once Antoine constants are fitted to four or five (T, P^{sat}) points, $P^{\mathrm{sat}}(T)$ becomes evaluable at any T in the validity range — including T values nobody measured. Multiply this by ten components and the simulator can evaluate 10×10 binary flash diagrams from a few hundred numbers. The same compression is also where the failure comes from: a model fitted between 280 and 380 K may extrapolate badly to 500 K; a model fitted on Raoult mixtures may misbehave on a hydrogen-bonded one; a model fitted at 1 bar may need a Poynting correction at 100 bar.

The simulator does **not** hide this from the user: every model records its declared validity range in the data file (`Trange` in the Antoine block, $T_b < T < 0.95\,T_c$ for Watson,...) and warns when it is asked to extrapolate. Chapter 7 will return to this and ask, model by model, what is genuinely fittable and what is hard-coded.

2.5 Power and limit, the running theme

A process simulator is at heart **compression plus extrapolation**. A short list of measured numbers (component properties, binary pair parameters) is compressed into a stack of models. The models are then extrapolated to a vast space of process conditions and topologies the original measurements never covered.

The compression is what makes the simulator useful: it would be absurd to measure the duty of every conceivable ethanol/water heater to a precision of a percent. The extrapolation is what makes the simulator dangerous: every prediction sits at some distance from the measurements that fed the models, and the engineer's job is to keep that distance reasonable.

> *Intuition.* The right relationship with a process simulator is the same as the right relationship with any instrument: trust the readings inside the calibration range, treat them as estimates outside it, and always ask which underlying measurement contributed to which output number. The glass-box mission of Choupo exists precisely so that question can be answered by reading the source, not by trust.

2.6 What this manual does

The rest of the manual is a tour of pillar 3 — the models the simulator uses, the parameters they need, and the boundaries within which they can be trusted — followed by the numerical methods that glue them together into a working flowsheet.

Part	content
Part I	this chapter; the notation and units the simulator uses internally
Part II	thermodynamic models for *pure* components: Antoine, Watson, Cp, plus integrals into h, s, g
Part III	thermodynamic models for *mixtures*: ideal mixing, activity coefficients, fugacity, γ-φ, K-values
Part IV	numerical methods: Newton-1D, Newton-ND, Levenberg-Marquardt, Wegstein, RK4
Part V	transport properties: viscosity (Chung), thermal conductivity (Eucken), diffusivity (Fuller), mixture rules – sl
Part VI	selected unit operations as they are wired
Part VII	equipment sizing and Guthrie/Turton costing

The order is roughly: *what does the simulator know about a single pure substance?* then *what changes when substances mix?* then *how does the code actually solve the resulting equations?* then *how does it apply them to a unit operation and to design?* By Part VII you can read any line of the simulator's run log and know which equation produced it, which parameter is upstream of that equation, and where in the data file the parameter came from.

Runnable case. **The part numbering above is the target layout.** of this manual currently presents Parts II, III, IV (mixtures and numerical methods) in a slightly different order; reorganisation into the table above is the active editorial work. See the session log in CLAUDE.md §14 for the current state.

3 Notation, units, and basis

What you'll learn. The conventions used throughout this manual and inside the simulator, so you can read the run-time log without translation.

3.1 Units

The simulator works in SI-derived process units, not strict SI:

Quantity	Symbol	Unit
Molar flow rate	F, L, V	kmol/h
Temperature	T	K
Pressure	P	bar
Volume	V_R	m^3
Heat duty	Q	kJ/kmol of feed
Specific heat capacity	C_p	J/(mol K)
Heat of vaporisation	ΔH_{vap}	J/mol
Saturation pressure	P^{sat}	bar

Intuition. Plant data sheets use these units. SI purists wince at bar and `kmol/h`, but these are what every plant data sheet uses and the arithmetic is friendlier. The simulator does no hidden conversions — what you read in the log is in the unit above.

3.2 Vectors and components

For a mixture with N_c components labelled $i = 1, \dots, N_c$:

- z_i — feed mole fraction of component i; vector $\mathbf{z}$.
- x_i — liquid-phase mole fraction; vector $\mathbf{x}$.
- y_i — vapour-phase mole fraction; vector $\mathbf{y}$.
- K_i — equilibrium K-value, $K_i \equiv y_i / x_i$; vector $\mathbf{K}$.

Mole fractions always sum to unity:

$$\sum_{i=1}^{N_c} z_i = 1, \qquad \sum_{i=1}^{N_c} x_i = 1, \qquad \sum_{i=1}^{N_c} y_i = 1.$$

3.3 Vapour fraction

For a feed of total molar flow F partitioning into a vapour stream V and a liquid stream L:

$$\beta \equiv \frac{V}{F}, \qquad 1 - \beta = \frac{L}{F}, \qquad \beta \in [0, 1]. \tag{3}$$

$\beta = 0$ is a saturated liquid (bubble point); $\beta = 1$ is a saturated vapour (dew point). All flash problems are, in the end, problems for β.

Part II — Pure-component thermodynamic models

1 Critical properties and corresponding states

What you'll learn. Every data file ships three numbers — T_c, P_c and the acentric factor ω — that look mysterious until you place them on a P-T phase diagram and ask what they actually mark. This chapter unpacks the geometry: T_c and P_c are the coordinates where the vapour-pressure line stops because liquid and vapour cease to be distinguishable; reduced coordinates collapse the behaviour of all simple fluids onto a single master curve (the law of corresponding states); ω is the third parameter that corrects for non-spherical, polar or hydrogen-bonded molecules. Three downstream correlations in the simulator (Watson, Sato-Riedel, the planned cubic EOS) consume nothing more than these three numbers, plus T_b.

1.1 The pure-component P-T diagram

Pure substances exist as solid, liquid, or vapour depending on the (T, P) chosen. The boundaries on a P-T diagram are three coexistence curves — sublimation (S/V), melting (S/L), and vapourisation (L/V) — meeting at the *triple point*. Only the vapourisation curve is relevant to this manual, because every unit op deals with liquid + vapour fluid streams (no solids yet).

Tracing the L/V curve from the triple point upward, both T and P rise. But at a specific (T_c, P_c), the curve simply **ends**: above that point, no matter how high P, no separate liquid and vapour exist — you have a single supercritical fluid phase. These two coordinates are the *critical temperature* and *critical pressure*, the values listed in every component's `.dat` file:

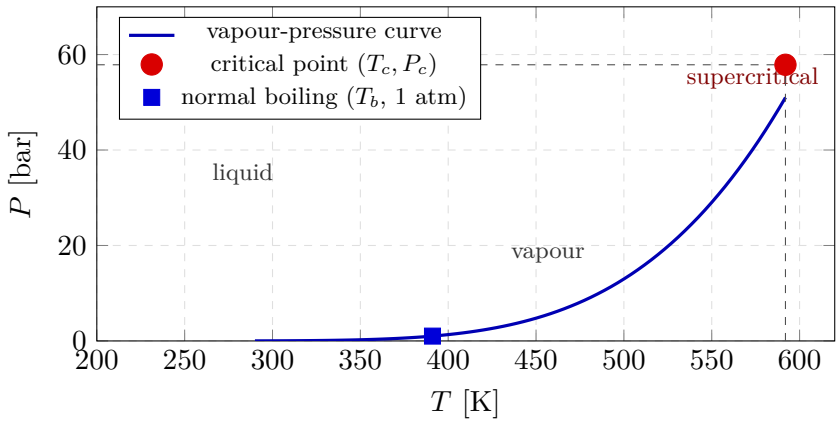

Figure 1: Vapour-pressure (saturation) curve of acetic acid in the P-T plane, computed from the Antoine constants in `data/standards/components/aceticAcid.dat`. The square marks the normal boiling point ($T_b = 391$ K at 1 atm). The curve terminates at the critical point ($T_c = 591.95$ K, $P_c = 57.86$ bar): beyond it there is no liquid/vapour distinction.

Two physical readings of the critical point are worth carrying:

- **It is where liquid and vapour become indistinguishable.** Both phases approach a common density as $T \to T_c$; the latent heat Δh^{vap} smoothly decays to zero (this is exactly what Watson (39) encodes).
- **It is the upper limit of the saturation curve.** Any Antoine fit, valid by definition only along the saturation line, ceases to make sense beyond T_c. That is why every `vaporPressure` block in the data files carries a `Trange` that always lies below T_c.

1.2 The law of corresponding states

van der Waals' great insight (1873) was that if you express T and P in units of their critical values, all simple fluids collapse onto the same curves. Define the *reduced* coordinates

$$T_r \equiv \frac{T}{T_c}, \qquad P_r \equiv \frac{P}{P_c}, \qquad V_r \equiv \frac{V}{V_c}, \tag{4}$$

and the two-parameter law of corresponding states asserts: any property expressed in reduced form is the *same function* of (T_r, P_r) for every fluid. Argon and methane and benzene, plotted as P_r versus T_r, give one universal saturation curve to within a few percent.

The simulator exploits this collapse repeatedly without naming it:

- **Watson** (39) writes $\Delta h^{\text{vap}}(T)/\Delta h^{\text{vap}}(T_b) = [(1-T_r)/(1-T_{br})]^{0.38}$ — a function of the single reduced variable $1-T_r$.
- **Sato-Riedel** (233) for the liquid thermal conductivity is a function of T_r and T_{br} only.
- **The planned cubic EOS** (SRK/PR, v1.x) writes the attractive parameter a as $a(T) = a_c \cdot \alpha(T_r, \omega)$, with α a universal function.

> *Intuition.* Corresponding states is what makes a thermophysical-property data sheet *short*. Without it, every fluid would need a custom correlation for every property. With it, we measure T_c, P_c (and one extra parameter — next subsection) for every component and reuse universal forms. This is the same engineering compression mentioned in Chapter 2: a few numbers per component, many predictions.

1.3 The acentric factor ω

Two-parameter corresponding states works for simple, near-spherical molecules (argon ∼0, methane ∼0.01). Real molecules deviate: longer chains, polar groups, hydrogen bonds all add **shape and stickiness** that two scalars cannot capture. Pitzer's 1955 fix was a third reduced parameter, the *acentric factor*,

$$\boxed{\omega \equiv -\log_{10}\left[P_r^{\text{sat}}(T_r = 0.7)\right] - 1.} \tag{5}$$

The definition takes the reduced saturation pressure at $T_r = 0.7$ (a representative point on the curve) and measures how far below 0.1 it lies. Simple fluids sit at $\omega \approx 0$ by construction (argon, oxygen). Hydrocarbons grow ω with chain length (*n*-hexane: 0.30; *n*-decane: 0.49). Hydrogen-bonded and dimerising species reach $\omega \approx 0.4$–0.7 (water: 0.34; ethanol: 0.65; acetic acid: 0.47 — moderated by vapour-phase dimerisation).

With ω in hand, the corresponding-states recipe becomes **three**-parameter: every dimensionless property is some universal function $f(T_r, P_r, \omega)$. This is the empirical backbone of the SRK and PR cubic EOS that will land in v1.x.

Worked example 1.1. Computing ω from Antoine for acetic acidAt $T_r = 0.7$ we need $T = 0.7 \times 591.95 = 414.36$ K. From the Antoine constants in `aceticAcid.dat` ($A = 4.68206$, $B = 1642.54$, $C = -39.764$):

$$\log_{10} P^{\text{sat}}_{[\text{bar}]}(414.36) = 4.68206 - \frac{1642.54}{414.36 - 39.764} = 4.68206 - 4.3863 = 0.2957,$$

so $P^{\text{sat}}(414.36) = 1.977$ bar. In reduced form, $P_r^{\text{sat}} = 1.977/57.86 = 0.0342$, hence

$$\omega = -\log_{10}(0.0342) - 1 = 1.466 - 1 = 0.466,$$

which agrees with the tabulated $\omega = 0.4665$ in the data file to three significant figures. Notice that ω is not an independent measurement: it is derivable from T_c, P_c and a single (T, P^{sat}) point at $T_r = 0.7$.

1.4 The compressibility factor Z

The compressibility factor measures how far a real fluid departs from the ideal-gas law:

$$Z \equiv \frac{PV}{nRT}, \qquad Z_{\text{idealgas}} = 1. \tag{6}$$

For low pressures, $Z \approx 1$ for any gas. As $P \to P_c$, Z drops below 1 (attraction-dominated, vapour denser than ideal) or rises above 1 (repulsion-dominated, vapour less dense than ideal) depending on the molecule. Corresponding states tells us that $Z(T_r, P_r, \omega)$ is a *universal* function — generalised compressibility charts (Pitzer-Curl, Lee-Kesler) tabulate it once for all fluids. takes the ideal-gas limit $Z = 1$ throughout: justified for the vapour-pressure ranges of our 9 stock components at ambient operating pressures, breaks down above ∼10 bar. When a cubic EOS lands (post-v1) it will compute $Z(T_r, P_r, \omega)$ via the roots of the appropriate cubic. The data files already carry all the inputs.

1.5 Where critical properties land in the stack

which property uses it	how	in source code
$\Delta h^{\text{vap}}(T)$ via Watson	T_c, T_b, $\Delta h^{\text{vap}}_{T_b}$	`Component::Hvap_latent`
$\kappa^{\text{liq}}(T)$ via Sato-Riedel	T_c, T_b, M	v1.x roadmap
φ_i via cubic EOS	T_c, P_c, ω	post-v1
$\mu^{\text{gas}}(T)$ via Chapman-Enskog	σ, ϵ/k (correspond to T_c, P_c)	v1.x roadmap
$P^{\text{sat}}(T)$ validity ceiling	T_c as the upper bound on Antoine `Trange`	data-file metadata

> *Intuition.* A practical sanity check when staring at the numbers in a fresh `.dat` file: T_b/T_c should sit in roughly the range 0.55–0.70 for non-polar molecules (acetic acid: $391/592 = 0.66$; benzene: $353/562 = 0.63$). If you see T_b/T_c near 1 or below 0.5, you almost certainly have a data-entry error. The ratio T_b/T_c is what

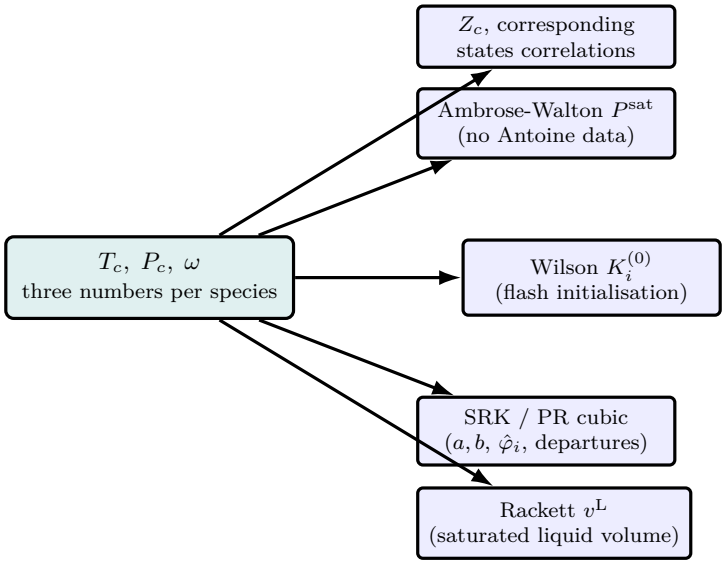

Figure 2: The corresponding-states payoff: three numbers per species — the critical temperature T_c, critical pressure P_c, and acentric factor ω — seed an entire thermodynamic stack. The same triple supplies the Ambrose-Walton vapour pressure when no Antoine constants exist, the Wilson K-value that initialises every flash, the $a(T)$ and b of the SRK/PR cubic (and through it $\hat{\varphi}_i$ and the enthalpy/entropy departures), the Rackett saturated-liquid volume, and the generalised Z_c correlations. Carrying T_c, P_c, ω in every component data file is what keeps the stack forward-compatible: select `model SRK;` and those three numbers *become* the equation of state, no new data required.

> Watson, Sato-Riedel and the corresponding- states machinery rely on; getting it wrong propagates everywhere.

2 Vapour pressure: from Clausius-Clapeyron to Antoine

What you'll learn. Antoine is not magic. It is the engineering refinement of a thermodynamic identity — the Clausius-Clapeyron equation — after two physical approximations and one empirical fix. This chapter walks through that derivation so the three constants A, B, C have names, the $\log_{10}$ has a reason, and the failure modes of the correlation are visible by construction. Watson then enters as the γ-φ-free way to get ΔH_{vap} at any T from a single calorimetric measurement at T_b.

2.1 The thermodynamic starting point

Along the liquid-vapour coexistence curve, the chemical potentials of liquid and vapour are equal: $\mu^{\text{L}}(T, P) = \mu^{\text{V}}(T, P)$. Take the differential along the curve,

$$\mathrm{d}\mu^{\text{L}} \;=\; \mathrm{d}\mu^{\text{V}} \;\Longrightarrow\; -s^{\text{L}}\,\mathrm{d}T + v^{\text{L}}\,\mathrm{d}P \;=\; -s^{\text{V}}\,\mathrm{d}T + v^{\text{V}}\,\mathrm{d}P,$$

where s and v are the molar entropy and molar volume of each phase. Solving for $\mathrm{d}P/\mathrm{d}T$ and using $\Delta s_{\text{vap}} = \Delta H_{\text{vap}}/T$ (from the Gibbs relation $\Delta g = 0$ at coexistence) gives the **Clausius-Clapeyron equation**:

$$\boxed{\frac{\mathrm{d}P^{\text{sat}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{vap}}}{T\,\Delta v_{\text{vap}}},} \tag{7}$$

where $\Delta v_{\text{vap}} = v^{\text{V}} - v^{\text{L}}$. Equation (7) is exact: no approximation has been made. It is a consequence of the second law (pillar 2 of Chapter 2) and no fitted constants enter.

2.2 Two physical approximations

Equation (7) is not yet usable because v^{V} is itself a function of T and P. Two approximations make it algebraic:

1. **The vapour behaves as an ideal gas:** $v^{\text{V}} = RT/P^{\text{sat}}$. Valid well below the critical point where vapour density is low.
2. **The liquid volume is negligible against the vapour volume:** $v^{\text{L}} \ll v^{\text{V}}$. At 1 bar, $T = T_b$, water has $v^{\text{L}} \approx 1.8 \times 10^{-5}$ m³/mol and $v^{\text{V}} \approx 3.1 \times 10^{-2}$ m³/mol — three orders of magnitude apart. The approximation worsens as $T \to T_c$ where the two volumes converge.

Under both approximations, $\Delta v_{\text{vap}} \approx RT/P^{\text{sat}}$, and (7) reduces to

$$\frac{\mathrm{d}P^{\text{sat}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{vap}}\,P^{\text{sat}}}{RT^2}, \tag{8}$$

or, dividing through by P^{sat},

$$\boxed{\frac{\mathrm{d}\ln P^{\text{sat}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{vap}}}{RT^2}.} \tag{9}$$

This is the form usually called "simple Clausius-Clapeyron". It relates the slope of the log-saturation pressure to the latent heat divided by RT^2.

2.3 Integrating with constant ΔH_{vap}: the rigid form

The simplest closure is to treat ΔH_{vap} as temperature-independent. Then (9) integrates immediately:

$$\ln P^{\text{sat}}(T) \;=\; -\frac{\Delta H_{\text{vap}}}{R\,T} \;+\; C_0, \tag{10}$$

where C_0 is a constant of integration fixed by one reference point (e.g. the normal boiling point: at T_b, $P^{\text{sat}} = 1$ atm). The rigid form (10) is a straight line on a $\ln P$-vs-$1/T$ plot, with slope $-\Delta H_{\text{vap}}/R$.

It is also pedagogically powerful: *measuring* P^{sat} at two temperatures gives you ΔH_{vap} from the slope, with no calorimetry needed.

Intuition. The rigid form is the simplest closed expression a thermodynamic identity can be turned into. Its weakness is the constancy assumption on ΔH_{vap}. Watson (39) already told us ΔH_{vap} varies with T — decaying to zero at T_c — so the rigid form must drift from the truth as T moves away from the reference point. Across a 100 K range it is typically wrong by 5–10% in P^{sat}, ample reason to want a better fit.

2.4 The empirical fix: Antoine

What if ΔH_{vap} were not constant but *weakly varied* with T? Then (9) would integrate to something like $-F(T)/R+C_0$ with $F(T)$ a slowly-varying function. Fitting a flexible-enough empirical form to thirty years of measurement across the laboratory range, Antoine (1888) proposed

$$\boxed{\log_{10} P_i^{\text{sat}}(T)\,[\text{bar}] \;=\; A_i \;-\; \frac{B_i}{T\,[\text{K}] + C_i}.} \tag{11}$$

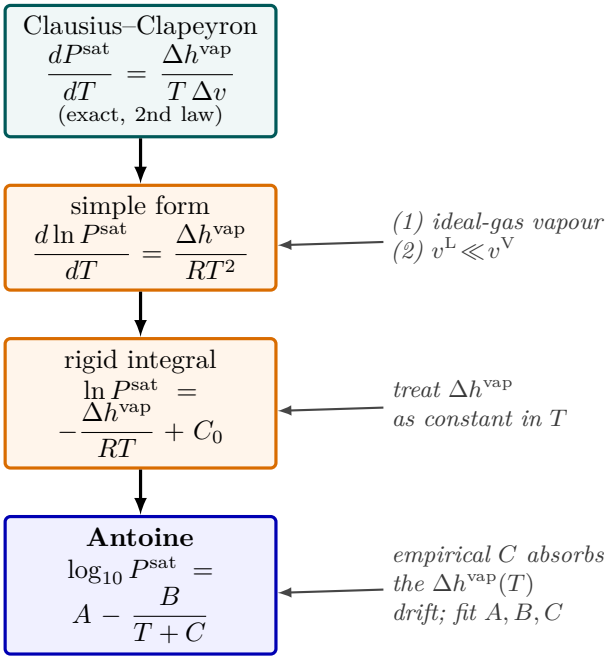

Figure 3: From a thermodynamic identity to the Antoine fit, in three honest steps. The exact Clausius–Clapeyron equation (teal, no fitted constants) becomes algebraic only after *two physical approximations* (orange): an ideal-gas vapour and a negligible liquid volume. Integrating with a constant latent heat gives the rigid straight-line-on-$\ln P$-vs-$1/T$ form; the empirical Antoine constant C (blue) then absorbs the temperature drift of Δh^{vap} that the rigid form ignores. Each downward arrow is a named assumption, so the student sees exactly what was traded for usability.

Recognise the shape: it is the rigid Clausius-Clapeyron form (10) with one cosmetic and one substantive modification.

- $\log_{10}$ **vs** $\ln$ **(cosmetic).** Antoine historically used base-10 logs because engineers read tables with them. $A \log 10$ and $B \log 10$ differ from the natural-log constants by a factor of $\ln 10 \approx 2.303$; the physics is unchanged.
- $T + C$ **in the denominator (substantive).** Instead of $1/T$ the denominator is $1/(T + C)$. The constant C absorbs the leading-order temperature dependence of ΔH_{vap}. In the rigid form B was identified with $\Delta H_{\text{vap}}/(R \ln 10)$; in Antoine B no longer has that clean meaning, but the three-parameter fit gains an extra order of magnitude in accuracy over the same range.

> *Intuition.* The Antoine constants are *not three independent measurements*; they are the three numbers that best track $\ln P^{\text{sat}}$ across the range in which the fit was done. Reusing them outside that range is asking the formula to extrapolate. Asking a fit to extrapolate is asking the fit to invent.

2.5 Failure modes of Antoine

Three ways Antoine breaks down, all visible from the structure of (11):

- **Beyond the validity range.** Antoine is calibrated on $[T_{\min}, T_{\max}]$, usually straddling T_b. Push far below $T_{\min}$ and the rigid Clausius-Clapeyron approximation (constant ΔH_{vap}) starts to lie; push past $T_{\max}$ approaching T_c and *both* approximations (constant ΔH_{vap} *and* ideal-gas vapour) collapse.
- $T + C$ **flips sign.** Antoine fits with C negative produce undefined logarithms when $T = |C|$. Typically $|C| < T_{\min}$ so this never bites inside the range; if you extrapolate below $|C|$ you get a domain error.
- **Near the critical point.** Above $\sim 0.95\,T_c$ the vapour is no longer dilute. No three-parameter form can recover the critical exponents; Pitzer/Watson-style corresponding-states relations take over.

Component data files in `data/standards/components/` declare a `Trange` explicitly:

```
vaporPressure
{
    model         Antoine;
    coefficients  (4.68206  1642.540  -39.764);   // aceticAcid
    Trange        (290  392);                     // K
}
```

does not yet *enforce* the range at run time; this is on the bug-watch. But the data are there and the reader of a `.dat` is expected to honour them.

2.6 Watson for $\Delta H_{\text{vap}}(T)$

The latent heat is already derived in Chapter 6 (Eq. 39). Repeated here for completeness:

$$\Delta H_{\text{vap}}(T) \;=\; \Delta H_{\text{vap}}(T_b)\left(\frac{T_c - T}{T_c - T_b}\right)^{0.38}. \tag{12}$$

Watson is a corresponding-states result: it depends only on T_r and T_{br}, and the 0.38 exponent is the universal value Watson extracted from a broad survey of vapour-pressure curves in 1943. The simulator computes it in `Component::Hvap_latent`.

2.7 Worked example: water from $\ln P$ vs $1/T$ to Antoine

Worked example 2.1. Two ways to compute P^{sat} of water at 320 KData from `data/standards/components/water.dat` (DIPPR-style, valid 273–373 K): $A = 5.40221$, $B = 1838.675$, $C = -31.737$, $T_b = 373.15$ K, $T_c = 647.10$ K, $\Delta H_{\text{vap}}(T_b) = 40\,650$ J/mol.
Antoine, direct.

$$P^{\text{sat}}(320) = 10^{\,5.40221 - 1838.675/(320-31.737)} = 10^{\,5.40221 - 6.3781} \approx 10^{-0.9759} \approx 0.1058 \text{ bar}.$$

Rigid Clausius-Clapeyron, for comparison. Anchored at $T_b = 373.15$ K, $P^{\text{sat}} = 1.01325$ bar, using $\Delta H_{\text{vap}}(T_b) = 40\,650$ J/mol and the rigid form (10),

$$\ln\frac{P^{\text{sat}}(320)}{1.01325} = -\frac{40\,650}{8.314}\left(\frac{1}{320} - \frac{1}{373.15}\right) \approx -2.171,$$

giving $P^{\text{sat}} \approx 1.01325 \cdot e^{-2.171} \approx 0.116$ bar.
Difference. Antoine: 0.106 bar. Rigid form: 0.116 bar. A ∼10% gap, exactly the order of magnitude we predicted in §Integrating with constant ΔH_{vap}. Antoine wins because its extra fitted constant C corrects for the weak temperature dependence of ΔH_{vap} that the rigid form ignored.
Watson at 320 K, for the energy balance.

$$\Delta H_{\text{vap}}(320) = 40\,650\left(\frac{647.10 - 320}{647.10 - 373.15}\right)^{0.38} \approx 40\,650 \cdot 1.094 \approx 44\,460 \text{ J/mol}.$$

Water below its boiling point is harder to evaporate than at T_b itself, by about $\sim 9\%$ — consistent with $\Delta H_{\text{vap}} \to 0$ as $T \to T_c$ being a slow approach.

Intuition. Antoine and Watson serve two different masters. Antoine asks "what is the saturation pressure?" — the answer is needed *every time* we compute a K-value in the flash. Watson asks "how much heat does this phase change move?" — the answer is needed every time the energy balance crosses the L/V boundary (adiabatic flash, distillation reboiler, condenser duty). The two correlations are independent in the simulator; you can use one without the other.

2.8 Ambrose-Walton: P^{sat} when there are no Antoine data

Antoine needs three numbers *fitted to measured vapour pressures*. A component you have just *estimated* from its molecular structure (Joback groups → T_c, P_c; Lee-Kesler → ω; Chapter 1) has no such measurements — so Antoine cannot be filled, and without P^{sat} the species is not flashable. The Ambrose-Walton correlation closes exactly that gap. It is a *three-parameter corresponding-states* form: it asks for only T_c, P_c and ω — precisely the three constants the estimate already produced — and returns the whole saturation curve.

The idea continues the corresponding-states reasoning of §1. Pitzer's three-parameter principle says the reduced vapour pressure P^{sat}/P_c is a *universal* function of the reduced temperature $T_r = T/T_c$ once the acentric factor ω is

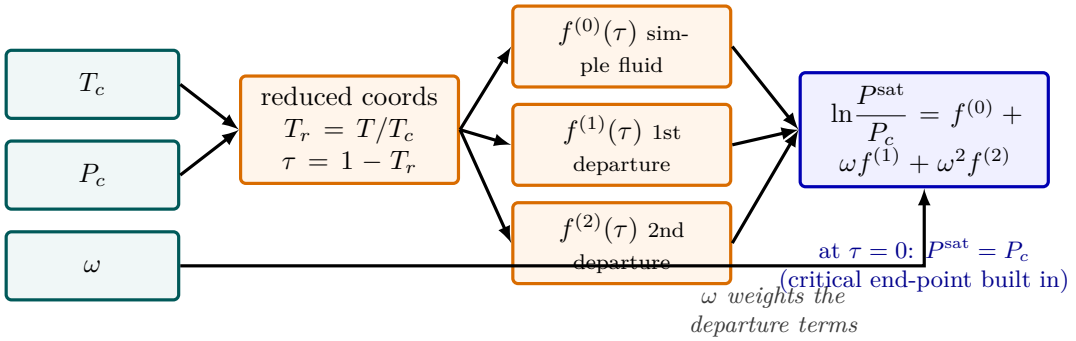

Figure 4: Ambrose–Walton turns the *three* corresponding-states constants an estimate already produces (T_c, P_c, ω, teal) into the *whole* saturation curve, with no measured vapour pressure. The reduced temperature feeds three *universal* shape functions $f^{(0)}, f^{(1)}, f^{(2)}$ (orange, the same for every fluid); the acentric factor ω weights the two departure terms and the sum returns $\ln(P^{sat}/P_c)$ (blue). Because every $f^{(i)}$ vanishes at $\tau = 1 - T_r = 0$, the curve passes through $P^{sat} = P_c$ at the critical point *by construction*—which is exactly what makes a freshly estimated component flashable.

admitted as a third coordinate. Expanding $\ln(P^{sat}/P_c)$ as a power series in ω and truncating at ω^2,

$$\ln \frac{P^{sat}}{P_c} \;=\; f^{(0)}(T_r) \;+\; \omega\, f^{(1)}(T_r) \;+\; \omega^2 f^{(2)}(T_r), \tag{13}$$

where $f^{(0)}$ is the simple-fluid reference (argon/krypton/xenon, $\omega \approx 0$) and $f^{(1)}, f^{(2)}$ are the first- and second-order departures. Ambrose and Walton (1989) fitted the three shape functions to high-quality data using a Wagner-style basis in

$$\tau \;\equiv\; 1 - T_r,$$

which vanishes at the critical point and grows toward the triple point. Their result, the form the simulator uses verbatim (`AmbroseWalton::psat`), is

$$\begin{aligned} f^{(0)} &= \left(-5.97616\,\tau + 1.29874\,\tau^{1.5} - 0.60394\,\tau^{2.5} - 1.06841\,\tau^{5}\right)/T_r, \\ f^{(1)} &= \left(-5.03365\,\tau + 1.11505\,\tau^{1.5} - 5.41217\,\tau^{2.5} - 7.46628\,\tau^{5}\right)/T_r, \\ f^{(2)} &= \left(-0.64771\,\tau + 2.41539\,\tau^{1.5} - 4.26979\,\tau^{2.5} + 3.25259\,\tau^{5}\right)/T_r. \end{aligned} \tag{14}$$

The fractional powers $\tau^{1.5}$ and $\tau^{2.5}$ are exactly the Wagner exponents that let a smooth analytic form track the steep curvature of $\ln P^{sat}$ down toward the triple point — the same curvature Antoine captures with its single empirical constant C. At the critical point $\tau = 0$, every term vanishes, so $\ln(P^{sat}/P_c) = 0$, i.e. $P^{sat} = P_c$ at $T = T_c$ *by construction*. The simulator caps the output at P_c for any $T \geq T_c$ (there is no saturation above the critical point).

What the code does. `AmbroseWalton::psat` is a pure function of (T, T_c, P_c, ω) — it carries no fitted state, so the estimate op (`estimateComponent`) calls the *same* function to fill a `.dat` and the flash calls it at run time. Note one bookkeeping detail visible in the source: P_c is stored in **bar** everywhere in Choupo (as the equations of state also assume), and the constructor converts it to Pa once so that `Psat_Pa` returns pure SI. The three constants are *not* re-declared inside the `vaporPressure` block — they already live at component level, and `Component::Component` injects them, so a case author writes only

```
vaporPressure { model AmbroseWalton; }   // Tc, Pc, omega taken from the component
```

Worked example 2.2. Ambrose-Walton P^{sat} of benzeneBenzene is a near-simple fluid: $T_c = 562.05$ K, $P_c = 48.95$ bar, $\omega = 0.210$ (Poling et al., 5th ed., App. A). Evaluate at the normal boiling point $T_b = 353.25$ K — a self-consistency check, since the answer should land on 1 atm.

First the reduced coordinates,

$$T_r = \frac{353.25}{562.05} = 0.6285, \qquad \tau = 1 - T_r = 0.3715.$$

The τ powers: $\tau^{1.5} = 0.22643$, $\tau^{2.5} = 0.08412$, $\tau^{5} = 0.00708$. The shape functions from (14) (each already

divided by T_r):

$$f^{(0)} = -3.1574, \quad f^{(1)} = -3.3820, \quad f^{(2)} = -0.0475.$$

Assemble (13) with $\omega = 0.210$,

$$\ln\frac{P^{\text{sat}}}{P_c} = -3.1574 + (0.210)(-3.3820) + (0.210)^2(-0.0475) = -3.8697,$$

so

$$P^{\text{sat}} = 48.95\,\text{bar} \times e^{-3.8697} = 48.95 \times 0.020862 = 1.021\ \text{bar},$$

within 0.8% of the 1.013 bar it *should* return at T_b. The residual gap is honest: ω here came from a Lee-Kesler relation, while the saturation curve came from the Ambrose-Walton fit — two different correlations, so they disagree by a fraction of a percent even on a textbook fluid. At 25 °C (298.15 K) the same evaluation gives 0.132 bar against a measured ≈ 0.127 bar, about 4% high — the expected accuracy band for a constants-only estimate.

Power and limit. The power is that Ambrose-Walton needs

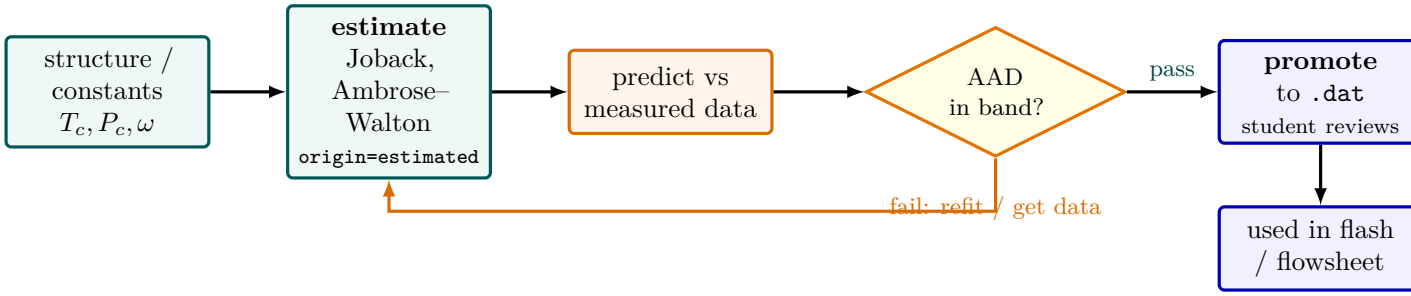

Figure 5: The glass-box property pipeline: **estimate**, then *validate*, then promote. From a molecular structure or three critical constants a group-contribution or corresponding-states method produces a value tagged `origin=estimated` (teal). Before it is trusted it is compared against measured data and scored by the average-absolute-deviation (AAD); the gate (yellow) is the project's no-silent-crutch rule applied to data quality. Only a value that clears its accuracy band is *promoted* into the curated `.dat` the student reviews (blue) and the solver then reads. A failing estimate loops back to be refitted or replaced by laboratory data—it is never silently used.

nothing measured about the vapour pressure itself: feed it the three critical/acentric constants — which corresponding states gives you even for a brand-new molecule — and it returns the entire curve from triple point to critical point, with the correct critical end-point built in. This is what makes an *estimated* component flashable without a single laboratory point. The limit is that it is an *estimate*, not data. It is good to a few percent for non-polar and weakly polar species; it *degrades* for strongly polar, hydrogen-bonding and associating compounds — water, alcohols, carboxylic acids — where the universal $f^{(i)}$ shapes simply do not describe the real curve. The glass-box rule therefore stands: the estimate op overlays the predicted Ambrose-Walton curve on any measured points in the component's `reference{}` / `experimental{}` blocks, and you look before you trust it for design. When you *do* have measured vapour pressures, fit Antoine (§2) and use that — Ambrose-Walton is the fallback that keeps the simulator running until the data arrive, not a replacement for them.

> *Intuition.* Antoine and Ambrose-Walton answer the same question — "what is $P^{\text{sat}}(T)$?" — from opposite directions. Antoine is *data-first*: three constants regressed from measurements, accurate inside the fitted window, silent about everything outside it. Ambrose-Walton is *theory-first*: a universal corresponding-states shape with no compound-specific tuning beyond (T_c, P_c, ω), defined everywhere from triple to critical point but only as accurate as the universality assumption holds. A polar molecule breaks the universality; a molecule outside its Antoine `Trange` breaks the fit. The honest workflow is to use Antoine where the data live and fall back to Ambrose-Walton where they do not — and to plot both whenever you can.

3 Sublimation: the solid-vapour curve and the triple point

What you'll learn. The same Clausius-Clapeyron identity that produced Antoine (§2) for the liquid-vapour curve produces the *solid-vapour* (sublimation) curve when the latent heat is the heat of sublimation rather than of vapourisation. This section derives the sublimation line, shows why the three coexistence curves must meet at one point with $\Delta H_{\text{sub}} = \Delta H_{\text{fus}} + \Delta H_{\text{vap}}$, and documents the exact algorithm `PurePhaseDiagram` uses to draw the solid/liquid/vapour P-T diagram — sublimation by Clausius-Clapeyron, melting by the full Clapeyron slope — so the triple-point picture of §1 is finally closed on both ends.

3.1 Why a solid sublimes: the same identity, a different phase

The derivation of §2 never used the fact that the condensed phase was a liquid. It used only that two phases coexist, so their chemical potentials are equal along the boundary, and that the Gibbs relation gives $\Delta s = \Delta h/T$ at coexistence. Replace "liquid" by "solid" everywhere and the *exact* Clapeyron equation (7) reads, for the solid-vapour boundary,

$$\frac{\mathrm{d}P^{\text{sub}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{sub}}}{T\,\Delta v_{\text{sub}}}, \qquad \Delta v_{\text{sub}} = v^{\text{V}} - v^{\text{S}}, \tag{15}$$

where ΔH_{sub} is the molar enthalpy of sublimation (solid → vapour, directly, with no liquid in between). Apply the *same* two approximations as before — the vapour is ideal, $v^{\text{V}} = RT/P$, and the solid volume is negligible against the vapour, $v^{\text{S}} \ll v^{\text{V}}$ (a solid is even denser than the liquid, so this is *safer* than for the L/V curve) — and (15) collapses to the simple form

$$\frac{\mathrm{d}\ln P^{\text{sub}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{sub}}}{R\,T^2}. \tag{16}$$

This is (9) with ΔH_{vap} swapped for ΔH_{sub} — "Antoine extended to the solid", except that we do not bother with a three-constant Antoine fit here. Sublimation pressures are tiny and rarely measured across a wide range, so the simulator integrates (16) in its *rigid* (constant-ΔH_{sub}) form, anchored at the triple point (T_t, P_t):

$$\boxed{P^{\text{sub}}(T) \;=\; P_t\,\exp\!\Big[-\frac{\Delta H_{\text{sub}}}{R}\Big(\frac{1}{T}-\frac{1}{T_t}\Big)\Big].} \tag{17}$$

This is exactly the line in `PurePhaseDiagram::run`,

```
const scalar P = Pt * std::exp(-(Hsub / R) * (1.0 / T - 1.0 / Tt));
```

evaluated on a grid of $m = \max(6, n/2)$ points from $0.78\,T_t$ up to T_t (the lower bound keeps P^{sub} within about three decades of P_t so it does not vanish off the log axis).

3.2 The triple point and $\Delta H_{\text{sub}} = \Delta H_{\text{fus}} + \Delta H_{\text{vap}}$

The three coexistence curves — sublimation (S/V), melting (S/L) and vapourisation (L/V) — are not independent. At the unique *triple point* (T_t, P_t) all three phases coexist, so the solid, liquid and vapour chemical potentials are equal there. Enthalpy is a state function, so the energy to go solid → vapour must equal solid → liquid → vapour around the triple point:

$$\boxed{\Delta H_{\text{sub}} \;=\; \Delta H_{\text{fus}} \;+\; \Delta H_{\text{vap}}} \qquad \text{(at the triple point)}. \tag{18}$$

Because $\Delta H_{\text{sub}} > \Delta H_{\text{vap}}$, the sublimation line is *steeper* than the vapourisation line on a $\ln P$ vs $1/T$ plot (slope $-\Delta H_{\text{sub}}/R$ against $-\Delta H_{\text{vap}}/R$, both negative): the two curves leave the triple point at different slopes, with the sublimation curve below and to the left. This kink at T_t is the qualitative signature of the triple point, and (18) is the consistency check the curator applies before writing a `sublimation{}` block.

3.3 The melting line: the full Clapeyron slope

The solid-liquid boundary is the one curve where neither phase is a gas, so the ideal-gas trick of §2 is *unavailable*: $\Delta v_{\text{fus}} = v^{\text{L}} - v^{\text{S}}$ is a small difference of two condensed-phase volumes, not an RT/P that dominates. We must keep the full Clapeyron equation (7):

$$\frac{\mathrm{d}P^{\text{fus}}}{\mathrm{d}T} \;=\; \frac{\Delta H_{\text{fus}}}{T\,\Delta v_{\text{fus}}}. \tag{19}$$

Over the modest temperature span of a melting line, T barely moves from T_t and ΔH_{fus}, Δv_{fus} are nearly constant, so (19) integrates to a near-straight, very steep line. The simulator linearises it directly, anchored at the triple point:

$$T^{\text{fus}}(P) \;=\; T_t \;+\; \frac{T_t\,\Delta v_{\text{fus}}}{\Delta H_{\text{fus}}}\,(P - P_t), \tag{20}$$

the line `const scalar T = Tt + (Tt * dVfus / Hfus) * (P - Pt)` in `PurePhaseDiagram`, swept from P_t up to $\max(P_c, 10^8\ \text{Pa})$. The sign of Δv_{fus} is what makes water famous: for almost every substance the solid is denser than

the liquid, so $\Delta v_{\text{fus}} > 0$ and the melting line leans *right* (higher pressure raises the melting point). Water is the anomaly — ice floats, so $\Delta v_{\text{fus}} < 0$ and the line leans *left* (pressure *lowers* the melting point). In `phase01_water_full` the curator enters `deltaVfus -1.63e-6` (m^3/mol, *negative*), and the melting line tilts the right way by construction.

3.4 What the algorithm needs — and what it draws

`PurePhaseDiagram` reads the solid anchor from two places, in precedence order (this matches the code exactly):

1. the component's curated `sublimation{}` block — `tripleT` T_t, `tripleP` P_t, `Hfus`, `Hsub` — the intrinsic pure-compound constants that live *with* the species (`Component::subTripleT()` etc.);
2. the op-dict `solid{}` block, which fills any field the component left blank *and* is the sole source of `deltaVfus` Δv_{fus} — the fusion-line slope, deliberately kept out of the component block because it is a sample/measurement quantity, not a clean intrinsic constant.

The drawing logic then branches on what is available:

- **L/V only** (no solid anchor): the saturation curve runs from $0.45\,T_c$ to T_c and the critical point is marked — the historical behaviour, every existing `.dat` unaffected.
- **+ sublimation** ($T_t, P_t, \Delta H_{\text{sub}}$ all present, flag `hasSub`): the saturation curve now starts at T_t, the sublimation curve (17) is drawn, and the triple point is marked.
- **+ fusion** (additionally $\Delta H_{\text{fus}} > 0$ and $\Delta v_{\text{fus}} \neq 0$, flag `hasFusion`): the melting line (20) is added.

Each row of the output CSV is tagged `saturation`, `sublimation`, `fusion`, `triple` or `critical`, so the GUI can colour the three curves and place the two singular points without re-deriving anything.

Worked example 3.1. Triple-point closure for `watertutorials/props/scan/phase01_water_full` draws the full water diagram with the op-dict anchor

$$T_t = 273.16\ \text{K}, \quad P_t = 611.657\ \text{Pa}, \quad \Delta H_{\text{fus}} = 6010, \quad \Delta H_{\text{sub}} = 51059\ \text{J/mol}.$$

Consistency by (18). The implied heat of vapourisation at the triple point is $\Delta H_{\text{vap}} = \Delta H_{\text{sub}} - \Delta H_{\text{fus}} = 51059 - 6010 = 45049$ J/mol, against the calorimetric $\approx 45\,054$ J/mol for water at 0 °C — the block is internally consistent to better than 0.02%.
A point on the sublimation curve. Evaluate (17) at $T = 260$ K (ice subliming below freezing):

$$P^{\text{sub}}(260) = 611.657\,\exp\!\left[-\frac{51059}{8.31446}\left(\tfrac{1}{260} - \tfrac{1}{273.16}\right)\right] = 611.657\,e^{-1.1383} = 195.9\ \text{Pa},$$

about 1.96 mbar — the vapour pressure of ice at −13 °C, within a few percent of the tabulated ≈ 196 Pa, and the basis of freeze-drying (lyophilisation): pull the chamber below P_t and ice leaves as vapour without ever melting.
The melting tilt. With $\Delta v_{\text{fus}} = -1.63 \times 10^{-6}$ m^3/mol, the slope (20) is

$$\frac{\mathrm{d}T}{\mathrm{d}P} = \frac{T_t\,\Delta v_{\text{fus}}}{\Delta H_{\text{fus}}} = \frac{273.16 \times (-1.63 \times 10^{-6})}{6010} = -7.4 \times 10^{-8}\ \text{K/Pa},$$

i.e. raising the pressure by 100 bar (10^7 Pa) lowers the melting point by about 0.74 K — the classic regelation number, and negative, as only water and a handful of other anomalies are.

Intuition. Dry ice and water ice are the two halves of the same picture. CO_2 has its triple point *above* 1 atm ($T_t = 216.6$ K, $P_t = 5.18$ bar), so at atmospheric pressure the liquid field is never crossed: warm a block of solid CO_2 and it goes straight to gas along the sublimation curve — that is why it is "dry". Water's triple point is *below* 1 atm (611 Pa), so at room pressure you cross the melting line first and ice becomes a puddle. Same three curves, same Clausius-Clapeyron slope $-\Delta H_{\text{sub}}/R$; the only thing that decides "dry" versus "wet" is whether the ambient pressure sits above or below P_t.

Power and limit. The power is reuse: one identity (Clausius-Clapeyron) plus one bookkeeping law ($\Delta H_{\text{sub}} = \Delta H_{\text{fus}} + \Delta H_{\text{vap}}$) gives the entire solid region from four curated numbers, with the triple point and critical point as the two anchors that make the diagram close. The limit is the same constant-ΔH_{sub} rigidity that makes the rigid L/V form (10) only approximate: (17) treats ΔH_{sub} as temperature-independent, exact only near T_t and drifting as T falls (which is why the curve is drawn only down to $0.78\,T_t$); and the melting line (20) is linearised, fine over its narrow span but never extrapolate it to the exotic high-pressure ice polymorphs. As always in Choupo, the diagram is *drawn, not asserted*: the CSV carries every (T, P) point and its curve tag, so a student sees the kink at the triple point and the direction of the melting tilt rather than taking them on faith.

4 Saturated-liquid molar volume: the Rackett equation

Ambrose-Walton (§2.8) gave us $P^{\text{sat}}(T)$ from the same three constants (T_c, P_c, ω) an estimate produces. Its sibling on the *liquid* side answers the companion question: how much volume does one mole of saturated liquid occupy? The simulator needs this in three independent places — the liquid **density** reported for any stream (`ThermoPackage::density`), the pure-component liquid molar volumes V_i^L that Wilson's $\Lambda_{ij} = \frac{V_j^L}{V_i^L}\exp(\cdots)$ demands ((68)), and the `Vliq` field an `estimateComponent` writes into a fresh `.dat`. All three route through one closure, `closures::rackettVliq`, so that what the estimate stores and what the flash later reads are the *same* number from the *same* formula — the glass-box invariant we hold for every estimated property.

4.1 Why a special correlation for the liquid?

A cubic equation of state (SRK, PR; Chapter 8) will happily return a liquid root v^L, but that root is the EoS's notorious weak spot: SRK over-predicts saturated-liquid volumes by 10–30%, and even the volume-translated forms drift with temperature. The liquid is nearly incompressible, so its molar volume is governed almost entirely by how close T sits to T_c — a one-dimensional problem along the saturation line — and a purpose-built corresponding-states correlation beats a general EoS by an order of magnitude here. That correlation is Rackett's.

4.2 From the critical point to the Rackett equation

Start at the critical point, where by definition

$$Z_c \;=\; \frac{P_c V_c}{R_{\text{g}} T_c}, \qquad \text{so} \qquad V_c \;=\; \frac{R_{\text{g}} T_c}{P_c}\, Z_c. \tag{21}$$

Rackett (1970) observed that if you plot $\ln V^{\text{sat}}$ against $(1-T_r)^{2/7}$ for a saturated liquid, the data fall on a remarkably straight line that, extrapolated to $T_r \to 1$, passes through V_c. Anchoring the line at the critical point and letting its slope be set by Z_c gives the one-constant Rackett equation,

$$V^{\text{sat}}(T) \;=\; V_c\, Z_c^{(1-T_r)^{2/7}} \;=\; \frac{R_{\text{g}} T_c}{P_c}\, Z_c^{\,1+(1-T_r)^{2/7}}, \qquad T_r = \frac{T}{T_c}. \tag{22}$$

The two forms are identical: substituting $V_c = (R_{\text{g}}T_c/P_c)Z_c$ folds the leading Z_c into the exponent, raising the 1. The peculiar exponent $2/7 \approx 0.2857$ is purely empirical — it is the power that linearises the most species at once — and the same $2/7$ you will see in COSTALD and other liquid-volume correlations descends from Rackett. At $T = T_c$ the bracket $(1-T_r)^{2/7} = 0$, the exponent is 1, and (22) returns exactly V_c, as a correlation seeded at the critical point must.

4.3 The Spencer-Danner / Yamada-Gunn substitution: $Z_c \to Z_{RA}$

Equation (22) has a flaw: the *true* Z_c is itself only known to a percent or two, and using it makes the correlation miss the actual measured volume at lower T_r where you care most. Spencer and Danner (1972) fixed this by replacing Z_c with an *adjustable* Rackett parameter Z_{RA} — a number fitted per compound so that (22) hits one known liquid density. Z_{RA} is numerically close to Z_c but not equal to it. When a fitted Z_{RA} is *not* available — exactly the situation for an estimated component — Yamada and Gunn (1973) supplied a corresponding-states correlation for it in terms of the acentric factor,

$$Z_{RA} \;=\; 0.29056 \;-\; 0.08775\,\omega. \tag{23}$$

This is the form the simulator hard-codes. The constant 0.29056 is the generic small-molecule $Z_c \approx 0.29$; the $-0.08775\,\omega$ term shrinks it as molecules become more elongated and acentric, mirroring how Z_c falls along a homologous series. Putting (23) into (22) is the operational formula:

$$\boxed{V^L(T) \;=\; \frac{R_{\text{g}} T_c}{P_c}\, Z_{RA}^{\,1+(1-T_r)^{2/7}}, \qquad Z_{RA} = 0.29056 - 0.08775\,\omega.} \tag{24}$$

What the code does. `closures::rackettVliq(T, Tc, Pc_Pa, omega)` is (24) verbatim: it forms $T_r = T/T_c$, computes $Z_{RA} = 0.29056 - 0.08775\,\omega$, raises it to $1+(1-T_r)^{2/7}$, and multiplies by $R_{\text{g}}T_c/P_c$ (note P_c passed in **Pa** here, so the result is SI, m^3/mol). Like the Ambrose-Walton closure it carries no fitted state — the estimate op and the run-time density call the identical function. This correlation is variously labelled "Rackett", "modified Rackett (Spencer-Danner)" or "Rackett/Yamada-Gunn" depending on whether Z_{RA} is fitted or predicted; Choupo predicts it from ω, so the diagnostic JSON tags the value `Rackett / Yamada-Gunn`.

4.4 Mixtures: mole-weighted molar volume

For a liquid mixture the simulator does *not* mix the constants; it computes each pure-component saturated volume at the mixture T and mole-averages them (`ThermoPackage::density`, branch (3)),

$$V^L_{\text{mix}}(T,\mathbf{x}) \;=\; \sum_i x_i\, V_i^L(T) \;=\; \sum_i x_i\, \frac{R_g T_c^{(i)}}{P_c^{(i)}}\, Z_{RA,i}^{\,1+(1-T/T_c^{(i)})^{2/7}}, \tag{25}$$

and the reported liquid density is then $\rho = \overline{M}/V^L_{\text{mix}}$ with $\overline{M} = \sum_i x_i M_i$ the mole-averaged molar mass. This is the simplest defensible mixing rule: it assumes an *ideal liquid mixture* (no excess volume, $V^E = 0$), so the partial molar volume of each species equals its pure-component value. It is the same Amagat-additivity assumption that underlies treating the liquid as volumetrically ideal, and it is exactly the assumption Wilson's V_j^L/V_i^L ratio also makes. Real mixtures show a few-percent V^E (ethanol-water famously contracts), which this rule does not capture — the honest limit flagged below.

Worked example 4.1. Rackett liquid volume and density of benzeneBenzene: $T_c = 562.05$ K, $P_c = 48.95$ bar $= 4.895 \times 10^6$ Pa, $\omega = 0.210$, $M = 78.11$ g/mol. Evaluate at 25 °C ($T = 298.15$ K), the temperature `estimateComponent` pins.
Reduced temperature and the Rackett parameter,

$$T_r = \frac{298.15}{562.05} = 0.5305, \qquad Z_{RA} = 0.29056 - 0.08775(0.210) = 0.27213.$$

The exponent,

$$1 + (1 - T_r)^{2/7} = 1 + (0.4695)^{0.2857} = 1 + 0.8084 = 1.8084.$$

The prefactor,

$$\frac{R_g T_c}{P_c} = \frac{8.314 \times 562.05}{4.895 \times 10^6} = 9.547 \times 10^{-4}\ \text{m}^3/\text{mol}.$$

Assemble (24),

$$V^L = 9.547 \times 10^{-4} \times (0.27213)^{1.8084} = 9.547 \times 10^{-4} \times 0.09367 = 8.94 \times 10^{-5}\ \text{m}^3/\text{mol} = 89.4\ \text{cm}^3/\text{mol}.$$

The measured saturated-liquid volume of benzene at 25 °C is ≈ 89.4 cm^3/mol — agreement to better than 1% for a non-polar, near-spherical molecule. The density follows,

$$\rho = \frac{M}{V^L} = \frac{78.11 \times 10^{-3}\ \text{kg/mol}}{8.94 \times 10^{-5}\ \text{m}^3/\text{mol}} = 874\ \text{kg/m}^3,$$

against the handbook 874 kg/m^3. Rackett is at its best precisely here.

Power and limit. The power mirrors Ambrose-Walton's: from only (T_c, P_c, ω) — the three constants corresponding states hands you even for a freshly estimated molecule — Rackett returns the whole saturated-liquid volume curve, correct to 1–2% for normal fluids and seeded exactly at the critical point. It is far better than reading the liquid root of a cubic EoS, and it costs one `pow`. The limits are three. (i) It is a *saturated*-liquid correlation: it carries *no* pressure dependence, so it ignores the (small) compression of a sub-cooled liquid above its saturation pressure — a good approximation because liquids are nearly incompressible, but not exact for high-pressure work. (ii) Accuracy *degrades* for the same polar, hydrogen-bonding species that defeat Ambrose-Walton — water, alcohols, acids — because the universal $Z_{RA}(\omega)$ shape no longer describes them; with a measured density you should fit Z_{RA} (Spencer-Danner) rather than predict it from ω. (iii) The mole-weighting (25) assumes $V^E = 0$, so it misses the volume change on mixing of strongly non-ideal solutions. For pedagogy and for the density figure on a flowsheet stream these are acceptable; for a custody-transfer liquid density they are not, and the glass-box rule applies — the estimate op overlays the Rackett curve on any measured liquid densities in the component file so you look before you trust it.

> *Intuition.* Ambrose-Walton and Rackett are the two halves of one corresponding-states bargain. Give up three constants (T_c, P_c, ω) and you get back *both* legs of the saturation envelope — the pressure $P^{\text{sat}}(T)$ *and* the liquid volume $V^L(T)$ — each anchored at the critical point and each accurate to a few percent for normal fluids. Neither needs a single measurement of the property it predicts; both fail on the same polar molecules; both are fallbacks that keep an estimated component fully simulatable until real data arrive, and both are overlaid on data, never substituted for it.

5 Heat capacity

The molar heat capacity is parametrised as a polynomial in temperature:

$$C_p^{\mathrm{ig}}(T) \;=\; a_0 + a_1 T + a_2 T^2 + a_3 T^3 + a_4 T^4 \quad [\mathrm{J/(mol\,K)}]. \tag{26}$$

Two such polynomials per component: one for the ideal gas (C_p^{ig}, used in vapour-phase energy balances) and one for the liquid (C_p^{L}, used in liquid-phase energy balances).

For an enthalpy difference between T_{ref} and T:

$$h(T) - h(T_{\mathrm{ref}}) \;=\; \int_{T_{\mathrm{ref}}}^{T} C_p(T')\,dT'. \tag{27}$$

With the polynomial form, this integral is closed-form term by term — the simulator does it analytically, not by quadrature.

6 From component data to integral properties

What you'll learn. Each `data/standards/components/*.dat` file holds a short list of numbers: T_c, P_c, ω, T_b, $\Delta h^{\mathrm{vap}}_{T_b}$, V^{liq}, Antoine constants, c_p polynomials. This chapter shows what each one *means*, where the simulator reads it, and how those tiles are assembled into the integral quantities that drive every energy balance and every Gibbs-energy minimisation: pure-component enthalpy and entropy, the latent heat at an arbitrary T via Watson, mixture enthalpy and mixture Gibbs energy, and Gibbs-Duhem as the consistency check on activity-coefficient fits.

6.1 Anatomy of a component data file

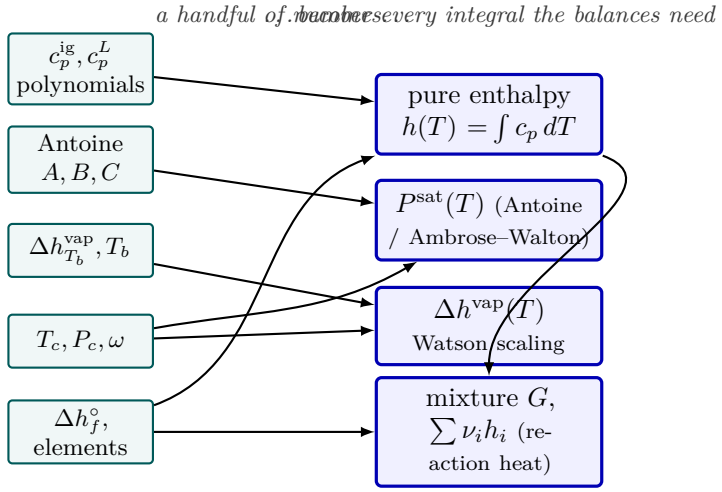

Figure 6: What a component `.dat` file is *for*. Each entry is a small tile (teal): a few c_p coefficients, Antoine or critical constants, a single latent heat at T_b, the formation enthalpy and its element basis. The simulator assembles those tiles—analytically, term by term—into the integral quantities (blue) that every energy balance and every Gibbs-energy minimisation actually consumes: pure-component enthalpy, the saturation curve, the temperature-corrected latent heat via Watson, and the mixture Gibbs energy whose $\sum_i \nu_i h_i$ returns the heat of reaction. One number can feed several integrals (note the shared T_c and Δh°_f arrows).

A typical entry, slightly trimmed for the page:

```
name        aceticAcid;
formula     C2H4O2;
CAS         64-19-7;

MW          60.052;         // kg/kmol
Tc          591.95;         // K
Pc          57.86;          // bar
omega       0.4665;         // [-]
Tb          391.05;         // K
HvapTb      24390;          // J/mol
Vliq        5.752e-5;       // m^3/mol  (57.52 cm^3/mol)

vaporPressure
{
    model         Antoine;
    coefficients  (4.68206  1642.540  -39.764);
    Trange        (290  392);
}

idealGasHeatCapacity {... }
liquidHeatCapacity   {... }
```

Each scalar plays a specific role. The right-hand column below points to where the simulator reads it:

field	unit	meaning, and where it is consumed
`MW`	kg/kmol	Molar mass. Carries through mass-flow conversions and the sizing chapters; entered in every stoichiometry / yield calculation.
`Tc`	K	Critical temperature. Sets the upper bound of every saturation curve (vapour and liquid become indistinguishable at T_c) and is the explicit input to the Watson correlation (39). Also fed forward to a future SRK/PR EOS.
`Pc`	bar	Critical pressure. Carried for the same future EOS work and for acentric-factor consistency checks. Not yet read by the ideal-gas φ.
`omega`	–	Pitzer's *acentric factor*, defined as $\omega = -\log_{10} P_r^{\mathrm{sat}}(T_r = 0.7) - 1$. Zero for spherical molecules (argon, methane), positive for elongated and hydrogen-bonded ones (acetic acid is on the high side at 0.47 because of its dimerisation in the vapour). Used by SRK/PR's $\alpha(T, \omega)$; carries it inertly.
`Tb`	K	Normal boiling point ($P^{\mathrm{sat}}(T_b) = 1$ atm). Anchors the Watson correlation: the temperature where the latent heat is tabulated. Also a sanity-check ruler for Antoine fits.
`HvapTb`	J/mol	Latent heat of vaporisation *at* T_b, the reference value Watson extends to other temperatures.
`Vliq`	m^3/mol	Liquid molar volume at 25 °C. The PFR reads it to convert molar feed to a volumetric flow $Q = F \cdot \bar{V}^{\mathrm{liq}}$ for the concentration $C_i = F_i/Q$; the stirred-tank sizing chain uses it for liquid hold-up. Treated as constant; pressure-volume corrections would be a ten-line edit.
`vaporPressure`	Antoine + Trange	$\log_{10} P^{\mathrm{sat}}_{[\mathrm{bar}]} = A - B/(T_{[\mathrm{K}]} + C)$. Used everywhere VLE shows up (flash, bubble-T, dew-T, distillation).
`idealGasHeatCapacity`	polynomial	$c_p^{\mathrm{ig}}(T) = a_0 + a_1 T + a_2 T^2 + a_3 T^3$, J/(mol · K). Drives the gas-phase part of every energy balance.
`liquidHeatCapacity`	polynomial	Same shape, for the saturated liquid. Drives the liquid-phase part of energy balances and the entropy integral below.

Intuition. The data file looks like a flat list of numbers because it *is* one: a thermodynamic ground truth the simulator's models do not re-derive, only consult. Every entry is either **measured** (Tc, Pc from Reid/Prausnitz/Poling tables; Tb, HvapTb from boiling-point calorimetry; Vliq from density at 25 °C; Antoine from regression of vapour-pressure tables; c_p from calorimetric measurements) or **empirical** (ω from a defined regression formula). Nothing is invented; nothing is fitted by the simulator at run time. Chapter 7 maps out which numbers in the whole stack *would* be fittable in principle, and which the simulator currently takes as fixed.

6.2 Pure-component enthalpy

The thermodynamic ground level is fixed by convention: the simulator sets

$$h_i^{\mathrm{liq}}(T_{\mathrm{ref}}) \equiv 0, \qquad T_{\mathrm{ref}} = 298.15\ \mathrm{K}, \tag{28}$$

for every component in the liquid phase. Departures from T_{ref} in the liquid follow from c_p^{liq}:

$$h_i^{\mathrm{liq}}(T) = \int_{T_{\mathrm{ref}}}^{T} c_{p,i}^{\mathrm{liq}}(T')\,\mathrm{d}T'. \tag{29}$$

With the polynomial $c_p^{\mathrm{liq}} = a_0 + a_1 T + a_2 T^2 + \dots$ shipped in the data file, the integral is term-by-term elementary; the simulator evaluates it in `src/thermo/heatCapacity/Polynomial.cpp`, called via `Component::Hliq_pure(T, Tref)`.

The vapour enthalpy of a pure component sits a latent heat above the liquid:

$$\boxed{h_i^{\mathrm{vap}}(T) = h_i^{\mathrm{liq}}(T) + \Delta h_i^{\mathrm{vap}}(T).} \tag{30}$$

The two terms are computed in `Component::Hvap_pure` / `Component::Hliq_pure` / `Component::Hvap_latent`; the next subsection is about how the latent heat at an arbitrary T is obtained from the single tabulated value $\Delta h_{T_b}^{\mathrm{vap}}$.

> *Intuition.* The choice $h_i^{\mathrm{liq}}(T_{\mathrm{ref}}) = 0$ is a reference zero, like sea level in altitude. For a unit with *no reaction* only differences enter the energy balance ($h_{\mathrm{out}} - h_{\mathrm{in}}$, $Q = \dot{n}\,\Delta h_{\mathrm{stage}}$), and the per-component zero cancels — a flash, a column, a heater, an evaporator all balance correctly on this datum. If you compare these h values against a table that picked $h_i^{\mathrm{ig}}(298) = 0$ for an *ideal gas* reference, they differ by exactly $\Delta h_i^{\mathrm{vap}}(T_{\mathrm{ref}})$.
> **But beware: this datum does NOT survive a chemical reaction.** With a separate zero *for each component*, the difference $h_B(T) - h_A(T)$ for a reaction $A \to B$ at fixed T is essentially zero (both are zeroed at T_{ref} plus a near-equal sensible term) — the heat of reaction has vanished. An energy balance written across a reactor on this datum is simply wrong. The next subsection fixes this.

6.3 The reference state, and why a reaction needs the elements

The sensible+latent datum of the previous subsection sets $h_i^{\mathrm{liq}}(T_{\mathrm{ref}}) = 0$ *per component* — every species gets its own private zero. That is harmless as long as the set of species is conserved (no reaction): in $h_{\mathrm{out}} - h_{\mathrm{in}}$ each species' zero appears once on each side and cancels.

A reaction breaks that bookkeeping. Consider the isomerization neo$C_5 \to n$-C_5 (used in the `process05_isomerization_recycle` tutorial). On the per-component datum, $h_{nC_5}(T) - h_{\mathrm{neo}C_5}(T) \approx 0$ at equal T, so an energy balance over the reactor reports $\Delta H \approx 0$. The *true* heat of reaction, however, is the difference of formation enthalpies,

$$\Delta h_{\mathrm{rxn}}(T_{\mathrm{ref}}) \;=\; \sum_i \nu_i\, \Delta h_{f,i}^{\circ} \;=\; \Delta h_{f,nC_5}^{\circ} - \Delta h_{f,\mathrm{neo}C_5}^{\circ} \;=\; -146{,}760 - (-168{,}050) \;=\; +21{,}290\ \mathrm{J/mol}, \tag{31}$$

where the index i runs over the *species* taking part in the reaction and ν_i is the **stoichiometric coefficient** of species i — *negative for reactants, positive for products.* For neo$C_5 \to n$-C_5 that is $\nu_{\mathrm{neo}C_5} = -1$ and $\nu_{nC_5} = +1$, so the sum is $(+1)\,\Delta h_{f,nC_5}^{\circ} + (-1)\,\Delta h_{f,\mathrm{neo}C_5}^{\circ}$ — "products minus reactants", each weighted by how many moles the balanced equation consumes or produces. (The same i indexes a component everywhere in this guide: x_i its mole fraction, h_i its molar enthalpy, $\Delta h_{f,i}^{\circ}$ its formation enthalpy.) The result is +21,290 J/mol, i.e. *endothermic* — the isothermal reactor must be *heated*. The per-component datum simply cannot see this number.

The cure is to measure every substance's enthalpy from the same floor: the **chemical elements it is made of**, each in its naturally stable form at 298.15 K, 1 bar.

What "the elements datum" actually means. By universal convention, each chemical element is assigned enthalpy *zero* in its standard state — its stable form at 298.15 K, 1 bar:

$$h^{\circ}(\mathrm{C,\ graphite}) = 0, \quad h^{\circ}(\mathrm{H_2, gas}) = 0, \quad h^{\circ}(\mathrm{O_2, gas}) = 0, \quad h^{\circ}(\mathrm{N_2, gas}) = 0, \ \ldots$$

The **standard enthalpy of formation** $\Delta h_{f,i}^{\circ}$ of a compound is then the heat of *building one mole of it from those elements.* For methane,

$$\mathrm{C(graphite)} + 2\,\mathrm{H_2(gas)} \;\longrightarrow\; \mathrm{CH_4(gas)}, \qquad \Delta h_{f,\mathrm{CH_4}}^{\circ} = -74{,}520\ \mathrm{J/mol}.$$

Since the elements on the left are worth zero, the compound's *absolute* enthalpy on this datum simply *is* its formation enthalpy: $h_{\mathrm{CH_4}}^{\circ}(298) = \Delta h_{f,\mathrm{CH_4}}^{\circ} = -74{,}520$ J/mol. Negative means methane sits *below* its elements — energy was released when it formed.

> *Intuition.* Think of altitude. Measuring each city's height from its own local sea level (the per-component datum) is useless for comparing a Portuguese city with a Spanish one. Pick *one* global sea level and all height differences become meaningful. The chemical elements are that global sea level for enthalpy: because *every* compound is measured from the same atomic floor, the atoms cancel in "products minus reactants" (mass is conserved in a reaction), and what is left over is exactly the heat of reaction. That is why this is the only datum that survives a reactor.

Worked example 6.1. Burning methane — the elements never appear in the sum, yet they make it workEveryone knows methane burns and releases a lot of heat. Let us *compute* how much, using only formation enthalpies, and watch the elements datum do its job. The balanced combustion is

$$\mathrm{CH_4} + 2\,\mathrm{O_2} \;\longrightarrow\; \mathrm{CO_2} + 2\,\mathrm{H_2O(g)}.$$

Standard formation enthalpies (kJ/mol), straight from the data files:

species	Δh_f° [kJ/mol]	note
CH_4	-74.5	a compound
O_2	0	**an element — zero by definition**
CO_2	-393.5	a compound
H_2O(g)	-241.8	a compound

Apply "products minus reactants", each weighted by its stoichiometric ν:

$$\Delta h_{\text{rxn}} = \big[\,1\cdot(-393.5) + 2\cdot(-241.8)\,\big] \;-\; \big[\,1\cdot(-74.5) + 2\cdot 0\,\big]$$
$$= (-877.1) - (-74.5) \;=\; \boxed{-802.6\ \text{kJ/mol}}.$$

Negative and large — strongly *exothermic*, exactly as the blue flame on a stove tells you. Two things to notice. First, O_2 contributed nothing (it is an element, its enthalpy is the zero of the scale) — yet had we measured each compound from its *own* private zero, the three formation numbers would have been on three incompatible scales and this subtraction would have been meaningless. The common elemental floor is what lets us add them. Second, no separate "heat of reaction" had to be supplied to the simulator: it is simply $h_{\text{out}} - h_{\text{in}}$ once every stream carries its Δh_f°.

This is the datum behind $\Delta h_{f,i}^\circ$ and the one `Component::h_pure_ig` already uses (introduced for the Gibbs reactor, Chapter 12). An ideal gas at temperature T is its formation enthalpy plus the sensible heat from 298.15 K up to T:

$$h_i^{\text{ig}}(T) \;=\; \underbrace{\Delta h_{f,i}^\circ}_{\text{build it from elements}} \;+\; \underbrace{\int_{298.15}^{T} c_{p,i}^{\text{ig}}(T')\,\mathrm{d}T'}_{\text{then heat it to } T}. \tag{32}$$

On this datum the liquid sits a latent heat *below* the ideal gas, so the elements-referenced liquid enthalpy is

$$\boxed{\,h_i^{\text{liq}}(T) \;=\; h_i^{\text{ig}}(T) \;-\; \Delta h_i^{\text{vap}}(T),\,} \tag{33}$$

and now *both* phases share the same zero. Because each species carries its $\Delta h_{f,i}^\circ$, the sum $\sum_i \nu_i h_i$ over a reaction returns the heat of reaction automatically — no separate Δh_{rxn} term is needed, it falls out of $h_{\text{out}} - h_{\text{in}}$. This is the datum every process simulator uses internally, for exactly this reason.

> *Intuition.* [The standard enthalpy-reference convention] The standard convention defines the enthalpy reference state as *the elements at 298.15 K*, with the ideal-gas enthalpy pinned to the standard heat of formation, $h_i^{\text{ig}}(T_{\text{ref}}) = \Delta h_{f,i}^\circ$ — the internal calculation path is elements → ideal-gas compounds at 298 K → ideal gas at T → real fluid at T, P, the same ladder as Figure 7. It is why a stream of pure ethanol at 298 K reports a *negative* enthalpy: that number is its (negative) heat of formation, not a bug. Choupo follows this convention, with the reference *pressure* pinned at 1 bar (the post-1982 NIST/JANAF convention against which the `S_ig` method was validated). The gap is *zero* for enthalpy (ideal-gas h does not depend on pressure) and a flat $R\ln(1.01325) \approx 0.11$ J/(mol · K) for entropy — negligible, and never accumulating.

> *Intuition.* Two datums, one rule. *Sensible+latent* (per-component zero) is fine for any non-reacting unit and is what the flash / column / evaporator unit ops use internally for their local duties. The *elements* datum (formation-enthalpy zero) is the only one that survives a reactor, so it is what the `energyBalance` report uses when every component carries a `gibbsFormation` block; the report falls back to the sensible datum (and flags the row) only when some species lacks Δh_f°. The two agree exactly on any unit where nothing reacts — which is why `flash01` reports the same 267 kW duty on either datum, while `process05`'s reactor only shows its +4.5 kW of reaction heat on the elements datum.

The whole picture fits on one ladder. Figure 7 stacks the states of a pure substance on a vertical enthalpy axis: the two candidate zeros, the two latent jumps (Δh^{fus}, Δh^{vap}), and the sensible ramps ($\int c_p\,dT$) that connect them. Read it once and the reference-state question stops being mysterious: an enthalpy value is just a *height* on this ladder, and a "datum" is just the choice of where to nail $h = 0$.

6.4 Which rung does the data file pin? The `phase` keyword

The ladder of Figure 7 shows that the formation enthalpy $\Delta h_{f,i}^\circ$ is a *distance*: from the elements baseline (the green dashed line) up to one of the rungs of the species i. But to which rung exactly? Different tables answer this question differently:

- NIST and JANAF tabulate $\Delta h_{f,i}^\circ$ in the **ideal-gas state at** 298.15 **K,** 1 bar for compounds that can be brought to the gas phase — water, ethanol, methane, the 45 standard volatiles and radicals shipped with Choupo. This is the standard convention used internally for every species.

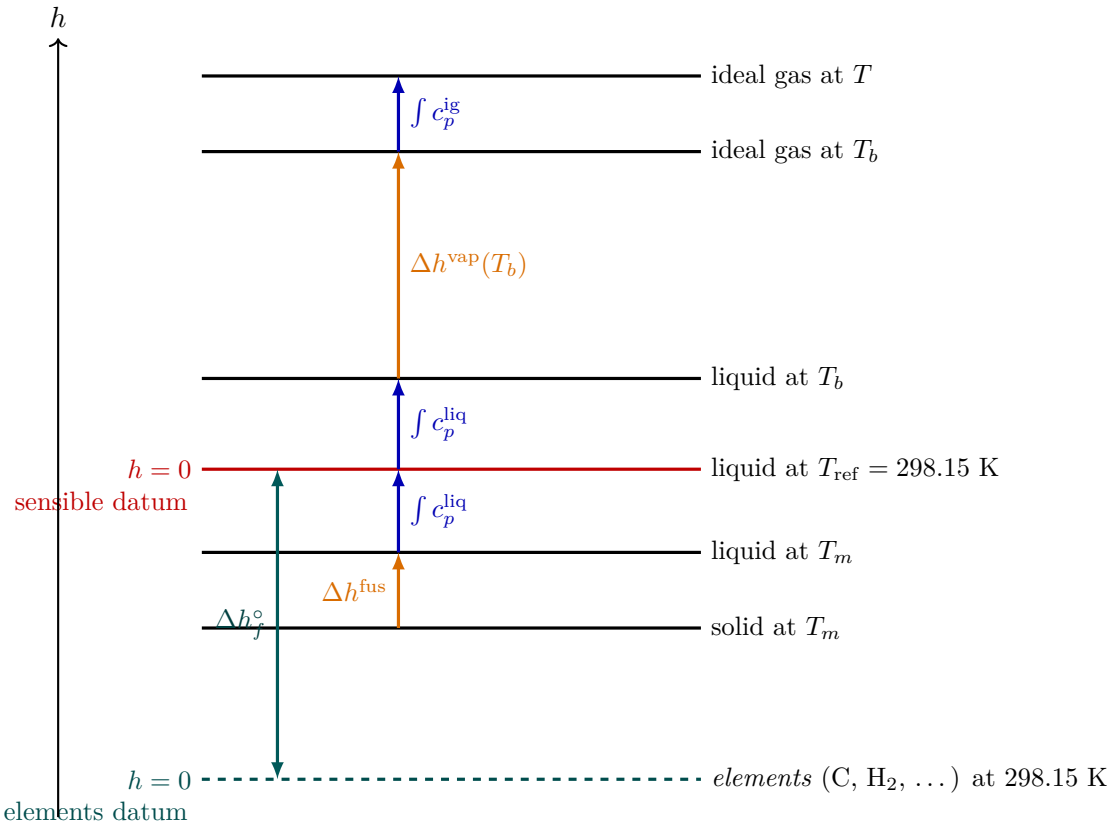

Figure 7: The enthalpy ladder of a pure substance (heights qualitative, not to scale). Each state is a rung; latent transitions (Δh^{fus}, Δh^{vap}) and sensible ramps ($\int c_p\,dT$) connect them. The **sensible datum** nails $h = 0$ at the liquid at T_{ref}, *per component* — convenient, but every species has its own zero, so it loses the heat of reaction. The **elements datum** nails $h = 0$ at the chemical elements; the rung-to-rung distance up to the substance is its formation enthalpy Δh_f°, and because all species are measured from the same elemental floor, $\sum_i \nu_i h_i$ over a reaction returns the reaction enthalpy automatically. Choupo's vapour enthalpy is $h^{\text{vap}} = h^{\text{liq}} + \Delta h^{\text{vap}}$ (one rung up); on the elements datum the liquid is $h^{\text{ig}} - \Delta h^{\text{vap}}$ (one rung down from the gas).

- For **crystalline non-volatiles** that decompose before they vaporise (sucrose, glucose, the bulk of solid solutes), NIST tabulates Δh_f° in the **solid** state — there is no honest ideal-gas value to give.
- For a small group of compounds (mostly experimentally-measured condensed-phase enthalpies of formation), the tabulated number is the **liquid** datum.

Choupo records the chosen rung as a one-word field inside the component's `gibbsFormation` block. Every entry in `data/standards/components/` declares it explicitly:

```
gibbsFormation
{
    dHf_298    -241826.0;        // J/mol  (ideal-gas reference)
    s_298      188.834;          // J/(mol*K)  (third-law absolute)
    phase      gas;              // <-- tells the solver which rung
}
```

```
gibbsFormation                   // sucrose, from NIST WebBook
{
    dHf_298    -2226100.0;       // J/mol  (crystalline)
    s_298          392.4;        // J/(mol*K)
    phase      solid;            // <-- never goes through a gas leg
}
```

The reason the field matters is operational. When the energy balance asks for $h_i^{\mathrm{liq}}(T)$ on the elements datum, the solver builds a path through the ladder:

$$\text{natural } \texttt{phase = gas}: \quad h_i^{\mathrm{liq}}(T) = \Delta h_{f,i}^{\circ} + \int_{298.15}^{T} c_p^{\mathrm{ig}}\, dT - \Delta h^{\mathrm{vap}}(T)$$

$$\text{natural } \texttt{phase = liquid}: \quad h_i^{\mathrm{liq}}(T) = \Delta h_{f,i}^{\circ} + \int_{298.15}^{T} c_p^{\mathrm{liq}}\, dT$$

$$\text{natural } \texttt{phase = solid}: \quad h_i^{\mathrm{liq}}(T) \approx \Delta h_{f,i}^{\circ} + \int_{298.15}^{T} c_p^{\mathrm{liq}}\, dT \qquad (c_p^{\mathrm{solid}} \approx c_p^{\mathrm{liq}},\ \Delta h^{\mathrm{fus}} \approx 0)$$

That is why a sucrose-bearing fermentation stream **does not need an ideal-gas c_p for sucrose**: the solver never integrates through a sucrose vapour leg, because sucrose's natural rung is on the solid line of the ladder and `phase = solid` tells it so. Forcing every species through a strict-IG path — as a naive strict-IG implementation would — would require the case author to invent an artificial c_p^{ig} for crystalline sucrose just so the equations work. Choupo refuses that compromise: the natural-phase convention keeps the per-species data *honest*, and the energy-balance report still closes through the reactor because every species carries its own Δh_f°.

> *Intuition.* Think of `phase` as the answer to "starting from the elements, where on the ladder does the data sheet stop?" For gases the table stops at the top rung (ideal gas at 298 K). For sucrose it stops at the bottom rung (the crystal). When the energy balance later asks the liquid enthalpy at some other temperature, the solver knows how to walk from the recorded rung to the one the balance needs — and how to refuse the walk when the latent jump it would need does not exist (sucrose → vapour throws a clear error rather than fabricate numbers).

Default for backward compatibility. If `phase` is absent the solver assumes `gas` (the NIST / JANAF convention, correct for the overwhelming majority of volatile species). All standard-catalogue components have been audited and declare `phase` explicitly. Per-case overlays under `<case>/constant/components/<name>.dat` are subject to the same rule: new entries should always declare `phase` so the datum is unambiguous at the source of truth.

6.5 Where every number comes from — the three questions a black-box tool won't answer

The `phase` keyword of the previous subsection is one instance of a larger discipline. The elements datum (§6.3) tells you *which floor* a formation enthalpy is measured from; it does not tell you *where the number on the data sheet came from*, *which model consumed it*, or *at which level of the case it was overridden*. A black-box simulator answers none of these: you pick "Property Method: NRTL-RK" from a dropdown and a thousand numbers are sourced, substituted, and defaulted silently behind it. Choupo's governing rule is the opposite — a student points at any number on the screen and asks *"where did THIS come from?"*, and the run has already answered it, at the point of use. Three questions, three rules.

(1) An "in-water" property is pair data wearing a convenience hat. The formation enthalpy of crystalline sucrose, $\Delta h_f^{\circ} = -2226.1$ kJ/mol, is *intrinsic*: its definition names sucrose and its elements, nothing else. Its enthalpy of *solution*, $\Delta h_{\mathrm{soln}} = +5.40$ kJ/mol (crystal → aqueous), is a different animal: its definition names *two* species, sucrose *and water*. That is *arity-2* data, and it does not live in `sucrose.dat` — it lives in a by-name catalogue tier, `data/standards/solution/sucrose-water.dat`, exactly as the infinite-dilution ion enthalpies live in `data/standards/electrolyte/ions.dat` on the $h_f^{\circ}(\mathrm{H}_{aq}^{+}) = 0$ convention. Water is ubiquitous enough to earn *one canonical, named aqueous reference tier* — never an *implied component slot*. When a solution property is needed and no solvent is named, the resolver applies the package default and *announces it*:

```
[thermo] dHsoln(sucrose): solvent not named -> DEFAULT solvent = water
         source: solution/sucrose-water.dat  [Putnam & Kilday 1986]
```

Off-default, it refuses with a remedy rather than silently substitute the water value. The student therefore always sees that a second species was assumed, which one, from which file, and from whose paper — the four things a dropdown hides.

(2) A model's parameters live in the model's own block. A cubic equation of state (SRK, PR) consumes only the corresponding-states triad $\{T_c, P_c, \omega\}$ and derives its own a_c, b, κ *in source* — adding PR after SRK touched no data file. A model that needs parameters it *cannot* derive — PC-SAFT's segment number m,

segment diameter σ, well depth ε/k — carries them in a *model-keyed* sub-block on the component, `eosParameters{ PCSAFT{ ...} }`, each with its own provenance. One molecule then carries SRK, PR, PC-SAFT and the next EOS simultaneously, never colliding, and the run announces *per component* whether its numbers were derived from corresponding states or read from a fitted block, and from whose paper:

```
[thermo] EOS = PR     -- benzene: a_c,b,kappa derived from Tc,Pc,omega (intrinsic)
[thermo] EOS = PCSAFT -- benzene: m,sigma,epsilon/k from eosParameters.PCSAFT
                         [Gross & Sadowski 2001]  origin: regressed
```

A model missing a required block fails with a remedy — never a silent corresponding-states fallback.

(3) A molecule is whole at every level. The case tree is fractal (case ⊃ sector ⊃ unit), and a lower node may *overlay* a refined number when its scope makes it more true — a particular powder's measured sorption isotherm, say. But the overlay is a *patch, never a fork*: the molecule's identity $(\mathrm{MW}, T_c, \omega)$ is born once, at the standard tier, and is the same molecule at every level. The overlay merges *block-by-block* — override the whole `solid{}` reference-state block, never a lone scalar inside it (a lone-scalar overlay would silently splice a sample's ρ_p onto the catalogue's k_v, a hidden hybrid). And a number that is really a *rate* (crystallisation kinetics) or a *geometry* (a particle-size distribution) is not component data at all — it belongs to the equipment, in the operation's `constant/`, and the PSD travels as a *stream* attribute. The cascade line tells the student which level won.

> *Intuition.* *All models are wrong, some are useful* — and Choupo tells you exactly *which* one you used and *where its numbers came from*, so you judge the usefulness yourself. A datum's home is decided by **arity** (does its definition name a second species?), then by **scope** (at what level is it true?), then by **kind** (is it the molecule, or the molecule-in-a-machine?). Nothing — no solvent, no model, no level — is ever implicit on disk. The curator's full decision procedure is `docs/ai/data-doctrine.md`.

6.6 The Gibbs side: s°_{298}, third-law entropy, and the absolute ideal-gas free energy

The previous subsections settled the *enthalpy* half of the formation datum: $\Delta h^\circ_{f,i}$ pins the substance's height above the elements floor. But the Gibbs reactor of Chapter 12 minimises the *Gibbs* energy, and Eq. (283) needs the absolute standard-state free energy $g^\circ_i(T)$, which carries an entropy term. This subsection derives the second half of the `gibbsFormation` block — the field s°_{298} — and the chain `Component::g_pure_ig` walks to assemble $g^\circ_i(T)$.

Why entropy needs a *different* reference from enthalpy. Enthalpy has no natural zero, so we *chose* one (the elements). Entropy does have a natural zero: the third law of thermodynamics fixes $s = 0$ for a perfect crystal at $T = 0$ K. The number tabulated in s°_{298} is therefore the **absolute (third-law) entropy** of the substance at 298.15 K, 1 bar — the integral of c_p/T all the way up from 0 K, plus the entropy of every phase change crossed on the way:

$$s^\circ_i(298) \;=\; \int_0^{298.15} \frac{c_{p,i}(T')}{T'}\,\mathrm{d}T' \;+\; \sum_{\text{transitions}} \frac{\Delta h^{\text{trs}}}{T^{\text{trs}}}. \tag{34}$$

This is *not* an "entropy of formation" — it is an absolute value, and the elements carry a non-zero s°_{298} of their own. That distinction is why the data file stores Δh°_f (a *difference* from the elements) but a plain absolute s°_{298} (a *third-law* value). The reaction's entropy change reconstructs automatically as $\sum_i \nu_i\, s^\circ_i$, because the atoms cancel exactly as they did for enthalpy.

From the two stored numbers to $g^\circ_i(T)$. Both stored values are pinned at 298.15 K; the simulator transports them to the reactor temperature T by integrating the ideal-gas heat capacity. Enthalpy uses $\int c_p\,dT$, entropy uses $\int c_p/T\,dT$:

$$h^\circ_i(T) = \Delta h^\circ_{f,i} \;+\; \int_{298.15}^{T} c^{\text{ig}}_{p,i}(T')\,\mathrm{d}T', \tag{35}$$

$$s^\circ_i(T) = s^\circ_i(298) \;+\; \int_{298.15}^{T} \frac{c^{\text{ig}}_{p,i}(T')}{T'}\,\mathrm{d}T', \tag{36}$$

$$\boxed{\; g^\circ_i(T) \;=\; h^\circ_i(T) \;-\; T\,s^\circ_i(T). \;} \tag{37}$$

These are exactly `Component::h_pure_ig(T)`, `Component::s_pure_ig(T)` and `Component::g_pure_ig(T)` in `src/thermo/Component.`
The two integrals are the H and S methods of the heat-capacity model (`src/thermo/heatCapacity/`); for the polynomial form $c^{\text{ig}}_p = a_0 + a_1T + a_2T^2 + a_3T^3$ both are elementary:

$$\int_{298.15}^{T} c_p\,dT' = \sum_k \frac{a_k}{k+1}\left(T^{k+1} - 298.15^{k+1}\right), \quad \int_{298.15}^{T} \frac{c_p}{T'}\,dT' = a_0 \ln\frac{T}{298.15} + \sum_{k\geq 1} \frac{a_k}{k}\left(T^k - 298.15^k\right).$$

The *element-potential* solver of the Gibbs reactor reads only the combination $g_i^\circ(T)/RT$ (it appears as `g_pure_ig(T)/RT` in `gibbsMethod/ElementPotential.cpp`), so the 1 bar standard reference cancels into the $\ln(y_i P)$ term of Eq. (283) and never appears as a separate offset.

Worked example 6.2. Reforming: the entropy term decides the equilibriumThe water-gas-shift reaction $CO + H_2O \rightleftharpoons CO_2 + H_2$ has almost no entropy change (two moles to two moles), so it is dominated by Δh; but the steam-reforming reaction $CH_4 + H_2O \rightleftharpoons CO + 3\,H_2$ goes from two moles of gas to four, and Δs° is large and positive. Below ~ 900 K the $-T\Delta s$ term in $\Delta g^\circ = \Delta h^\circ - T\Delta s^\circ$ is too small to overcome the endothermic $\Delta h^\circ \approx +206$ kJ/mol, and equilibrium sits on the methane side; raise T and the entropy term flips the sign of Δg°, driving conversion to syngas. A Gibbs reactor *cannot* reproduce this temperature swing from Δh_f° alone — it needs each species' s_{298}°. That is why the `gibbsFormation` block carries *both* numbers, and why a component lacking s_{298}° cannot be placed in a Gibbs reactor.

6.7 Where the two numbers come from: the NASA-CEA import

A component's Δh_f° and s_{298}° are *curated*, not computed at run time — they are measured thermochemistry the simulator only consults. Choupo's gas-phase formation data are imported from the **NASA Glenn** thermodynamic database (NASA-7 polynomials), via the curation tool `bin/curate/import_gibbs_nasa.py`. The primary source is McBride, Gordon & Reno, NASA TM-4513 (1993) [1], a public-domain US-government work.

The connection between a NASA-7 polynomial and the two fields Choupo needs is direct and exact. The NASA polynomials are themselves referenced to the elements, so the standard-state enthalpy they return at 298.15 K *is* the enthalpy of formation, and the entropy they return is the third-law absolute entropy:

$$\Delta h_{f,i}^\circ \;=\; H_i^0(298.15), \qquad s_i^\circ(298) \;=\; S_i^0(298.15), \tag{38}$$

which is precisely what the import reads off the polynomial (`standard_enthalpies_RT` and `standard_entropies_R` evaluated at 298.15 K, 1 atm).

Two glass-box guards (why the import is trustworthy). The tool refuses to write a number it cannot defend:

- **Anchor verify.** Before any file is touched, the tool re-derives Δh_f° for five well-known species (CO_2, H_2O, NH_3, benzene, CO) and aborts if any deviates from the textbook value by more than 0.5 kJ/mol — a unit-conversion or transcription slip is caught at the door.
- **Isomer guard.** NASA names many species by formula only, and the trap is real: "C3H6O" in TM-4513 is *propylene oxide* ($\Delta h_f^\circ \approx -93.7$ kJ/mol), not acetone (-217.1). Every name-to-NASA mapping therefore carries an *expected* Δh_f°, and the import **refuses** any species whose NASA value deviates by more than 5 kJ/mol. A wrong-isomer match is a transcription error to catch, never a number to trust.

A real, curated measured value already in `data/standards/components/` is never overwritten; the import only *fills* an absent block or *refits* one tagged as an estimate (Joback, `origin=estimated`). Species with no clean, isomer-unambiguous NASA match (acetone, the longer n-alkanes, the xylenes, diethyl ether) are deliberately left carrying their honestly-tagged Joback estimate rather than a guessed value — the gap is kept visible, not papered over.

> *Intuition.* [Power and limit] **Power:** two small numbers per species — Δh_f° and s_{298}° — plus a $c_p^{\mathrm{ig}}(T)$ polynomial are enough to give the absolute Gibbs energy of *any* ideal-gas mixture at *any* temperature, which is all the element-potential equilibrium solver needs. The data are public-domain, element-referenced, and audited. **Limit:** the chain is strictly *ideal-gas*. $g_i^\circ(T)$ carries no pressure correction beyond the ideal $RT\ln(y_i P)$, no real-gas fugacity, and no liquid- or solid-phase free energy — so a Choupo Gibbs reactor is a gas-phase equilibrium tool. And the values are only as good as the NASA fit: TM-4513 c_p polynomials are reliable to $\sim$ 1000–6000 K depending on species, and an estimated (Joback) entry can be off by several kJ/mol, which the per-value provenance tag makes the student check before trusting an equilibrium prediction.

6.8 Latent heat at any T: Watson

Each data file lists $\Delta h_i^{\mathrm{vap}}(T_b)$ at the normal boiling point only. The simulator transports it to any other temperature with Watson's correlation,

$$\boxed{\Delta h_i^{\mathrm{vap}}(T) \;=\; \Delta h_i^{\mathrm{vap}}(T_b)\left(\frac{T_c - T}{T_c - T_b}\right)^{0.38}.} \tag{39}$$

[1] B. J. McBride, S. Gordon, M. A. Reno, *Coefficients for Calculating Thermodynamic and Transport Properties of Individual Species*, NASA Technical Memorandum 4513 (1993). A US-government work, in the public domain; also the data behind the NASA CEA program (Gordon & McBride, NASA RP-1311, 1994).

This is not a fit; it is a dimensionally motivated power law that collapses to $\Delta h^{\text{vap}} \to 0$ as $T \to T_c$ (vapour and liquid become indistinguishable) and reproduces the tabulated $\Delta h^{\text{vap}}(T_b)$ at $T = T_b$ by construction. The exponent 0.38 is the empirical Watson value; Pitzer's later acentric-factor formulation refines it but the simple form is good to a few percent across the practically interesting range $T_b < T < 0.95\,T_c$.

Implementation: `Component::Hvap_latent(T)` in `src/thermo/Component.cpp`, 6 lines.

Worked example 6.3. Acetic acid latent heat at $T = 350$ KWith $T_c = 591.95$ K, $T_b = 391.05$ K, $\Delta h^{\text{vap}}_{T_b} = 24{,}390$ J/mol from `data/standards/components/aceticAcid.dat`,

$$\Delta h^{\text{vap}}(350) \;=\; 24390 \cdot \left(\frac{591.95 - 350}{591.95 - 391.05}\right)^{0.38} \;\approx\; 24390 \cdot (1.2042)^{0.38} \;\approx\; 26{,}120 \text{ J/mol}.$$

Acetic acid is harder to evaporate at 350 K than at its boiling point (by $\sim 7\%$). The trend is universal: Δh^{vap} *decreases* as T rises toward T_c.

6.9 Pure-component entropy and Gibbs energy

Entropy is the analogous integral against c_p/T,

$$s_i^{\text{liq}}(T) - s_i^{\text{liq}}(T_{\text{ref}}) \;=\; \int_{T_{\text{ref}}}^{T} \frac{c_{p,i}^{\text{liq}}(T')}{T'}\,\mathrm{d}T', \tag{40}$$

and the vapour entropy is a latent step above:

$$s_i^{\text{vap}}(T) \;=\; s_i^{\text{liq}}(T) \;+\; \frac{\Delta h_i^{\text{vap}}(T)}{T}. \tag{41}$$

With h and s in hand, the pure-component Gibbs energy follows by the Legendre transform that defines it,

$$g_i(T) \;=\; h_i(T) - T\,s_i(T), \tag{42}$$

and the same reference convention ($g_i^{\text{liq}}(T_{\text{ref}}) = -T_{\text{ref}}\,s_i(T_{\text{ref}})$) is carried through.

> *Intuition.* None of the unit ops actually call s or g directly — the energy balances are all in h alone, because the systems considered are non-reactive plus elementary kinetics where ΔG of reaction is not yet needed. When chemical-equilibrium reactors land (post-v1), the same Cp / Watson chain feeds $\Delta G_r = \Delta h_r - T\,\Delta s_r$ for the equilibrium constant $K_{\text{eq}}(T) = \exp(-\Delta G_r/RT)$. Wiring is one afternoon's work; the data are already in place.

6.10 Mixing: ideal versus real

The simulator's mixture enthalpy is the ideal-mixing combination of the pure values, weighted by phase fractions:

$$h_{\text{mix}}(T) \;=\; (1 - V)\sum_i x_i\,h_i^{\text{liq}}(T) \;+\; V\sum_i y_i\,h_i^{\text{vap}}(T), \tag{43}$$

exactly as implemented in `ThermoPackage::Hmixture`. This assumes the *excess enthalpy* h^E of mixing is negligible. For Raoult systems (benzene/toluene, n-hexane/n-heptane) and moderately non-ideal systems with weak hydrogen bonding, h^E is indeed tens of J/mol against pure h's of tens of kJ/mol — a sub-percent correction. For strongly hydrogen-bonded mixtures (ethanol/water, acetic acid/water) h^E can reach ~ 500 J/mol and the energy balance will under-predict, e.g., the duty of a condenser by a fraction of a percent up to a few percent.

The Gibbs energy of mixing is where activity coefficients enter. For a single-phase mixture

$$\boxed{\Delta G_{\text{mix}} \;=\; RT\sum_i x_i \ln\!\big(x_i\gamma_i\big) \;=\; \underbrace{RT\sum_i x_i \ln x_i}_{\text{ideal mixing, always} \le 0} \;+\; \underbrace{RT\sum_i x_i \ln\gamma_i}_{\equiv\, G^E,\ \text{excess Gibbs}}\,.} \tag{44}$$

The ideal part is the universal $-T\,\Delta s^{\text{ideal}}_{\text{mix}} = RT\sum x_i \ln x_i$ that always drives mixing forward at finite T for components that mix at all. The excess part G^E is what NRTL, Wilson and friends actually parameterise, via $\ln\gamma_i = \partial(n\,G^E/RT)/\partial n_i$ at constant $T, P, n_{j\neq i}$. A consistency check is built in: an activity model has to satisfy the integrability constraint that follows next.

6.11 Gibbs-Duhem: the consistency check

At constant T and P, partial molar quantities of a mixture are not independent. The Gibbs-Duhem identity for the activity coefficients reads

$$\boxed{\sum_i x_i\,\mathrm{d}(\ln\gamma_i) \;=\; 0 \quad (\text{constant } T, P).} \tag{45}$$

It says: you cannot fit $\ln\gamma_1$ and $\ln\gamma_2$ to data independently and freely — once $\ln\gamma_1(x_1)$ is chosen, $\ln\gamma_2(x_1)$ is determined up to a constant of integration fixed by the pure-limit $\gamma_i(x_i \to 1) = 1$.

For a binary system the constraint linearises to a useful test on experimental data: at a series of $(x_1, \gamma_1, \gamma_2)$ measurements, the area under $\ln(\gamma_1/\gamma_2)$ over the full composition range must integrate to zero:

$$\int_0^1 \ln\frac{\gamma_1(x_1)}{\gamma_2(x_1)}\,\mathrm{d}x_1 \;=\; 0. \tag{46}$$

This is the celebrated Redlich-Kister consistency test. When the `fitParameters` regression fits NRTL or Wilson parameters to a (x_1, T_{bubble}) dataset (the `tutorials/fitNRTL01_ethanol_water` case), the chosen functional form for G^E *automatically* satisfies Eq. (45) by construction — NRTL and Wilson are both derived from a G^E model, and any pair of $\ln\gamma_i$ derived from a single G^E obeys Gibbs-Duhem identically. The fit quality therefore measures how well the data themselves are thermodynamically consistent: a poor LM convergence is sometimes the data's fault, not the optimiser's.

Common pitfall. Gibbs-Duhem is exact for a binary mixture as written, but its appearance changes in n-component systems — there it becomes $\sum_i x_i\,\mathrm{d}\ln\gamma_i = 0$ along any infinitesimal path in composition space. Most published binary γ-data sets satisfy GD to within experimental noise; ternary data sets often do not (a hint that one of the three binaries is missing the cross-term). If you ever fit a ternary by combining three independent binary NRTL pairs, you have implicitly trusted the GD consistency of each pair.

6.12 Putting it together: an example energy balance

Take an isobaric heater that brings 100 kmol/h of equimolar ethanol/water from $T_{\text{in}} = 300$ K to $T_{\text{out}} = 360$ K, no phase change ($V = 0$ throughout, liquid only). The duty per kmol of feed is

$$\begin{aligned} q \;&=\; h_{\text{liq}}(360) - h_{\text{liq}}(300) \\ &=\; \sum_i x_i \left[h_i^{\text{liq}}(360) - h_i^{\text{liq}}(300)\right] \\ &=\; \sum_i x_i \int_{300}^{360} c_{p,i}^{\text{liq}}(T)\,\mathrm{d}T. \end{aligned} \tag{47}$$

Using the constant-c_p approximations shipped (ethanol: $c_p^{\text{liq}} \approx 165$ J/(mol · K); water: ≈ 75.4 J/(mol · K)), this evaluates to

$$q \;\approx\; 0.5 \cdot 165 \cdot 60 + 0.5 \cdot 75.4 \cdot 60 \;\approx\; 7{,}212 \text{ J/mol feed.}$$

At 100 kmol/h that is a duty of about $7{,}212 \cdot 10^5/3600 \approx 200$ kW — which is exactly what `tutorials/hx01_heater` reports when you run it. Every step uses only Eqs. (28)–(43) and the polynomial c_p from the data file.

Runnable case. `runCase tutorials/hx01_heater` executes the duty calculation above end-to-end and writes the result to the run log. Modify `tutorials/hx01_heater/system/flowsheetDict` to change T_{out} and watch how Q scales linearly with ΔT — a direct consequence of constant c_p.

7 What is fittable, what is not

Every model in this Part has parameters that ultimately come from *somewhere*. The simulator distinguishes three sources, in descending order of trust:

1. **Literature curation.** Antoine constants (A_i, B_i, C_i), critical properties (T_c, P_c, ω), normal boiling points, latent heats, heat-capacity polynomials, liquid molar volumes — these live in `data/standards/components/<name>.dat` and are cited per-file at the top of each file. They are *not* estimated by the simulator. To add a new component, copy an existing `.dat` file and populate it from DIPPR / Reid-Prausnitz-Poling [18] / NIST.
2. **Literature pair coefficients.** NRTL $(a_{ij}, b_{ij}, a_{ji}, b_{ji}, \alpha_{ij})$ and Wilson Λ_{ij} live in `data/standards/binaryPairs/<model>/<c1>-<c` also with provenance. The same DECHEMA / RPP sources apply.
3. **In-simulator regression.** For binary interaction parameters *only*, the simulator can refit (a, b) against an experimental T_b-vs-x dataset by the Levenberg-Marquardt outer driver of Chapter 4. Pure-component properties cannot be refitted by the simulator — if your candidate fluid is missing Antoine constants, you must either find them in the literature or estimate them by a group-contribution method (Lydersen, Joback, UNIFAC) outside the simulator, then write them into a `.dat` file.

Common pitfall. Pure-component property gaps will silently bend any flowsheet result. If `<name>.dat` has Antoine constants fitted only between 273 and 373 K, do not run a simulation that requires P^{sat} at $T = 600$ K — the extrapolation can give negative pressures. The simulator does not yet enforce `validityRange`; that is on the bug-watch.

Part III — Mixtures: from ideal to real

1 Ideal mixing

What you'll learn. Before any activity coefficient, every multi-component thermodynamic model is anchored in an *ideal* reference: a hypothetical mixture where molecules of different species feel no preference for who their neighbours are. This chapter builds the ideal-mixture chemical potential $\mu_i^{\mathrm{id}} = \mu_i^* + RT\ln x_i$ piece by piece (the $\ln x_i$ term is pure combinatorial entropy, no model), shows how it gives the ideal-gas mixture (Dalton's law) and the ideal-solution mixture (Raoult's law) as special cases, and ends with the binary $\Delta G_{\mathrm{mix}}^{\mathrm{id}}$ curve that explains why mixing is thermodynamically favourable at any finite temperature. The next chapter promotes γ_i to the role of the excess factor that captures *real* non-ideality on top of this ideal baseline.

1.1 What "ideal mixing" means

Take N_A molecules of species A and N_B molecules of species B, mix them at constant T and P. An *ideal* mixture is the limit in which:

- the volume of mixing is zero ($\Delta V_{\mathrm{mix}} = 0$);
- the enthalpy of mixing is zero ($\Delta H_{\mathrm{mix}} = 0$);
- the molecules retain their identity — A feels the same molecular field whether surrounded by A or by B.

These three statements are equivalent under mild regularity assumptions, and together they fix *every* mixture property once the pure-component properties are known. The only thing that changes on mixing is the **configurational entropy**: there are many more ways to arrange N_A A's and N_B B's among a fixed set of sites than there are ways to arrange the pure components separately. This is the only "mixing" effect.

1.2 The combinatorial entropy of mixing

Place $N = N_A + N_B$ indistinguishable-within-species molecules on a lattice. The number of distinct arrangements is the binomial coefficient

$$W \;=\; \binom{N}{N_A} \;=\; \frac{N!}{N_A!\,N_B!}. \tag{48}$$

Boltzmann's statistical entropy is $S = k_B \ln W$. For molar amounts ($N_i = x_i N_{\mathrm{Avogadro}}$), Stirling's approximation gives, per mole of mixture,

$$\boxed{\Delta s_{\mathrm{mix}}^{\mathrm{id}} \;=\; -R\sum_i x_i \ln x_i.} \tag{49}$$

This is a *pure* consequence of how many ways the molecules can be arranged — there is no fitted constant, no model assumption about intermolecular forces. It is positive (since $x_i < 1$ and $\ln x_i < 0$), maximised at the equimolar composition where the combinatorial freedom is greatest, and zero at the pure-component limits.

Intuition. The $-R\sum x_i \ln x_i$ term shows up in every mixing calculation that follows — ideal or not, gas or liquid, in chemical-equilibrium expressions, in distillation ΔG profiles. It is the same function that appears in Shannon's information entropy of a discrete distribution, for the same combinatorial reason. Engineers and information theorists rediscovered each other's result decades apart.

1.3 The ideal-mixture chemical potential

Combining $\Delta s_{\mathrm{mix}}^{\mathrm{id}} = -R\sum x_i \ln x_i$ with $\Delta h_{\mathrm{mix}}^{\mathrm{id}} = 0$ gives the molar Gibbs energy of mixing

$$\Delta g_{\mathrm{mix}}^{\mathrm{id}} \;=\; \Delta h_{\mathrm{mix}}^{\mathrm{id}} - T\Delta s_{\mathrm{mix}}^{\mathrm{id}} \;=\; RT\sum_i x_i \ln x_i. \tag{50}$$

The full molar Gibbs energy of the mixture is then the sum of the pure-component values, weighted by mole fraction, plus this mixing correction:

$$g^{\mathrm{id}}(T,P,\mathbf{x}) \;=\; \sum_i x_i\, g_i^*(T,P) \;+\; RT\sum_i x_i \ln x_i, \tag{51}$$

where g_i^* is the pure-component molar Gibbs energy. Take the partial derivative with respect to n_i at constant $T, P, n_{j\neq i}$ to recover the **ideal-mixture chemical potential**:

$$\boxed{\mu_i^{\mathrm{id}}(T,P,\mathbf{x}) \;=\; \mu_i^*(T,P) \;+\; RT\ln x_i.} \tag{52}$$

Equation (52) is the single most important formula in mixture thermodynamics. The pure-component term μ_i^* carries all the physical chemistry of species i alone (saturation pressure, latent heat, c_p); the $RT\ln x_i$ term is the combinatorial correction — universal across substances and models.

Intuition. Notice what (52) does *not* contain: any parameter that depends on which other species i is mixed with. In an ideal mixture the chemical potential of A at $x_A = 0.3$ is the same whether A is dissolved in B, C, or in $A + B + C$. That is exactly the simplification that breaks for real mixtures, where neighbour identity matters. The activity coefficient γ_i of the next chapter is the multiplicative correction (52) needs — $\mu_i = \mu_i^* + RT\ln(x_i\gamma_i)$ — to absorb that effect.

1.4 Ideal-gas mixtures and Dalton's law

Specialise (52) to a gas mixture in which every component obeys the ideal-gas law. The pure-component chemical potential is

$$\mu_i^{*,\mathrm{ig}}(T,P) \;=\; \mu_i^{0,\mathrm{ig}}(T) \;+\; RT\ln(P/P_0), \tag{53}$$

with P_0 a reference pressure (1 bar). Substituting into (52) and using $P_i = y_i P$ (the partial pressure):

$$\mu_i^{\mathrm{ig,mix}}(T,P,\mathbf{y}) \;=\; \mu_i^{0,\mathrm{ig}}(T) \;+\; RT\ln(y_iP/P_0) \;=\; \mu_i^{0,\mathrm{ig}}(T) \;+\; RT\ln(P_i/P_0). \tag{54}$$

The mixture chemical potential is therefore identical to that of pure i at its partial pressure P_i. Going back through the ideal-gas equation of state, this is exactly Dalton's law:

$$P \;=\; \sum_i P_i, \qquad P_i \;=\; y_iP. \tag{55}$$

For the simulator: the fugacity coefficient of every species in an ideal-gas mixture is $\varphi_i = 1$ *by construction*. This is the vapour-phase model.

1.5 Ideal solutions and Raoult's law

Apply (52) to a liquid mixture in which the intermolecular forces between unlike molecules are the same as between like ones — the classical *ideal solution*. The pure-component liquid chemical potential at saturation is

$$\mu_i^{*,\mathrm{L}}(T,P) \;=\; \mu_i^{*,\mathrm{V,sat}}(T,P_i^{\mathrm{sat}}) \;\approx\; \mu_i^{0,\mathrm{ig}}(T) \;+\; RT\ln(P_i^{\mathrm{sat}}/P_0), \tag{56}$$

where the second equality uses ideal-gas vapour fugacity. Phase equilibrium $\mu_i^{\mathrm{L,mix}} = \mu_i^{\mathrm{V,mix}}$ then gives, after cancelling the reference,

$$\boxed{\;y_iP \;=\; x_i\,P_i^{\mathrm{sat}}(T).\;} \tag{57}$$

This is **Raoult's law**: the partial pressure of species i above an ideal solution is its pure-component saturation pressure weighted by its liquid mole fraction. In the simulator, the activity coefficient $\gamma_i = 1$ *by construction*.

Raoult's law is exact for ideal solutions; for many real systems (chemically similar species, e.g. benzene/toluene, n-hexane/n-heptane) it is accurate to a percent or so. Where it breaks — ethanol/water, acetic acid/water, ethyl acetate/ethanol — is the territory of activity coefficients in the next chapter.

1.6 The driving force for mixing

For a binary at any composition, (50) reads

$$\Delta g_{\mathrm{mix}}^{\mathrm{id}}(x_1) \;=\; RT\big[x_1\ln x_1 + (1-x_1)\ln(1-x_1)\big], \tag{58}$$

which is concave-up and negative, with its minimum at $x_1 = 0.5$ where $\Delta g_{\mathrm{mix}}^{\mathrm{id}} = -RT\ln 2 \approx -1.72$ kJ/mol at 300 K. Because $\Delta g_{\mathrm{mix}}^{\mathrm{id}} < 0$ for every $x_1 \in (0,1)$, **ideal mixing is always spontaneous**. This is why miscibility is the rule rather than the exception for chemically similar liquids; breaking it requires real intermolecular forces that the ideal model cannot represent.

Intuition. The cusps at $x_1 = 0$ and $x_1 = 1$ are characteristic of combinatorial mixing: an infinitesimal amount of impurity in a pure liquid produces a finite drop in g, because the configurational entropy gain $-Rx\ln x \to +\infty$ as $x \to 0^+$. This is the thermodynamic root of why ultra-purification is hard: every additional decade of purity costs an infinite Gibbs energy.

1.7 Where the ideal limit goes wrong

Ideal mixing is the right *reference*, not the right *description*, for most real liquids:

- **Chemically similar species, weak interactions (alkanes, aromatics):** Raoult holds to within 1–5%. Activity coefficients are close to 1; the ideal model is useful directly.
- **Polar / non-polar pairs (water with hydrocarbons):** $\gamma_i \gg 1$, often > 10. Ideal mixing under-predicts the escape tendency of one component in the other by orders of magnitude; liquid-liquid splits become possible.
- **Hydrogen-bonded systems (ethanol/water, acid/water):** γ_i varies smoothly but non-trivially across composition; can hit azeotropes (minimum or maximum on the bubble curve) that the ideal model cannot represent at all.

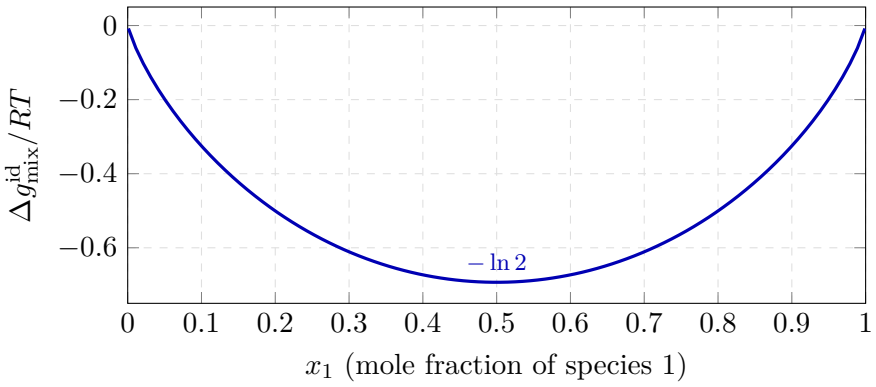

Figure 8: Ideal-mixture Gibbs energy of mixing for a binary at fixed T. Negative everywhere except at the pure-component endpoints ($x_1 = 0$ or 1, where the curve is tangent to zero with a vertical slope from the ln singularity). The minimum at $x_1 = 0.5$ is exactly $-RT \ln 2$.

- **Strong specific interactions (acid/base, hydrogen- bond donors with acceptors):** $\gamma_i < 1$ (negative deviations), maximum-boiling azeotropes; again outside the ideal-mixture description.

The activity coefficient γ_i is the multiplicative correction that puts (52) back on top of the experimental data:

$$\mu_i(T, P, \mathbf{x}) \;=\; \mu_i^*(T, P) \;+\; RT \ln(x_i\, \gamma_i) \;=\; \mu_i^{\mathrm{id}}(T, P, \mathbf{x}) \;+\; RT \ln \gamma_i. \tag{59}$$

The next chapter is about how that correction is parametrised (NRTL, Wilson) and how the excess Gibbs energy G^E generates γ_i through a partial-derivative recipe.

2 Activity coefficients

What you'll learn. Three models cover the simulator's needs. They share a common derivation: an expression for excess Gibbs energy G^E from which γ_i falls out by differentiation.

2.1 The excess Gibbs energy bridge

For a real liquid mixture, the molar Gibbs energy is split as

$$G^L(T,P,\mathbf{x}) \;=\; G^{L,\text{ideal}}(T,P,\mathbf{x}) \;+\; G^E(T,\mathbf{x}). \tag{60}$$

The activity coefficient is the composition derivative:

$$\boxed{\ln\gamma_i \;=\; \frac{1}{R_\text{g}T}\left(\frac{\partial(n\,G^E)}{\partial n_i}\right)_{T,P,n_{j\neq i}}} \tag{61}$$

Pick a model for G^E, do the derivative, and you have an activity model. That is what NRTL, Wilson, UNIQUAC and friends are: each is one choice of $G^E(\mathbf{x},T)$.

2.2 Ideal solution

If $G^E = 0$, then $\gamma_i = 1$ for every component. Raoult's law: $y_i P = x_i P_i^{\text{sat}}(T)$. Useful as a baseline, a sanity check, and a starting point for the Wegstein outer loop.

2.3 NRTL (Renon and Prausnitz, 1968)

The model that earns its keep in this simulator is NRTL — the *Non-Random Two-Liquid* model. It is the default for non-ideal VLE and the engine behind every liquid–liquid split: NRTL is the only G^E model in Choupo that can represent demixing, because (unlike Wilson, §2.4) its denominators admit the doubly-tangent G^E shape that two coexisting liquids require. Everything below is implemented in `src/thermo/activityCoefficient/NRTL.cpp`.

The local-composition idea. Renon and Prausnitz's [36] starting point — the same postulate that underlies Wilson (66) — is that a real liquid is not randomly mixed. Around a central molecule of type i, the local mole fraction of j-neighbours, x_{ji}, is biased

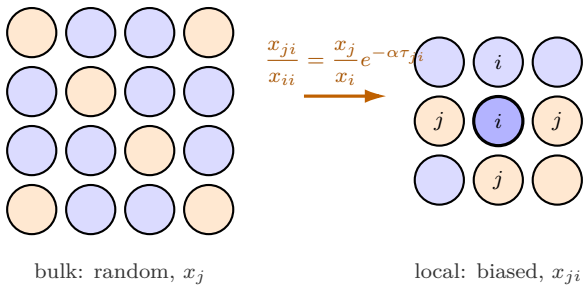

Figure 9: The local-composition postulate behind NRTL and Wilson. In a *randomly* mixed liquid (left) every molecule's neighbour shell reflects the bulk fractions x_j. Real mixtures are not random: around a central molecule i, the local fraction of j-neighbours x_{ji} is re-weighted by a Boltzmann factor in the pair-energy difference (right). NRTL's twist is the extra non-randomness constant α in the exponent — $\alpha \to 0$ recovers the random regular-solution lattice, larger α sharpens the local ordering. Every τ_{ij} and Λ_{ij} in the activity-coefficient formulas is just this biased shell, counted.

away from the bulk x_j by the difference in pair energy. NRTL's distinctive twist is a *two-liquid* Boltzmann weighting that carries an extra non-randomness factor α:

$$\frac{x_{ji}}{x_{ii}} \;=\; \frac{x_j}{x_i}\,\exp\!\Big[-\alpha_{ji}\,\frac{g_{ji}-g_{ii}}{R_\text{g}T}\Big], \tag{62}$$

where g_{ji} is the energy of a j–i pair and α_{ji} *quenches* the non-randomness: $\alpha \to \infty$ forces complete

local order, $\alpha = 0$ recovers a random (regular-solution) mixture. In practice α is an adjustable but *transferable* constant, taken symmetric ($\alpha_{ij} = \alpha_{ji}$) and in the narrow band $\alpha \in [0.20, 0.47]$: $\sim$0.2 near ideality, 0.3 for typical polar mixtures (the simulator's default when a pair file omits it), $\sim$0.47 for strongly self-associating or phase-splitting systems.

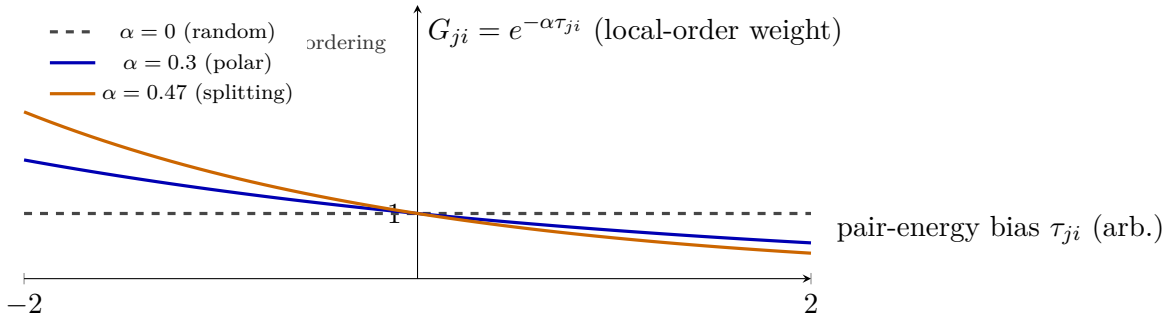

Figure 10: What the NRTL non-randomness constant α actually does. The factor $G_{ji} = e^{-\alpha\tau_{ji}}$ is the weight a j-neighbour gets in the shell around an i-molecule. At $\alpha = 0$ (flat dashed line) $G = 1$ for every bias — the shell is random and NRTL collapses to a regular solution. Cranking α up steepens the curve, so the same pair-energy bias τ produces a sharper deviation of the local shell from the bulk: $\alpha \approx 0.3$ for typical polar mixtures (Choupo's default when a pair file omits it), $\alpha \approx 0.47$ for the strongly self-associating, phase-splitting systems where the extra ordering is what carries the liquid–liquid split. Because α is held fixed during a fit, a poor choice cannot be repaired by the energy parameters.

The grouped parameters. Collecting the two quantities the model actually stores,

$$\boxed{\tau_{ij} \;=\; \frac{g_{ij} - g_{jj}}{R_{\mathrm{g}} T} \;=\; a_{ij} + \frac{b_{ij}}{T}, \qquad G_{ij} \;=\; \exp(-\alpha_{ij}\,\tau_{ij})} \tag{63}$$

with the diagonal conventions $\tau_{ii} = 0$, $G_{ii} = 1$, $\alpha_{ii} = 0$. Two facts matter. First, τ is *asymmetric*: $\tau_{ij} \neq \tau_{ji}$ in general, because the energy to place a j next to an i differs from the reverse once each is referenced to its own pure-fluid pair. Second, the temperature dependence is the affine form $\tau_{ij} = a_{ij} + b_{ij}/T$ — exactly the storage in the code: each ordered pair carries four energy parameters $(a_{ij}, b_{ij}, a_{ji}, b_{ji})$ and one shared α_{ij} (the `aMat_`, `bMat_`, `alphaMat_` matrices). Here a_{ij} is the high-T residual (entropic, temperature-independent) and b_{ij} [K] is the enthalpic slope. A b-only fit ($a = 0$) forces $\tau \to 0$ as $T \to \infty$ — the mixture becomes ideal when thermal energy swamps the pair energy — whereas carrying $a \neq 0$ admits a residual that survives.

Excess Gibbs energy. Substituting the local compositions (62) into the two-fluid lattice energy and referencing to the pure components gives

$$\frac{G^E}{R_{\mathrm{g}} T} \;=\; \sum_{i=1}^{N_c} x_i \, \frac{\displaystyle\sum_{j=1}^{N_c} \tau_{ji} G_{ji} x_j}{\displaystyle\sum_{k=1}^{N_c} G_{ki} x_k}. \tag{64}$$

Activity coefficient. Apply the partial-molar derivative (61). Write $n_i = n x_i$ and abbreviate the denominators $S_j \equiv \sum_k G_{kj} x_k$ (these are `S[j]` in the code) and the numerators $D_j \equiv \sum_k \tau_{kj} G_{kj} x_k$ (`Tsum[j]`). Differentiating $nG^E/R_{\mathrm{g}}T = \sum_i n_i D_i / S_i$ with respect to n_i at fixed $n_{m \neq i}$ — using $\partial S_j / \partial n_i = (G_{ij} - S_j)/n$ and $\partial D_j / \partial n_i = (\tau_{ij} G_{ij} - D_j)/n$ — the $1/n$ factors cancel and the result is the standard form

$$\boxed{\ln \gamma_i \;=\; \underbrace{\frac{\displaystyle\sum_j \tau_{ji} G_{ji} x_j}{\displaystyle\sum_k G_{ki} x_k}}_{D_i / S_i} \;+\; \sum_j \frac{x_j G_{ij}}{S_j} \left[\tau_{ij} - \frac{D_j}{S_j} \right]} \tag{65}$$

which is term-for-term the loop in `NRTL::gamma()`: `termA` is D_i/S_i, `termB` accumulates the j-sum, and the two are exponentiated. Pre-computing S_j and D_j once per temperature (rather than inside every i, j pass) is the only optimisation; the algebra is otherwise the textbook formula. Check a hand derivation against Renon and Prausnitz [36] or Reid–Prausnitz–Poling [18], Eq. 8-10.

Intuition. Read (65) as "the environment i creates plus the environments i disturbs." The first term is how strongly i is held by its own surroundings; the sum is the back-reaction — i entering each cell j and changing its tally. At infinite dilution ($x_i \to 0$) the cell of the majority solvent dominates and $\ln \gamma_i^{\infty}$ collapses to a closed expression in the pair parameters — the quantity most sensitive to a fit.

Pairs and the ideal-default rule. Binary parameter sets are tabulated in the DECHEMA Vapor–Liquid Equilibrium Data Collection or regressed from data; the tutorials carry literature values from Reid–Prausnitz–Poling [18]. The simulator resolves each pair in a fixed precedence (`locatePairFile`): an inline `pairs(...)` block in the `thermoPackage`, then a per-node `constant/binaryPairs/NRTL/<a>-<b>.dat`, then the case root, then `data/standards/binaryPairs/NRTL/`. For an N_c-component mixture there are $N_c(N_c-1)/2$ binaries; NRTL is a strictly *pairwise* model with *no* ternary parameters, so a multicomponent prediction is only as good as its constituent binaries.

A pair with no parameters anywhere does *not* abort the run: it **defaults to ideal** ($\tau = 0 \Rightarrow$ no G^E contribution; α set to 0.3 harmlessly). This is deliberate — it lets a thermo package be built one pair at a time — but it is never silent: every ideal-defaulted pair is announced on `stdout` and in the advisory log, so the student *sees* exactly which interactions are unconstrained. To produce one's own fitted set from a T_b-vs-x dataset, run the parameter-estimation driver of Chapter 4 (`tutorials/steady/optimisation/fitNRTL01_ethanol_water`).

Power. Two energy parameters per direction plus one non-randomness constant span ideality, strong positive deviation, and liquid–liquid immiscibility within a single algebraic form; the affine $\tau(T)$ extrapolates across a modest temperature range; and the model is analytically differentiable (used for the bubble-point Jacobian).

Limit. NRTL is a *fitting* model, not a predictive one: the parameters have no first-principles values and must come from data (contrast UNIFAC, which predicts them from group contributions). It carries no information about the vapour or the solid — it gives γ for the liquid only; you still supply P^{sat} from Antoine (§2) and any K_{sp} separately. The affine $\tau(T)$ is calorimetrically uncalibrated unless each pair is refit against measured H^E (the `calorimetricFit` gate that NRTL exposes), so excess-enthalpy predictions from a VLE-only fit are not to be trusted. And because α is held fixed during a fit, a poor α choice cannot be repaired by the energy parameters.

2.4 Wilson (1964)

Simpler in form than NRTL, faster numerically (no α, no nested G-exponentials), and built on a more physical picture — but with one structural cost: it *cannot represent* a liquid-liquid split. That is a limitation of the functional form, not of the fit, and the reason is worth understanding rather than memorising (§2.4).

The idea: local compositions. Wilson's [37] starting point is that the composition *immediately around* a central molecule is not the bulk composition. If molecule 1 attracts molecule 2 more strongly than it attracts its own kind, then the local mole fraction of 2 near a 1 is enriched above x_2. Wilson postulated that these *local* mole fractions x_{ji} (species j around a central i) are related to the bulk ones through Boltzmann factors of the interaction energies λ_{ji}:

$$\frac{x_{ji}}{x_{ii}} \;=\; \frac{x_j \,\exp(-\lambda_{ji}/R_{\mathrm{g}}T)}{x_i \,\exp(-\lambda_{ii}/R_{\mathrm{g}}T)}. \tag{66}$$

Weighting each local environment by the molar volume of the central species and collecting the result into an excess Gibbs energy yields the one-line model below. The same local-composition postulate underlies NRTL and UNIQUAC; Wilson is its earliest and leanest realisation.

Excess Gibbs energy.

$$\boxed{\frac{G^E}{R_{\mathrm{g}}T} \;=\; -\sum_{i=1}^{N_c} x_i \ln\!\left(\sum_{j=1}^{N_c} x_j \Lambda_{ij}\right)} \tag{67}$$

with the asymmetric, dimensionless interaction parameter

$$\Lambda_{ij} \;=\; \frac{V_j^L}{V_i^L}\,\exp\!\Big[-\underbrace{(\lambda_{ij}-\lambda_{ii})}_{A_{ij}}\big/(R_{\mathrm{g}}T)\Big], \qquad \Lambda_{ii}=1. \tag{68}$$

Here V_i^L is the pure-component liquid molar volume and λ_{ij} the energy of an i–j interaction. Only the *difference* $A_{ij} \equiv \lambda_{ij} - \lambda_{ii}$ ever appears, so Wilson carries **two** energy parameters per binary, A_{ij} and A_{ji} (in J/mol), and these are *asymmetric*: $A_{ij} \neq A_{ji}$ in general, hence $\Lambda_{ij} \neq \Lambda_{ji}$. There is no third "non-randomness" factor as in NRTL.

From G^E to γ_i. Apply the partial-molar recipe (61) to (67). Writing $n_i = nx_i$ and $nG^E/R_gT = -\sum_i n_i \ln\big(\sum_j n_j\Lambda_{ij}/n\big)$, the derivative $\partial/\partial n_i$ contributes one term from the explicit ln at i and one from every other molecule's denominator in which i appears. After re-dividing by n the result collapses to the classic two-term form:

$$\boxed{\ln\gamma_i \;=\; 1 - \ln\left(\sum_j x_j\Lambda_{ij}\right) \;-\; \sum_k \frac{x_k\,\Lambda_{ki}}{\sum_j x_j\,\Lambda_{kj}}} \tag{69}$$

This is exactly the loop in `src/thermo/activityCoefficient/Wilson.cpp`: the code first builds the Λ matrix from (68), precomputes $S_k = \sum_j x_j\Lambda_{kj}$ once, then assembles (69) as $1 - \ln S_i - \sum_k x_k\Lambda_{ki}/S_k$.

Built-in consistency. Two checks fall out for free, and the code reproduces them. At infinite dilution of i in pure j ($x_j \to 1$) the second sum keeps only the $k = j$ term, giving the limiting activity coefficient

$$\ln\gamma_i^{\infty} \;=\; 1 - \ln\Lambda_{ij} - \Lambda_{ji}, \tag{70}$$

a convenient handle for fitting to a single γ^{∞} datum. And in the pure limit $x_i \to 1$ every $S_k \to \Lambda_{ki}$ and (69) returns $\ln\gamma_i = 1 - \ln 1 - 1 = 0$, i.e. $\gamma_i \to 1$ as it must.

Temperature dependence — and an honest caveat. The T-dependence in Choupo's Wilson is *implicit only*, through the Boltzmann factor $\exp(-A_{ij}/R_gT)$ and the (weakly T-varying) volume ratio in (68): the energy parameters A_{ij} themselves are stored as single constants in J/mol, not as a T-expansion. This is deliberately *unlike* the NRTL implementation, whose $\tau_{ij} = a_{ij} + b_{ij}/T$ (64) carries an explicit linear-in-$1/T$ term. If a wider temperature range demands it, the physically standard refinement is to let $A_{ij} = A^0_{ij} + A^1_{ij}\,T$ (a linear-in-T energy, the form used by DECHEMA for Wilson); that extra slope is not yet wired into the loader, so over a broad T window prefer a pair fitted near the operating temperature. The single-constant choice keeps the glass-box small: one number per direction, visible in the `.dat`.

Wilson vs NRTL at a glance.

	Wilson	**NRTL**
Parameters / binary	A_{ij}, A_{ji} (2)	$a_{ij}, b_{ij}, a_{ji}, b_{ji}, \alpha$ (5)
Needs V_i^L?	yes (liquid molar volume)	no
T-model in Choupo	$\exp(-A/R_gT)$ only	explicit $a + b/T$ in τ
Liquid-liquid split	**impossible**	yes (its main strength)
Cost per γ eval	one ln, one nested sum	two nested sums + exp

Why Wilson cannot demix. A liquid-liquid split requires G^E to be non-convex in composition (a region where $\partial^2 G/\partial x^2 < 0$). For a binary, $G^E/R_gT = -x_1\ln(x_1 + x_2\Lambda_{12}) - x_2\ln(x_2 + x_1\Lambda_{21})$ is mathematically incapable of producing two minima for any positive

$\Lambda_{12}, \Lambda_{21}$ — the single logarithm per term keeps the curve convex. No choice of parameters fixes this; it is the price of the one-parameter-pair-per-direction simplicity. For partially miscible systems (e.g. the LLE Gibbs-minimisation of Chapter 9) the simulator falls back to NRTL or UNIQUAC, which carry the extra freedom needed for a double-well G^E.

Parameter sources. Binary (A_{ij}, A_{ji}) sets live in `data/standards/binaryPairs/Wilson/<i>-<j>.dat` (or a case-local `constant/binaryPairs/Wilson/`); a pair with no file anywhere *defaults to ideal* for that binary ($\Lambda = 1$, no contribution) and the run announces it aloud rather than aborting — the same no-silent-crutch hybrid used by NRTL. Literature values are tabulated in the DECHEMA collection and Reid–Prausnitz–Poling [18]; fit your own with the parameter-estimation driver of Chapter 4.

Worked example 2.1. Wilson γ for ethanol(1)/water(2) at 350 K, $x_1 = 0.20$Parameters $A_{12} = 1157.95$, $A_{21} = 4081.65$ J/mol (the values shipped in the `ethanol-water.dat` Wilson pair) and liquid molar volumes $V_1^L = 5.87 \times 10^{-5}$, $V_2^L = 1.81 \times 10^{-5}$ m^3/mol. With $R_gT = 8.314 \times 350 = 2910$ J/mol:

$$\Lambda_{12} = \tfrac{V_2}{V_1}e^{-A_{12}/R_gT} = \tfrac{1.81}{5.87}\,e^{-0.398} = 0.207, \quad \Lambda_{21} = \tfrac{V_1}{V_2}e^{-A_{21}/R_gT} = \tfrac{5.87}{1.81}\,e^{-1.403} = 0.797.$$

At $x_1 = 0.20$: $S_1 = x_1 + x_2\Lambda_{12} = 0.20 + 0.80(0.207) = 0.366$ and $S_2 = x_2 + x_1\Lambda_{21} = 0.80 + 0.20(0.797) = 0.959$. Then from (69),

$$\ln\gamma_1 = 1 - \ln 0.366 - \left[\frac{x_1\Lambda_{11}}{S_1} + \frac{x_2\Lambda_{21}}{S_2}\right] = 1 - (-1.005) - \left[\tfrac{0.20}{0.366} + \tfrac{0.80\cdot 0.797}{0.959}\right] \approx 0.84,$$

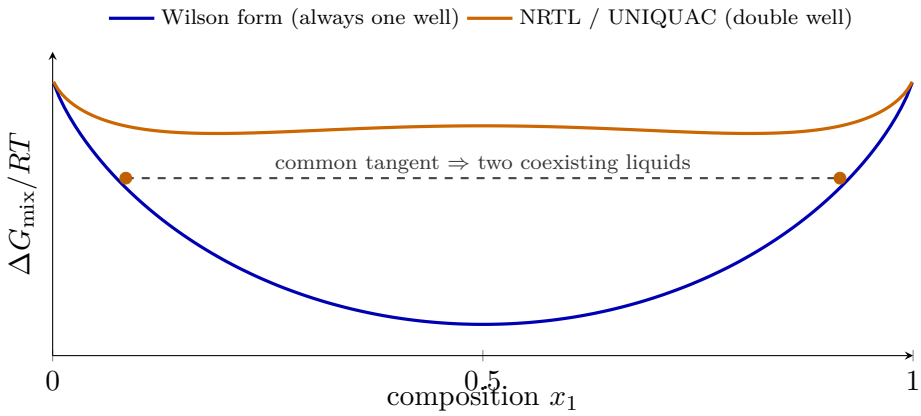

Figure 11: Why the model's *form* — not its fit — decides whether a liquid can split. A liquid–liquid split needs ΔG_{mix} to be non-convex: a hump with two wells, so a common tangent touches two distinct compositions (the orange curve). Wilson's single logarithm per term, $G^E/RT = -x_1 \ln(x_1 + x_2\Lambda_{12}) - x_2 \ln(x_2 + x_1\Lambda_{21})$, is mathematically incapable of producing that hump for any positive Λ — it stays convex (blue), so no parameter set makes Wilson demix. NRTL and UNIQUAC carry the extra freedom for a double well, which is exactly why Choupo's LLE Gibbs-minimisation accepts those two and refuses Wilson.

so $\gamma_1 \approx 2.3$. Ethanol is strongly non-ideal here — the same qualitative verdict as the NRTL example above, with a different model and two fewer parameters.

2.5 UNIQUAC (Abrams and Prausnitz, 1975)

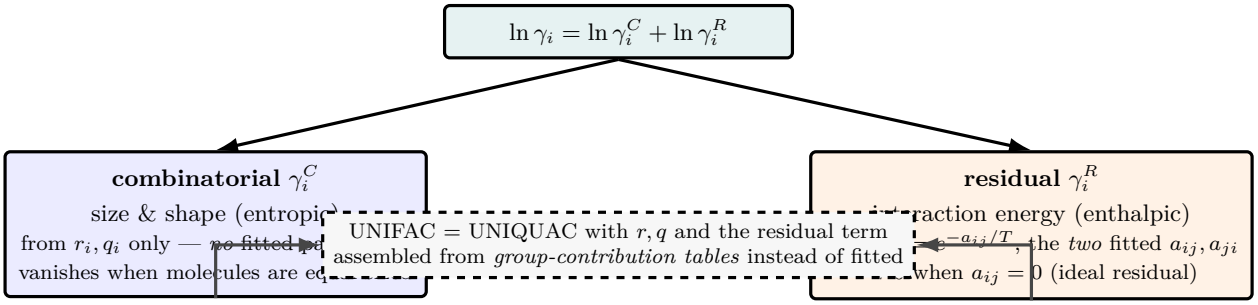

Figure 12: UNIQUAC's defining move: split the excess Gibbs energy into two physically distinct pieces. The **combinatorial** term (blue) carries the entropic penalty of packing molecules of unequal size and shape — it needs only the structural constants r_i, q_i and has *zero* fitted parameters, vanishing entirely when all molecules are the same size. The **residual** term (orange) carries the interaction energy through $\tau_{ij} = e^{-a_{ij}/T}$ and holds the *two* binary energies (a_{ij}, a_{ji}) — one fewer than NRTL, which also needs α. Because the size/shape physics lives in r, q, this is the exact structure UNIFAC reuses: keep the combinatorial term as is and build the residual from group-contribution tables instead of a binary fit.

NRTL and Wilson are *energy-only* models: they describe non-ideality through interaction parameters and quietly assume the molecules are the same size and shape. That assumption frays for mixtures of very unequal molecules — a polymer in a solvent, a sugar in water, an alkane in an alcohol — where much of the deviation from ideality is *entropic*, born of the difficulty of packing big and small molecules onto a common lattice. UNIQUAC (UNIversal QUAsi-Chemical) [40] splits the excess Gibbs energy into two physically distinct pieces and gives each its own term:

$$\frac{G^E}{R_{\mathrm{g}}T} = \underbrace{\frac{G^E_{\mathrm{comb}}}{R_{\mathrm{g}}T}}_{\text{size/shape (entropic)}} + \underbrace{\frac{G^E_{\mathrm{res}}}{R_{\mathrm{g}}T}}_{\text{energy (enthalpic)}}, \qquad \Longrightarrow \qquad \ln\gamma_i = \ln\gamma_i^C + \ln\gamma_i^R. \tag{71}$$

The *combinatorial* part γ_i^C depends only on *structural* constants; the *residual* part γ_i^R carries the two adjustable

binary energies. This is exactly the structure that lets UNIFAC *predict* γ from group counts: UNIFAC is UNIQUAC with r, q and the residual term assembled from group-contribution tables instead of being fitted.

Structural constants. Each component carries two pure-component numbers, computed from its van der Waals group volumes and surface areas (Bondi, 1968):

$$r_i \;=\; \text{relative molecular } \textit{volume}, \qquad q_i \;=\; \text{relative molecular } \textit{surface area}. \tag{72}$$

These are not fitted — they are tabulated constants the student *cites* (Poling–Prausnitz–O'Connell [18], Table 8E; original values in [40]). In the dict they live in an `rq { ...}` block, one $\{r;\, q;\}$ pair per component; the engine refuses to run if any is missing (no silent default for a structural property). From r and q the model builds, at a given composition, the *segment fraction* ϕ_i and *area fraction* θ_i:

$$\phi_i \;=\; \frac{r_i x_i}{\sum_j r_j x_j}, \qquad \theta_i \;=\; \frac{q_i x_i}{\sum_j q_j x_j}. \tag{73}$$

Two limits aid intuition: if all r_i are equal then $\phi_i = x_i$, and if additionally all q_i are equal then $\theta_i = x_i$ and the combinatorial term vanishes — equal-sized molecules carry no packing penalty.

Combinatorial (size/shape) term. The entropic piece is a Staverman–Guggenheim lattice count, with activity coefficient

$$\ln\gamma_i^{C} \;=\; \ln\frac{\phi_i}{x_i} \;+\; \frac{z}{2}\,q_i \ln\frac{\theta_i}{\phi_i} \;+\; l_i \;-\; \frac{\phi_i}{x_i}\sum_j x_j l_j, \qquad l_i \;=\; \frac{z}{2}(r_i - q_i) - (r_i - 1), \tag{74}$$

where the *lattice coordination number* is fixed at $z = 10$ (each molecule is taken to have ten nearest neighbours — the canonical UNIQUAC value, hard-coded as `z_coord_` in the engine, not a tunable). This term contains *no* adjustable parameters: feed it the r's and q's and it is fully determined. It alone already predicts the bulk of athermal (zero-energy) non-ideality.

Residual (energy) term. The enthalpic piece uses the *quasi-chemical* idea — a molecule's surroundings are biased by interaction energies through Boltzmann factors

$$\tau_{ij} \;=\; \exp\!\Big(-\frac{u_{ij} - u_{jj}}{R_{\mathrm{g}} T}\Big) \;=\; \exp\!\Big(-\frac{a_{ij}}{T}\Big), \qquad a_{ij} \;=\; \frac{u_{ij} - u_{jj}}{R_{\mathrm{g}}} \;\;[\mathrm{K}], \tag{75}$$

with u_{ij} the i–j interaction energy. Note $\tau_{ii} = 1$ and the matrix is asymmetric ($\tau_{ij} \neq \tau_{ji}$); each binary contributes *two* energies a_{ij}, a_{ji} — one fewer than NRTL, which also needs α. The engine stores a_{ij} in *kelvin* and supports an optional linear extension $a_{ij}^{\mathrm{eff}} = a_{ij} + b_{ij}\,T$ for sets fitted over a wide temperature range (most literature sets use a alone, $b = 0$). The residual activity coefficient is

$$\ln\gamma_i^{R} \;=\; q_i \left[\, 1 - \ln\!\Big(\sum_j \theta_j \tau_{ji}\Big) - \sum_j \frac{\theta_j \tau_{ij}}{\sum_k \theta_k \tau_{kj}} \,\right]. \tag{76}$$

Equations (74) and (76) are exactly what `src/thermo/activityCoefficient/UNIQUAC.cpp` evaluates in `gamma()` — read the two side by side.

Parameter sources and the ideal default. The two energies (a_{ij}, a_{ji}) per binary come from VLE regression (DECHEMA, or the parameter-estimation driver of Chapter 4), supplied inline as `pairs (...)` or from a UNIQUAC pair catalogue (`constant/binaryPairs/UNIQUAC/<a>-<b>.dat`, the same precedence NRTL and Wilson use). A binary with no parameters anywhere defaults to $a_{ij} = a_{ji} = 0$, i.e. $\tau = 1$ (ideal residual contribution), and the run *announces* the defaulted pairs aloud — the same no-silent-crutch hybrid as NRTL and Wilson.

Power and limit. UNIQUAC needs only *two* energy parameters per binary (vs three for NRTL) yet, because the size/shape physics is built in through r and q, it extrapolates over composition and to multicomponent mixtures better than NRTL or Wilson for molecules of disparate size. Unlike Wilson it *can* describe liquid–liquid demixing (its G^E admits a double well), so the LLE Gibbs-minimisation of Chapter 9 accepts it alongside NRTL. And it is the structural parent of UNIFAC, so a UNIQUAC *fit* and a UNIFAC *prediction* are directly comparable on the same components. The **limit**: the two-parameter residual term is less flexible than NRTL's three-parameter form for a single tightly-fitted binary — for a strongly hydrogen-bonded pair fitted on its own data NRTL often edges it. The combinatorial term is a mean-field lattice count that sees molecular shape only through the single number q, so it is imperfect for very flat or branched species. And, like every G^E model, it is silent about the vapour ($\varphi^{\mathrm{sat}} = 1$ here) and about ions (use eNRTL/Pitzer, §3).

Worked example 2.2. UNIQUAC γ for ethanol(1)/water(2) at 351 K, $x_1 = 0.20$The `tutorials/steady/flash/bubbleT02_uniquac_ethanol_water` system. Structural constants $r_1 = 2.1055,\ q_1 = 1.9720,\ r_2 = 0.9200,\ q_2 = 1.4000$; energies $a_{12} = -21.44\,\mathrm{K}$, $a_{21} = 222.47\,\mathrm{K}$.
Step 1 — τ at $T = 351\,K$ (Eq. 75): $\tau_{12} = e^{21.44/351} = 1.063$, $\tau_{21} = e^{-222.47/351} = 0.531$ (and $\tau_{11} = \tau_{22} = 1$).
Step 2 — fractions (Eq. 73) at $x = (0.20, 0.80)$: $\sum r_j x_j = 1.157$, $\sum q_j x_j = 1.514$, hence $\phi_1 = 0.364,\ \theta_1 = 0.260$ (and $\phi_2 = 0.636,\ \theta_2 = 0.740$).
Step 3 — combinatorial (Eq. 74) with $z = 10$: $l_1 = 5(2.1055 - 1.9720) - (2.1055 - 1) = -0.438$, $l_2 = -2.320$, $\sum x_j l_j = -1.944$, giving $\ln\gamma_1^C = 0.398$.
Step 4 — residual (Eq. 76): $\sum_j \theta_j\tau_{j1} = 0.260 + 0.740(0.531) = 0.653$, the second sum evaluates to 0.247, so $\ln\gamma_1^R = 1.972\,[\,1 - \ln 0.653 - 0.247\,] = 0.501$.
Result: $\ln\gamma_1 = 0.398 + 0.501 = 0.899 \Rightarrow \gamma_1 \approx 2.46$ (and $\gamma_2 \approx 1.12$). Driving $x_1 \to 0$ gives the infinite-dilution value $\gamma_1^\infty \approx 7.7$ — the strongly-non-ideal range that makes ethanol/water a minimum-boiling azeotrope, and the very value the tutorial's `thermoPackage` energies were fitted to reproduce.

Runnable case. `tutorials/steady/flash/bubbleT02_uniquac_ethanol_water` — the bubble temperature of 30/70 ethanol/water at 1 atm, landing on the same $T_{\mathrm{bubble}} \approx 354\,\mathrm{K}$ as its NRTL and Wilson twins: a clean three-way model comparison on identical components.

2.6 Worked example

Worked example 2.3. NRTL γ for ethanol/water at 350 K, $x_1 = 0.20$Components 1 =ethanol, 2 =water. Literature parameters (the same set carried in `data/standards/binaryPairs/NRTL/ethanol-water.dat`, from [18]): $a_{12} = -0.8009,\ b_{12} = 246.18\,\mathrm{K},\ a_{21} = 3.4578,\ b_{21} = -586.0809\,\mathrm{K},\ \alpha = 0.30$.
Step 1 — τ and G at $T = 350\,K$ from (63):

$$\tau_{12} = -0.8009 + \tfrac{246.18}{350} = -0.0975, \quad \tau_{21} = 3.4578 - \tfrac{586.08}{350} = \quad 1.7833,$$

$$G_{12} = e^{-0.3(-0.0975)} = 1.0297, \quad G_{21} = e^{-0.3(1.7833)} = 0.5857.$$

Step 2 — the binary $\ln\gamma_1$. For two components (65) reduces to a two-term sum ($\tau_{11} = \tau_{22} = 0$, $G_{11} = G_{22} = 1$). With $x_1 = 0.20$, $x_2 = 0.80$ the denominators are $S_1 = x_1 + G_{21}x_2 = 0.20 + 0.5857(0.80) = 0.6686$ and $S_2 = G_{12}x_1 + x_2 = 1.0297(0.20) + 0.80 = 1.0059$, so

$$\ln\gamma_1 = \frac{\tau_{21}G_{21}x_2}{S_1} + \frac{x_2G_{12}}{S_2}\left[\tau_{12} - \frac{\tau_{12}G_{12}x_1}{S_2}\right] = 1.250 + (-0.063) = 1.186,$$

giving $\gamma_1 \approx 3.27$.
Step 3 — read it. Ethanol is strongly non-ideal at this dilute composition; Raoult's law ($\gamma_1 = 1$) would under-predict the ethanol partial pressure threefold. Pushing $x_1 \to 0$ in the same formula gives the infinite-dilution value $\gamma_1^\infty \approx 5.4$ — the steepest, most fit-sensitive point of the curve. At the other end the same parameters place the well-known minimum-boiling azeotrope near $x_1 \approx 0.894$, where $\gamma_1 \approx \gamma_2 P^{\mathrm{sat}}$ balance crosses (`tutorials/batch/still/still02_ethanol_water_azeo`).

Runnable case. `tutorials/steady/flash/flash02_ethanol_water` — watch the Wegstein outer loop converge in ∼7 iterations vs ∼29 by direct substitution.

2.7 UNIFAC: predicting γ_i from molecular structure

NRTL and Wilson are *correlative*: they reproduce a binary's non-ideality only after someone has fitted (a_{ij}, b_{ij}, α) or (A_{ij}, A_{ji}) to measured VLE for *that pair*. Confront them with a binary no one has measured and they have nothing to say. UNIFAC (**UNI**QUAC **F**unctional-group **A**ctivity **C**oefficients) is *predictive*: it computes $\gamma_i(T, \mathbf{x})$ from the *functional groups* that make up each molecule, using one published parameter table shared across all of chemistry. No binary fit. The headline glass-box win is to predict the ethanol–water azeotrope from structure alone and then *see* the residual error against data. UNIFAC is the group-contribution realisation of UNIQUAC [38, 18].

The central idea: a solution of groups. A molecule is decomposed into a multiset of *subgroups* (e.g. ethanol = 1

`CH3` + 1 `CH2` + 1 `OH`). The mixture is then treated as a solution not of molecules but of those groups. A `CH2` in ethanol is assumed to interact with a `H2O` exactly as a `CH2` in any other molecule would — the *group-additivity hypothesis*. This is what lets one parameter matrix (group m against group n) serve every conceivable molecule built from those groups.

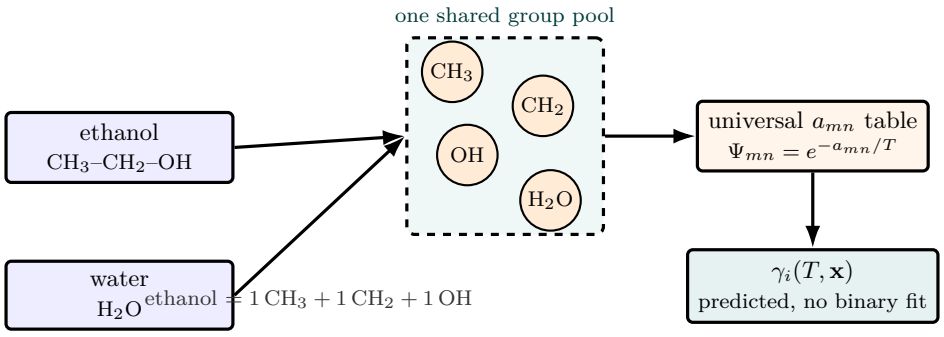

Figure 13: UNIFAC's central idea, drawn: a molecule is dissolved into a *multiset of functional groups* (ethanol = CH_3+CH_2+OH), and the mixture is treated as a solution of those groups rather than of molecules. A CH_2 is assumed to interact with H_2O the same way in every molecule it appears in — the group-additivity hypothesis — so a *single universal* a_{mn} table ($\Psi_{mn} = e^{-a_{mn}/T}$) serves all of chemistry. That is what turns UNIQUAC's *fitted* residual term into a *prediction*: feed in the group counts and out comes $\gamma_i(T, \mathbf{x})$ for a binary nobody ever measured. The price is proximity effects and isomers the group count cannot see (see the limits).

Two-part structure. As in UNIQUAC, the activity coefficient splits into a size/shape (entropic) part and an energetic part:

$$\ln \gamma_i \;=\; \underbrace{\ln \gamma_i^{C}}_{\text{combinatorial}} \;+\; \underbrace{\ln \gamma_i^{R}}_{\text{residual}} . \tag{77}$$

Group constants. Each subgroup k carries a van der Waals volume R_k and surface area Q_k (tabulated, T-independent). Summing over the $\nu_k^{(i)}$ groups in molecule i gives its molecular volume and surface parameters

$$r_i = \sum_k \nu_k^{(i)} R_k, \qquad q_i = \sum_k \nu_k^{(i)} Q_k. \tag{78}$$

These are exactly the UNIQUAC r_i, q_i, which is why UNIFAC and UNIQUAC share the combinatorial term. In the simulator R_k, Q_k live in `data/standards/unifac/groups.dat` (Hansen *et al.* [39], open-literature original-UNIFAC values).

Combinatorial term (Staverman–Guggenheim). Purely geometric — it depends on r_i, q_i only, never on temperature:

$$\ln \gamma_i^{C} = \ln \frac{\phi_i}{x_i} + \frac{Z}{2}\, q_i \ln \frac{\theta_i}{\phi_i} + l_i - \frac{\phi_i}{x_i} \sum_j x_j l_j, \tag{79}$$

with the volume and surface-area fractions

$$\phi_i = \frac{r_i x_i}{\sum_j r_j x_j}, \qquad \theta_i = \frac{q_i x_i}{\sum_j q_j x_j}, \qquad l_i = \frac{Z}{2}(r_i - q_i) - (r_i - 1), \tag{80}$$

and lattice coordination number $Z = 10$. This term alone already captures the entropic non-ideality of mixing molecules of unequal size (it is what the Flory–Huggins correction does for polymers). In the code the quantity actually stored is l_i; ϕ_i/x_i and θ_i/ϕ_i are formed directly from the running sums $\sum_j r_j x_j$ and $\sum_j q_j x_j$.

Residual term: Γ_k on two scales. The energetic part is the difference between how each group "feels" in the mixture versus in the pure component:

$$\ln \gamma_i^{R} \;=\; \sum_k \nu_k^{(i)} \left[\ln \Gamma_k - \ln \Gamma_k^{(i)} \right], \tag{81}$$

where Γ_k is the residual activity coefficient of group k in the actual mixture, and $\Gamma_k^{(i)}$ is the *same* group's residual activity in a reference solution containing only the groups of pure component i. The subtraction is what enforces $\gamma_i \to 1$ as $x_i \to 1$: a group surrounded by its own molecule has $\Gamma_k = \Gamma_k^{(i)}$, so the bracket vanishes. Both scales use the identical functional form:

$$\ln \Gamma_k = Q_k \left[1 - \ln\Big(\sum_m \Theta_m \Psi_{mk} \Big) - \sum_m \frac{\Theta_m \Psi_{km}}{\sum_n \Theta_n \Psi_{nm}} \right], \tag{82}$$

with the group surface fraction and the group-interaction Boltzmann factor

$$\Theta_m = \frac{Q_m X_m}{\sum_n Q_n X_n}, \qquad \Psi_{mn} = \exp\!\left(-\frac{a_{mn}}{T}\right). \tag{83}$$

X_m is the mole fraction of group m over all groups in the relevant solution. The *main-group* interaction parameter a_{mn} [K] is asymmetric ($a_{mn} \neq a_{nm}$); groups in the same main group have $a_{mn} = 0$ (so $\Psi_{mn} = 1$). These are the only adjustable numbers in the whole model, and they are *universal*, not per-binary — `data/standards/unifac/interactions.dat` (Hansen *et al.* [39]). The implementation in `src/thermo/activityCoefficient/UNIFAC` uses this single-parameter a_{mn} (original UNIFAC); the later temperature-extended form $\Psi_{mn} = \exp[-(a_{mn} + b_{mn}T + c_{mn}T^2)/T]$ of modified UNIFAC (Dortmund) is *not* implemented.

Subgroups vs. main groups. A subtle but load-bearing point: Ψ_{mn} is indexed by *main group*, while R_k, Q_k and the group counting use the *subgroup*. Subgroups sharing a main group (e.g. `CH3`, `CH2`, `CH` all in main group `CH2`) have distinct R_k, Q_k but *share* every interaction parameter. That is how UNIFAC keeps the a-matrix small while still telling a methyl apart from a methylene by size.

The algorithm (glass-box). For each component i the code: (1) builds r_i, q_i, l_i from the declared groups (78); (2) evaluates the combinatorial term (79) from the mixture composition; (3) computes $\ln\Gamma_k$ once for the *mixture* group composition and once for each *pure*-component group composition $X^{(i)}$ via (82); (4) assembles the residual (81); (5) returns $\gamma_i = \exp(\ln\gamma_i^C + \ln\gamma_i^R)$. The pure-component $\Gamma_k^{(i)}$ are temperature dependent (through Ψ), so they are recomputed at every call rather than cached. The author declares each component's groups in the `activityModel` block: `model UNIFAC;` then a `groups{...}` sub-dict listing `group/count` per component. A component absent from `groups` falls back to $\gamma_i = 1$ and logs the gap (no silent crutch).

> **Worked example 2.4.** Decomposing acetone + *n*-hexaneAcetone = 1 `CH3CO` + 1 `CH3`; *n*-hexane = 2 `CH3` + 4 `CH2`. With the tabulated $R_{\mathrm{CH3}} = 0.9011$, $Q_{\mathrm{CH3}} = 0.848$ and $R_{\mathrm{CH2}} = 0.6744$, $Q_{\mathrm{CH2}} = 0.540$, hexane gets $r = 2(0.9011) + 4(0.6744) = 4.500$ and $q = 2(0.848) + 4(0.540) = 3.856$ from (78) alone — no fitting, no data. The non-zero a_{mn} between main groups `CH2CO` and `CH2` then drives the group Γ_k apart in the residual term and predicts the strong positive deviations (and the homogeneous azeotrope) of this notoriously non-ideal pair. Compare the prediction against the measured Txy through the validation weapon (Chapter 4) — that overlay, prediction vs. data, is the whole pedagogical point.

Power. A genuine prediction — γ_i for an *unmeasured* binary, even an azeotrope, from structure and a single universal table. Indispensable when no fitted NRTL/Wilson set exists, and the natural default when screening many candidate solvents at once.

Limit. (1) *Only as good as the group table.* A missing a_{mn} pair is silently treated as $a_{mn} = 0$ (ideal between those groups) — the header of `interactions.dat` flags this as a known honesty gap, and a declared group with no a-rows yields an unreliable γ. (2) *Group-additivity breaks for proximity effects* — neighbouring polar groups, intramolecular hydrogen bonds, and isomers the group count cannot distinguish (UNIFAC sees no difference between two positional isomers with the same groups). (3) *Weak near a mixture's critical region and for strongly dilute or asymmetric systems*, and the original-UNIFAC $\Psi = \exp(-a/T)$ gives only a crude temperature trend (the reason modified UNIFAC was developed). (4) *No liquid–liquid split* from this code path; UNIFAC-LLE uses a separate parameter table not bundled here. The standing lesson: a prediction to be *checked against data*, never trusted blind — if you *have* measured VLE for the pair, a fitted NRTL will beat UNIFAC, and UNIFAC's value is precisely when you do not.

3 Electrolyte solutions: ionic strength, the osmotic coefficient, and Pitzer

What you'll learn. A dissolved salt does not obey Raoult's law and does not have a meaningful vapour pressure, yet it governs the single most important number in reverse osmosis: the *osmotic pressure* π, the hydraulic head a membrane must overcome before a drop of water will cross. This chapter builds π from first principles — the equality of solvent chemical potential across the membrane — arrives at the ideal-dilute *van't Hoff* law $\pi = \nu R_g T c$ (the Raoult of electrolytes), then shows why it over-predicts at seawater concentration and how *Pitzer's osmotic coefficient* $\phi(I)$ corrects it. This is the electrolyte branch of the same "ideal → real" arc that took us from Raoult to NRTL.

3.1 Why an electrolyte needs its own model

The activity models of the previous chapter (γ_i from G^E) describe *neutral* molecules whose non-ideality is short-range: hydrogen bonds, size differences, weak dipoles. An ion is a different animal. When NaCl dissolves it dissociates,

$$\mathrm{NaCl} \longrightarrow \mathrm{Na^+} + \mathrm{Cl^-},$$

into $\nu = 2$ particles, and those particles carry charge. The dominant non-ideality is now the *long-range* Coulomb interaction between ions — an energy that decays as $1/r$, far slower than the $1/r^6$ of a van der Waals force, and which therefore matters even in a dilute solution. No G^E polynomial in mole fraction captures a $\sqrt{\text{concentration}}$ singularity at infinite dilution; the electrolyte needs its own functional form, and that form is what Debye–Hückel (1923) and later Pitzer (1973) [46] supply.

In Choupo this matters in exactly one place so far — the `spiralWoundModule` (NF/RO) — where π is the back-pressure that opposes the permeate water flux. A component flagged `nonvolatile true; dissociation` ν`;` in its `.dat` is a salt; it is invisible to the VLE machinery ($K_i = 0$) and visible to the osmotic model.

3.2 How Choupo carries the split: one salt, model ions, reference rung

The physics above is also the *data* architecture. The user writes one component, `NaCl`; the solver reasons in the two *model species* it dissociates into, $\mathrm{Na^+}$ and $\mathrm{Cl^-}$. Choupo records exactly that, without duplicating the salt: the component (`components/NaCl.dat`) carries the identity and the ion stoichiometry `dissociatesTo { Na 1; Cl 1; }` — a formula-like fact, like the molecular formula itself — while the *model species* (`species/aqueous/Na.dat`, `Cl.dat`) carry the charge and the aqueous reference data. We deliberately do not call the ions "true" species: an ion is not more real than the salt — complete dissociation is itself a model, the $K \to \infty$ limit of an association equilibrium, and the moment a real K appears (an ion pair, a hydrate) the split moves from the component's stoichiometry into `chemistry/` as an equilibrium.

The *reference state* matters as much as the model. A neutral activity model is anchored on pure-liquid Raoult ($\gamma_i \to 1$ as $x_i \to 1$); an ion has no pure liquid, so its datum is the *aqueous infinite-dilution* reference on the $\mathrm{H^+(aq)} = 0$ convention. Choupo does not hardcode "aqueous": each *property method* record declares its reference basis per phase — a Pitzer method declares the aqueous infinite-dilution rung for the liquid and the ion-derived datum for the salt, where an NRTL method declares pure-liquid Raoult. A case then selects a *property package* that names the method, the salt (whose `dissociatesTo` names the ions), and the Pitzer pairs; the Developer Guide (*the thermophysical property architecture*) gives the full picture. Here we keep to the physics: why $\phi(I)$, built next, is the electrolyte's correction to van't Hoff.

3.3 Osmotic pressure from the solvent chemical potential

Picture a membrane permeable to water but not to salt, with pure water on the left at pressure P and a salt solution on the right. At equilibrium the *solvent* chemical potential must be equal across the membrane (the salt cannot equilibrate — it cannot cross). The dissolved salt lowers the water's chemical potential through its activity $a_w < 1$; the only way to raise it back to the pure-water value is to raise the pressure on the solution side by an amount π:

$$\mu_w^*(T, P) \;=\; \mu_w^*(T, P + \pi) \;+\; R_g T \ln a_w, \tag{84}$$

where the star denotes pure water. Expand the pressurised pure-water term with the (nearly pressure-independent) molar volume of pure water $\bar{V}_w$,

$$\mu_w^*(T, P + \pi) \;=\; \mu_w^*(T, P) + \int_P^{P+\pi} \bar{V}_w \, dP' \;\approx\; \mu_w^*(T, P) + \bar{V}_w \, \pi,$$

substitute into (84), and cancel $\mu_w^*(T, P)$:

$$\boxed{\;\pi \;=\; -\frac{R_g T}{\bar{V}_w} \ln a_w .\;} \tag{85}$$

This is exact apart from treating $\bar{V}_w$ as pressure-independent — an excellent approximation for liquid water. Everything that follows is a model for the one unknown it contains, the solvent activity a_w.

3.4 The osmotic coefficient ϕ

For an electrolyte it is conventional to carry the deviation of $\ln a_w$ from ideality not in an activity coefficient but in the *osmotic coefficient* ϕ, defined by

$$\ln a_w \;\equiv\; -\,\phi\, M_w \sum_i m_i \;=\; -\,\phi\, M_w\, \nu m, \tag{86}$$

where m is the salt molality (mol per kg solvent), ν the number of ions it releases, and M_w the solvent molar mass (kg/mol). The sum $\sum_i m_i = \nu m$ runs over all ionic species. By construction $\phi \to 1$ as $m \to 0$: at infinite dilution the solution is ideal.

Substituting (86) into (85) and using $M_w/\bar{V}_w = \rho_w$ (the solvent density) together with the dilute-solution identity $m\,\rho_w \approx c$ (molality times solvent density $\approx$ molarity):

$$\boxed{\;\pi \;=\; \phi\, \nu\, R_{\mathrm{g}} T\, c.\;} \tag{87}$$

This is the form the code evaluates (`OsmoticModel::osmoticPressure` returns $\phi\,\nu\,RTc$, with R in J/(kmol K) and c in kmol/m^3). The whole physics of the electrolyte has been distilled into a single dimensionless number ϕ.

> *Intuition.* ϕ is to an electrolyte what γ is to a neutral mixture: a multiplicative "reality factor" on an ideal law. $\phi = 1$ is the ideal (van't Hoff) baseline; $\phi < 1$ means the real solution exerts *less* osmotic pressure than ν free particles would — the ions are not free, they are partly paired and screened, so they push on the membrane less hard than their count suggests.

3.5 The van't Hoff baseline: $\phi = 1$

Setting $\phi = 1$ in (87) gives van't Hoff's law,

$$\pi_{\mathrm{vH}} \;=\; \nu\, R_{\mathrm{g}} T\, c, \tag{88}$$

the colligative analogue of the ideal-gas law (van't Hoff, 1887, drew the analogy explicitly: solute particles in a solvent behave, for π, like gas molecules in a vacuum). It is the simulator's default osmotic model (`model vanHoff;`) and the right first answer: exact in the dilute limit, requiring nothing but the dissociation number ν. Its failure is quantitative, not qualitative. In the seawater regime ($m \lesssim 1$ mol/kg) it fails on the high side — $\phi < 1$, so van't Hoff over-predicts π, the cautious-but-wasteful side for a membrane designer (an over-estimated π under-estimates the achievable flux and recovery). The error is *not* one-signed for all concentrations, though: for NaCl ϕ bottoms out near $m \approx 0.7$, climbs back through $\phi = 1$ around $m \approx 2.4$, and *exceeds* 1 in concentrated brine — where van't Hoff *under*-predicts. A high-recovery design, a brine concentrator, or simply the polarised wall can reach that regime, so even the sign of the correction is concentration-dependent.

3.6 Ionic strength

The natural concentration variable for ion–ion electrostatics is not molality but the *ionic strength*

$$I \;=\; \tfrac{1}{2} \sum_i m_i\, z_i^2, \tag{89}$$

which weights each ion by the *square* of its charge z_i — the Coulomb energy scales as charge squared, so a divalent ion counts four times a monovalent one. For a 1:1 salt such as NaCl, $z_+ = +1$, $z_- = -1$, $m_+ = m_- = m$, so

$$I \;=\; \tfrac{1}{2}\,(m \cdot 1 + m \cdot 1) \;=\; m \qquad \text{(1:1 salt)}. \tag{90}$$

The single-electrolyte Pitzer model in Choupo assumes this 1:1 case (seawater RO); $I = m$ throughout.

3.7 Pitzer's osmotic coefficient for a 1:1 salt

Pitzer's ion-interaction model writes $\phi - 1$ as the sum of a *long-range* electrostatic term (a Debye–Hückel limiting law, softened to stay finite) and *short-range* ion-specific corrections. For a single 1:1 electrolyte it collapses to the compact form Choupo implements:

$$\boxed{\;\phi \;=\; 1 \;\underbrace{-\,A_\phi\, \frac{\sqrt{I}}{1 + b\sqrt{I}}}_{\text{Debye–Hückel}} \;+\, m\, B^{\phi} \;+\; m^2\, C^{\phi},\;} \tag{91}$$

with the second-virial coefficient itself a function of ionic strength,

$$B^{\phi} \;=\; \beta^{(0)} + \beta^{(1)}\, e^{-\alpha\sqrt{I}}. \tag{92}$$

Read term by term:

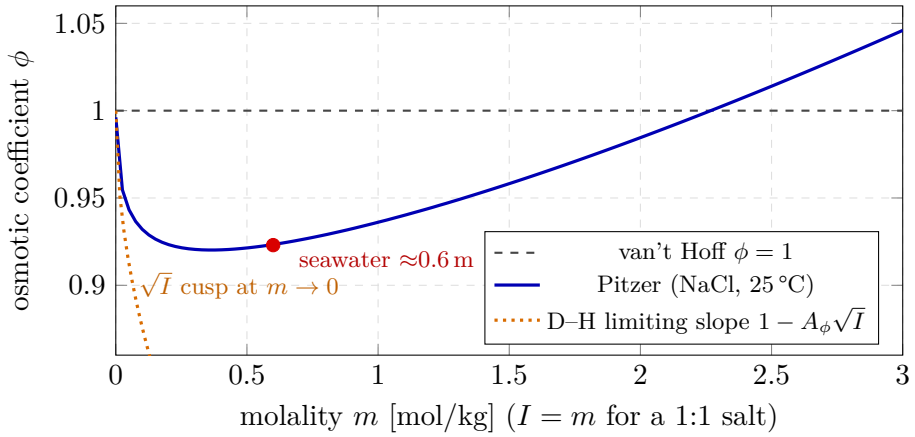

Figure 14: Pitzer's osmotic coefficient for NaCl, the electrolyte analogue of γ correcting an ideal law. The dashed grey line is the van't Hoff ideal baseline $\phi = 1$. The dotted orange curve is the Debye–Hückel limiting law $\phi \to 1 - A_\phi\sqrt{I}$: note its infinite slope (the $\sqrt{I}$ cusp) at infinite dilution—a singularity no mole-fraction polynomial can reproduce. The full Pitzer curve (blue) follows this electrostatic dip, then the salt-specific pair term pulls it back up, crossing $\phi = 1$ near $m \approx 2.4$. At seawater concentration (red point, $\phi \approx 0.92$) the real osmotic pressure is about 8 % below van't Hoff—directly the back-pressure an RO membrane need not overcome.

- $-A_\phi\sqrt{I}/(1 + b\sqrt{I})$ — the Debye–Hückel term. $A_\phi \approx 0.391\,\mathrm{kg}^{1/2}\,\mathrm{mol}^{-1/2}$ at 25 °C is the (universal, solvent-only) limiting-law slope; $b = 1.2$ is the universal closest-approach parameter. At low I this $\sim -A_\phi\sqrt{I}$ behaviour is the exact electrostatic limit — the famous square-root dependence that no mole-fraction polynomial can reproduce. It *lowers* ϕ: ion screening reduces the effective particle count.
- $m\,B^\phi$ — the short-range, salt-specific pair interaction. $\beta^{(0)}, \beta^{(1)}$ are tabulated per salt and *raise* ϕ back up at higher m. The exponential in (92) lets the pair term decay smoothly from its low-I value $\beta^{(0)} + \beta^{(1)}$ toward $\beta^{(0)}$; $\alpha = 2.0$ is the universal decay rate for 1:1 salts.
- $m^2\,C^\phi$ — a small triple-ion correction, important only in concentrated brine.

For NaCl at 25 °C the simulator ships the standard set

$$\beta^{(0)} = 0.0765, \quad \beta^{(1)} = 0.2664, \quad C^\phi = 0.00127, \quad A_\phi = 0.391, \quad b = 1.2, \quad \alpha = 2.0,$$

every one of which is a parameter the `osmotic` sub-block can override (and a future `FitParameters` run could calibrate to a measured $\phi(m)$ curve — exactly the spirit of Chapter 4).

Worked example 3.1. NaCl at seawater concentrationA 35 g/L NaCl feed ($M = 58.44\,\mathrm{g/mol}$) is $c \approx 0.60\,\mathrm{kmol/m^3}$, i.e. molality $m \approx 0.60\,\mathrm{mol/kg}$, so $I = 0.60$ and $\sqrt{I} = 0.775$.
Debye–Hückel term:

$$-\frac{0.391 \times 0.775}{1 + 1.2 \times 0.775} = -\frac{0.303}{1.930} = -0.157.$$

Pair term: $B^\phi = 0.0765 + 0.2664\,e^{-2.0\times0.775} = 0.0765 + 0.2664 \times 0.213 = 0.133$, so $mB^\phi = 0.60 \times 0.133 = 0.080$.
Triple term: $m^2C^\phi = 0.36 \times 0.00127 = 0.0005$.

$$\phi = 1 - 0.157 + 0.080 + 0.0005 \approx 0.923.$$

The osmotic pressure is therefore about **7.7 % lower** than van't Hoff predicts:

$$\pi_{\mathrm{vH}} = \nu R_g T c = 2 \times 8.314 \times 298 \times 599 \approx 29.7\ \mathrm{bar}, \qquad \pi_{\mathrm{Pitzer}} \approx 0.923 \times 29.7 \approx 27.4\ \mathrm{bar}.$$

Against a 55 bar applied pressure that ∼2.3 bar relief in the back-pressure is not cosmetic: it raises the net driving pressure $\Delta P - \Delta\pi$ and therefore the water flux and the recovery.

3.8 Why this matters for NF/RO — and where it is worst

The osmotic correction looks modest at the feed (8 % here), but the quantity that actually throttles the flux is the osmotic pressure *at the membrane wall*, not in the bulk. Concentration polarisation (Chapter 1 and the membrane tutorials) piles solute up against the wall, $c_m = c_b\,e^{J_w/k}$, so the wall concentration can be several times the feed. The correction $(1 - \phi)\,\nu R_g T c$ therefore reaches its largest magnitude — and, at brine concentrations, can even change sign — exactly at the wall, where the membrane is most starved; getting ϕ right matters most there. This is

the bread-and-butter regime of Vítor's NF/RO research, and the reason the Pitzer model is the first electrolyte refinement Choupo carries.

Key idea. The osmotic pressure is built in two independent layers, mirroring the whole simulator's "ideal baseline + correction" design. The *exact* thermodynamics is $\pi = -(R_{\mathrm{g}}T/\bar{V}_w)\ln a_w$; the *model* supplies $\ln a_w$ through one dimensionless number ϕ. van't Hoff sets $\phi = 1$ (ideal); Pitzer makes ϕ a function of ionic strength, capturing the $\sqrt{I}$ electrostatic limit that no polynomial activity model can. The two are interchangeable `OsmoticModel` sub-models — a one-word change in the case.

Runnable case. `tutorials/steady/membranes/membrane06_pitzer` — the seawater RO case of `membrane01` with `osmotic { model Pitzer; }`; $\phi \approx 0.92$ lifts recovery from 15.6 % to 17.2 %. `tutorials/steady/membranes/membrane01_RO_NaCl_seawater` is the van't Hoff baseline for comparison; `membrane03` sweeps k_{film} to expose the concentration-polarisation curve where the osmotic model matters most.

4 Electrolyte-NRTL: the activity coefficient of the *ion*

What you'll learn. Pitzer's $\phi(I)$ (§3) gives the deviation of the *solvent* — the osmotic pressure that throttles an RO membrane. But an evaporator, a crystalliser, or a drowning-out precipitator needs the deviation of the *salt*: the mean ionic activity coefficient $\gamma_\pm$, which fixes the boiling-point rise and the supersaturation ratio. This chapter derives the *electrolyte-NRTL* (eNRTL) model of Chen, Song and Evans [52, 51], the natural extension of the molecular NRTL of §2 to a solution of ions. Its premise is a clean *three-layer* split of the excess Gibbs energy — a long-range Pitzer–Debye–Hückel (PDH) term for the ion atmosphere, a short-range local-composition (NRTL) term for the molecular neighbourhood, and (for a mixed solvent) a Born term for transferring an ion between dielectrics — each one a separately readable contribution to $\ln\gamma_\pm$. We write the closed form Choupo evaluates for one fully-dissociated 1:1 salt, give the T-dependence, and validate it against Hamer & Wu NaCl data to AAD 1.8 %.

4.1 Why a second electrolyte model?

The Pitzer osmotic model of §3 answers exactly one question — *what is the water activity a_w?* — and answers it beautifully (NaCl ϕ to $< 0.2\,\%$). That is all an RO designer needs, because the flux is set by $\Delta\pi \propto -\ln a_w$. But move to a process that *concentrates* or *precipitates* the salt and the governing quantity flips to the salt's own activity:

- an **evaporator** raises the boiling point by $\propto -\ln a_w$, but the duty and the achievable concentration are bounded by where the salt *saturates*, i.e. by $\gamma_\pm$;
- a **crystalliser** nucleates when the activity product $(\gamma_\pm m)^\nu$ exceeds K_{sp} — the driving force is $\gamma_\pm$, not m;
- a **drowning-out** (antisolvent) precipitator works *because* adding ethanol drops the dielectric constant, which *raises* $\gamma_\pm$ and so the supersaturation — an effect with no counterpart in a single-solvent model.

eNRTL supplies $\gamma_\pm$ *and* a_w from one consistent excess-Gibbs surface, and — unlike Pitzer, whose virial coefficients are tied to molality in pure water — it lives on the mole-fraction scale, so it extends naturally to the mixed solvent. In Choupo it is the `activity { model eNRTL; }` sub-model of the same electrolyte slot that carries Pitzer (`src/thermo/electrolyte/ENRTLSingleSalt.H`).

4.2 The three-layer excess Gibbs energy

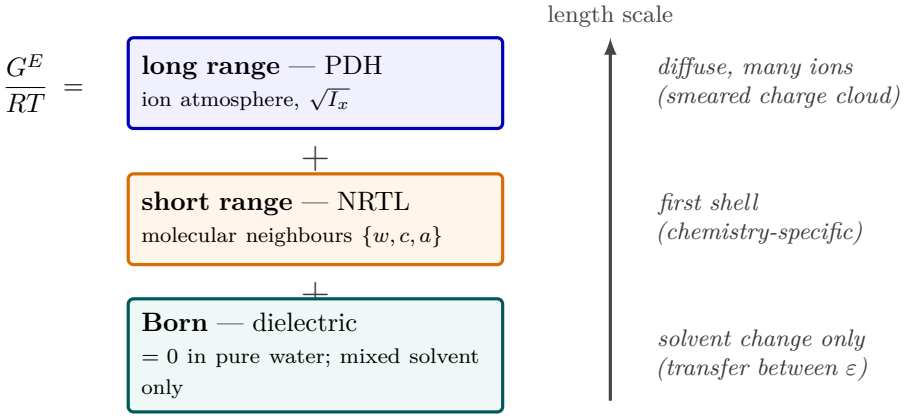

Figure 15: Electrolyte-NRTL splits the excess Gibbs energy into three separately-readable layers, each acting on a different length scale—the model's whole idea. The *long-range* Pitzer–Debye–Hückel term (blue) sees only the diffuse ion atmosphere and its $\sqrt{I_x}$ electrostatics; the *short-range* local-composition NRTL term (orange) sees an ion's immediate molecular neighbours $\{w, c, a\}$ and carries the chemistry-specific interactions; the Born term (teal) enters only when the solvent's dielectric constant changes, and is identically zero in pure water. Because they act on different scales they simply *add*, and $\ln\gamma_\pm$ for each species is the corresponding derivative—an evaporator and a crystalliser read the same surface that an RO membrane does.

eNRTL writes the molar excess Gibbs energy as a sum of physically distinct contributions,

$$\frac{G^E}{RT} = \underbrace{\frac{G^{E,\mathrm{PDH}}}{RT}}_{\text{long range}} + \underbrace{\frac{G^{E,\mathrm{lc}}}{RT}}_{\text{short range (NRTL)}} + \underbrace{\frac{G^{E,\mathrm{Born}}}{RT}}_{\text{dielectric transfer}}, \tag{93}$$

and the activity coefficient of every species is the corresponding derivative. The split is the model's whole idea: an ion feels the diffuse *ion atmosphere* (a smeared-out, charge-only effect that only the PDH term knows about) *and* its immediate *molecular neighbours* (a short-range, chemistry-specific effect that only the local-composition term knows about). The two act on different length scales, so they add. The Born term enters only when the solvent itself changes (the mixed-solvent case, §4.6); in pure water it is identically zero.

Reference state — unsymmetric for ions. A neutral component's $\gamma \to 1$ as $x \to 1$ (the pure liquid, §2). An ion has no pure liquid, so its reference is the *unsymmetric* (aqueous infinite-dilution) convention: $\gamma_i^* \to 1$ as the salt $\to 0$ in pure water. In the code this is enforced literally — every ionic local term is evaluated at the actual composition *minus* its value at $\mathbf{X}_{\text{ref}} = (x_w{=}1,\, x_c{=}0,\, x_a{=}0)$ — so the subtraction $lc(\mathbf{X}) - lc(\mathbf{X}_{\text{ref}})$ in `ENRTLSingleSalt.H` *is* the unsymmetric normalisation. The solvent (water) keeps the ordinary symmetric reference ($\gamma_w \to 1$ as $x_w \to 1$).

4.3 Long range: the Pitzer–Debye–Hückel term

For the long-range part eNRTL borrows Pitzer's mole-fraction-scale Debye–Hückel expression. Its contribution to the *unsymmetric* ionic $\ln\gamma$ is

$$\ln\gamma_i^{\text{PDH}} \;=\; -A_{\text{DH}}\left[\frac{2z_i^2}{\rho}\,\ln\!\big(1+\rho\sqrt{I_x}\big) \;+\; \frac{z_i^2\sqrt{I_x} - 2\,I_x^{3/2}}{1+\rho\sqrt{I_x}}\right], \tag{94}$$

where $I_x = \tfrac{1}{2}\sum_i x_i z_i^2$ is the *mole-fraction* ionic strength (note: x_i, not molality m_i — eNRTL lives on the mole-fraction scale), $\rho = 14.9$ is the closest-approach parameter (Pitzer's universal value), and A_{DH} is the mole-fraction-scale Debye–Hückel slope,

$$A_{\text{DH}} \;=\; \frac{1}{3}\sqrt{\frac{2\pi N_A}{\bar V_w}}\left(\frac{e^2}{4\pi\varepsilon_w\varepsilon_0 k_B T}\right)^{3/2}, \tag{95}$$

which evaluates to $A_{\text{DH}} = 2.9127$ for water at 25 °C ($\bar V_w = 1.807\times10^{-5}\,\text{m}^3/\text{mol}$, $\varepsilon_w = 78.45$). The solvent term gets the symmetric companion $\ln\gamma_w^{\text{PDH}} = 2A_{\text{DH}}\,I_x^{3/2}/(1+\rho\sqrt{I_x})$. This is the same $\sqrt{I}$ electrostatic limit as Pitzer's (91) — the part no polynomial activity model can reproduce — just written for $\ln\gamma$ instead of ϕ and on the mole-fraction basis.

4.4 Short range: the local-composition (NRTL) term

The short-range term is the NRTL local-composition idea of §2, now applied to a mixture of *three* species $\{w, c, a\}$ (water, cation, anion) with two rules borrowed from Chen's electrolyte theory:

- **like-ion repulsion**: no cation sits next to a cation (Coulomb forbids it), so a like-charged species is *excluded* from an ion's local cell;
- **local electroneutrality**: the immediate neighbourhood of a central molecule is charge-balanced.

With a single salt the energy landscape collapses to just *two* adjustable numbers — the molecule → electrolyte and electrolyte → molecule interaction energies $\tau_{m,ca}$ and $\tau_{ca,m}$ — plus the nonrandomness factor α. The Boltzmann factors are the familiar NRTL form $G_{ij} = \exp(-\alpha\,\tau_{ij})$, and all ion–ion same-pair terms vanish ($\tau_{ca} = 0$, $G_{ca} = 1$). The resulting ionic local-composition $\ln\gamma$ is the Chen–Song–Evans three-term expression (as implemented in `ENRTLSingleSalt::lnGammaX`, following IDAES Eqns. 26–27),

$$\ln\gamma_s^{\text{lc}} \;=\; z_s\left[\frac{X_w G_{sw}}{\sum_k X_k G_{kw}}\left(\tau_{sw} - \frac{\sum_k X_k G_{kw}\tau_{kw}}{\sum_k X_k G_{kw}}\right) + \frac{\sum_k' X_k G_{ks}\tau_{ks}}{\sum_k' X_k G_{ks}} + \frac{X_{s'} G_{ss'}}{\sum_k'' X_k G_{ks'}}\left(\tau_{ss'} - \frac{\sum_k'' X_k G_{ks'}\tau_{ks'}}{\sum_k'' X_k G_{ks'}}\right)\right], \tag{96}$$

where s' is the ion of *opposite* charge to s; the primed sums $\sum'$ exclude the species like-charged with s, and the double-primed sums $\sum''$ exclude the species like-charged with s' (the repulsion rule). Read it as the three neighbourhoods an ion sees: term 1 the solvent cell, term 2 its own charge cell, term 3 the counter-ion cell. The mean ionic value follows from the salt's stoichiometry,

$$\ln\gamma_\pm^{(x)} \;=\; \frac{\nu_c\big(\ln\gamma_c^{\text{PDH}} + \ln\gamma_c^{\text{lc}}\big) + \nu_a\big(\ln\gamma_a^{\text{PDH}} + \ln\gamma_a^{\text{lc}}\big)}{\nu_c + \nu_a}, \qquad \gamma_\pm = \gamma_\pm^{(x)}\Big/\big(1 + 10^{-3} M_w\,\nu\,m\big), \tag{97}$$

the divisor converting the mole-fraction-scale $\gamma_\pm^{(x)}$ to the molality-scale $\gamma_\pm$ that lab data report (M_w the solvent molar mass in g/mol, $\nu = \nu_c + \nu_a$).

4.5 Temperature dependence

Three places in the model carry T, and Choupo handles each by anchoring to the validated 25 °C kernel so that every existing golden master stays byte-identical at the reference and moves physically away from it:

1. **NRTL energies.** Like the molecular NRTL's $\tau = a + b/T$, the electrolyte τ is taken in the $\Delta g/RT$ form with a T-independent interaction energy,

$$\tau_{ij}(T) = \tau_{ij}^{\mathrm{ref}} \frac{T_{\mathrm{ref}}}{T}, \qquad T_{\mathrm{ref}} = 298.15\,\mathrm{K}, \tag{98}$$

   i.e. $a = 0$, $b = \tau^{\mathrm{ref}} T_{\mathrm{ref}}$ — the single published 25 °C value sets b and the form reduces to τ^{ref} at the reference.
2. **Debye–Hückel slope.** $A_{\mathrm{DH}}(T) = A_{\mathrm{DH}}(25)\, f(T)$ with the Born–Bjerrum factor

$$f(T) = \left(\frac{\rho_w(T)}{\rho_w(25)}\right)^{1/2} \left(\frac{\varepsilon_w(25)\, T_{25}}{\varepsilon_w(T)\, T}\right)^{3/2}, \tag{99}$$

   evaluated through the Malmberg–Maryott $\varepsilon_w(T)$ and Kell $\rho_w(T)$ correlations (`src/thermo/electrolyte/SolventProperties.H`); $f(25) = 1$.
3. **Born and PDH** are evaluated at the actual T, never a hard-coded 298.15.

> **Common pitfall.** The anchored $\tau(T) \propto 1/T$ is a *calorimetrically uncalibrated* guess: it reproduces $\gamma_\pm$ and ϕ at and near 25 °C, but its temperature *slope* (hence any heat of dilution derived from it) has not been refitted against dH data. This is the same caveat that gates the Pitzer L_ϕ enthalpy branch; the eNRTL package deliberately does *not* inherit the `calorimetricFit` flag, so it stays in the honest, uncalibrated regime. Use it for $\gamma_\pm$, a_w, boiling-point rise and solubility; do not read a heat of dilution off its T-slope without a refit.

4.6 The mixed solvent: drowning-out and the Born term

Add an antisolvent (ethanol) at salt-free mole fraction x_{anti} and two things happen, both of which eNRTL captures from (93):

1. **Dielectric mixing.** The mixture permittivity $\varepsilon_{\mathrm{mix}}$ (a mass-fraction average of the two pure values), molar volume $\bar{V}_{\mathrm{mix}}$ and molar mass M_{mix} replace the pure-water values in (95). Because $\varepsilon_{\mathrm{mix}} < \varepsilon_w$, the slope A_{DH} *grows* (electrostatics is less screened in a less polar medium) — the PDH term strengthens.
2. **Born transfer.** Moving an ion from pure water (ε_w) into the lower-ε mixture costs the Born self-energy difference,

$$\ln \gamma_i^{\mathrm{Born}} \;=\; \frac{e^2}{8\pi\varepsilon_0 k_B T} \left(\frac{1}{\varepsilon_{\mathrm{mix}}} - \frac{1}{\varepsilon_w}\right) \frac{1}{r_i}, \tag{100}$$

   with r_i the Born radius (calibrated to KCl in water+ethanol). It is positive — the ion is *less* comfortable in the mixture — so $\gamma_\pm$ rises.

Both effects push $\gamma_\pm$ up, which lifts the activity product $(\gamma_\pm m)^\nu$ above K_{sp} and *precipitates* the salt without removing any water — the thermodynamic engine of antisolvent crystallisation. In Choupo this is the `gammaPMMixed` entry point; $x_{\mathrm{anti}} \to 0$ recovers the aqueous $\gamma_\pm$ exactly, and $T = 298.15$ recovers the validated 25 °C kernel.

> **Worked example 4.1.** NaCl mean ionic activity coefficient at 25 °CTake $m = 1.0$ mol/kg NaCl ($\nu_c = \nu_a = 1$, $z_c = z_a = 1$, $M_w = 18.015$). The solvent amount is $n_w = 1000/18.015 = 55.51$ mol/kg, so $n_{\mathrm{tot}} = 55.51 + 2(1.0) = 57.51$ and the mole fractions are $X_w = 0.9652$, $X_c = X_a = 0.01739$, giving a mole-fraction ionic strength $I_x = \frac{1}{2}(0.01739 + 0.01739) = 0.01739$, $\sqrt{I_x} = 0.1319$.
> *PDH term* (Eq. 94, $z = 1$, $A_{\mathrm{DH}} = 2.9127$, $\rho = 14.9$):
>
> $$\ln \gamma_c^{\mathrm{PDH}} = -2.9127 \left[\frac{2}{14.9} \ln(1 + 14.9 \times 0.1319) + \frac{0.1319 - 2(0.01739)^{3/2}}{1 + 14.9 \times 0.1319}\right] \approx -0.252.$$
>
> *Local term* (Eq. 96, with $\tau_{m,ca} = 8.885$, $\tau_{ca,m} = -4.549$, $\alpha = 0.2$, evaluated unsymmetrically) adds a positive short-range contribution; summed and mole-fraction-to-molality converted via (97) the kernel returns
>
> $$\gamma_\pm \;\approx\; 0.648,$$
>
> against the Hamer & Wu tabulated value 0.657 — a 1.3 % error, typical of the model across the range.

4.7 Validation and the power-and-limit

The kernel was checked term-by-term against an independent Python prototype (`bin/curate/enrtl_ref/enrtl_validated.py`, agreeing to 5×10^{-5}) and against the classic NaCl reference data of Hamer & Wu [53] (and Robinson & Stokes) over 0.1–6 mol/kg at 25 °C:

m (mol/kg)	eNRTL $\gamma_\pm$	lit. $\gamma_\pm$	error
0.1	0.773	0.778	−0.6 %
0.5	0.670	0.681	−1.7 %
1.0	0.648	0.657	−1.3 %
2.0	0.674	0.668	+0.9 %
3.0	0.732	0.714	+2.5 %
6.0	0.950	0.986	−3.6 %
AAD $\gamma_\pm = 1.8$ %;		osmotic ϕ AAD = 1.4 %	

The eNRTL captures the entire shape: the dip near $m \approx 1$ (PDH screening dominates) and the strong rise past $m \approx 3$ (the local term takes over) — the same non-monotone curve van't Hoff cannot see.

Key idea. **Power.** One mole-fraction-scale excess-Gibbs surface delivers both $\gamma_\pm$ *and* a_w from two energy parameters per salt, extends without re-parameterisation to mixed solvents (the Born term is parameter-light), and reproduces the non-monotone $\gamma_\pm(m)$ to ∼2 % — the engine behind boiling-point rise, solubility, and antisolvent crystallisation.
Limit. Choupo's kernel is a *single fully-dissociated salt*; it does not by itself do multi-salt common-ion mixtures, ion pairing, or weak-electrolyte speciation (those live on the multi-ion Pitzer speciation path of §5). Its T-slope is calorimetrically uncalibrated (`calorimetricFit` stays off), and for high-concentration *aqueous* accuracy a per-salt-fitted Pitzer (∼0.2 %) beats it (∼1.8 %) — eNRTL's edge is the mixed solvent and the unified $\gamma_\pm/a_w$ surface, not single-salt aqueous precision.

Runnable case. `tutorials/steady/evaporation/evaporator07_nacl_enrtl` — a NaCl evaporator with `activity { model eNRTL; }`, where $\gamma_\pm$ sets the boiling-point rise and the saturation limit. `tutorials/steady/crystallisation/crystalliser06_nacl_antisolvent` exercises the mixed-solvent Born term (ethanol drowning-out). Contrast with the Pitzer membrane cases of §3 (`membrane06_pitzer`), which solve the *solvent* side of the same physics.

4.8 Multi-salt eNRTL: the symmetric multicomponent formulation

When two salts share the water — NaCl + KCl in a crystalliser purge, a mixed brine — each ion's environment contains *foreign* ions, and the single-salt expressions no longer apply. The multicomponent formulation (Song & Chen 2009, DOI `10.1021/ie9004578`) keeps the two eNRTL hypotheses (local electroneutrality, like-ion repulsion) and writes the local term species-wise, with every effective ion–molecule and ion–ion binary assembled from the *pair* parameters by charge-fraction mixing rules:

$$Y_c = \frac{X_c}{\sum_{c'} X_{c'}}, \qquad G_{am} = \sum_c Y_c\, G_{ca,m}, \qquad G_{ca} = \sum_{c'} Y_{c'}\, G_{ca,c'a}, \qquad \tau = -\ln G/\alpha, \tag{101}$$

so a chloride ion in a Na/K mixture sees a *composition-weighted* water–salt energy. The long-range term is the same PDH expression with the multicomponent ionic strength. Two structural results, both golden-locked in the tutorial:

1. **Exact single-salt reduction.** With one salt the charge fractions collapse to unity and the model IS the single-salt kernel of the previous sections (the tutorial pins the pure endpoints at $\sim 10^{-15}$ relative).
2. **Harned's rule emerges.** At constant total molality, $\ln\gamma_\pm$ of each salt is nearly *linear* in the salt fraction ($R^2 > 0.999$ at 4 mol/kg with zero electrolyte–electrolyte parameters) — the century-old empirical rule falls out of the local-composition structure.

Intuition. **A declared inconsistency — and its fix, side by side.** The published multicomponent activity-coefficient expressions differentiate the excess Gibbs energy *treating the mixing-rule binaries as constants*, yet those binaries depend on composition through the charge fractions Y. The result is not exactly Gibbs–Duhem consistent — the motivation for the *refined* eNRTL of Bollas, Chen & Barton (2008). Choupo ships **both formulations** (`formulation published;` vs `consistent;`): the consistent one keeps the chain rule through the Y-dependent mixing rules, so γ *is* the exact derivative of G^{ex} and the finite-difference oracle closes ($\sim 10^{-9}$) at *every* composition — while the published one prints its mid-sweep deviation (∼0.5) next to the pure-salt state where it, too, is exact. At 4 mol/kg NaCl–KCl the two differ by ∼4 % in mid-sweep $\gamma_\pm$. Robinson's 1961 isopiestic data judge them: the published form wins the dominant Harned coefficient (α_{NaCl} +14 % vs −29 %), both overstate the small α_{KCl}, and no single τ_{EE} closes both — the residual is structural. Bollas et al.'s own Figure 1 (this very system) cross-checks both branches: their original-eNRTL panel matches our published curves to the pixel, and their refined panel shows the same signature our consistent form shows — the τ fan nearly collapses, because an exact derivative is almost insensitive to τ_{EE}.

Runnable case. `tutorials/props/electrolyte/enrtl_multisalt_nacl_kcl` — the paper's own demonstration (NaCl–KCl–H_2O, 4 mol/kg Harned sweep), reproducing its Figure 1 $\tau_{ca,c'a}$ sensitivity fan (in $\log_{10}$ — the figure's axis label says ln but plots $\log_{10}$) and locking the reduction pin, the Harned slopes, and the

declared Gibbs–Duhem deviation.

5 Multi-ion Pitzer: the Harvie–Møller–Weare model

What you'll learn. The single-salt osmotic coefficient of the previous section answers "how much does *this one* dissolved salt lower the water activity?" Real brines, seawater, scaling waters and the inside of an evaporator carry *many* ions at once — Na^+, K^+, Ca^{2+}, Mg^{2+}, Cl^-, SO_4^{2-}, OH^-, the carbonates — and an ion's activity then depends on *every* ion present, not just its own counter-ion. This section builds the Harvie–Møller–Weare (HMW) assembly of Pitzer's model, which gives a *per-ion* activity coefficient γ_i from the same long-range Debye–Hückel term plus binary virials *plus* ternary mixing terms. We show exactly the sums Choupo evaluates (`PitzerHMW::evaluate`), the higher-order electrostatic term $E_\theta(I)$ that no other Pitzer code makes optional, and the self-test that proves the multi-ion model collapses to the single-salt kernel of §3.

5.1 From one salt to many ions

The compact form (91) is what you get when the general Pitzer expansion is specialised to a *single* fully dissociated salt. The general expansion does not work with salts at all — it works with *ions*. Its natural output is not a mean activity coefficient $\gamma_\pm$ but an individual γ_i for each ionic species i, on the molality scale, referenced to infinite dilution in pure water. Two reasons force this move:

- **A speciation solver needs γ_i, not $\gamma_\pm$.** When the same water carries dissolved CO_2 that speciates into HCO_3^- and CO_3^{2-}, or when you solve for pH, the chemical-equilibrium solver (`SpeciationSolver`) writes a mass-action law for each reaction and needs the activity of *each* ion individually. A lumped $\gamma_\pm$ cannot be unbundled.
- **Cross interactions are real.** In a mixed $Na^+/Mg^{2+}/Cl^-/SO_4^{2-}$ brine the activity of Na^+ is changed by the presence of Mg^{2+} (a like-sign neighbour) and of SO_4^{2-} (a second anion it never pairs with as a pure salt). These are the *mixing terms*; they are zero in any single-salt solution and are precisely what HMW adds.

Choupo implements the HMW assembly as the `pitzer AqueousActivity` model (`PitzerHMW`), selected by the same one word a single-salt case uses — `activityModel pitzer;` — but now reached through the speciation path, where it replaces the cheap Davies fallback with a full per-ion γ_i. All the arithmetic comes from the *same* `PitzerForm` header the single-salt kernel uses, so there is one Pitzer surface in the code, never two that can drift apart.

5.2 The per-ion working equations

For a cation M (charge z_M) and an anion X (charge z_X, here $z_X<0$), in a solution of cations $\{c\}$, anions $\{a\}$ and neutrals $\{n\}$ at molalities m_i, ionic strength $I=\tfrac12\sum_i z_i^2 m_i$, the HMW equations Choupo evaluates are (Pitzer 1991, ch. 3; [50])

$$\begin{aligned}\ln\gamma_M \;=\; & z_M^2\,F \;+\; \sum_a m_a\big(2B_{Ma}+Z\,C_{Ma}\big) \;+\; |z_M|\sum_c\sum_a m_c m_a\,C_{ca} \\ & +\sum_c m_c\Big(2\Phi_{Mc}+\sum_a m_a\,\psi_{Mca}\Big) \;+\; \sum_{a<a'} m_a m_{a'}\,\psi_{Maa'},\end{aligned} \tag{102}$$

$$\begin{aligned}\ln\gamma_X \;=\; & z_X^2\,F \;+\; \sum_c m_c\big(2B_{cX}+Z\,C_{cX}\big) \;+\; |z_X|\sum_c\sum_a m_c m_a\,C_{ca} \\ & +\sum_a m_a\Big(2\Phi_{Xa}+\sum_c m_c\,\psi_{Xac}\Big) \;+\; \sum_{c<c'} m_c m_{c'}\,\psi_{Xcc'},\end{aligned} \tag{103}$$

with the long-range function F collecting the Debye–Hückel slope and the ionic-strength *derivatives* of the binary and mixing terms,

$$F \;=\; \underbrace{-A_\phi\left[\frac{\sqrt{I}}{1+b\sqrt{I}}+\frac{2}{b}\ln\!\big(1+b\sqrt{I}\big)\right]}_{\text{Debye–Hückel}} \;+\; \sum_c\sum_a m_c m_a\,B'_{ca} \;+\; \sum_{c<c'} m_c m_{c'}\Phi'_{cc'} \;+\; \sum_{a<a'} m_a m_{a'}\Phi'_{aa'}, \tag{104}$$

and the charge-weighted total molality

$$Z \;=\; \sum_i |z_i|\,m_i. \tag{105}$$

Reading these against the code (`PitzerHMW::evaluate`), every symbol is one of a handful of `PitzerForm` calls:

- $B_{ca}=\beta^{(0)}_{ca}+\beta^{(1)}_{ca}\,g(\alpha_1\sqrt{I})+\beta^{(2)}_{ca}\,g(\alpha_2\sqrt{I})$, with $g(x)=2\big[1-(1+x)e^{-x}\big]/x^2$ — the binary pair term, per (cation, anion) read from the shared catalogue `data/standards/electrolyte/pairs.dat`. The $\beta^{(2)}$ slot is the 2:2 ion-pairing term ($\alpha_1=1.4$, $\alpha_2=12$); it is zero for 1:1/2:1/1:2 salts.

- $B'_{ca} = \left[\beta^{(1)}_{ca}\, g'(\alpha_1\sqrt{I}) + \beta^{(2)}_{ca}\, g'(\alpha_2\sqrt{I})\right]/I$, with $g'(x) = -2\left[1 - (1 + x + \tfrac{1}{2}x^2)e^{-x}\right]/x^2$ — the I-derivative that lives in F.
- $C_{ca} = C^{\phi}_{ca}/\left(2\sqrt{|z_c z_a|}\right)$ — the triplet (concentrated-brine) coefficient. The $|z_i|\sum_c\sum_a m_c m_a C_{ca}$ term is common to every ion (only $|z_i|$ differs); the code computes the double sum once and reuses it.
- $\Phi_{ij} = \theta_{ij} + E_{\theta\,ij}(I)$ and $\Phi'_{ij} = E'_{\theta\,ij}(I)$ — the *like-sign* mixing parameter (§5.3). θ_{ij} and the triplet ψ_{ijk} are read from `data/standards/electrolyte/mixing.dat`; an absent entry is 0.

Neutral solutes. A neutral species ($z = 0$, e.g. dissolved CO_2) is excluded from every charged sum and from I; it interacts only through the salting-out terms

$$\ln\gamma_n \;=\; 2\sum_{\text{ion}} m_{\text{ion}}\,\lambda_{n,\text{ion}} \;+\; \sum_c\sum_a m_c m_a\,\zeta_{n,c,a}, \tag{106}$$

with the symmetric back-reactions $+2m_n\lambda_{n,i}$ added to each ion's $\ln\gamma_i$ and $+m_n m_a\zeta_{n,c,a}$ to the cation/anion legs. A neutral with no tabulated λ/ζ gets $\gamma_n = 1$ exactly — there is no Debye–Hückel term for an uncharged species.

5.3 The higher-order electrostatic term $E_\theta(I)$

The like-sign mixing parameter Φ_{ij} is *not* simply the fitted constant θ_{ij}. Pitzer (1975) [47] showed that two ions of the same sign but *different charge* (e.g. Na^+ and Mg^{2+}, or Cl^- and SO_4^{2-}) feel an additional, purely electrostatic, ionic-strength-dependent interaction — the *unsymmetrical higher-order* term $E_\theta(I)$ — that exists even when $\theta_{ij} = 0$. It is built from a single integral $J(x)$:

$$E_{\theta\,ij}(I) \;=\; \frac{z_i z_j}{4I}\Big[\,J(x_{ij}) - \tfrac{1}{2}J(x_{ii}) - \tfrac{1}{2}J(x_{jj})\,\Big], \qquad x_{ij} = 6\,z_i z_j A_\phi\sqrt{I}, \tag{107}$$

$$E'_{\theta\,ij}(I) \;=\; -\frac{E_{\theta\,ij}}{I} + \frac{z_i z_j}{8I^2}\Big[\,x_{ij}J'(x_{ij}) - \tfrac{1}{2}x_{ii}J'(x_{ii}) - \tfrac{1}{2}x_{jj}J'(x_{jj})\,\Big]. \tag{108}$$

The single integral $J(x)$ Choupo evaluates is Pitzer's published closed-form approximation (the same one used by the USGS PHRQPITZ / Plummer et al. 1988),

$$J(x) \;=\; \frac{x}{4 + C_1\,x^{-C_2}\exp\!\left(C_3\,x^{C_4}\right)}, \qquad \begin{aligned} &C_1 = 4.581,\;\; C_2 = 0.7237,\\ &C_3 = 0.0120,\;\; C_4 = 0.528, \end{aligned} \tag{109}$$

with $J'(x)$ taken *analytically* (not by finite difference) from (109). Two limits make E_θ harmless where it should be: as $x \to 0$, $J \sim x^2$ so $E_\theta \to 0$ and the Debye–Hückel limiting law is untouched; as $x \to \infty$, $J \to x/4$. The crucial algebraic fact is that for two ions of the *same charge* ($z_i = z_j$) one has $x_{ij} = x_{ii} = x_{jj}$, so the bracket in (107) vanishes *identically*: $E_\theta = E'_\theta = 0$. Same-charge mixing (e.g. Na^+/K^+) therefore keeps the plain constant-θ form, and *every single salt* — which has one cation and one anion — has no like-sign pairs at all, so E_θ never fires.

> **Key idea.** $E_\theta(I)$ is the term most Pitzer tutorials wave away and most codes hard-wire on. Choupo computes it from first principles via the $J(x)$ kernel and *announces it*: at `verbosity` ≥ 3 the model prints which θ, ψ, λ, ζ binaries and triplets are active for the present ion set (`announceParameters`), so a student sees the *actual* parameter set in play — never a black box. The same kernel that carries E_θ is the difference between a correct Mg^{2+}-rich seawater γ and a 10–20 % error.

5.4 The multi-ion osmotic coefficient and the water activity

The per-ion γ_i answers "how non-ideal is each solute?" Its thermodynamic *sibling* answers the question that matters for the *solvent*: "how much do all these ions, together, lower the activity of the water?" That is the osmotic coefficient ϕ of the whole brine, and it is built from the *same* virial parameters as the γ_i — the Gibbs–Duhem relation forces it. Skipping it (the $\phi = 1$ dilute approximation) is the single most common way a Pitzer code quietly cheats: the ions get the full treatment, the water is left ideal. In a real brine the water is *not* ideal — $a_w < x_w$ — and that departure *is* the osmotic pressure $\Pi = -(R_g T/\bar{V}_w)\ln a_w$ that drives every membrane process, so a $\phi = 1$ water activity is exactly the error that wrecks an osmosis or evaporator calculation at high concentration.

Choupo therefore evaluates the full Harvie–Møller–Weare osmotic coefficient ([46], Pitzer 1991 ch. 3; HMW 1984), the osmotic-form companion of the γ_i sums above:

$$\begin{aligned}\Big(\phi - 1\Big)\sum_i m_i \;=\; 2\Bigg[&-\frac{A_\phi\, I^{3/2}}{1 + b\sqrt{I}} \;+\; \sum_c\sum_a m_c m_a\left(B^{\phi}_{ca} + Z\,C_{ca}\right)\\ &+\sum_{c<c'} m_c m_{c'}\left(\Phi^{\phi}_{cc'} + \textstyle\sum_a m_a\,\psi_{cc'a}\right) + \sum_{a<a'} m_a m_{a'}\left(\Phi^{\phi}_{aa'} + \textstyle\sum_c m_c\,\psi_{aa'c}\right)\\ &+\sum_n\sum_i m_n m_i\,\lambda_{ni} \;+\; \sum_n\sum_c\sum_a m_n m_c m_a\,\zeta_{nca}\Bigg],\end{aligned} \tag{110}$$

with $Z = \sum_i |z_i| m_i$, the osmotic second virial $B^{\phi}_{ca} = \beta^{(0)}_{ca} + \beta^{(1)}_{ca} e^{-\alpha_1\sqrt{I}} + \beta^{(2)}_{ca} e^{-\alpha_2\sqrt{I}}$ (note: the bare exponentials, not the $g(x)$ of the γ form), and $C_{ca} = C^{\phi}_{ca}/(2\sqrt{|z_c z_a|})$. Term by term this is the same parameter set the γ_i used: the long-range Debye–Hückel head $-A_\phi I^{3/2}/(1 + b\sqrt{I})$ is $I\, f^{\phi}(I)$, the binaries reuse $\beta^{(0,1,2)}, C^{\phi}$, and the ternaries reuse θ, ψ.

The one genuinely different object is the like-sign mixing term, which takes its *osmotic* form

$$\Phi^{\phi}_{ij} \;=\; \theta_{ij} \;+\; {}^{E}\theta_{ij}(I) \;+\; I\,{}^{E}\theta'_{ij}(I), \tag{111}$$

which carries the extra $I\,{}^{E}\theta'$ piece that the γ mixing term $\Phi_{ij} = \theta_{ij} + {}^{E}\theta_{ij}$ does *not* — both built from the *same* $J(x)$ kernel (109). For two same-charge ions ${}^{E}\theta = {}^{E}\theta' = 0$, so (111) collapses to the plain θ_{ij} and every single salt is untouched. The water activity then follows from the ϕ definition (87),

$$\ln a_w \;=\; -\,M_w\,\phi \sum_i m_i, \tag{112}$$

the same shape as the dilute form (124) but with the *real* ϕ in front instead of 1. In code this is `PitzerHMW::evaluate` filling `ActivityResult::osmotic` from (110), which `SpeciationSolver` reads for (112); Davies leaves it null and keeps $\phi = 1$, announced as such.

Key idea. The same virials, two faces. Equation (110) shares *every* parameter with the γ_i sums — as Gibbs–Duhem demands — so a brine's water activity is no longer a free guess but a *prediction*. Validated where the literature is sharpest: the NaCl osmotic coefficient from (110) tracks Pitzer–Peiper–Busey/Robinson–Stokes to better than 0.2 % from 0.1 to 6 mol/kg ($\phi = 0.936$ at 1 m, 1.045 at 3 m, 1.27 at 6 m), and standard seawater ($S = 35$, $I \approx 0.7$) returns $a_w = 0.981$ against the measured 0.980–0.981. Before this, Choupo (like many codes) used $\phi = 1$; that is now a *declared* fallback of the Davies model only, never a silent default under Pitzer.

5.5 The single-salt collapse — the skeptic's gate

The multi-ion sums *must* reduce to the closed single-salt kernel of §3 when only one salt is present, or there is a summation bug. This is not asserted by faith: `PitzerHMW::verify` loops over *every* binary in `pairs.dat`, builds the single-salt solution at $m \in \{0.1, 0.5, 1, 3\}$ mol/kg, forms the mean

$$\ln \gamma^{\mathrm{HMW}}_{\pm} = \frac{\nu_c \ln \gamma_M + \nu_a \ln \gamma_X}{\nu_c + \nu_a}, \tag{113}$$

and compares it to `PitzerSingleSalt::lnGammaPM`. The max relative deviation across all binaries is $\sim 1.6 \times 10^{-14}$ — floating-point reassociation only. The same routine checks the Debye–Hückel limiting law (as $I \to 0$, $\ln \gamma_M \to -3A_\phi z_M^2\sqrt{I} = -A_\gamma z_M^2\sqrt{I}$, ratio $\to 1$) and the $J(x)$ kernel against the tabulated anchors $J(0.1) \approx 0.00353$, $J(1) \approx 0.1158$, $J(10) \approx 2.040$ (the $\sim 10^{-3}$ deviation there is the published accuracy of the approximation (109) itself, not an error).

Worked example 5.1. Why E_θ is not optional in seawaterStandard seawater is roughly Na^+ 0.49, Mg^{2+} 0.055, Cl^- 0.57, SO_4^{2-} 0.029 mol/kg, giving $I \approx 0.72$ mol/kg. For the like-sign pair Cl^-/SO_4^{2-} ($z_i = -1$, $z_j = -2$), the arguments are $x_{ij} = 6 \cdot 2 \cdot 0.391\sqrt{0.72} = 3.98$, $x_{ii} = 6 \cdot 1 \cdot 0.391\sqrt{0.72} = 1.99$, $x_{jj} = 6 \cdot 4 \cdot 0.391\sqrt{0.72} = 7.96$. From (109), $J(3.98) \approx 0.62$, $J(1.99) \approx 0.27$, $J(7.96) \approx 1.55$, so

$$E_\theta = \frac{(-1)(-2)}{4 \times 0.72}\left[0.62 - \tfrac{1}{2}(0.27) - \tfrac{1}{2}(1.55)\right] = 0.694 \times (-0.29) \approx -0.20.$$

This -0.20 *adds* to whatever constant $\theta_{\mathrm{Cl,SO_4}}$ the catalogue carries (typically ~ 0.02): the electrostatic part *dominates* the fitted constant here. Dropping E_θ — the "constant-theta" shortcut — would mis-state the Cl^-/SO_4^{2-} interaction by an order of magnitude, and with it the γ of every ion that sees both anions.

Power. One header of pure $J(x)$ arithmetic turns the single-salt osmotic coefficient into a full multi-component aqueous activity model: any mix of Na, K, Ca, Mg, H cations and Cl, SO_4, OH (plus the carbonate subsystem) anions, with the higher-order electrostatics that make 2:1 and 2:2 brines correct, all driven from two flat catalogues (`pairs.dat`, `mixing.dat`) that cite a primary source per value and accept a per-case overlay. The same surface feeds the speciation solver, the osmotic pressure, and — through the Gibbs–Helmholtz identity $L_\phi = -RT^2\,\partial g^{ex}/\partial T$ — the heats of dilution.

Limit. The shipped parameters are the *25 °C* HMW base set; the full $\beta(T)$, $\theta(T)$ temperature surface is deferred (the dominant temperature dependence enters only through $A_\phi(T)$, scaled by the Born–Bjerrum factor, and through the single-salt $\partial\beta/\partial T$ calorimetric slots). The ion roster is the core seawater/brine system plus carbonates — borate,

H_4SiO_4, HSO_4^- and the higher neutral system are intentionally out. And $J(x)$ is Pitzer's *approximation* to the defining integral (good to $\sim 10^{-3}$), not the integral itself — accurate enough that the limiting law and the single-salt collapse hold to machine precision, but a Chebyshev-grade $J(x)$ would be the next refinement if sub-0.1% E_θ ever mattered.

Runnable case. The HMW model is the `pitzer AqueousActivity` reached through the speciation path (`SpeciationSolver`); a single-salt case still uses the closed `PitzerSingleSalt` kernel and the osmotic-pressure membrane tutorials (`membrane06_pitzer`). Run any speciation case at `verbosity 4` to see `announceParameters` list the active binaries, θ/ψ ternaries and E_θ pairs.

6 Aqueous speciation: mass action, charge balance, and the pH closure

What you'll learn. A water analysis hands you a short list of *total* concentrations — so much calcium, so much carbonate, so much chloride — but the quantities that actually matter (the CO_3^{2-} that scales a membrane, the free H^+ that sets the pH, the $CaSO_4$(aq) ion pair that hides calcium from a saturation index) are *distributed* among dozens of dissolved species by chemical equilibrium. This section builds the *speciation* calculation that does the distributing: a small set of mass-action equilibria written against a handful of *master* ions, closed either by a measured pH or, when none is reliable, by *electroneutrality*. We solve it the way the code does — one Newton iteration in $\ln m$ inside a frozen-γ fixed point — and we meet the PHREEQC *percent-error* convention that tells you, before you trust a single number, whether the lab sheet even balances its charges. This is the engine behind the `speciate` and `scalingScan` property operations and behind every membrane-scaling and softener audit downstream.

6.1 Master ions and the mass-action network

The previous two chapters gave the *activity* of an ion that is already free in solution. Speciation answers the prior question: given the totals, how much of each ion *is* free? When a carbonate water sits at equilibrium, the dissolved inorganic carbon is split between H_2CO_3, HCO_3^-, CO_3^{2-} and pairs like $CaHCO_3^+$; calcium is split between free Ca^{2+}, $CaHCO_3^+$, $CaCO_3$(aq), $CaSO_4$(aq), and so on. Each non-master *species* s is the product of a *formation reaction* written against a fixed, minimal basis of *master* ions (Ca, Mg, Na, K, Cl, SO_4, HCO_3, …) plus H^+ and water. Its concentration is fixed by a mass-action law,

$$m_s\,\gamma_s \;=\; K_s(T)\,\prod_j a_j^{\,\nu_{sj}}, \qquad a_j = \gamma_j\, m_j, \tag{114}$$

where m_j are the *free* master molalities (mol per kg water), ν_{sj} the stoichiometric coefficient of master j in the formation of s, $K_s(T)$ the formation constant, and γ the single-ion activity coefficients of the previous chapter. Water enters with its own activity a_w raised to $\nu_{s,\mathrm{H_2O}}$ (so the autoprotolysis $H_2O = H^+ + OH^-$ writes OH^- with $a_{\mathrm{OH}} = K_w\, a_w / a_{\mathrm{H}}$). The whole network — the species list, their stoichiometries, $\log_{10} K_s$ at 25 °C and an optional temperature law — is read from a `speciation.dat` catalogue derived from the public-domain USGS PHREEQC databases [42].

Intuition. Speciation is not a new kind of physics; it is "more $\partial/\partial n$ of the same Gibbs surface." Equation (114) is just $\Delta G_{\mathrm{rxn}} = 0$ for each formation reaction — the same condition that gives an equilibrium constant in any reactor. What is new is the *bookkeeping*: many simultaneous equilibria sharing the same free ions, closed by conservation of mass and charge.

6.2 The conservation equations: mole balances

There are far more species than masters, so (114) alone cannot close the system. The closure is conservation. For each master ion j the analysis specifies a *total* molality T_j (everything containing that element, free or bound), and the model must reproduce it:

$$T_j \;=\; m_j \;+\; \sum_s \nu_{sj}\, m_s \qquad (j = 1, \dots, n). \tag{115}$$

Substituting the mass-action expression (114) for each m_s turns (115) into n equations in the n free master molalities — everything else is algebra. This is the core problem the solver faces. What remains is to decide what to do with H^+, the master that a water analysis almost never reports as a total.

6.3 Two closures for H^+: pH given vs pH solved

pH given. If a trustworthy pH is on the sheet, set

$$a_{\mathrm{H}} \;=\; 10^{-\mathrm{pH}}, \tag{116}$$

treat H^+ as *known*, and remove its mole balance from the system: H^+ (and the OH^- that chains off it) are simply algebraic functions of the fixed a_{H}. The unknowns stay the n master molalities. This is the dict form `pH 7.8;`.

pH solved (electroneutrality). Often the reported pH is stale, drifted in the sample bottle, or simply missing. Then H^+ becomes an *extra unknown* and an extra equation must replace its missing total. That equation is *electroneutrality*: a real solution carries no net charge,

$$\boxed{\;\sum_i z_i\, m_i \;=\; 0\;} \tag{117}$$

summed over *all* species — masters, H^+, OH^-, and every complex. The pH is then an *output*: the free H^+ adjusts until the cations and anions balance. This is the dict form `pH solve;`.

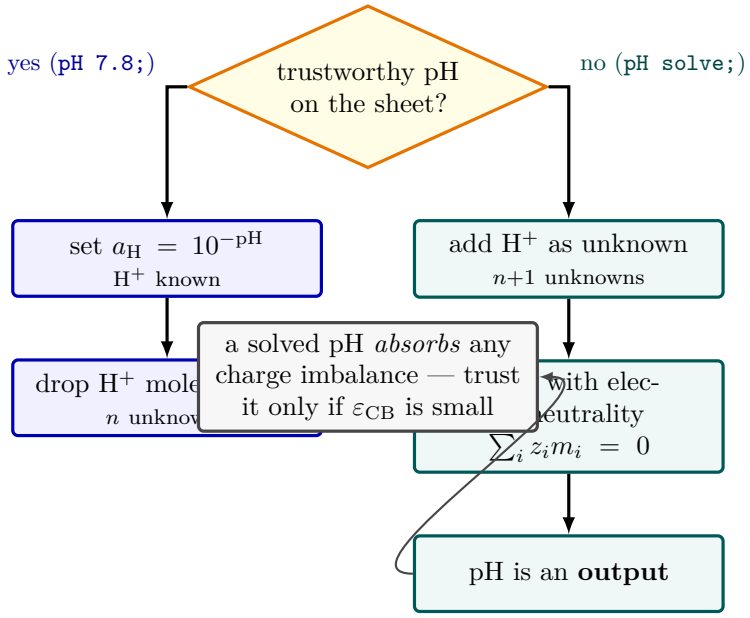

Figure 16: The two ways the speciation solver closes the proton balance. If a trustworthy pH is on the analysis (left, blue), H^+ is *known*: fix $a_{\mathrm{H}} = 10^{-\mathrm{pH}}$ and drop its mole balance, leaving n unknowns. If the pH is stale or missing (right, teal), H^+ becomes an extra unknown and *electroneutrality* $\sum_i z_i m_i = 0$ supplies the missing equation—the pH then falls out as an output, exactly as a titration determines it physically. The caveat (grey): a solved pH silently absorbs any charge-balance error in the input, so it is only as good as $\varepsilon_{\mathrm{CB}}$.

> *Intuition.* Charge balance closing the pH is exactly what a titration does physically. When the cation and anion totals on a sheet do not add up to zero charge, something is wrong — a missing analyte, a unit slip, an alkalinity mis-reported — and the *solved* pH silently *absorbs* that error: it moves H^+ (and hence OH^-, HCO_3^-, CO_3^{2-}) to make the books balance. A solved pH is only as trustworthy as the charge balance of the analysis behind it. That is why the next subsection insists on measuring the imbalance *before* believing the answer.

6.4 The PHREEQC percent-error convention

A defensible speciation report must say how far the input was from charge balance. Following PHREEQC [42], Choupo reports the *charge-balance percent error*

$$\varepsilon_{\mathrm{CB}} \;=\; 100 \cdot \frac{\left| S_+ - S_- \right|}{\tfrac{1}{2}\left(S_+ + S_-\right)} \;=\; 200 \cdot \frac{\left| S_+ - S_- \right|}{S_+ + S_-}, \tag{118}$$

where $S_+ = \sum_{z_j>0} |z_j|\, T_j$ and $S_- = \sum_{z_j<0} |z_j|\, T_j$ are the cation and anion *equivalent* sums over the *master totals* ($\mathrm{H}^+/\mathrm{OH}^-$ are not part of a water analysis, so they are excluded). The factor 200 (not 100) normalises by the *average* of the two sums, so a $+5\,\%$ cation excess and a $-5\,\%$ anion deficit each register as the same $\approx 5\,\%$. Choupo prints $\varepsilon_{\mathrm{CB}}$ *before* it solves, and when it exceeds $\approx 5\,\%$ raises a loud advisory: the analysis is internally inconsistent and the solved pH is not trustworthy because it has absorbed the lab error. This is the simulator's no-silent-crutch rule applied to data quality — the user is told what the number is carrying.

6.5 The solution algorithm: Newton in log-molality

The mole balances (115) are stiff: molalities span many orders of magnitude (Na^+ at 10^{-2}, CO_3^{2-} at 10^{-5}, H^+ at 10^{-8}) and must stay strictly positive. Both problems vanish under the substitution

$$x_j \;\equiv\; \ln m_j \qquad\Longrightarrow\qquad m_j = e^{x_j} > 0 \;\text{ automatically,} \tag{119}$$

and the mass-action law (114) becomes *linear* in the log unknowns:

$$\ln m_s \;=\; \ln 10\, \log_{10} K_s - \ln \gamma_s + \nu_{s,\mathrm{H_2O}} \ln a_w + \sum_j \nu_{sj}\big(\ln \gamma_j + x_j\big). \tag{120}$$

The residual vector is the set of mole balances (plus, when H^+ is solved, the electroneutrality row (117) with x_{H} as its unknown). Because each m_s depends on x_j through (120), the Jacobian is *analytic*: a mole-balance row

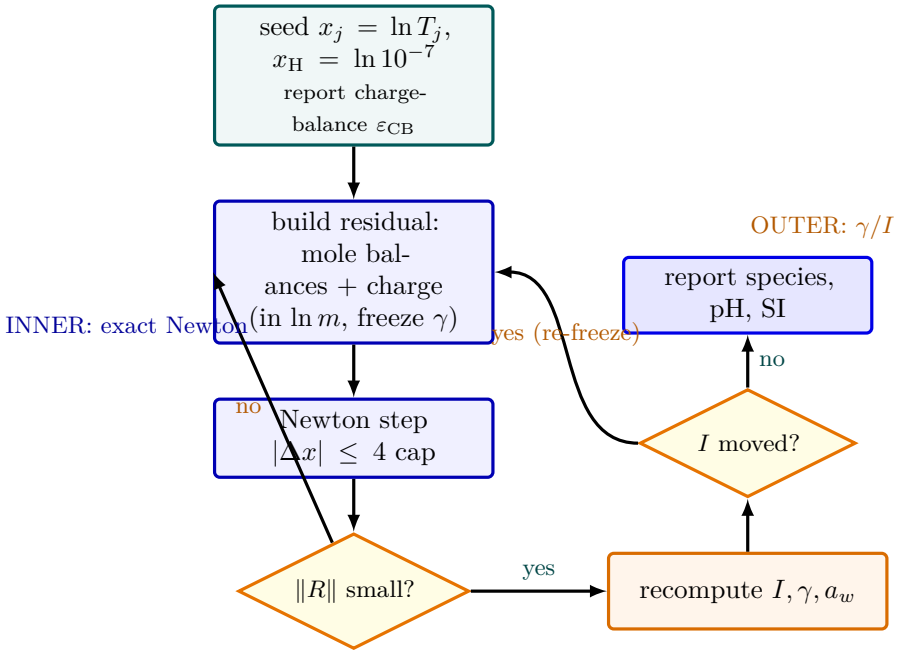

Figure 17: The speciation solver as a two-loop machine—the same freeze-the-slow-coupling pattern as a recycle tear. The *inner* loop (blue) is an exact Newton on the mole balances (plus electroneutrality when the pH is solved), written in $\ln m$ so molalities stay positive and the mass-action laws turn linear; each step is capped at $|\Delta x| \le 4$ so the iteration cannot leap out of its basin. The *outer* loop (orange) freezes the activity coefficients, solves the inner system exactly, then re-evaluates γ and a_w at the new ionic strength, repeating until I stops moving. Before any of it runs, the charge-balance error ε_{CB} is reported (teal)—a solved pH is no better than the analysis it balances.

has $\partial R_j/\partial x_k = -\delta_{jk}\, m_j - \sum_s \nu_{sj}\nu_{sk}\, m_s$, and the dense $n \times n$ (or $(n{+}1) \times (n{+}1)$) system is solved by Gaussian elimination (Chapter 3). Newton steps in $\ln m$ are capped at $|\Delta x| \le 4$ (m may change by at most $e^4 \approx 55\times$ per step) so the iteration cannot leap out of the basin.

The activity coefficients are the wrinkle: γ_j depends on the ionic strength $I = \frac{1}{2}\sum_i z_i^2 m_i$, which depends on the very molalities being solved. Choupo breaks this with a *frozen-γ fixed point*: hold all γ (and the water activity a_w) constant, run Newton to convergence on the molalities, recompute I, γ and a_w, and repeat the outer pass until I stops moving. The two-loop structure — an outer γ/I self-consistency wrapping an inner exact Newton — is the same pattern used for the recycle tear (Chapter 6): freeze the slow coupling, solve the fast system exactly, update, repeat.

Key idea. Speciation is solved as a **two-loop** problem. The *inner* loop is an exact Newton on mole balances (and electroneutrality, if pH is solved) written in $\ln m$ so molalities stay positive and the mass-action laws become linear. The *outer* loop freezes the activity coefficients and water activity, then re-evaluates them at the new ionic strength — iterating $I \to \gamma \to m \to I$ to self-consistency. The default γ model is Davies (the teaching rung below Pitzer); Pitzer HMW is selectable for high ionic strength.

6.6 The Davies equation: a parameter-free single-ion γ_i

The speciation network of the previous subsections needs, for every charged species, a *single-ion* activity coefficient γ_i — not the mean $\gamma_\pm$ that the eNRTL (§4) and the osmotic Pitzer (§3) deliver. The cheapest rung that still gets the electrostatics right is the *Davies equation* [26], the model Choupo's `davies` kernel (`AqueousActivity`) implements and the speciation solver freezes once per outer iteration. This subsection derives it, states exactly what the code evaluates, and is honest about where it runs out.

From the limiting law to Davies. The Debye–Hückel theory (1923) treats each ion as a point charge surrounded by a diffuse *ionic atmosphere* of opposite net charge. Solving the linearised Poisson–Boltzmann equation for the work of charging that atmosphere gives the celebrated *limiting law*, valid only as $I \to 0$,

$$\log_{10}\gamma_i \;=\; -A\, z_i^2 \sqrt{I}, \tag{121}$$

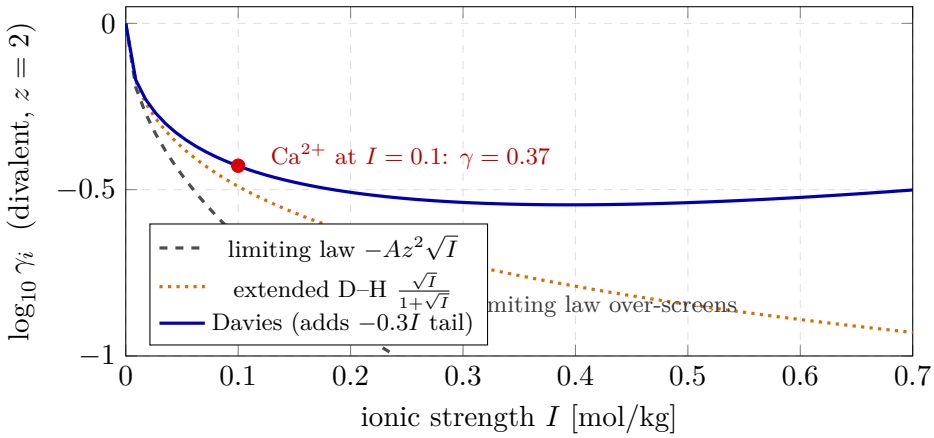

Figure 18: The three rungs of single-ion non-ideality for a divalent ion, showing why Davies is the right teaching default. The bare Debye–Hückel limiting law (dashed grey) over-screens at finite I, driving γ unphysically low. The extended law's closest-approach denominator $1+\sqrt{I}$ (dotted orange) tames it. Davies (blue) adds one universal, salt-*in*dependent empirical tail $-0.3I$ that turns the decline back upward—mimicking the short-range repulsion a per-salt $\beta^{(0)}$ would otherwise carry, with *no* fitted parameter. At $I=0.1$ the worked Ca^{2+} value (red, $\gamma=0.37$) sits 40 % above the limiting-law answer—exactly the gap that decides whether $CaCO_3$ reads as supersaturated at a membrane wall.

where the square-root in ionic strength $I=\frac{1}{2}\sum_i m_i z_i^2$ is the electrostatic signature no mole-fraction polynomial can reproduce. The slope A is a property of the *solvent only* (no salt-specific input). The limiting law over-screens at finite concentration; the *extended* Debye–Hückel law tames it with a closest-approach denominator $1+\mathring{a}B\sqrt{I}$, and the **Davies** fix takes the universal choice $\mathring{a}B=1$ and adds a single empirical linear tail $-0.3\,I$ that turns the otherwise monotone decline back upward, mimicking the short-range repulsion that a per-salt $\beta^{(0)}$ would otherwise carry:

$$\boxed{\log_{10}\gamma_i \;=\; -A\,z_i^2\left(\frac{\sqrt{I}}{1+\sqrt{I}} \;-\; 0.3\,I\right).} \tag{122}$$

The remarkable thing about (122) is that it carries *no adjustable, salt-specific parameter at all* — only A (the solvent) and the ion charge z_i. That is its entire pedagogical and practical appeal: a defensible γ_i for *any* ion in water from charge alone.

The slope A and its temperature scaling. A is the $\log_{10}$-base Debye–Hückel slope; Choupo anchors it at the textbook

$$A \;=\; 0.51\ \mathrm{kg}^{1/2}\,\mathrm{mol}^{-1/2} \quad (25\,\text{řC}),$$

which is exactly $3A_\phi/\ln 10$ with the Pitzer $A_\phi=0.3915$, so the Davies and Pitzer rungs share one consistent Debye–Hückel constant. Off 25 °C, A inherits its temperature dependence from the *solvent* through

$$A(T) \;=\; 0.51\times f_{\mathrm{DH}}(T), \qquad f_{\mathrm{DH}}(T)=\left(\frac{\rho_w(T)}{\rho_w^{25}}\right)^{1/2}\left(\frac{\varepsilon_w^{25}\,T_{25}}{\varepsilon_w(T)\,T}\right)^{3/2}, \tag{123}$$

the same dimensionless factor (`SolventProperties::debyeHuckelFactor`) that scales the Pitzer A_ϕ and the eNRTL A_{DH} — so the three electrolyte kernels move together with temperature, never independently. The factor is built from the static permittivity $\varepsilon_w(T)$ (Malmberg–Maryott [27]) and the density $\rho_w(T)$ (Kell [28]), each written as a *ratio* anchored so that $f_{\mathrm{DH}}(298.15\,\mathrm{K})\equiv 1$: at 25 °C the value is byte-identically $A=0.51$, and only off 25 °C does the physics of a colder, denser, more polar solvent (a *larger* A, hence stronger non-ideality) appear. The $T^{-3/2}\varepsilon^{-3/2}$ form is the textbook Debye–Hückel scaling of the charging integral.

What the code actually evaluates. The `davies` kernel is deliberately tiny and glass-box (`AqueousActivity::evaluate`):

- it computes $A=0.51\,f_{\mathrm{DH}}(T)$ once, then returns a closure $\gamma(z)=10^{-Az^2(\sqrt{I}/(1+\sqrt{I})-0.3I)}$ evaluated with `std::pow(10.0,...)` (not $e^{x\ln 10}$, to keep the golden masters byte-stable);
- **neutral species** ($z=0$) and the $I\le 0$ guard return $\gamma=1$ exactly — there is *no* Setchenow salting-out term ($\log_{10}\gamma_{\mathrm{neutral}}=b\,I$) in this rung, stated honestly rather than faked;

- it is a pure function of $(|z|, I, A)$, so the solver freezes the whole γ table at the iteration's I (the Newton in log-molality stays linear in the activities — see the solution algorithm above).

Worked example 6.1. Davies vs. the limiting law for Ca^{2+}Take a brackish water at $I = 0.10\,\text{mol/kg}$, 25 °C, so $A = 0.51$ and $\sqrt{I} = 0.316$. For a divalent ion ($z = 2$, $z^2 = 4$):

$$\frac{\sqrt{I}}{1+\sqrt{I}} - 0.3I = \frac{0.316}{1.316} - 0.030 = 0.240 - 0.030 = 0.210,$$

$$\log_{10}\gamma_{\text{Ca}} = -0.51 \times 4 \times 0.210 = -0.428, \qquad \gamma_{\text{Ca}} = 10^{-0.428} = 0.373.$$

The bare limiting law (121) would give $\log_{10}\gamma = -0.51 \times 4 \times 0.316 = -0.645$, i.e. $\gamma = 0.226$ — a *40 % lower* activity coefficient. The denominator and the $-0.3I$ tail together pull the divalent ion back from the unphysically strong screening the limiting law predicts; this is precisely the difference that decides whether $CaCO_3$ shows up as supersaturated ((125)) at the membrane wall. A monovalent ion at the same I has only a quarter of the exponent, $\gamma_{\text{Na}} = 10^{-0.107} = 0.78$ — the z^2 weighting in action.

Power and limit. *Power:* a single, parameter-free equation gives every ion a defensible γ_i from its charge alone, with the correct $\sqrt{I}$ electrostatic limit and a sensible turn-up at moderate I — the ideal teaching rung between "ideal solution" ($\gamma_i = 1$) and a fully fitted Pitzer. *Limit:* the $-0.3I$ tail is empirical, so Davies is trustworthy only to $I \approx 0.5\,\text{mol/kg}$; the speciation solver therefore raises an explicit advisory once I exceeds $\approx 0.7\,\text{mol/kg}$ and points the student at the multi-ion Pitzer HMW kernel (§5, [46]), which carries the salt-specific virial coefficients Davies lacks. Neutral species are ideal ($\gamma = 1$), and the calorimetric (T-derivative) accuracy of the slope is only as good as the solvent correlations (123). The model is honest about all three.

6.7 Activity model and water activity

The default single-ion γ is the Davies equation (122) of §6.6, with $A \approx 0.51$ at 25 °C and the $\varepsilon_w(T), \rho_w(T)$ scaling (123) shared with the Pitzer kernel; it is trustworthy to $I \approx 0.5\,\text{mol/kg}$, the solver raises an advisory above $I \approx 0.7$, and Pitzer HMW [46] is the documented replacement. Neutral species take $\gamma = 1$. The water activity is

$$\ln a_w \;=\; -\,M_w\,\phi \sum_i m_i, \tag{124}$$

with M_w the molar mass of water in kg/mol and ϕ the osmotic coefficient — the definition (86). Under **Davies** the solver keeps the dilute approximation $\phi = 1$ (declared in the run header), consistent with Davies being indicative beyond $I \approx 0.5\,\text{mol/kg}$. Under **Pitzer** ϕ is the *real* multi-ion osmotic coefficient of (110), built from the same virials as the γ_i, so the brine's water activity — the gypsum a_w^2 leg, the boiling-point elevation, the osmotic pressure — is Pitzer-consistent and not a $\phi = 1$ guess (§5.4).

6.8 Temperature dependence of K

Formation constants are anchored at 25 °C and corrected to the run temperature by a three-tier priority, *all* pinned so that $\log_{10} K(298.15\,\text{K}) \equiv \log_{10} K_{25}$ exactly (the 25 °C run is byte-stable by construction):

1. **Analytic** (PHREEQC `-analytical_expression`): the fitted shape $\text{ana}(T) = A_1 + A_2T + A_3/T + A_4\log_{10}T + A_5/T^2 + A_6T^2$ is applied as an *anchored* correction, $\log_{10}K(T) = \log_{10}K_{25} + [\text{ana}(T) - \text{ana}(298.15)]$, so the published fit's absolute offset cancels and only its T-shape is borrowed; outside the fitted range the run announces "$K(T)$ extrapolated" (no silent extrapolation).
2. **van't Hoff** (constant ΔH_r): $\log_{10}K(T) = \log_{10}K_{25} + \dfrac{\Delta H_\text{r}}{R_\text{g}\ln 10}\left(\dfrac{1}{298.15} - \dfrac{1}{T}\right)$.
3. **Flat**: a genuinely bare entry holds K at its 25 °C value.

Each form is listed at verbosity ≥ 2, so the student sees exactly which constants were temperature-corrected and how.

6.9 From speciation to saturation: the scaling link

Once the free-ion activities are known, the *saturation index* of any mineral in `minerals.dat` follows immediately:

$$\text{SI} \;=\; \log_{10}\frac{\text{IAP}}{K_{sp}(T)}, \qquad \text{IAP} = \prod_{\text{ions}} a_{\text{ion}}^{\nu}\; a_w^{\nu_{\text{H}_2\text{O}}}, \tag{125}$$

SI < 0 undersaturated, $= 0$ at equilibrium, > 0 supersaturated (scaling risk). The `scalingScan` op walks this index along an RO concentration trajectory; the `equilibrate` option goes further and finds the precipitated amounts by an *active-set* complementarity solve (admit the most-supersaturated phase, drive its SI to zero, evict any phase whose precipitated amount goes negative) layered on top of exactly the Newton above. Saturation indices are the bridge from the chemistry of this section to the membrane and crystalliser chapters.

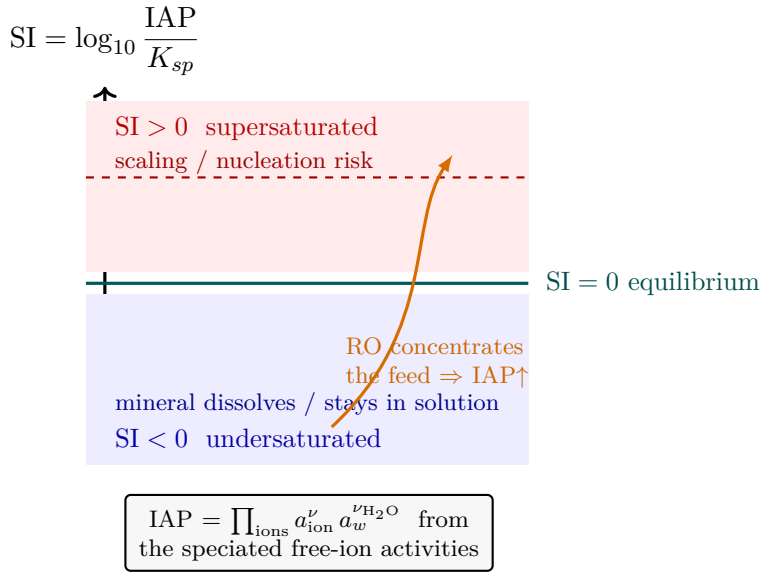

Figure 19: The saturation index turns speciated free-ion activities into a single scaling verdict. Once the solver knows every free-ion activity, the ion-activity product IAP is compared to the solubility product $K_{sp}(T)$: $\mathrm{SI} = \log_{10}(\mathrm{IAP}/K_{sp})$. Below the teal equilibrium line the mineral is undersaturated (blue, it dissolves or stays dissolved); above it the water is supersaturated (red, scaling and nucleation risk). As a reverse-osmosis stage concentrates the feed the activities climb and the operating point walks *up* the ladder (orange)—the `scalingScan` op traces exactly this trajectory, and the `equilibrate` option drops the precipitate once a mineral crosses $\mathrm{SI} = 0$.

Power and limit. The power: from seven total concentrations and a temperature the model recovers the entire distribution of dozens of species, a defensible pH (closed by physics, not by a stale measurement), and the saturation state of every candidate scale mineral — the same calculation a regulatory water-chemistry report rests on, with every $\log K$, every γ, and every Newton iteration visible. The limits are deliberate and announced: the Davies default is trustworthy only to $I \approx 0.5\,\mathrm{mol/kg}$ (use Pitzer beyond); the water activity uses the $\phi = 1$ dilute approximation; neutral species are ideal ($\gamma = 1$); a solved pH is no better than the charge balance of its input, which is precisely why $\varepsilon_{\mathrm{CB}}$ is reported and gated before the answer is offered.

Worked example 6.2. A brackish RO feed, pH solvedA brackish feed lists (mg/L) Ca 84, Mg 27, Na 363, K 12, Cl 510, SO_4 250, HCO_3 183 at 25 °C, with no reliable pH. Run with `pH solve;`. The solver first reports the charge-balance error $\varepsilon_{\mathrm{CB}} = 200\,|S_+ - S_-|/(S_+ + S_-)$ over the master equivalents; here it is a couple of percent (a clean analysis), well under the 5 % advisory. It then seeds $x_j = \ln T_j$, $x_{\mathrm{H}} = \ln 10^{-7}$, freezes Davies γ at the initial I, and Newton-solves the seven mole balances plus electroneutrality in $\ln m$; recomputing I and γ and re-solving a few times lands $I \approx 0.03\,\mathrm{mol/kg}$ and a solved pH ≈ 7.72 — close to the lab's reported 7.8, the charge balance vindicating the analysis. The carbonate now distributes into HCO_3^-, CO_3^{2-} and $CaHCO_3^+$, and the calcite SI from (125) reads the scaling risk that drives the downstream RO audit (`tutorials/props/electrolyte/scaling_ro_brackish`).

Runnable case. `tutorials/props/electrolyte/scaling_ro_brackish` — the worked feed above, with analytic "pins" (pure-water K_w, gypsum-saturated $CaSO_4$, neutral pure water) that nail the solver against hand-checkable answers, then the feed speciated with `pH solve;` and a closed-vs-open-CO_2 scaling scan. `tutorials/props/electrolyte/softener_brackish` adds Gaines–Thomas cation exchange on the same Newton.

7 Fugacity and the vapour phase

What you'll learn. The activity coefficient γ_i of the previous chapter corrects the liquid-side chemical potential for non-ideality. On the vapour side, the analogous correction is the *fugacity coefficient* φ_i. Together (γ_i, φ_i) are the two multiplicative gauges that take the ideal-mixture reference of Chapter 1 all the way to real-fluid phase equilibrium. fixes $\varphi_i = 1$ (ideal-gas vapour), which is adequate at the operating pressures of the shipped tutorials; the cubic equations of state that produce $\varphi_i \neq 1$ at higher pressures are on the v1.x roadmap.

7.1 Why we need fugacity at all

The chemical potential of a real gas is not $\mu = \mu^0(T) + RT\ln(P/P_0)$. That expression is the ideal-gas case where the molecules do not interact. Real molecules feel attractive and repulsive forces; their chemical potential depends on more than just pressure in a logarithmic-of-ratio way.

G. N. Lewis (1901) defined the *fugacity* f as the auxiliary variable that restores the simple logarithmic form for any fluid:

$$\boxed{\mu(T,P) \;=\; \mu^0(T) \;+\; RT\ln\frac{f(T,P)}{f^0}.} \tag{126}$$

Reference $f^0 = 1$ bar by convention. The defining condition is that $f \to P$ as $P \to 0$: in the dilute-gas limit, the fugacity becomes the ordinary pressure. The *fugacity coefficient* is the ratio

$$\varphi(T,P) \;\equiv\; \frac{f(T,P)}{P}, \tag{127}$$

with $\varphi \to 1$ as $P \to 0$ and $\varphi = 1$ identically for an ideal gas at any P. Deviations of φ from unity measure how far the real gas departs from the ideal-gas equation of state.

Intuition. Fugacity is the engineering trick that lets every formula derived under the ideal-gas assumption survive in a real-gas world: replace P with f everywhere and the algebra continues to apply. All of the complexity of non-ideal molecular interactions is bundled into the single multiplicative correction φ, which is then computed from an equation of state. $\varphi < 1$ for fluids dominated by attraction (compression denser than ideal); $\varphi > 1$ for fluids dominated by repulsion (e.g. helium near T_c).

7.2 Fugacity of a pure liquid

For a pure substance at saturation $(T, P^{\mathrm{sat}}(T))$, liquid and vapour coexist and their chemical potentials are equal, so their fugacities are equal too:

$$f^{\mathrm{L}}(T,P^{\mathrm{sat}}) \;=\; f^{\mathrm{V}}(T,P^{\mathrm{sat}}) \;=\; \varphi^{\mathrm{sat}}(T)\,P^{\mathrm{sat}}(T). \tag{128}$$

For a pure liquid at a different P, the small pressure dependence is the *Poynting correction* $\exp\!\big[v^{\mathrm{L}}(P-P^{\mathrm{sat}})/(RT)\big]$, which is ~ 1 until P exceeds several bar above P^{sat}. At operating conditions the simulator drops the Poynting factor entirely:

$$f_i^{\mathrm{L},*}(T) \;\approx\; \varphi_i^{\mathrm{sat}}(T)\,P_i^{\mathrm{sat}}(T). \tag{129}$$

With the ideal-gas $\varphi_i^{\mathrm{sat}} = 1$ assumption, this collapses further to $f_i^{\mathrm{L},*} \approx P_i^{\mathrm{sat}}$.

7.3 Partial fugacity in mixtures

For component i in a mixture, the partial fugacity $\hat{f}_i$ plays the role of an "effective partial pressure". In a gas mixture

$$\hat{f}_i^{\mathrm{V}}(T,P,\mathbf{y}) \;=\; y_i\,\hat{\varphi}_i(T,P,\mathbf{y})\,P, \tag{130}$$

where $\hat{\varphi}_i$ is the partial fugacity coefficient of i in the mixture (note the hat: it depends on composition, unlike the pure-component φ_i). In an ideal-gas mixture $\hat{\varphi}_i = 1$ for every i, recovering Dalton's law $\hat{f}_i = y_i P$ that we already derived in Chapter 1.

In a liquid mixture the partial fugacity is built from the activity coefficient and the pure-component liquid fugacity:

$$\hat{f}_i^{\mathrm{L}}(T,P,\mathbf{x}) \;=\; x_i\,\gamma_i(T,\mathbf{x})\,f_i^{\mathrm{L},*}(T) \;\approx\; x_i\,\gamma_i\,\varphi_i^{\mathrm{sat}}\,P_i^{\mathrm{sat}}. \tag{131}$$

The two corrections separate cleanly: γ_i absorbs *composition-driven* non-ideality (next-neighbour interactions in the liquid); φ_i^{sat} absorbs the *vapour-phase* non-ideality at the saturation pressure of the pure component. In both are taken at the simplest end (γ_i from NRTL/Wilson but $\varphi_i^{\mathrm{sat}} = 1$).

7.4 Phase-equilibrium criterion in fugacity form

Chapter 2 laid out the criterion: $\mu_i^{\mathrm{L}} = \mu_i^{\mathrm{V}}$ for every component across the phase boundary. Substituting (126) into both sides and cancelling μ^0 and f^0,

$$\boxed{\hat{f}_i^{\mathrm{L}}(T,P,\mathbf{x}) \;=\; \hat{f}_i^{\mathrm{V}}(T,P,\mathbf{y}) \quad \text{for every component } i.} \tag{132}$$

Substituting (130) and (131) produces the γ-φ **formulation**:

$$x_i\,\gamma_i(\mathbf{x},T)\,\varphi_i^{\mathrm{sat}}(T)\,P_i^{\mathrm{sat}}(T) \;=\; y_i\,\hat{\varphi}_i(\mathbf{y},T,P)\,P. \tag{133}$$

This is the equation the manual's current Part I Chapter 2 presents out of context. Now you can read it: liquid-side fugacity equals vapour-side fugacity, both built up from the ideal-mixture baseline (x_i or y_i times a reference) times two real-fluid corrections (the γ's and the φ's).

Setting $\hat{\varphi}_i = \varphi_i^{\mathrm{sat}} = 1$ (the ideal-gas-vapour assumption) reduces (133) to

$$\boxed{x_i\,\gamma_i(\mathbf{x},T)\,P_i^{\mathrm{sat}}(T) \;=\; y_i\,P.} \tag{134}$$

Often called *modified Raoult's law*. It is the closed-form phase-equilibrium relation every flash, bubble-T, dew-T and distillation stage ultimately uses. Setting $\gamma_i = 1$ on top gives plain Raoult.

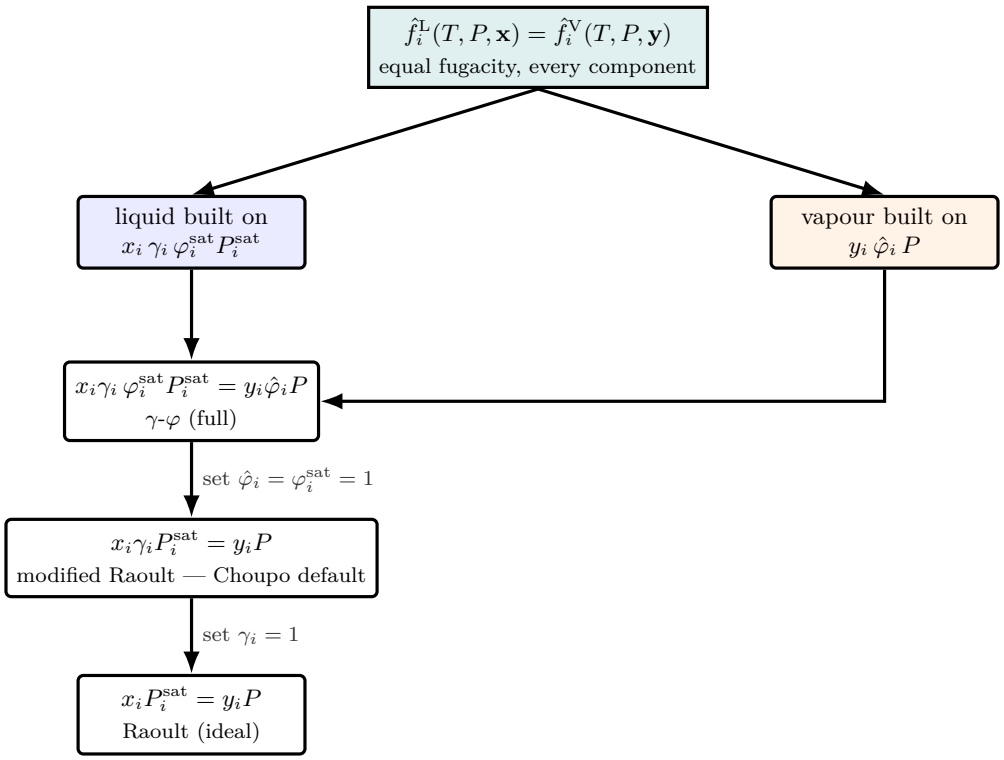

Figure 20: The fugacity route from the exact equilibrium criterion down to the closed form every flash actually solves. Both phases start from the same statement — liquid fugacity equals vapour fugacity — each built from an ideal-mixture baseline (x_i or y_i) times real-fluid corrections (the γ's on the liquid, the φ's on the vapour). Dropping the two vapour corrections ($\hat{\varphi}_i = \varphi_i^{\mathrm{sat}} = 1$) collapses the γ-φ formula to *modified Raoult's law* — the relation Choupo's `ThermoPackage::Kvec` evaluates — and dropping γ_i too gives plain Raoult. Each downward step is an assumption made visible, not a hidden default.

7.5 When $\varphi_i = 1$ fails

Three regimes where the ideal-gas simplification breaks down, and the simulator quietly mispredicts:

- **High pressure** ($P \gtrsim 10$ bar for most species, $\gtrsim 1$ bar for water near T_c). Attractive interactions condense the gas below ideal density; $\varphi < 1$ by 5–20%, pushing the predicted bubble pressure of a flash low by a similar margin.

- **Dimerising vapours** (acetic acid, formic acid, nitrogen tetroxide). The vapour contains an equilibrium fraction of dimers, halving the apparent number of moles. No EoS short of an explicit dimerisation model gets this right. $\omega = 0.47$ for acetic acid is the indirect symptom; the apparent φ for the monomer can sit around 0.5–0.7.
- **Supercritical and near-critical**. The very concept of a vapour breaks down, φ varies dramatically with small changes in T and P, and cubic EoS take over from the simpler approximations.

7.6 Where φ_i comes from when not 1

When the ideal-gas assumption breaks down, Choupo replaces the `idealGas` block with a cubic equation of state for the vapour:

$$P \;=\; \frac{RT}{v-b} \;-\; \frac{a(T)}{v^2 + u\,b\,v + w\,b^2},$$

parametrised by T_c, P_c and ω for each species (the data files already carry these; see Chapter 1). Soave-Redlich-Kwong (SRK): $u = 1, w = 0$. Peng-Robinson (PR): $u = 2, w = -1$. Both produce $\hat{\varphi}_i$ in closed form from the mixture parameters $a_{\mathrm{mix}}, b_{\mathrm{mix}}$ via the standard derivation $\ln\hat{\varphi}_i = (\partial(n\,\bar{A}^{\mathrm{R}}/RT)/\partial n_i)_{T,V,n_{j\neq i}}$, where $\bar{A}^{\mathrm{R}}$ is the residual Helmholtz energy. The next chapter (Chapter 8) derives the SRK form Choupo implements end to end — the parameters, the cubic in Z, the fugacity coefficient, and the enthalpy/entropy departure functions — and contrasts it with Peng-Robinson.

The point of carrying T_c, P_c and ω in every data file: those three numbers *become* the cubic EoS the moment a case selects `model SRK;` (or `PR;`). The thermodynamic stack is forward-compatible by design.

7.7 The γ-φ implementation

The simulator's `ThermoPackage::Kvec` computes

$$K_i(\mathbf{x}, T, P) \;=\; \frac{\gamma_i(\mathbf{x}, T)\,P_i^{\mathrm{sat}}(T)}{P} \tag{135}$$

exactly as Eq. (134), with γ_i from NRTL/Wilson (or 1 for the ideal-solution model). No φ_i multiplications appear in the source — they are all collapsed to unity at compile time by the `equationOfState { model idealGas; }` block.

7.8 Henry's law: the dilute-solute K-value

Modified Raoult's law, Eq. (134), anchors the liquid-phase fugacity of component i on the *pure liquid* i: the reference is $x_i \to 1$, where $\gamma_i \to 1$ and the liquid escapes at its own saturation pressure $P_i^{\mathrm{sat}}(T)$. That reference is useless for a gas that is only sparingly soluble. A bubble of oxygen, CO_2 or NH_3 never *is* a liquid at the temperature of the water it dissolves in — $P_{\mathrm{O_2}}^{\mathrm{sat}}(298\ \mathrm{K})$ does not even exist below the critical point — so the pure-liquid anchor is a fiction. The physically natural reference for a solute is the *opposite* limit, $x_i \to 0$.

The constant. In a sufficiently dilute solution the partial pressure of the solute is linear in its liquid mole fraction. This is **Henry's law** (Henry, *Phil. Trans. R. Soc.* 1803):

$$\boxed{\;p_i \;=\; H_i(T)\,x_i, \qquad x_i \to 0.\;} \tag{136}$$

H_i is the **Henry's-law constant** (here in pressure per mole-fraction units, Pa — Choupo's H_{xp} convention). It is the slope of the p_i–x_i curve at the origin, exactly as P_i^{sat} is the slope at $x_i = 1$ under Raoult. The two pictures are the two tangents of the same real $p_i(x_i)$ curve, taken at its two ends; for a species that obeys $\gamma_i^{\infty} \neq 1$ they relate through the infinite-dilution activity coefficient,

$$H_i(T) \;=\; \gamma_i^{\infty}(T)\,P_i^{\mathrm{sat}}(T), \tag{137}$$

which is why a large positive deviation ($\gamma_i^{\infty} \gg 1$, the solute "dislikes" the solvent) gives a large H_i and a low solubility. Eq. (137) is also the bridge a curator uses to cross-check a tabulated H_i against an activity model.

The K-value. Combining $p_i = y_i P$ (Dalton, ideal vapour) with Eq. (136) gives the equilibrium ratio in the same $K_i = y_i/x_i$ form every other model produces:

$$\boxed{\;K_i \;=\; \frac{y_i}{x_i} \;=\; \frac{H_i(T)}{P}.\;} \tag{138}$$

Set side by side with the γ-φ result $K_i = \gamma_i\,P_i^{\mathrm{sat}}/(\varphi_i P)$, the only change is the *reference fugacity*: P_i^{sat} (pure-liquid anchor) is replaced by H_i (infinite-dilution anchor), and γ_i drops out because it has been folded into H_i at $x_i \to 0$ via Eq. (137). Both leave $K_i \propto 1/P$ — a solute is always squeezed into the liquid by raising the pressure.

Temperature dependence: van't Hoff. Dissolution is a phase change with a molar enthalpy of dissolution ΔH_{diss} (the heat released when one mole of solute leaves the gas and enters the dilute liquid). Applying the van't Hoff relation to the dissolution equilibrium constant $K_{\mathrm{sol}} = 1/H$,

$$\frac{\mathrm{d}\ln K_{\mathrm{sol}}}{\mathrm{d}T} = \frac{\Delta H_{\mathrm{diss}}}{RT^2} \quad\Longrightarrow\quad \frac{\mathrm{d}\ln H}{\mathrm{d}T} = -\frac{\Delta H_{\mathrm{diss}}}{RT^2},$$

and integrating from a reference temperature T_{ref} with ΔH_{diss} taken constant over the (narrow) validity window gives the form Choupo implements:

$$\boxed{H(T) \;=\; H_{\mathrm{ref}}\,\exp\!\left[\frac{\Delta H_{\mathrm{diss}}}{R}\left(\frac{1}{T}-\frac{1}{T_{\mathrm{ref}}}\right)\right].} \tag{139}$$

The sign deserves care, and the source comment in `HenrysLaw::H` flags a historical bug here. For an *exothermic* dissolution $\Delta H_{\mathrm{diss}} < 0$ (the common case — CO_2, NH_3, H_2S, O_2 all release heat on dissolving). For $T > T_{\mathrm{ref}}$ both factors in the exponent are negative, so H *rises*: the gas becomes *less* soluble when the liquid is warmed. This is the everyday fact that warm soda goes flat and that ammonia boils out of warm water — and it is why a *stripper* runs hot and an *absorber* runs cold (Chapter 14).

Worked example: CO_2 in water. The standard catalogue ships `henrysLaw/CO2-water.dat` with $H_{\mathrm{ref}} = 1.64\times10^{8}$ Pa (= 1640 bar) at $T_{\mathrm{ref}} = 298.15$ K and $\Delta H_{\mathrm{diss}} = -19\,400$ J mol^{-1} (Sander, 2015). At 25 °C and $P = 1$ atm $= 1.013\times10^{5}$ Pa,

$$K_{\mathrm{CO_2}} = \frac{H_{\mathrm{ref}}}{P} = \frac{1.64\times10^{8}}{1.013\times10^{5}} \approx 1.6\times10^{3} \quad (\gg 1:\ \text{strongly favours the vapour}).$$

Equivalently, the liquid mole fraction in equilibrium with a pure CO_2 atmosphere is $x = p/H = 1.013\times10^{5}/1.64\times10^{8} \approx 6\times10^{-4}$ — about 0.034 mol L^{-1}, the familiar carbonation figure. Warming to 40 °C (313.15 K),

$$H(313.15) = 1.64\times10^{8}\,\exp\!\left[\tfrac{-19400}{8.314}\left(\tfrac{1}{313.15}-\tfrac{1}{298.15}\right)\right] \approx 2.39\times10^{8}\ \text{Pa},$$

a 46% rise in H over a 15 K warming — the soda goes flat.

In the simulator. `ThermoPackage::Kvec` branches to the Henry path for a component whose data file declares `role solute;` *and* for which a `(solute, solvent)` pair file exists for the case's named solvent; it then equates the FULL unsymmetric-convention fugacities (Prausnitz, *Molecular Thermodynamics*, Ch. 10),

$$y_i\,\varphi_i^{V}\,P \;=\; x_i\,\gamma_i^{*}\,H_i(T)\,\exp\!\left[\frac{\bar v_i^{\infty}\,(P-P_{\mathrm{solv}}^{\mathrm{sat}})}{RT}\right] \quad\Longrightarrow\quad K_i \;=\; \frac{\gamma_i^{*}\,H_i(T)\,\mathcal{P}}{\varphi_i^{V}\,P}, \tag{140}$$

with three physically distinct corrections on top of the textbook H_i/P (each optional in the pair file, each defaulting to the dilute low-pressure limit):

- φ_i^{V} — the *vapour* fugacity coefficient from the package EoS: the gas phase is real at 200 bar.
- $\mathcal{P} = \exp[\bar v_i^{\infty}(P - P^{\mathrm{sat}})/RT]$ — the **Poynting** term of **Krichevsky–Kasarnovsky** (1935): compressing the *liquid* raises the dissolved gas's fugacity through its partial molar volume $\bar v_i^{\infty}$ (`v_inf` in the pair file).
- γ_i^{*} — the **unsymmetric activity coefficient** of **Krichevsky–Ilinskaya** (1945), normalised so $\gamma^{*}\to1$ as $x_i\to0$; a two-suffix Margules, $\ln\gamma_i^{*} = (A/RT)(x_{\mathrm{solv}}^{2}-1)$ (`margulesA`). This is where the liquid's activity model lives in the Henry world — not NRTL on the solvent, but the solute's departure from its own dilute limit.

Worked fit: hydrogen in ammonia to 1000 atm. Plot Wiebe & Tremearne's isotherms as $\ln(f_2/x_2)$ against P (the KK coordinates) and the data collapse onto straight lines: slope $\bar v_{\mathrm{H_2}}^{\infty} = 24$ cm^3 mol^{-1} (25 and 50 °C agree), intercept $H(298) = 1.18\times10^{9}$ Pa — the apparent "drift of H with pressure" IS the Poynting term, and the two-constant line reproduces the full 1000-atm isotherm to ∼2% ($x_{\mathrm{H_2}}$ at 400 atm: computed 0.0287 vs. measured 0.0282). The same plot on Larson & Black's nitrogen data gives $\bar v_{\mathrm{N_2}}^{\infty} = 23$ cm^3 mol^{-1}. Both fits keep $\gamma^{*} = 1$: hydrogen stays on the KK line up to $x\sim9\%$, so the Margules slot is available but the data do not demand it. Everything downstream (flash, the absorber/stripper tridiagonal of Chapter 14) consumes that K_i identically — the Henry model is invisible to the stage solver, which only ever sees K_i. The same data file also feeds the heat-of-absorption energy balance: the absorber reads $\Delta H_{\mathrm{abs}} = -\Delta H_{\mathrm{diss}}$ straight from the pair file (`Absorber.cpp`), so the constant that sets the solubility *and* the constant that warms the column are one number, never two.

Supercritical solutes: the ammonia loop. Henry's law is not merely convenient at high pressure — above a solute's critical temperature it is the *only* honest anchor. In the ammonia synthesis loop the separator condenses NH_3 at 250 K and 200 bar with H_2 ($T_c = 33$ K) and N_2 ($T_c = 126$ K) far above their critical points: neither HAS a vapour pressure there, so the Raoult anchor P_i^{sat} does not exist, and extrapolating an Antoine fit above T_c manufactures a number with no physical meaning (it once predicted 13% dissolved H_2 — forty times the measured value). The shipped pairs are fitted to the classical syntheses-loop measurements: `H2-NH3` to Wiebe & Tremearne (1934), and `N2-NH3` to Larson & Black (1925) as critically evaluated in the IUPAC Solubility Data Series (Vol. 5/6), whose evaluation confirms the two gases dissolve *independently*, each linear in its partial pressure — exactly the independent-Henry-pairs model. Both carry $\Delta H_{\text{diss}} > 0$: unusually, hydrogen and nitrogen are *more* soluble in hot ammonia. With these constants the separator liquid carries ∼0.5% H_2 + ∼0.2% N_2 — the physical numbers (`tutorials/steady/flowsheets/ammonia02_full_plant`).

Power and limit. The power: with two numbers (H_{ref}, ΔH_{diss}) per pair, Henry's law gives a quantitative, temperature-correct K-value for any sparingly soluble gas — precisely the regime where the pure-liquid Raoult anchor does not exist and a cubic EoS would be a heavy, poorly constrained hammer. The limit: Eq. (136) is the *tangent at* $x_i \to 0$. It is accurate only in the dilute limit; for a water-miscible species (an alcohol, ketone, amine, or acid that spans the full composition range) the linear law is just the $x_i \to 0$ end of a curved $p_i(x_i)$, and an activity-coefficient model (NRTL/UNIQUAC, §2.3–§2.5) is the right tool. For strong electrolytes or high loadings, switch to a Pitzer / eNRTL model (Chapter 2). The shipped constants are infinite-dilution values from Sander's compilation (Sander, R., *Atmos. Chem. Phys.* **15** (2015) 4399–4981, CC-BY), valid roughly 273–373 K near 1 atm; ΔH_{diss} is taken constant across that window, so do not extrapolate far beyond it.

7.9 Why the simulator uses two solver layers, not one

In code, Eq. (134) is a strong coupling between $\mathbf{x}$, $\mathbf{y}$, T, and P. Solving it in one shot with full Newton on the joint variable would work in principle, but the simulator splits the work into two nested layers, exploiting the fact that γ_i depends on the liquid composition only:

1. **Outer loop on γ_i.** Hold γ_i fixed at the current estimate. Then the $K_i = \gamma_i P_i^{\text{sat}}/P$ are also fixed and the phase-fraction-plus-composition problem becomes a single scalar equation in the vapour fraction V (the Rachford-Rice equation; see Chapter 9). Solve it by Newton-1D.
2. **Recompute γ_i.** With the updated $\mathbf{x}$ from step 1, evaluate $\gamma_i(\mathbf{x}, T)$ from NRTL or Wilson. Go back to step 1. Wegstein (Chapter 5) accelerates this outer loop.

The reason this beats a joint Newton: γ_i is smooth in $\mathbf{x}$ but mildly non-monotone — a finite-difference Jacobian on a joint $(V, \mathbf{x}, \mathbf{y})$ system is expensive to build per iteration and easy to mis-step on. The inner Rachford-Rice problem, in contrast, is one-dimensional in V with a strictly monotone residual; Newton-1D nails it in ~ 5 iterations. Splitting buys cost-per-iteration plus robustness, at the price of an outer convergence that needs Wegstein to be fast.

> **Runnable case.** In any tutorial's `constant/propertyDict` you will find:
>
> ```
> equationOfState  { model  idealGas; }
> ```
>
> That single line is the assumption $\varphi_i = 1$. Switching to a real-gas vapour is a one-word edit — the line becomes `model SRK;` or `model PR;` — and nothing else in the case file changes. Forward compatibility by design.

8 The SRK cubic equation of state

What you'll learn. How Choupo turns the three numbers (T_c, P_c, ω) into a real-fluid vapour model: the Soave-Redlich-Kwong $a(T,\omega)$ and b, the Cardano solution of the cubic in Z, the fugacity coefficient $\hat{\varphi}_i$ that replaces the ideal-gas $\varphi_i = 1$, and the enthalpy/entropy *departure functions* $H^{\mathrm{R}}, S^{\mathrm{R}}$ that feed the energy balance at elevated pressure — all in closed form, exactly as `src/thermo/equationOfState/SRK.cpp` computes them. Then where Peng-Robinson differs by one pair of constants.

8.1 From van der Waals to Soave

Every cubic EoS descends from van der Waals (1873): a repulsive term $R_{\mathrm{g}}T/(v-b)$ that keeps molecules from overlapping (the co-volume b), and an attractive term $-a/v^2$ that pulls them together. Redlich and Kwong (1949) sharpened the attractive denominator to $v(v+b)$ and made a scale as $T^{-1/2}$. Soave's 1972 contribution [34] was to replace that fixed temperature scaling with a fitted function $\alpha(T;\omega)$ tied to the acentric factor, so the EoS reproduces pure-component vapour pressures along the whole saturation curve — the single change that made cubic EoS accurate enough for VLE. The result is the Soave-Redlich-Kwong (SRK) equation Choupo implements:

$$P \;=\; \frac{R_{\mathrm{g}}T}{v-b} \;-\; \frac{a(T)}{v\,(v+b)}. \tag{141}$$

This is the $u=1,\ w=0$ member of the generalised cubic $P = R_{\mathrm{g}}T/(v-b) - a(T)/(v^2 + u\,b\,v + w\,b^2)$ introduced in Chapter 7.

8.2 Pure-component parameters

The co-volume b and the critical attraction a_c come from forcing Eq. (141) to satisfy the critical conditions $(\partial P/\partial v)_{T_c} = (\partial^2 P/\partial v^2)_{T_c} = 0$. For SRK this fixes the two universal constants $\Omega_a = 0.42747$ and $\Omega_b = 0.08664$:

$$a_{c,i} = \Omega_a \,\frac{R_{\mathrm{g}}^2 T_{c,i}^2}{P_{c,i}} = 0.42747\,\frac{R_{\mathrm{g}}^2 T_{c,i}^2}{P_{c,i}}, \tag{142}$$

$$b_i = \Omega_b \,\frac{R_{\mathrm{g}} T_{c,i}}{P_{c,i}} = 0.08664\,\frac{R_{\mathrm{g}} T_{c,i}}{P_{c,i}}. \tag{143}$$

The temperature dependence lives entirely in Soave's α:

$$a_i(T) \;=\; a_{c,i}\,\alpha_i(T), \qquad \alpha_i(T) = \left[\,1 + m_i\,(1 - \sqrt{T/T_{c,i}})\,\right]^2, \tag{144}$$

with the Soave slope a quadratic in the acentric factor [34]:

$$m_i \;=\; 0.48508 + 1.55171\,\omega_i - 0.15613\,\omega_i^2. \tag{145}$$

Note $\alpha_i(T_{c,i}) = 1$ exactly (the curve is anchored at the critical point), and α rises as T falls below T_c, strengthening attraction in the subcritical region where the vapour pressure must be matched.[2]

The temperature derivative $\mathrm{d}a_i/\mathrm{d}T$ — needed for the departure functions below — follows by the chain rule on Eq. (144):

$$\frac{\mathrm{d}a_i}{\mathrm{d}T} = -\,a_{c,i}\,\frac{m_i\left[1 + m_i(1-\sqrt{T/T_{c,i}})\right]}{\sqrt{T\,T_{c,i}}}. \tag{146}$$

8.3 Mixing rules

A mixture needs single effective $a_{\mathrm{mix}}, b_{\mathrm{mix}}$. Choupo uses the classical van der Waals one-fluid rules with a binary interaction parameter k_{ij}:

$$a_{\mathrm{mix}}(T) = \sum_i \sum_j y_i\,y_j\,(1-k_{ij})\,\sqrt{a_i(T)\,a_j(T)}, \tag{147}$$

$$b_{\mathrm{mix}} = \sum_i y_i\,b_i. \tag{148}$$

The k_{ij} are small empirical corrections (zero by default, symmetric, $k_{ii}=0$) read from the optional `binaryInteractions` sub-list. They are the only fitted mixture quantity; everything else is pure-component. The mixture derivative required later is

$$\frac{\mathrm{d}a_{\mathrm{mix}}}{\mathrm{d}T} = \sum_i \sum_j y_i y_j (1-k_{ij})\,\tfrac{1}{2}\left(\sqrt{\tfrac{a_j}{a_i}}\,\frac{\mathrm{d}a_i}{\mathrm{d}T} + \sqrt{\tfrac{a_i}{a_j}}\,\frac{\mathrm{d}a_j}{\mathrm{d}T}\right). \tag{149}$$

[2] Choupo uses the original Soave slope of Eq. (145); some texts quote the rounded $0.480 + 1.574\,\omega - 0.176\,\omega^2$. The difference is within the model's own accuracy.

8.4 The cubic in Z and its Cardano root

Writing $Z = Pv/R_{\mathrm{g}}T$ and the dimensionless groups

$$A = \frac{a_{\mathrm{mix}}\,P}{(R_{\mathrm{g}}T)^2}, \qquad B = \frac{b_{\mathrm{mix}}\,P}{R_{\mathrm{g}}T}, \tag{150}$$

Eq. (141) rearranges to the cubic

$$\boxed{Z^3 - Z^2 + (A - B - B^2)\,Z - A\,B = 0} \tag{151}$$

Choupo solves Eq. (151) *analytically* by Cardano's method (`cardano_vapourRoot`), not by iteration — a cubic has a closed-form solution, so there is no Newton loop and no initial guess to get wrong. The discriminant $\Delta = R_d^2/4 + Q_d^3/27$ of the depressed cubic decides the branch: $\Delta > 0$ gives one real root (single fluid phase); $\Delta \le 0$ gives three real roots, evaluated through the trigonometric form $w_k = 2\sqrt[3]{r}\cos\!\big((\phi + 2\pi k)/3\big)$. When three roots exist the *largest above* B is the vapour root (largest molar volume); the smallest would be the liquid root. Choupo's EoS path returns the **vapour root** throughout — it is the $\hat{\varphi}_i$ supplier for the gas phase of the γ-φ flash.

8.5 The fugacity coefficient

Differentiating the residual Helmholtz energy with respect to mole number (the recipe of Chapter 7) gives the SRK partial fugacity coefficient in closed form:

$$\boxed{\ln\hat{\varphi}_i = \frac{b_i}{b_{\mathrm{mix}}}(Z-1) - \ln(Z-B) - \frac{A}{B}\left(\frac{2\sum_j y_j(1-k_{ij})\sqrt{a_i a_j}}{a_{\mathrm{mix}}} - \frac{b_i}{b_{\mathrm{mix}}}\right)\ln\!\left(1+\frac{B}{Z}\right)} \tag{152}$$

This is exactly the expression in `SRK::phi`; the inner sum $\sum_j y_j(1-k_{ij})\sqrt{a_i a_j}$ is the i-th row of the same matrix that built a_{mix}, so it costs nothing extra. In the ideal-gas limit ($A, B \to 0$, $Z \to 1$) every term vanishes and $\hat{\varphi}_i \to 1$, recovering the assumption the tutorials ship with.

8.6 Departure (residual) functions for energy balances

The flash gives compositions; the *energy* balance needs how far real-gas enthalpy and entropy sit from the ideal-gas reference at the same T, P. These are the *departure* (residual) functions. For the SRK form ($\sigma = 1, \varepsilon = 0$ in the generalised cubic), Choupo evaluates [33]

$$H^{\mathrm{R}}(T,P,\mathbf{y}) = R_{\mathrm{g}}T\,(Z-1) + \frac{T\,\dfrac{\mathrm{d}a_{\mathrm{mix}}}{\mathrm{d}T} - a_{\mathrm{mix}}}{b_{\mathrm{mix}}}\,\ln\frac{Z}{Z+B}, \tag{153}$$

$$S^{\mathrm{R}}(T,P,\mathbf{y}) = R_{\mathrm{g}}\,\ln(Z-B) + \frac{1}{b_{\mathrm{mix}}}\,\frac{\mathrm{d}a_{\mathrm{mix}}}{\mathrm{d}T}\,\ln\frac{Z}{Z+B}. \tag{154}$$

These are `SRK::H_residual` and `SRK::S_residual` line for line. The real-fluid enthalpy used in the energy ledger is then the ideal-gas enthalpy (from the heat-capacity integral of Chapter 5) *plus* H^{R}; likewise for entropy. Both departures vanish as $P \to 0$ ($Z \to 1$, $B \to 0$), so the high-pressure correction switches itself off where it is not needed — the same self-consistency as $\hat{\varphi}_i \to 1$.

8.7 Peng-Robinson: the same machine, two constants

Peng-Robinson (1976) [35] keeps the entire structure above and changes only the attractive denominator to $v^2 + 2bv - b^2$, i.e. the generalised-cubic constants $\varepsilon = 1 - \sqrt{2}$, $\sigma = 1 + \sqrt{2}$ (so $u = 2, w = -1$). Three substitutions convert SRK into PR:

	SRK	**Peng-Robinson**
Ω_a	0.42747	0.45724
Ω_b	0.08664	0.07780
$m(\omega)$ / $\kappa(\omega)$	$0.48508 + 1.55171\omega - 0.15613\omega^2$	$0.37464 + 1.54226\omega - 0.26992\omega^2$
attractive term	$a/[v(v+b)]$	$a/[v^2 + 2bv - b^2]$
ln factor	$\ln\dfrac{Z+B}{Z}$	$\dfrac{1}{2\sqrt{2}}\ln\dfrac{Z+(1+\sqrt{2})B}{Z+(1-\sqrt{2})B}$

In the code this is literally the difference between `SRK.cpp` and `PR.cpp`: the constants `OMEGA_A`, `OMEGA_B`, the slope function, and the single logarithm `ln_genB`. PR's third critical condition forces $Z_c = 0.307$ versus SRK's 0.333 — both above the true $Z_c \approx 0.27$ of most fluids, but PR's is closer, which is why PR gives better *liquid* densities. SRK is often marginally better for light-gas *vapour* fugacities. Choupo ships both; the case picks one with a single keyword.[3]

[3] Choupo uses the original 1976 PR slope $\kappa(\omega)$; the 1978 high-ω refinement is not applied (see the comment in `PR.cpp`).

Worked example 8.1. SRK a, b for CO_2 at 320 KWith $T_c = 304.13$ K, $P_c = 73.77$ bar $= 7.377 \times 10^6$ Pa, $\omega = 0.225$ and $R_g = 8.314$ J mol^{-1}K^{-1}:

$$b = 0.08664 \frac{8.314 \times 304.13}{7.377 \times 10^6} = 2.97 \times 10^{-5} \text{ m}^3 \text{ mol}^{-1},$$

$$a_c = 0.42747 \frac{8.314^2 \times 304.13^2}{7.377 \times 10^6} = 0.366 \text{ Pa m}^6 \text{ mol}^{-2}.$$

The Soave slope $m = 0.48508 + 1.55171(0.225) - 0.15613(0.225)^2 = 0.827$. At $T = 320$ K, $T_r = 1.052$, so $\sqrt{T_r} = 1.026$ and $\alpha = [1 + 0.827(1 - 1.026)]^2 = 0.957$, giving $a(320) = 0.350$ Pa m^6 mol^{-2}. At $P = 50$ bar these yield $A = aP/(R_gT)^2 = 0.247$, $B = bP/(R_gT) = 0.0558$, and the Cardano vapour root of Eq. (151) is $Z \approx 0.80$ — a 20% departure from the ideal gas, exactly the regime where keeping `model idealGas` would corrupt both the energy balance and the fugacity coefficient.

Power and limit. Three numbers per species (T_c, P_c, ω) and one optional k_{ij} per pair buy a closed-form real-gas model: $\hat{\varphi}_i \neq 1$, real Z, and the H^R, S^R departures that make the energy balance correct at tens of bar. No iteration (Cardano is analytic), no per-species curve fitting beyond the acentric factor, and the exact same algebra serves SRK and PR. The **limit**: cubic EoS are vapour/non-polar workhorses. They are poor for strongly hydrogen-bonded or associating fluids (water, alcohols, acids — where the activity-coefficient γ-route of Chapter 2 is the right tool for the liquid), and the van der Waals one-fluid mixing rule degrades for very asymmetric or polar mixtures. Choupo returns the *vapour* root only; it is not (yet) a liquid-density or full-VLLE EoS engine, and a single k_{ij} cannot rescue a system the underlying α-function gets wrong. As always: all models are wrong, some are useful — SRK/PR earn their keep precisely where the ideal gas stops being so and association has not yet taken over.

Runnable case. Select the cubic EoS in any case's `constant/propertyDict`:

```
equationOfState
{
    model  SRK;            // or  PR;
    binaryInteractions     // optional; k_ij = 0 by default
    (
        { i  CO2;   j  CH4;   kij  0.0919; }
    );
}
```

The components must carry `Tc`, `Pc` and `omega` in their `.dat` files (the loader rejects the EoS otherwise — see `requiredComponentProperties`).

9 K-values and Rachford-Rice

What you'll learn. The single equation at the heart of every flash, bubble-point, dew-point, and column stage in the simulator — and why it has exactly one root when a two-phase region exists.

9.1 Defining the K-value

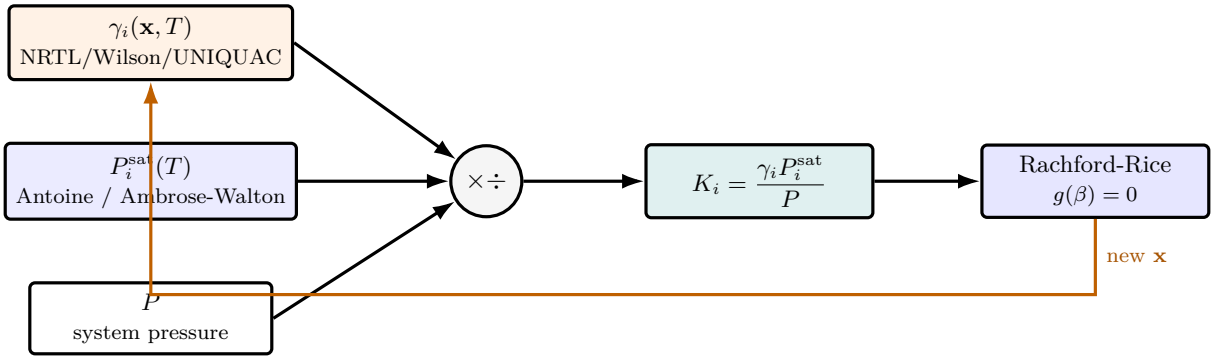

Figure 22: What the K-value is, and where it sits in the flash loop. Modified Raoult's law assembles each $K_i = \gamma_i P_i^{\text{sat}}/P$ from three glass-box ingredients: the activity coefficient (from the chosen G^E model), the pure-component vapour pressure (Antoine), and the system pressure. Once $\mathbf{K}$ is fixed, the flash is a *pure mass-balance* problem — Rachford-Rice for β. But γ_i depends on the liquid composition $\mathbf{x}$, which Rachford-Rice updates, so the orange arrow closes an *outer* loop: re-evaluate $\mathbf{K}$ at the new $\mathbf{x}$ and repeat. This composition–K coupling is exactly why the flash needs two solver layers, not one.

The K-value of component i is the partition ratio between vapour and liquid:

$$K_i \equiv \frac{y_i}{x_i}. \tag{155}$$

Combining with (134):

$$\boxed{K_i = \frac{\gamma_i(\mathbf{x}, T)\, P_i^{\text{sat}}(T)}{P}} \tag{156}$$

At each outer iteration, K_i is a function of $T, P, \mathbf{x}$ alone — once $\mathbf{K}$ is fixed, the flash becomes a pure mass-balance problem.

9.2 Material balance and Rachford-Rice

A single-stage flash takes feed $\mathbf{z}$ at F and splits it into vapour V at composition $\mathbf{y}$ and liquid L at composition $\mathbf{x}$. The component balance is:

$$F z_i = V y_i + L x_i = F[\beta y_i + (1-\beta) x_i]. \tag{157}$$

Cancelling F and using $y_i = K_i x_i$:

$$z_i = \beta K_i x_i + (1-\beta) x_i = x_i\,[1 + \beta(K_i - 1)],$$

so

$$x_i = \frac{z_i}{1 + \beta(K_i - 1)}, \qquad y_i = \frac{K_i z_i}{1 + \beta(K_i - 1)}. \tag{158}$$

Imposing $\sum_i y_i - \sum_i x_i = 0$ (the closure) and collecting terms yields the *Rachford-Rice equation*:

$$\boxed{g(\beta) \equiv \sum_{i=1}^{N_c} \frac{z_i\,(K_i - 1)}{1 + \beta(K_i - 1)} = 0} \tag{159}$$

9.3 Why exactly one root, in $(0, 1)$, when there is two-phase

Two properties make Rachford-Rice numerically pleasant:

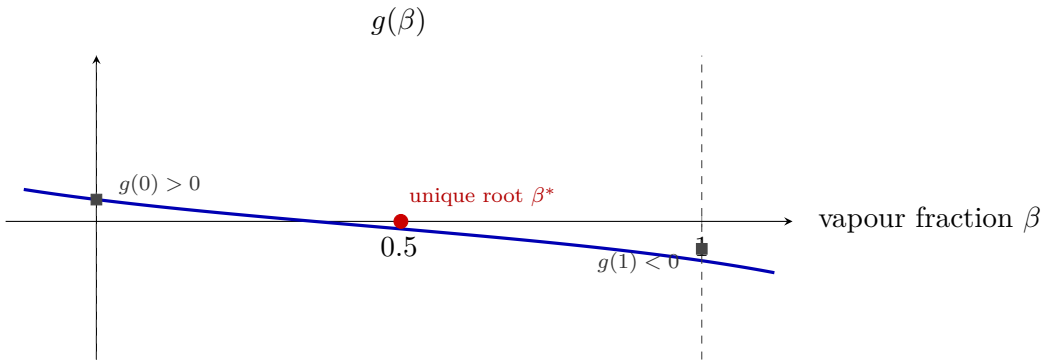

Figure 23: Why Rachford-Rice is numerically tame. When a two-phase region exists, some $K_i > 1$ (volatiles) and some $K_i < 1$ (heavies), so $g(0) > 0$ and $g(1) < 0$ — the function changes sign across the physical window $[0, 1]$. And $g'(\beta) < 0$ everywhere it is defined (each term in the derivative is a negative square), so g is *strictly decreasing*. A monotone function with one sign change has exactly one root, and Newton-Raphson from any interior β_0 converges to it quadratically. If the flash will not converge, the bug is upstream — a bad K_i (Antoine out of range, γ_i blowing up), never Rachford-Rice itself.

Monotonicity. Differentiating (159):

$$g'(\beta) \;=\; -\sum_i \frac{z_i\,(K_i-1)^2}{[1+\beta(K_i-1)]^2} \;<\; 0.$$

Since each term in the sum is non-negative, $g'(\beta) < 0$ wherever the sum is defined — g is strictly decreasing.

Sign change. If a two-phase region exists, some $K_i > 1$ (the volatile components) and some $K_i < 1$ (the heavy ones). Then $g(0) = \sum_i z_i(K_i - 1)$ has the sign of "are most components volatile?", and $g(1) = \sum_i z_i(1 - 1/K_i)$ has the opposite sign. Both endpoints differ in sign, so a root exists in $(0, 1)$.

The combination of monotonicity and a sign change in $(0, 1)$ means Newton-Raphson starting from *any* interior β_0 will converge, quadratically, to the unique root.

> **Key idea.** Rachford-Rice is convex enough to make Newton-Raphson tame. If your flash refuses to converge, the bug is upstream — almost always a bad K_i (Antoine outside its range; γ_i blowing up; T below the lower phase boundary).

9.4 Initialisation: the Wilson K-value

A robust initial guess for $\mathbf{K}$ before any activity-coefficient information is available comes from Wilson's correlation:

$$K_i^{(0)}(T,P) \;=\; \frac{P_{c,i}}{P}\,\exp\!\Big[\,5.373\,(1+\omega_i)\,\big(1-T_{c,i}/T\big)\Big]. \tag{160}$$

This uses only the critical point and the acentric factor — both in the component database — and works astonishingly well as a starting point.

9.5 Worked example

> **Worked example 9.1.** Flash of benzene/toluene at 370 K, 1 barA feed at $\mathbf{z} = (0.40,\ 0.60)$ (benzene then toluene), $T = 370\,K$,
> $P = 1$ bar. At this temperature, Antoine gives $P_1^{\text{sat}} \approx 1.46$ bar and $P_2^{\text{sat}} \approx 0.61$ bar. Ideal solution: $\gamma_i = 1$, so $K_1 = 1.46$ and $K_2 = 0.61$.
> Rachford-Rice:
>
> $$g(\beta) \;=\; \frac{0.40\cdot 0.46}{1+0.46\beta} + \frac{0.60\cdot(-0.39)}{1-0.39\beta} \;=\; 0.$$
>
> At $\beta = 0$: $g = 0.184 - 0.234 = -0.050$. At $\beta = 1$: $g = 0.126 - 0.383 = -0.257$. Both endpoints are negative — so this feed is below the bubble point. Iterating Newton on g starting from $\beta_0 = -0.1$ converges quickly to $\beta \approx -0.13$, which is unphysical: confirmation that the mixture is sub-cooled at $370\,K$. To get a two-phase split we need $T \gtrsim 375$ K. This is exactly the `flash01_benzene_toluene` setup; the simulator will report the unphysical β and warn.

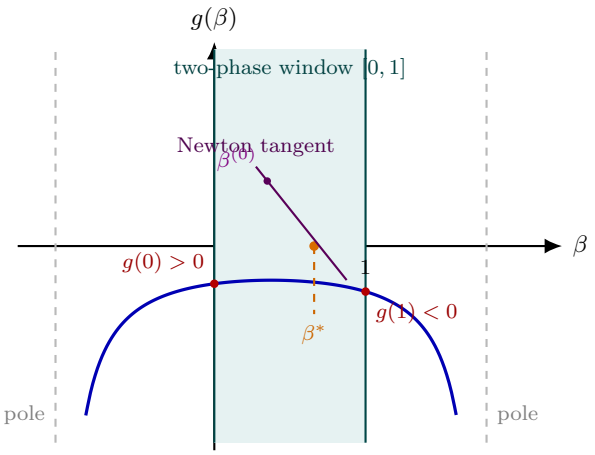

Figure 24: Why Rachford–Rice is tame. Each term has a pole where $1 + \beta(K_i - 1) = 0$; for a genuine two-phase mixture all the poles lie *outside* the physical window $[0, 1]$ (shaded), so on that window g is a single, smooth, strictly *decreasing* branch ($g'(\beta) < 0$ always). When a split exists the endpoints straddle zero — $g(0) > 0$, $g(1) < 0$ — guaranteeing exactly one root β^* in $(0, 1)$. Monotone plus a sign change is the textbook recipe for a trouble-free Newton: the tangent from *any* interior $\beta^{(0)}$ slides to β^* and converges quadratically. If the endpoints share a sign, the root is pushed outside $[0, 1]$ — the subcooled / superheated case the flash reports honestly.

> **Runnable case.** `tutorials/flash01_benzene_toluene` — run with `verbosity 3` and watch the Newton trace converge in 3–4 iterations.

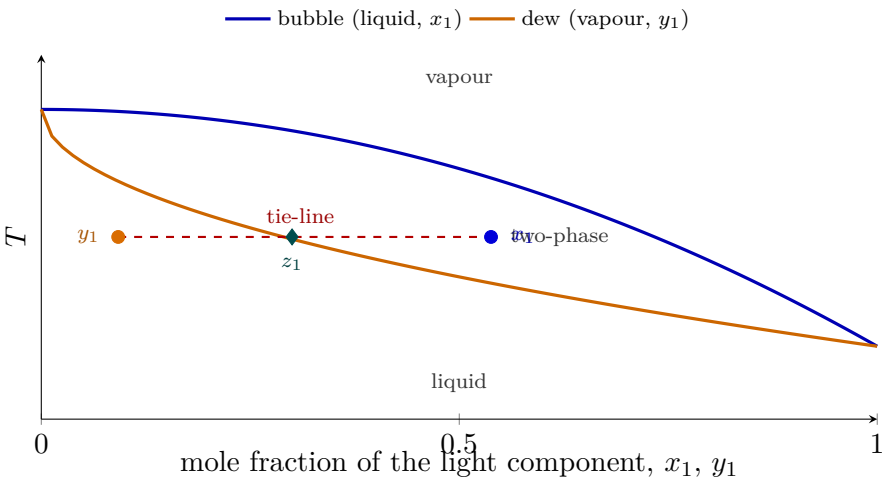

Figure 25: A temperature–composition (Txy) diagram, the phase-diagram face of the flash. The lower (blue) curve is the bubble locus, plotted against the *liquid* composition x_1; the upper (orange) curve is the dew locus, plotted against the *vapour* composition y_1. Between them lies the two-phase region. At a flash temperature, the horizontal *tie-line* (red) connects the coexisting liquid x_1 and vapour y_1 in equilibrium ($y_1 = K_1 x_1$). A feed of composition z_1 (diamond) lands inside the lens, and the lever rule along the tie-line fixes the vapour fraction β that Rachford-Rice computes. Outside the lens there is a single phase — below it sub-cooled liquid, above it superheated vapour.

Part IV — Numerical methods

1 Numerical methods, gently

What you'll learn. A first look at root-finding, with no Jacobians and no Taylor expansions yet. We will pose one concrete chemical-engineering problem — the bubble temperature of a benzene/toluene mixture — and solve it four times: by hand, by bisection, by secant, and finally by Newton-Raphson. By the end you will have a feel for *why* the simulator uses Newton everywhere instead of just guessing, and the heavier chapters that follow (Newton-1D, Newton in n dimensions, Levenberg-Marquardt) will read as natural generalisations of what you just did by hand.

1.1 One problem, four strategies

Mix 50 mol% benzene and 50 mol% toluene in a sealed vessel at $P = 1$ **bar** and slowly heat the liquid. At what temperature does the first vapour bubble form?

For an ideal Raoult mixture the bubble condition is

$$x_b\, P_b^{\mathrm{sat}}(T) \;+\; x_t\, P_t^{\mathrm{sat}}(T) \;=\; P, \tag{161}$$

which with $x_b = x_t = 0.5$ and $P = 1$ bar reduces to $P_b^{\mathrm{sat}}(T) + P_t^{\mathrm{sat}}(T) = 2$ bar. Both saturation pressures come from Antoine,

$$\log_{10} P^{\mathrm{sat}}(T) \;=\; A \;-\; \frac{B}{T + C},$$

with the constants shipped in `data/standards/components/`: $A_b = 4.0181$, $B_b = 1203.84$, $C_b = -53.23$ K for benzene; $A_t = 4.0783$, $B_t = 1343.94$, $C_t = -53.77$ K for toluene.

Define the **residual**

$$f(T) \;\equiv\; \tfrac{1}{2}\big[P_b^{\mathrm{sat}}(T) + P_t^{\mathrm{sat}}(T)\big] - 1, \tag{162}$$

which is zero exactly at the bubble temperature. Knowing $T_b \approx 353$ K and $T_t \approx 384$ K, the answer must lie somewhere between those, but where?

Intuition. Every numerical method in this book solves the same shape of problem: **given a function f, find a T for which $f(T) = 0$.** The strategies differ only in how cleverly they use the available information about f. The dumbest method needs only the ability to evaluate f; the smartest also needs f'. More information you exploit, fewer evaluations you need.

1.2 Strategy 1 — guess and adjust

Pick a temperature, compute f, see whether it is positive or negative, adjust. Honest, slow, but absolutely transparent:

T [K]	$f(T)$ [bar]
340	-0.55
360	-0.13
380	$+0.53$
365	$+0.0053$
364	-0.024

After five tries we know the root is between 364 and 365 K and closer to the upper bound. Two more refinements get us to within 0.05 K of the answer. The procedure works but you do all the thinking.

1.3 Strategy 2 — bisection

Trial-and-error implicitly used a fundamental fact: f is continuous, $f(340) < 0$, $f(390) > 0$, so by the intermediate value theorem the root sits somewhere in $[340, 390]$. Bisection mechanises that intuition: take the midpoint, evaluate f, keep the half that still brackets the root.

Worked example 1.1. Bisection on Eq. (161)Starting bracket $[340, 390]$. Midpoint $T_1 = 365$, $f(T_1) = +0.0053$. Since f changed sign on the lower half, replace b with T_1 and repeat:

iter	T [K]	f [bar]
1	365.000	$+5.34 \times 10^{-3}$
2	352.500	-3.15×10^{-1}
3	358.750	-1.67×10^{-1}
5	363.438	-4.02×10^{-2}
8	364.805	-4.43×10^{-4}
12	364.817	-8.24×10^{-5}
20	364.820	$\pm 7 \times 10^{-7}$

After 20 iterations $T \approx 364.820$ K.

Bisection is a tortoise: every iteration halves the bracket, so the error decays exactly by a factor 2. To get from a 50 K wide bracket down to a 10^{-7} K tolerance takes roughly $\log_2(50/10^{-7}) \approx 29$ steps. Never blows up, never overshoots — but never fast either.

1.4 Strategy 3 — secant

Bisection only uses the *sign* of f at each midpoint. We can do better by exploiting the *value*: if f has changed by a known amount between two trial T's, a linear extrapolation predicts where it would hit zero. Geometrically, draw the secant line through the last two points and read off the T-intercept:

$$T_{k+1} \;=\; T_k \;-\; f(T_k)\,\frac{T_k - T_{k-1}}{f(T_k) - f(T_{k-1})}. \tag{163}$$

Worked example 1.2. Secant on Eq. (161)Starting from the same two points $(T_0, T_1) = (340, 390)$ K:

iter	T [K]	f [bar]
1	357.817	-1.91×10^{-1}
2	363.009	-5.24×10^{-2}
3	364.975	$+4.60 \times 10^{-3}$
4	364.816	-9.74×10^{-5}
5	364.81968	-1.76×10^{-7}
6	364.81969	$+6.7 \times 10^{-12}$

Six iterations to machine precision.

Each step uses two function evaluations of history, no derivatives. The order of convergence is $\varphi = (1+\sqrt{5})/2 \approx 1.618$ — the golden ratio. In practice this means the number of correct digits multiplies by 1.6 each iteration: 1 digit, 2, 3, 5, 8, 12 — so it doubles roughly every two iterations.

Common pitfall. The secant formula breaks if $f(T_k) = f(T_{k-1})$ (division by zero). Bisection cannot break that way. In practice the simulator falls back to bisection whenever the secant step would carry the iterate outside the current bracket.

1.5 Strategy 4 — Newton-Raphson, in one breath

Secant approximated the slope of f from two adjacent evaluations. What if we knew the slope exactly — the derivative $f'(T)$? Then we could shoot a line off the current point and intersect zero in *one* step:

$$\boxed{\,T_{k+1} \;=\; T_k \;-\; \frac{f(T_k)}{f'(T_k)}.\,} \tag{164}$$

That is Newton-Raphson. The next chapter derives it rigorously from a Taylor expansion; here we just apply it.

Worked example 1.3. Newton on Eq. (161)Start from $T_0 = 340$ K with $f'(T)$ computed numerically (or, more typically in this simulator, analytically from $\mathrm{d}P^{\mathrm{sat}}/\mathrm{d}T$):

iter	T [K]	f [bar]
1	374.942	$+3.36 \times 10^{-1}$
2	365.872	$+3.15 \times 10^{-2}$
3	364.832	$+3.74 \times 10^{-4}$
4	364.8197	$+5.5 \times 10^{-8}$
5	364.8197	$+2.4 \times 10^{-15}$

Four iterations to 10^{-8}, five to machine precision.

Look at the residual column above. At iteration 3 it is 3.7×10^{-4}; at iteration 4 it is 5.5×10^{-8}. The exponent jumped from -4 to -8 — the error **squared**. That is the signature of **quadratic convergence**, and it is the reason every iterative solver in this simulator is Newton-based when a derivative is available.

> *Intuition.* "Quadratic" here means the number of correct decimal digits *doubles* every iteration: 2 digits, 4, 8, 16, machine precision. Bisection adds one bit per step; secant multiplies digits by 1.6; Newton multiplies digits by 2. This is why even though Newton needs the extra information f', it pays back so dramatically that it is essentially never not worth it.

1.6 The four strategies side by side

Figure 26 plots $|f(T_k)|$ versus iteration number for the four runs on a semi-log axis. The slope of each line is the convergence rate.

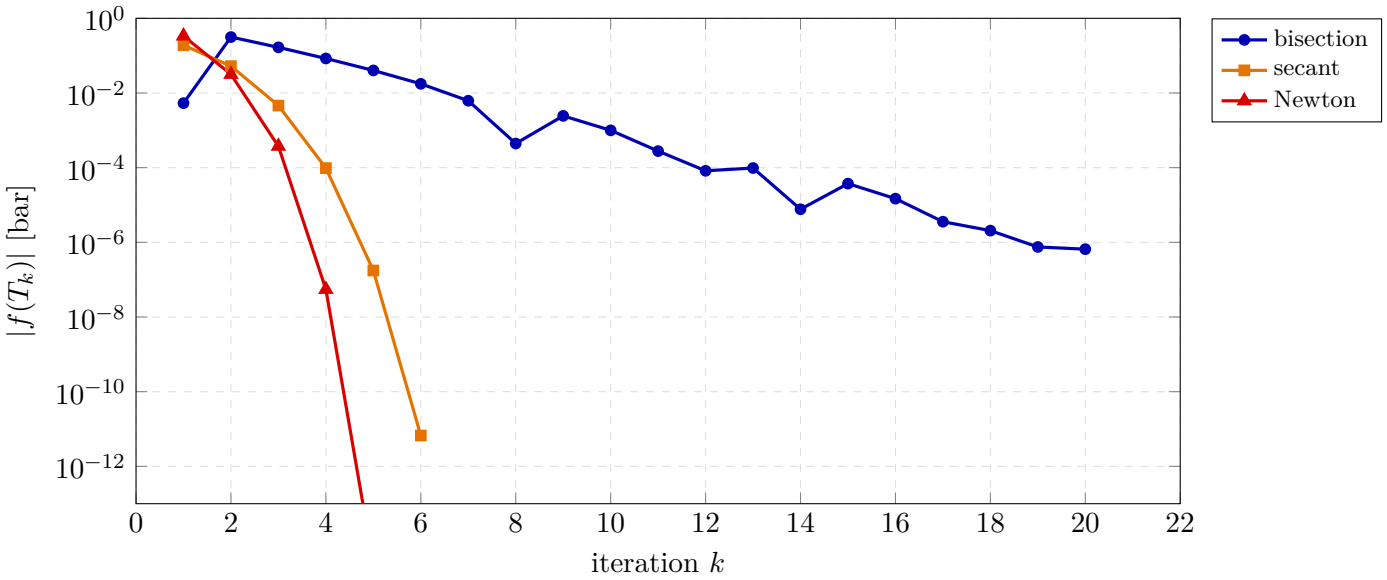

Figure 26: Residual decay for the four strategies on the same benzene/toluene bubble-temperature problem (initial bracket $[340, 390]$ K, target $T_b \approx 364.82$ K). Bisection is straight on the semi-log plot — a constant factor of ~ 2 per step. Secant accelerates with the slope steepening over time. Newton dives off the chart in five steps.

method	uses	per-iter cost	order p	iters to 10^{-8}
trial & error	f values + intuition	1 eval	—	many
bisection	f signs	1 eval	1 (linear)	∼30
secant	last 2 f values	1 eval	1.618	∼6
Newton	f *and* f'	1 eval + 1 deriv	2 (quadratic)	∼4

1.7 Why the rest of Part III matters

Every solver in the simulator is a generalisation of these four

moves:

- Chapter 2 re-derives Newton from Taylor's theorem, formalises "convergence," and shows how the simulator damps an overshoot or falls back to bisection when the local linear model misleads.

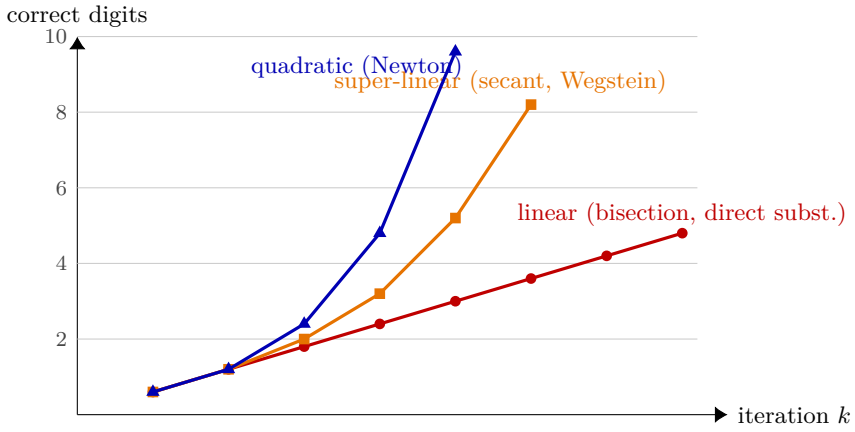

Figure 27: The three convergence *orders* that organise every solver in this manual, read as "how many correct digits each step buys." *Linear* convergence (red — bisection, direct substitution) adds a roughly constant number of digits per step, so the digit count climbs along a straight line. *Super-linear* (orange — the secant and Wegstein) speeds up as it goes: the per-step gain itself grows. *Quadratic* (blue — Newton and its n-D and tear-loop descendants) *doubles* the correct digits every step once it is near the root, which is why Newton clears twelve digits in a handful of iterations. Damping and finite-difference Jacobians are the price paid to keep a method on its fast curve without falling off into divergence far from the root.

- The next chapter generalises Newton to vector equations — the kind that pop up the moment you have multiple unknowns (multi-component flash, adiabatic flash, rigorous distillation). Replace "$f'(T_k)$" by a *Jacobian matrix*; replace "divide" by "solve a linear system." Nothing else changes.
- Levenberg-Marquardt is Newton plus a damping knob that lets it gracefully degrade to gradient descent when the linear model is bad. We use it for parameter estimation from experimental data.
- Wegstein and direct substitution attack a different shape of problem ($x = f(x)$ rather than $f(x) = 0$) and accelerate fixed-point iterations. We use them on activity-coefficient outer loops where computing a derivative is awkward.

Keep the four strategies in mind as you read on. Every more elaborate algorithm in this manual is, at heart, one of them dressed up.

2 Newton-Raphson, one dimension

What you'll learn. The single algorithm that solves every scalar fixed-point in the simulator. Why it converges quadratically when it converges, and what damping is for.

2.1 Derivation

Given a continuously differentiable $g\colon \mathbb{R} \to \mathbb{R}$ and a root x^* with $g(x^*) = 0$, Taylor-expand around an estimate x_k:

$$0 \;=\; g(x^*) \;=\; g(x_k) + g'(x_k)\,(x^* - x_k) + O((x^* - x_k)^2).$$

Solving for x^* and dropping the higher-order term defines the *Newton iteration*:

$$\boxed{\; x_{k+1} \;=\; x_k \;-\; \frac{g(x_k)}{g'(x_k)} \;} \tag{165}$$

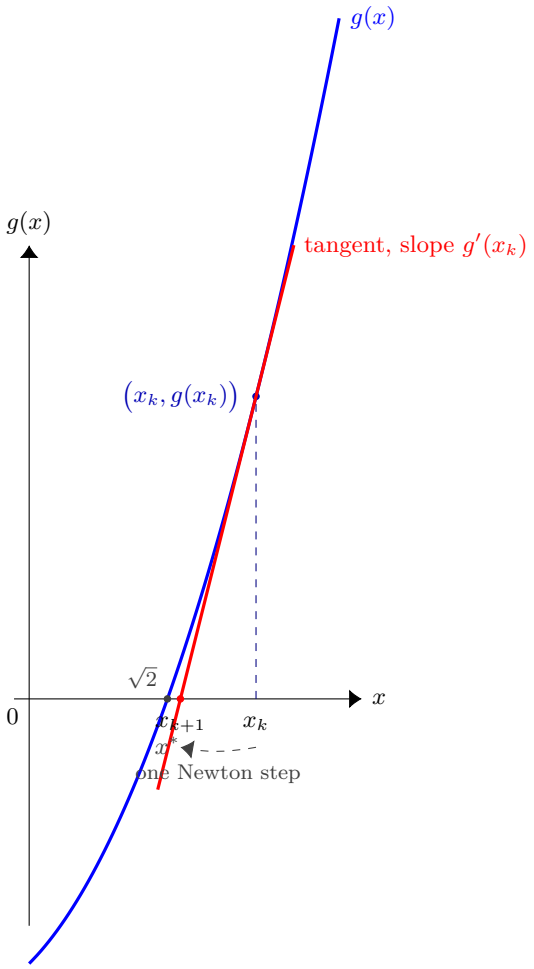

Figure 28: One Newton iteration on $g(x) = x^2 - 2$ (root $x^* = \sqrt{2} \approx 1.4142$). At the current estimate $x_k = 2$, draw the tangent to g at $(x_k, g(x_k))$ with slope $g'(x_k) = 4$; the tangent's x-intercept is the next estimate $x_{k+1} = 1.5$. Two more iterations from there converge to x^* to twelve digits — the quadratic doubling of correct digits per step that gives Newton its reputation.

2.2 Convergence

Define the error $e_k \equiv x_k - x^*$. A second-order Taylor expansion of g around x^* gives

$$g(x_k) \;=\; g'(x^*)\,e_k \;+\; \tfrac{1}{2} g''(x^*)\,e_k^2 \;+\; O(e_k^3),$$

and similarly $g'(x_k) = g'(x^*) + g''(x^*)e_k + O(e_k^2)$. Substituting into (165) and keeping leading order in e_k:

$$e_{k+1} \;=\; \frac{g''(x^*)}{2\,g'(x^*)}\,e_k^2 \;+\; O(e_k^3). \tag{166}$$

This is *quadratic convergence*: the error squares at each step, provided $g'(x^*) \neq 0$ and we start close enough to x^*.

2.3 Damping

Far from the root, the linearisation can over-shoot — pushing x_{k+1} into a region where g' has the wrong sign, or even outside the domain of g. A damping factor $\lambda \in (0, 1]$ tames this:

$$x_{k+1} \;=\; x_k - \lambda\, \frac{g(x_k)}{g'(x_k)}, \tag{167}$$

with λ reduced (typically halved) until $|g(x_{k+1})| < |g(x_k)|$. This loses quadratic convergence *near the root* (where λ returns to 1 anyway), but buys global robustness. The simulator's `NewtonRaphson` uses backtracking damping with a halving schedule.

2.4 Worked example

Worked example 2.1. Bubble pressure of benzene/tolueneSolve $g(P) = \sum_i x_i \gamma_i P_i^{\mathrm{sat}}(T)/P - 1 = 0$ for P at $T = 365\,\mathrm{K}$, $x_1 = 0.4$ benzene, ideal solution.
At $T = 365\,\mathrm{K}$ Antoine gives $P_1^{\mathrm{sat}} \approx 1.20$ bar, $P_2^{\mathrm{sat}} \approx 0.49$ bar. So $g(P) = (0.4 \cdot 1.20 + 0.6 \cdot 0.49)/P - 1 = 0.774/P - 1$.
This is linear in $1/P$ — the root is $P^* = 0.774$ bar in one "Newton step". Real cases with $\gamma_i = \gamma_i(\mathbf{x}, T)$ are non-linear: typical convergence is 3–5 steps to $|g| < 10^{-8}$.

3 Newton-Raphson, n dimensions

What you'll learn. Most non-trivial equations in a process simulator have more than one unknown. This chapter generalises the Newton step of §2 to a system of n coupled equations in n unknowns. The key conceptual leap is that a derivative becomes a *Jacobian matrix*, and the division g/g' becomes a *linear-system solve* $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$. Everything else — damping, finite-difference approximation, convergence behaviour — carries over from the scalar case once that leap is made.

3.1 Why we need a multi-dimensional Newton

The 1-D Newton iteration solves one equation in one unknown. Lots of chemistry is not like that. A few representative examples from this simulator:

- **Multi-component flash.** At fixed $(T, P, \mathbf{z})$, the Rachford-Rice equation determines V/F *given* the K_i's; but the K_i's themselves depend on the unknown liquid composition $\mathbf{x}$, which in turn depends on V/F. If we close this loop simultaneously instead of with an outer Wegstein loop, we get n_c coupled equations in n_c unknowns.
- **Adiabatic flash.** Both T and V/F are unknown; the system is two equations (component-balance closure + energy balance) in two unknowns.
- **Rigorous distillation.** Each stage contributes its own MESH equations (Mass, Equilibrium, Summations, Heat balance); a column with N_s stages and n_c components builds a system of $O(N_s n_c)$ coupled equations. Naphtali–Sandholm uses Newton-ND on the whole block.
- **Reactive distillation.** A reaction on the catalytic stages adds, per reactive stage j, the source $\nu_i\,\xi_j$ to that stage's component balance, with the molar extent ξ_j set either by chemical *equilibrium*, $\sum_i \nu_i \ln\!\big(\gamma_i\, x_{j,i}\big) = \ln K_a(T_j)$, (one extra unknown ξ_j and one extra equation per stage), or by a *rate* law $\xi_j = r_j\, m_{\mathrm{cat},j}$ (no extra unknown — ξ_j is a function of x_j, T_j). Because the reaction makes the residual far more nonlinear, it is converged by *homotopy*: solve the column non-reactive first, then switch the reaction on, seeded from that profile. For a mole-conserving ($\sum_i \nu_i = 0$) reaction the constant-molal-overflow flow profile is undisturbed; the worked methyl-acetate case is in the User Guide, §**??** of that document.

The pattern is universal: residuals F_i that all depend on all the unknowns x_j, and zero of them can be solved independently.

3.2 Generalising Taylor: the Jacobian matrix

We want to solve

$$\mathbf{F}(\mathbf{x}) \;=\; \mathbf{0}, \qquad \mathbf{F}\colon \mathbb{R}^n \to \mathbb{R}^n. \tag{168}$$

Each F_i is a function of all n variables $x_1, \dots, x_n$. Around an estimate $\mathbf{x}_k$, the analog of the scalar Taylor expansion $g(x^*) = g(x_k) + g'(x_k)(x^* - x_k) + \cdots$ is the *matrix-vector* Taylor expansion

$$\mathbf{0} \;=\; \mathbf{F}(\mathbf{x}^*) \;=\; \mathbf{F}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)\,(\mathbf{x}^* - \mathbf{x}_k) + O(|\mathbf{x}^* - \mathbf{x}_k|^2), \tag{169}$$

where the role of g' is now played by the *Jacobian matrix* $\mathbf{J} \in \mathbb{R}^{n\times n}$ whose entries are all the partial derivatives,

$$\mathbf{J}_{ij}(\mathbf{x}_k) \;\equiv\; \left.\frac{\partial F_i}{\partial x_j}\right|_{\mathbf{x}_k}. \tag{170}$$

Row i of $\mathbf{J}$ is the gradient of F_i; column j describes how all n residuals change when we move x_j alone.

Geometric reading. The map $\mathbf{x} \mapsto \mathbf{F}(\mathbf{x})$ sends $\mathbb{R}^n \to \mathbb{R}^n$; the Jacobian $\mathbf{J}(\mathbf{x}_k)$ is the *best linear approximation* of that map at $\mathbf{x}_k$. Equation (169) says that $\mathbf{F}$ is, to first order, $\mathbf{J}\cdot\Delta\mathbf{x}$ plus a constant shift.

3.3 The Newton step in n dimensions

Dropping the higher-order term in (169) and solving for $\mathbf{x}^*$ gives, formally, $\mathbf{x}^* \approx \mathbf{x}_k - \mathbf{J}^{-1}\mathbf{F}(\mathbf{x}_k)$. In practice we do *not* compute $\mathbf{J}^{-1}$; we solve the linear system

$$\boxed{\mathbf{J}(\mathbf{x}_k)\,\Delta\mathbf{x} \;=\; -\mathbf{F}(\mathbf{x}_k), \qquad \mathbf{x}_{k+1} \;=\; \mathbf{x}_k + \lambda\,\Delta\mathbf{x},} \tag{171}$$

where $\lambda \in (0, 1]$ is the damping factor (next subsection). Solving the system is both *cheaper* (one LU factorisation, $O(n^3/3)$ flops) and *numerically more stable* than forming the explicit inverse ($O(n^3)$ flops, more accumulated round-off, hard to keep accurate when $\mathbf{J}$ is poorly conditioned).

Intuition. The scalar Newton step $x_{k+1} = x_k - g(x_k)/g'(x_k)$ asks *"how far do I have to slide x to make the linear model of g hit zero?"*. The n-D version asks the same question, but the answer is a *direction vector* in $\mathbb{R}^n$ rather than a single scalar, and we find it by solving a linear system instead of doing a division. Setting

$n = 1$ in (171) recovers (165): $\mathbf{J}$ becomes the scalar g', and the "linear system" degenerates to one equation $g'(x)\Delta x = -g(x)$.

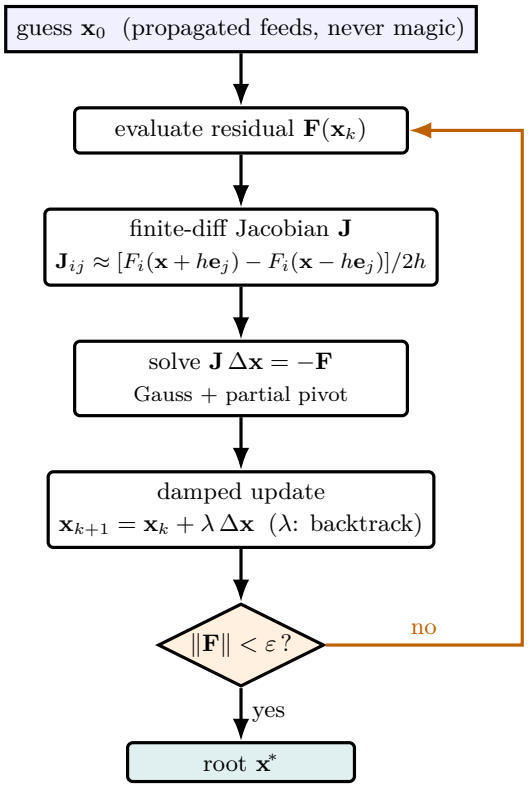

Figure 29: The n-dimensional Newton loop, exactly as `src/solver/NewtonND.cpp` runs it. Read it top to bottom: start from an honest guess (feeds propagated through the topology, never a universal constant), measure how wrong you are ($\mathbf{F}$), build the local linear model ($\mathbf{J}$, by finite differences so the engineer only codes $\mathbf{F}$), solve *one* linear system for the step direction $\Delta\mathbf{x}$, walk a damped fraction λ of it, and test convergence. If $\|\mathbf{F}\|$ is still too big, the orange arrow loops back and the whole pass repeats with the improved $\mathbf{x}$. The damping λ is the only knob that distinguishes "which direction" (the Jacobian's job) from "how far" (safe progress far from the root).

3.4 Finite-difference Jacobian

Analytic Jacobians for chemical models (NRTL γ derivatives, say, or distillation MESH equations) are tedious to derive and a maintenance burden once derived: every time the activity model is extended, the Jacobian code has to change too. The simulator opts for the universal approach of *finite differences* on the residuals. For each column j, perturb x_j by $\pm h$ keeping all other components fixed and compute

$$\mathbf{J}_{ij}(\mathbf{x}) \;\approx\; \frac{F_i(\mathbf{x} + h\,\mathbf{e}_j) - F_i(\mathbf{x} - h\,\mathbf{e}_j)}{2h}, \tag{172}$$

where $\mathbf{e}_j$ is the j-th standard basis vector. The step size is taken relative to the magnitude of x_j,

$$h \;=\; \max\!\big(\epsilon_{\text{rel}}\,|x_j|,\, \epsilon_{\text{abs}}\big), \qquad \epsilon_{\text{rel}} \approx 10^{-6}.$$

Cost: two extra residual evaluations per Jacobian column, so $2n$ per full Jacobian. Benefit: the engineer only has to implement $\mathbf{F}$, never $\mathbf{J}$ — decisive for pedagogical visibility and for adding new unit ops.

Common pitfall. Choosing h is a delicate compromise. Truncation error of the central-difference formula grows as h^2; round-off error grows as $\epsilon_{\text{machine}}/h$. For double precision ($\epsilon_{\text{machine}} \approx 10^{-16}$) the optimum sits near $h \sim \epsilon_{\text{machine}}^{1/3} \approx 10^{-5}$. Pick h too small (e.g. 10^{-12}) and the subtraction in the numerator of (172) loses most of its digits to cancellation.

3.5 Solving the linear system

The Jacobian is dense, square, and small ($n \lesssim 50$ in nearly every simulator unit op). Iterative methods (CG, GMRES) shine on large sparse systems; for our size they are slower than direct factorisation. The simulator uses Gauss elimination with partial pivoting on $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$ followed by back-substitution — the same algorithm taught in undergraduate linear algebra, implemented in ~ 60 LOC in `src/solver/NewtonND.cpp`.

Partial pivoting. At each elimination step, swap the

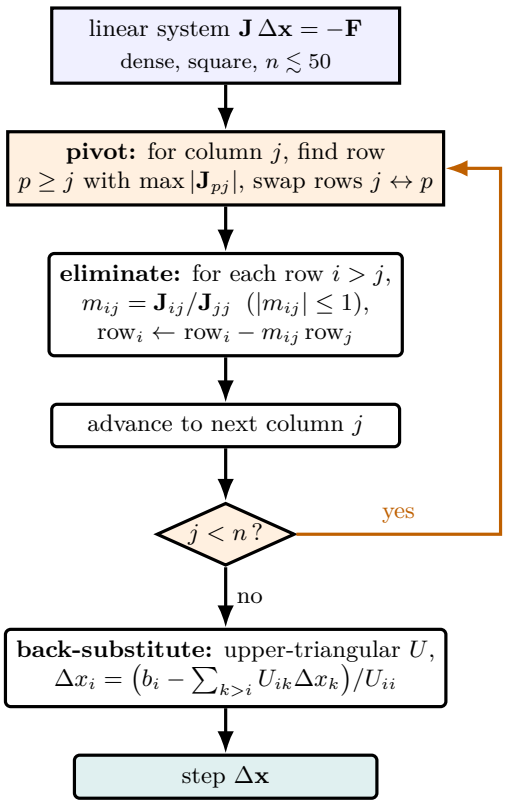

Figure 30: Gauss elimination with partial pivoting — the ∼60-line direct solver that every Newton step in the simulator calls to turn $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$ into a step. Column by column it first *pivots* (swaps in the row with the largest entry in the active column, the orange box) so the multipliers m_{ij} never exceed one in magnitude — the guard against round-off blow-up — then *eliminates* everything below the diagonal, leaving an upper-triangular system. When the last column is reached, one *back-substitution* sweep reads off $\Delta\mathbf{x}$ from the bottom up. No matrix inverse is ever formed: the factorisation is both cheaper ($O(n^3/3)$) and steadier than $\mathbf{J}^{-1}$.

current pivot row with whichever row below has the largest absolute entry in the pivot column. This keeps the multipliers ≤ 1 in magnitude and prevents catastrophic growth of round-off in the elimination. Without pivoting, a Jacobian with widely varying entries can produce numerically meaningless results.

3.6 Damping by backtracking line search

The full Newton step ($\lambda = 1$) is correct to second order in $\Delta\mathbf{x}$. Far from the root, second-order error can be huge and $\mathbf{x}_k + \Delta\mathbf{x}$ can land somewhere worse than $\mathbf{x}_k$ — in chemistry, "worse" often means a non-physical state (negative concentration, pressure below the saturation curve, temperature outside the Antoine range). The fix is the same as in 1-D (§2, Damping): try $\lambda = 1$; if $\|\mathbf{F}(\mathbf{x}_{k+1})\|_2^2$ does not decrease, halve λ and try again, down to a minimum $\lambda_{\min} \sim 2^{-10}$.

> *Intuition.* The Jacobian decides *which direction* to walk; damping decides *how far* to walk in that direction. When the linear model is trustworthy — close to the root, where higher-order terms are small — $\lambda \to 1$ and Newton recovers its quadratic-doubling speed. When the linear model is not trustworthy — far from the root, or in a region where $\mathbf{J}$ is nearly singular — λ shrinks until the step is short enough to stay in a safe basin.

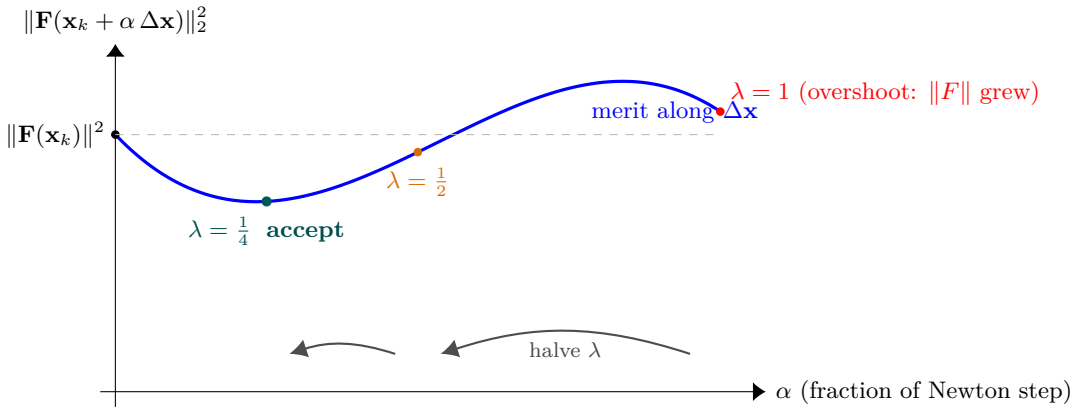

Figure 31: Backtracking damping seen as a one-dimensional slice of the residual norm *along* the fixed Newton direction $\Delta\mathbf{x}$. The Jacobian has already chosen the direction; damping only chooses how far. The full step ($\lambda = 1$, red) overshoots into a region where $\|\mathbf{F}\|^2$ is *larger* than at $\mathbf{x}_k$ — in chemistry often a non-physical state. The simulator halves λ ($1 \to \frac{1}{2} \to \frac{1}{4}$) until the trial point drops below the dashed current-residual line (the sufficient-decrease test), and accepts the first λ that does. Near the root the curve is monotone down from $\alpha = 0$, the first try $\lambda = 1$ passes, and quadratic convergence is recovered untouched.

3.7 Worked example: a 2×2 system

Worked example 3.1. Two coupled algebraic equationsSolve the system

$$F_1(\mathbf{x}) \;=\; x_1^2 + x_2^2 - 5 \;=\; 0, \qquad F_2(\mathbf{x}) \;=\; x_1 x_2 - 2 \;=\; 0.$$

The pair has solutions $(2, 1)$ and $(1, 2)$ (and the two reflections through the origin). We seek $(2, 1)$ from the initial guess $\mathbf{x}_0 = (2.5, 0.5)$.

Step 1. Evaluate $\mathbf{F}$ and $\mathbf{J}$:

$$\mathbf{F}(\mathbf{x}_0) = \begin{pmatrix} 2.5^2 + 0.5^2 - 5 \\ 2.5 \cdot 0.5 - 2 \end{pmatrix} = \begin{pmatrix} 1.50 \\ -0.75 \end{pmatrix}, \quad \mathbf{J}(\mathbf{x}_0) = \begin{pmatrix} 2x_1 & 2x_2 \\ x_2 & x_1 \end{pmatrix}\Big|_{\mathbf{x}_0} = \begin{pmatrix} 5.0 & 1.0 \\ 0.5 & 2.5 \end{pmatrix}.$$

Solve $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$:

$$\begin{pmatrix} 5.0 & 1.0 \\ 0.5 & 2.5 \end{pmatrix} \begin{pmatrix} \Delta x_1 \\ \Delta x_2 \end{pmatrix} \;=\; \begin{pmatrix} -1.50 \\ 0.75 \end{pmatrix}.$$

Gauss elimination: pivot row 1, multiplier $0.5/5.0 = 0.1$. Subtract 0.1 times row 1 from row 2: row 2 becomes $(0, 2.4 \mid 0.90)$. Back-substitute: $\Delta x_2 = 0.375$, $\Delta x_1 = (-1.50 - 1.0 \cdot 0.375)/5.0 = -0.375$. So $\mathbf{x}_1 = (2.5, 0.5) + (-0.375, +0.375) = (2.125, 0.875)$.

Step 2. $\mathbf{F}(\mathbf{x}_1) = (0.281, -0.141)$, with $\|\mathbf{F}\|_2 = 0.314$ (down from 1.677); take another Newton step. After it, $\mathbf{x}_2 \approx (2.012, 0.988)$, $\|\mathbf{F}\|_2 \approx 0.025$. Two more steps recover machine precision. The error is squaring roughly each iteration — the expected quadratic convergence of Newton near the root.

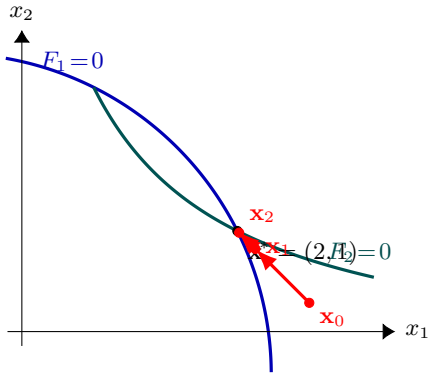

Figure 32: The two Newton steps of the worked 2×2 example, drawn in the (x_1, x_2) plane. The two residual equations each define a zero-curve — a circle ($F_1 = x_1^2 + x_2^2 - 5 = 0$, blue) and a hyperbola ($F_2 = x_1 x_2 - 2 = 0$, teal) — and the root is where they cross. Each Newton step solves $\mathbf{J}\,\Delta\mathbf{x} = -\mathbf{F}$ for a step *direction*, and the iterates $\mathbf{x}_0 \to \mathbf{x}_1 \to \mathbf{x}_2$ march straight at the intersection, the gap to $\mathbf{x}^*$ roughly squaring every step (quadratic convergence). The same picture in n dimensions is a path on the intersection of n residual hypersurfaces — only the drawing gets harder, never the algorithm.

4 Parameter estimation: Levenberg-Marquardt

What you'll learn. How the simulator turns a table of experimental bubble-temperature data into a fitted set of NRTL (or Wilson) interaction parameters. We will derive each piece from scratch: why *least squares*; why the gradient of χ^2 is exactly $\mathbf{J}^\top\mathbf{r}$; why its Hessian splits into two terms and why one of them can be dropped; how the resulting Gauss-Newton step can overshoot; Levenberg's damping fix; Marquardt's invariance fix; and the simple trust-region rule the simulator uses to update the damping. The runnable case is `tutorials/fitNRTL01_ethanol_water/`.

4.1 A different question

Every solver in Part III so far has had the same shape: *given a model, find the inputs that satisfy a residual.* Flash, bubble-T, CSTR, distillation — all instances of "solve $\mathbf{F}(\mathbf{x}) = 0$".

Parameter estimation flips the question. We have:

- a *model with adjustable knobs* — here, NRTL with its four binary-interaction parameters $a_{ij}, b_{ij}, a_{ji}, b_{ji}$ (the α_{ij} non-randomness is held fixed by convention);
- a *table of experimental data* that the model should reproduce — here, N pairs $(x_i^{\text{exp}}, T_i^{\text{exp}})$ at fixed pressure P;
- and we want to find the knob setting that makes the model agree with the data as closely as possible.

This is an *over-determined* problem: $N \gg m$, so we cannot satisfy every data point exactly, only minimise the total mismatch.

4.2 Problem statement

Let $\mathbf{p} \in \mathbb{R}^m$ be the parameter vector (for an NRTL binary, $m = 4$: $a_{ij}, b_{ij}, a_{ji}, b_{ji}$). The simulator provides a *forward model*: for each experimental composition x_i^{exp}, run a bubble-T calculation with the current $\mathbf{p}$ and return the predicted temperature,

$$T_i^{\text{pred}}(\mathbf{p}) \;=\; \text{bubble-}T \text{ at } (x_i^{\text{exp}},\, 1 - x_i^{\text{exp}},\, P) \text{ with NRTL}(\mathbf{p}). \tag{173}$$

The **residual** at point i is the prediction error,

$$r_i(\mathbf{p}) \;\equiv\; T_i^{\text{pred}}(\mathbf{p}) - T_i^{\text{exp}}, \tag{174}$$

and we minimise the sum of squared residuals,

$$\boxed{\chi^2(\mathbf{p}) \;\equiv\; \tfrac{1}{2}\sum_{i=1}^{N} r_i(\mathbf{p})^2 \;=\; \tfrac{1}{2}\,\mathbf{r}(\mathbf{p})^\top\mathbf{r}(\mathbf{p}).} \tag{175}$$

Intuition. Why squared, not absolute value or fourth power? Squaring is the unique choice that is everywhere differentiable, penalises large mismatches disproportionately (so a single outlier still pulls the fit), and is the maximum-likelihood estimator when the experimental errors are independent Gaussians of equal variance. The factor $\tfrac{1}{2}$ is bookkeeping — it cancels the 2 that drops out of $\frac{d}{dp_j}(r_i^2) = 2\,r_i\,\partial r_i/\partial p_j$ in the gradient.

4.3 The gradient of χ^2: deriving $\mathbf{J}^\top\mathbf{r}$

At a minimum, every partial derivative of χ^2 vanishes. Let us compute $\partial\chi^2/\partial p_j$ explicitly. Using the chain rule on Eq. (175),

$$\frac{\partial\chi^2}{\partial p_j} \;=\; \frac{\partial}{\partial p_j}\Big[\tfrac{1}{2}\sum_{i=1}^{N} r_i^2\Big] \;=\; \tfrac{1}{2}\sum_{i=1}^{N} 2\,r_i\,\frac{\partial r_i}{\partial p_j} \;=\; \sum_{i=1}^{N} r_i\,\frac{\partial r_i}{\partial p_j}. \tag{176}$$

Now collect the partial derivatives of the residual into the **residual Jacobian**

$$\mathbf{J}_{ij}(\mathbf{p}) \;\equiv\; \frac{\partial r_i}{\partial p_j}, \qquad \mathbf{J} \in \mathbb{R}^{N\times m}, \tag{177}$$

so that the j-th component of the gradient is exactly $\sum_i r_i\mathbf{J}_{ij} = (\mathbf{J}^\top\mathbf{r})_j$. Stacking all m partials:

$$\boxed{\nabla_{\mathbf{p}}\chi^2(\mathbf{p}) \;=\; \mathbf{J}(\mathbf{p})^\top\,\mathbf{r}(\mathbf{p}).} \tag{178}$$

A minimum therefore satisfies $\mathbf{J}^\top\mathbf{r} = 0$, the *normal-equation condition*. This is the same shape of system we have already met: m equations, m unknowns. The natural thing to do is run a Newton iteration on it.

4.4 The Hessian splits in two: keep one piece, drop the other

To apply Newton-Raphson to $\nabla\chi^2 = 0$ we need the Hessian $\nabla^2\chi^2 \in \mathbb{R}^{m\times m}$. Differentiate $(\mathbf{J}^\top \mathbf{r})_j = \sum_i r_i\, \partial r_i/\partial p_j$ once more, with respect to p_k:

$$\frac{\partial^2 \chi^2}{\partial p_j \partial p_k} \;=\; \sum_{i=1}^{N} \frac{\partial r_i}{\partial p_k}\,\frac{\partial r_i}{\partial p_j} \;+\; \sum_{i=1}^{N} r_i\,\frac{\partial^2 r_i}{\partial p_j \partial p_k}. \tag{179}$$

In matrix form,

$$\nabla^2\chi^2 \;=\; \underbrace{\mathbf{J}^\top\mathbf{J}}_{\text{first-derivative information}} \;+\; \underbrace{\sum_{i=1}^{N} r_i\,\nabla^2 r_i}_{\text{second-derivative information}}. \tag{180}$$

The first term costs us nothing extra — we already have $\mathbf{J}$. The second term requires the $m \times m$ Hessian of every single residual: N matrices of m^2 second partials, each obtained from forward model calls. Forming it by finite differences would multiply the cost of one LM iteration by roughly a factor of m.

The *Gauss-Newton approximation* is to drop the second term:

$$\nabla^2\chi^2 \;\approx\; \mathbf{J}^\top\mathbf{J}. \tag{181}$$

> *Intuition.* Why is dropping $\sum_i r_i\nabla^2 r_i$ defensible? Look at the sum: it is weighted by the residuals themselves. At the optimum $\mathbf{p}^*$ the residuals are small (the whole point of the fit), so the dropped term is small *compared to* $\mathbf{J}^\top\mathbf{J}$ (which is a sum of squares and grows with N). Near $\mathbf{p}^*$, r_i errors of ~ 0.4 K with a $\mathbf{J}$ whose columns have norm ~ 1 make the dropped term roughly $100\times$ smaller than the kept one. Far from $\mathbf{p}^*$ the approximation can mislead — and that is exactly where Levenberg's damping (next subsection) saves us.

With the Gauss-Newton Hessian, Newton-Raphson on $\nabla\chi^2 = 0$ becomes

$$\left(\mathbf{J}^\top\mathbf{J}\right)\Delta\mathbf{p} \;=\; -\mathbf{J}^\top\mathbf{r}, \qquad \mathbf{p}_{k+1} \;=\; \mathbf{p}_k + \Delta\mathbf{p}. \tag{182}$$

This $m \times m$ linear system is the **normal equations** of the linear least-squares problem $\min_{\Delta\mathbf{p}} \|\mathbf{r} + \mathbf{J}\,\Delta\mathbf{p}\|^2$. You can read it either way: as a Newton step on $\nabla\chi^2 = 0$ (the derivation above), or as the closed-form minimiser of the linearised residual $\mathbf{r}(\mathbf{p}+\Delta\mathbf{p}) \approx \mathbf{r} + \mathbf{J}\,\Delta\mathbf{p}$.

> **Common pitfall.** Gauss-Newton is locally a descent direction for χ^2, but only *locally*. If the linearisation $\mathbf{r}(\mathbf{p}+\Delta\mathbf{p}) \approx \mathbf{r} + \mathbf{J}\,\Delta\mathbf{p}$ is poor — because we are far from $\mathbf{p}^*$, or some r_i are large, or the residual surface curves sharply — the full Gauss-Newton step can overshoot wildly and *increase* χ^2. You will see this in the simulator's LM trace as iterations where the trial χ^2 jumps by orders of magnitude before being rolled back.

4.5 Levenberg's idea: a knob between two extremes

We have two algorithms in hand to minimise χ^2, with opposite failure modes:

- **Gauss-Newton**, Eq. (182), takes *long* steps in the direction the linearised model prefers. Near $\mathbf{p}^*$ it converges quadratically. Far from $\mathbf{p}^*$ it overshoots.
- **Steepest descent**, $\Delta\mathbf{p} = -\alpha\,\nabla\chi^2 = -\alpha\,\mathbf{J}^\top\mathbf{r}$, takes *short* steps in the direction χ^2 decreases fastest. It provably reduces χ^2 for a small enough α. But near $\mathbf{p}^*$ it zigzags through narrow valleys and crawls.

Levenberg [59] proposed to interpolate between the two by adding a tunable damping term to the normal-equation matrix:

$$\left(\mathbf{J}^\top\mathbf{J} + \lambda\,\mathbf{I}\right)\Delta\mathbf{p} \;=\; -\mathbf{J}^\top\mathbf{r}. \tag{183}$$

The two limits of the λ-knob are illuminating:

- $\lambda \to 0$: the damping vanishes and the system reduces to Eq. (182) — pure Gauss-Newton, with local quadratic convergence.
- $\lambda \to \infty$: the damping term dominates and $\Delta\mathbf{p} \approx -\lambda^{-1}\,\mathbf{J}^\top\mathbf{r} = -\lambda^{-1}\,\nabla\chi^2$ — an arbitrarily short step along the steepest-descent direction. Linear convergence, but *guaranteed* to decrease χ^2 if the step is short enough.

Notice the dual role λ plays. Increasing it does not only *rotate* the step toward the gradient: it also *shrinks* the step, because the diagonal of $\mathbf{J}^\top\mathbf{J} + \lambda\mathbf{I}$ grows and divides $\Delta\mathbf{p}$ down. That is why λ doubles as an implicit trust-region radius: big λ means "I do not trust the linear model out to a long step, take a short careful one."

4.6 Marquardt's invariance fix

Levenberg's $\lambda\,\mathbf{I}$ has a subtle defect: it is not invariant under a diagonal rescaling of the parameters. Suppose we re-expressed every p_j in units that doubled it: $p_j \to 2p_j$. The columns of $\mathbf{J}$ would halve, $\mathbf{J}^\top\mathbf{J}$ would scale by $1/4$

along its diagonal, but $\lambda\mathbf{I}$ would not change. A λ that was small compared to one diagonal entry of $\mathbf{J}^\top\mathbf{J}$ might now dominate it.

For NRTL parameters this matters in practice: a_{ij} is a dimensionless $\mathcal{O}(1)$ number, but b_{ij} carries units of K and runs $\mathcal{O}(100\text{–}1000)$. A single λ that damps the b direction reasonably will completely lock the a direction (or vice versa).

Marquardt [60] fixed this by scaling the damping with the diagonal of $\mathbf{J}^\top\mathbf{J}$ itself:

$$\boxed{\left(\mathbf{J}^\top\mathbf{J} + \lambda\ \mathrm{diag}(\mathbf{J}^\top\mathbf{J})\right)\Delta\mathbf{p} \;=\; -\mathbf{J}^\top\mathbf{r}.} \tag{184}$$

This is invariant under diagonal reparametrisation: doubling all b_{ij} shrinks both terms in the matrix by the same factor, so their relative weighting is preserved.

The simulator implements Eq. (184) in the *multiplicative* form that is common in practice: each diagonal entry of $\mathbf{J}^\top\mathbf{J}$ is multiplied by $1+\lambda$. That is the same equation with the λ knob shifted by one — $\lambda = 0$ in the multiplicative form is pure Gauss-Newton; $\lambda \to \infty$ is pure scaled gradient descent.

> *Intuition.* The diagonal entry $(\mathbf{J}^\top\mathbf{J})_{jj} = \sum_i (\partial r_i/\partial p_j)^2$ is the local curvature of χ^2 along the p_j axis — the second derivative of χ^2 in direction j, in the Gauss-Newton approximation. Damping in proportion to this curvature means: take small steps in well-determined (stiff) parameter directions, take longer steps in poorly-determined (slack) ones. Without the scaling, one fixed λ ends up calibrated for the stiffest direction and is needlessly conservative everywhere else — which is what makes plain Levenberg's convergence so unsteady on real problems.

4.7 The trust-region update: 3 down, 5 up

The damping λ is not fixed — it is adjusted every iteration based on whether the proposed step *actually* decreased χ^2. The logic is simple:

- If $\chi^2(\mathbf{p}_k + \Delta\mathbf{p}) < \chi^2(\mathbf{p}_k)$, the linearisation was good enough. *Accept* the step ($\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta\mathbf{p}$) and *shrink* λ to push toward Gauss-Newton next time. The simulator uses $\lambda \leftarrow \lambda/3$.
- Otherwise the linearisation overshot. *Reject* the step ($\mathbf{p}_{k+1} = \mathbf{p}_k$) and *grow* λ to take a shorter, more gradient-aligned step next time. The simulator uses $\lambda \leftarrow 5\,\lambda$.

> *Intuition.* Read λ as a trust-region radius: small λ = "I trust the linear model out to a long step"; large λ = "I do not, keep close to the current $\mathbf{p}$." Each LM iteration is one experiment that tests how far the linear model can be trusted; the trust radius shrinks or grows based on the experimental outcome. This is the single algorithmic ingredient that distinguishes LM from plain Gauss-Newton — and it is the reason LM converges on problems where Gauss-Newton diverges.

The 3-down/5-up asymmetry is conservative on purpose: a rejected step has already cost a full Jacobian rebuild ($2m$ extra forward solves), so we want to avoid the *next* rejection more aggressively than we want to chase Gauss-Newton speed. The exact factors are tunable; 3 and 5 follow Press *et al.* [65] with one click more caution.

4.8 Finite-difference Jacobian

The model $T_i^{\mathrm{pred}}(\mathbf{p})$ is a black-box solver call, not a closed-form expression. We compute $\mathbf{J}$ by central differences in $\mathbf{p}$:

$$\mathbf{J}_{ij}(\mathbf{p}) \;\approx\; \frac{r_i(\mathbf{p} + h_j\,\mathbf{e}_j) - r_i(\mathbf{p} - h_j\,\mathbf{e}_j)}{2\,h_j}, \tag{185}$$

with a relative step

$$h_j \;=\; \epsilon_{\mathrm{rel}}\ \max(1, |p_j|), \qquad \epsilon_{\mathrm{rel}} \;=\; 10^{-4}. \tag{186}$$

Each column of $\mathbf{J}$ costs 2 forward solves of the simulator. For the canonical ethanol-water case ($m = 4$ parameters, $N = 11$ data points), one LM iteration is therefore $1 + 2\,m = 9$ vector forward evaluations, each of which loops over N bubble-T calls.

> **Common pitfall.** The step size h_j has two error sources working against each other: **truncation error** (the second derivative we ignore in the FD formula) grows as h^2; **round-off error** grows as $\epsilon_{\mathrm{machine}}/h$ for an arithmetic resembling double precision. The optimum is near $h \sim \epsilon_{\mathrm{machine}}^{1/3} \approx 10^{-5}$ for central differences. The simulator's 10^{-4} is a shade larger — chosen because the inner bubble-T solve is itself accurate to about 10^{-8} K, so a smaller h would not buy more accuracy and would amplify the inner solver's noise.

4.9 Stopping criteria

The LM loop exits successfully when the relative improvement in χ^2 falls below `tolerance` (default 10^{-6}):

$$\frac{\chi^2_{k-1} - \chi^2_k}{\chi^2_{k-1}} < \texttt{tolerance}. \tag{187}$$

It exits unsuccessfully on either `maxIter` iterations (default 30) or a damping blow-up $\lambda > 10^{10}$, which signals that the trust region has collapsed — the Hessian approximation is indefinite or the residual surface is too non-smooth for the FD Jacobian. In practice the second failure mode shows up when the dataset contains near-duplicate x_i values or when the initial guess is so far from $\mathbf{p}^*$ that the inner bubble-T Newton itself fails to converge.

4.10 The simulator's implementation

The operation is registered as `type fitParameters;` in `system/propsDict` (its choupoSolve-era predecessor `fitBinaryPair` is retired — the factory throws, naming the replacement). Its main loop lives in `src/propertyOps/FitParameters.cpp`, which calls into:

- `Dictionary::setScalarAtPath()` to overwrite the four NRTL parameters in `thermoDict_` for each forward evaluation;
- the existing `BubblePoint` unit-op (which emits `T_bubble` as a KPI from onwards) for the inner bubble-T;
- `solver::gaussSolve` (the same Gauss elimination used by `NewtonND`, just on the 4×4 normal-equations system).

No external library, no auto-differentiation, no hand-coded analytical Jacobian: a deliberate constraint for pedagogical visibility.

Runnable case. `runCase tutorials/fitNRTL01_ethanol_water` produces the full LM trace, the per-point residual breakdown, and writes the proposal file `constant/binaryPairs/NRTL/ethanol-water.fit-<date>.dat`. See the case's `README.md` for the promote-by-`mv` step.

Worked example 4.1. Reading the LM trace for ethanol-water at 1 atmThe simulator prints one row per iteration:

```
  iter     chi^2          RMS(K)     lambda    step
  ----  ------------  ----------  --------  --------
  init  6.0195e+01    2.3393e+00  1.0e-03               (DECHEMA literature)
  1     54.0505       2.2167      1.0e-03   523.9       (accepted, lambda /= 3)
  2     17990.5239    40.4413     3.3e-04   0.0009      (REJECTED, lambda *= 5)
  3     6020.4407     23.3947     1.7e-03   0.0009      (REJECTED)
...
  18    1.8006        0.4046      4.5e+00   6.8971      (accepted)
  19    6109.9721     23.5680     1.5e+00   0.0000      (REJECTED)
  20    1.8006        0.4046      7.4e+00   0.0000      (converged: rel<tol)
```

Read row by row:

- **Initial $\chi^2 = 60.2$, RMS $= 2.34$ K.** With the literature DECHEMA parameters, the model overpredicts T_b by about 2 K on average across this dataset.
- **Iter 1 ($\lambda = 10^{-3}$):** small damping, pure Gauss-Newton step. χ^2 drops marginally; step norm is large (~ 524) — the gradient was steep.
- **Iter 2 ($\lambda = 3.3 \times 10^{-4}$):** the previous step shrank λ, so the next is even more Gauss-Newton-ish. But the linearisation no longer holds at this displaced point; χ^2 rockets to 1.8×10^4. *Rejected*; λ jumps to 1.7×10^{-3}.
- **Iters 3–17:** the loop oscillates between accept (with λ decreasing 3-fold) and reject (with λ increasing 5-fold). χ^2 drifts down on the accept rows. The `step` column shrinks as the active subspace of $\mathbf{p}$ narrows.
- **Iter 18:** an accepted step that drops χ^2 to 1.80, RMS = 0.40 K. Five-fold improvement over literature for THIS dataset.
- **Iter 19:** a final overreach, rejected.
- **Iter 20:** same χ^2 as iter 18, but with the relative improvement now $< 10^{-6}$. Converged.

The lesson visible in the trace: the trust region tightens the more non-linear the residual surface is around the current $\mathbf{p}$, and relaxes when the linearisation holds. That dance is exactly what distinguishes LM from plain Gauss-Newton — the latter would have diverged at iteration 2.

Intuition. Levenberg-Marquardt is best understood not as a single algorithm but as a continuously-tuned *mixture* of two limits: Gauss-Newton (fast but fragile) and gradient descent (slow but bullet-proof). The λ knob slides between them automatically based on whether the last step worked. The Marquardt diagonal scaling makes the knob behave the same regardless of how you re-express the parameters.

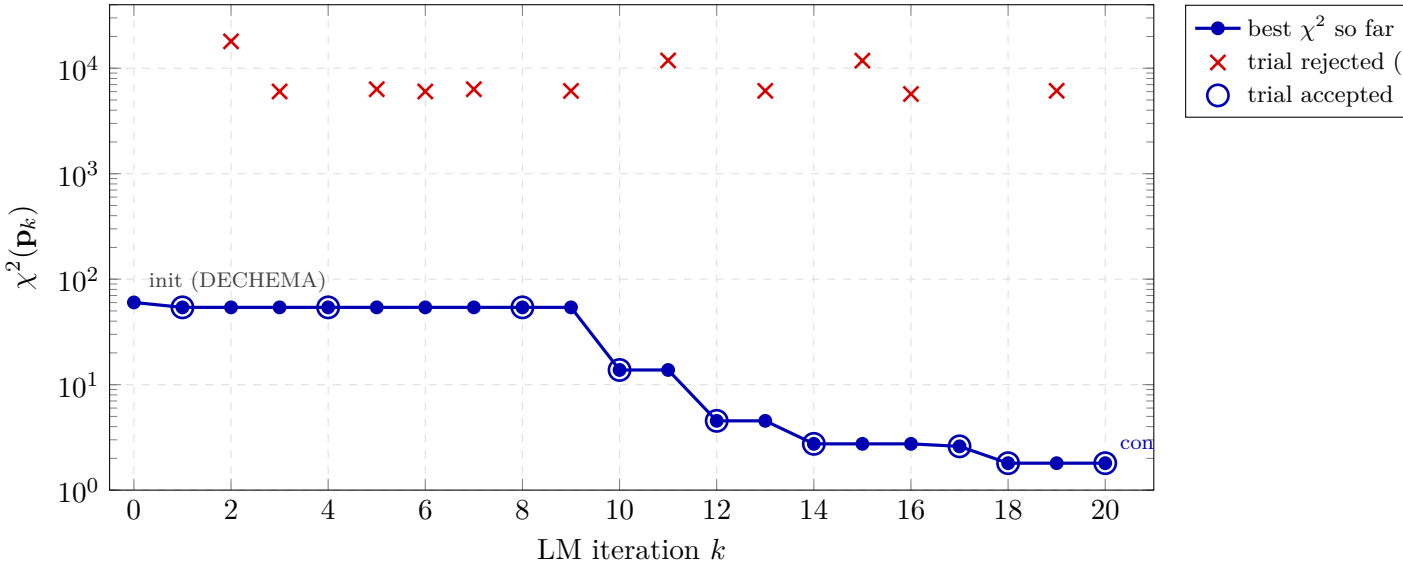

Figure 33: Levenberg-Marquardt χ^2 history for the ethanol/water NRTL fit in `tutorials/fitNRTL01_ethanol_water` (real run, all 20 iterations). The solid blue line tracks the best χ^2 achieved so far — the parameters that are carried forward. The red crosses are trial χ^2 values the LM proposed but *rejected* because they were larger than the best, each triggering $\lambda \leftarrow 5\lambda$. The blue circles are the accepted trials, each triggering $\lambda \leftarrow \lambda/3$. The 3-up/5-down trust-region update keeps the accepted sequence monotone even though per-iteration trials oscillate over three orders of magnitude.

5 Wegstein acceleration

What you'll learn. *Fixed-point* problems — equations of the form $x = f(x)$ — are at the heart of every simulator's outer loops. The naive solution method, called *direct substitution*, can be glacially slow. This chapter derives Wegstein's acceleration from first principles, step by step: from the slowness of direct substitution, to the idea of approximating f as linear, to the closed-form formula that the simulator uses for its activity-coefficient loop.

5.1 The setting: fixed-point problems

A *fixed-point problem* is the search for a value x^* that satisfies

$$x^* \;=\; f(x^*), \tag{188}$$

where $f\colon \mathbb{R} \to \mathbb{R}$ is some function we can evaluate. Geometrically, x^* is the value where the curve $y = f(x)$ crosses the diagonal $y = x$. In a process simulator a typical f takes a current estimate of the activity coefficients and returns an updated estimate (we'll get to the details in §9 and §2); what matters here is the abstract problem.

5.2 Direct substitution and why it can be slow

The most obvious way to solve $x = f(x)$ is to start somewhere, plug in, and repeat:

$$x_{k+1} \;=\; f(x_k), \qquad k = 0, 1, 2, \ldots \tag{189}$$

This is *direct substitution*. When it works, it works because each step shrinks the error. Let $e_k \equiv x_k - x^*$. Linearising f around x^*,

$$f(x_k) \;=\; f(x^*) + f'(x^*)\, e_k + O(e_k^2) \;=\; x^* + f'(x^*)\, e_k + O(e_k^2),$$

so

$$e_{k+1} \;=\; x_{k+1} - x^* \;=\; f'(x^*)\, e_k + O(e_k^2). \tag{190}$$

The error shrinks by the factor $|f'(x^*)|$ each step. If this *contraction factor* is 0.1, ten iterations give six digits; if it is 0.9, ten iterations give roughly half a digit.

Worked example 5.1. Direct substitution on a slow problemConsider $f(x) = 0.9\,x + 0.5$, with fixed point $x^* = 5$ (you can verify by substitution). Here $f'(x) = 0.9$ everywhere, so the contraction factor is exactly 0.9. Starting at $x_0 = 0$:

$$x_0 = 0,\;\; x_1 = 0.5,\;\; x_2 = 0.95,\;\; x_3 = 1.36,\;\; x_4 = 1.72,\;\; \ldots,\;\; x_{20} \approx 4.39,\;\; x_{30} \approx 4.79.$$

After thirty iterations we are still 0.21 away from the answer. This kind of slow contraction is exactly what happens to the activity-coefficient loop near an azeotrope.

5.3 The Wegstein idea: model f as a straight line

Wegstein's observation [54] is this: *if we already have two iterates x_{k-1} and x_k, we know not just one point on the curve $y = f(x)$ but* two. *Two points define a line. We can use that line as a local linear approximation to f, and solve the linear problem $x =$ (line) exactly in one step.*

The model. Approximate the curve $y = f(x)$ by the secant line through $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$. Its slope is

$$s_k \;\equiv\; \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}. \tag{191}$$

The line itself can be written, point-slope form anchored at $(x_k, f(x_k))$, as

$$y \;=\; f(x_k) + s_k\,(x - x_k).$$

The next iterate. Treat this line as if it *were* the true f and solve the linear fixed-point equation $y = x$:

$$x \;=\; f(x_k) + s_k\,(x - x_k).$$

Collect x-terms on the left,

$$x\,(1 - s_k) \;=\; f(x_k) - s_k\,x_k,$$

and divide by $(1 - s_k)$:

$$x_{k+1} \;=\; \frac{f(x_k) - s_k\,x_k}{1 - s_k}. \tag{192}$$

Rearranging into Wegstein's form. Equation (192) is the answer, but it is rarely written this way. Define a single bookkeeping coefficient

$$q_k \equiv \frac{s_k}{s_k - 1} \tag{193}$$

(so $1 - q_k = -1/(s_k - 1) = 1/(1 - s_k)$, and $1 - q_k = 1 + 1/(1 - s_k)$ when written symmetrically — the algebra is in the next paragraph). Multiplying numerator and denominator of (192) by -1 and using q_k:

$$x_{k+1} = \frac{s_k\, x_k - f(x_k)}{s_k - 1} = \frac{s_k}{s_k - 1}\, x_k - \frac{1}{s_k - 1}\, f(x_k) = q_k\, x_k + (1 - q_k)\, f(x_k),$$

because $-1/(s_k - 1) = 1 - s_k/(s_k - 1) = 1 - q_k$. This is the Wegstein step:

$$\boxed{x_{k+1} = q_k\, x_k + (1 - q_k)\, f(x_k), \quad \text{with} \quad q_k = \frac{s_k}{s_k - 1}, \quad s_k = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.} \tag{194}$$

Reading the formula. The Wegstein step is a *linear combination* of the current iterate x_k and the direct-substitution next iterate $f(x_k)$. The coefficient q_k controls the mix. Three limiting cases give the intuition:

- $s_k = 0$ (the function is locally flat): $q_k = 0$. The step becomes $x_{k+1} = f(x_k)$ — ordinary direct substitution. Makes sense: a flat f has nothing to accelerate.
- $s_k \to 1^-$ (the contraction is becoming slow: $f'(x^*)$ near 1): q_k becomes a large negative number, $1 - q_k$ a large positive one. The step throws the iterate far past $f(x_k)$ in the same direction. This is the whole point: direct substitution would take many steps to cover that distance; Wegstein takes one.
- $s_k > 1$: the secant slope is steeper than the diagonal, so the linear model is not a contraction at all. The formula gives a positive q_k that can be arbitrarily large, throwing the iterate in the *wrong* direction. We will fix this in §5.5.

> *Intuition.* Direct substitution says "my next guess is wherever f took me". Wegstein says "f is approximately linear locally; let me solve the linear problem directly and jump straight to the answer". When f really is linear, Wegstein hits x^* in one step (the secant is the curve itself). When f is non-linear, the secant is only a chord; we land near x^*, take a new secant from the latest two iterates, and try again. Each round the secant is closer to the local tangent at x^*, so convergence accelerates as we close in. This is *super-linear* convergence: the per-step error reduction itself improves over time, unlike direct substitution's fixed contraction factor.

5.4 A worked iteration

Take the slow problem from the earlier example: $f(x) = 0.9\,x + 0.5$, $x^* = 5$. Start from two direct-substitution iterates $x_0 = 0$ and $x_1 = f(x_0) = 0.5$. Then:

$$\begin{aligned} s_1 &= \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{0.95 - 0.5}{0.5 - 0} = 0.9, \\ q_1 &= \frac{s_1}{s_1 - 1} = \frac{0.9}{-0.1} = -9, \\ x_2 &= q_1\, x_1 + (1 - q_1)\, f(x_1) = -9 \cdot 0.5 + 10 \cdot 0.95 = -4.5 + 9.5 = 5.0. \end{aligned}$$

One Wegstein step reaches the fixed point exactly — whereas direct substitution needed thirty iterations to get within 0.21. The miracle is that f is linear, so Wegstein's linear model is *exact*, and the linear problem is solved in closed form.

5.5 When the linear model misleads: the q-clip

The miracle case (f linear) is rare; the typical case is f non-linear and the secant slope s_k a noisy approximation to $f'(x^*)$. Two pathologies show up.

Pathology 1: the wrong contraction direction ($s_k > 1$). If the secant slope is greater than 1, the linear model is not a contraction. Geometrically, the secant line is steeper than the diagonal $y = x$ and intersects it on the *far side* of both data points — the extrapolation lands somewhere unrelated to the actual fixed point. Plug $s_k > 1$ into (193): $q_k = s_k/(s_k - 1)$ is positive and possibly large, and the formula sends the iterate in the wrong direction.

Pathology 2: over-extrapolation. Even with $s_k < 1$ (genuine contraction), if s_k is very close to 1 the denominator $s_k - 1$ is tiny and q_k becomes a huge negative number. The Wegstein step then throws the iterate *far* past the secant intercept — correct in direction but wildly overshooting. This is fine when f really is approximately linear over a wide range; it is destabilising when it is not.

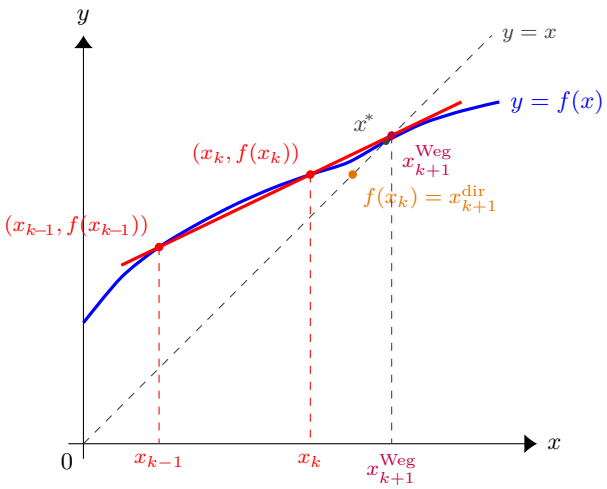

Figure 34: Geometric construction of one Wegstein step. Two prior iterates lie on the curve $y=f(x)$ as $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$. Their *secant line* (red) approximates f locally as an implicit linear model; the next iterate x^{Weg}_{k+1} is where this secant crosses the fixed-point diagonal $y=x$. Direct substitution would jump only as far as $f(x_k)$ along the diagonal (orange) — visibly short of x^* — while the secant extrapolation lands almost exactly on the fixed point. For a *linear* f the secant coincides with the curve and Wegstein hits x^* in one step; for the non-linear f shown, the secant is a chord and one step gets within $\sim$2% of the root.

The fix: clip q_k to a safe interval. The simulator forces

$$q_k \in [\,q_{\min},\, q_{\max}\,], \qquad \text{typical defaults: } q_{\min} = -5,\ \ q_{\max} = 0. \tag{195}$$

These bounds carry concrete meaning:

- $q_{\max} = 0$ disables wrong-direction steps. With $q = 0$ Wegstein degrades to direct substitution ($x_{k+1} = f(x_k)$) — slow but never wrong.
- $q_{\min} = -5$ caps the maximum acceleration to a factor of 6: the step travels at most six times further than direct substitution would. When the secant flattens ($s_k \to 1^-$), this is what keeps the iterate from escaping the basin of attraction.

> *Intuition.* The clip turns Wegstein into a kind of adaptive method that uses the secant when it is informative (modest s_k, $-5 \le q_k < 0$) and falls back to safe direct substitution when it is not ($s_k > 1 \Rightarrow q_k \to 0$). The student should picture q_k as a knob between two regimes, with $q_{\min}$ and $q_{\max}$ as safety limits chosen empirically across decades of process simulation.

5.6 Where this matters in the simulator: the γ-loop

Wegstein is the workhorse of the simulator's activity-coefficient outer loop. After a Rachford-Rice solve, the calculated compositions $\mathbf{x}^*$ depend on the assumed γ-values; we recompute $\gamma_i(\mathbf{x}^*, T)$, get new K-values, redo Rachford-Rice, and so on. Symbolically the structure is

$$\gamma^{(k+1)} \;=\; \text{Wegstein}\big(\gamma^{(k)},\, f(\gamma^{(k)})\big), \qquad f(\gamma) \;\equiv\; \gamma_i\big(\mathbf{x}^*(\gamma),\, T\big),$$

i.e. f is the composition of the Rachford-Rice solver with a re-evaluation of the activity model. Wegstein is applied *component-wise*: N_c independent scalar accelerations, one per chemical species, not a single vector norm and not the residual. This component-wise design is important because different components in a non-ideal mixture can have very different local contraction factors — a single global q_k would slow the fast ones to match the slow ones.

In simulation runs near the ethanol/water azeotrope, where the ideal-mixing contraction factor is close to 1, Wegstein routinely converges in ~ 7 outer iterations against the ~ 29 that direct substitution would need on the same case. That four-fold speed-up across thousands of flash calculations per simulator pass is what makes a non-ideal flowsheet practical to run on a laptop.

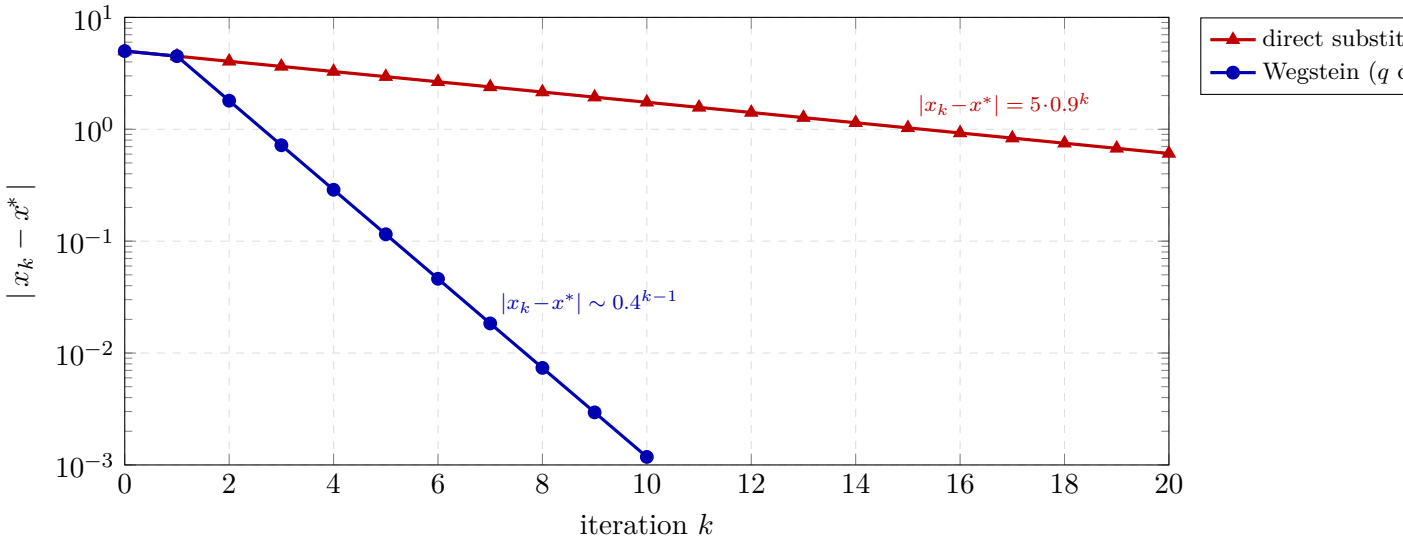

Figure 35: Wegstein vs. direct substitution on the test problem $g(x) = 0.9\,x + 0.5$ with fixed point $x^* = 5$ (chosen so that $g'(x^*) = 0.9$ is close to one — direct substitution converges slowly). Direct substitution contracts at rate 0.9 (geometric, dotted red); Wegstein with q clipped to $[-5, 0]$ contracts at rate ~ 0.4, two-and-a-half times faster on a logarithmic scale. After ten iterations Wegstein has reduced the error by three orders of magnitude where direct substitution has barely halved it; after twenty iterations the gap is six orders.

6 Closing a recycle: Newton on the tear stream

What you'll learn. A flowsheet with a recycle has a directed cycle: a downstream unit feeds an upstream one, so no unit can be solved until the stream entering the loop is already known — a chicken-and-egg the simulator breaks by *tearing* the loop, guessing the torn stream, sweeping once, and iterating until the guess reproduces itself. The previous chapter accelerated that iteration scalar-by-scalar with Wegstein. This chapter takes the alternative the simulator now uses by default: treat the whole torn stream as one vector and drive its residual to zero with the n-dimensional Newton of Chapter 3. We derive the tear residual, justify the choice of tear variables, and — via a real bug the change exposed — explain why the residual must be *relative*.

6.1 The recycle as a fixed-point in the tear stream

Recall the tearing picture from Chapter 1. Cut one stream in the cycle — the *tear stream* — and let $\mathbf{x}$ be its state (flows, temperature). Guessing $\mathbf{x}$ makes the loop acyclic: every unit now has a known inlet, so one pass of the ordinary sequential-modular executive can be run, unit by unit, all the way around the loop until it recomputes the torn stream. Call the result of that one full sweep

$$\mathbf{x}^{\text{new}} \;=\; G(\mathbf{x}), \tag{196}$$

where G is "run the whole flowsheet once, starting from this tear guess". The recycle is converged when the guess reproduces itself,

$$G(\mathbf{x}^*) \;=\; \mathbf{x}^* \qquad\Longleftrightarrow\qquad \mathbf{r}(\mathbf{x}^*) \;\equiv\; G(\mathbf{x}^*) - \mathbf{x}^* \;=\; \mathbf{0}. \tag{197}$$

Wegstein attacks the left form (a fixed point $\mathbf{x} = G(\mathbf{x})$, accelerated component-wise). Newton attacks the right form (a root $\mathbf{r}(\mathbf{x}) = \mathbf{0}$), all components at once. Same G, same sweep, same per-unit pedagogy — only the outer iteration differs.

6.2 What goes in the tear vector

A process stream carries a total flow, a composition, a temperature, a pressure and a phase split — but these are not all independent, and a good tear vector should use only independent coordinates. The tempting choice "total flow F plus all mole fractions z_i" is *redundant*: the fractions obey $\sum_i z_i = 1$, so $(F, z_1, \ldots, z_{N_c})$ carries $N_c + 1$ numbers for a stream whose physical content is only its N_c component flows. The redundancy is not catastrophic — it does *not*, on its own, make the Jacobian singular: the surplus direction (the one that holds every component flow Fz_i fixed while trading F off against the z_i) leaves G unchanged, so $\partial G/\partial \mathbf{x}$ annihilates it and the Jacobian $\mathbf{J} = \partial G/\partial \mathbf{x} - \mathbf{I}$ maps it to the harmless eigenvalue -1, not 0. It is simply wasteful and awkward: one extra variable (hence an extra pair of Jacobian sweeps), and the iterate's z_i must be re-normalised onto the simplex every step or they drift and F loses its meaning.

Choupo sidesteps this by tearing on *component molar flows* plus temperature,

$$\mathbf{x} \;=\; (F_1,\, F_2,\, \ldots,\, F_{N_c},\, T), \qquad F_i \equiv F z_i, \tag{198}$$

which are mutually independent — there is no normalisation tying them together (the total $F = \sum_i F_i$ and the fractions $z_i = F_i/F$ are recovered afterwards). Pressure and phase split are not torn: pressure is set by the units, and the phase fractions follow from a flash on the converged (F_i, T). For N_c components the tear vector has $N_c + 1$ entries.

6.3 Newton on the tear residual

With the residual (197) in hand, the rest is the machinery of Chapter 3 applied verbatim. The Jacobian

$$\mathbf{J} \;=\; \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \;=\; \frac{\partial G}{\partial \mathbf{x}} - \mathbf{I} \tag{199}$$

is built by *central* finite differences: perturb tear variable j by $\pm\delta$, run a full sweep G at each, and the symmetric difference $[\,G(\mathbf{x} + \delta\mathbf{e}_j) - G(\mathbf{x} - \delta\mathbf{e}_j)\,]/2\delta$ gives column j. That is *two* sweeps per variable, $2(N_c + 1)$ sweeps per Jacobian — the price Newton pays that Wegstein (one sweep per step) does not. Central differences cost twice what forward ones would, but are $O(\delta^2)$ accurate rather than $O(\delta)$, which keeps the Jacobian — and the convergence rate it drives — clean. Each Newton step then solves the linear system

$$\mathbf{J}\,\Delta\mathbf{x} \;=\; -\,\mathbf{r}(\mathbf{x}), \tag{200}$$

by the same Gauss elimination with partial pivoting, takes $\mathbf{x} \leftarrow \mathbf{x} + \alpha\,\Delta\mathbf{x}$ under a backtracking line search (start at $\alpha = 1$ and halve it until the residual norm $\|\mathbf{r}\|$ actually drops, so the method stays robust far from the solution), and repeats. In code this is literally a call to `solver::newtonND` with $\mathbf{r}$ as the residual functor — the recycle reuses the project's one general-purpose n-D Newton, no special-case algebra. Near the solution it converges essentially *quadratically* — the number of correct digits roughly doubles each step, the $O(\delta^2)$ central-difference Jacobian holding that rate down to a very low floor — where Wegstein only improves super-linearly.

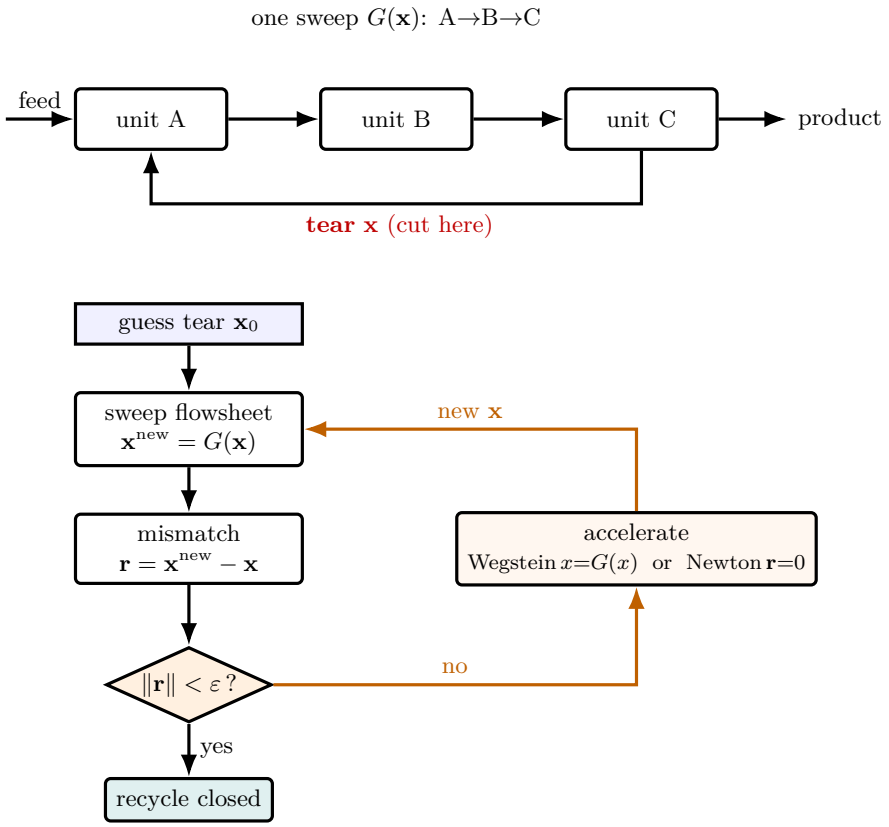

Figure 36: Closing a recycle. *Left:* the physical flowsheet has a cycle (C feeds back to A); cutting the recycle at the dashed **tear** turns it into an open chain whose one sweep $G(\mathbf{x})$ = "run A→B→C from this tear guess" is just the ordinary sequential pass. *Right:* the outer loop guesses the tear, sweeps once, measures the mismatch $\mathbf{r} = G(\mathbf{x}) - \mathbf{x}$, and — if it is still too large — hands $\mathbf{r}$ to an accelerator (Wegstein on the fixed point, or Newton on the root) to propose a better tear, then sweeps again. The convergence is reached when the guess reproduces itself. Crucially, the per-unit physics inside G never changes; only the box that proposes the *next* tear differs between Wegstein and Newton.

6.4 Why the residual must be relative — a bug story

There is a subtlety that bites hard, and the simulator learned it the painful way. Write the convergence test naively as an *absolute* norm, $\|\mathbf{r}\| < \varepsilon$ with, say, $\varepsilon = 10^{-5}$. Now look at what is inside $\mathbf{r}$ for a typical tear: a temperature near 365 and component flows that, for a small recycle, can be of order 2×10^{-4} kmol/s. The norm is *dominated by the temperature residual*. A solver can satisfy $\|\mathbf{r}\| < 10^{-5}$ with the temperature pinned to six digits while the tiny tear flow is still off by 5 % — and report success.

That is exactly what happened. The `process03_recycle` tutorial ran for versions with its recycle flow converged to only a few percent; the recorded golden value was itself under-converged, and nobody noticed because the exit code was zero and the temperature looked perfect. The fix is to make every component of the residual *dimensionless and comparable* by scaling each by the magnitude of its own variable:

$$\tilde{r}_i = \frac{G_i(\mathbf{x}) - x_i}{s_i}, \qquad s_i = \begin{cases} F_{\text{tot}} & \text{for a component-flow entry,} \\ T & \text{for the temperature entry,} \end{cases} \tag{201}$$

so that "converged" means *every* variable — the dominant temperature and the faint trace flow alike — has stopped moving to the same relative precision. With (201) the process03 recycle converges to its true fixed point ($F \approx 2.008 \times 10^{-4}$ kmol/s, $\|\tilde{\mathbf{r}}\| \approx 5 \times 10^{-9}$ in five Newton steps), and the golden master was re-recorded against that genuinely-converged answer.

> *Intuition.* A residual norm is only as trustworthy as its worst-scaled component. Mixing a temperature ($\sim 10^2$) and a trace flow ($\sim 10^{-4}$) in one absolute norm is like checking that a bridge is level by averaging the height of the towers and the rivets — the towers swamp the average and a crooked rivet passes. Scale first,

then compare.

6.5 Wegstein or Newton? Choosing the recycle solver

Both are available; `recycleSolver Newton;` is the default and

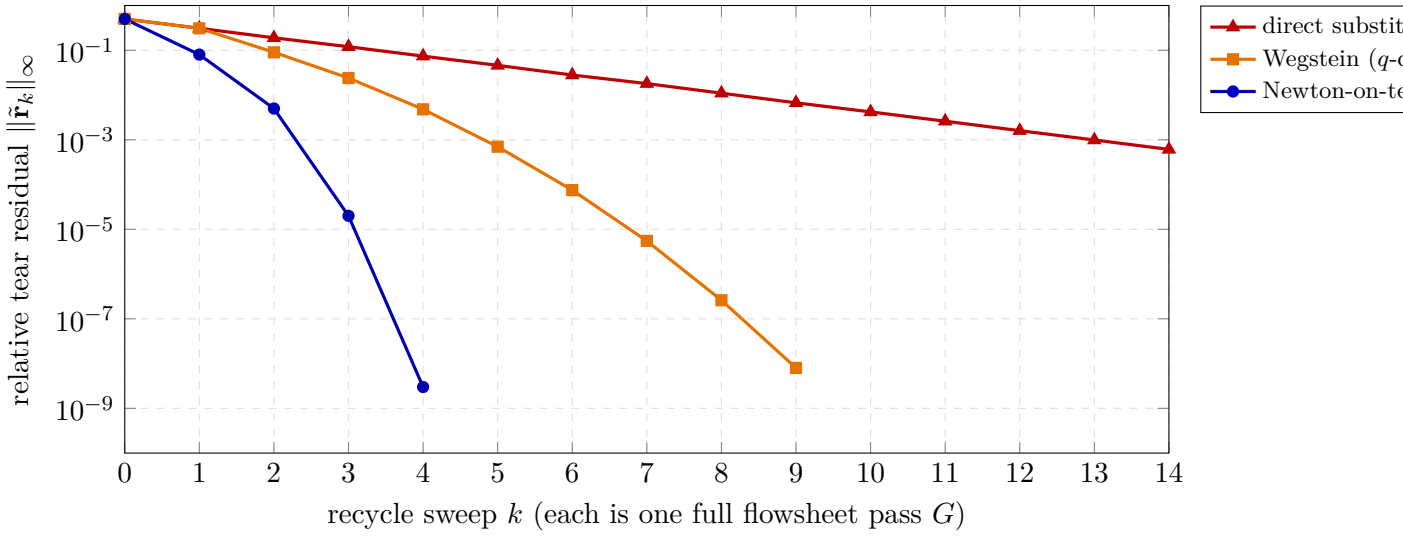

Figure 37: The three ways to close the same coupled recycle, plotted as relative tear residual versus outer sweep (`process03_recycle`). Direct substitution contracts at a fixed geometric rate — a straight line on the log scale, slow whenever the loop gain is near one. Wegstein bends downward (super-linear: each step's reduction improves as the secant sharpens), and Newton-on-tears dives quadratically, clearing machine precision in $\sim$5 outer steps. The catch hidden by the x-axis: a Newton step costs $2(N_c{+}1)$ sweeps to build its central-difference Jacobian, while Wegstein and direct substitution cost one sweep each — so Newton wins on *coupled* tears where its Jacobian pays for itself, and Wegstein wins when sweeps are dear and the tear is nearly diagonal.

`recycleSolver Wegstein;` keeps the accelerator. They trade off cleanly:

	Wegstein	**Newton-on-tears**
cost per outer step	one sweep	$2(N_c + 1)$ sweeps (central-difference Jacobian)
convergence	super-linear	quadratic
coupling between tear variables	ignored (component-wise scalars)	captured (full Jacobian)
robustness on counter-current / strongly-coupled tears	needs damping (q-clip)	strong
best when	loosely-coupled single tear	coupled and/or multiple tears

Newton wins when the tear variables are coupled — a change in one component flow swings the others through the units — because its Jacobian sees that coupling, which Wegstein's independent scalars cannot. Wegstein can still be the better tool when sweeps are expensive and the tear is nearly diagonal, or as a robust damped fallback: the counter-current double-effect evaporator `evaporator05` runs damped Wegstein ($q_{\min} = -0.7$) to tame the over-shoot endemic to backward-feed vapour recycles, where an undamped step would oscillate.

6.6 This is not equation-oriented — and that is the point

It is tempting to call Newton-on-tears "equation-oriented", but it is not. A true equation-oriented (EO) solver collapses *every* unit's internal equations and *every* stream connection into one giant sparse system and Newtons on all of it at once; the units lose their identity as solvable modules. Choupo deliberately does not do that (see the v1.0 note in `CLAUDE.md`): full EO needs hand-rolled sparse linear algebra, and it dissolves the per-unit transparency that is the whole pedagogical point — a student can no longer watch a single flash or reactor converge in isolation.

Newton-on-tears keeps the sequential-modular skeleton intact — each unit still solves itself, G is still one honest sweep — and wraps *only the recycle* in a Newton. It is the Pareto step: most of the convergence robustness of EO, at none of its architectural cost, on the handful of variables (the tears) where the coupling actually lives. If the

project ever adds a true EO mode it will be an opt-in alternative (`solver eo;`) offered *alongside* the modular path, precisely so the manual can teach the contrast — not a replacement for it.

> **Runnable case.** `tutorials/steady/flowsheets/process03_recycle` — reactor–separator with a recycle, `recycleSolver Newton;` explicit; the genuinely-tight fixed point of the bug story above. `tutorials/steady/flowsheets/process05_isomerization_recycle` — a second Newton recycle. `tutorials/steady/evaporation/evaporator05_counter_current` — two tear streams closed with damped `recycleSolver Wegstein;`, the counter-current case where damping earns its keep.

7 Direct search: Nelder-Mead

What you'll learn. Newton and Levenberg-Marquardt need derivatives. Wegstein needs a fixed-point iteration. When the objective is the output of a long, discontinuous, or noisy simulator pass — as it is whenever we ask "what reflux ratio minimises the reboiler duty?" — derivatives may not exist as smooth analytical objects, and finite differences may amplify noise. Nelder-Mead is the canonical *derivative-free* optimiser: it maintains a geometric simplex of $n+1$ points in $\mathbb{R}^n$ and crawls downhill by reflecting and contracting that shape. It is the algorithm behind `outerDict type optimization`.

The problem

We want to minimise a scalar

$$f : \mathbb{R}^n \longrightarrow \mathbb{R},$$

where f is provided as a black box — evaluating $f(\mathbf{x})$ runs one full simulator pass on a flowsheet with the design variables $\mathbf{x}$ written into the dictionary, and (optionally) a post-processing chain (sizing + costing) on top of the result. We have:

- no symbolic derivative;
- finite differences are expensive (each one is another full simulator pass) *and* noisy when the inner solvers carry their own tolerance;
- bounds on each x_i (refluxes cannot be negative, reactor volumes cannot exceed your budget).

The Nelder-Mead simplex algorithm [61] sidesteps all three obstacles.

The simplex

A simplex in $\mathbb{R}^n$ is a set of $n+1$ vertices,

$$\mathcal{S} = \{\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\},$$

in general position — they span $\mathbb{R}^n$ as an affine basis. In two dimensions a simplex is a triangle; in three, a tetrahedron. At each iteration we order the vertices by their objective values:

$$f(\mathbf{x}_{\text{best}}) \;\le\; f(\mathbf{x}_{\text{second}}) \;\le\; \cdots \;\le\; f(\mathbf{x}_{\text{worst}}).$$

We then try to replace the worst vertex by something better, leaving the other n alone.

The four moves

Let $\mathbf{x}_c$ be the centroid of all vertices *except* the worst:

$$\mathbf{x}_c \;=\; \frac{1}{n} \sum_{k \neq \text{worst}} \mathbf{x}^{(k)}.$$

The candidates are formed along the line from the worst point through the centroid. In order of cost (each move is one extra evaluation):

1. Reflection.

$$\mathbf{x}_r \;=\; \mathbf{x}_c + \alpha\,(\mathbf{x}_c - \mathbf{x}_{\text{worst}}), \qquad \alpha = 1.$$

The worst vertex is reflected through the centroid. If $f(\mathbf{x}_r)$ falls between $f(\mathbf{x}_{\text{best}})$ and $f(\mathbf{x}_{\text{second}})$, the reflection is accepted and the iteration ends.

2. Expansion.

If $f(\mathbf{x}_r) < f(\mathbf{x}_{\text{best}})$, the reflection direction looks *very* promising; we extend further:

$$\mathbf{x}_e \;=\; \mathbf{x}_c + \gamma\,(\mathbf{x}_r - \mathbf{x}_c), \qquad \gamma = 2.$$

Whichever of $\mathbf{x}_e, \mathbf{x}_r$ has the lower f replaces the worst vertex.

3. Contraction.

If $f(\mathbf{x}_r) \ge f(\mathbf{x}_{\text{second}})$, the reflection landed on a hill; we shrink toward the centroid instead:

$$\mathbf{x}_o \;=\; \mathbf{x}_c + \rho\,(\mathbf{x}_{\text{worst}} - \mathbf{x}_c), \qquad \rho = 0.5.$$

(In Choupo we use *outside contraction* — $\mathbf{x}_c + \rho\,(\mathbf{x}_r - \mathbf{x}_c)$ — when $f(\mathbf{x}_r) < f(\mathbf{x}_{\text{worst}})$, otherwise the inside form above. Either way the new point is on the line through $\mathbf{x}_c$, closer to it.)

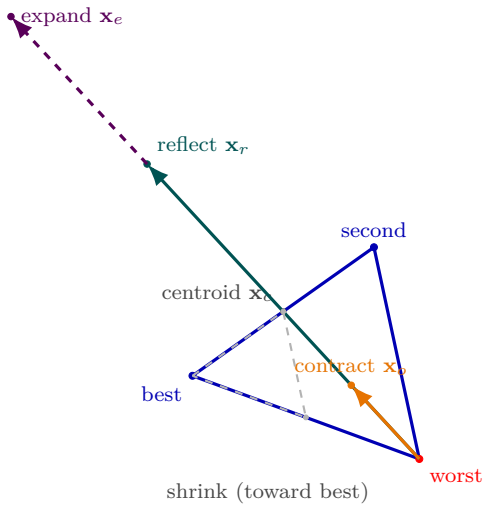

Figure 38: The four Nelder–Mead moves as geometry, on a triangular simplex ($n = 2$). All candidates lie on the line through the worst vertex and the centroid $\mathbf{x}_c$ of the kept vertices. *Reflection* (teal) flips the worst point through $\mathbf{x}_c$; if that is wildly successful, *expansion* (violet, dashed) pushes further along the same ray; if reflection lands on a hill, *contraction* (orange) pulls back toward $\mathbf{x}_c$; and if even that fails, *shrink* (grey, dashed) drags every vertex but the best halfway in, making the whole simplex smaller and more local. Only shrink re-evaluates more than one vertex — it is the algorithm conceding the reflection ray has stalled. The decision tree of Fig. 39 is exactly the rule for picking which of these four points to keep.

4. Shrink. If contraction also fails, every vertex except the best is pulled halfway toward the best:

$$\mathbf{x}^{(k)} \leftarrow \mathbf{x}_{\text{best}} + \sigma\,(\mathbf{x}^{(k)} - \mathbf{x}_{\text{best}}), \qquad \sigma = 0.5,$$

for $k \neq$ best. This is the only move that re-evaluates multiple vertices. It is the algorithm's confession that progress along the reflection line has stalled and the simplex needs to become smaller and more local.

Box bounds without changing the algorithm

Nelder-Mead is, by construction, unconstrained. When the user supplies bounds $\mathbf{x} \in [\mathbf{x}_{\text{lo}}, \mathbf{x}_{\text{hi}}]$ for each variable, Choupo *clips* every candidate vertex back into the box element-wise *before* calling f:

$$x_i \leftarrow \min(\max(x_i, \text{lo}_i), \text{hi}_i).$$

The simplex learns very quickly that probing the violation region pays the same as the wall, so it contracts away from the wall. This is the simplest defensible handling; if you want a smooth penalty term or a true active-set strategy, that is the job of an SQP or interior-point wrapper, not Nelder-Mead.

Stopping criteria

Two natural tests:

- **tolX** — the simplex has shrunk. We compute the maximum coordinate-wise extent of the simplex, normalised *per axis* by that axis's bound width:

$$\text{tolX trip} \;:\; \max_i \frac{\max_k |x_i^{(k)} - x_i^{\text{best}}|}{\text{hi}_i - \text{lo}_i} \;<\; \text{tolX}.$$

  The per-axis normalisation is crucial: if one variable is a reactor volume in m^3 and the other a temperature in K, a global $\|\Delta\mathbf{x}\|$ would be dominated by the larger scale and would stop the algorithm before the small axis has converged. Choupo uses this scaled criterion.

- **tolF** — the range of f over the simplex is small relative to $|f_{\text{best}}|$. Useful when f is flat near the optimum.

Plus the usual safety net of `maxIter`.

When Nelder-Mead is the right hammer

- Modest number of design variables (say $n \leq 10$). The simplex needs $n + 1$ vertices and each iteration costs at least one new evaluation, so the wall-clock cost grows gracefully in n.
- Black-box, possibly noisy, possibly discontinuous objective. The simulator pass is exactly that.
- Local search. Nelder-Mead converges to a local minimum near the initial simplex; for a global search you would restart with several random simplexes (*multi-start*) and keep the best.

When to reach for something else

- High dimension ($n \gg 10$). Simplex moves become inefficient because each move only affects one vertex; on a 50-D problem, a quasi-Newton method with finite differences typically wins despite the gradient cost.
- Smooth, differentiable objective with cheap derivatives. Use a Newton-family method directly (Choupo's NewtonND, or LM for least-squares, both already in §3 and §4).
- Hard constraints, especially non-linear ones. Use SQP — Choupo's constrained optimiser, §8 — or an interior-point wrapper (Choupo offers no interior-point method).

What you see in Choupo

The `OptimizationDriver` prints one line per simplex iteration, showing the move type (`reflect` / `expand` / `contract` / `shrink`), the current best f value, the current best $\mathbf{x}$, and the simplex size. Identical rows go into `optimization_history.csv` so the student can plot the convergence trajectory. On termination the simulator is replayed once at the optimum with full verbosity, so the case directory ends with the converged final state on standard output — exactly as a direct (non-outer) run would leave it.

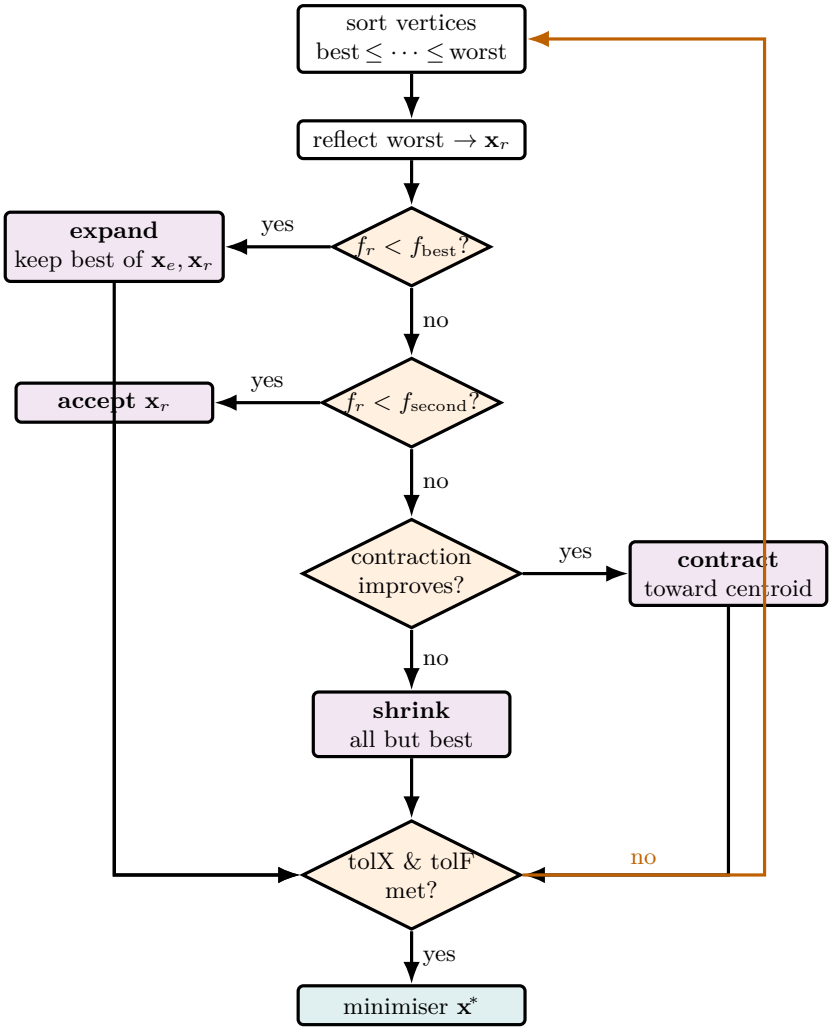

Figure 39: The Nelder–Mead move decision tree — a derivative-free descent that needs only f values, used in Choupo wherever a Jacobian is treacherous: the *direct* Gibbs-energy minimisation of an LL or VLL flash (§9), and parameter-fitting in `OptimizationDriver`. Each pass orders the $n{+}1$ vertices, reflects the worst through the centroid, and then — by a short cascade of cheap tests on the reflected value — chooses the *expand* / *accept* / *contract* / *shrink* move, each just a point on the line through the centroid. Only *shrink* re-evaluates several vertices; it is the algorithm conceding the reflection line has stalled. The simplex crawls downhill until it is both small (tolX) and flat (tolF). For the symmetric γ-models where a Newton flash would collapse onto the trivial $K = 1$ saddle, this "crawl the Gibbs surface" search — multi-started — is what finds the true two-phase split.

8 Constrained optimisation: line-search SQP

What you'll learn. Nelder-Mead minimises a black box but cannot honour a hard constraint. Sequential Quadratic Programming (SQP) does: at every iterate it linearises the constraints, builds a convex *quadratic* model of the Lagrangian, solves that quadratic program (QP) for a step, then shortens the step with an ℓ_1 merit function until it makes honest progress. We derive the QP from the KKT conditions, the damped-BFGS curvature update that keeps the QP convex, the merit function that arbitrates objective against feasibility, the Armijo backtracking line search, and the five-residual KKT certificate Choupo prints at the optimum. The inner convex QP is solved by an active-set method, derived in §8.7.

The flowsheet optimiser often comes with real constraints: a purity that must be *at least* 0.99, a duty that must *equal* a target, a recovery bounded above. Nelder-Mead (§7) cannot see any of these — it only ranks objective values, so a constraint can only enter as a hand-tuned penalty, which distorts the objective and is brittle. SQP [62, 63] is the classical answer: it treats the constraints as first-class citizens and converges to a point that provably satisfies the Karush–Kuhn–Tucker (KKT) optimality conditions. Choupo's `solver::sqp` is a hand-rolled, finite-difference, line-search SQP — no external library, every quantity inspectable.

8.1 The problem and the KKT conditions

The general nonlinear program (NLP) Choupo's SQP solves is

$$\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad \begin{cases} h_j(\mathbf{x}) = 0, & j = 1,\dots,m_{\mathrm{e}}, \\ g_k(\mathbf{x}) \le 0, & k = 1,\dots,m_{\mathrm{i}}, \\ \mathrm{lo} \le \mathbf{x} \le \mathrm{hi}, \end{cases} \tag{202}$$

with equalities h_j (e.g. a DesignSpec residual), inequalities g_k (written in the ≤ 0 convention — a purity floor $x_{\mathrm{purity}} \ge 0.99$ becomes $g = 0.99 - x_{\mathrm{purity}} \le 0$), and box bounds. The Lagrangian collects every constraint with a multiplier:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) \;=\; f(\mathbf{x}) \;+\; \sum_j \lambda_j\, h_j(\mathbf{x}) \;+\; \sum_k \mu_k\, g_k(\mathbf{x}). \tag{203}$$

A point $\mathbf{x}^\star$ is a (first-order) constrained minimiser when the **KKT conditions** hold:

$$\nabla_{\mathbf{x}}\mathcal{L} = \nabla f + \textstyle\sum_j \lambda_j \nabla h_j + \sum_k \mu_k \nabla g_k = \mathbf{0} \qquad \text{(stationarity)}, \tag{204}$$

$$h_j(\mathbf{x}^\star) = 0, \quad g_k(\mathbf{x}^\star) \le 0 \qquad \text{(primal feasibility)}, \tag{205}$$

$$\mu_k \ge 0 \qquad \text{(dual feasibility)}, \tag{206}$$

$$\mu_k\, g_k(\mathbf{x}^\star) = 0 \qquad \text{(complementary slackness)}. \tag{207}$$

Complementarity (207) says: a constraint that is not

binding ($g_k < 0$) carries no multiplier ($\mu_k = 0$); only *active* constraints ($g_k = 0$) can push back with $\mu_k > 0$.

8.2 Shadow prices: what the multiplier is worth

The multipliers are not bookkeeping — they answer the question a process engineer actually asks: *"what is this constraint costing me, and what would I pay to loosen it?"* Perturb the right-hand side of an active inequality, $g_k(\mathbf{x}) \le b_k$, by a small Δb_k. The feasible set opens up, the optimum slides along the relaxed boundary, and — because at the optimum $\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{0}$, so only the *explicit* dependence on b_k survives (the envelope / sensitivity theorem) — the optimal objective moves by

$$\frac{\partial f^\star}{\partial b_k} \;=\; -\,\mu_k \qquad\Longleftrightarrow\qquad \Delta f^\star \;\approx\; -\,\mu_k\,\Delta b_k. \tag{208}$$

That is the whole reason the multiplier is called a **shadow price**. Read it economically — with the sign flipped for a *maximisation*, since Choupo maximises NPV by minimising −NPV — and μ_k is the **marginal € value of one more unit of the capped resource**: one more m^3 of reactor, one more € of capital budget, one more kg/h of market demand. A binding capital budget with $\mu = 0.8$ says the next euro of CAPEX would buy €0.80 of extra NPV (build it); a binding cap with $\mu = 0.05$ is barely worth relaxing; a *slack* constraint has $\mu = 0$ and can be ignored entirely. This is why `solver::sqp` prints the active set and its multipliers every iteration and again in the closing KKT certificate — the shadow prices *are* the design answer, not a diagnostic. The tutorial `tutorials/steady/optimisation/optim05_economic_design` maximises a small plant's NPV against a capital budget and a capacity cap and reads the binding cap's shadow price straight off the certificate.

The multipliers are *shadow prices* — λ_j (or μ_k for an inequality) is the marginal change in the optimal f per unit relaxation of its constraint — which is exactly the sensitivity a process engineer wants, so Choupo prints them every iteration.

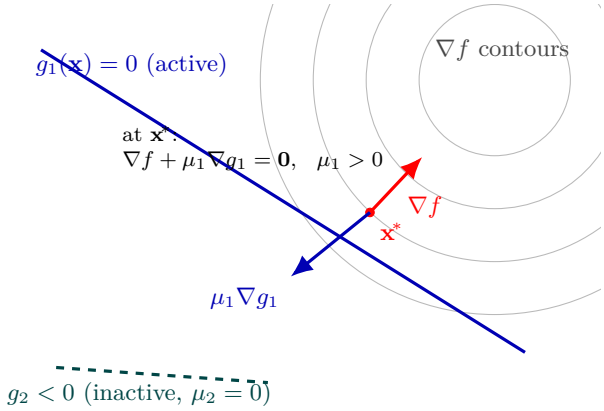

Figure 40: The KKT stationarity condition as a force balance at a constrained minimiser. The grey arcs are level sets of the objective f; absent constraints, f would keep decreasing toward its unconstrained minimum (upper right). The active inequality $g_1 = 0$ (blue line) blocks the way, so the constrained optimum $\mathbf{x}^*$ sits *on* that boundary, where an objective contour is tangent to it. There the objective gradient ∇f (red, pointing uphill) is exactly cancelled by the constraint's push $\mu_1 \nabla g_1$ (blue) — stationarity $\nabla f + \mu_1 \nabla g_1 = \mathbf{0}$ with multiplier $\mu_1 > 0$. The second constraint g_2 (dashed teal) is slack: it does not touch $\mathbf{x}^*$, so by complementary slackness its multiplier is zero and it exerts no force. The active multiplier μ_1 is the *shadow price* — how much f would improve per unit of relaxing g_1 — which is why Choupo prints it.

8.3 From KKT to a quadratic program

Newton's method on the nonlinear KKT system (204)–(205) is the heart of SQP. Linearise the constraints about the current $\mathbf{x}$ and replace the Lagrangian by a quadratic model in the step $\mathbf{d} = \mathbf{x}_+ - \mathbf{x}$, with curvature matrix $\mathbf{B} \approx \nabla^2_{\mathbf{xx}} \mathcal{L}$ (note: the Hessian of the *Lagrangian*, not of f — this is the single most common SQP mistake; see §8.4). The step solves the convex QP

$$\min_{\mathbf{d} \in \mathbb{R}^n} \ \tfrac{1}{2}\, \mathbf{d}^\top \mathbf{B}\, \mathbf{d} + \nabla f^\top \mathbf{d} \quad \text{s.t.} \quad \begin{cases} \nabla h_j^\top \mathbf{d} + h_j = 0, \\ \nabla g_k^\top \mathbf{d} + g_k \le 0, \\ \mathrm{lo} - \mathbf{x} \le \mathbf{d} \le \mathrm{hi} - \mathbf{x}. \end{cases} \tag{209}$$

The constant $f(\mathbf{x})$ is dropped (it does not move the minimiser). The linear-in-$\mathbf{d}$ constraint rows are precisely the first-order Taylor predictions $h_j(\mathbf{x} + \mathbf{d}) \approx h_j + \nabla h_j^\top \mathbf{d}$ driven to feasibility. The QP multipliers returned for (209) *are* the SQP's estimate of $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ for the next iterate — the QP solve does double duty, producing both the primal step $\mathbf{d}$ and the dual estimate.

Finite-difference gradients. Choupo carries no analytic sensitivities — the objective and every constraint come out of one simulator pass. So ∇f and the full constraint Jacobian are built by finite differences from a *single* perturbation of each variable: forward differences

$$\frac{\partial f}{\partial x_j} \approx \frac{f(\mathbf{x} + \delta_j \mathbf{e}_j) - f(\mathbf{x})}{\delta_j}, \qquad \delta_j = \varepsilon_{\mathrm{FD}} \max(|x_j|,\, x_j^{\mathrm{typ}}), \tag{210}$$

cost $n + 1$ response evaluations per iteration (n perturbations + the base point); the opt-in *central* scheme costs $2n$ but is $O(\delta^2)$ accurate. Crucially, one perturbation of x_j yields column j of ∇f *and* of the whole Jacobian at once — the responses f, $\mathbf{h}$, $\mathbf{g}$ are read off the same pass.

8.4 Keeping the QP convex: damped-BFGS on the Lagrangian

The QP (209) is convex — and the active-set solver of §8.7 only works — if $\mathbf{B}$ is symmetric positive definite (SPD). Choupo never forms the true Lagrangian Hessian (it would need second derivatives by finite difference: $O(n^2)$ extra passes). Instead it carries a quasi-Newton approximation $\mathbf{B}$, started at $\mathbf{B}_0 = \mathbf{I}$ and updated each step from the curvature pair

$$\mathbf{s} = \mathbf{x}_+ - \mathbf{x}, \qquad \mathbf{y} = \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}_+, \boldsymbol{\lambda}_+) - \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}_+). \tag{211}$$

The subtlety that students miss: $\mathbf{y}$ is the difference of the *Lagrangian* gradient (204) (objective *plus* constraint terms) evaluated with the *same* new multipliers $\boldsymbol{\lambda}_+$ at *both* points. Using ∇f alone silently builds the wrong curvature, solves the wrong QP, and stalls on active constraints.

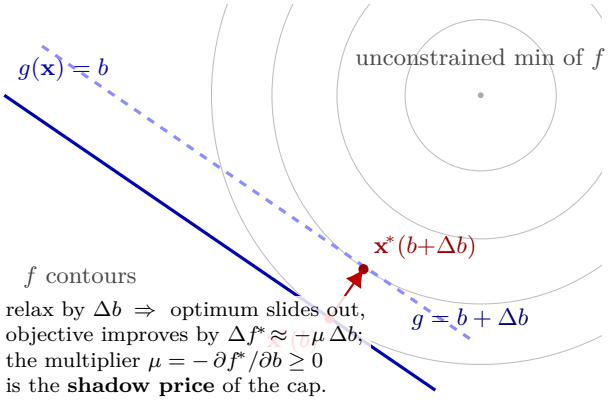

Figure 41: The multiplier as a price. Contours of the objective f (grey) tighten toward its unconstrained minimum (upper right), which the active cap $g = b$ (solid blue) keeps out of reach, so the constrained optimum $\mathbf{x}^*(b)$ sits on that boundary, tangent to the outer contour. Loosen the cap to $g = b + \Delta b$ (dashed) and the boundary shifts toward the unconstrained minimum; the optimum slides to $\mathbf{x}^*(b+\Delta b)$ on a *smaller* contour — a better objective. The rate of that improvement per unit of relaxation is exactly the Lagrange multiplier, $\Delta f^* \approx -\mu\,\Delta b$ (Eq. 208): μ is the shadow price. A constraint that does not touch the optimum is slack, its multiplier is zero, and moving it changes nothing — complementary slackness, seen geometrically.

The plain BFGS update preserves SPD only when the curvature condition $\mathbf{s}^\top\mathbf{y} > 0$ holds, which a nonconvex Lagrangian can violate. Powell's *damped* BFGS [63] repairs this by replacing $\mathbf{y}$ with a blend toward $\mathbf{Bs}$:

$$\theta = \begin{cases} 1, & \mathbf{s}^\top\mathbf{y} \geq 0.2\,\mathbf{s}^\top\mathbf{Bs}, \\ \dfrac{0.8\,\mathbf{s}^\top\mathbf{Bs}}{\mathbf{s}^\top\mathbf{Bs} - \mathbf{s}^\top\mathbf{y}}, & \text{otherwise}, \end{cases} \qquad \mathbf{r} = \theta\,\mathbf{y} + (1-\theta)\,\mathbf{Bs}, \tag{212}$$

which guarantees $\mathbf{s}^\top\mathbf{r} \geq 0.2\,\mathbf{s}^\top\mathbf{Bs} > 0$, then applies the BFGS rank-two update with $\mathbf{r}$ in place of $\mathbf{y}$:

$$\mathbf{B} \;\leftarrow\; \mathbf{B} - \frac{\mathbf{Bs}\,\mathbf{s}^\top\mathbf{B}}{\mathbf{s}^\top\mathbf{Bs}} + \frac{\mathbf{r}\,\mathbf{r}^\top}{\mathbf{s}^\top\mathbf{r}}. \tag{213}$$

$\theta = 1$ recovers ordinary BFGS; $\theta < 1$ damps toward the positive-definite $\mathbf{Bs}$ direction. Choupo prints θ each step — a run of $\theta \ll 1$ warns that the Lagrangian curvature is fighting the update (often finite-difference noise), and the update is *skipped* entirely if even the damped $\mathbf{s}^\top\mathbf{r}$ collapses.

8.5 The ℓ_1 merit function and the Armijo line search

The QP step $\mathbf{d}$ is a direction, not a commitment. A full step $\alpha = 1$ may overshoot, especially far from the optimum where the linearised constraints lie. SQP needs a scalar that decides whether a trial step is *good* — but "good" must balance two competing goals: lower the objective *and* reduce infeasibility. Choupo uses the exact ℓ_1 *merit function* [62]

$$\phi_\mu(\mathbf{x}) \;=\; f(\mathbf{x}) \;+\; \mu\left(\sum_j |h_j(\mathbf{x})| + \sum_k \max(0,\, g_k(\mathbf{x}))\right), \tag{214}$$

where the penalty parameter μ weights the total constraint violation. "Exact" means: for μ large enough, an unconstrained minimiser of ϕ_μ coincides with the constrained minimiser of (202) — no $\mu \to \infty$ limit is needed. Choupo raises μ monotonically to stay above the multiplier estimates,

$$\mu \;\geq\; \mu_{\text{safety}}\,\max(\|\boldsymbol{\lambda}\|_\infty, \|\boldsymbol{\mu}\|_\infty), \qquad \mu_{\text{safety}} = 1.1, \tag{215}$$

the standard rule that makes $\mathbf{d}$ a descent direction for ϕ_μ.

A backtracking line search then picks α. The QP step is a descent direction with directional derivative

$$D\phi_\mu(\mathbf{x};\mathbf{d}) \;=\; \nabla f^\top\mathbf{d} \;-\; \mu\left(\sum_j |h_j| + \sum_k \max(0, g_k)\right), \tag{216}$$

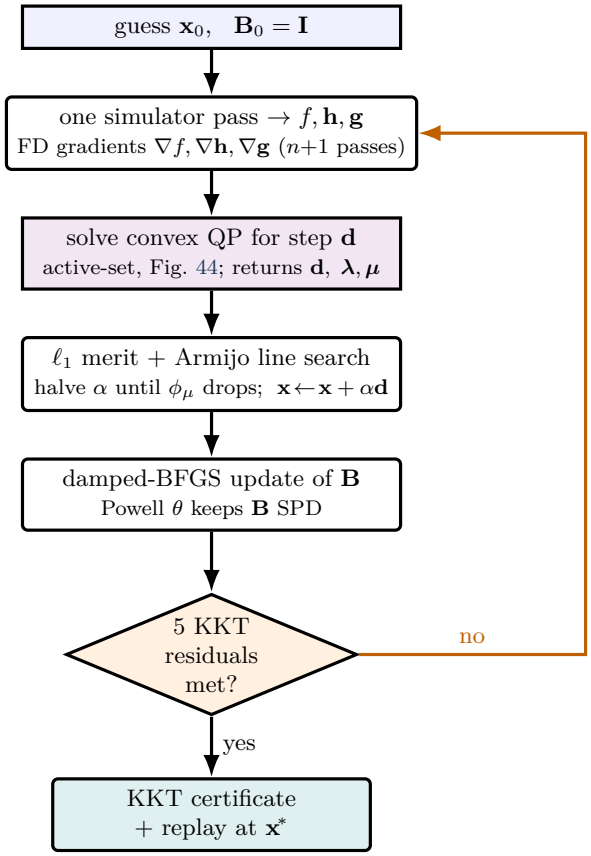

Figure 42: One pass of Choupo's line-search SQP, top to bottom. Each iterate runs *one* simulator pass to read off the objective and all constraints, builds finite-difference gradients from it, then solves a convex quadratic program (the inner active-set solver of Fig. 44) for both a step $\mathbf{d}$ and fresh multiplier estimates. An ℓ_1 merit function with an Armijo backtracking line search decides how far along $\mathbf{d}$ to actually move — balancing lower objective against less constraint violation — and a damped-BFGS update keeps the curvature matrix $\mathbf{B}$ symmetric positive definite so the next QP stays convex. The loop exits only when all five KKT residuals clear tolerance, and the printed certificate lets the student verify optimality by hand.

(the second term is the predicted reduction of the violation the QP constraints drive to zero). Choupo halves α from 1 until the **Armijo** sufficient-decrease condition holds:

$$\phi_\mu(\mathbf{x} + \alpha\mathbf{d}) \;\le\; \phi_\mu(\mathbf{x}) + c_1\,\alpha\,D\phi_\mu(\mathbf{x};\mathbf{d}), \qquad c_1 = 10^{-4}. \tag{217}$$

If α falls below $\alpha_{\min} = 10^{-8}$ without acceptance the

solver stops *honestly* ("line search failed — no merit decrease"), rather than pretend convergence — usually a sign the finite-difference gradients have hit their noise floor.

The Maratos effect and the watchdog. A known pathology: near the optimum the ℓ_1 merit can reject the *full* Newton step even though it improves both objective and feasibility — the *Maratos effect* [62] — which destroys the fast local rate. Choupo's v1 mitigation is deliberately minimal: a *watchdog* that forces the full $\alpha = 1$ step every $K = 5$ iterations regardless of the merit, breaking the merit's veto. It also *reports* the symptom (full step improved feasibility yet the merit rejected it) so the student sees the effect, not just its cure.

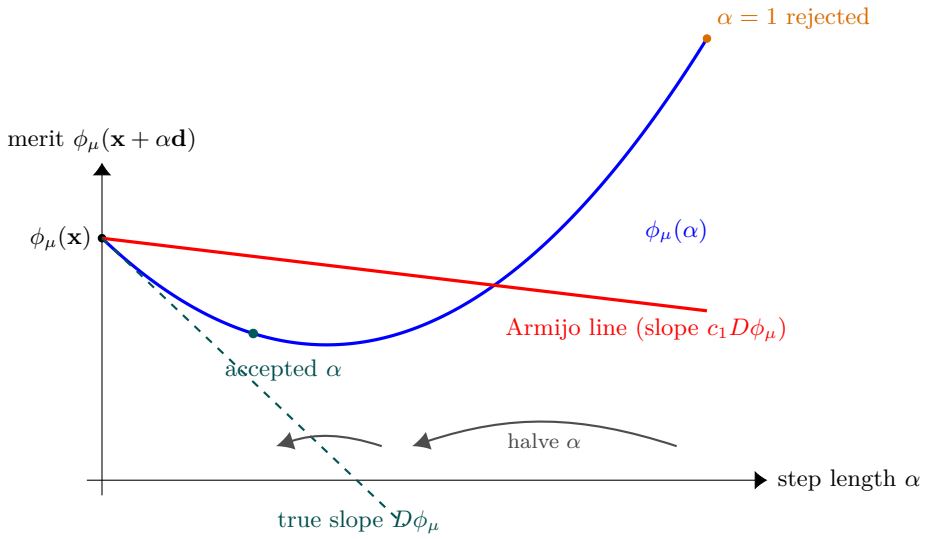

Figure 43: The Armijo backtracking line search on the ℓ_1 merit function, the SQP's "how far" decision. The QP gives a descent direction $\mathbf{d}$; we walk along it and watch the merit $\phi_\mu(\mathbf{x} + \alpha\mathbf{d})$, which trades objective against constraint violation. The full step $\alpha = 1$ overshoots the dip and sits *above* the gentle Armijo sufficient-decrease line (red, slope $c_1 D\phi_\mu$ with $c_1 = 10^{-4}$), so it is rejected; the simulator halves α until a trial point falls *below* that line — a small but genuine merit decrease, enough to guarantee progress. If α shrinks past 10^{-8} with no acceptance, SQP stops honestly rather than fake convergence — usually the finite-difference gradients have hit their noise floor.

8.6 Termination: the five-residual KKT certificate

SQP stops when the KKT conditions (204)–(207) are met within tolerance. Choupo measures, at the current iterate with the latest QP multipliers, five residuals:

$$\begin{aligned} r_{\text{stat}} &= \|\nabla_{\mathbf{x}}\mathcal{L}\|_\infty && (\le 10^{-5}), \qquad r^{\text{eq}}_{\text{feas}} = \max_j |h_j| && (\le 10^{-6}), \\ r^{\text{ineq}}_{\text{feas}} &= \max_k \max(0, g_k) && (\le 10^{-6}), \quad r_{\text{compl}} = \max_k |\mu_k\, g_k| && (\le 10^{-6}), \\ r_{\text{dual}} &= \max_k \max(0, -\mu_k) && (\ge 0). \end{aligned} \tag{218}$$

for stationarity, equality and inequality feasibility, complementary slackness, and dual feasibility. When the first four clear their tolerances the solver returns `converged`, and prints all five as a *certificate* the student can verify by hand. Active box bounds are folded in as ordinary inequality rows in the QP, so a binding bound contributes its multiplier into r_{stat} — a bound that holds is a genuine active constraint, not a free direction.

8.7 The inner active-set QP

Every SQP iteration must solve the strictly-convex QP (209). Choupo does this with a hand-rolled **active-set** method (`solver::activeSetQP`), the classical algorithm for convex QPs [62, Alg. 16.3]. The idea: maintain a *working set* W of constraints treated as equalities (all true equality rows, plus the inequality rows currently guessed active), and at each step solve the equality-constrained QP over W exactly via its KKT linear system

$$\begin{bmatrix} \mathbf{B} & A_W^\top \\ A_W & \mathbf{0} \end{bmatrix} \begin{bmatrix} \mathbf{p} \\ \boldsymbol{\lambda}_W \end{bmatrix} = \begin{bmatrix} -\mathbf{g}_W \\ -\mathbf{c}_W \end{bmatrix}, \tag{219}$$

solved by the same Gauss elimination with partial pivoting as the rest

of the simulator (§3). Then:

- If the resulting sub-step $\mathbf{p}$ keeps every *inactive* inequality feasible, take it and inspect the multipliers of the active inequality rows. If all $\lambda_W \ge 0$ the KKT conditions hold over the full set — **optimal**, return. Otherwise *drop* the inequality with the most-negative multiplier from W (it wants to move off its bound).
- If $\mathbf{p}$ would violate an inactive inequality, take the maximal feasible fraction $\alpha \in [0, 1)$ toward the nearest *blocking* constraint, stp, and *add* that constraint to W.

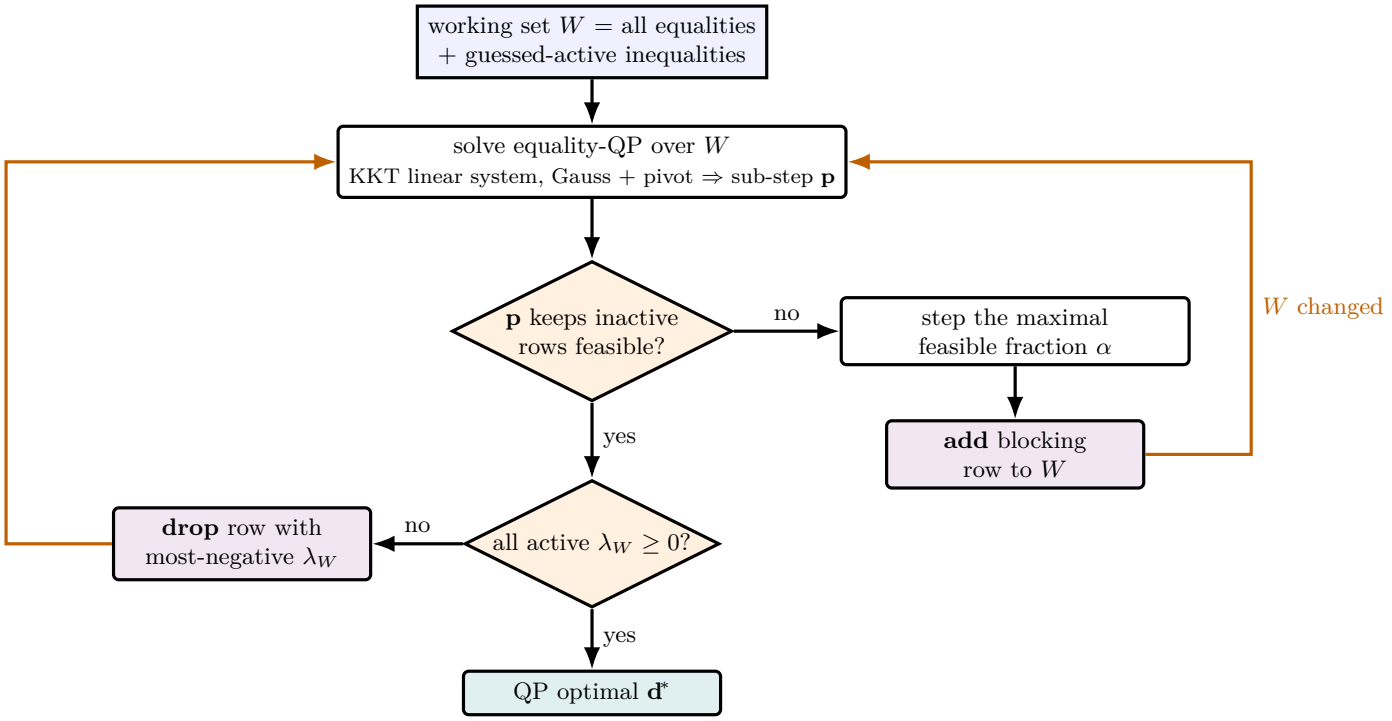

Figure 44: The inner active-set method that solves each SQP quadratic program. It maintains a *working set* W — the constraints it currently treats as equalities — and at every pass solves the equality-constrained QP over W exactly through its KKT linear system (the same Gauss-with-pivoting solver, Fig. 30). Then two questions steer it: if the sub-step would cross an inactive inequality, take the largest feasible fraction toward it and *add* that blocking row to W; otherwise check the active multipliers — if any is negative, that constraint wants to leave, so *drop* the most-negative one; if all are non-negative, the KKT conditions hold and the QP is optimal. Because $\mathbf{B}$ is strictly convex, every working-set change strictly lowers the QP objective, so the loop terminates in finitely many add/drop steps.

Strict convexity of $\mathbf{B}$ guarantees a strict decrease at every working-set change, hence *finite* termination [62, Sec. 16.5]. Box bounds $\text{lo} \leq \mathbf{d} \leq \text{hi}$ are folded in as ordinary inequality rows, so one machinery handles everything.

Honest failure modes. A singular KKT matrix (219) means the active constraint gradients are linearly dependent — a *LICQ failure*. Choupo recovers when the deficiency is among active *inequalities* (drop the most-recently-added, and forbid it from re-entering this solve, breaking the drop/re-add cycle two nearly-parallel constraints would form); if the deficiency is in the equality rows it is genuine and unrecoverable, and the solver throws a clear diagnostic rather than returning garbage. A working-set-change cap aborts loudly on suspected cycling — never a silent spin.

8.8 A worked self-check: Hock–Schittkowski

Because the SQP is finite-difference and the simulator pass is noisy, Choupo *re-proves the algorithm* before every constrained run, on three analytic problems from the Hock–Schittkowski test set [64] (`verifySQP`, a merge-blocking gate). Take **HS35** (Beale):

$$\begin{aligned} &\min_{\mathbf{x}\geq 0}\ 9 - 8x_1 - 6x_2 - 4x_3 + 2x_1^2 + 2x_2^2 + x_3^2 + 2x_1x_2 + 2x_1x_3 \\ &\text{s.t.}\ \ x_1 + x_2 + 2x_3 \leq 3, \end{aligned} \tag{220}$$

whose published optimum is $f^\star = \frac{1}{9} = 0.1111\ldots$ at $\mathbf{x}^\star = (\frac{4}{3}, \frac{7}{9}, \frac{4}{9})$. From $\mathbf{x}_0 = (\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ Choupo's SQP linearises (one constraint row plus three lower-bound rows), solves the convex QP, damped-BFGS updates $\mathbf{B}$, and converges to $|f - f^\star| < 10^{-4}$. The companion `verifyActiveSetQP` solves a hand-worked three-inequality QP whose active set $\{d_1 + d_2 \leq 1\}$, primal solution $\mathbf{d}^\star = (0, 1)$ and multipliers $\boldsymbol{\mu} = (1, 0, 0)$ are known by hand, and asserts the inner solver reproduces them to 10^{-9}. If either self-check fails the run aborts — the tolerance is never loosened to mask a bug.

Power and limit. **Power:** SQP is the method of choice for smooth, equality- and inequality-constrained NLPs — it honours hard constraints exactly at the solution, returns shadow-price multipliers for free, and converges superlinearly near the optimum because $\mathbf{B}$ approximates the Lagrangian curvature. A flowsheet DesignSpec, a purity floor, or a duty-equals-target all map directly onto (202). **Limit:** it is a *local* method (converges to the nearest KKT point — multi-start for global search), the gradients are finite-difference so it inherits the simulator's noise floor (a noisy pass poisons $\mathbf{B}$ and the merit), and it assumes the responses are smooth in $\mathbf{x}$ — a discrete decision or a phase-appearance discontinuity breaks the linearisation. For a black-box *unconstrained* objective with no derivatives, reach back for Nelder-Mead (§7); SQP earns its keep precisely when the constraints are real.

What you see in Choupo

Selecting `method sqp;` in an `outerDict` optimisation block makes `OptimizationDriver` print, per iteration, the damped-BFGS θ, the merit penalty μ, the active set, the Lagrange multipliers (shadow prices), the Armijo step α, and the four KKT residuals — the whole inner workings, not just a final answer. The self-checks `verifySQP` / `verifyActiveSetQP` run first and their pass is announced. On convergence the five-line KKT certificate is printed and the simulator is replayed once at the optimum with full verbosity, exactly as the Nelder-Mead path does.

9 Runge-Kutta 4 for the PFR

What you'll learn. The PFR model integrates concentrations along the reactor volume by the canonical fourth-order Runge-Kutta scheme. The independent variable is V, not t — a small but important reinterpretation.

9.1 The reactor as an ODE

In a steady-state PFR with molar flow rate F and reaction extent per unit volume $\dot{r}$:

$$\frac{d\mathbf{x}}{dV} = \frac{\boldsymbol{\nu} \cdot r(\mathbf{x}, T)}{F}, \tag{221}$$

with $\boldsymbol{\nu}$ the stoichiometric vector and $\mathbf{x}$ the mole-fraction vector along the reactor. The state $\mathbf{x}$ at $V = V_R$ is the outlet.

9.2 RK4 step

For a step of size $h = V_R/N$:

$$\mathbf{k}_1 = f(\mathbf{x}_n) \tag{222}$$
$$\mathbf{k}_2 = f(\mathbf{x}_n + \tfrac{h}{2}\,\mathbf{k}_1) \tag{223}$$
$$\mathbf{k}_3 = f(\mathbf{x}_n + \tfrac{h}{2}\,\mathbf{k}_2) \tag{224}$$
$$\mathbf{k}_4 = f(\mathbf{x}_n + h\,\mathbf{k}_3) \tag{225}$$
$$\mathbf{x}_{n+1} = \mathbf{x}_n + \tfrac{h}{6}\,(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{226}$$

Local truncation error is $O(h^5)$ per step; global error is $O(h^4)$. At $N = 100$ (the simulator's default) this is well below the significance of any kinetics one can realistically know.

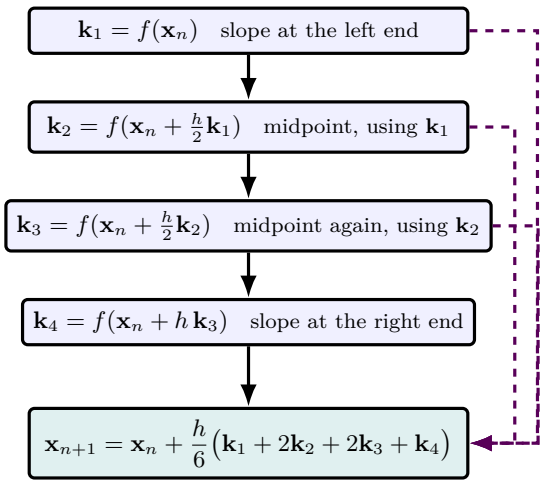

Figure 45: The four-stage structure of classical RK4 for one step $\mathbf{x}_n \to \mathbf{x}_{n+1}$ of size h (here $h = V_R/N$ marching down the PFR). Each stage is one evaluation of the right-hand side f: a slope at the left end ($\mathbf{k}_1$), two probes of the midpoint slope that reuse the previous stage ($\mathbf{k}_2, \mathbf{k}_3$), and a slope at the right end ($\mathbf{k}_4$). The solid arrows are the data dependency — each $\mathbf{k}$ feeds the next — and the dashed violet arrows show all four slopes returning to be combined with Simpson-like weights 1:2:2:1. That weighting is exactly what buys the $O(h^5)$ local accuracy from only four function calls. The same packed-state machinery drives the batch integrators of Chapter 1.

Part V — Transport properties and mixing rules

1 Why transport: where pressure drop, heat transfer, and HETP come from

What you'll learn. A flash needs no transport coefficients — the answer depends only on phase equilibrium. A packed column, a tube-side heat exchanger or a fixed-bed reactor lives or dies on viscosity, thermal conductivity and diffusivity. This Part introduces those three property families in the Reid-Prausnitz-Poling tradition (classic, non-cubic models, two or three parameters per pure component, plus a mixture rule per property), documents the planned extension of the `*.dat` schema, and points forward to the unit-operation correlations they feed.

1.1 Three coefficients, three modes of irreversibility

Thermodynamics tells you where a system wants to go. Transport tells you how fast it gets there and what it costs. Three Newtonian constitutive laws — one per mode of irreversibility — show up in every unit operation that involves flow, heat or mass transfer:

mode	flux	gradient driver	coefficient
momentum	τ (Pa)	$\partial u/\partial y$	viscosity μ
heat	q (W/m²)	$\partial T/\partial y$	thermal conductivity κ
mass (i)	J_i (mol/m²s)	$\partial c_i/\partial y$	diffusivity D_i

Newton's law of viscosity ($\tau = -\mu\,\mathrm{d}u/\mathrm{d}y$), Fourier's law of heat conduction ($q = -\kappa\,\mathrm{d}T/\mathrm{d}y$) and Fick's first law of diffusion ($J_i = -D_i\,\mathrm{d}c_i/\mathrm{d}y$) share the same linear-response form. Each transport coefficient is a *thermophysical* property of the fluid: it depends on T, mildly on P (strongly for gases above ∼10 bar), and on composition. Unlike thermodynamic properties they describe irreversible processes; they do not derive from an energy.

Intuition. A common student trap is to expect that, just like enthalpy follows from c_p, viscosity should follow from something equally fundamental. It does not. Transport coefficients ultimately come from the underlying kinetic theory — collision integrals for gases, free-volume theory for liquids — but for engineering work we use *correlations*: 2-3-parameter fits to empirical data, identical in spirit to Antoine for vapour pressure.

1.2 Where each property is consumed

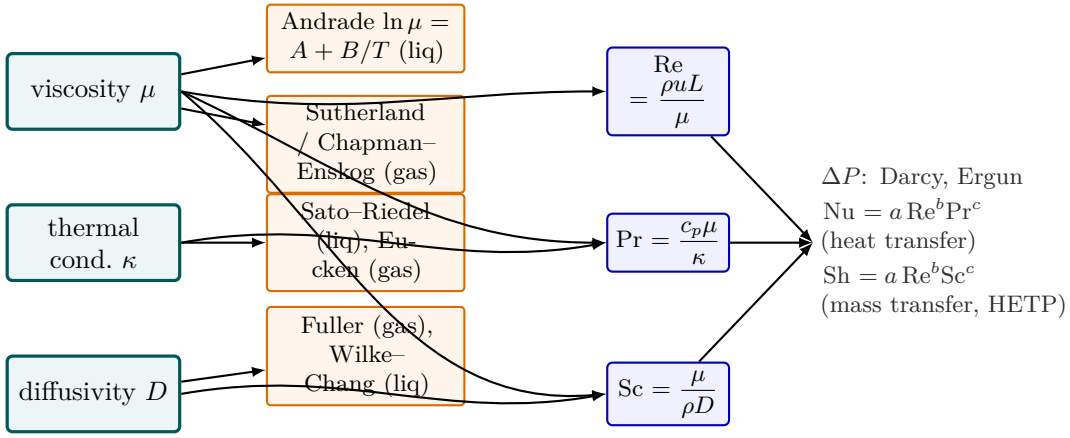

Figure 46: The transport map: three coefficients, their correlations, and where they are consumed. Each Newtonian coefficient (teal) is supplied by a two- or three-parameter correlation (orange)—Andrade or Sutherland/Chapman–Enskog for viscosity, Sato–Riedel/Eucken for conductivity, Fuller and Wilke–Chang for diffusivity. The coefficients then assemble into the three dimensionless groups Re, Pr, Sc (blue), and nearly every downstream design equation is a power law in them ($\mathrm{Nu} = a\,\mathrm{Re}^b\mathrm{Pr}^c$, $\mathrm{Sh} = a\,\mathrm{Re}^b\mathrm{Sc}^c$). Note that viscosity feeds all three groups—one good μ correlation unlocks pressure drop, heat *and* mass transfer.

property	feeds	through correlation
viscosity μ	pressure drop in tubes, packings	Darcy-Weisbach, Ergun
	shear-controlled mixing	power number correlations
	convective heat transfer	Dittus-Boelter (turbulent), Sieder-Tate
	column flooding limits	Souders-Brown coefficient
thermal cond. κ	conductive heat transfer	Fourier across pipe walls / fins
	convective heat transfer	via Nusselt = f(Re, Pr)
diffusivity D	mass-transfer coefficient	Sherwood = f(Re, Sc)
	HETP / NTU in absorption, dist.	Onda, Eduljee
	gas-side concentration polarisation	Sherwood again

Intuition. The three dimensionless groups Re ($= \rho u L/\mu$), Pr ($= c_p\mu/\kappa$) and Sc ($= \mu/(\rho D)$) capture each property's role in a heat- or mass-transfer correlation, normalised against the flow. Most empirical Nusselt or Sherwood relations are functions of the form $\mathrm{Nu} = a\ \mathrm{Re}^b\ \mathrm{Pr}^c$ or $\mathrm{Sh} = a\ \mathrm{Re}^b\ \mathrm{Sc}^c$, so *one good correlation per coefficient* unlocks the entire family of design equations downstream.

1.3 Status note

None of this is implemented yet. Pressure drops in the simulator are zero; heat-exchanger duties bypass the UA product (a Tout or Q is given, not derived from area); column efficiency is taken as 100% per stage. Part IV documents the theory and the planned data-file extension; the C++ wiring is on the v1.x roadmap. Every chapter that follows closes with a candid status note.

2 Viscosity

What you'll learn. The viscosity of a pure liquid follows the Andrade equation $\ln \mu = A + B/T$ — a two-parameter fit that captures the strong Arrhenius-like decay of liquid viscosity with temperature. Gases take the opposite trend (μ grows with T); the classical model is Sutherland. The corresponding-states alternative for either phase is the Chapman-Enskog formula in terms of Lennard-Jones σ and ϵ/k. Reid-Prausnitz-Poling Chapter 9.

2.1 Pure liquid: Andrade

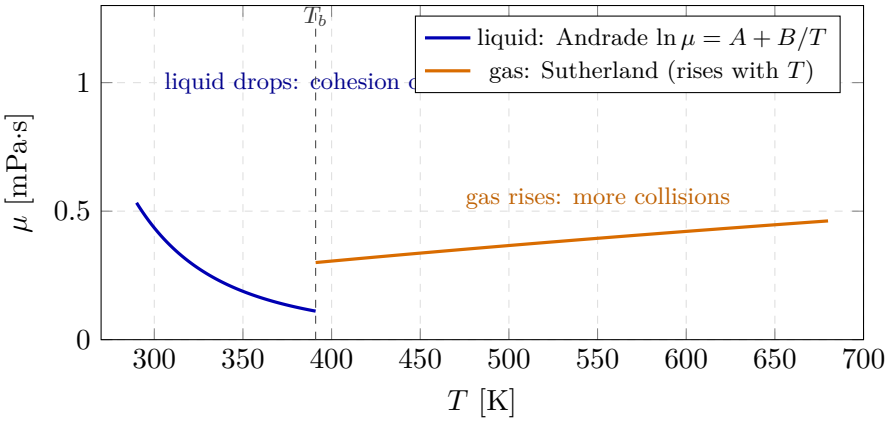

Figure 47: Liquid and gas viscosity move in *opposite* directions with temperature—the most counter-intuitive fact in transport. Below the boiling point the liquid follows Andrade ($\ln \mu = A + B/T$, blue): viscosity falls steeply as thermal energy lets molecules slip past one another. Above it the gas follows Sutherland (orange): viscosity *rises* because hotter molecules carry more momentum across the shear gradient. Curves are illustrative (acetic-acid liquid constants $A = -13.61$, $B = 1761$ K; the gas branch is a representative Sutherland shape, not to the same vertical scale).

Liquid viscosity drops sharply with temperature because molecules move past each other more easily once thermal energy starts to overcome the cohesive potential. The empirical fit is

$$\boxed{\ln \mu^{\text{liq}}(T) \;=\; A \;+\; \frac{B}{T},} \qquad \mu \text{ in Pa·s, } T \text{ in K.} \tag{227}$$

Two parameters, fitted to two or more (μ, T) data points. For the practically interesting range $0.4\,T_c < T < 0.95\,T_c$ this two-parameter form is within ∼5% of experimental viscosity. For wider ranges (cryogenic to near-critical) Reid recommends the extended four-parameter Andrade

$$\ln \mu^{\text{liq}}(T) \;=\; A \;+\; \frac{B}{T} \;+\; C\,\ln T \;+\; D\,T^{E}, \tag{228}$$

but the two-parameter form is what we ship.

Worked example 2.1. Acetic acid at 300 K and 380 KFitting Andrade to two literature data points ($\mu(300) = 1.07$ mPa · s, $\mu(380) = 0.38$ mPa · s) gives $A = -13.61$, $B = 1761$ K. At an intermediate $T = 340$ K, Eq. (227) predicts $\mu(340) \approx 0.63$ mPa · s, against a measured ≈ 0.60 mPa · s — a 5% prediction without any 340 K measurement.

2.2 Pure gas: Sutherland

Gas viscosity *increases* with temperature: more thermal energy means more momentum-carrying collisions across a flow gradient. The classical fit is Sutherland (originally derived from a square-well potential),

$$\boxed{\mu^{\text{gas}}(T) \;=\; \mu_0 \left(\frac{T}{T_0}\right)^{3/2} \frac{T_0 + S}{T + S},} \tag{229}$$

where μ_0 is the viscosity at the reference temperature T_0 (commonly 273.15 K) and S is the Sutherland constant (a positive material parameter that controls the deviation from $T^{1/2}$ behaviour of hard-sphere theory). Three parameters per component. Air: $\mu_0 = 1.716 \times 10^{-5}$ Pa · s, $T_0 = 273.15$ K, $S = 110.4$ K.

2.3 Either phase: Chapman-Enskog with Lennard-Jones

A corresponding-states alternative falls out of kinetic theory. For a dilute gas of molecules interacting through a Lennard-Jones (12–6) pair potential characterised by σ (collision diameter, in ångström) and ϵ/k (well depth normalised to Boltzmann's constant, in K),

$$\mu^{\text{gas}}(T) \;=\; 2.6693 \times 10^{-6}\,\frac{\sqrt{M\,T}}{\sigma^{2}\,\Omega_{\mu}(T^{*})} \qquad \text{[Pa·s; } M \text{ in kg/kmol; } T \text{ in K; } \sigma \text{ in Å]}, \tag{230}$$

where $T^{*} = kT/\epsilon$ and the dimensionless collision integral Ω_{μ} is tabulated (Reid Table 9-2) or approximated by the Neufeld correlation

$$\Omega_{\mu}(T^{*}) \;\approx\; \frac{1.16145}{(T^{*})^{0.14874}} \;+\; \frac{0.52487}{e^{0.7732\,T^{*}}} \;+\; \frac{2.16178}{e^{2.43787\,T^{*}}}. \tag{231}$$

The appeal of the LJ approach is that *two* parameters (σ, ϵ/k) determine viscosity, thermal conductivity and diffusivity — not separately fitted per property. Tabulations exist for most common species.

2.4 The Chung correlation — what Choupo actually uses

The Lennard-Jones form needs σ and ϵ/k, which are tabulated for only some species. Choupo instead ships the **Chung et al. (1988)** corresponding-states realisation of the SAME kinetic theory, which ESTIMATES the collision parameters from the critical properties every component already carries — so a gas viscosity follows from T_c, P_c, ω, M alone, with no separate viscosity datum:

$$\mu^{\text{gas}}(T) \;=\; 40.785 \times 10^{-7}\,\frac{F_c\sqrt{M\,T}}{V_c^{2/3}\,\Omega_{\mu}(T^{*})} \qquad \text{[Pa·s; } M \text{ in kg/kmol; } V_c \text{ in cm}^3\text{/mol]}, \tag{232}$$

with the critical volume from the acentric-corrected compressibility $V_c = Z_c RT_c/P_c$, $Z_c = 0.2918 - 0.0928\,\omega$; the reduced temperature $T^{*} = 1.2593\,T/T_c$ feeding the SAME Neufeld collision integral Ω_{μ}; and the polarity factor $F_c = 1 - 0.2756\,\omega$ (non-polar here — the factor does not capture strong dipoles, so Chung degrades for polar species). For a simple non-polar gas the accuracy is excellent: at 298 K it returns $\mu_{\text{N}_2} = 1.78 \times 10^{-5}$ Pa·s, within ∼0.3% of the measured value. The model is DECLARED (`transport { model Chung; }` in the thermoPackage) and the engine REFUSES to guess a viscosity without it — no silent default. (Water and steam take the higher-accuracy IAPWS transport formulation instead.)

2.5 Data-file extension (optional per-component override)

The Chung/Sutherland corresponding-states route above needs no per-component viscosity datum. A component MAY still carry an explicit measured correlation to OVERRIDE the estimate where higher accuracy is warranted:

```
viscosityLiq
{
    model         Andrade;
    coefficients  (A B);             // ln mu_Pa.s = A + B/T
    Trange        (270 390);
}

viscosityGas
{
    model         Sutherland;
    coefficients  (mu0 T0 S);        // [Pa.s], [K], [K]
    Trange        (200 700);
}

lennardJones
{
    sigma         3.617;             // angstrom
    epsOverK      97.0;              // K
}
```

> **Runnable case.** **Status note.** No viscosity model is wired into any unit op. When this lands (v1.x), the call site will be `ProcessStream::viscosity(T, x)` consumed by `HeatExchanger`, `PFR` (for the Reynolds number), and `DistillationColumn` (for flooding checks).

3 Thermal conductivity

What you'll learn. Liquid thermal conductivity is well represented by the Sato-Riedel correlation — a corresponding-states formula that needs only M, T_c and T_b (data the simulator already has). Gas conductivity follows from the viscosity through Eucken's relation $\kappa = (\mu/M)(c_p + 5R/4)$ or the modified Eucken with a correction for polyatomics. Reid Chapter 10.

3.1 Pure liquid: Sato-Riedel

$$\kappa^{\mathrm{liq}}(T) \;=\; \frac{1.11}{\sqrt{M}}\,\frac{3 + 20\,(1 - T_r)^{2/3}}{3 + 20\,(1 - T_{br})^{2/3}} \qquad [\mathrm{W/(m{\cdot}K)}], \tag{233}$$

where $T_r = T/T_c$, $T_{br} = T_b/T_c$ and M is in g/mol (equivalently kg/kmol). The formula contains zero fitted constants per component — it is pure corresponding states, calibrated once across the Reid-Prausnitz-Poling reference set. Accuracy: ∼8% on average across non-polar liquids, ∼15% for polar ones (water is an outlier and gets its own tabulation).

Worked example 3.1. Acetic acid at 350 KWith $M = 60.052$, $T_c = 591.95$, $T_b = 391.05$: $T_r = 0.591$, $T_{br} = 0.661$. Eq. (233) gives $\kappa \approx 0.162$ W/(m · K) against a literature value of ∼0.158 W/(m · K).

3.2 Pure gas: modified Eucken

Eucken combined heat conduction and viscosity through the kinetic energy budget of a colliding-molecule model:

$$\kappa^{\mathrm{gas}}(T) \;=\; \frac{\mu(T)}{M}\left[\,c_p(T) + \tfrac{5}{4}R\,\right] \qquad [\mathrm{W/(m{\cdot}K)}]. \tag{234}$$

The factor $\frac{5}{4}R$ corrects an Euler-type derivation that otherwise overpredicts κ for monatomics. Polyatomic molecules carry extra internal degrees of freedom; the *modified Eucken* replaces the kernel with $(c_p + 1.32R)$ or similar. Either way, gas thermal conductivity is a derived quantity in this scheme — no new fitted parameter beyond those already in μ and c_p.

3.3 Data-file extension (v1.x)

Sato-Riedel needs no coefficients; modified Eucken needs none either once μ and c_p are in place. The data file just declares which model to use:

```
thermalConductivityLiq  { model SatoRiedel; }
thermalConductivityGas  { model modifiedEucken; }
```

Runnable case. **Status note.** κ is not consumed by any unit op. When the cubic EOS work brings real-gas heat exchangers (v1.x), the call site will be `ProcessStream::kappa(T, x)`.

4 Diffusivity

What you'll learn. Diffusivity is fundamentally a *pair* property: it describes the motion of species i relative to species j, and an isolated component cannot carry a single number. For binary diffusion in gases the workhorse is Fuller-Schettler-Giddings; for binary diffusion in dilute liquids it is Wilke-Chang. Multicomponent extensions (Vignes, Maxwell-Stefan) build on the binary values. Reid Chapters 11–12.

4.1 Gas binary: Fuller-Schettler-Giddings

$$D_{AB}(T,P) \;=\; \frac{1.013\times10^{-7}\,T^{1.75}\,[(M_A+M_B)/(M_AM_B)]^{1/2}}{P\left[(\Sigma v)_A^{1/3}+(\Sigma v)_B^{1/3}\right]^2} \qquad [\mathrm{m^2/s};\ T \text{ in K},\ P \text{ in bar},\ M \text{ in g/mol}]. \tag{235}$$

The $(\Sigma v)_i$ are *atomic diffusion volumes*: a small table of element-additive contributions (C: 16.5, H: 1.98, O: 5.48, N: 5.69; complete tables in Reid 11-1). No experimental binary diffusivity is needed — the prediction is geometric, from atomic group contributions only. Accuracy: ∼10% for gases at ∼1 bar; degrades sharply above 10 bar where high-pressure corrections become necessary.

4.2 Liquid binary, dilute: Wilke-Chang

$$D_{AB}^{\infty} \;=\; 7.4\times10^{-12}\,\frac{(\phi_B\,M_B)^{1/2}\,T}{\mu_B\,V_A^{0.6}} \qquad [\mathrm{m^2/s};\ T \text{ in K},\ M \text{ in g/mol},\ \mu_B \text{ in Pa}\cdot\text{s},\ V_A \text{ in cm}^3\text{/mol}], \tag{236}$$

where V_A is the molar volume of solute A at its boiling point (LeBas group-contribution table, or just V_A^{liq} from the data file as a first approximation) and ϕ_B is the association factor of solvent B (water: 2.6; methanol: 1.9; ethanol: 1.5; benzene/other unassociated: 1.0). The result is the *infinite-dilution* diffusivity — valid as $x_A \to 0$.

Intuition. Liquid diffusivities are six to seven orders of magnitude lower than gas diffusivities at 1 bar ($\sim 10^{-9}$ vs $\sim 10^{-5}$ m²/s). This single fact is why almost every mass-transfer-limited unit op is dominated by the liquid film resistance.

4.3 Composition dependence: Vignes

Wilke-Chang gives only the limiting values D_{AB}^{∞} (solute A in solvent B) and D_{BA}^{∞} (the reverse). Across the full composition range Vignes interpolates logarithmically:

$$D_{AB}(x_A) \;=\; \left(D_{AB}^{\infty}\right)^{1-x_A}\left(D_{BA}^{\infty}\right)^{x_A}. \tag{237}$$

For multicomponent systems the Maxwell-Stefan framework promotes each binary D_{ij} to a matrix that combines them rigorously. Reid Chapter 12 treats this in depth; we will not need it before v1.x of the simulator.

4.4 Data-file extension (v1.x)

Single-component files carry no diffusivity entries (diffusion is a pair property). Binary pairs get their own files alongside `constant/binaryPairs/`:

```
data/standards/diffusionVolumes/ethanol.dat  // for Fuller
{ sumV  50.36; }                              // atomic-volume sum

data/standards/associationFactors/water.dat   // for Wilke-Chang
{ phi   2.6; }
```

Runnable case. Status note. Diffusivity is unused. When mass-transfer correlations land (post-v1, for the membrane/NF chapter that motivates the project) the call site will be `ProcessStream::Dij(T, P, x, i, j)`.

5 Mixture rules

What you'll learn. Pure-component transport properties are useless on their own — every real stream is a mixture. Reid-Prausnitz-Poling Chapter 9 recommends Wilke's rule for gas viscosity, a log-mole-fraction average for liquid viscosity, a mass-fraction average for liquid thermal conductivity, and the Vignes interpolation for binary diffusivity. Each rule has a domain of validity and a ten-line implementation.

5.1 Gas viscosity: Wilke

$$\mu_{\text{mix}}^{\text{gas}} \;=\; \sum_i \frac{y_i\,\mu_i}{\sum_j y_j\,\phi_{ij}}, \qquad \phi_{ij} \;=\; \frac{\left[1+(\mu_i/\mu_j)^{1/2}(M_j/M_i)^{1/4}\right]^2}{\left[8(1+M_i/M_j)\right]^{1/2}}. \tag{238}$$

The ϕ_{ij} are dimensionless interaction terms. Reid reports $\sim$2% accuracy for binary gas mixtures across the full composition range, with the rule reducing exactly to μ_i in the pure-component limit by construction.

5.2 Liquid viscosity: log-mole-fraction average

$$\ln \mu_{\text{mix}}^{\text{liq}} \;=\; \sum_i x_i\,\ln \mu_i. \tag{239}$$

The motivation is empirical (Arrhenius rule for ideal liquid mixtures) but the form is well behaved: it is exact when the mixing is athermal and the simplest non-trivial answer otherwise. For strongly hydrogen-bonded mixtures the Grunberg-Nissan correction adds a binary interaction term $\sum_i \sum_j x_i x_j G_{ij}$ but we will not need it.

5.3 Liquid thermal conductivity: mass-fraction average

$$\kappa_{\text{mix}}^{\text{liq}} \;=\; \sum_i w_i\,\kappa_i, \qquad w_i \;=\; \frac{x_i M_i}{\sum_j x_j M_j}. \tag{240}$$

The mass weighting is empirical; Filippov's binary correlation is more accurate but needs an interaction term per pair. For engineering accuracy mass-fraction mixing is enough.

5.4 Gas thermal conductivity: Wassiljewa with Mason-Saxena

Same shape as Wilke for viscosity, with ϕ_{ij} rebuilt from the ratio of thermal conductivities:

$$\kappa_{\text{mix}}^{\text{gas}} \;=\; \sum_i \frac{y_i\,\kappa_i}{\sum_j y_j\,A_{ij}}. \tag{241}$$

The A_{ij} are usually approximated as ϕ_{ij} of Wilke — the kinetic-theory connection between μ and κ ensures the two rules share the same interaction structure.

5.5 Diffusivity in multicomponent: Vignes plus Maxwell-Stefan

The binary Vignes formula (237) handles ternary and beyond if generalised to a full Maxwell-Stefan diffusivity matrix $\left[D\right]$ whose i,j entries come from the binaries D_{ij}^{∞}. Practical implementation requires inverting that matrix to recover Fick's-law-equivalent diffusivities. This is non-trivial; Reid Chapter 12 covers it in depth and we will adopt his Eq. (12-3.5) when v1.x of the simulator needs it.

Intuition. Notice the pattern: every mixture rule reduces to the pure-component value when $x_i = 1$, and most are weighted means (linear, logarithmic, or harmonic) of the pure values with an interaction correction. When in doubt, the simulator should default to the weighted mean and warn the user; refinements (Grunberg-Nissan, Filippov, Maxwell-Stefan) get switched on by an explicit `mixingRule` key in the dict.

6 Where transport feeds the downstream chapters

What you'll learn. A brief look forward: once the transport properties of the previous chapters are wired in, they unlock pressure-drop, heat-transfer and mass-transfer correlations in the unit operations. This chapter sketches each consumer without rigorously deriving it — those correlations live in Part V (unit operations) and the post-processing chapters — so you know where the cables go before v1.x starts plugging them.

6.1 Pressure drop

- **Pipes (single phase):** Darcy-Weisbach, $\Delta P = f\,\rho u^2 L/(2D)$, with $f = f(\mathrm{Re}, \epsilon/D)$ via Colebrook or Churchill. $\mathrm{Re} = \rho u D/\mu$ ties ΔP directly to viscosity.
- **Packed beds (catalyst pellets, structured packing):** Ergun, $\Delta P/L = 150(1-\varepsilon)^2\mu u/(\varepsilon^3 d_p^2) + 1.75(1-\varepsilon)\rho u^2/(\varepsilon^3 d_p)$. Viscous and inertial terms, both viscosity-dependent in the laminar regime.
- **Two-phase flow (column trays, slug flow):** Lockhart-Martinelli phenomenological correlation in $X^2 = (\Delta P_L/\Delta P_G)$, both legs computed with their own viscosity.

6.2 Heat-transfer coefficients

Convective h on either side of a tube wall comes from a Nusselt correlation $\mathrm{Nu} = f(\mathrm{Re}, \mathrm{Pr})$. Two workhorses:

- **Dittus-Boelter (turbulent, smooth tubes):** $\mathrm{Nu} = 0.023\,\mathrm{Re}^{0.8}\,\mathrm{Pr}^n$, with $n = 0.4$ for heating, 0.3 for cooling.
- **Sieder-Tate** (same with a wall-viscosity correction $(\mu/\mu_w)^{0.14}$ for liquids with steep $\mu(T)$).

With Nu in hand, $h = \mathrm{Nu}\,\kappa/D$ and the overall coefficient U falls out of the standard series resistance $1/(UA) = 1/(h_i A_i) + R_{\mathrm{wall}} + 1/(h_o A_o)$. The heat-exchanger sizing chapter (Part VI Chapter 1's sibling) currently uses a hardcoded U; with transport in place, U will be a result, not a constant.

6.3 Mass-transfer coefficients, NTU and HETP

For absorption or distillation, the mass-transfer film coefficient k_c comes from a Sherwood correlation $\mathrm{Sh} = f(\mathrm{Re}, \mathrm{Sc})$ of the same shape as Nu(Re, Pr). HETP for a packed column then follows from Onda's or Eduljee's correlations. For the distillation column each stage is assumed perfectly efficient (Murphree $E_{\mathrm{MV}} = 1$); the v1.x extension will introduce E_{MV} as a Sherwood-derived quantity, varying with composition and flow regime.

Runnable case. Status note (consolidated). No transport coefficient is read or computed by any unit op. Pressure drops are zero, heat-exchanger U is constant (or absent — the user supplies T_{out} or Q), column efficiency is 100% per stage. This Part is the theoretical scaffolding for the v1.x rollout; the data-file extensions listed in Chapters 2–4 are the contract the implementation will honour.

Part VI — Selected unit operations

1 A unit operation is a physical equipment

What you'll learn. The framing Choupo adopts for every unit operation in this Part: the **unit op object models a real piece of hardware** (a vessel with a heat-exchange surface, a column with N trays, a tube of length L), with inputs the engineer can point to in the plant ("this feed line", "this heating-steam header") and outputs the engineer can sample ("this concentrated stream", "this distillate drum"). Anything that is *not* a property of the equipment itself — a target outlet concentration, an equal-area design rule, a desired conversion — is handled by an outer **DesignSpec controller** that wraps the simulation, not by the unit-op spec.

1.1 The friction we want to remove

A student writing her first single-effect evaporator dict tends to freeze on the question "*which* variable do I specify — duty, vapour fraction, area, both?" The same paralysis returns for every unit op in a typical undergraduate flowsheeting course: distillation columns asking the student to choose between "R, N and F" or "$D/F, R$ and N" or "D, R and N"; heat exchangers asking "T-out or duty"; reactors asking "volume or X_A". These degree-of-freedom puzzles are mathematically equivalent and every chem-eng curriculum eventually explains why — but in the first *week* of a course on simulators, the puzzle is friction. The student does not yet own the mental model that justifies the choice; she fills out some boxes, gets an unreasonable answer, and concludes (often correctly) that "the simulator is being weird again".

Choupo takes a different stance, learned from twenty-plus years of Choupo's convention: the dict that defines a unit op describes the **hardware**, not a math problem. An evaporator is a heat-exchange surface of area A with an overall transfer coefficient U operating at a process-side pressure P and heated by a stream coming in on the chest. A heat exchanger is a tube bank of area A with a coefficient U. A distillation column has N trays at a feed stage f with a reflux ratio R. A reactor is a vessel of volume V at temperature T with a named reaction. None of these dicts contains the word "target" or "conversion" or "V/F". Those are *outcomes* of the simulation, not inputs to it.

Concretely, the evaporator (and the case files in `tutorials/steady/evaporation/evaporator0*`) keeps the operation block to four scalars:

```
operation
{
    P     1.0  bar;
    area  20.0 m2;
    U   2000.0 W/m2/K;
    Tref 298.15 K;          // optional
}
```

and the student reads off "V/F", "Q", "T_boil", "steam demand" and "economy" from the run log. The mode-switch between `duty` and `vaporFraction` that carried is *gone* — a single physical specification in, all the operating quantities out.

1.2 Where this idea comes from

The "unit op = physical equipment" stance is older than sequential-modular simulation itself. Rudd & Powers' *Process Synthesis* [67] opens with the observation that a process flowsheet is a directed graph of *real-world equipment connected by real-world streams* — the design problem is then framed as choosing the equipment shape and the stream topology, not as solving an algebraic system. Westerberg *et al.*'s *Process Flowsheeting* [68] formalises this view: every node in the graph is an object that owns its connections, owns its sizing parameters, and exposes a small, fixed set of physical inputs to the user. The book makes the further claim — still true — that "the user should not be required to know which variables the solver will choose to iterate on".

This is the standard layered model, where each unit op block has a small fixed set of physical parameters and external *Design Specs* sit on top. "Calculator"- and "Design-Specification"-style blocks let the engineer write "manipulate the column reflux ratio so that the distillate methanol mole fraction equals 0.95" *without* altering the column block itself — the block still owns R, N, f; the Design Spec owns the targeting. Stephanopoulos' textbook treatment of the sequential-modular paradigm [69] draws this distinction explicitly: *operating parameters* belong on the block (the engineer sets the hardware), *design constraints* belong on a controller (the engineer sets the target).

Biegler, Grossmann, and Westerberg's *Systematic Methods* [70] go on to formalise the **decomposition** that this layering enables: the inner problem ("simulate this fixed hardware with these fixed inputs") is well-posed and fast, the outer problem ("find the hardware that satisfies these constraints") is the *design* problem, and the two solvers do not need to know about each other's variables. Choupo is built on the same decomposition: the unit ops in this Part are inner-problem solvers; the `DesignSpec` driver scheduled for is the outer problem, and it manipulates the same dict-level scalars that the user could have edited by hand.

1.3 What the student sees

In each unit-op section below, the dict block carries only physical quantities:

- **Flash**: T, P and a phase set (the vessel sees the same T, P on inlet and outlet by definition; nothing computed enters the dict).
- **Evaporator**: P on the process side, area A, overall HTC U, and (via the heating-steam input) the chest temperature. V/F, Q, T_{boil} and the steam demand are RESULTS.
- **Heat exchanger**: hot-side T_{in}, cold-side T_{in}, UA (and a flow arrangement); duty Q and both outlet temperatures come out.
- **CSTR / PFR**: vessel volume V (or tube length L + cross-section A), T (isothermal) or thermal mode, plus a named reaction. Conversion and outlet composition are results.
- **Distillation column**: N, f, R, D (or some physical analogue). Tray temperatures and the full $\mathbf{x}, \mathbf{y}$ profile come out.

and the student's homework consists of **describing the hardware**, not solving the math. Where the assignment is "find the area such that the outlet concentration is 25%", that statement maps onto a `DesignSpec` on top of the unit op — the same way a real plant engineer thinks: I have an evaporator with area A; the question is, "*what A* delivers the spec the customer wrote into the PO?".

> *Intuition.* The discriminator between an operating parameter (lives on the unit op) and a design target (lives on an outer DesignSpec) is the question: *would you walk out to the plant and measure this with a wrench or a thermometer?* If yes, it is hardware — write it on the block. If you would only see it on the DCS trend afterwards, it is a result — let the simulator compute it and a DesignSpec target it.

2 A stream is a thermodynamic state, not five numbers

What you'll learn. The companion framing for the unit-op pattern of §1: a process **stream** is the thermodynamic state of a fluid at a point in the plant. Every state has a fixed number of degrees of freedom, and the engineer specifies just enough of them — not all at once — to fix the state uniquely. The simulator computes the rest, including a sanity-check pass for the variables left unspecified. This "flash on import" is what lets the case file carry only the *given data* of a problem instead of pre-computed answers.

2.1 Degrees of freedom for a single-phase pure-component stream

For one mole of fluid at thermodynamic equilibrium, the Gibbs phase rule fixes the variance. For a pure ($C = 1$) single-phase ($\pi = 1$) system,

$$F = C - \pi + 2 = 2,$$

so two intensive variables (any two of T, P, density, molar enthalpy, ...) fix the state completely. Multiply the stream by the extensive total molar flow F_{total} and you have the full description: $\{F_{\text{total}}, T, P\}$ (3 numbers; one extensive + two intensive).

If the stream is two-phase (still $C = 1$, $\pi = 2$, $F = 1$), one intensive plus the phase fraction β is enough — the other intensive collapses to the saturation curve $P_{\text{sat}}(T)$. So $\{F_{\text{total}}, T \text{ or } P, \beta\}$.

2.2 The Choupo stream record carries six fields

The C++ struct (`src/streams/ProcessStream.H`) is:

```
struct ProcessStream {
    std::string name;
    scalar      F;          // total molar flow      [kmol/s]
    scalar      T;          // temperature            [K]
    scalar      P;          // pressure               [Pa]
    sVector     z;          // mole fractions       (Sigma = 1)
    scalar      vf;         // vapour fraction        [-]
    bool        demandDriven;   // utility-tap marker
};
```

For a multi-component stream the degrees of freedom are $C - \pi + 2 + (C - 1) = 2C - \pi + 1$ if you count the composition (the $C - 1$ independent mole fractions). Choupo's record exposes T, P, $\mathbf{z}$, β explicitly — one intensive variable too many for a single-phase stream, exactly right for a two-phase stream, and one too few only if the user also wants enthalpy and entropy stored explicitly (Choupo recomputes those from the equation-of-state layer on demand).

2.3 Specification + consistency check on import

The case-file syntax (`system/flowsheetDict`) lets the user write *any consistent subset* of the state variables, with the simulator filling in the rest. The most-used patterns are:

What you write	What the simulator does	When it applies
T + P + $\mathbf{z}$	checks if 1- or 2-phase; fills β	general process feed
P + `state saturatedVapour`	inverts $P_{\text{sat},i}(T) = P$; $\beta = 1$	heating steam, condensate
P + `state saturatedLiquid`	same inversion, $\beta = 0$	cooling water, refrigerant
T + `state saturatedVapour`	evaluates $P_{\text{sat},i}(T)$; $\beta = 1$	utility-side temperature
T + P + `state subcooled`	validates $T < T_{\text{sat}}(P)$; $\beta = 0$	feed integrity check
T + P + `state superheated`	validates $T > T_{\text{sat}}(P)$; $\beta = 1$	post-heater check

The `state` keyword tags the expected phase behaviour; the simulator either *computes* the missing variable from the saturation curve or *validates* the declared values against it. Three failure modes are caught at import time:

- Declaring `state saturatedVapour` with T and P that disagree with $P_{\text{sat}}(T)$ by more than 5 %: thrown with a message naming the stream, the declared T, P, and the actual $P_{\text{sat}}(T)$.
- Declaring `state subcooled` with $T > T_{\text{sat}}(P)$: a physical contradiction; thrown.
- The `state` keyword is PURE-component only (it inverts a single $P_{\text{sat},i}$). For a *mixture* saturated or two-phase state, use `vaporFraction` (next subsection), which is mixture-aware.

2.4 `vaporFraction`: the general two-spec rule (why β is often essential)

The `state` keyword above is convenient but pure-only. The general, mixture-aware spec is: **a stream is fixed by z + F + exactly TWO of** {T, P, `vaporFraction`} (vaporFraction is the vapour fraction β; the legacy key `vf` is an alias). The engine resolves whichever pair you give:

What you write	What the engine does	Typical use
T + P	Rachford–Rice flash → β	general feed (single- or two-phase)
P + `vaporFraction`	solve T so the flash gives β	saturated-liquid (β=0) / vapour (β=1) feed
T + `vaporFraction`	solve P so the flash gives β	feed pinned at a known temperature
all three	**REFUSED** (over-specified)	—
only one	**REFUSED** (under-specified)	—

> *Intuition.* Why `vaporFraction` is not optional sugar but often *essential*: on the phase boundary, T and P are **not independent**. For a pure component a two-phase state has $P = P_{\text{sat}}(T)$, so (T, P) is degenerate — you *cannot* pin a saturated state with T and P; you must give β with one of them. For a mixture (T, P) between bubble and dew does fix β uniquely, but `vaporFraction` 0 (or 1) is the unambiguous way to say "saturated liquid (or vapour) feed" and let the engine find the bubble (or dew) point. This is the most common point of confusion in stream specification, and the over-/under-specification refusals above exist so the engine never silently resolves an ill-posed request.

The resolver (`Flowsheet.cpp::solveStateForVf`) brackets and bisects on the (monotone) flash β — β=0 is the bubble point, β=1 the dew point, in between a flash-at-β — and announces the resolved $(T, P, \beta, \text{phase})$ loudly. **A feed's thermal state lives in the STREAM, never duplicated on the consuming unit.** A distillation column reads its feed quality $q = 1 - \beta$ directly from the inlet stream (§29); a redundant `operation.feeds.quality` that *disagrees* with the stream is refused at import, so the two can never drift apart (the silent "feed declared liquid but flashing to vapour" bug that broke an early multi-feed energy balance).

2.5 The flow F: extensive, source-driven vs. demand-driven

For feed streams the case author always knows F — it is the basis of the design problem. For utility streams (heating steam, cooling water) the situation is reversed: the engineer knows the *conditions* (pressure header, temperature) but the *quantity* is whatever the consuming unit operation demands. Choupo allows omitting F when the stream is declared with `state saturatedVapour` or `state saturatedLiquid`; the simulator marks the `ProcessStream::demandDriven` flag and waits for the consuming unit op to publish its F_{steam} via `consumedInputs()`. After the unit's `solve()` returns, the Flowsheet writes the demanded F back into the stream registry, so the Streams panel (and the structured JSON output) shows the actual consumption rather than a placeholder zero.

> *Intuition.* The mental model the student should leave with: a stream declaration is a *question* asked of the thermodynamic package ("what state is this fluid in?"), not a five-number data dump that has to be filled out completely. This is the standard contract, and Choupo adopts it too. The pedagogical payoff is that the case file shows the engineer's intent — *this stream is the live steam at 170 kPa from the plant header* — and not the result of a pre-computed arithmetic exercise (Antoine inversion to find $T = 388.36\,\text{K}$).

2.6 Where in the code this happens

The single entry point for stream import is `readSourceStream()` in `src/unitOperations/flowsheet/Flowsheet.cpp`, called by `Flowsheet::initialiseFromDict()` once per top-level stream in `flowsheetDict.streams {... }`. Its work breaks into three numbered phases visible in the source:

1. **Composition.** Read one of the five composition flavours (`molarComposition`, `massComposition`, `composition`, `molarFlows`, `massFlows`). Convert to canonical mole-fraction vector `z`; record the sum of per-species flows when applicable.
2. **T, P, β via `state`.** If the `state` key is present, find the dominant pure component (the one with $x_i \geq 0.999$; mixtures fall through to the explicit-T-and-P branch). Invert Antoine if needed via `invertPsat()` — a Newton-1D on $f(T) = P_{\text{sat}}(T) - P$ that converges in $\sim 5{-}10$ iterations because P_{sat} is monotone increasing.
3. **F.** Pick the source: $F_{\text{from per-species}}$, the explicit `F` entry, or *omit* (`demandDriven = true`) when the stream is utility-shaped.

A failed Antoine inversion (component below its triple point, above its critical point, or with an Antoine block whose Trange does not cover the given P) throws a runtime_error with a message that names the stream, the dominant component, and the out-of-range value.

3 Isothermal flash, vapour–liquid

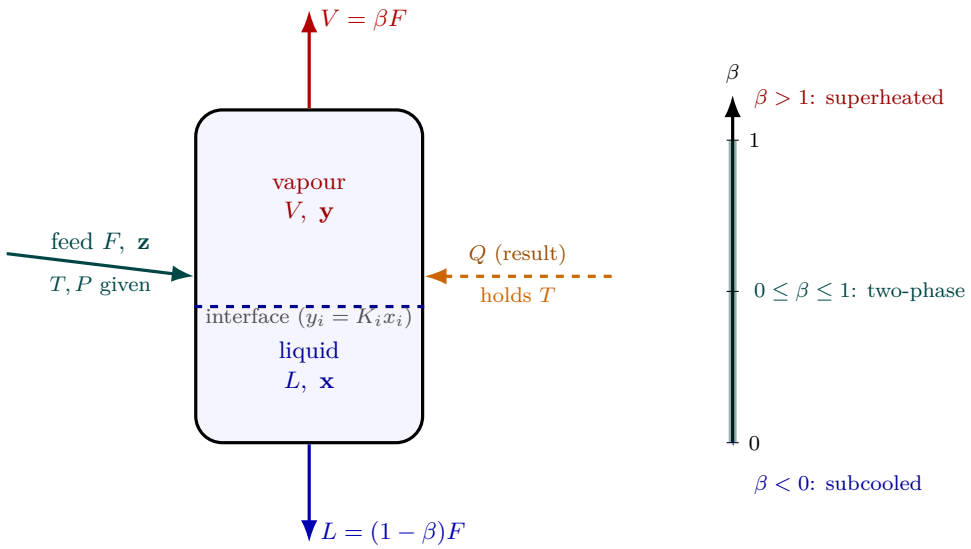

Figure 48: The flash drum as hardware. One feed $(F, \mathbf{z})$ at the specified (T, P) enters and disengages into a vapour $V = \beta F$ off the top and a liquid $L = (1-\beta)F$ off the bottom, the two phases meeting at the interface where $y_i = K_i x_i$. The vapour fraction $\beta = V/F$ is *not* an input — it is the Rachford–Rice root forced by the mass balance. The duty Q (dashed, orange) is whatever heat must cross the wall to *hold* T while part of the feed changes phase: a *result* of the converged energy balance, shown as a stream, never a hidden number. The ruler on the right reads β back as a regime tag: a β outside $[0, 1]$ is the honest signal that the feed is single-phase (subcooled or superheated) at the chosen T, P.

> *What you'll learn.* The end-to-end algorithm the simulator runs for a V-L flash: outer Wegstein loop on γ, inner Newton on β, exit conditions and diagnostics.

3.1 Problem statement

Given: $\mathbf{z}$, T, P, a thermo package (Antoine, activity model).
Find: β, $\mathbf{x}$, $\mathbf{y}$ such that (156) and (159) hold simultaneously.

3.2 Degrees of freedom: what you specify, what the flash computes

A flash vessel takes *one* feed and splits it, at equilibrium, into a vapour and a liquid product. Counting variables against equations tells you exactly how many numbers the engineer must fix — and which ones the simulator is then free to move. This is the single most common point of confusion, so we spell it out.

The feed is given, not a degree of freedom. Its total flow F, composition $\mathbf{z}$ (with $\sum_i z_i = 1$, hence $C-1$ independent mole fractions), and thermal state all arrive on the inlet stream. Nothing about the feed is yours to choose at the flash — it is whatever the upstream unit delivers.

With the feed fixed, exactly two degrees of freedom remain. The unknowns of the split are $\{\beta,\ \mathbf{x},\ \mathbf{y},\ T,\ P,\ Q\}$. Against them stand the equilibrium relations $y_i = K_i x_i$, the C component balances $z_i = \beta\, y_i + (1-\beta)\, x_i$, the summation condition $\sum_i x_i = \sum_i y_i$ (the Rachford-Rice constraint), and the energy balance $Q = H_{\text{out}} - H_{\text{in}}$. The bookkeeping leaves *two* free. This is *Duhem's theorem*: the equilibrium state of a closed system of known overall composition is fixed by exactly two independent variables, regardless of how many components or phases are present. (Equivalently via Gibbs' phase rule: a two-phase C-component system has $\mathcal{F} = C - 2 + 2 = C$ intensive degrees of freedom; the feed composition consumes $C-2$ of them through the lever rule, leaving two.) You close those two by specifying *any two* of:

You specify	Flash type
T and P	*isothermal* flash — Choupo's `isothermalFlash`
Q and P	*adiabatic* ($Q = 0$) or fixed-duty flash; T is found
β and P	fixed vapour fraction (V/F); T is found

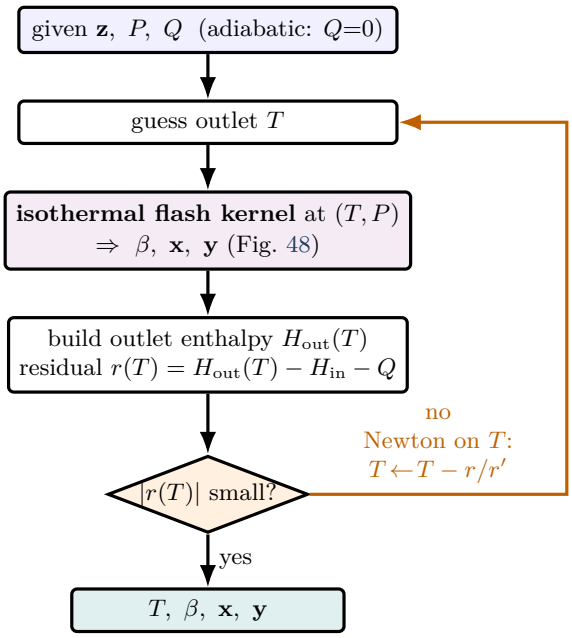

Figure 49: The adiabatic (or fixed-duty) flash is the isothermal flash wearing one more loop. Here T is *not* given — the spec is the duty Q (zero for a true adiabatic throttle) and the pressure P. The outer Newton (orange) searches for the outlet temperature: at each trial T it runs the *whole* isothermal kernel of Fig. 48 (itself two loops) to get the split, builds the outlet enthalpy $H_{\text{out}}(T)$, and forms the energy residual $r(T) = H_{\text{out}}(T) - H_{\text{in}} - Q$. Newton on T drives r to zero — the temperature at which the phase split absorbs exactly the specified duty. The same kernel, reused: an isothermal flash fixes T and *reports* Q; the adiabatic flash fixes Q and *finds* T. This is Duhem's two degrees of freedom, closed from the other side.

Specified vs. manipulated, for the isothermal flash. In the `operation` block you **specify** T and P. The simulator then **manipulates** the remaining unknowns until every equation holds:

- the vapour fraction $\beta = V/F$ (the root of Rachford-Rice),
- the phase compositions $\mathbf{x}$ and $\mathbf{y}$,
- the heat duty Q that must cross the boundary to *hold* T constant while part of the feed changes phase — read off the converged energy balance, $Q = H_{\text{out}} - H_{\text{in}}$. On the flowsheet this Q appears as the heat *stream* feeding the unit (a docked utility stub + dashed wire), never as a hidden number.

Where the duty lives — a flash *gives* a Q, a heater *takes* one. Because T, P already spend both degrees of freedom, Q is a *result* of the isothermal flash, never an input — the `operation` block has no Q key (it was removed on purpose). The unit whose hardware knob *is* the duty is the `heater`: there you **specify** Q and the outlet T is the result. So "inject a known 500 kW, then separate" is a `heater` → `flash` chain (the heat enters on a visible wire), and "hold the outlet at a target T" is a *design spec* that manipulates the heater's Q in a visible outer loop — exactly as outlet pressure is handled on a compressor (specify W_{shaft}, target P_{out}). This is the "heat is a stream, not a hidden number" rule applied to the flash.

> *Intuition.* The trap is to count T, P *and* β as three inputs. Once T and P are set, β is *not* free — it is whatever the two-phase mass balance forces, and it may even land outside $[0, 1]$ (subcooled liquid or superheated vapour, see the regime tags below). Symmetrically, the duty Q is *never* specified for an isothermal flash; it is the answer, not the question. Pick the two specifications that match the equipment: a flash drum held by a thermostat is T, P; an adiabatic throttle is Q=0, P; a partial condenser sized for a target recovery is β, P.

3.3 Algorithm

1. Initialise $\mathbf{K}$ from Wilson (160).
2. Solve Rachford-Rice (159) for β by Newton-1D.
3. Compute $\mathbf{x}, \mathbf{y}$ from (158).
4. Compute $\boldsymbol{\gamma}^{(k+1)} = \gamma(\mathbf{x}, T)$.
5. Update $K_i = \gamma_i \, P_i^{\mathrm{sat}}(T)/P$.
6. Wegstein-accelerate $\mathbf{K}$ (component-wise; (194)).
7. If $\|\mathbf{K}^{(k+1)} - \mathbf{K}^{(k)}\|_\infty < \varepsilon$, exit; else goto 2.

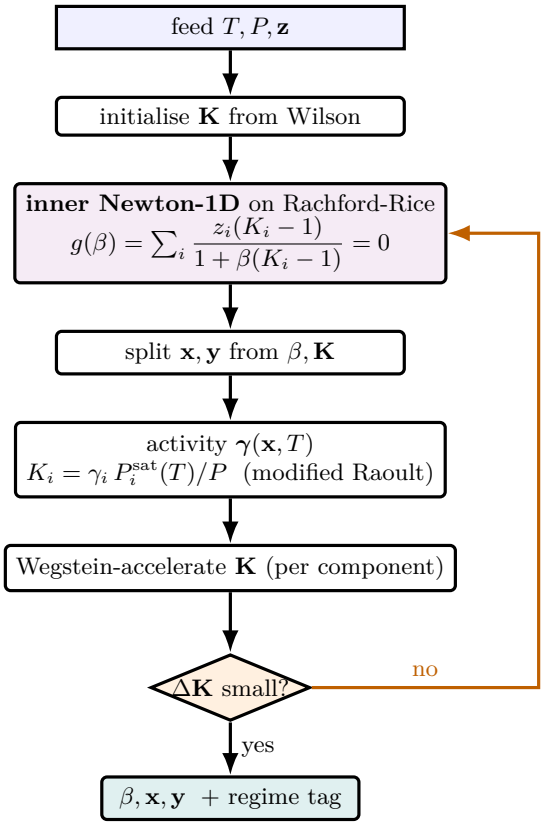

Figure 50: The isothermal V–L flash is *two nested loops*. The **inner** loop (violet) is a Newton-1D that solves the Rachford–Rice equation for the vapour fraction β at the current K-values — a clean monotone root. The **outer** loop (orange) updates the K-values: from the trial split it re-evaluates the activity coefficients $\gamma_i(\mathbf{x}, T)$, rebuilds $K_i = \gamma_i P_i^{\mathrm{sat}}/P$ through modified Raoult, and Wegstein-accelerates them. When the K-values stop moving the two loops are mutually consistent and the flash is converged; the final β also classifies the regime (subcooled $\beta < 0$, two-phase $0 \le \beta \le 1$, superheated $\beta > 1$). For an *ideal* system the activity step is a no-op and only the inner loop runs once — the same code, fewer passes.

3.4 Diagnostics in the log

At `verbosity 3`, the simulator prints:

- Inner Newton trace on Rachford-Rice: $(\beta^{(j)}, g(\beta^{(j)}), g'(\beta^{(j)}))$ per iteration.
- Outer Wegstein trace: $\gamma_i^{(k)}$ and $q_i^{(k)}$ per component.
- Final regime tag: `V-L (subcooled)` if $\beta < 0$, `V-L (superheated)` if $\beta > 1$, `V-L (two-phase)` otherwise.

> *Intuition.* The most informative number to watch is $q_i^{(k)}$. If q sits near $q_{\min}$, the system is strongly non-linear (azeotrope nearby) and the simulator is choosing the most aggressive accepted extrapolation. If $q \approx 0$, the system is mildly non-linear and direct substitution would have been fine.

4 Phase changer: the dome-crossing boiler / condenser

What you'll learn. How Choupo's `phaseChanger` (and its aliases `boiler` and `condenser`) takes one stream across the saturation dome — subcooled liquid, anywhere inside the two-phase region, or superheated vapour — by specifying *one* outlet coordinate and letting the heat duty Q fall out as the result. How the saturation temperature is read off P^{sat} (or an IF97 $T^{\text{sat}}(P)$ kernel for a pure fluid), how the latent/sensible split is recovered, and why "boiler" and "condenser" are not separate models but emergent labels.

4.1 Where it sits between a heater and a flash

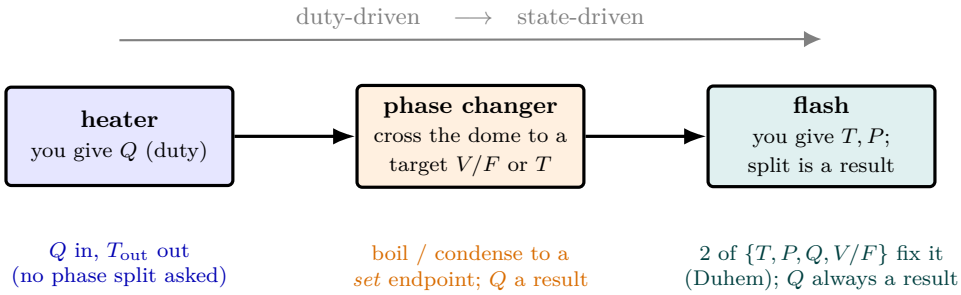

Figure 51: Three single-vessel thermal units, ordered by what the student specifies. The **heater** is duty-driven: you furnish Q and the outlet temperature follows — no phase split is even asked. The **flash** is state-driven: you fix two of $\{T, P, Q, V/F\}$ (Duhem's rule), and the vapour–liquid split is the result, with Q *always* an output, never an input key. The **phase changer** sits between them — a boiler/condenser that crosses the saturation dome to a *set* endpoint (a target V/F or T), reporting the duty it took. Reading the units this way — duty-driven on the left, state-driven on the right — keeps the degrees of freedom straight and avoids the classic "is Q an input or an output?" confusion.

A `heater` (Sec. 33) is *sensible-only*: you give it a duty Q, it returns the outlet T, and it must *not* cross the saturation line — it refuses loudly if the energy you inject would start vaporising the stream. An `isothermalFlash` (Sec. 3) fixes T and P and *reports* the duty Q that holds T while the feed splits. The `phaseChanger` is the unit built for the case in between: a piece of hardware whose explicit *job* is to land the outlet somewhere on the (T, vf) surface that the heater forbids and the isothermal flash cannot target directly. One stream in, one (two-phase-aware) stream out, Q a result.

4.2 Degrees of freedom: one outlet coordinate

By Duhem's theorem (Sec. 3) the outlet of a single feed at known **z** is fixed by exactly two numbers. The `phaseChanger` spends the first on pressure — P_{out} is *held* from the inlet unless you set `P` explicitly — and asks you to spend the second by picking **exactly one** outlet coordinate:

You specify	Meaning	Result
`Q`	duty added/removed (kW)	T_{out}, vf_{out}
`outletT`	outlet temperature	Q
`outletQuality`	target vapour fraction $\text{vf} \in [0, 1]$	Q
`outletState`	`saturatedLiquid` / `saturatedVapour`	Q
`superheat`	$T = T^{\text{sat}}(P) + \Delta K$ (vapour side)	Q
`subcool`	$T = T^{\text{sat}}(P) - \Delta K$ (liquid side)	Q

Supplying zero or more than one coordinate is a hard error — the degree-of-freedom count is enforced, never silently patched. Every mode but the first is an inverse problem: the unit *self-targets*, solving for the Q that lands the outlet on the coordinate you declared, and *announces* that inverse solve in its report (the no-silent-crutch rule — there is no hidden `outerDict` or design-spec loop, the solve is built in but visible). The single-phase states `subcooledLiquid` / `superheatedVapour` are refused as bare `outletState` words precisely because they need that extra distance from saturation; you must give `subcool`/`superheat` instead.

4.3 The saturation temperature

Every spec mode needs the position of the dome at P_{out}, i.e. the saturation temperature $T^{\text{sat}}(P)$. Choupo locates it from the *dominant* component (the one with the largest mole fraction) by two routes:

- **Pure-fluid route.** If the dominant species carries an IF97-class pure-fluid kernel *and* the feed is effectively pure, the kernel's closed-form $T^{\mathrm{sat}}(P)$ is used directly.
- **Generic route.** Otherwise the dominant component's vapour-pressure curve is inverted by bisection,

$$P_{\mathrm{dom}}^{\mathrm{sat}}\big(T^{\mathrm{sat}}\big) = P_{\mathrm{out}}, \tag{242}$$

over $T \in [150, 1200]\,\mathrm{K}$ (the Antoine root, Sec. 2).

For a *pure* (or dominant-component) dome the saturation plateau is a single $T = T^{\mathrm{sat}}$ and the vapour fraction vf is the free coordinate along it: H spans $[h_f, h_g]$ at *constant* T. For a genuine multicomponent mixture the dome has *width* in T (the bubble-to-dew interval), so $\mathrm{vf}(T)$ rises smoothly from 0 at the bubble point to 1 at the dew point, and the engine brackets the sign-changing interval on a flash scan before driving Newton inside it. This split — step-at-T^{sat} for a pure dome, smooth-vf(T) for a mixture — is the one subtlety that distinguishes the two cases throughout the solve.

4.4 Dome-aware enthalpy and the vf closure

The whole unit is built on the existing flash kernel: `IsothermalFlash::solveCore` returns the split $(\beta, \mathbf{x}, \mathbf{y})$ at any (T, P_{out}), and the stream enthalpy is the lever-rule blend of the equilibrium phases,

$$H(T,P) \;=\; (1-\beta)\, h_f(\mathbf{x},T,P) \;+\; \beta\, h_g(\mathbf{y},T,P), \tag{243}$$

the very enthalpies the produced liquid and vapour streams carry. The molar latent heat at saturation is $h_g - h_f$.

The vf=1 trap, and the ε-step that avoids it. Evaluating the enthalpy at *exactly* $\mathrm{vf} = 1$ on the saturation line is dangerous: at $P = P^{\mathrm{sat}}$ the property routing falls onto the *liquid* branch (region 1 of an IF97 kernel; the single-phase side of a cubic EoS) and silently *loses the latent heat*. Choupo therefore probes a tiny step $\varepsilon = 10^{-4}$ *into* the two-phase region and extrapolates the endpoints linearly back out:

$$\begin{aligned} h_{\mathrm{lo}} &= H(T^{\mathrm{sat}}, P,\ \varepsilon), & h_{\mathrm{hi}} &= H(T^{\mathrm{sat}}, P,\ 1-\varepsilon),\\ h_f &= \frac{h_{\mathrm{lo}} - \varepsilon\, h_{\mathrm{hi}}}{1-2\varepsilon}, & h_g &= \frac{h_{\mathrm{hi}} - \varepsilon\, h_{\mathrm{lo}}}{1-2\varepsilon}. \end{aligned} \tag{244}$$

This recovers the true h_f, h_g (hence the latent heat) without ever asking the thermo for an ambiguous on-line point.

Duty mode — closing on vf. In duty mode you give Q; the target outlet enthalpy is

$$H_{\mathrm{target}} \;=\; H_{\mathrm{in}} \;+\; \frac{Q}{F}, \tag{245}$$

(F in $\mathrm{mol\,s^{-1}}$, Q in W). On a pure dome the engine first tests whether H_{target} falls on the plateau, $h_f \le H_{\mathrm{target}} \le h_g$; if so the outlet sits at T^{sat} and the vapour fraction is read straight off the lever rule,

$$\mathrm{vf}_{\mathrm{out}} \;=\; \frac{H_{\mathrm{target}} - h_f}{h_g - h_f}, \tag{246}$$

clipped to $[0,1]$. Only *outside* the plateau — where $H(T)$ is smooth and monotone — does a single Newton-on-T run, with the pure-fluid kernel's triple-point floor ($T \ge 274\,\mathrm{K}$) protecting the search from the kernel's forbidden region. A multicomponent dome has no plateau, so duty mode Newtons on T throughout.

4.5 The duty is always a result; latent / sensible split

Whatever coordinate you picked, the duty is computed last, from the converged endpoints:

$$Q \;=\; F\,\big(H_{\mathrm{out}} - H_{\mathrm{in}}\big), \qquad Q>0 \text{ added}, \quad Q<0 \text{ removed}. \tag{247}$$

It is reported as `DUTY (result)` and, on the flowsheet, appears as a heat *stream* (a utility stub + dashed wire), never as a number painted on the node. Choupo further attributes the duty to a *latent* share — the part tied to the change in vapour fraction at saturation — and a *sensible* remainder:

$$Q_{\mathrm{latent}} \;=\; F\,\big(\mathrm{vf}_{\mathrm{out}} - \mathrm{vf}_{\mathrm{in}}\big)\,(h_g - h_f), \qquad Q_{\mathrm{sensible}} \;=\; Q - Q_{\mathrm{latent}}, \tag{248}$$

with $h_g - h_f$ the latent heat from (244) at T^{sat}.

4.6 Boiler and condenser are emergent labels

There is *one* class. After convergence Choupo reads the sign of Q and the outlet vapour fraction and prints a *role*: a positive duty landing on saturated vapour is a `boiler`, partway up the dome a `partial boiler`; a negative duty landing on saturated liquid is a `condenser`, partway down a `partial condenser`; a duty that stays single-phase is just a heater/cooler. The aliases `boiler` and `condenser` register the *same* class — the direction is emergent from the physics, not a model switch, so a case that mis-labels a condensing duty as a `boiler` still computes the correct (negative) Q and says so.

Worked example 4.1. Saturated-vapour boiler on a pure domeFeed: saturated-or-subcooled water at $P = 1\,\text{bar}$, $F = 1\,\text{kmol}\,\text{s}^{-1}$, with `outletState saturatedVapour;`. The engine finds $T^{\text{sat}}(1\,\text{bar}) \approx 372.8\,\text{K}$ from the pure-fluid kernel, sets $T_{\text{out}} = T^{\text{sat}}$ and forces $\text{vf}_{\text{out}} = 1$ (vf is the free coordinate on the plateau). With $H_{\text{out}} = h_g$ and $H_{\text{in}} = h_f$ the latent heat $h_g - h_f \approx 40.7\,\text{kJ}\,\text{mol}^{-1}$ gives $Q = F(h_g - h_f) \approx 1000 \cdot 40.7 = 4.07 \times 10^4\,\text{kW}$, reported as a positive duty, role `boiler`, $Q_{\text{latent}} \approx Q$ and $Q_{\text{sensible}} \approx 0$. Re-specify `outletQuality 0.5;` and the same plateau gives half the latent duty.

Power and limit. **Power:** one class spans every dome-crossing service with a single, honest degree of freedom; the duty is always a visible result; the latent/sensible split and the boiler/condenser role come for free from the converged state; and the same code path serves both the generic cubic-EoS + Antoine route and the high-accuracy pure-fluid (IF97) route, the dispatch hidden inside the thermo. **Limit:** the saturation dome is located from the *dominant* component, so the single T^{sat} used for the latent split is a pure/dominant-component approximation — for a wide multicomponent dome it is only the bracket centre, and a strongly non-ideal mixture's true bubble-to-dew width is captured by the flash but not by that one plateau temperature. The unit is a thermodynamic phase-change *specifier*, not a rating model: it does not size area or compute a real heat-transfer coefficient unless you switch to `model geometry;` (the Nusselt film-condensation / Rohsenow nucleate-boiling branch with a Zuber CHF ceiling), documented in the heat-transfer rating material.

Primary references: the saturation curve and Clausius–Clapeyron latent heat (Sec. 2); Smith, Van Ness & Abbott, *Introduction to Chemical Engineering Thermodynamics*, on the lever rule and dome enthalpies; IAPWS R7-97 (IF97) for the pure-fluid $T^{\text{sat}}(P)$ kernel; for the geometry branch, Nusselt (1916) film condensation and Rohsenow (1952) / Zuber (1959) nucleate boiling and critical heat flux.

5 Continuous stirred-tank reactor (CSTR)

What you'll learn. A single-reaction steady-state CSTR collapses to a one-dimensional Newton solve in the reaction extent ξ. The simulator does exactly that.

5.1 Material balance with reaction extent

For one independent reaction with stoichiometry $\boldsymbol{\nu}$ and rate $r(\mathbf{x}, T)$ (units mol/(m^3 s)), the steady-state component balance over the reactor is

$$F_{\text{in}}\,\mathbf{z}_{\text{in}} \;=\; F_{\text{out}}\,\mathbf{z}_{\text{out}} \;-\; V_R\,\boldsymbol{\nu}\,r(\mathbf{z}_{\text{out}}, T). \tag{249}$$

Using the extent of reaction ξ (mol/h) we write $\mathbf{z}_{\text{out}}F_{\text{out}} = \mathbf{z}_{\text{in}}F_{\text{in}} + \boldsymbol{\nu}\,\xi$. Eliminating $\mathbf{z}_{\text{out}}$:

$$g(\xi) \;\equiv\; \xi - V_R\,r\big(\mathbf{z}_{\text{out}}(\xi), T\big) \;=\; 0. \tag{250}$$

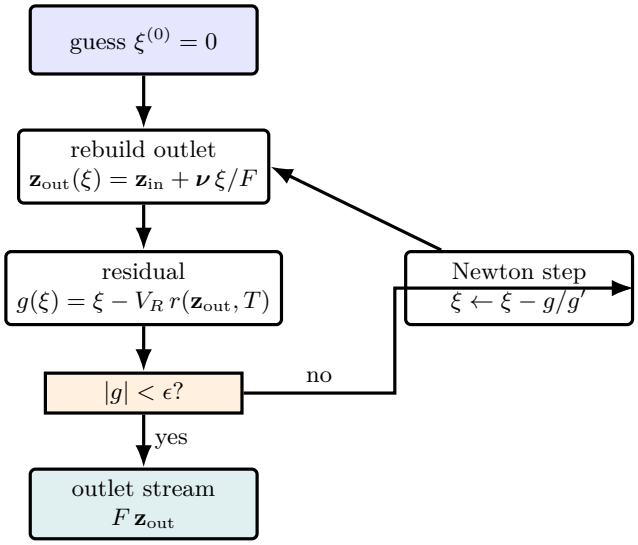

Figure 52: The steady CSTR is *not* a big system — it collapses to one scalar root-find. The single unknown is the reaction extent ξ; every outlet mole fraction is rebuilt from it through stoichiometry $\mathbf{z}_{\text{out}} = \mathbf{z}_{\text{in}} + \boldsymbol{\nu}\,\xi/F$, the rate $r(\mathbf{z}_{\text{out}}, T)$ is evaluated there, and the residual $g(\xi) = \xi - V_R r$ measures how far the guessed extent is from production-equals-consumption. A damped **Newton-1D** (right loop) drives $g \to 0$ in 3–5 iterations. The whole reactor is one equation in one number.

A single non-linear equation in a single unknown ξ: **Newton-1D in one variable**, with damping. Convergence is typically 3–5 iterations.

5.2 First-order kinetics — analytic check

For an Arrhenius first-order reaction $r = k\,c_A$ with $k = k_0\,e^{-E_a/R_{\text{g}}T}$, the steady-state CSTR has the analytic solution

$$X_A \;\equiv\; 1 - \frac{c_A}{c_{A,0}} \;=\; \frac{\text{Da}}{1+\text{Da}}, \qquad \text{Da} = k\,\tau, \tag{251}$$

where $\tau = V_R/Q$ is the residence time. The simulator reproduces this to machine precision in `tutorials/cstr01_first_order`.

5.3 Worked example

Worked example 5.1. First-order $A \to B$ in a 0.5 m^3 CSTRInlet: $c_{A,0} = 100$ mol/m^3, volumetric flow $Q = 0.01$ m^3/s, kinetic constant $k = 0.02$ s^{-1}. Then $\tau = V_R/Q = 50$ s, Da $= 1.0$, $X_A = 0.5$. Newton starting from $\xi^{(0)} = 0$ converges in 4 iterations to $\xi = 0.5\,c_{A,0}\,Q = 0.5$ mol/s.

Runnable case. `tutorials/cstr01_first_order` — includes the analytic check in the case output.

6 The rate law: from mass action to Langmuir–Hinshelwood

What you'll learn. Why a power law cannot describe a catalytic reaction at any order, where the denominator of an LHHW rate law comes from, and why a rate constant means nothing without the rate law and the activity model it was regressed with.

Every reactor so far has been handed a rate $r(\mathbf{c}, T)$ and asked what to do with it: a CSTR sets $\xi = rV_R$, a PFR integrates it along the volume, a batch vessel integrates it in time. None of them cares what r is. This section is about r itself, and in Choupo it is one object — the shared `RateLaw` — so a batch vessel and a continuous one cannot disagree about it.

6.1 Where mass action stops

For a homogeneous elementary step the rate is a product of concentrations raised to the stoichiometric orders: mass action, the law of a well-mixed molecular collision. A *catalytic* reaction is not that. The molecules do not meet in the fluid; they meet on a surface, and the surface has a finite number of sites.

Follow Langmuir. Let θ_i be the fraction of sites carrying species i and θ_v the fraction left vacant. Adsorption and desorption of i balance at equilibrium,

$$k_{a,i}\, c_i\, \theta_v \;=\; k_{d,i}\, \theta_i \qquad\Longrightarrow\qquad \theta_i \;=\; K_i\, c_i\, \theta_v,$$

with $K_i = k_{a,i}/k_{d,i}$ the *adsorption equilibrium constant* of i. Every site is either vacant or occupied, so $\theta_v + \sum_i \theta_i = 1$ and therefore

$$\theta_v \;=\; \frac{1}{1+\sum_i K_i c_i}, \qquad \theta_i \;=\; \frac{K_i c_i}{1+\sum_i K_i c_i}. \tag{252}$$

Now let the rate-determining step be a surface reaction between adsorbed species. Its rate is proportional to the coverages of the reactants — not to their bulk concentrations — so for a bimolecular step $A_{\mathrm{ads}} + B_{\mathrm{ads}}$ occupying two sites,

$$r \;=\; k\,\theta_A\,\theta_B \;=\; \frac{k\,K_A c_A\,K_B c_B}{\left(1+\sum_i K_i c_i\right)^2}. \tag{253}$$

That denominator is the whole point, and no power law contains it. Look at what it does. A *product* that adsorbs strongly appears in $\sum_i K_i c_i$ but not in the numerator: as the reaction makes it, it covers the surface, crowds the reactants off, and **throttles the reaction that produced it**. This is product inhibition. You cannot reproduce it by tuning the order of a power law, because a power law's dependence on a product's concentration is monotone in one direction only. Equation (253) is the Langmuir–Hinshelwood–Hougen–Watson (LHHW) form [1, 2, 3].

6.2 The one formula Choupo evaluates

Choupo does not carry a taxonomy of LHHW shapes. It carries one expression, and the shapes are what you get by choosing its parts. Write θ_i for the intensive variable of species i — a concentration, or an *activity* $a_i = \gamma_i x_i$ when `basis activity` — and weight it by its adsorption constant to get the species *as the surface sees it*,

$$\bar{a}_i \;=\; K_i(T)\,\theta_i, \qquad K_i(T) = K_i^{0}\,\mathrm{e}^{B_i/T}$$

(B_i the van 't Hoff slope; $K_i = 1$ for a species that declares no adsorption). Then

$$\boxed{\,r \;=\; \frac{k_f \prod_i \bar{a}_i^{\,\alpha_i} \;-\; k_r \prod_i \bar{a}_i^{\,\beta_i}}{\left(v+\sum_{i\in\mathrm{ads}} \bar{a}_i\right)^{p}}\,} \tag{254}$$

Three dials, and every rate law in this guide is a setting of them.

- $p = 0$ (no `adsorption` block): the denominator is 1 and what remains is the plain power law, `type Arrhenius`. Mass action.
- $p > 0$: LHHW. The exponent is the number of sites the rate-determining step occupies — $p = 2$ in Eq. (253).
- v is the vacant-site term. It is 1 in the classical Hougen–Watson forms, and 0 where the surface is taken as saturated — an ion-exchange resin swollen with liquid, say, on which every site holds *something* and only the competition matters.

Note that $\bar{a}_i$ sits in the numerator too. That is not a convention: the rate-determining surface step sees the *adsorbed* species, as Eq. (253) shows. Any constant K_i can of course be absorbed into k_f, which is exactly why textbook LHHW forms look different from one another — and it is the first hint of the pitfall below.

The reverse leg is either **detailed balance** (`reversible true`: $k_r = k_f/K_c$, so equilibrium sits precisely where the thermodynamics puts it, and cannot drift with a fitted number), or an **explicitly regressed** Arrhenius pair $(k_r^0, E_{a,r})$ — which is what a kinetics paper reports. Choupo refuses both at once, and refuses detailed balance on an activity basis, since K_c is a concentration-basis constant.

6.3 Per gram of catalyst

A heterogeneous rate constant is reported per gram of dry catalyst, and a reactor is measured in cubic metres. Declare the bulk loading ρ_{cat} and the conversion is exact:

$$r\left[\tfrac{\mathrm{kmol}}{\mathrm{m^3\,s}}\right] \;=\; \rho_{\mathrm{cat}}\left[\tfrac{\mathrm{kg}}{\mathrm{m^3}}\right]\cdot r\left[\tfrac{\mathrm{mol}}{\mathrm{g\,s}}\right].$$

It is a declared, printed number — change the packing and the reactor changes with it.

6.4 A worked, cited example, and a trap

Methyl-acetate synthesis over the ion-exchange resin Amberlyst 15, $\mathrm{CH_3OH + CH_3COOH \rightleftharpoons CH_3COOCH_3 + H_2O}$, follows Eq. 16 of Pöpken, Götze and Gmehling [4] — the adsorption-based model the authors recommend (mean relative error 5.4 %, against 13.9 % for the pseudo-homogeneous power law on the same data). It is Eq. (254) with $v = 0$, $p = 2$, activities from UNIQUAC, and $\bar{a}_i = K_i a_i / M_i$.

Water and methanol adsorb hardest ($K_i/M_i = 0.29$ and 0.18, against 0.052 for the acid). So the water the reaction *makes* crowds the acid off the resin. That is why reactive distillation, which pulls the water out as it forms, pays for its capital — and here it is a line in a dictionary rather than a sentence in a textbook.

> **Common pitfall.** Do not try to see the inhibition by deleting the denominator and keeping k_f. It shows nothing. In this parameterisation the reaction gets *faster*, because the adsorbed activities are small numbers and dividing by their square is a multiplication. That is an arithmetic accident, not physics: the constants were regressed *for* that rate law, on *those* activities. **A rate constant without its rate law and its activity model is a number without a meaning.** Swap UNIQUAC for UNIFAC in `cstr07_lhhw_methylAcetate` and $k_1 = 8.497 \times 10^6$ stops being the authors' k_1.
> To *see* the inhibition, hold the reactants fixed and add the product. In `batch07_lhhw_methylAcetate`, charging water at unchanged acid and alcohol fractions halves the initial rate, from 58.3 to 29.0 mmol/(L s). The water is not a reactant. It is sitting on the sites.

Three reactors evaluate the same Eq. (254). The steady CSTR (`cstr07`) runs kinetically limited at $X = 49\,\%$; grow its volume and it climbs to 76 %. The batch vessel (`batch07`) integrates to $X = 76.6\,\%$ and stops. That plateau is asserted nowhere: it is where the numerator of Eq. (254) vanishes, i.e. the activity-based $K_a = 25.50$ that the authors' *independent* Eq. 17 recovers from the same eight constants. Two reactors, two integration schemes, one equilibrium — and a cheap, honest check that the parameters were entered right.

> **Runnable case.** `cstr07_lhhw_methylAcetate` (steady, activity basis) · `batch07_lhhw_methylAcetate` (the same law integrated in time, plus the water-spike experiment) · `ctrl07_lhhw_inhibition` (a PID loop against a reaction whose exotherm fades as its own product covers the catalyst).

7 Plug-flow reactor (PFR)

What you'll learn. The CSTR's continuous-flow counterpart with *no* backmixing. A single differential balance integrated along the reactor volume by the fourth-order Runge-Kutta scheme of Chapter 9. For first-order kinetics it admits the textbook closed-form $X = 1 - e^{-k\tau}$ that the simulator's tutorial reproduces.

7.1 The differential material balance

A PFR is a tube in which the molar flow of each component changes continuously with the cumulative volume V ($0 \le V \le V_R$). For a single liquid-phase reaction $\sum_i \nu_i A_i = 0$ proceeding at rate $r(\mathbf{x}, T)$ (mol per unit volume per unit time), a shell balance on a differential slab of volume $\mathrm{d}V$ yields

$$\boxed{\frac{\mathrm{d}F_i}{\mathrm{d}V} \;=\; \nu_i\, r(\mathbf{x}, T), \qquad i = 1, \ldots, N_c,} \tag{255}$$

where $F_i = F_i(V)$ is the molar flow of i, and $x_i = F_i / \sum_k F_k$ rebuilds the mole fractions inside r at each volume. Equation (255) is an autonomous, first-order ODE system in V; the inlet condition $F_i(0) = F_{i,0}$ closes the problem.

Intuition. The CSTR is a single zero-dimensional point where everything is at the outlet conditions. The PFR is a one-dimensional continuum where conditions evolve from inlet to outlet — mathematically a *plug* of fluid sliding down the tube without mixing axially. The CSTR's algebraic equation becomes the PFR's ODE because the time the plug spends between V and $V + \mathrm{d}V$ is $\mathrm{d}V/Q$.

7.2 Algorithm: RK4 integration

The simulator integrates (255) from $V = 0$ to V_R with the classical fourth-order Runge-Kutta scheme of §9. With `nSteps` sub-intervals (default 100) the step size is $h = V_R/\texttt{nSteps}$ and the local truncation error is $O(h^5)$ — amply sufficient for any pedagogical kinetic. Per-step cost: four evaluations of $r(\mathbf{x}, T)$, which dominates because each evaluation calls the kinetics object's `rate()` method.

The reactor outlet $\mathbf{F}(V_R)$ feeds whatever is downstream in `flowsheetDict`, exactly like a CSTR outlet — the unit-op contract does not distinguish.

7.3 First-order kinetics: the analytic check

For an irreversible first-order reaction $A \to B$ at constant T, $r = k\, c_A = k\, F_A/Q$, and (255) reduces (for the limiting reactant A) to

$$\frac{\mathrm{d}F_A}{\mathrm{d}V} \;=\; -\,k\,\frac{F_A}{Q}. \tag{256}$$

Integrating,

$$\boxed{X_A \;\equiv\; 1 - \frac{F_A(V_R)}{F_{A,0}} \;=\; 1 - e^{-k\,\tau},} \tag{257}$$

with the residence time $\tau = V_R/Q$. The PFR achieves higher conversion than a CSTR with the same τ: at $k\tau = 1$, $X^{\mathrm{PFR}} = 1 - e^{-1} = 0.632$ versus $X^{\mathrm{CSTR}} = 1/(1 + k\tau) = 0.500$ — the canonical "no backmixing wastes less feed at the front of the reactor"

result.

7.4 Worked example

Worked example 7.1. Same kinetics as `cstr01_first_order`, in a PFR$k\tau = 1.10$. The PFR conversion from (257) is $X = 1 - e^{-1.10} = 0.667$. The CSTR of the same volume gives $X = 1/(1 + 1.10) = 0.476$. The simulator's RK4 integration (100 steps) returns $X = 0.667$ to six digits — the difference between CSTR and PFR for this kinetic is exactly the backmixing effect.

Runnable case. `tutorials/pfr01_first_order` — prints conversion every `writeInterval` steps along the axial coordinate; the final $X = 0.667$ matches (257) to RK4 truncation error.

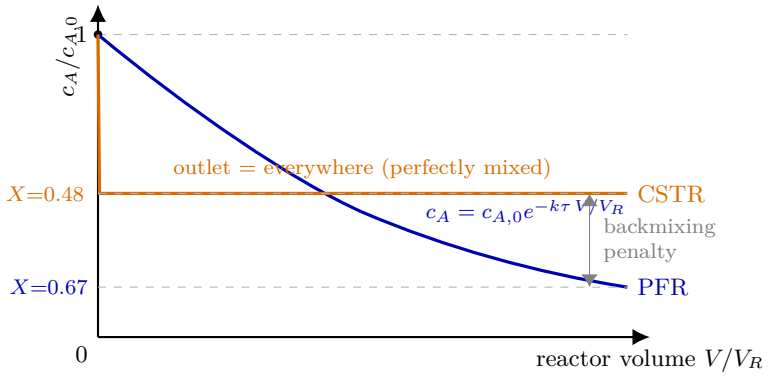

Figure 53: Same first-order kinetics, same residence time τ ($k\tau = 1.1$), two reactor idealisations. The **PFR** (blue) is a plug sliding down the tube: c_A falls continuously from inlet to outlet, so the reaction runs at high concentration near the front — $X = 1 - e^{-k\tau} = 0.67$. The **CSTR** (orange) mixes the feed *instantly* to the outlet state, so the *entire* vessel reacts at the low outlet concentration: the residual horizontal line is the whole reactor at once, $X = k\tau/(1 + k\tau) = 0.48$. The shaded gap is the price of backmixing — the same volume converts less feed when it is stirred to uniformity.

8 Single-effect evaporator: the credo case study

What you'll learn. The evaporator is the unit operation where Choupo's credo earned its scars. Three iterations of the unit op design (hardware- spec F-spec Mode-2) ended in a credo-pure interface that is now the template every other unit op is being aligned to. The credo: *the operation block holds only hardware parameters; the operating state of the vessel (pressure, boiling temperature, vapour fraction) is computed by the simulator from the heat balance and the saturation curve.* Any design question that the textbook poses — "find the area", "find the steam flow", "find the intermediate pressures" — is handled by an outer `DesignSpec`, never by switching modes inside the unit op.

8.1 What the evaporator solves

Inputs (ports):

- One **feed** liquid stream (process side): $F_{\text{feed}}, T_{\text{feed}}, P_{\text{feed}}, \mathbf{z}_{\text{feed}}$.
- One **heating steam** stream (chest side): $F_{\text{steam}}, T_{\text{steam}}, P_{\text{steam}}, \mathbf{z}_{\text{steam}}$. Typically declared with `state saturatedVapour;` so the stream parser computes T from P via the solvent's saturation curve (§2).

Parameters (operation block):

- **area** — heat-exchange surface A [m^2]
- **U** — overall heat-transfer coefficient [$\text{W}/(\text{m}^2\,\text{K})$]
- **Tref** (optional) — reference temperature for the sensible-enthalpy integrals.

Conspicuously *absent* from the operation block: the operating pressure P_{op}, the boiling temperature T_{boil}, the vapour fraction V/F, the duty Q, the steam mass demand — everything else. Those are all **computed**.

8.2 The Mode-2 equations

The unit op solves four equations in this order, each carrying

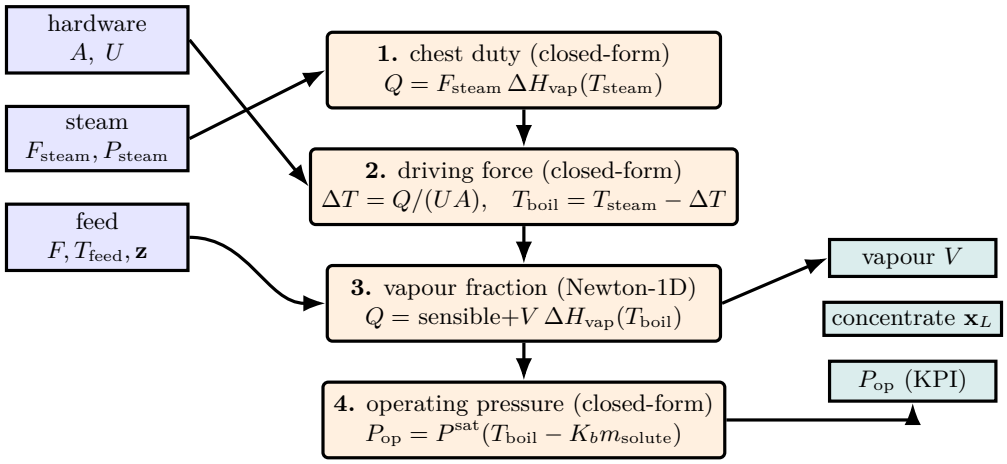

Figure 54: The credo evaporator solves four equations *in order*, and the operating pressure P_{op} — which the student never writes — falls out at the end. Only hardware (A, U) and the streams enter (blue). **1.** The condensing steam fixes the duty Q. **2.** The hardware turns Q into a driving force ΔT and hence the boiling temperature T_{boil}, set by chest and geometry *alone*. **3.** A single Newton-1D closes the process-side energy balance for the vapour flow V. **4.** The Antoine curve, corrected for boiling-point elevation $K_b m$, inverts T_{boil} back to the pressure the vessel must sit at. No mode switch, no P_{op} in the dict — the crown jewel of the credo is a computed output, not an input.

its own physical meaning, none of them touching the user dict:

1. Chest duty (closed-form). The heating steam supplies its full latent heat as it condenses on the chest tubes:

$$Q \;=\; F_{\text{steam}} \cdot \Delta H_{\text{vap, solv}}(T_{\text{steam}}). \tag{258}$$

2. Driving force from hardware (closed-form). The geometry of the heat exchanger gives the LMTD-equivalent driving force at the achievable boiling point:

$$\Delta T \;=\; \frac{Q}{U \cdot A}, \qquad T_{\text{boil}} \;=\; T_{\text{steam}} \;-\; \Delta T. \tag{259}$$

Note that T_{boil} is independent of the process-side composition in Mode 2 — it is fixed by chest + hardware alone.

3. Vapour fraction from energy balance (Newton-1D). The duty must close the process-side energy balance: heating the feed sensibly from T_{feed} to T_{boil}, plus vaporising the produced solvent at T_{boil}:

$$Q \;=\; F_{\text{feed}} \sum_i z_i \left[H_i^{l}(T_{\text{boil}}) - H_i^{l}(T_{\text{feed}}) \right] \;+\; V \cdot \Delta H_{\text{vap, solv}}(T_{\text{boil}}). \tag{260}$$

Solving for V (the vapour molar flow) is one Newton-1D iteration: the residual is monotone in V/F at fixed T_{boil} and Q, so the bracket $[0, 0.999]$ is safe.

4. Operating pressure from saturation (closed-form). T_{boil} is the boiling point of the liquid at the vessel's pressure, corrected for the boiling-point elevation caused by the non-volatile solute:

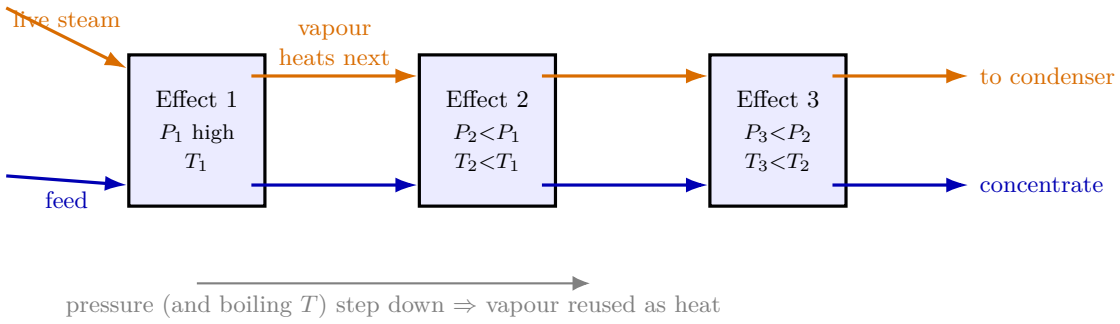

Figure 55: Why evaporators are run in a train. Each effect's **vapour** (orange) becomes the heating steam in the *next* effect's chest, so one unit of live steam boils off water roughly N times across N effects — the steam economy. This works only because the pressure (and therefore the boiling temperature) *steps down* from effect to effect: the cooler next effect can be heated by the vapour the warmer one produced. The **liquor** (blue) cascades forward, growing more concentrated at each stage until it leaves as product. Each box is exactly one single-effect credo evaporator of Figure 54; Choupo builds the train by piping the vapour port of one into the steam port of the next in `flowsheetDict`.

$$T_{\text{boil}} \;=\; T_{\text{sat, pure}}(P_{\text{op}}) \;+\; K_b \cdot m_{\text{solute}}(\mathbf{x}_L), \tag{261}$$

where m_{solute} is the molality of the solute in the concentrated outlet $\mathbf{x}_L$ and K_b is the ebulioscopic constant. Inverting (261) gives

$$P_{\text{op}} \;=\; P_{\text{sat, pure}}\big(T_{\text{boil}} - K_b\, m_{\text{solute}}\big),$$

a single call to the solvent's Antoine equation. This is the *credo's crown jewel*: the user never wrote P_{op}; the simulator computes it from physics and emits it as a KPI (`evap.P`) and as the pressure stamped on both liquid and vapour output streams.

8.3 What *TU* write vs what the simulator gives back

Variable	In Mode 2, this is…	Where it shows up
$F_{\text{feed}}, T_{\text{feed}}, \mathbf{z}_{\text{feed}}$	USER INPUT	`streams.feed`
P_{steam}	USER INPUT	`streams.liveSteam.P`
`state saturatedVapour`	USER FLAG	`streams.liveSteam.state`
T_{steam}	COMPUTED (P+state, stream parser)	`streams.liveSteam.T`
F_{steam}	USER INPUT (often via $variable)	`streams.liveSteam.F`
A, U	USER INPUT (hardware)	`operation.{area,U}`
Q	COMPUTED, eq. (258)	KPI `evap.duty`
ΔT	COMPUTED, eq. (259)	KPI `evap.dT`
T_{boil}	COMPUTED, eq. (259)	KPI `evap.T_boil`
V/F	COMPUTED, eq. (260)	KPI `evap.V_over_F`
P_{op}	COMPUTED, eq. (261)	KPI `evap.P`
$\mathbf{x}_L$ (concentrate composition)	COMPUTED, species mass balance	`streams.productConc.z`
BPE	COMPUTED, $K_b\, m_{\text{solute}}$	KPI `evap.BPE`

The student writes a small handful of physically tangible quantities; everything else falls out. The same code path works for every evaporator case — there is no mode switch.

8.4 Why mode-switching inside the unit op was wrong

The path from to went through two redesigns whose common mistake was offering *alternative spec modes* inside the unit op:

- hardware-spec: A, U, P_{op} given, Q and F_{steam} computed.
- F-spec: F_{steam} given, Q computed from chest condensation; A became a consistency report (`A_needed` KPI).
- Mode-2: A, U given, P_{op} computed from the heat balance.

The first two flavours each had a defensible physics narrative. But they violated the credo because the *operation* block contained, at different times, a vessel pressure (P_{op}) or a chest-flow placeholder, neither of which is hardware: P_{op} is a stream property of the outputs; F_{steam} is a stream property of the chest. Different design questions ("find A for a given P_{op}", "find P_{op} for a given A") were being answered by silently switching which variable the `solve()` was iterating on. Pedagogically, the student could not tell from the dict alone what was input and what was output.

The redesign removes the choice: *always* hardware in, *always* state computed. Every design variation is hoisted to an outer `DesignSpec` that manipulates user-level $variables — including A itself when the textbook makes it the unknown.

8.5 The three canonical design questions, one unit op

The textbook problems map directly onto outer-driver formulations. All three use the same evaporator underneath; only the outerDict changes.

Q1 — "Given hardware, what comes out?" No outerDict. The cascade runs once, sequentially. Tutorial `evaporator02_triple_effect_sugar` demonstrates this for a fixed $A = 100\,\text{m}^2$ cascade.

Q2 — "Given product spec, find the missing hardware." 1-D or 2-D `DesignSpec`: manipulate A (and possibly F_{steam}); target the outlet concentration (and possibly the textbook operating pressure). Tutorial `evaporator04_orange_juice_simple` is the canonical example — the dict expresses the textbook MCFT Ex. 10.2 problem in 50 lines.

Q3 — "Given product spec and terminal vacuum, find the multi-effect design." N-D `DesignSpec` on A and F_{steam} with two targets (product mass + terminal P_N). Co-current is a DAG, no recycle; counter-current is a recycle problem with two tear streams (one per inter-effect pair, see §5). Tutorials `evaporator03_co_current` and `evaporator05_counter_current` cover both wirings.

> *Intuition.* The mental model the student should leave with: the evaporator is a black box with two stream inputs, three stream outputs, and two parameters (A and U). It always computes the same set of unknowns from the same set of inputs. Every "what if" becomes a question for the outer DesignSpec, not a modification of the operation block. The credo enforces this single contract — and the rest of the unit ops in Choupo are being incrementally aligned to the same shape.

9 Liquid-liquid equilibrium by Gibbs minimisation

What you'll learn. Two-phase liquid-liquid equilibrium is, classically, solved by writing the equal-activity constraint $\gamma_{i,\alpha}x_{i,\alpha} = \gamma_{i,\beta}x_{i,\beta}$ for every component and feeding the resulting $2n+1$ algebraic system to Newton. For LLE landscapes with the same activity model on both phases — and water / butanol is the textbook example — this method fails: the trivial $K = 1$ solution at $\mathbf{x}_\alpha = \mathbf{x}_\beta = \mathbf{x}_z$ is a *saddle* of the Gibbs landscape but a stable fixed point of the residual map, and damped Newton from any nearby start collapses to it. Direct minimisation of the per-mole Gibbs energy on the composition simplex bypasses the saddle entirely. This is what `IsothermalFlash` does for the `phaseSet LL` branch. But before paying for that multi-start minimisation, the flash first asks a cheaper, sharper question — *is the feed even unstable?* — with Michelsen's tangent-plane-distance test, the subject of the next subsection.

9.1 The phase-stability pre-check: Michelsen's tangent plane

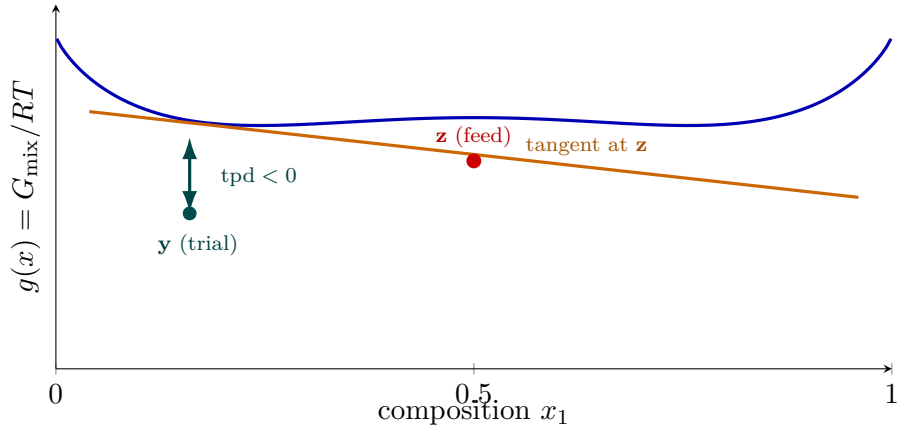

Figure 56: Michelsen's tangent-plane stability test, read off the molar-Gibbs-of-mixing surface $g(x)$. Draw the hyperplane tangent to g at the feed $\mathbf{z}$ (orange). The tangent-plane distance $\mathrm{tpd}(\mathbf{y})$ is the vertical gap between the surface and that tangent, measured at a trial composition $\mathbf{y}$. Here the surface dips *below* the tangent near $\mathbf{y}$: $\mathrm{tpd}(\mathbf{y}) < 0$, which *proves* the single feed is unstable and a liquid–liquid split exists — and that $\mathbf{y}$ is a ready-made guess for the incipient phase. A feed is stable (one phase) iff $\mathrm{tpd} \geq 0$ everywhere. At a converged stationary point this whole picture collapses to one number, $\mathrm{tm} = 1 - \sum_i Y_i$, the quantity `michelsenTPD` actually tests.

What you'll learn. Before any split is computed, the flash must decide *whether a split exists at all.* A single liquid is stable when no trial phase can lower the total Gibbs energy; it is unstable — a split exists — when some trial composition $\mathbf{y}$ lies *below* the tangent plane drawn to the molar-Gibbs surface at the feed. Michelsen (1982) turned this geometric statement into a one-number criterion, the tangent-plane distance tm, and a cheap successive-substitution iteration that finds the deepest-cutting trial phase. `michelsenTPD` in `src/solver/StabilityTest.cpp` implements it as the gate of the `phaseSet LL` branch.

The tangent-plane distance

Fix T and P and let $g(\mathbf{x}) = G_{\mathrm{mix}}(\mathbf{x})/RT$ be the molar Gibbs energy of mixing, in units of RT, of a homogeneous liquid of composition $\mathbf{x}$. The feed sits at $\mathbf{x} = \mathbf{z}$. Now imagine pinching off an infinitesimal amount of a *trial* phase of composition $\mathbf{y}$ from the feed. The change in total Gibbs energy, per mole of trial phase, relative to the value the same material had *at the feed composition*, is the **tangent-plane distance**

$$\mathrm{tpd}(\mathbf{y}) \;=\; \sum_{i=1}^{n} y_i \Big[\, \mu_i(\mathbf{y}) - \mu_i(\mathbf{z}) \,\Big]/RT \;=\; \sum_{i=1}^{n} y_i \big[\, \ln\!\big(y_i\gamma_i(\mathbf{y})\big) - \ln\!\big(z_i\gamma_i(\mathbf{z})\big) \,\big], \tag{262}$$

where the second form uses the symmetric activity-coefficient expression $\mu_i/RT = \mu_i^\circ/RT + \ln(x_i\gamma_i)$, so the reference chemical potentials μ_i° cancel between the two phases. Geometrically, $\mathrm{tpd}(\mathbf{y})$ is the vertical gap between the surface $g(\mathbf{y})$ and the hyperplane tangent to g at $\mathbf{z}$, measured at $\mathbf{y}$. The stability statement of Baker–Pierce–Luks and Michelsen is then a single line:

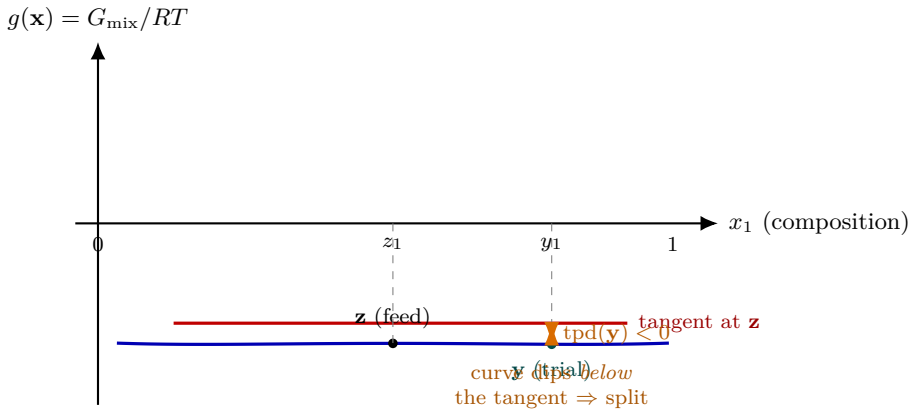

Figure 57: Michelsen's tangent-plane-distance test, drawn for a binary. $g(\mathbf{x}) = G_{\text{mix}}/RT$ is the molar Gibbs energy of mixing; the feed sits at $\mathbf{z}$. Draw the line tangent to g at $\mathbf{z}$ (red). The tangent-plane distance $\text{tpd}(\mathbf{y})$ is the *vertical gap* from that tangent down to the surface at a trial composition $\mathbf{y}$. If the surface stays *above* the tangent everywhere, $\text{tpd} \geq 0$ and the single feed is stable. But here the curve has a hump and dips *below* the tangent near $\mathbf{y}$: $\text{tpd}(\mathbf{y}) < 0$ (equivalently $\text{tm} < 0$, or $\sum_i Y_i > 1$) — one negative value proves a phase split exists, and that $\mathbf{y}$ is a ready-made guess for the incipient phase that the Gibbs-min flash then refines.

Key idea. The feed $\mathbf{z}$ is **stable** (one phase) if and only if $\text{tpd}(\mathbf{y}) \geq 0$ for *every* trial composition $\mathbf{y}$ on the simplex. A single $\mathbf{y}$ with $\text{tpd}(\mathbf{y}) < 0$ proves a phase split exists — and that $\mathbf{y}$ is a good guess for the incipient phase.

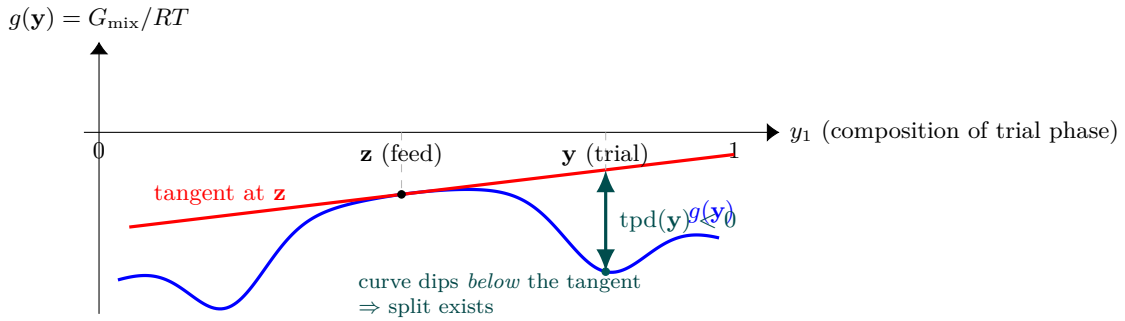

Figure 58: Michelsen's tangent-plane distance, drawn for a binary feed whose Gibbs-of-mixing curve $g(\mathbf{y})$ has two wells (an LLE system). Draw the plane tangent to g at the feed composition $\mathbf{z}$ (red). The tangent-plane distance $\text{tpd}(\mathbf{y})$ is the *vertical gap* from the curve down to (or up to) that plane, measured at a trial composition $\mathbf{y}$. Here the curve dips *below* the tangent at the right-hand well, so $\text{tpd}(\mathbf{y}) < 0$: a trial phase there lowers the total Gibbs energy, which *proves* the single feed is unstable and a split exists. If the curve stayed on or above the tangent everywhere ($\text{tpd} \geq 0$), the feed would be one stable phase and the flash would short-circuit. Michelsen finds the deepest such dip by a cheap successive-substitution walk to the curve's stationary points, never a brute simplex sweep.

From a global search to stationary points

Searching the whole simplex for a negative value is impractical, so Michelsen searches only the *stationary points* of tpd. Setting $\partial\,\text{tpd}/\partial y_i = 0$ subject to $\sum_i y_i = 1$ gives the condition

$$\ln y_i + \ln \gamma_i(\mathbf{y}) - h_i \;=\; k \qquad \text{for all } i, \qquad h_i \;\equiv\; \ln\!\big(z_i\,\gamma_i(\mathbf{z})\big), \tag{263}$$

with the same Lagrange constant k for every component. Following Michelsen, introduce *unnormalised* mole numbers $Y_i = y_i\,e^{-k}$, so that $\sum_i Y_i = e^{-k}$ and $y_i = Y_i / \sum_j Y_j$. In these variables the stationarity condition (263) becomes

the fixed point

$$\ln Y_i \;=\; h_i - \ln\gamma_i(\mathbf{y}) \qquad\Longleftrightarrow\qquad Y_i \;=\; \exp\!\big[\,h_i - \ln\gamma_i(\mathbf{y})\,\big], \tag{264}$$

which suggests the **successive-substitution** iteration that `runProbe` performs:

$$Y_i^{(m+1)} \;=\; \exp\!\Big[\,h_i - \ln\gamma_i\big(\mathbf{y}^{(m)}\big)\,\Big], \qquad y_i^{(m+1)} \;=\; \frac{Y_i^{(m+1)}}{\sum_j Y_j^{(m+1)}}. \tag{265}$$

The $\gamma_i(\mathbf{z})$ in h_i is computed *once*; each iteration costs one activity-coefficient evaluation at the trial composition. At a converged stationary point Michelsen shows the tangent-plane distance reduces to a function of the unnormalised Y alone,

$$\mathrm{tm} \;=\; 1 \;+\; \sum_{i=1}^{n} Y_i\,\big(\ln Y_i - 1\big) \;-\; \sum_{i=1}^{n} Y_i\,h_i, \tag{266}$$

which is exactly the `evalTm` helper in the source. The useful identity is $\mathrm{tm} = 1 - \sum_i Y_i$ at convergence (because $\ln Y_i = h_i - \ln\gamma_i$ there), so the classic shortcut "$\sum_i Y_i > 1 \Rightarrow$ unstable" is the same test as $\mathrm{tm} < 0$. Choupo reports and decides on tm directly, declaring instability when

$$\mathrm{tm}_{\min} \;<\; \mathrm{tm}_{\mathrm{tol}} \qquad (\text{default } \mathrm{tm}_{\mathrm{tol}} = -10^{-3}), \tag{267}$$

the small negative threshold absorbing round-off so that a marginally negative tm at the trivial point is not mistaken for a true split.

Multi-component probing: which starting points

Equation (265) only finds *a* stationary point —

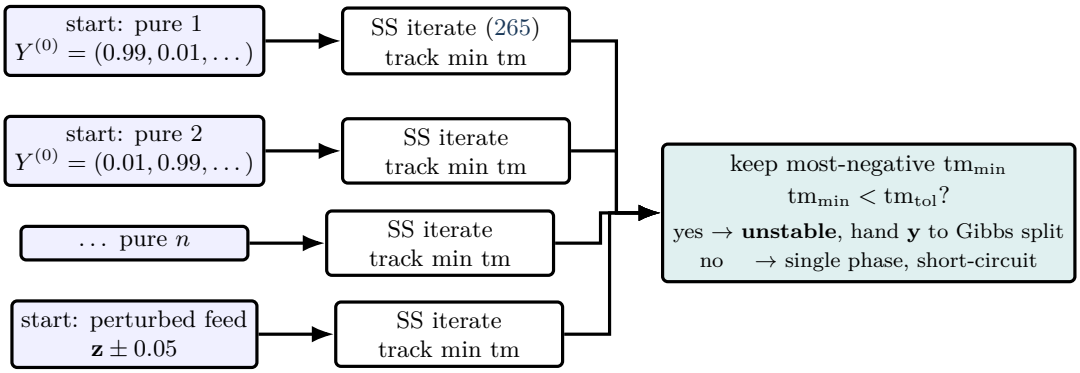

Figure 59: Why Michelsen's test launches *several* probes. The tangent-plane-distance surface usually has more than one stationary point — one per liquid well, plus the trivial feed point that is always stationary — and successive substitution (265) only finds the one in whose basin it starts. So `michelsenTPD` fires $n{+}1$ starts: one heavily biased toward each pure component (to land in that species' well) and one mildly perturbed feed (to catch shallow off-feed instabilities). Each probe records the *lowest* tm visited *along its whole trajectory*, not just the endpoint — the cheap guard against drifting back to the trivial saddle on a symmetric system. The most-negative tm across all probes decides: below the tolerance it declares a split and hands its $\mathbf{y}$ to the Gibbs minimisation; otherwise the feed is one stable phase and the expensive search is skipped.

the one in whose basin the start $\mathbf{y}^{(0)}$ lives. The TPD surface generally has several stationary points (one per liquid "well", plus the trivial point $\mathbf{y} = \mathbf{z}$ which is always stationary), and *any* of them with $\mathrm{tm} < 0$ proves instability. To hit every well at least once, `michelsenTPD` launches $n + 1$ probes for an n-component feed:

- n **pure-component starts.** Probe k begins with a heavy bias toward pure k: $Y_k^{(0)} = 0.99$ and the remaining 0.01 split evenly. These find the wells dominated by each species — the water-rich and butanol-rich liquids in an LLE gap.
- **one mildly perturbed-feed start.** $Y^{(0)} = \mathbf{z}$ nudged by ± 0.05 in the first component, then renormalised. This catches "weak" instabilities where a pure-component start overshoots the shallow off-feed well.

The probe with the lowest (most negative) tm wins; its converged $\mathbf{y}$ is returned as `trialY`, the incipient-phase seed handed to the Gibbs-minimisation split of §9, and `probeIndex` records which start found it (pedagogically: *which* pure component the second liquid is rich in).

Common pitfall. For a *symmetric* LLE — the same NRTL model on both phases, as in water/n-butanol — the feed itself, $\mathbf{y} = \mathbf{z}$, is a stationary point of (265) where the gradient of tpd vanishes. Plain successive substitution can drift *back* to this trivial point even from a strongly biased start, reporting tm ≈ 0 and missing the real split. Choupo's `runProbe` guards against this without paying for Michelsen's full second-order Newton: it records the *lowest* tm *visited along the whole trajectory*, not just the endpoint. The path from a pure-component start *passes through* the genuine off-diagonal well on its way back to the trivial saddle, so the trajectory minimum captures the true incipient composition. This is a deliberate, cheap substitute — adequate for the binary and ternary LLE/VLLE cases Choupo targets; a high-component industrial flash would want the real Newton-on-tpd of Michelsen Part I.

Worked example 9.1. TPD verdict on water/1-butanol at 25 °CTake the `extraction01_waterButanol` feed, $\mathbf{z} = (0.70, 0.30)$ (water, butanol), with the gap-tuned NRTL. The reference vector is $h_i = \ln(z_i \gamma_i(\mathbf{z}))$, evaluated once. The pure-water probe ($Y^{(0)} = (0.99, 0.01)$) iterates Eq. (265) and its trajectory dips to a markedly negative tm as $\mathbf{y}$ passes through the aqueous well ($x_{\text{water}} \approx 0.98$); the pure-butanol probe finds the organic well. The reported $\text{tm}_{\min} \ll 0$ clears the threshold (267), so the flash prints

```
Michelsen TPD stability test on phase 'liquid':
      tm_min = -X.XXXXe-02   unstable = YES
```

and proceeds to the two-liquid Gibbs minimisation, seeded with the winning `trialY`. Had the same routine been run on an ideal or fully-miscible feed, every probe would converge to tm ≥ 0, the test would print `unstable = no`, and the flash would short-circuit to "single liquid (no LL split detected by TPD)" without ever paying for the simplex search — the whole point of a *pre*-check.

Intuition. The TPD test and the Gibbs minimisation are the two halves of one honest answer. The minimisation *would* return the feed itself ($\beta \to 0$) when no split exists, so in principle it could decide stability on its own — but only after paying for a full multi-start search. The TPD test answers the cheap yes/no question first, and *when* the answer is yes it hands over a vetted starting phase, turning the expensive search into a short polish. Glass-box pedagogy: the student sees the tm number that justified attempting a split, not a silent branch.

Power and limit. The power of the tangent-plane test is that it converts a global, geometric question — "does any trial phase undercut the tangent plane?" — into a handful of cheap fixed-point iterations whose endpoints are the only places a negative value can hide, and it does so using *only* the activity model already in hand (no second derivatives, no EoS). Its limit is inherited from successive substitution: it locates stationary points but does not *guarantee* it has found the global minimum of tpd unless every basin is seeded. Choupo's $n + 1$ canonical starts plus the trajectory-minimum guard cover the binary and ternary landscapes it targets; a feed with many near-degenerate liquid wells could still slip a split past a too-sparse probe set — which is why Michelsen Part I's second-order Newton on tpd remains the rigorous tool on the roadmap.

The Gibbs surface

For a two-phase split of a feed of composition $\mathbf{x}_z$ at temperature T and pressure P, with $\beta \in (0, 1)$ moles of phase β per mole of feed, the Gibbs energy of mixing per mole of feed (in units of RT) is

$$\frac{G_{\text{mix}}}{RT} \;=\; (1-\beta)\sum_i x_{\alpha,i}\left[\ln x_{\alpha,i} + \ln \gamma_{\alpha,i}(\mathbf{x}_\alpha)\right] \;+\; \beta \sum_i x_{\beta,i}\left[\ln x_{\beta,i} + \ln \gamma_{\beta,i}(\mathbf{x}_\beta)\right].$$

The first term is the ideal-mixing contribution, $x \ln x$, summed over each phase; the second is the excess part $x \ln \gamma$ that encodes the activity model. Overall mass balance ties the two phase compositions:

$$z_i \;=\; (1-\beta)\, x_{\alpha,i} + \beta\, x_{\beta,i} \quad\Longrightarrow\quad x_{\beta,i} \;=\; \frac{z_i - (1-\beta)\, x_{\alpha,i}}{\beta}.$$

We minimise G_{mix} over the free variables $(\beta, x_{\alpha,1}, \ldots, x_{\alpha,n-1})$; the last alpha component follows from $\sum_i x_{\alpha,i} = 1$, and the entire $\mathbf{x}_\beta$ from the mass balance above. An infeasible probe (one where some $x_{\beta,i}$ would come out negative) gets a large penalty so the optimiser steers back into the physical simplex.

Why direct minimisation succeeds where Newton fails

At $\mathbf{x}_\alpha = \mathbf{x}_\beta = \mathbf{x}_z$ (the K = 1 trivial), the equal-activity constraint $\gamma_i(\mathbf{x}_z)\, z_i = \gamma_i(\mathbf{x}_z)\, z_i$ is satisfied for every i, the mass balance is trivially closed for any β, and the normalisation constraints are met. All residuals are zero. A Newton method tracking $\mathbf{F}(\mathbf{v}) = \mathbf{0}$ has no way to know that this point is a saddle of G rather than a minimum — residual zero is residual zero — so it accepts the point. Even worse, damped Newton from $(\mathbf{x}_\alpha, \mathbf{x}_\beta)$ biased toward pure components overshoots and lands on the trivial because the basin of attraction extends out into the entire simplex along the equal-composition diagonal.

Direct minimisation never makes this mistake. The simplex algorithm samples G values; a saddle point of G has neighbouring points with *lower* G, so any simplex vertex placed there will be displaced on the next move. Multi-start gives global search — each start follows a different downhill path — and the lowest G across all converged points is the global minimum (or the feed-single phase, in which case $G(\mathbf{x}_z) < G_{\text{split}}$ and we report "no LL split exists for these parameters").

What Choupo reports

For `extraction01_waterButanol` (`phaseSet LL`, NRTL parameters tuned to the 25 °C water / 1-butanol gap), the routine converges in 4 multi-starts × ∼ 30 simplex iterations to:

Phase	x_{water}	x_{butanol}	flow [kmol/h]
feed ($\mathbf{x}_z$)	0.700	0.300	100.0
aqueous (α)	0.984	0.016	50.6
organic (β)	0.408	0.592	49.4

The aqueous phase carries essentially all the water; the organic phase has substantial water dissolved in butanol, consistent with the literature tie-line at 25 °C. The 2n+1 Newton that the same case ran returned $\beta = -0.5$ (clamped to zero on output) and the regime "two-phase liquid" with both phases indistinguishable — the classic K = 1 trap.

Beyond two liquids: VLLE (delivered)

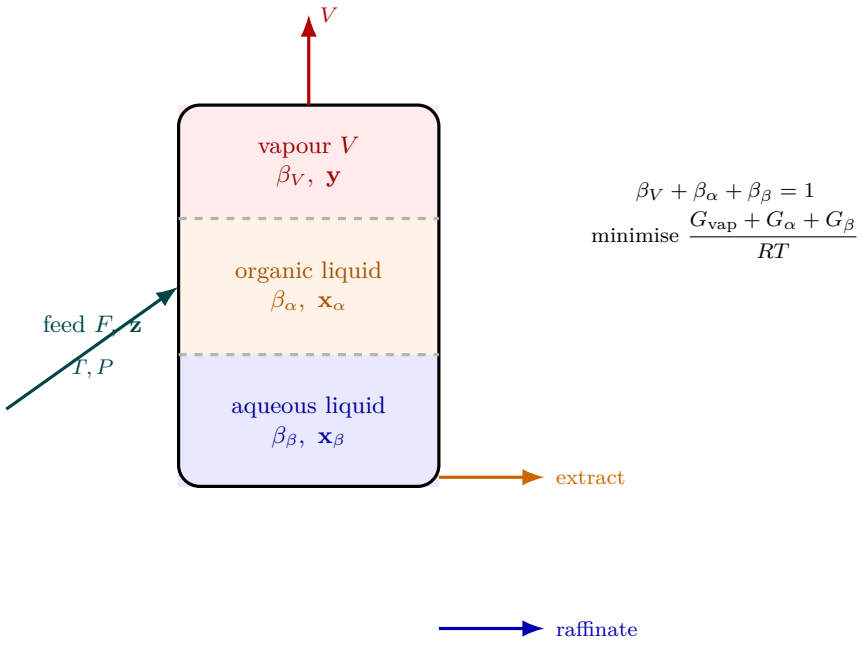

Figure 60: The three-phase (VLLE) flash: one feed disengages into a vapour and *two* immiscible liquids that the Gibbs-min flash finds by adding a vapour term (ideal-gas reference relative to pure liquid) to the two-liquid surface. The free variables grow to $2n$ — the phase fractions β_V, β_α (with $\beta_\beta = 1 - \beta_V - \beta_\alpha$) plus the vapour and α-liquid compositions — and direct Nelder–Mead minimisation of the *total* G/RT over the simplex locates the global split. Six multi-starts deliberately bias the aqueous liquid to water-rich corners ($x_{\alpha,\text{water}} \approx 0.97$–$0.99$); without that seeding the simplex misses the narrow heteroazeotrope basin and collapses to a two-phase answer. For a binary this coexistence is a single curve in (T, P) (phase rule $F = 2 + 2 - 3 = 1$).

The same direct-minimisation approach extends to three coexisting phases (vapour plus two liquids) by adding a vapour Gibbs term with the ideal-gas reference state relative to pure liquid:

$$\frac{G_{\text{vap}}}{RT} \;=\; \beta_V \sum_i y_i \left[\ln y_i + \ln\!\left(\frac{P}{P_i^{\text{sat}}(T)} \right) \right].$$

Free variables grow to $2n$: $(\beta_V, \beta_\alpha, y_0, \ldots, y_{n-2}, x_{\alpha,0}, \ldots, x_{\alpha,n-2})$, with β_β from the phase fractions summing to 1 and the entire $\mathbf{x}_\beta$ from overall mass balance per component. The `IsothermalFlash::PhaseSet::VLLE` branch implements exactly this, with six multi-starts that explicitly bias the aqueous liquid simplex to extreme water-rich

compositions ($x_{\alpha,\text{water}} \approx 0.97$–$0.99$) — without this seeding, Nelder-Mead misses the narrow heteroazeotrope basin and the optimum collapses to two-phase V+L or LL. Binary three-phase coexistence is a one-dimensional curve in (T, P) by Gibbs phase rule ($F = 2 + 2 - 3 = 1$); ternaries give a two-dimensional region ($F = 2$) and are pedagogically more robust.

Beyond Gibbs-minimisation Nelder-Mead

Nelder-Mead works well up to about ten dimensions. For very high-component LLE/VLLE, or for cases that need quadratic convergence, the Naphtali-Sandholm formulation of Newton on log-K-values (a piece) reaches a smaller cost per iteration once the basin is correctly identified by Gibbs minimisation. The minimisation thus plays the role of a globally robust pre-conditioner; Newton finishes the polish.

9.2 Ternary phase diagrams: sweeping the simplex

A ternary diagram is not a new piece of physics — it is the flash of this chapter (and the bubble point of §3) evaluated over a grid of feed compositions on the composition triangle $x_1 + x_2 + x_3 = 1$. At each interior node the engine calls the *same* solver a single `isothermalFlash` would, so the map and a one-off flash can never disagree:

- **Solubility (LLE) map.** The liquid–liquid flash classifies each node as one phase or two liquids; the boundary between them traces the *binodal* (miscibility) curve, and the equilibrium pair of liquid compositions at a split node is a *tie-line*. Using the liquid–liquid branch (no vapour) gives the clean extraction/decanter triangle that students recognise — water / ethanol / benzene, say, with a large two-liquid region and tie-lines sloping toward the water–benzene edge.
- **Boiling surface (VLE).** Bubble-point at each node gives a temperature *surface* over the triangle; it needs only the three vapour pressures, so even ideal binaries produce a meaningful map.

Predicting the gap without fitted pairs. The miscibility gap is governed by the activity model (§2). NRTL/Wilson need a *regressed* binary pair for every edge; absent one, that edge defaults to ideal and the gap vanishes. **UNIFAC** sidesteps this: it predicts γ from molecular *groups* (CH_2, OH, ACH, H_2O, ...) with no system-specific fit, so a real ternary shows a real liquid–liquid gap with zero curated pair data — the practical key that makes the solubility map usable for arbitrary mixtures whose groups are tabulated. A component without a group decomposition simply falls back to $\gamma = 1$ (ideal) for itself; the model never invents data.

10 Counter-current liquid–liquid extraction

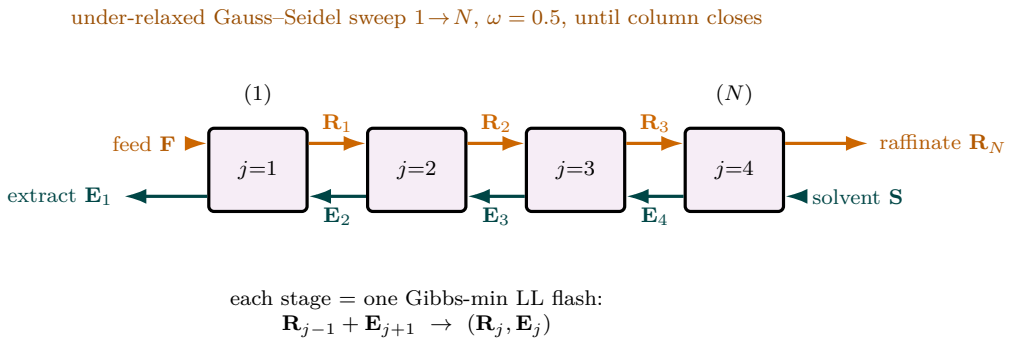

Figure 61: The counter-current extraction cascade. Feed **F** enters stage 1 and fresh solvent **S** enters stage N; the feed-derived *raffinate* (orange) flows $1 \to N$ while the solvent-rich *extract* (teal) flows $N \to 1$, so the leaving extract has met the richest feed and the leaving raffinate the cleanest solvent. Each stage is *exactly* one liquid–liquid Gibbs-min flash (Fig. 60's two-liquid cousin): it takes the combined inlet $\mathbf{R}_{j-1} + \mathbf{E}_{j+1}$ and splits it into the two equilibrium liquids. Because the two directions are coupled, the cascade is a fixed-point problem; Choupo tears it by under-relaxed Gauss–Seidel sweeps ($\omega = 0.5$, the raw update oscillates) and stops not on a noisy per-stage delta but on the honest test that what leaves the *column* equals what entered it.

> *What you'll learn.* A single liquid–liquid flash (Chapter 9) splits one mixed feed into two equilibrium liquids — a decanter. A *column* does better: a feed and a fresh solvent enter at *opposite* ends and pass each other counter-currently over N theoretical stages, so the leaving extract has seen the richest feed and the leaving raffinate has seen the cleanest solvent. The chemistry of each stage is exactly the Gibbs-minimisation LL flash you already know; the new content is the *cascade* — how the stages are linked and torn. Choupo solves it by under-relaxed stage-by-stage successive substitution (Gauss–Seidel sweeps), the liquid–liquid twin of the Wang–Henke bubble-point loop used for distillation (Chapter 13) and of the coupled absorber/stripper sweep (Chapter 14). The implementation is in `src/unitOperations/separation/Extractor.cpp`; the type is `extractor` (with `extract` as an alias). It calls the shared flash kernel `IsothermalFlash::solveCore` read-only — it never re-implements the equilibrium split.

10.1 The cascade and its variables

Number the stages $j = 1, \dots, N$. The **feed** enters at stage 1 and the **fresh solvent** at stage N. Two liquid streams flow in opposite directions:

- the *raffinate* (feed-derived, carrier-rich) cascades $1 \to N$ and leaves the column at stage N;
- the *extract* (solvent-rich) cascades $N \to 1$ and leaves at stage 1.

Write $\mathbf{R}_j$ and $\mathbf{E}_j$ for the component-molar flow vectors (n species each) of the raffinate and extract *leaving* stage j. The two streams arriving at stage j are the raffinate descending from $j-1$ and the extract rising from $j+1$, with the column ends supplied by the external feeds:

$$\text{into stage } j \;=\; \underbrace{\mathbf{R}_{j-1}}_{(\text{feed } \mathbf{F} \text{ if } j=1)} \;+\; \underbrace{\mathbf{E}_{j+1}}_{(\text{solvent } \mathbf{S} \text{ if } j=N)} . \tag{268}$$

The two leaving streams $(\mathbf{R}_j, \mathbf{E}_j)$ are the two liquid phases that this combined inlet splits into at the column temperature T and pressure P. Counting unknowns: $2nN$ component flows, tied by N two-phase equilibria. This is precisely the structure of the MESH equations restricted to two liquid phases with no vapour and (by the isothermal assumption below) no per-stage energy balance.

10.2 One stage is one Gibbs-min flash

Stage j takes its combined inlet (268), of total moles $F_j = \sum_i (\text{into stage } j)_i$ and overall composition $z_{j,i} = (\text{into stage } j)_i / F_j$, and hands it verbatim to the liquid–liquid flash of Chapter 9:

$$z_{j,i} \;=\; (1-\beta_j)\, x_{\alpha,i} + \beta_j\, x_{\beta,i}, \qquad \beta_j = \arg\min_{\beta,\,\mathbf{x}_\alpha} \frac{G_{\text{mix}}(\beta, \mathbf{x}_\alpha, \mathbf{x}_\beta)}{RT}, \tag{269}$$

solved by multi-start Nelder–Mead on the Gibbs surface, *not* by Newton on the equal-activity residual — this is the whole point of Chapter 9, and the Extractor inherits it for free. Of the two phases the flash returns, the one richer in the designated **solvent species** s (the largest component of the solvent feed, or the user's `solvent` key) is labelled the extract; the other is the raffinate:

$$\mathbf{E}_j \;=\; \text{phase with the larger } x_s, \qquad \mathbf{R}_j \;=\; \text{the other phase.} \tag{270}$$

The same species choice seeds the flash's α/β basin bias (s for the extract-rich phase, the feed's dominant carrier for the raffinate-rich phase), so each stage's Gibbs minimisation starts in the correct well. **Robust degeneracy handling:** if a stage's flash finds no genuine split ($\beta_j \le 10^{-6}$ or $\ge 1-10^{-6}$, or it fails to converge), the whole inlet leaves as raffinate ($\mathbf{R}_j = $ inlet, $\mathbf{E}_j = \mathbf{0}$) so the cascade mass balance stays exact; the count of two-phase stages is reported (`splitStages`) so a column operating below its solubility window is *visible* rather than silently wrong.

10.3 Tearing the cascade: under-relaxed Gauss–Seidel

The inter-stage flows in (268) are coupled in both directions, so the cascade is a fixed-point problem. Choupo tears it by successive substitution exactly as a distillation column tears its bubble-point loop:

1. **Initialise** honestly by topology propagation — every stage's extract $\mathbf{E}_j \leftarrow \mathbf{S}$ (the fresh solvent) and every raffinate $\mathbf{R}_j \leftarrow \mathbf{F}$ (the feed). No magic constant; the feeds themselves are the seed (cf. the no-silent-crutch credo).
2. **Sweep** $j = 1 \to N$ (Gauss–Seidel: each stage uses the *freshest* available neighbours). Build the inlet (268), flash it (269), and *under-relax* the update with factor $\omega = 0.5$:

$$\mathbf{R}_j \leftarrow (1-\omega)\,\mathbf{R}_j + \omega\,\mathbf{R}_j^{\text{flash}}, \qquad \mathbf{E}_j \leftarrow (1-\omega)\,\mathbf{E}_j + \omega\,\mathbf{E}_j^{\text{flash}}. \tag{271}$$

   Under-relaxation is essential: a raw Gauss–Seidel update of counter-current cascades oscillates, because changing $\mathbf{E}_j$ changes the inlet of stage $j-1$, which changes *its* extract, and so on back up the column.
3. **Iterate** until the *column mass balance closes*.

Why the convergence test is the overall closure, not $|\Delta F|$. Each per-stage flash splits its own inlet to machine tolerance, but the Nelder–Mead simplex stops at a finite `tolX` $\sim 10^{-5}$ on compositions, so the inter-stage flows carry a small flash-noise floor that the under-relaxed sweeps grind down. The per-step residual $\max_j |\Delta\mathbf{R}_j|, |\Delta\mathbf{E}_j|$ limit-cycles on that floor and is a poor convergence proxy. The physically meaningful metric is whether what leaves the *column* equals what entered it:

$$\varepsilon_{\text{close}} \;=\; \frac{\big|\,(F_{\mathbf{F}} + F_{\mathbf{S}}) - (F_{\mathbf{E}_1} + F_{\mathbf{R}_N})\,\big|}{F_{\mathbf{F}} + F_{\mathbf{S}}}, \tag{272}$$

the relative gap between the two feeds in and the extract-plus-raffinate out. The sweep stops when $\varepsilon_{\text{close}} < 10^{-4}$, or when it *stalls* (60 sweeps with no closure improvement). On a stall the column reports the best state reached but only flags `converged` if the closure it achieved is still acceptable ($< 10\,\varepsilon_{\text{close}}^{\text{tol}}$); otherwise the `converged` KPI stays 0 and the run warns aloud — numerical honesty over a green light. The closure $\varepsilon_{\text{close}}$ is pushed to `recordResidual` each sweep so the convergence plot shows the cascade settling.

10.4 What the column reports

The two products and the per-species accounting follow directly from the torn cascade:

$$\text{extract product} = \mathbf{E}_1 \qquad \text{(solvent-rich, leaves stage 1)}, \tag{273}$$

$$\text{raffinate product} = \mathbf{R}_N \qquad \text{(feed-derived, leaves stage } N\text{)}, \tag{274}$$

$$\text{recovery}_i = \frac{E_{1,i}}{F_{\mathbf{F},i} + F_{\mathbf{S},i}} \qquad \text{(fraction of species } i \text{ leaving in the extract)}, \tag{275}$$

$$\text{distribution}_i = \frac{z_{\mathbf{E}_1,i}}{z_{\mathbf{R}_N,i}} \qquad \text{(the partition ratio } K_i = x_{i,E}/x_{i,R}\text{)}. \tag{276}$$

The **solute** is identified automatically as the species — other than the solvent s and the feed carrier — most enriched into the extract, and its recovery and distribution ratio are headlined; a full per-species breakdown follows. Note the credo-pure interface: the `operation` block carries only *hardware* — `stages` N and the optional shared `temperature` — while the solute recovery, the distribution ratios and the product flows are all *results* of turning the crank, never inputs.

Power and limit

Power. The column reuses the saddle-proof Gibbs-min flash, so it is robust on the same symmetric-γ systems that defeat equal-activity Newton; combined with UNIFAC (§9.2) it can screen extraction of a solute by an arbitrary solvent with *zero* fitted pair data. The counter-current cascade captures the central design trade-off — more stages or more solvent buy more recovery — and the stage profile (`xE_`/`xR_` versus stage) lets a student watch the solute

concentrate toward stage 1 exactly as in a McCabe–Thiele construction, but read off a rigorous activity model rather than a single constant K.

Limit. The model is *isothermal*: one temperature for the whole column (the feed's, unless overridden), because LLE is weakly enthalpic and a per-stage energy balance is deliberately out of scope — the same fallback the absorber and stripper take when c_p data are absent. Stages are *theoretical* (Murphree-style stage efficiency and the HETP/mass-transfer rate that fix a real packed-column height are not modelled). The two products are taken to be the two declared `liquid` phases, so the thermoPackage must carry ≥ 2 liquid phases or the unit refuses to run. There is no entrainer/decanter solvent-recovery loop — the extract leaves with its solvent, and recovering it (e.g. by distillation) is a downstream unit in the flowsheet. Finally, the cascade is solved by successive substitution, not the quadratically convergent Naphtali–Sandholm Newton; for routine ternary extraction it converges in tens of sweeps, but very stiff, many-component columns would benefit from the Newton polish noted at the end of Chapter 9.

Worked sketch: a 3-stage solvent wash

Take a feed of 100 kmol/h with mole fractions $(\text{carrier}, \text{solute}) = (0.80, 0.20)$ contacted counter-currently with 150 kmol/h of pure solvent over $N = 3$ stages at fixed T. Initialise $\mathbf{E}_j = \mathbf{S}$ and $\mathbf{R}_j = \mathbf{F}$ on all three stages. Sweep 1 flashes stage 1 (feed + extract-from-2): the solute distributes between the carrier-rich raffinate and the solvent-rich extract per (269), and under-relaxation moves the flows halfway. Stages 2 and 3 follow, stage 3 seeing the fresh solvent directly. Each subsequent sweep tightens the inter-stage flows; the closure (272) falls (modulo the flash-noise floor) and the run stops when it drops below 10^{-4}. The extract leaving stage 1 carries the solute that three equilibrium contacts could strip from the carrier; pushing N or the solvent-to-feed ratio up raises the headline `soluteRecovery` — the lever the student turns. The natural runnable analogue is the water/1-butanol gap of `extraction01_waterButanol` promoted from a single decanter to a multistage column.

11 Stoichiometric conversion reactor

What you'll learn. The CSTR and PFR ask *how far* a reaction goes given a rate law and a residence time. Sometimes you do not have — and do not want — a rate law: you simply *know* the per-pass conversion of a reactant (measured in a pilot plant, set by the catalyst, quoted in a textbook problem). The `conversionReactor` takes that single number and turns the crank of stoichiometry: one specified conversion fixes the reaction extent ξ, and every outlet flow follows from $n_{i,\mathrm{out}} = n_{i,\mathrm{in}} + \nu_i\,\xi$. No kinetics, no iteration, no liquid volume — an *algebraic* reactor. This chapter derives that one line, shows why a *finite* conversion is exactly what makes a recycle loop meaningful, and contrasts it with the Gibbs reactor of the next chapter. The implementation is in `src/unitOperations/reactor/ConversionReactor.cpp`; the runnable case is `tutorials/steady/reactors/conversion01_hda/`.

11.1 The extent of reaction, made primary

In every reactor chapter so far the extent ξ was an *unknown* solved for — a Newton root (CSTR), the endpoint of an integration (PFR), or the minimiser of a free energy (Gibbs). The conversion reactor inverts the question: it makes ξ a *direct* consequence of one user-supplied number, the fractional conversion X of a chosen **limiting reactant** ℓ.

Recall the definition of conversion for reactant ℓ over a single reaction with stoichiometry $\boldsymbol{\nu}$ (negative for reactants, positive for products, exactly as in Section 6.3):

$$X \;\equiv\; \frac{n_{\ell,\mathrm{in}} - n_{\ell,\mathrm{out}}}{n_{\ell,\mathrm{in}}}, \qquad 0 \le X \le 1. \tag{277}$$

The extent that delivers this conversion follows by combining (277) with the stoichiometric outlet relation for the limiting species, $n_{\ell,\mathrm{out}} = n_{\ell,\mathrm{in}} + \nu_\ell\,\xi$:

$$\boxed{\;\xi \;=\; \frac{X\,n_{\ell,\mathrm{in}}}{-\nu_\ell}\;} \qquad (\nu_\ell < 0). \tag{278}$$

The denominator $-\nu_\ell > 0$ converts "moles of ℓ consumed" into "moles of reaction-as-written". With the feed molar flow F_{in} and feed mole fractions $\mathbf{z}$, the inlet flow of the limiting reactant is $n_{\ell,\mathrm{in}} = z_\ell\,F_{\mathrm{in}}$, exactly the line `xi = X * molesLim_in / (-nu[iLim])` in the source.

Once ξ is known, *every* species — reactants, products, inerts — is propagated by the one stoichiometric balance:

$$\boxed{\;n_{i,\mathrm{out}} \;=\; n_{i,\mathrm{in}} + \nu_i\,\xi \;=\; z_i\,F_{\mathrm{in}} + \nu_i\,\xi, \qquad i = 1,\dots,N_c\;} \tag{279}$$

with $\nu_i = 0$ for any species not in the reaction (inerts pass through untouched). The outlet flow and composition are then $F_{\mathrm{out}} = \sum_i n_{i,\mathrm{out}}$ and $z_{i,\mathrm{out}} = n_{i,\mathrm{out}}/F_{\mathrm{out}}$. That is the *entire* mass balance: no equation to solve, no Jacobian, no tolerance — the reactor is one pass of arithmetic.

Intuition. The extent ξ is the single "turn of the crank" of the balanced equation. Conversion X measures how far one specific reactant turned; ξ measures how far the reaction-as-written turned. Equation (278) is just the gearing between the two — $-\nu_\ell$ is the gear ratio. Choose the gear (ℓ), read off how far it turned (X), and (279) reports where every other gear ended up. Because all species share the *same* ξ, stoichiometry — hence atom balance — is satisfied exactly and for free.

11.2 Honesty guards in the algorithm

The glass-box implementation refuses to silently produce nonsense:

- The named `limitingReactant` must have $\nu_\ell < 0$ (it must actually be consumed); a non-negative ν_ℓ is a hard error, because (278) would otherwise divide by a product or give a negative extent.
- X is range-checked to $[0, 1]$.
- Every outlet flow is checked for negativity after (279). If $n_{i,\mathrm{out}} < 0$, the conversion was placed on the *wrong* (non-limiting) reactant — some other reactant ran out first — and the run aborts with a message saying exactly that. A tiny negative round-off (within 10^{-12}) is tidied to zero.

This last guard is the pedagogical payoff of naming a limiting reactant: the code will not let you ask for 90 % conversion of a species that is only present in 50 % of the stoichiometric amount.

11.3 The isothermal duty

The conversion reactor runs *isothermal*: the outlet temperature is the operating temperature T (defaulting to the feed temperature if none is given). The duty needed to hold T is the heat of reaction times the extent. Using the elements (formation) datum of Section 6.3 — the only datum that survives a reactor — the heat of reaction per mole of extent is

$$\Delta h_{\mathrm{rxn}}(T) \;=\; \sum_i \nu_i \, h_i^{\mathrm{ig}}(T), \tag{280}$$

evaluated at the reactor temperature from each species' absolute ideal-gas enthalpy `Component::h_pure_ig(T)` (which carries Δh_f° as its floor). The isothermal duty is then

$$Q \;=\; \xi \, \Delta h_{\mathrm{rxn}}(T), \tag{281}$$

negative (heat removed) for an exothermic reaction, positive (heat added) for an endothermic one. If any participating species lacks ideal-gas c_p data, the duty is simply not reported rather than faked — the mass balance still stands on its own.

> *Intuition.* Notice what the conversion reactor does *not* do: it does not let the heat of reaction feed back on the conversion. X is yours to set; Q is the consequence. This is the deliberate opposite of the *adiabatic* Gibbs reactor, where T floats until the released heat and the equilibrium are mutually consistent. Here the chemistry is decoupled from the energy balance by construction — which is exactly right when the conversion is fixed by a catalyst bed cooled to a set temperature, and wrong if you meant "let it run adiabatically".

11.4 Why a *finite* conversion is the whole point of a recycle

A Gibbs reactor drives to equilibrium; a conversion reactor stops at the number you give it. The second is what real recycle processes need. Take hydrodealkylation of toluene (HDA), the canonical recycle flowsheet:

$$\mathrm{C_7H_8} + \mathrm{H_2} \;\longrightarrow\; \mathrm{C_6H_6} + \mathrm{CH_4},$$

run with hydrogen in excess so toluene is limiting. Per pass only ∼75 % of the toluene reacts; the unconverted 25 % is separated downstream and *recycled*. The per-pass conversion is set by the reactor (residence time, temperature) — it is an input, not an equilibrium output. If you modelled this reactor as a Gibbs reactor it would convert essentially *all* the toluene (the equilibrium lies far to the right), the recycle stream would shrink to nothing, and the whole reason for the loop would vanish. The conversion reactor's *finite, specified* X is precisely what leaves material in the recycle and makes the tear-stream solve of Chapter 6 non-trivial.

Worked example 11.1. One pass of the HDA reactorFeed: $F_{\mathrm{in}} = 400$ kmol/h at 900 K, 35 bar, mole fractions $z_{\mathrm{tol}} = 0.25$, $z_{\mathrm{H_2}} = 0.75$ (benzene and $\mathrm{CH_4}$ zero). Toluene is the limiting reactant with $\nu_{\mathrm{tol}} = -1$; specified conversion $X = 0.75$.
Extent. Inlet toluene $n_{\mathrm{tol,in}} = 0.25 \times 400 = 100$ kmol/h, so by (278)

$$\xi = \frac{0.75 \times 100}{-(-1)} = 75 \text{ kmol/h}.$$

Outlets by (279) (kmol/h):

species	n_{in}	$\nu_i\,\xi$	n_{out}
toluene	100	−75	25
$\mathrm{H_2}$	300	−75	225
benzene	0	+75	75
$\mathrm{CH_4}$	0	+75	75
total	400	0	400

Total moles are conserved here only because $\sum_i \nu_i = -1 - 1 + 1 + 1 = 0$ for this reaction; in general the outlet total shifts by $\xi \sum_i \nu_i$. The 25 kmol/h of unconverted toluene is what a downstream separator returns to the feed — the recycle. No Newton iteration was performed; the answer is exact stoichiometry.

> **Runnable case.** `tutorials/steady/reactors/conversion01_hda` — the single-pass reactor above. The full recycle loop (with separation and tear-stream Newton) is `tutorials/plant/hda`.

11.5 Conversion vs. Gibbs: two opposite idealisations

The conversion reactor and the Gibbs reactor of the next chapter are

the two extreme idealisations of a reactor, and choosing between them is a modelling decision the student should make consciously:

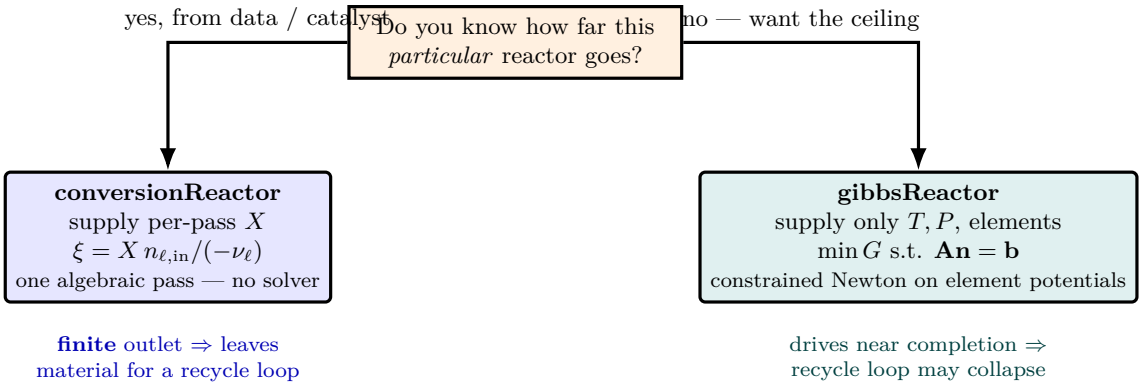

Figure 62: The two extreme reactor idealisations, and the conscious modelling choice between them. The **conversion reactor** answers "what does *this* reactor actually do?" — you supply the per-pass conversion X (known from a catalyst or plant data) and the outlet falls out of stoichiometry in a single algebraic pass, no solver. The **Gibbs reactor** answers "what is the *best* this chemistry could do?" — you supply nothing about extent, only T, P and the element inventory, and the outlet is the composition that minimises G subject to atom conservation. The recycle consequence is the deciding factor: a finite conversion leaves material for the loop, while equilibrium often drives to near-completion and the loop collapses.

	conversionReactor	**gibbsReactor**
what you supply	a per-pass conversion X	nothing about extent — only T, P, elements, species
what fixes the outlet	stoichiometry, $\xi = X n_{\ell,\mathrm{in}}/(-\nu_\ell)$	minimisation of G over the atom-conserving set
computation	one algebraic pass	constrained Newton on element potentials
reactions	one specified reaction	all combinations the species list allows
recycle behaviour	**finite outlet** — leaves material for the loop	drives (often) to near-completion — loop may collapse
when it is right	conversion known from data / catalyst; recycle processes	fast kinetics, conversion unknown, want the equilibrium ceiling

Intuition. The Gibbs reactor answers "what is the *best* this chemistry could do?"; the conversion reactor answers "what does this *particular* reactor actually do?". A recycle flowsheet lives or dies on the second question — which is why the conversion reactor, despite being the simplest unit in the whole simulator (no solver at all), is the one that makes the most interesting flowsheets work.

12 Gibbs reactor: equilibrium by free-energy minimisation

What you'll learn. The CSTR and PFR of the previous chapters are *kinetic* reactors: they tell you what composition you get from a given residence time and rate law, irrespective of whether the result is at equilibrium. The Gibbs reactor is the opposite limit — it assumes kinetics are infinitely fast and computes the composition that minimises the total Gibbs energy at the specified T and P, subject to the atomic elements being conserved. Useful whenever the timescale for reaction is much shorter than the residence time (reforming, high-temperature combustion, ammonia synthesis at scale), or as an upper bound on conversion when kinetic data are unavailable. This chapter derives the algorithm Choupo uses — Lagrangian multipliers on the element-balance constraints, one π_j per element — and walks through the water-gas-shift tutorial end-to-end. The implementation is in `src/unitOperations/reactor/GibbsReactor.cpp`, and the runnable case is `tutorials/gibbs01_water_gas_shift/`.

12.1 The problem

Given a feed of mole numbers $\mathbf{n}^{\text{in}}$ over N species, a reactor temperature T, a reactor pressure P and the element-composition matrix $\mathbf{A} \in \mathbb{R}^{M\times N}$ (so A_{ji} is the number of atoms of element j in species i), find the equilibrium mole numbers $\mathbf{n}$ that minimise the total molar Gibbs energy of the (single-phase, ideal-gas) mixture:

$$\min_{\mathbf{n}} \; G(\mathbf{n}) \;=\; \sum_{i=1}^{N} n_i\,\mu_i(T,P,\mathbf{n}), \qquad \text{subject to} \quad \mathbf{A}\,\mathbf{n} \;=\; \mathbf{b}, \quad n_i \ge 0, \tag{282}$$

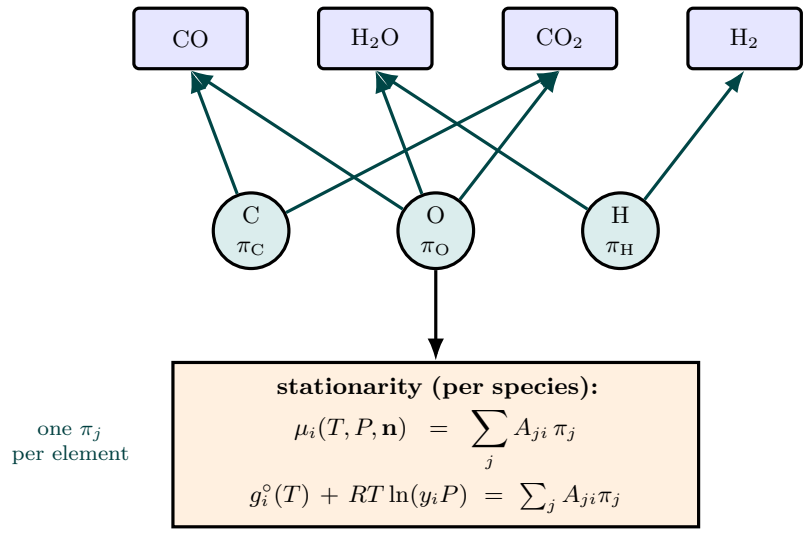

Figure 63: The Gibbs reactor never enumerates reactions. Instead it attaches one Lagrange multiplier π_j — an *element chemical potential* — to each conserved element (C, O, H …, teal circles), and the atom-balance constraints $\mathbf{A}\mathbf{n} = \mathbf{b}$ wire every species (blue) to the elements it contains. At the constrained minimum of G each species' chemical potential equals the sum of its element potentials, $\mu_i = \sum_j A_{ji}\pi_j$ (orange box). That is N stationarity equations plus M element balances — a small Newton system in $(\mathbf{n}, \boldsymbol{\pi})$ — replacing the combinatorial mess of "which reactions occur". The water-gas-shift species shown here resolve themselves automatically.

where $\mathbf{b} = \mathbf{A}\,\mathbf{n}^{\text{in}}$ records the total atoms of each element present in the feed (conserved across the reaction). Note that the feed itself trivially satisfies the constraint, so the optimisation lives in a non-empty affine set.

For an ideal-gas mixture at the reactor pressure P, the chemical potential is the one we derived in Chapter 7:

$$\mu_i(T,P,\mathbf{n}) \;=\; g_i^{\circ}(T) \;+\; RT\,\ln\!\big(y_i\,P\big), \qquad y_i = \frac{n_i}{N_{\text{tot}}}, \quad N_{\text{tot}} = \sum_i n_i, \tag{283}$$

with P taken in bar and the standard reference $P^{\circ} = 1$ bar suppressed. The standard-state $g_i^{\circ}(T)$ is the absolute ideal-gas Gibbs energy computed by the chain $h_f^{\circ} + s^{\circ} + \int c_p^{\text{ig}}$ of Chapter 6, available in `Component::g_pure_ig(T)`.

12.2 The Lagrangian

Introduce a multiplier λ_j for each of the M element-balance constraints:

$$L(\mathbf{n}, \boldsymbol{\lambda}) \;=\; G(\mathbf{n}) \;-\; \sum_{j=1}^{M} \lambda_j \left(\sum_{i=1}^{N} A_{ji}\, n_i - b_j \right). \tag{284}$$

Stationarity with respect to n_i gives, at an interior solution ($n_i > 0$ for every i),

$$\frac{\partial L}{\partial n_i} \;=\; \mu_i \;-\; \sum_{j=1}^{M} \lambda_j\, A_{ji} \;=\; 0 \qquad \forall i. \tag{285}$$

Substituting Eq. (283) and dividing through by RT:

$$\frac{g_i^\circ(T)}{RT} + \ln y_i + \ln P \;=\; \sum_{j=1}^{M} \pi_j\, A_{ji}, \qquad \pi_j \equiv \frac{\lambda_j}{RT}. \tag{286}$$

Solving for $n_i = N_{\mathrm{tot}} y_i$ gives the closed form the simulator uses to reconstruct mole numbers from the iteration variables:

$$\boxed{\; n_i \;=\; \exp\!\left(\ln N_{\mathrm{tot}} - \ln P - \frac{g_i^\circ(T)}{RT} + \sum_{j=1}^{M} \pi_j\, A_{ji} \right). \;} \tag{287}$$

> *Intuition.* Each element multiplier π_j has units of inverse RT and ought to be thought of as the "thermodynamic activity" of an atom of element j in the equilibrium mixture. Species i that contains more of an element with a high π_j is correspondingly more populated at equilibrium. At $T = 0$ K only the species with the lowest g_i° survive; at $T \to \infty$ the entropy term $\ln y_i$ wins and all species become equally populated.

12.3 The residual system

Define the iteration variables

$$\mathbf{x} \;=\; (\, \pi_1, \,\ldots,\, \pi_M,\; \ln N_{\mathrm{tot}} \,) \;\in\; \mathbb{R}^{M+1}. \tag{288}$$

There are M element-balance residuals and one mass-conservation residual:

$$f_j(\mathbf{x}) \;=\; \sum_{i=1}^{N} A_{ji}\, n_i(\mathbf{x}) \;-\; b_j, \qquad j = 1, \ldots, M, \tag{289}$$

$$f_{M+1}(\mathbf{x}) \;=\; \sum_{i=1}^{N} n_i(\mathbf{x}) \;-\; \exp(\ln N_{\mathrm{tot}}), \tag{290}$$

with $n_i(\mathbf{x})$ from Eq. (287). These $M+1$ non-linear equations in $M+1$ unknowns are solved by the `solver::newtonND` routine of Chapter 3 with a finite-difference Jacobian; one Gauss-eliminated linear system per iteration, backtracking line search.

12.4 The initial guess that makes this work

Equation (287) contains an exponential of a quantity that, at typical reactor temperatures, has g_i°/RT of order -100 for products of formation. A naive initial guess of $\pi_j = 0$ then asks the simulator to evaluate $\exp(+100) \approx 10^{43}$ on the very first residual call, drowning all gradient information and refusing to converge.

The simulator's trick is to seed $\boldsymbol{\pi}$ from a **least-squares fit** of the stationarity condition (285) to the *feed* composition. Treat the feed mole fractions y_i^{feed} (perturbed up from zero to a floor of 10^{-6} for inert species so the log stays finite) as if they were already the equilibrium answer, and solve the over-determined $N \times M$ linear system

$$\sum_{j=1}^{M} \pi_j\, A_{ji} \;=\; \frac{g_i^\circ(T)}{RT} \;+\; \ln y_i^{\mathrm{feed}} + \ln P \qquad i = 1, \ldots, N \tag{291}$$

in the least-squares sense by forming the $M \times M$ normal equations

$$(\mathbf{A}\,\mathbf{A}^{\top})\,\boldsymbol{\pi} \;=\; \mathbf{A}\,\mathbf{b}_{\mathrm{rhs}}, \tag{292}$$

and solving by `solver::gaussSolve`. After this seed the Newton iteration converges with visibly quadratic decay in five to ten steps.

12.5 Worked example: water-gas shift at 800 K

The water-gas shift reaction

$$\mathrm{CO} + \mathrm{H_2O} \;\rightleftharpoons\; \mathrm{CO_2} + \mathrm{H_2}$$

runs over a $\mathrm{Cu/ZnO/Al_2O_3}$ low-temperature catalyst and an $\mathrm{Fe_2O_3/Cr_2O_3}$ high-temperature catalyst; the equilibrium composition is the practical upper bound on hydrogen production. Take $T = 800$ K, $P = 1$ bar, equimolar $\mathrm{CO/H_2O}$ feed.

The atom matrix over elements (C, H, O) for species (CO, $\mathrm{H_2O}$, $\mathrm{CO_2}$, $\mathrm{H_2}$) is

$$\mathbf{A} \;=\; \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 2 \\ 1 & 1 & 2 & 0 \end{pmatrix},$$

so $M = 3$, $N = 4$. With $b_\mathrm{C} = b_\mathrm{H} = 2\,n_0$, $b_\mathrm{O} = 2\,n_0$ where $n_0 = 0.5\,F$ is the half-feed.

The Gibbs energies at 800 K from the data files are

species	$g^\circ(800\,\mathrm{K})$ [kJ/mol]	g°/RT
CO	-282.0	-42.4
$\mathrm{H_2O}$	-389.3	-58.5
$\mathrm{CO_2}$	-577.5	-86.8
$\mathrm{H_2}$	-117.8	-17.7

Running `./choupoSolve tutorials/gibbs01_water_gas_shift` gives, after six Newton steps with residual decay [$0.499 \to 0.235 \to 0.142 \to 0.015 \to 2.1{\times}10^{-4} \to 4.3{\times}10^{-8}$]:

species	y_i^feed	y_i^eq
CO	0.500	0.164
$\mathrm{H_2O}$	0.500	0.164
$\mathrm{CO_2}$	0.000	0.336
$\mathrm{H_2}$	0.000	0.336

CO conversion is 67.2%. The equilibrium constant

$$K_\mathrm{eq}(800\,\mathrm{K}) \;=\; \frac{y_{\mathrm{CO_2}}\,y_{\mathrm{H_2}}}{y_{\mathrm{CO}}\,y_{\mathrm{H_2O}}} \;=\; \frac{0.336 \times 0.336}{0.164 \times 0.164} \;\approx\; 4.19, \tag{293}$$

against the tabulated value 4.04 from Smith-Van Ness-Abbott Table 13.2 — agreement at the 3.7% level, which is the residual error in the Δh_f° and s° tabulations themselves.

> *Intuition.* The independent reaction count for a Gibbs problem is $R = N - M$ (species minus elements). Water-gas shift has $R = 4 - 3 = 1$: one independent reaction. Steam reforming of methane ($\mathrm{CH_4}$ + $\mathrm{H_2O}$, five species: $\mathrm{CH_4}$, $\mathrm{H_2O}$, CO, $\mathrm{CO_2}$, $\mathrm{H_2}$, three elements C, H, O) has $R = 5 - 3 = 2$ — two independent reactions, typically written as reforming itself and water-gas shift. Combustion of methane in air with possible incomplete burning ($\mathrm{CH_4}$, $\mathrm{O_2}$, CO, $\mathrm{CO_2}$, $\mathrm{H_2O}$, $\mathrm{N_2}$, six species, four elements C, H, O, N) has $R = 6 - 4 = 2$. The simulator does not care about R — it solves the $M + 1$ unknowns regardless of how many independent reactions that corresponds to.

12.6 When the ideal-gas Gibbs reactor stops being right

- **Kinetic limitation.** If the residence time is shorter than the slowest elementary step (e.g. NO formation in a furnace, ammonia synthesis below 700 K), the equilibrium composition is an upper bound but the actual exit composition is dictated by kinetics. Run a CSTR or PFR instead.
- **Non-ideal gas.** Above ~ 10 bar the ideal-gas $\varphi_i = 1$ approximation breaks down; the chemical potential picks up a fugacity-coefficient term that the Gibbs reactor ignores. Post-v1 cubic EOS will fix this.
- **Condensation.** Single-phase Gibbs is correct only if all species stay in the same phase at the reactor outlet. For reformers running at temperatures where water condenses on cooling, a multi-phase Gibbs (with liquid Gibbs energy + vapour-liquid equilibrium) would be needed; does not do this.
- **Element rank deficiency.** If $\mathbf{A}$ does not have full row rank (e.g. a redundant element constraint), the normal-equation matrix in the initial-guess step becomes singular. The implementation falls back to $\pi = 0$ in that case and hopes the iteration recovers — in practice the rank deficiency means the user has set the problem up wrong.

> **Runnable case.** `runCase tutorials/gibbs01_water_gas_shift` executes the worked example above. Try editing `system/flowsheetDict` to change the reactor T to 1000 K and watch K_eq drop (the WGS is mildly

exothermic, so the forward reaction is favoured at lower temperature). At 1000 K the simulator returns $K_{\mathrm{eq}} \approx 1.4$, against the literature 1.40 — the same accuracy as the 800 K case.

12.7 Equilibrium maps and the temperature approach

A single Gibbs solve answers "what is the equilibrium composition at *this* (T, P)?". Sweep the solve over a $T \times P$ grid and you get an *equilibrium map* — iso-lines of a chosen quantity (a product mole fraction, an element yield) over the operating plane. This is the picture that explains, at a glance, why an industrial reactor fixes its temperature and pressure in a narrow window. Take ammonia synthesis, $N_2 + 3H_2 \rightleftharpoons 2NH_3$:

- the reaction is exothermic, so by Le Chatelier the equilibrium NH_3 fraction *rises as T falls* — the contours climb toward the cold side;
- it consumes moles ($\Delta n = -2$), so the fraction *rises with pressure* — the contours climb with P;
- yet the plant runs *hot* (400–500 °C) because the iron catalyst is inert below ∼400 °C: without a rate there is no approach to that beautiful cold equilibrium at all.

The map makes this tension a single image: the equilibrium optimum sits in one corner, the industrial window (a labelled box) sits where the catalyst can work, and the arrow between them *is* the compromise every ammonia plant lives with. Choupo draws the map with the `gibbsMap` property operation (the Explorer's "Equilibrium map" view is the same engine, interactive); unconverged grid cells are marked, never interpolated over.

The temperature approach to equilibrium. A real reactor does not reach equilibrium at its outlet temperature; a rate-limited, catalyst-limited bed falls short. The classical engineering device for this — the one a commercial equilibrium reactor exposes without explaining — is the *temperature approach* ΔT: evaluate the *reaction* equilibrium at $T + \Delta T$ while the physical state (enthalpy, any condensation, the energy balance) stays at the real T. For an exothermic reaction the equilibrium conversion falls with temperature, so the conversion a real bed reaches equals the equilibrium conversion of a *higher* temperature: ΔT is that distance, in kelvin, and industry quotes it routinely (ammonia beds ∼10–20 °C, methanol ∼10–30, shift converters ∼5–25).

Choupo implements it exactly (Song's convention, ratified by the design forum, `docs/design/gibbs-map-forum-2026-07-02.md`): the outlet is a legitimate *non-equilibrium* state — composition $n(T + \Delta T)$, physical state (T, P) — so its enthalpy is $H = \sum_i n_i(T + \Delta T)\, h_i(T, P)$, evaluated at the physical T, and its affinity at T is nonzero (that nonzero affinity *is* the distance from equilibrium, not a bug). Two honesty rules the forum demanded:

Intuition. **ΔT is calibrated, never predicted.** It lumps kinetics, catalyst activity and transport into one empirical number you fit against a plant measurement — *all models are wrong, some are useful.* A single global ΔT is meaningful for one dominant reaction (ammonia, methanol); applied to a multi-reaction pool it shifts reactions in physically inconsistent directions (in a reformer, methane reforming and the water-gas shift have *opposite* thermicity and different real approaches), so Choupo announces the global-only limitation aloud. And at high pressure an ideal-gas Gibbs kernel will let ΔT silently absorb the missing fugacity corrections — announced too, so a student never calibrates non-ideality and calls it kinetics. The value is echoed at every point of consumption — the streams table, the KPI epilogue, a column in the map CSV, the anchor-KPI names — because a ΔT that survives unnoticed in a copied case file is the classic silent-fudge accident.

13 Distillation column, Wang-Henke bubble-point

What you'll learn. The simulator's column model: stage-by-stage equilibrium, CMO (constant molar overflow), bubble-point inner loop, K-value outer loop. Robust on ideal systems; struggles near azeotropes.

13.1 Stage model

For a column with N_s equilibrium stages numbered 1 (condenser) to

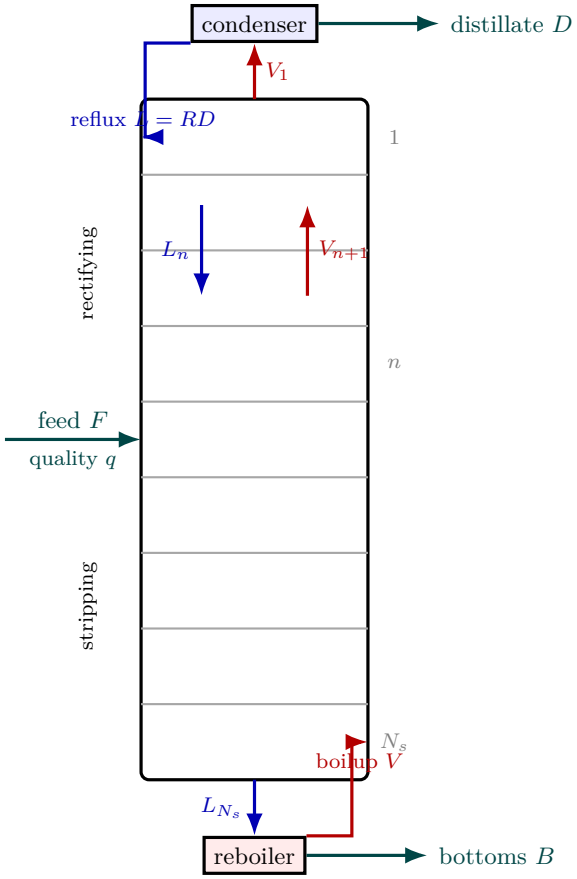

Figure 64: Anatomy of the distillation column the stage model describes. Stages are numbered 1 (condenser) at the top to N_s (reboiler) at the bottom. Liquid (blue) cascades *down* stage to stage, vapour (red) rises *up*; on every tray the leaving streams are in equilibrium, $y_{n,i} = K_{n,i}x_{n,i}$. The condenser returns reflux $L = RD$ and draws distillate D; the reboiler returns boilup and draws bottoms B. The feed enters at its own stage and divides the column into a *rectifying* section above and a *stripping* section below — the feed quality q sets how the internal flows step across it. *Constant molar overflow* is the simplifying assumption that the internal L_n, V_n are constant *within* each section ($L = RD$, $V = (R{+}1)D$ in the rectifier), replacing a per-stage energy balance — exact for equal molar latent heats, forgiving otherwise.

N_s (reboiler), each interior stage n satisfies, at steady state:

- Component balance: $L_{n-1}\,x_{n-1,i} + V_{n+1}\,y_{n+1,i} = L_n\,x_{n,i} + V_n\,y_{n,i}$.
- Equilibrium: $y_{n,i} = K_{n,i}(T_n, P_n, \mathbf{x}_n)\,x_{n,i}$.
- Stoichiometry: $\sum_i x_{n,i} = 1$, $\sum_i y_{n,i} = 1$.
- Energy balance (here simplified: CMO).

13.2 Constant molar overflow

CMO replaces the per-stage energy balance with the assumption that liquid and vapour internal flows are constant within each section:

$$L_n = L = R\,D, \quad V_n = V = (R+1)\,D \quad \text{(rectifying section)}, \tag{294}$$

where R is the reflux ratio and D the distillate rate. For the stripping section, $L' = L + qF$, $V' = V - (1-q)F$ with q the feed quality. This is exact only for ideal-enthalpy mixtures, but remarkably forgiving even when it is wrong.

13.3 Wang-Henke bubble-point method

The algorithm (Wang & Henke 1966, [55]) breaks the coupled system into two nested loops:

1. Guess a temperature profile T_n, $n = 1, \dots, N_s$.
2. For each stage, given T_n and the current $\mathbf{x}_n$, compute $K_{n,i}$ and a new $\mathbf{x}_n$ from the tri-diagonal component-balance system.
3. For each stage, update T_n by solving the bubble-point equation $\sum_i K_{n,i}(T_n) x_{n,i} = 1$ (Newton-1D in T_n).
4. Iterate steps 2–3 until $\|T^{(k+1)} - T^{(k)}\|_\infty < \varepsilon$.

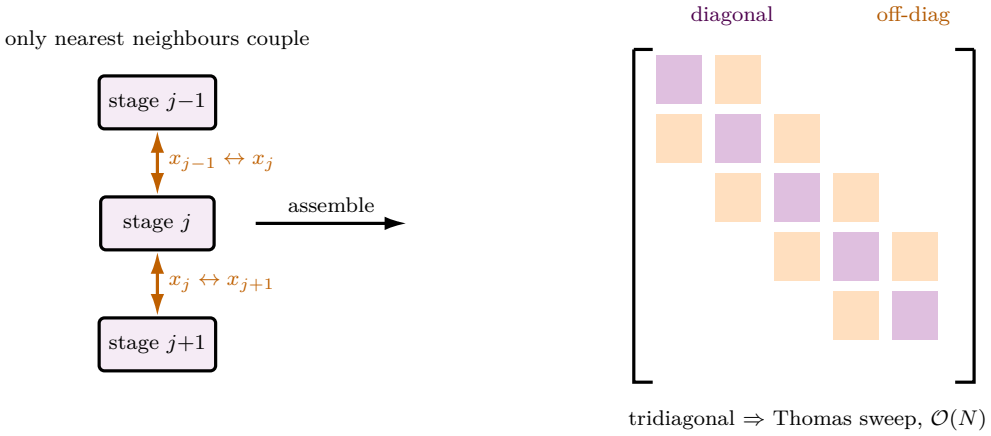

Figure 65: Why a column's component balance is *tridiagonal*. Around stage j the steady balance ties only three liquid compositions — x_{j-1} descending in, x_j leaving, x_{j+1} rising in (one species shown). Because each stage couples to *just its two neighbours*, stacking the N balances gives a matrix with non-zeros only on the main diagonal and the two adjacent ones: a banded, tridiagonal system, one per component. Such a system is solved exactly by the Thomas algorithm in a single forward-then-backward sweep at $\mathcal{O}(N)$ cost — no general matrix inversion — which is what Wang–Henke's inner step and the absorber cascade both exploit. The simultaneous MESH Newton keeps the same band structure in its Jacobian, which is why even the rigorous column stays cheap per iteration.

The simulator's implementation is in `src/unitOperations/distillation/DistillationColumn.cpp`.

13.4 When this method struggles

- **Azeotropes.** If $K_i \to 1$ for the key components, the bubble-T solve becomes ill-conditioned and the outer loop can pass through the azeotrope into a non-physical region. The ethanol/water case at $x_1 \approx 0.94$ is the canonical trouble spot.
- **High reflux.** The CMO assumption is exact only if molar latent heats are equal; at $R \gg 1$ small enthalpy differences accumulate, and stage temperatures drift.

The remedy for both is Naphtali-Sandholm equation-oriented Newton [56] — on the roadmap.

> **Runnable case.** `tutorials/column01_benzene_toluene` — the ideal case, converges in 30–50 outer iterations.
> `tutorials/sensitivity01_column_reflux` — the same column swept over $R \in [1.5, 5]$ by the outer driver.

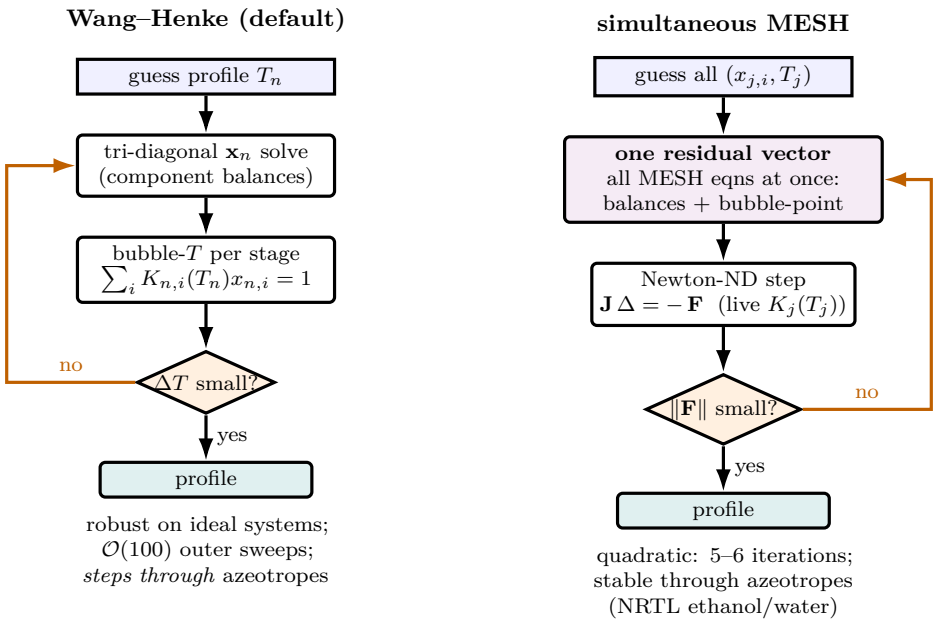

Figure 66: Two ways to solve the same column. *Left* — **Wang–Henke** (the default) decouples the problem into nested loops: with a guessed temperature profile it solves a tri-diagonal component-balance system for the stage compositions, then updates each stage temperature from its own bubble-point equation, and sweeps until the profile stops moving. Simple and robust on ideal mixtures, but slow (∼100 sweeps) and prone to stepping *through* an azeotrope into a non-physical region. *Right* — `method simultaneous` (the rigorous MESH Newton) packs *every* stage equation into one residual vector and drives the whole thing to zero with the project's one n-D Newton (live $K_j(T_j)$). It converges quadratically in 5–6 iterations and stays physical on azeotropic systems — refusing to invent a "past the azeotrope" answer that does not exist. Same column, same thermodynamics; only the *shape* of the iteration differs.

14 Gas absorption and stripping: the coupled stage cascade

What you'll learn. An absorber washes a solute out of a gas with a liquid solvent; a stripper does the reverse, blowing a solute out of a liquid with a gas. Both are counter-current multistage cascades, and both are governed by one dimensionless group — the absorption factor $A = L/(KV)$. This chapter sets up the per-stage balance as a tridiagonal system (solved by the Thomas algorithm, not the closed-form Kremser), then adds the *energy* balance that the textbook Kremser ignores — and which turns out to dominate: the heat of absorption warms the column, raises K, and self-limits the very absorption that releases it.

14.1 The counter-current cascade and the absorption factor

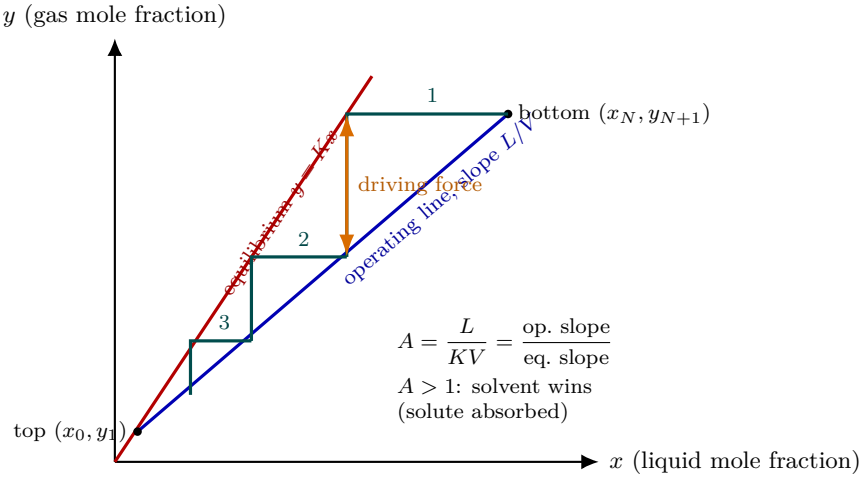

Figure 67: The McCabe–Thiele staircase for a counter-current absorber. The *equilibrium line* $y = Kx$ (red) says what gas and liquid *leaving a stage* must satisfy; the *operating line* (blue), slope L/V, is the mass balance tying the streams *passing between* stages. For an absorber the operating line sits *above* the equilibrium line, and the vertical gap between them is the driving force that pulls solute from gas into liquid. Each theoretical stage is one step of the staircase — horizontal to the operating line (a passing mass balance), vertical to the equilibrium line (a stage equilibrium) — so the number of steps from bottom to top is the stage count. The single group that governs it all is the absorption factor $A = L/(KV)$, exactly the ratio of the two slopes: $A > 1$ widens the staircase and the solvent wins; $A < 1$ pinches it shut and the solute escapes. Choupo solves the same balance as a tridiagonal (Thomas), then *couples in the self-heating energy balance* the dry staircase ignores.

Number the equilibrium stages 1 (top) to N (bottom). Lean solvent (L) enters the top, rich gas (V) enters the bottom; they pass counter-currently. On each stage the leaving streams are in equilibrium, $y_{i,j} = K_i(T_j)\, x_{i,j}$. A component balance around stage j (constant molar L and V) relates three neighbouring liquid compositions — x_{j-1}, x_j, x_{j+1} — so the whole column is a *tridiagonal* system in $\mathbf{x}_i$, one per component. The controlling group is the *absorption factor*

$$A_i \;=\; \frac{L}{K_i V}, \tag{295}$$

the ratio of the solvent's capacity to carry the solute (L) to the gas's tendency to keep it ($K_i V$). $A_i > 1$ means the solvent wins and the solute is absorbed; $A_i < 1$ means it escapes. The classic Kremser equation writes the recovery in closed form as a function of A_i and N; Choupo instead solves the tridiagonal directly (the Thomas algorithm), which costs nothing extra and generalises cleanly to the non-isothermal case below.

14.2 Why the energy balance matters

Dissolving a gas releases its heat of absorption (the negative of the

heat of solution — here recovered from the Henry's-law van't Hoff slope, Chapter 3's sibling). Choupo solves a *second* tridiagonal, in the stage temperatures T_j, with that heat as a source, and iterates the two balances to convergence (falling back to isothermal cleanly when no heat data exist). The coupling is a vicious circle:

$$\text{absorb} \;\Rightarrow\; \text{heat released} \;\Rightarrow\; T\!\uparrow \Rightarrow\; K(T)\!\uparrow \Rightarrow\; A = \tfrac{L}{KV}\!\downarrow \Rightarrow\; \text{less absorption.} \tag{296}$$

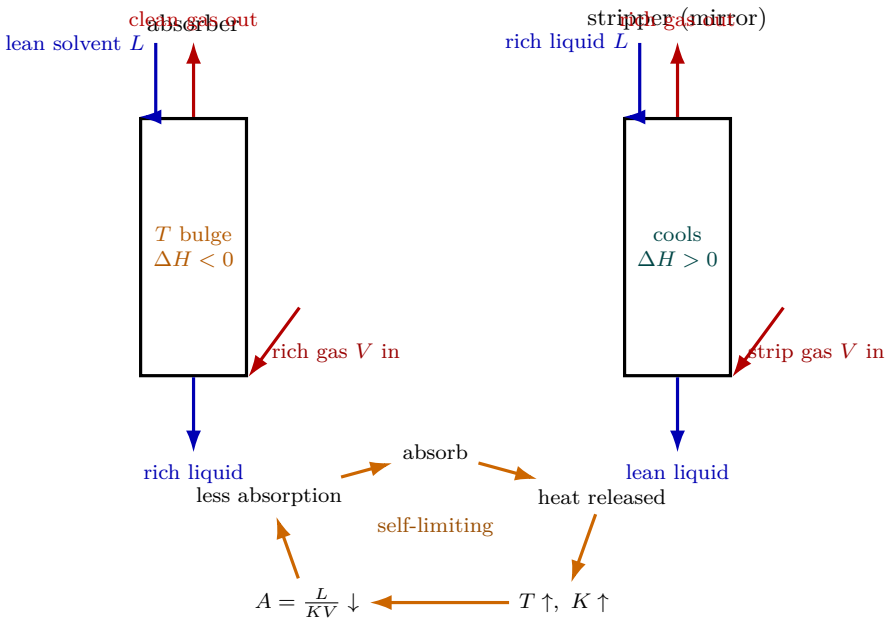

Figure 68: The absorber and its mirror, the stripper, plus the feedback loop that makes the energy balance non-optional. In the absorber lean solvent enters the top, rich gas the bottom; the dissolving gas releases its heat of absorption ($\Delta H < 0$), so the liquid *warms* — a "temperature bulge". The stripper is the same cascade inverted (rich liquid down, strip gas up) and its desorption is *endothermic*, so it *cools*. The loop at the bottom is why the isothermal Kremser is dangerously optimistic: absorbing releases heat, which raises T, which raises K, which *lowers* the absorption factor $A = L/(KV)$, which suppresses the very absorption that released the heat. The column throttles itself exactly where it works hardest — the NH_3 scrubber heats ∼17 K and recovers 67.5%, not the ∼97% an isothermal balance would promise.

> *Intuition.* The isothermal Kremser is dangerously optimistic for a soluble, exothermic gas. The column heats itself up exactly where it is working hardest (a "temperature bulge"), and the warmer it gets the less it can hold — a built-in throttle. This is why real ammonia and amine absorbers are cooled, and why an energy balance is not optional for them. The same machinery, run in reverse, gives the stripper its *endothermic* cooling.

Worked example 14.1. Ammonia scrubbing, `absorber01_NH3_water`Six stages, NH_3 from air into water. The exotherm ($\Delta H \approx -34$ kJ/mol) heats the liquid by ∼17 K (top 309, peak ∼318, bottom 315 K), pushing K up and A down, so the recovery is **67.5 %** — far below the ∼97 % an isothermal Kremser would promise. The exotherm self-limits the absorption; the energy balance is the whole story.

14.3 The stripper: the same cascade, mirrored

A stripper (`stripper01_NH3_water`) is the absorber inverted: liquid feed enters the top, stripping gas the bottom, and the rich stream is now the *gas*. The tridiagonal is identical; only the reported quantities differ. Its controlling group is the *stripping factor*

$$S_i = \frac{K_i V}{L} = \frac{1}{A_i}, \tag{297}$$

and its energy balance is the absorber's with the sign flipped: desorption is endothermic, so the liquid *cools* as it is stripped (the NH_3 case drops ∼16.5 K, the mirror of the absorber's +17 K). $S_i > 1$ strips, $S_i < 1$ retains.

> **Runnable case.** `tutorials/steady/absorption/absorber01_NH3_water` — 6-stage NH_3 scrubber, the exothermic temperature bulge.
> `tutorials/steady/absorption/stripper01_NH3_water` — the mirror, NH_3 stripped from water by air, endothermic cooling.

15 Crystallisation: the population balance and the method of moments

What you'll learn. Every unit operation so far balances how much of each species is present. A crystalliser must also track *how big the crystals are* — the particle-size distribution (PSD) is the product specification, not a by-product. That demands a balance over a new, *internal* coordinate, crystal size L: the *population balance*. This chapter builds it from the supersaturation driving force, solves the steady continuous (MSMPR) case in closed form $n(L) = n^0 e^{-L/G\tau}$, and shows how the *method of moments* collapses that distribution to a handful of ordinary numbers — mean size, coefficient of variation, magma density — with no size grid at all. It is the first unit in Choupo whose state lives in a coordinate other than composition.

15.1 The driving force: solubility and supersaturation

A solute crystallises only when its dissolved concentration c exceeds the saturation value c_{sat} — read either from a measured solubility curve on a molecular component (`solubility { coefficients (...) }`, a polynomial in T) or, for a dissociating salt, from the ion-activity product derived in §15.2. The ratio

$$S \equiv \frac{c}{c_{\text{sat}}(T)} \tag{298}$$

is the *supersaturation*: $S > 1$ drives both the birth of new crystals (nucleation) and the growth of existing ones. It is the crystalliser's analogue of the distillation column's relative volatility — the single number the whole unit turns on.

The simplest model (`model equilibrium`, the cooling crystalliser) assumes the kinetics are *infinitely fast*: the mother liquor leaves exactly saturated, $S = 1$, and the yield is pure thermodynamics,

$$\text{crystals} \;=\; \text{solute}_{\text{in}} \;-\; c_{\text{sat}}(T_{\text{op}}) \cdot \text{solvent}. \tag{299}$$

This answers "how much" but says nothing about "how big". For that we need the kinetics, and the kinetics need the population balance.

15.2 Where the saturation comes from for a salt: the solubility product

Equation (299) — and every model below — needs c_{sat}. For a molecular solute (sucrose, a pharmaceutical) it is read directly from a measured `solubility { coefficients (...) }` curve, a polynomial $c_{\text{sat}}(T)$. For a dissociating *salt* there is no such single curve, and — this is the crux — the electrolyte activity model of §3 *cannot supply one.*

An activity model is blind to the solid. The eNRTL or Pitzer model computes the mean ionic activity coefficient $\gamma_\pm(m, T)$ — how non-ideal the *liquid* solution of ions is. It knows nothing about the crystal: the lattice, the chemical potential of the solid. But solubility is a *solid–liquid* equilibrium,

$$\mathrm{NaCl(s)} \rightleftharpoons \mathrm{Na^+} + \mathrm{Cl^-}, \qquad \mu_{\mathrm{NaCl(s)}} = \mu_{\mathrm{Na^+}} + \mu_{\mathrm{Cl^-}},$$

and the position of that equilibrium — the *solubility product*

$$K_{sp} \;=\; a_{\mathrm{Na^+}}\, a_{\mathrm{Cl^-}} \;=\; \big(\gamma_\pm\, m_{\text{sat}}\big)^{\nu} \;=\; \exp\!\Big(-\frac{\Delta G^\circ_{\text{dissol}}}{R_{\text{g}} T}\Big) \tag{300}$$

($\nu = \nu_+ + \nu_-$, $= 2$ for NaCl) depends on $\Delta G^\circ_{\text{dissol}}$, the free energy of the hydrated ions *relative to the crystal.* That difference is a property of the *solid*, to which a liquid-phase γ-model has no access.

Intuition. The salt's solubility is to the crystalliser what a volatile's vapour pressure is to the flash: the pure-phase anchor the activity model does *not* provide. NRTL gives you γ for the bubble point but you still supply Antoine's P^{sat}; the eNRTL gives you $\gamma_\pm$ for the saturation but you still supply one K_{sp} (equivalently, one measured solubility). An activity model predicts *non-ideality*, never the reference state of a phase it cannot see.

One anchor, a whole surface. Choupo takes the practical route: anchor K_{sp} to a single *measured* solubility m°_{sat} at 25°C (`solubility 6.144;` in the salt's `electrolyte{}` block), so that, with $\gamma_\pm$ from the model,

$$K_{sp} \;=\; \big(\gamma_\pm(m^\circ_{\text{sat}})\, m^\circ_{\text{sat}}\big)^{\nu}. \tag{301}$$

This looks circular — a solubility in to get a solubility out. The payoff is that the *same* K_{sp} now fixes saturation at *every other* condition: solving

$$\big(\gamma_\pm(m, T, \mathbf{x}_{\text{solv}})\, m\big)^{\nu} \;=\; K_{sp} \tag{302}$$

for m returns m_{sat} as a function of temperature *and solvent composition.* The model turns one number into the full solubility surface — whereas the molecular route would need a freshly measured curve for every solvent.

Why store the solubility and not K_{sp} itself. It is tempting to tabulate the "fundamental" constant K_{sp} directly. Resist it: the K_{sp} back-computed from a solubility is *model-contaminated*. The *same* measured $m^{\circ}_{\mathrm{sat}} = 6.144$ gives $K_{sp} = 34.8$ through the eNRTL ($\gamma_{\pm} = 0.96$) but 38.2 through Pitzer ($\gamma_{\pm} = 1.01$) — a 10% split for one salt, because the two models disagree about non-ideality at 6 molal. A solubility product and its activity-coefficient model are an *inseparable pair*: serious brine parameterisations (Harvie–Møller–Weare; the $\log K$ values in PHREEQC) regress the mineral K_{sp} *jointly* with the ion-interaction terms on the same data, so the K_{sp} is never portable to another convention. The *solubility* is the model-independent observable — you read it off a balance — so it is what the curated `.dat` stores; the engine derives the model-consistent K_{sp} at load time and reports it, so a student who swaps Pitzer↔ eNRTL *sees* K_{sp} shift while the solubility holds. (The standard $\Delta G^{\circ}_{\mathrm{dissol}}$ of (300) is the one genuinely model-free anchor, but routing it through an imperfect $\gamma_{\pm}$ would inject the model's $\sim$2% error into the predicted solubility at the very reference point. We keep the observable exact there and make the model earn its keep on the *extrapolation* away from 25°C and pure water.)

Temperature: van't Hoff on K_{sp}. Two distinct things move m_{sat} with T: the activity coefficient $\gamma_{\pm}(T)$ (the liquid; §3) and the solubility product $K_{sp}(T)$ itself (the solid–liquid equilibrium constant). The latter obeys van't Hoff,

$$\frac{d\ln K_{sp}}{dT} = \frac{\Delta H_{\mathrm{sol}}}{R_{\mathrm{g}}T^2} \implies K_{sp}(T) = K_{sp}(T_0)\,\exp\!\Big[-\frac{\Delta H_{\mathrm{sol}}}{R_{\mathrm{g}}}\Big(\tfrac{1}{T}-\tfrac{1}{T_0}\Big)\Big], \tag{303}$$

with ΔH_{sol} the enthalpy of solution (`dissolutionEnthalpy`; optional — omit it and K_{sp} is held flat in T). For NaCl $\Delta H_{\mathrm{sol}} \approx +3.9$ kJ/mol is small, which is precisely why NaCl's solubility is famously almost flat with temperature — and why NaCl is purified by evaporation or by antisolvent, never by cooling.

15.3 The two-layer $K_{sp}(T)$: which heat of solution drives van't Hoff?

Equation (303) hides a subtlety that the caustic-soda case forces into the open. The constant `dissolutionEnthalpy` is the *standard* (infinite-dilution) enthalpy of solution $\Delta H^{\infty}_{\mathrm{sol}}$ — the heat absorbed when one mole of the crystal dissolves into an *infinite* amount of water. But van't Hoff on a *saturated* solution is not about diluting to infinity; it is about the equilibrium between the crystal and a liquid that is already at m_{sat}. The thermodynamically correct driver of $d\ln K_{sp}/dT$ at saturation is the *differential* (partial-molar) heat of solution *at that molality*,

$$\Delta H_{\mathrm{diff}}(m_{\mathrm{sat}}) \;=\; \Delta H^{\infty}_{\mathrm{sol}} \;+\; \overline{L}_2(m_{\mathrm{sat}}), \qquad \overline{L}_2 \;=\; L_{\phi} + m\,\frac{\partial L_{\phi}}{\partial m}, \tag{304}$$

where $L_{\phi}(m,T)$ is the salt's *relative apparent molar enthalpy* (the heat-of-dilution curve) and $\overline{L}_2$ its partial-molar sibling — the heat associated with adding one more mole of salt to a solution *already at* m, rather than to pure water. At infinite dilution $L_{\phi} \to 0$ and (304) collapses back to $\Delta H^{\infty}_{\mathrm{sol}}$; at a real saturation $\overline{L}_2$ can be large and of *either* sign.

> *Intuition.* Why this matters so much for NaOH. The infinite-dilution figure is $\Delta H^{\infty}_{\mathrm{sol}} \approx -44.5$ kJ/mol — strongly *exothermic*, which through (303) alone would predict solubility *falling* with temperature. Caustic soda does the opposite: its solubility *rises* steeply with T (you concentrate it by evaporation, and it crash-crystallises on *cooling*). The resolution is $\overline{L}_2(m_{\mathrm{sat}})$: at $\sim$29 molal the partial-molar relative enthalpy is large and *positive*, enough to overturn the sign, so $\Delta H_{\mathrm{diff}} > 0$ and the solubility climbs. Use the wrong (infinite-dilution) enthalpy and the model gets the *direction* of the temperature effect backwards.

Where L_{ϕ} comes from: $-RT^2\,\partial g^{ex}/\partial T$. The heat-of-dilution curve is not a fourth dataset — it is the *temperature derivative of the very activity surface* already in hand. By the Gibbs–Helmholtz identity, writing the dimensionless excess Gibbs energy per mole of salt as $g^{ex} = \nu\,[(1-\phi) + \ln\gamma_{\pm}]$ (§3),

$$L_{\phi}(m,T) \;=\; -\,R_{\mathrm{g}}\,T^2\,\frac{\partial g^{ex}}{\partial T}, \tag{305}$$

which Choupo evaluates by a central finite difference ($\Delta T = 0.05$ K) *on the same Pitzer kernel* that returns ϕ and $\gamma_{\pm}$ — one implementation, used identically by the props bench, the activity adapter and the curation fit (`PitzerSingleSalt::Lphi`). The partial-molar $\overline{L}_2$ of (304) is then a second central difference of L_{ϕ} in m (`partialMolarRelativeEnthalpy`). Glass-box and sign-safe: no hand-coded enthalpy expansion can drift out of step with the activity coefficient it must be consistent with.

The Silvester–Pitzer T-slots. Equation (305) needs the *temperature slope* of the virial parameters, not just their 25°C values. The single-salt kernel carries three optional derivatives applied *linearly* (Silvester & Pitzer 1977 [48]),

$$\beta^{(0)}(T) = \beta^{(0)} + \beta^{(0)}_T\,(T-T_0), \quad \beta^{(1)}(T) = \beta^{(1)} + \beta^{(1)}_T\,(T-T_0), \quad C^{\phi}(T) = C^{\phi} + C^{\phi}_T\,(T-T_0), \tag{306}$$

with $T_0 = 298.15$ K, so the 25°C $\gamma_{\pm}/\phi$ anchor is *never moved* (every slope vanishes at T_0) and the slots carry *only* the heat-of-dilution information. The dominant long-range slope is already in $A_{\phi}(T)$ (the Born–Bjerrum factor, §3); the three slots add the short-range correction. With all slots zero, L_{ϕ} is still computed — but from $A_{\phi}(T)$ alone, the *calorimetrically uncalibrated* guess.

The `calorimetricFit` gate. Here is the discipline that keeps the model honest. A $\beta(T)$ slope of zero gives a non-zero L_ϕ that is *Gibbs–Helmholtz-consistent but calorimetrically wrong* — consistency is not correctness. So L_ϕ is admitted into the energy balance (and into ΔH_{diff} above) *only* when the salt's pair row carries `calorimetricFit true;`: the flag is set *by* the curation tool `bin/curate/fit_lphi_pitzer.py`, which regresses the three slots of (306) by exact linear least squares (they enter L_ϕ linearly) against the measured relative apparent molar enthalpy $\Phi_L(m, 25°\text{C})$ of Parker's NSRDS–NBS 2 compilation [49], and sets the flag only if the fit passes the gate (relative AAD $< 5\%$ over $m = 0.1$–6 mol/kg, skipping the $|\Phi_L| < 50$ cal/mol band where the curve crosses zero twice and a ratio is noise). The row also records `lphiValidityMax`, the upper molality of the fit window; a saturation *beyond* it (NaOH's ∼29 molal is far past a 6-molal fit) still gets the sign-correct $\overline{L}_2$, but the run *announces* the extrapolation in the advisory log rather than pretending to accuracy it does not have. Without the flag, $K_{sp}(T)$ reverts byte-for-byte to the infinite-dilution form (303) — every legacy golden master is untouched. The eNRTL package deliberately does *not* inherit the flag (§4): its $\tau(T) \propto 1/T$ is the uncalibrated form, and the T-slots were regressed for the Pitzer surface, not the eNRTL one.

> **Common pitfall.** The gate is per-*kernel* and per-*salt*, not global. Turning on `calorimetricFit` for a salt whose pair row has zero T-slots does *not* make the enthalpy correct — it makes the uncalibrated $A_\phi(T)$-only L_ϕ believed. The flag is an output of the fitting tool's gate, never a knob the case author flips by hand to "enable enthalpy". If `fit_lphi_pitzer.py` fails its 5% gate, the flag stays `false` and the salt keeps the infinite-dilution van't Hoff form: an honest reduced model, not a silent wrong one.

Power and limit. Power: a *single* measured solubility plus a *single* calorimetric refit turn one number into the full $m_{\text{sat}}(T, \mathbf{x}_{\text{solv}})$ surface *with the correct temperature direction* — and the same L_ϕ surface feeds the crystalliser's heat of crystallisation $\Delta H_{\text{cryst}}(m_{\text{sat}}) = -\Delta H_{\text{diff}}(m_{\text{sat}})$ and any evaporator's heat of concentration, all from one Gibbs–Helmholtz-consistent kernel. Limit: the T-slots are *linear* (Silvester–Pitzer's first correction), fitted to 25°C Φ_L data on the 0–6 molal window; a saturation far outside it is honestly flagged, and the heat-*capacity* of solution ($\partial^2 g^{ex}/\partial T^2$, the J_ϕ channel) is a later slice. The model gets the *sign and order of magnitude* of the T-dependence right where it matters — it is not a substitute for a full $\beta(T)$ surface regressed over a wide temperature range.

The antisolvent: drowning-out. The most striking use of (302) is *antisolvent* crystallisation. Adding ethanol to a brine lowers the mixture dielectric constant ε (from ∼78 to ∼24); the ions, less screened, attract more strongly, so $\gamma_\pm$ *rises* steeply and (302) forces m_{sat} *down*. The mixed-solvent eNRTL captures this through two terms: the Debye–Hückel coefficient $A \propto (\varepsilon T)^{-3/2}$, which grows as ε falls, and a *Born* term for the energy of moving an ion from water into the poorer solvent. No cooling and no boil-off: the supersaturation $S = c/c_{\text{sat}}$ is created purely by displacing the saturation. In Choupo this is the difference between $m_{\text{sat}} = 6.14$ mol/kg in water and 0.88 mol/kg at 40 % ethanol — the *same* K_{sp}, a different $\gamma_\pm$.

With c_{sat} in hand — by either route — the supersaturation $S = c/c_{\text{sat}}$ is defined and everything below proceeds identically. The kinetics see only S; the crystalliser does not care whether it came from a sugar solubility polynomial or from an eNRTL ion-activity product.

15.4 The population balance: a balance over size

Let $n(L)\,dL$ be the number of crystals per unit suspension volume with size in $[L, L + dL]$; $n(L)$ is the *population density* [$\#\,\text{m}^{-4}$]. Crystals enter and leave a size interval not only by flowing in and out of the vessel but by *growing* across its boundaries, at the linear growth rate $G = dL/dt$. Writing the balance on $[L, L + dL]$ for a well-mixed vessel of volume V with volumetric throughput Q gives the *population balance equation* (Randolph & Larson [31]):

$$\frac{\partial n}{\partial t} + \frac{\partial (G\,n)}{\partial L} = \underbrace{B^0\,\delta(L)}_{\text{nucleation}} + \frac{Q_{\text{in}} n_{\text{in}} - Q\,n}{V}. \tag{307}$$

The $\partial(Gn)/\partial L$ term is the novelty: a *convection in size space*, crystals marching up the L-axis as they grow. Nucleation appears as a source of zero-size particles at $L = 0$ (a boundary condition $n(0) = n^0 = B^0/G$, the nuclei density), where B^0 is the nucleation rate [$\#\,\text{m}^{-3}\text{s}^{-1}$].

> *Intuition.* A composition balance is bookkeeping over a *finite* list of species. A population balance is bookkeeping over a *continuum* of sizes — the same conservation logic, but the "species" is now "a crystal of size L", and growth lets one continuously turn into the next. This is why crystallisation (and aerosols, polymers, bubbles, cells) needs its own balance: the property that defines the product varies continuously and the process moves material along that continuum.

15.5 The MSMPR steady state: the famous straight line

The reference idealisation is the *mixed-suspension, mixed-product-removal* (MSMPR) crystalliser: perfectly mixed, steady, clear feed ($n_{\text{in}} = 0$), product withdrawn at the suspension composition ($Q_{\text{in}} = Q$), and *size-independent*

growth (McCabe's ΔL law, $G \neq f(L)$) with no breakage or agglomeration. With $\tau = V/Q$ the residence time, (307) collapses to

$$G\,\frac{dn}{dL} + \frac{n}{\tau} = 0 \qquad \Longrightarrow \qquad \boxed{n(L) = n^0\, e^{-L/(G\tau)}.} \tag{308}$$

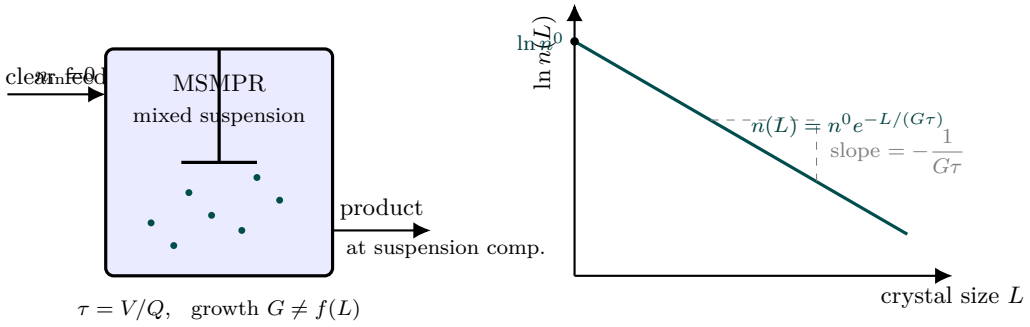

Figure 69: The MSMPR crystalliser and its signature diagnostic. Left: a perfectly mixed, steady vessel fed clear liquor ($n_{\text{in}} = 0$) and withdrawn at the suspension composition, with size-independent growth ($G \neq f(L)$). The population balance then collapses to a single exponential $n(L) = n^0 e^{-L/(G\tau)}$. Right: plot $\ln n$ against crystal size L and it is a *straight line* — the single most recognisable plot in crystallisation. The **intercept** hands you $\ln n^0 = \ln(B^0/G)$ (the nucleation-to-growth ratio) and the **slope** hands you $-1/(G\tau)$. Measure a product PSD, plot it this way, and the two kinetic numbers fall straight out.

Plotting $\ln n$ against L gives a *straight line* of slope $-1/(G\tau)$ and intercept $\ln n^0$ — the single most recognisable diagnostic in crystallisation practice. Measure a product PSD, plot it this way, and the slope hands you $G\tau$ while the intercept hands you the nucleation-to-growth ratio $n^0 = B^0/G$.

15.6 The method of moments

We rarely need the whole curve; we need numbers — a mean size, a spread, a mass. Define the moments

$$\mu_j \;=\; \int_0^{\infty} L^j\, n(L)\, dL. \tag{309}$$

For the MSMPR exponential (308) they evaluate in closed form (Hulburt & Katz [32]):

$$\mu_j \;=\; n^0\,(G\tau)^{j+1}\, j!. \tag{310}$$

Crucially, when G is size-independent the moment equations *close*: each μ_j depends only on lower moments and the kinetics, so we never discretise the L-axis. This is the method of moments, and it is exactly the kind of hand-rolled, no-grid solution Choupo favours. The moments carry direct physical meaning:

moment	meaning	MSMPR value
μ_0	total number / volume	$n^0 G\tau = B^0\tau$
μ_2	$\propto$ total surface area	$2n^0(G\tau)^3$
μ_3	$\propto$ total volume	$6n^0(G\tau)^4$

From them the engineering numbers fall out:

$$\underbrace{L_{10} = \frac{\mu_1}{\mu_0} = G\tau,}_{\text{number mean}} \quad \underbrace{L_{43} = \frac{\mu_4}{\mu_3} = 4G\tau,}_{\text{mass mean}} \quad \underbrace{L_{\text{d}} = 3G\tau}_{\text{dominant (mass mode)}}, \tag{311}$$

the magma (suspension) density

$$M_T \;=\; k_v\, \rho_c\, \mu_3, \tag{312}$$

(k_v the volume shape factor, ρ_c the crystal density — the *number↔mass bridge* on the component's `solid { rho_p; k_v; }` block), and a coefficient of variation that is a pure number:

$$\text{CV} = \sqrt{\frac{\mu_0\mu_2}{\mu_1^2} - 1} = \sqrt{2-1} = 1 \quad (= 100\%). \tag{313}$$

Key idea. An MSMPR crystalliser always produces a PSD with a coefficient of variation of exactly 100 % — independent of kinetics, residence time, everything. It is a structural property of the perfectly-mixed, size-independent-growth exponential, and a sharp sanity check: if your moments do not give CV = 1, the arithmetic is wrong. Choupo's `crystalliser02_msmpr` reports CV = 1.00.

15.7 Kinetics and closing for the supersaturation

Growth and nucleation are empirical power laws in the supersaturation (the FitParameters calibration targets), the standard data-anchored forms:

$$G = k_g\,(S-1)^g, \qquad B^0 = k_b\,(S-1)^b\,M_T^j, \tag{314}$$

the magma exponent j capturing secondary (crystal-assisted) nucleation. Substituting $\mu_3 = 6B^0G^3\tau^4$ and $M_T = k_v\rho_c\mu_3$ closes μ_3 explicitly in S:

$$\mu_3^{1-j} = 6\,G^3\tau^4\,k_b\,(S-1)^b\,(k_v\rho_c)^j \qquad (j<1). \tag{315}$$

The one remaining unknown is the steady supersaturation S itself, fixed by the *solute mass balance*: at steady state the crystal mass produced must equal the solute removed from the liquor,

$$\underbrace{Q\,M_T(S)}_{\text{crystallised}} \;=\; \underbrace{\text{solute}_{\text{in}} - S\,c_{\text{sat}}\,\text{solvent}}_{\text{removed from liquor}}. \tag{316}$$

This is one nonlinear equation in S, solved by a bracketed Newton-1D (Chapter 2) on $[1, S_{\text{feed}}]$ — the left side rises with S, the right side falls, so the root is unique. The residence time itself is a *result* of the hardware, $\tau = V/Q$, with $Q = \sum_i F_i\bar{V}_{\text{liq},i}$ the volumetric throughput.

Intuition. The finite-kinetics MSMPR refines the equilibrium model in a physically honest way. Because crystallisation takes time and the vessel offers only τ seconds, the liquor cannot reach saturation — it leaves *supersaturated* ($S>1$). So the MSMPR yield is always *less* than the equilibrium prediction (299); the gap is the price of running at a finite rate. Slower kinetics or a smaller vessel ⇒ higher residual S ⇒ less yield, but also (through $G\tau$) a different crystal size. The same hardware knob, V, trades yield against size — the central crystalliser design tension.

Worked example 15.1. Sugar MSMPR, `crystalliser02_msmpr`A near-saturated sucrose syrup ($S_{\text{feed}} = 1.66$) is fed to a $V = 5\,\text{m}^3$ cooled vessel at 20 °C; $Q = 1.31\times10^{-3}\,\text{m}^3/\text{s}$ so $\tau \approx 3820\,\text{s}$. The Newton-1D closes (316) at $S = 1.195$, giving $G = 1.95\times10^{-8}\,\text{m/s}$, hence $G\tau = 74.6\,\mu\text{m}$ and

$$L_{\text{d}} = 3G\tau = 224\,\mu\text{m}, \quad L_{43} = 4G\tau = 298.6\,\mu\text{m}, \quad \text{CV} = 1.00,$$

a magma density $M_T = 303\,\text{kg/m}^3$ and a yield of 27.8 % — below the equilibrium figure, because the liquor leaves supersaturated. The ratio $L_{43}/L_{\text{d}} = 4/3$ and CV = 1 are the analytic MSMPR signatures, reproduced to printer precision.

15.8 The batch crystalliser: integrating the moments in time

The MSMPR is the *steady* population balance. Its dynamic sibling is the *batch* crystalliser (`batchCrystalliser`, under choupoBatch): a closed vessel charged supersaturated, where the moments evolve in time. With no flow terms, the population balance (307) reduces to the *moment ODEs*

$$\frac{d\mu_0}{dt} = B^0, \qquad \frac{d\mu_j}{dt} = j\,G\,\mu_{j-1} \;\; (j\ge1), \tag{317}$$

nucleation feeding the count μ_0 at $L = 0$, growth convecting each moment up from the one below. They close (size-independent G) and are integrated by RK4 — the same time-stepping as the batch reactor (Chapter 1). The solute mass balance couples in: the dissolved solute lost is exactly the crystal mass grown,

$$\frac{dn_{\text{solute}}}{dt} = -\,\frac{V\,\rho_c\,k_v}{M_w}\,\frac{d\mu_3}{dt}, \tag{318}$$

so the supersaturation $S = c/c_{\text{sat}}$ falls as crystals form. The packed RK4 state is $(\mu_0,\mu_1,\mu_2,\mu_3,n_{\text{solute}})$. Unseeded, primary nucleation ($j=0$) bootstraps the population from the initial $S>1$; the supersaturation then relaxes toward equilibrium ($S\to1$) — the classic *desupersaturation curve*.

Worked example 15.2. Batch sugar, `batch05_crystalliser`A charge at $S_0 = 1.23$ (20 °C) relaxes over 3 h: S falls $1.23\to1.01$, the Sauter mean grows $0\to105\,\mu$m, and 229 kg of the 1249 kg dissolved sucrose crystallises. The dissolved-plus-crystal sucrose mass is conserved to the gram at every step — the moment↔solute coupling closes the balance.

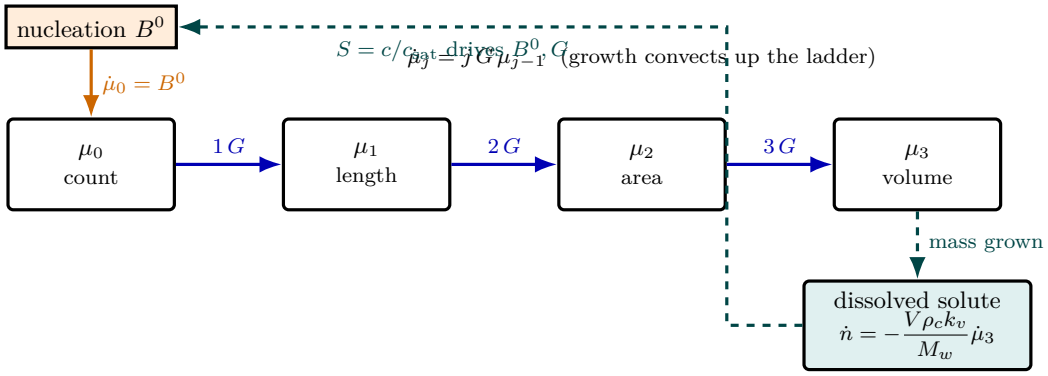

Figure 70: The method of moments turns the population-balance PDE into a small ladder of ODEs — no size grid. The state is just four moments: μ_0 (total crystal count), μ_1 (total length), μ_2 (area), μ_3 (volume $\propto$ mass). **Nucleation** B^0 injects new crystals at $L{=}0$, feeding only μ_0 (orange). **Growth** G convects every moment up from the one below, $\dot{\mu}_j = jG\mu_{j-1}$ (blue) — this is why the system *closes* when G is size-independent: each rate needs only lower moments. The crystal volume μ_3 sets the mass that leaves solution, so the dissolved solute falls (teal), the supersaturation $S = c/c_{\text{sat}}$ relaxes toward 1, and as S drops it feeds back to throttle B^0 and G — the desupersaturation loop. Choupo packs $(\mu_0, \mu_1, \mu_2, \mu_3, n_{\text{solute}})$ into one RK4 state and marches it in time.

15.9 When the moments do not close: the discretised PBE

The method of moments is exact, but only when the closure works. Let the growth law be *size-dependent* —

$$G(L) \;=\; G_0\,(1 + \alpha L), \qquad \alpha = \text{sizeFactor}\;\; [1/m] \tag{319}$$

— and the cascade $d\mu_j/dt = j\,G\,\mu_{j-1}$ no longer closes: multiplying the population balance (307) by L^j and integrating, the αL term pulls in a moment one step *above* the row we are integrating,

$$\frac{d\mu_j}{dt} \;=\; j\,G_0\,\big(\mu_{j-1} + \alpha\,\mu_j\big), \tag{320}$$

which is closed only by coincidence ($\alpha = 0$). The same trouble shows up in agglomeration, breakage, and any law where G depends on L explicitly. We need a representation of the PDF $n(L)$ that is *not* a finite list of moments.

The *finite-volume method* (FVM) discretises $n(L)$ on N evenly-spaced size bins,

$$L_k = (k + \tfrac{1}{2})\,\Delta L, \quad k = 0, \ldots, N{-}1, \quad \Delta L = L_{\max}/N, \tag{321}$$

with $n_k \equiv n(L_k)$ the cell-centred number density. The steady MSMPR PBE (307) for an arbitrary $G(L)$ reads

$$\frac{d}{dL}\big[G(L)\,n(L)\big] + \frac{n(L)}{\tau} \;=\; 0. \tag{322}$$

First-order upwind on the bin boundary gives the cell-to-cell recurrence

$$n_k \;=\; n_{k-1}\,\frac{G_{k-1}}{G_k + \Delta L/\tau}, \qquad k \geq 1, \tag{323}$$

seeded by the boundary condition $n_0 = B^0/G(0)$. Moments are then *post-computed* by quadrature,

$$\mu_j \;\approx\; \sum_k L_k^j\,n_k\,\Delta L, \tag{324}$$

and the outer closure on the supersaturation S is the same Newton-1D on the solute mass balance ($\rho_c k_v \mu_3 =$ solute consumed) that closes the MSMPR.

Sanity vs MSMPR. With $\alpha = 0$ the FVM scheme above reduces to a geometric sequence whose continuous limit is $n(L) = n^0\,e^{-L/(G\tau)}$ — the MSMPR exponential. The discretisation error is $\mathcal{O}(\Delta L/(G\tau))$, so 64 bins already give $\sim 5\,\%$ tail error and 256 bins close the gap to $\sim 1\,\%$. Tutorial `crystalliser03_pbe_size_independent` runs the FVM with $\alpha = 0$ against the same kinetics as `crystalliser02_msmpr` and reproduces every observable (L_{43}, CV, M_T, yield) to better than $1\,\%$ — the numerical sanity check before the model is trusted in the regime where the analytic answer does not exist.

The payoff. Tutorial `crystalliser04_size_dependent_growth` repeats the same case with $\alpha = 500\,\mathrm{m}^{-1}$ (+50 % growth enhancement at $L = 1\,\mathrm{mm}$). The big crystals out-grow the small ones: the $n(L)$ profile shifts visibly toward larger sizes, L_{43} climbs $298.6 \to 329.4\,\mu\mathrm{m}$ (+10 %), CV widens $1.00 \to 1.04$, and the magma density rises $306 \to 311\,\mathrm{kg/m}^3$. The closed-form MoM cannot deliver any of these numbers; the FVM does, on the same scaffold of kinetics + vessel + supersaturation closure.

What does NOT do. The current FVM is restricted to primary nucleation ($j = 0$ in $B^0 = k_b(S-1)^b M_T^j$); a secondary $j > 0$ would couple B^0 to μ_3 through M_T, demanding an inner iteration on the boundary condition. Agglomeration and breakage (integral source terms in the PBE, the third reason the moments do not close) are also out of scope — the FVM scaffold is ready for them (per-cell birth and death terms), but the kernel infrastructure is not.

> **Runnable case.** `tutorials/steady/crystallisation/crystalliser01_sugar` — the equilibrium (infinite-kinetics) yield model, `model equilibrium`.
> `tutorials/steady/crystallisation/crystalliser02_msmpr` — the continuous MSMPR population balance by the method of moments, `model MSMPR`; the profile is the $n(L)$ line and the product PSD.
> `tutorials/steady/crystallisation/crystalliser03_pbe_size_independent` — the discretised PBE on a size grid, `model FVM` with $\alpha = 0$: sanity validation against MSMPR, same observables within $\sim 1\,\%$.
> `tutorials/steady/crystallisation/crystalliser04_size_dependent_growth` — the FVM with $\alpha > 0$: the cut that justifies the discretised solver, where the MoM closure fails.
> `tutorials/batch/reactor/batch05_crystalliser` — the dynamic batch PBE (choupoBatch): the moments + solute integrated in time, the desupersaturation curve.

16 Spiral-wound membrane: solution-diffusion and concentration polarisation

What you'll learn. Pressure-driven membranes (reverse osmosis, nanofiltration) separate without a phase change: push a feed against a film and water crosses while salt is held back. Two coupled physics set the performance — *solution-diffusion* (how water and solute permeate the membrane) and *concentration polarisation* (how rejected solute piles up at the wall and fights back through osmotic pressure). This chapter derives the local flux, shows why it needs an inner Newton, and marches it down the feed channel. It is the research bridge of the simulator — Vítor's NF/RO domain — and the consumer of the Pitzer osmotic model of Chapter 3.

16.1 Solution-diffusion: two independent fluxes

In the solution-diffusion picture [20] the membrane is a dense film; water and solute dissolve into it and diffuse down their own potential gradients, so their fluxes are *independent*. The water (volume) flux is driven by the net pressure — the applied ΔP minus the osmotic back-pressure $\Delta\pi$ it must overcome:

$$J_w \;=\; A_w\,(\Delta P - \Delta\pi), \tag{325}$$

with A_w the membrane's water permeability. The solute flux is driven by the concentration difference across the membrane, independent of pressure:

$$J_s \;=\; B_s\,(c_m - c_p), \tag{326}$$

with B_s the solute permeability, c_m the concentration at the membrane wall and c_p in the permeate. These two coefficients (A_w, B_s) — carried per membrane in `data/standards/membranes/` — are the whole material model; a tight RO membrane has a tiny B_s, a loose NF membrane a larger one (hence NF's partial rejection).

The permeate concentration is not independent: the solute that crosses is swept along by the water, so $c_p = J_s/J_w$. Substituting (326),

$$c_p = \frac{B_s(c_m - c_p)}{J_w} \quad\Longrightarrow\quad \boxed{\,c_p = \frac{B_s}{J_w + B_s}\,c_m.\,} \tag{327}$$

Already a key result: as J_w grows (high pressure) $c_p \to 0$ — more water dilutes the fixed solute leak, so *rejection improves with pressure*.

16.2 Where the coefficients come from — and where the model ends

Equations (325)–(326) look phenomenological, but they follow from ONE structural premise that Wijmans & Baker [20] place at the centre of the model: **inside a dense membrane the pressure is uniform** (equal to the feed-side pressure), so the chemical-potential gradient that drives permeation is expressed entirely as a *concentration* (activity) gradient. A pore-flow membrane is the exact mirror — uniform concentration, pressure gradient — and the two pictures give different, testable flux laws. From that premise, water transport integrates to $J_w \propto \left(1 - e^{-\bar{V}_w(\Delta P - \Delta\pi)/RT}\right)$, whose linearisation (the exponent is $\ll 1$ at process conditions) is (325) with

$$A_w \;=\; \frac{D_w\,K_w\,c_w\,\bar{V}_w}{R\,T\,\ell}, \tag{328}$$

diffusivity × solubility × molar volume over RT and film thickness ℓ: the "permeability" is not a fudge factor but a composite of measurable membrane properties — which is why A_w is a *material* constant in `data/standards/membranes/` and not a per-case knob. The same integration for the solute (whose $\bar{V}_s\Delta P/RT$ term is negligible) yields (326): solute flux blind to pressure, the signature the classic RO experiments confirm.

Intuition. **Know where the model ends.** Wijmans & Baker close their review exactly where nanofiltration lives: between dense RO films (solution-diffusion) and open UF pores (pore-flow) sits a transition at virtual pore sizes of roughly 5–10 Å. A loose NF membrane (the catalogue's NF270) is *in* that transition: solution-diffusion with a fitted (A_w, B_s) still describes its water flux and partial rejection, but charge effects and convective coupling grow underneath — which is why the catalogue also carries `NF270_dspmde` (the Donnan-steric, charge-aware sibling) and why the two give different answers for multivalent ions. Choosing between them IS a modelling decision the case author owns.

16.3 Concentration polarisation: the film model

Water leaves through the membrane; the solute it rejects does not, so solute accumulates in a thin boundary layer at the wall. At steady

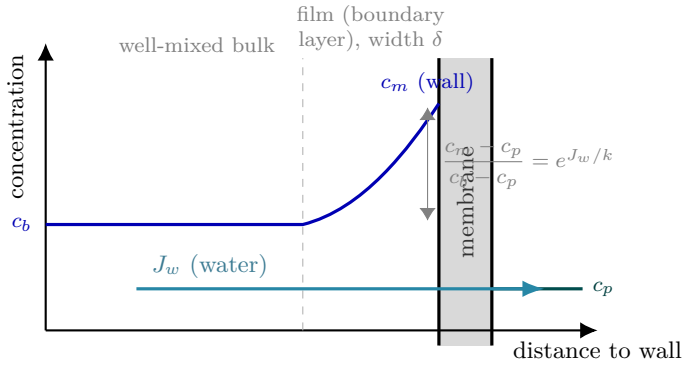

Figure 71: Concentration polarisation, the wall fighting back. Water crosses the membrane (J_w, cyan), but the rejected solute does not — it piles up in a thin film of width δ at the wall, so the wall concentration c_m swells *above* the well-mixed bulk c_b (blue). The film model balances convection toward the wall against back-diffusion (coefficient k) and gives the polarisation factor $E = e^{J_w/k}$: poor mixing (small k) or high flux makes c_m tower over c_b. Because the osmotic back-pressure is set by c_m, not c_b, polarisation is self-limiting — pushing harder raises J_w, which raises c_m, which raises $\Delta\pi$ and pushes back. The permeate c_p (teal) is the small leak that the inner Newton closes against.

state the convective solute carried toward the membrane balances back-diffusion plus the solute that permeates. Integrating across the film of mass-transfer coefficient k gives the *film model*:

$$\frac{c_m - c_p}{c_b - c_p} = \exp\!\left(\frac{J_w}{k}\right), \tag{329}$$

where c_b is the bulk (channel) concentration. The exponential $E = e^{J_w/k}$ is the *polarisation factor*: poor mixing (small k) or high flux makes c_m swell far above c_b.

Combining the permeate relation (327) with the film model (329) eliminates c_p and gives the wall concentration in closed form,

$$c_m = \frac{E\, c_b}{1 + \dfrac{B_s}{J_w + B_s}\,(E - 1)}, \tag{330}$$

exactly the expression `localFluxes` evaluates (with $\varphi = J_w + B_s$).

16.4 The coupling, and why it needs an inner Newton

Equations (325), (327), (330) are circular: J_w depends on $\Delta\pi$, which depends on c_m and c_p, which depend on J_w. The osmotic back-pressure is supplied by the selected osmotic model (Chapter 3) evaluated at each face,

$$\Delta\pi = \pi(c_m) - \pi(c_p) \quad \big[\text{van't Hoff } \nu R_g T c, \text{ or Pitzer } \phi(I)\,\nu R_g T c\big]. \tag{331}$$

Choupo closes the loop at each channel station with a one-dimensional Newton in J_w on the residual

$$F(J_w) = J_w - A_w\big(\Delta P - \Delta\pi(J_w)\big) = 0, \tag{332}$$

the $\Delta\pi$ derivative taken by central difference (the $A_w\Delta P$ part is constant), damped so J_w stays positive and bounded. It converges in a handful of iterations from the no-osmotic guess $J_w = A_w\Delta P$.

> *Intuition.* The osmotic term is what makes a membrane non-trivial. Without it $J_w = A_w\Delta P$ — a one-line answer. With it, pushing harder raises J_w, which polarises the wall (higher c_m), which raises $\Delta\pi$, which pushes back on J_w: a self-limiting feedback. The inner Newton finds the balance point. This is exactly why a real RO plant cannot simply crank the pressure — the wall fights back.

16.5 Marching down the channel

A spiral-wound element is not a point: as feed flows along it, water is removed, so the bulk concentration c_b rises and the pressure P_b falls. Choupo discretises the feed channel into stations and *marches*: at each station it solves the local flux above, removes the permeated water and solute from the bulk, updates c_b and P_b, and steps on — a 1-D plug-flow integration along the leaf. The profile (`J_w(z)`, $c_b(z)$, $P_b(z)$) is the output a membrane engineer reads. Two observed performance numbers summarise it:

$$\text{recovery} = \frac{\text{permeate flow}}{\text{feed flow}}, \qquad R_{\text{obs}} = 1 - \frac{c_p}{c_b} \;\; (\text{vs the } \textit{true}\; 1 - c_p/c_m). \tag{333}$$

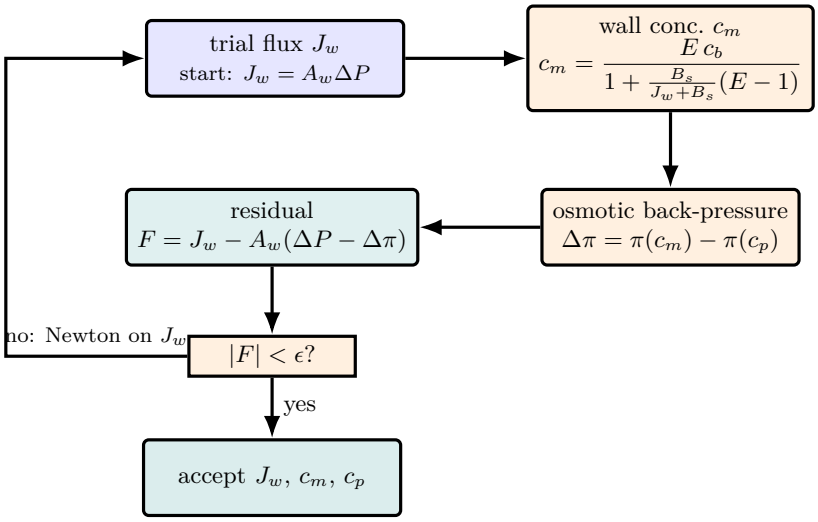

Figure 72: The local membrane flux is circular, so Choupo closes it with a one-dimensional Newton at every channel station. A trial water flux J_w sets the polarisation E and hence the wall concentration c_m (orange); c_m sets the osmotic back-pressure $\Delta\pi$; and $\Delta\pi$ feeds back into the solution-diffusion law through the residual $F = J_w - A_w(\Delta P - \Delta\pi)$ (teal). Starting from the no-osmotic guess $J_w = A_w \Delta P$, the Newton converges in a handful of iterations. This loop is exactly the physical feedback that stops a real RO plant from simply cranking the pressure: more flux → more polarisation → more back-pressure → the wall fights back.

The gap between observed and true rejection *is* the polarisation penalty.

Three pieces are selectable sub-models (the same factory pattern as the EoS or the cyclone), all living in the optional membrane module: k (constant, or a Schock–Miquel Sherwood correlation from the spacer hydraulics), ΔP along the channel (constant, or Schock–Miquel friction), and the osmotic model (van't Hoff or Pitzer). An `elements N` keyword chains N elements in series (the retentate of one is the feed of the next), with an inter-element ΔP loss — a full train.

Worked example 16.1. Seawater RO, `membrane01_RO_NaCl_seawater`35 g/L NaCl at 55 bar through an SW30HR-style element (A_w, B_s for NaCl, $k = 50\,\mu$m/s): the inner Newton balances J_w against the ∼28 bar osmotic back-pressure, giving $J_{w,\mathrm{avg}} \approx 13\,\mu$m/s, ∼15.6 % water recovery and **99.78 % observed NaCl rejection**; the NaCl mass balance closes to < 0.04 %. Switching the osmotic block to Pitzer (`membrane06_pitzer`) lowers $\Delta\pi$ by ∼8 % and lifts recovery to 17.2 %.

Runnable case. `tutorials/steady/membranes/membrane01_RO_NaCl_seawater` — canonical seawater RO.
`tutorials/steady/membranes/membrane02_NF_sugar` — loose NF, the defining *partial* rejection.
`tutorials/steady/membranes/membrane03_concentration_polarization` — sweeps k to draw the polarisation curve.
`tutorials/steady/membranes/membrane05_train` — a 3-element train, the diminishing-returns recovery down the series.

17 Nanofiltration with charge: the Donnan-Steric Pore Model with Dielectric Exclusion (DSPM-DE)

What you'll learn. Solution-diffusion (§16) treats every solute the same: one permeability B_s per salt, fitted, not predicted. It cannot say *why* a nanofiltration membrane rejects $MgSO_4$ at ∼98 % while letting half the NaCl through at the same pore size. The answer is *charge*. An NF membrane is a bundle of nm-scale pores carrying a fixed wall charge; ions partition into those pores against three barriers — steric, electrostatic (Donnan), and dielectric (Born) — and then move under the coupled gradient of concentration *and* electric potential (extended Nernst–Planck). This chapter derives the Donnan-Steric Pore Model with Dielectric Exclusion [21, 22], the charge-aware sibling of solution-diffusion in the same transport factory (`transport DSPM-DE;`), and shows how the divalent-co-ion rejection ordering falls out of the partition with no per-salt fit.

17.1 The pore picture and the three exclusions

DSPM-DE replaces the dense film with a bundle of cylindrical pores of effective radius r_p, fixed volumetric charge density X_d (signed; negative for a polyamide NF carrying carboxylate groups), porosity A_k and active-layer thickness Δx — the geometry is carried in the membrane `.dat poreModel{}` block as r_p, $A_k/\Delta x$, X_d and the pore dielectric ε_p. Each *ion* i (the salt is split into its cation and anion through the component's `electrolyte{ cation; anion; }` block) carries a radius r_i, a free-solution diffusivity D_i and a signed charge z_i.

An ion entering a pore pays three independent partition penalties; each multiplies the bulk concentration to give the pore concentration at the interface (Bowen & Welfoot, Eq. 7):

$$\frac{c_{\mathrm{pore},i}}{\gamma_i\, c_{b,i}} = \underbrace{\Phi_{S,i}}_{\text{steric}}\ \underbrace{\exp\!\Big(-\tfrac{z_i F}{R_{\mathrm{g}} T}\,\Delta\psi_D\Big)}_{\text{Donnan}}\ \underbrace{\exp\!\Big(-\tfrac{\Delta W_i}{k_B T}\Big)}_{\text{Born}}, \tag{334}$$

with γ_i the activity coefficient of ion i at the wall (Davies, §3), evaluated once per channel station — not inside the inner solve. The three factors are:

Steric. A finite ion cannot reach within r_i of the pore wall. The geometric partition for a sphere in a cylinder is

$$\Phi_{S,i} = (1-\lambda_i)^2, \qquad \lambda_i = \frac{r_i}{r_p}, \tag{335}$$

the same λ that sets the *hindrance* factors below. A purely steric membrane ($X_d = 0$, $\varepsilon_p = \varepsilon_b$) is a sieve — the uncharged starting point of the staged build.

Donnan. The fixed pore charge X_d must be balanced. At each pore mouth a Donnan potential jump $\Delta\psi_D$ develops that sets pore electroneutrality,

$$\sum_i z_i\, c_{\mathrm{pore},i} + X_d = 0. \tag{336}$$

With $X_d < 0$ the pore expels the *co-ion* (the anion). Because the penalty is $\exp(-z_i F\Delta\psi_D/R_{\mathrm{g}}T)$, a divalent co-ion ($z = \pm 2$) feels the jump *twice* as hard as a monovalent one — the origin of the NF fingerprint. (334) is monotone in $\Delta\psi_D$, so Choupo closes (336) by a robust *bisection* on $\psi = \Delta\psi_D F/R_{\mathrm{g}}T$ over $[-50, 50]$ rather than a Newton step that could fall onto the trivial $\psi = 0$ saddle (the $K = 1$ trap of §9; see the staged-build note).

Born (dielectric exclusion). Water confined in a nm pore is more ordered, so its dielectric constant ε_p is lower than the bulk ε_b (Choupo takes $\varepsilon_b = \varepsilon_b(T)$ from the solvent-properties table and ε_p from the membrane `.dat`). Moving a charged ion into the lower-ε medium costs Born solvation energy

$$\Delta W_i = \frac{z_i^2 e^2}{8\pi\varepsilon_0 r_i}\Big(\frac{1}{\varepsilon_p} - \frac{1}{\varepsilon_b}\Big), \tag{337}$$

(Bowen & Welfoot, Eq. 16; e the elementary charge, ε_0 the vacuum permittivity) — the very same Born form already met in the eNRTL solvent-shift of §3. Like Donnan it scales with z_i^2, so it reinforces divalent exclusion — and, unlike Donnan, it rejects *both* ions of a divalent salt, which is why $MgSO_4$ can be held above what charge alone would give. Set $\varepsilon_p < \varepsilon_b$ in the `.dat` to turn it on.

17.2 Transport through the pore: extended Nernst–Planck

Inside the pore each ion moves under the coupled gradient of its own concentration, the electric potential, and convection with the water (Bowen & Welfoot, Eq. 5):

$$j_i = -K_{d,i}\, D_i\, \frac{dc_i}{dx} - \frac{z_i c_i D_i K_{d,i} F}{R_{\mathrm{g}} T}\, \frac{d\psi}{dx} + K_{c,i}\, c_i\, J_v, \tag{338}$$

where J_v is the volume (water) flux and $K_{d,i}$, $K_{c,i}$ are the hydrodynamic *hindrance* factors — a confined ion diffuses and is convected more slowly than in free solution. Choupo uses Deen's centreline forms [23] as fitted by Bowen & Welfoot (Eqs. 14–15) in $\lambda = r_i/r_p$:

$$K_{d,i} = 1 + \tfrac{9}{8}\lambda \ln\lambda - 1.56034\,\lambda + 0.528155\,\lambda^2 + 1.91521\,\lambda^3 - 2.81903\,\lambda^4 + 0.270788\,\lambda^5 + 1.10115\,\lambda^6 - 0.435933\,\lambda^7, \tag{339}$$

$$K_{c,i} = (2 - \Phi_{S,i})\left(1 + 0.054\,\lambda - 0.988\,\lambda^2 + 0.441\,\lambda^3\right). \tag{340}$$

The pore potential gradient is not free: it is fixed by the *zero-net-current* condition (no external circuit on an NF element),

$$\sum_i z_i\, j_i = 0. \tag{341}$$

At steady state the flux each ion carries out is swept into the permeate by convection, $j_i = J_v\, c_{p,i}$. Integrating (338) across a thin pore with a uniform potential gradient gives the classical convective–diffusive pore *transmission*

$$\frac{c_{\mathrm{out},i}}{c_{\mathrm{in},i}} = \frac{K_{c,i}}{K_{c,i} + \left(1 - K_{c,i}\, e^{-\mathrm{Pe}_i}\right) R_i}, \qquad \mathrm{Pe}_i = \frac{K_{c,i}\, J_v}{K_{d,i}\, D_i\, (A_k/\Delta x)}, \tag{342}$$

where Pe_i is the ion Péclet number (convection vs hindered pore diffusion) and R_i is the ratio of the feed-face to permeate-face partition coefficients — the Donnan/Born asymmetry between the two mouths. The permeate-face Donnan potential ψ_p is again closed by electroneutrality on the permeate, $\sum_i z_i c_{p,i} = 0$, by the same bisection. This closed form is what keeps the model glass-box: no inner RK4 marching the pore, yet the full charge coupling is retained.

17.3 Closing the water flux and marching the channel

The water flux still obeys the solution-diffusion law (325), $J_w = A_w(\Delta P - \Delta\pi)$, but now $\Delta\pi = R_{\mathrm{g}}T\left(\sum_i c_{\mathrm{wall},i} - \sum_i c_{p,i}\right)$ is a van't Hoff sum over *all* ions, both faces. Choupo closes the J_w–$\Delta\pi$ loop with a damped fixed point (relaxation 0.5), seeded from the no-osmotic guess $J_w = A_w\Delta P$; each evaluation runs the feed-face partition, the transmission (342), and the permeate-face Donnan close. The per-ion permeate concentrations are then collapsed back to the per-salt c_p the spiral-wound march expects (stoichiometric, electroneutral), and the channel is marched station-by-station exactly as in §16.

Key idea. Staged exclusions — avoiding the $K = 1$ trap. A symmetric charged solve can collapse onto the trivial Donnan = 0 saddle (the same pathology as symmetric LLE, §9). DSPM-DE is therefore built as *visible switches*, each seeded from the simpler converged state: **(a)** steric only ($X_d = 0$, $\varepsilon_p = \varepsilon_b$) seeded from the solution-diffusion result; **(b)** +Donnan ($X_d \neq 0$) seeded from (a); **(c)** +Born ($\varepsilon_p < \varepsilon_b$) seeded from (b). Which stage is active is decided by the fields present in the membrane `.dat` and *announced in the run header* — never a silent fallback. The Donnan and permeate potentials are found by monotone bisection, which *cannot* land on the $\psi = 0$ branch when a non-trivial one exists.

Worked example 17.1. The divalent rejection ordering, `membrane10_dspmde_divalent`Three parallel NF270-archetype elements ($r_p = 0.60$ nm, $X_d = -5$ mol/m^3, $\varepsilon_p = 35$ vs $\varepsilon_b \approx 78$), each fed 5 mM of a single salt at 10 bar. With *no* per-salt permeability fit, the DSPM-DE partition reproduces the canonical NF fingerprint

$$R_{\mathrm{obs}}(\mathrm{MgSO_4}) \;>\; R_{\mathrm{obs}}(\mathrm{CaCl_2}) \;>\; R_{\mathrm{obs}}(\mathrm{NaCl}),$$

Choupo returning $R_{\mathrm{obs}}(\mathrm{NaCl}) \approx 55\,\%$ against near-total rejection of the divalent-co-ion salt $\mathrm{MgSO_4}$. The z^2 in both the Donnan (334) and the Born (337) terms is the whole story: the divalent co-ion $\mathrm{SO_4^{2-}}$ is excluded far harder than the monovalent $\mathrm{Cl^-}$ at the same pore. Plain solution-diffusion, with one B_s per salt, can be *fitted* to this ordering but cannot *predict* it. Toggling ε_p off (`membrane11_dspmde_born_toggle`) isolates the Born contribution.

Power and limit. The power is mechanism: DSPM-DE predicts the salt-by-salt and divalent rejection ordering from a handful of *physical* membrane parameters (r_p, X_d, ε_p, $A_k/\Delta x$) plus tabulated ion radii and diffusivities — no per-salt B_s to fit. It is the mechanistic NF model and the research bridge of the simulator's membrane domain. The limit is that every parameter is an *effective* fitted quantity: the pore radius depends on the ion-radius convention (Choupo's catalogue uses hydrated/Nightingale radii, so $r_p = 0.6$ nm is consistent with *those*, not with the Stokes-radius fits of the original papers); the charge density X_d is in reality pH- and concentration-dependent (it grows on dilution and with rising pH) but is a constant scalar here; ε_p is a single fitted dielectric; the closed-form transmission assumes a uniform potential gradient across a thin pore; and the model is for an aqueous electrolyte, not a general mixed-charge speciation solver. Treat the numbers as a mechanistic ordering tool, calibrated per operating point — *all models are wrong, some are useful.*

Runnable case. `tutorials/steady/membranes/membrane10_dspmde_divalent` — the divalent > mixed > monovalent rejection signature, three salts on one membrane.
`tutorials/steady/membranes/membrane11_dspmde_born_toggle` — the same case with the Born (dielectric) term toggled on/off to isolate its contribution.

18 Electrodialysis: Faraday transfer, the Nernst stack, and the limiting current

What you'll learn. Electrodialysis (ED) separates ions *electrically*, not by pressure or phase change. A stack of N alternating cation- (CEM) and anion- exchange (AEM) membranes is sandwiched between two electrodes; an applied DC current pulls cations one way and anions the other, draining a *diluate* channel and enriching a *concentrate* channel. This chapter shows the four equations the `electrodialysisStack` unit solves: **(i)** Faraday's law for the ion transfer, **(ii)** the Nernst membrane potential built from *real* Davies activities per channel, **(iii)** the ohmic drop across the membrane and solution resistances, and **(iv)** the Cowan–Brown limiting current density — the hard ceiling above which the boundary layer runs out of ions. You will see why ED reuses the electrolyte stack (Chapter 3) wholesale rather than re-deriving an activity model, and why the stack *refuses to clamp* an over-limiting current.

18.1 The stack: where charge meets selectivity

A *cell pair* is the repeating unit: one CEM, one diluate channel, one AEM, one concentrate channel. Stack N of them between two electrodes and the same current I passes through every membrane in series. Because a CEM is nearly impermeable to anions and an AEM nearly impermeable to cations, the cations migrating toward the cathode are trapped by the next AEM and the anions migrating toward the anode are trapped by the next CEM: salt accumulates in alternate channels (the concentrate) and is stripped from the others (the diluate). The electrodes themselves run the redox reactions that source and sink the electrons, but ED gathers *many* cell pairs per electrode pair, so the fixed electrode overpotential is amortised — the reason ED is an N-times-cheaper-per-membrane process than a single electrochemical cell.

Choupo's unit takes *two* feeds (`inputs ( diluateFeed concentrateFeed )`) and returns two products (`outputs ( diluate concentrate )`), plus an electrical-work stream published on an energy wire ($P = UI$), mirroring the shaft-work convention of the rotating gear in Chapter 21. The components must be ionic species with a row in `ions.dat` (`Na`, `Cl`, ...) plus `water` as the carrier — exactly the analysis the `ionExchanger` expects.

18.2 Faraday's law: charge in, moles out

The current is a flux of charge; each mole of a z-valent counter-ion carried across a membrane consumes zF coulombs, where $F = 96\,485$ C/mol is Faraday's constant. Inverting, the molar transfer rate of a counter-ion per cell pair is

$$\dot{n}_i \;=\; \xi\,\frac{I}{z_i F}, \tag{343}$$

and the whole stack moves $N\dot{n}_i$ moles per second out of the diluate and into the concentrate. Here $\xi \in (0, 1]$ is the *current efficiency* (default 0.9, announced in the log): not all of the charge does useful work. The shortfall $(1 - \xi)$ is co-ion leakage (a real membrane is not perfectly selective), electro-osmotic water drag, and any parasitic electrode reaction. Choupo treats ξ as a lumped, explicit operating number the student owns — there is no hidden permselectivity sub-model in v1.

Equation (343) is a hard upper bound on what current can remove: the diluate cannot give up more of an ion than it carries. The solver therefore *caps* the transfer at the diluate inflow and emits a LOUD warning when the cap binds (the diluate is fully demineralised for that ion) — the "no silent clamp" credo of Chapter 1 applied to charge.

Key idea. Two operating modes, exactly one of which you give. `current <I> A;` fixes the current and the demineralisation emerges; `targetDemin <fraction>;` fixes the demineralisation of the reference cation and Choupo solves (343) for the current I with a glass-box 1-D Newton (the residual $\dot{n}_{\text{removed}}/\dot{n}_{\text{in}} - \text{target}$, printed). Giving both, or neither, is a degrees-of-freedom error and the unit says so.

18.3 The membrane potential: Nernst on real activities

Even at zero current a cell pair sustains a potential, because each membrane separates two solutions of different ionic activity. A perfectly selective CEM is reversible to its counter-cation, so its equilibrium (Donnan + diffusion) potential is the Nernst expression

$$E \;=\; \frac{R_{\mathrm{g}} T}{zF}\,\ln\frac{a_{\text{conc}}}{a_{\text{dil}}}, \tag{344}$$

evaluated on the *activity ratio* of the counter-ion across the membrane, not the bare concentration ratio. The activity is $a_i = \gamma_i\, m_i$, and the activity coefficient γ_i is the very same **Davies** model the electrolyte stack uses (Chapter 3):

$$\log_{10}\gamma_i \;=\; -A\,z_i^2\left(\frac{\sqrt{I}}{1+\sqrt{I}} - 0.3\,I\right), \qquad I = \tfrac{1}{2}\sum_k m_k z_k^2, \tag{345}$$

with $A = 0.51$ at 25°C (scaled in T by the Debye–Hückel factor) [26]. The unit speciates each channel *once* per pass, freezes the γ_i, and reuses them everywhere — there is **no parallel γ implementation** for ED. The two membranes of a cell pair add their potentials (the CEM on the mean cation ratio, the AEM on the mean anion ratio, $z < 0$ flipping the ratio):

$$E_{\mathrm{mem}} \;=\; E_{\mathrm{CEM}} + E_{\mathrm{AEM}} \quad \text{(V per cell pair)}. \tag{346}$$

This E_{mem} is the *back-EMF*: it opposes the applied current, growing as the concentrate/diluate ratio widens — the thermodynamic "price" of separating against an activity gradient.

18.4 The ohmic drop: membranes and solutions in series

Driving the current I also pays an ohmic toll across the four resistances of a cell pair: the two membranes and the two solution channels. The membrane areal resistances $R^{A}_{\mathrm{CEM}}, R^{A}_{\mathrm{AEM}}$ [$\Omega\,\mathrm{m}^2$] come straight from the IEM `.dat` (measured in 0.5 M NaCl); divided by the active area they give resistances in ohms. The solution resistance of a channel of gap h and conductivity κ is

$$R_{\mathrm{sol}} \;=\; \frac{h}{\kappa\, A_{\mathrm{mem}}}, \qquad \kappa \;=\; \sum_i c_i\,\lambda_i, \qquad \lambda_i \;=\; \frac{z_i^2 F^2 D_i^0}{R_{\mathrm{g}} T}, \tag{347}$$

where the equivalent ionic conductance λ_i is built from the infinite-dilution diffusivity D_i^0 (the `ions.dat` transport tier, Nightingale 1959 [29]) via the **Nernst–Einstein** relation, and $c_i \approx m_i \rho_w$ in the dilute-carrier approximation ($\rho_w \approx 1000$ kg/m^3, announced). The per-cell-pair resistance is the series sum

$$R_{\mathrm{pair}} = \frac{R^{A}_{\mathrm{CEM}}}{A_{\mathrm{mem}}} + \frac{R^{A}_{\mathrm{AEM}}}{A_{\mathrm{mem}}} + R_{\mathrm{dil}} + R_{\mathrm{conc}}. \tag{348}$$

18.5 Assembling the stack voltage and power

The terminal voltage of the whole stack is the sum, over N cell pairs, of the back-EMF plus the ohmic drop, with the lumped electrode overpotential added once:

$$U \;=\; N\big(E_{\mathrm{mem}} + I\,R_{\mathrm{pair}}\big) + E_{\mathrm{electrodes}}, \qquad P = U I = W_{\mathrm{electric}}. \tag{349}$$

$E_{\mathrm{electrodes}}$ is a single ANNOUNCED number, not a resolved Butler–Volmer kinetics: ED v1 does not model the electrode reactions (the shared `electrochem` kernel ships a fully-implemented `butlerVolmer` stub for a future electrolysis unit, so the kernel need not fork). The electrical power P is published on the energy wire; divided by the diluate volumetric product it gives the headline *specific energy* in kWh/m^3.

18.6 The limiting current: the ceiling Cowan and Brown drew

The defining constraint of ED is the **limiting current density**. As current rises, the diluate-side boundary layer at the membrane face is depleted of ions faster than diffusion can replenish it; at the limiting current i_{lim} the interfacial concentration hits zero and the ohmic resistance there diverges. Past it, the only charge carriers left are H^+ and OH^- from water splitting — current efficiency collapses, the pH swings, and the membranes are stressed. Cowan and Brown [24] gave the working form

$$i_{\mathrm{lim}} \;=\; \frac{z F\, k\, c_{\mathrm{dil}}}{t_{\mathrm{cu}} - t_{\mathrm{co}}}, \qquad t_{\mathrm{co}} = 1 - t_{\mathrm{cu}}, \tag{350}$$

where c_{dil} is the bulk equivalent concentration of the *depleting* (diluate) channel, t_{cu} the counter-ion transport number *in the membrane* (from the IEM `.dat`), and t_{co} the co-ion number; the factor $(t_{\mathrm{cu}} - t_{\mathrm{co}}) = (2t_{\mathrm{cu}} - 1)$ measures how hard the membrane forces *all* the current onto the counter-ion, draining the boundary layer. The mass-transfer coefficient k is estimated from a Lévêque-type Sherwood correlation for the thin spacer slit,

$$\mathrm{Sh} = 1.85\left(\mathrm{Re}\,\mathrm{Sc}\,\frac{d_h}{L}\right)^{1/3}, \qquad k = \frac{\mathrm{Sh}\,D}{d_h}, \tag{351}$$

with $d_h \approx 2h$ for the slit, $\mathrm{Re} = v\,d_h/\nu$, $\mathrm{Sc} = \nu/D$, and the cross-flow velocity v supplied as `linearVelocity`. The student therefore *sees the limiting current rise with flow* — the central design lever of an ED stack. This is the honest "simple correlation over theory" rung: ν is taken as the dilute-water value ($\sim 10^{-6}$ m^2/s) and L defaults to the channel length, both ANNOUNCED.

> **Common pitfall.** Choupo **never clamps** the current at i_{lim}. If you specify a current with $i > i_{\mathrm{lim}}$, the unit returns the unclamped Faraday transfer of (343) *and* prints a loud over-limiting warning with the ratio i/i_{lim}. Silently capping would hide the single most important operating fault of an ED stack — glass-box honesty means showing you the physically-meaningless answer *labelled as such*, not a quietly "fixed" one.

Worked example 18.1. NaCl desalination, `electrochem/ed01_nacl_desalination`A 100-cell-pair Neosepta CMX/AMX stack ($R^A_{\mathrm{CEM}} = 3.0\ \Omega\,\mathrm{cm}^2$, $t_{\mathrm{cu}} = 0.98$; $R^A_{\mathrm{AEM}} = 2.4\ \Omega\,\mathrm{cm}^2$) desalinates a 0.1 mol/kg NaCl diluate against a 0.5 mol/kg concentrate, 0.2 m^2 per membrane, 0.5 mm channel gap, $v = 0.05$ m/s. With $I = 8$ A and $\xi = 0.9$, Eq. (343) removes $\xi IN/(zF) = 0.9 \cdot 8 \cdot 100/(1 \cdot 96485) \approx 7.5 \times 10^{-3}$ mol/s of Na^+, against a diluate Na^+ inflow of $5.0 \cdot 0.001795 \approx 9.0 \times 10^{-3}$ mol/s — about 83 % demineralisation. The log prints the Davies γ and κ of each channel, the Nernst E_{mem} from those real activities, the series ohmic R_{pair}, the assembled stack voltage U and power $P = UI$, and the Cowan–Brown i_{lim} with the verdict $i/i_{\mathrm{lim}} < 1$ (a safe sub-limiting design). The companion case `electrochem/ed02_over_limiting_current` pushes I past i_{lim} and trips the over-limiting warning.

Runnable case. `tutorials/electrochem/ed01_nacl_desalination` — canonical sub-limiting NaCl desalination; `current` mode.
`tutorials/electrochem/ed02_over_limiting_current` — the same stack driven past i_{lim}; the LOUD over-limiting warning, no silent clamp.

19 Ion-exchange softening: Gaines–Thomas mass action

What you'll learn. A water softener trades the *hardness* cations (Ca^{2+}, Mg^{2+}) for the sodium that a strong-acid cation (SAC) resin carries. The exchange is not a black-box "percent removal" — it is a *competitive chemical equilibrium* between the dissolved ions and a fixed-capacity solid phase, governed by the same mass-action thermodynamics as an aqueous complex. This chapter shows how Choupo writes that equilibrium as ONE extra unknown and ONE extra balance bolted onto the speciation Newton of Chapter 3, why the bound-species activity is an *equivalent fraction* rather than a concentration, where the $CaCO_3$-equivalent hardness and the unavoidable "salt penalty" come from, and — the honest limit — why this predicts the *limiting* effluent of a fully-equilibrated contactor, not the rising-leakage breakthrough curve of a bed in service.

The `ionExchanger` unit op (and its beaker-scale twin, the `exchange` property op) sits downstream of the aqueous-speciation machinery. Its purpose is concrete: take a brackish or hard water analysis, equilibrate it with a softening resin, and report the softened effluent — the feed a downstream NF/RO element (§16) sees once the scale-formers are stripped out. The case `membrane08_softened_scaling` chains exactly this: soften, then watch the calcite/gypsum saturation indices of Chapter 3 collapse.

19.1 The exchange half-reaction and the selectivity constant

A SAC resin is a cross-linked polymer skeleton hung with fixed, negatively charged sulfonate groups; we write the free site as X^-. Electroneutrality forces every site to carry a counter-cation, so the resin is best read as a separate, fixed-capacity *pseudo-phase*. A divalent cation Me^{z+} binds by displacing z singly-charged sites:

$$Me^{z+} + z\,X^- \rightleftharpoons MeX_z, \qquad K = \frac{a_{MeX_z}}{a_{Me}^{1}\, a_X^{z}}. \tag{352}$$

For a softener the players are $Na^+, K^+, Ca^{2+}, Mg^{2+}$ (plus Sr^{2+}, Ba^{2+} when scaling matters), with bound species $NaX, KX, CaX_2, MgX_2, \ldots$. Choupo reads this network from `exchange.dat` (curated from the USGS PHREEQC `EXCHANGE_SPECIES` block, public domain [42]); the Na^+ exchange is the reference reaction with $\log K = 0$, so each $\log K_{25}$ is a *selectivity relative to sodium*. The representative values in the catalogue are

$$\log K_{CaX_2} = 0.80, \quad \log K_{MgX_2} = 0.60, \quad \log K_{KX} = 0.70, \quad \log K_{NaX} = 0.00, \tag{353}$$

which encode the textbook affinity ordering $Ca^{2+} > Mg^{2+} > K^+ > Na^+$ for a sulfonated SAC resin. An optional van't Hoff ΔH slot lets $\log K(T)$ drift with temperature exactly as in the K-value chapter (§9).

19.2 The Gaines–Thomas convention: activity = equivalent fraction

The novelty of (352) is the activity of a *bound* species. We cannot use its molality — a sulfonate group is not free to roam — so we need a reference state for the solid pseudo-phase. The Gaines–Thomas convention [43] takes the activity of a bound species to be its *equivalent fraction* on the resin:

$$a_{MeX_z} = \beta_{MeX_z} = \frac{z\, m_{MeX_z}}{\mathrm{CEC}}, \qquad a_X = \beta_X = \frac{m_X}{\mathrm{CEC}}, \tag{354}$$

where m_{MeX_z} is the bound molality [mol/kg water], the factor z converts moles of bound cation to *equivalents* (charges occupied), and the cation-exchange capacity

$$\mathrm{CEC} = \sum_s z_s\, m_{sX} \qquad \text{[eq/kg water]} \tag{355}$$

is the total number of charged sites. The equivalent fractions sum to one, $\sum_s \beta_s + \beta_X = 1$, with the convention that the *free* site X^- carries the residual. The crucial point: in Choupo's kernel the resin is treated as always electroneutral — there is no thermodynamically "empty" free site that competes for cations. β_X is therefore not a populated species but the activity variable that the mass actions (352) solve *from*, while the conserved capacity (355) is the closure that pins it. The aqueous-ion activity $a_{Me} = \gamma_{Me}\, m_{Me}$ keeps its Davies γ from Chapter 3; only the resin phase uses $\gamma = 1$ with the equivalent-fraction reference.

Intuition. A bound cation is not dissolved — it is parked on a site. "How much is parked" is naturally a *fraction of the sites*, not a concentration in a volume that has no meaning for a solid. The equivalent fraction is the resin's mole fraction, weighted by charge because a divalent ion eats two sites. That single change of reference state is the whole difference between an aqueous-complexation equilibrium and an ion-exchange equilibrium; everything else — the mass action, the Newton, the Jacobian — is reused unchanged.

19.3 One extra unknown, one extra balance — folded into the speciation Newton

The implementation (`src/thermo/electrolyte/SpeciationSolver.cpp`) is deliberately frugal. The free site X^- joins the log-molality unknown vector as one extra coordinate $x_X = \ln m_X$, and the capacity (355) joins the residual system as one extra *equality*:

$$R_{\mathrm{CEC}} \;=\; \mathrm{CEC} \;-\; \sum_s z_s\, m_{s\mathrm{X}} \;=\; 0. \tag{356}$$

Each bound species reuses the same mass-action machinery as an aqueous complex. Taking logs of (352) with (354) and solving for the bound molality,

$$\ln m_{s\mathrm{X}} \;=\; \ln 10\,\log K \;+\; \ln\gamma_{\mathrm{Me}} + \ln m_{\mathrm{Me}} \;+\; z\,\ln m_{\mathrm{X}} \;+\; \underbrace{\left[\ln\tfrac{\mathrm{CEC}}{z} - z\ln\mathrm{CEC}\right]}_{\text{CEC constant, folded into } \log K}, \tag{357}$$

so the constant CEC term is pre-folded into an *effective* $\log K$ at build time, and the species is presented to the solver with stoichiometry $\{(\mathrm{Me},1),(\mathrm{X},z)\}$ and $\gamma \equiv 1$ — byte-identically to how an aqueous complex with the same legs is handled. The free site is *excluded* from the ionic strength, the water activity, and the aqueous electroneutrality balance, because it lives on the resin, not in solution. When the exchange spec is empty the whole branch is skipped and the result is identical to the no-exchange speciation, by construction — the glass-box guarantee that adding a feature cannot silently move an unrelated answer.

The same gamma-fixed-point / Newton solver of Chapter 3 then converges molalities, the free site, and (when `pH solve;`) the proton together. A resin shipped in the sodium form starts holding CEC equivalents of Na; that initial load is added to the conserved Na total, so the Na not retained on the sites is *released* to the water eq-for-eq — the model's account of where the displaced sodium goes.

19.4 Hardness, leakage, and the salt penalty

The headline output is *hardness*, reported in the practitioner's universal currency, mg/L as $CaCO_3$:

$$H \;=\; \big(m_{\mathrm{Ca}} + m_{\mathrm{Mg}}\big) \times M_{\mathrm{CaCO_3}} \times 1000, \qquad M_{\mathrm{CaCO_3}} = 100.09\ \mathrm{g/mol}, \tag{358}$$

with molalities [mol/kg] read as mg/L at the dilute closure $\rho \approx 1\,\mathrm{kg/L}$. Expressing both Ca^{2+} and Mg^{2+} as if they were calcium carbonate is the convention that lets a single number stand for "total hardness" regardless of which divalent cation supplies it. The removal is $100\,(H_{\mathrm{in}} - H_{\mathrm{out}})/H_{\mathrm{in}}$; the residual Ca^{2+}, Mg^{2+} in the effluent is the *leakage*. Every equivalent of hardness removed is balanced by an equivalent of sodium released, the *salt penalty*

$$\Delta m_{\mathrm{Na}} \;=\; m_{\mathrm{Na}}^{\mathrm{out}} - m_{\mathrm{Na}}^{\mathrm{in}} \;\approx\; 2\big(\Delta m_{\mathrm{Ca}} + \Delta m_{\mathrm{Mg}}\big) \quad \text{[eq-for-eq]}, \tag{359}$$

which is why a softener lowers hardness but *raises* the sodium load — a real cost for sodium-restricted feed waters and for any downstream RO that must then reject the added Na. Choupo also reports the resin loadings $\beta_{\mathrm{CaX_2}}, \beta_{\mathrm{MgX_2}}, \beta_{\mathrm{NaX}}$ and the CEC utilised ($\approx 100\,\%$, since the sites are always full).

Power and limit. **Power.** The competitive equilibrium falls straight out of the selectivity constants (353) — no fitted "removal efficiency". Because it reuses the speciation Newton, the softened water is a fully consistent analysis: its activities, ionic strength, and saturation indices are immediately available to the scaling and membrane chapters. The salt penalty (359) emerges automatically from charge conservation. **Limit.** The result is the *limiting effluent* — water *fully* equilibrated with the resin at its stated CEC, i.e. the best a fresh or fully-loaded contactor can do per pass. A real fixed bed leaks rising hardness as the exchange front migrates toward breakthrough, then must be regenerated; cycle length, bed-volumes-to-breakthrough, and regeneration chemistry are *transient* and are not modelled. The safe reading is therefore a bound: real leakage $\geq$ this equilibrium leakage — if the equilibrium effluent is already hard, no bed will soften it. The bed volume, if given, is announced but does not enter the per-pass equilibrium (it sets the cycle length, not the limiting composition). The Davies γ is trusted only to $I \lesssim 0.5\,\mathrm{mol/kg}$; beyond that the equilibrium is flagged *indicative*. Choupo prints all of this as a non-suppressible honesty banner — the limiting-case caveat is part of the answer, not a footnote.

Worked example 19.1. SAC softener on a hard water, `softener_brackish`A hard water carrying Ca^{2+}, Mg^{2+}, Na^+ and the matching anions is equilibrated with the `SAC_Na` resin — a sulfonated polystyrene-DVB strong-acid cation exchanger in the sodium form, nameplate CEC = 2.0 eq/L of settled bed (taken as eq/kg water at the dilute closure) [44, 45]. The kernel adds one unknown ($\ln m_X$) and one balance (the CEC capacity (356)) to the existing speciation Newton and converges in a handful of steps. At equilibrium the high-selectivity divalents (353) drive almost entirely onto the resin ($\beta_{\mathrm{CaX_2}}, \beta_{\mathrm{MgX_2}}$ dominate; the residual β_{NaX} is small), the hardness (358) collapses by well over 90 %, and the sodium rises by the eq-for-eq salt penalty (359). Feeding the softened analysis to the calcite/gypsum saturation check of Chapter 3 (case `membrane08_softened_scaling`) then shows the scaling indices dropping below zero — the point of softening ahead of an RO element.

Runnable case. `tutorials/props/electrolyte/softener_brackish` — the beaker-scale `exchange` property op: Gaines–Thomas softening of a hard water, with the resin-loading isotherm β_s vs. solution equivalent fraction written for the GUI to plot.
`tutorials/steady/membranes/membrane08_softened_scaling` — the `ionExchanger` unit op in a flowsheet, soften-then-watch-the-scaling coupled to the saturation indices of Chapter 3.

20 Adsorption: isotherms, LDF uptake, and the fixed bed

What you'll learn. A gas molecule sticking to a porous solid is an *equilibrium* (the isotherm, a curated pair datum: adsorbent × species), a *rate* (the linear driving force, an equipment datum: this particle, in this bed), and a *contactor* (the closed vessel or the 1-D fixed bed). This chapter gives the Henry and Langmuir isotherms with their van't Hoff temperature dependence, the extended (competitive) Langmuir together with its *declared* thermodynamic inconsistency, the LDF rate and why its k can never live in a standards catalogue, the conservative finite-volume bed balance with Danckwerts boundaries, the stoichiometric time every breakthrough curve must obey before a single time step is taken — and, closing the loop, how a "steady-state" pressure- or temperature-swing unit reads as the *time average of a twin-bed pair*.

20.1 The equilibrium loading: Henry and Langmuir

The state of the solid is the *loading* q_i — moles of adsorbate i per kilogram of regenerated, dry adsorbent (mol/kg) — and the driving force the isotherm sees is the partial pressure p_i (Pa). The extensive solid inventory is always $m_{\mathrm{ads}}\, q_i$, with m_{ads} the adsorbent mass of the *equipment*, never of the catalogue.

At vanishing coverage every isotherm is linear (Henry's law of the adsorbed phase):

$$q = H\, p, \qquad H = \lim_{p \to 0} \frac{q}{p}. \tag{360}$$

The Langmuir model adds the finite site count: a balance over identical, independent sites with fractional coverage $\theta = q/q_{\max}$, adsorption rate $\propto p\,(1-\theta)$ and desorption rate $\propto \theta$, gives at equilibrium

$$q(T,p) \;=\; q_{\max}\, \frac{b(T)\, p}{1 + b(T)\, p}, \tag{361}$$

with the two limits that make it teachable: $p \to 0$ recovers (360) with $H = q_{\max}\, b$, and $p \to \infty$ saturates at the monolayer $q_{\max}$. The affinity b carries the temperature dependence through van't Hoff on the adsorption equilibrium:

$$b(T) \;=\; b_{\mathrm{ref}}\, \exp\!\left[-\frac{\Delta H_{\mathrm{ads}}}{R} \left(\frac{1}{T} - \frac{1}{T_{\mathrm{ref}}} \right) \right], \tag{362}$$

with $\Delta H_{\mathrm{ads}} < 0$ for exothermic adsorption (the usual case), so b *falls* as T rises — beds regenerate hot. T_{ref} is a mandatory part of the curated record, never an implicit 298 K. The equilibrium record (its $q_{\max}$, b_{ref}, ΔH_{ads}, bases, validity window and primary citation) is a *pair* datum — adsorbent × species — and lives in the standards catalogue; the evaluation bench (`tutorials/props/adsorption/isotherm01_eval_13x_co2`) runs the Henry-limit, saturation, pin and unit-invariance gates aloud on every record it touches, and the regression bench (`isotherm02_fit_jaschik`) shows how such a record is *earned* from data — including the identifiability refusals (single-temperature data cannot yield ΔH_{ads}; Henry-regime data yields only the product $q_{\max} b$).

20.2 Competition: the extended Langmuir, honestly

For a mixture the workhorse teaching model is the *extended* (competitive) Langmuir — all species compete for the same sites, each term in the denominator crowding everyone else out:

$$q_i \;=\; q_{\max,i}\, \frac{b_i(T)\, p_i}{1 + \sum_j b_j(T)\, p_j}. \tag{363}$$

Now the honesty clause, stated here exactly as it is stated in the run headers: (363) is thermodynamically consistent *only* if all the $q_{\max,i}$ are equal. With distinct $q_{\max,i}$ the model violates the Gibbs–Duhem relation of the adsorbed phase — there is no single free-energy surface (no consistent spreading pressure) from which all the q_i derive, so the predicted loadings depend on a path that thermodynamics says must not matter. It remains a *useful, declared* teaching model, and Choupo keeps it as the default mixing rule precisely because its error is announced rather than hidden. The honest extension is the Ideal Adsorbed Solution Theory (IAST), which builds the mixture from the *pure-component* isotherms by equating spreading pressures — an explicit extension point of the mixing-rule layer, never a silent default.

20.3 The rate: linear driving force, and whose datum k is

Uptake is not instantaneous. The linear driving force (LDF) model lumps the whole diffusional resistance chain into one first-order relaxation toward the local equilibrium loading $q_i^* = q_i^*(T,p)$:

$$\frac{\mathrm{d}q_i}{\mathrm{d}t} \;=\; k_i\, (q_i^* - q_i). \tag{364}$$

The coefficient k_i (1/s) aggregates film, macropore and crystal diffusion *of that particle in that bed*; Glueckauf's classic estimate $k \approx 15\, D_e / r_p^2$ makes the point — k depends on the particle radius, i.e. on the *equipment*, not on

the chemical identity of the pair. It therefore lives in the case dict (with declared scope and source), never in the standards catalogue, and a missing k_i for an active species is a named refusal, not a default. In a closed vessel (364) couples to the gas inventory ($\mathrm{d}n_i/\mathrm{d}t = -m_{\mathrm{ads}}\,\mathrm{d}q_i/\mathrm{d}t$, $p_i = n_i RT/V_{\mathrm{gas}}$); in the Henry limit the coupled system is *linear* with

$$q(t) = q_\infty\big(1 - e^{-k_{\mathrm{eff}}t}\big), \qquad k_{\mathrm{eff}} = k\Big(1 + \frac{m_{\mathrm{ads}} H R T}{V_{\mathrm{gas}}}\Big), \tag{365}$$

the closed form the integrator must reproduce to 10^{-8} (`tutorials/batch/adsorber/batch11_adsorber_henry`) — the analytic gate that certifies the numerics before any nonlinear case is trusted.

20.4 The 1-D isothermal fixed bed

The bed balance is written the way it is solved: conservative finite volumes. For cell j of volume V_j, species i:

$$\varepsilon\, V_j\, \frac{\mathrm{d}c_{ij}}{\mathrm{d}t} \;=\; F^{\mathrm{conv}}_{i,j-\frac{1}{2}} - F^{\mathrm{conv}}_{i,j+\frac{1}{2}} \;+\; F^{\mathrm{disp}}_{i,j-\frac{1}{2}} - F^{\mathrm{disp}}_{i,j+\frac{1}{2}} \;-\; \rho_b\, V_j\, \frac{\mathrm{d}q_{ij}}{\mathrm{d}t}, \tag{366}$$

with upwind convective face fluxes $u\,A\,c$ (u the superficial velocity), dispersive fluxes $-\varepsilon D_{\mathrm{ax}} A\,\partial c/\partial z$ at the interior faces, and the LDF sink (364) evaluated at the local p_{ij}. Two boundary statements (Danckwerts): at the inlet the *flux* is imposed — the first face carries exactly $u\,c_{\mathrm{in}}$, so what enters is what you fed, dispersion included — and at the outlet $\partial c/\partial z = 0$, so nothing diffuses back in from the void beyond the bed.

Two honesty notes belong to this equation. First, the density trap: ρ_b is the *bulk* (packed-bed) density, kg of solid per m^3 of bed, so $\rho_b\, q$ is already mol per bed volume — writing $(1-\varepsilon)\rho_p$ with ε and ρ_p from different sources double-counts the porosity, the classic error, and is forbidden. Second, the declared closure: u, P, T are held constant in this phase of the model (momentum and energy balances are later extensions), yet a bed taking up 15 % of its feed *does* slow the gas. Holding u constant therefore fabricates carrier gas at exactly the net uptake rate — and the engine says so out loud, prints the fabricated moles, pins them as a KPI, and discounts them in the campaign ledger. The model's residual is *declared*, at a named magnitude, instead of being absorbed invisibly — the difference between a wrong number and an honest model.

20.5 The stoichiometric time: where the front must arrive

Before integrating anything, an overall mass balance already fixes where the breakthrough front must sit. Feed a clean bed at superficial velocity u and inlet concentration c_{in}; idealise the front as sharp. When it has just traversed the bed, everything fed has gone into filling the bed's two inventories — gas in the voids and adsorbate on the solid:

$$u\,A\,c_{\mathrm{in}}\,t_{\mathrm{st}} \;=\; L\,A\,\big[\varepsilon\,c_{\mathrm{in}} + \rho_b\,q^*(c_{\mathrm{in}})\big] \quad\Longrightarrow\quad t_{\mathrm{st}} \;=\; \frac{L}{u}\underbrace{\left[\varepsilon + \frac{\rho_b\,q^*(c_{\mathrm{in}})}{c_{\mathrm{in}}}\right]}_{R_f}, \tag{367}$$

where R_f is the retention factor and $u_{\mathrm{sh}} = u/R_f$ the front (shock) velocity — typically hundreds of times slower than the gas. Kinetics (k) and dispersion (D_{ax}) spread the front into a mass-transfer zone *around* t_{st}, but cannot move its centroid: because (366) telescopes exactly, the integral identity

$$\int_0^\infty \left(1 - \frac{c_{\mathrm{out}}}{c_{\mathrm{in}}}\right)\mathrm{d}t \;=\; t_{\mathrm{st}} \tag{368}$$

holds for the conservative scheme on *any* mesh, while the useful capacity t_b (breakthrough at, say, 5 %) converges with the mesh at the upwind scheme's first order — run the mesh study and watch both statements hold (`tutorials/batch/adsorber/batch13_breakthrough_co2`, `batch15_mesh_study`; the pure-transport twin `batch14_transport_only` pins the $k = 0$ bed against the analytic advection–dispersion solution).

20.6 Reading a "steady" swing unit: the twin-bed time average

A fixed bed is intrinsically transient — so what is a pressure-swing or temperature-swing unit doing in a *steady* flowsheet? It is the *time average of a pair of columns at cyclic steady state*: one bed adsorbs at high pressure (or low temperature) while its twin regenerates at low pressure (or high temperature), the valves swap, and over a full cycle the pair's averaged inlet and outlet flows are constant. The working capacity per cycle is bounded by the equilibrium swing $\Delta q = q^*(p_{\mathrm{high}}) - q^*(p_{\mathrm{low}})$ for PSA — or, through (362), by $q^*(T_{\mathrm{low}}) - q^*(T_{\mathrm{high}})$ for TSA. The case `tutorials/steady/separation/psa01_h2_psa` is exactly this abstraction for the pressure swing: competitive Langmuir loadings at P_{high} and P_{low} set the equilibrium ceiling, a declared utilisation derating and a purge sweep complete the shortcut, and the run header lists out loud what the shortcut does *not* model (breakthrough and cycle time, thermal effects, blowdown loss). The transient bed of the previous subsection is the machinery that will eventually replace the shortcut's derating with a computed one — the two levels of description are the same physics at two Pareto points.

> **Runnable case.** `tutorials/props/adsorption/` — the isotherm bench: evaluation gates (`isotherm01`), the multi-T regression with identifiability verdicts (`isotherm02`), the refusal case (`isotherm03`).
> `tutorials/batch/adsorber/` — the uptake staircase: closed vessel to inventory equilibrium (`batch09`),

extended-Langmuir competition (`batch10`), the analytic Henry gate (`batch11`), the pressure refusal (`batch12`), breakthrough with the announced t_{st} and the declared model residual (`batch13`), pure transport vs the analytic solution (`batch14`), the mesh study (`batch15`).
`tutorials/steady/separation/psa01_h2_psa` — the twin-bed equilibrium shortcut in a steady flowsheet.

21 Rotating equipment: compressors, turbines, pumps

What you'll learn. A compressor, turbine or pump is specified by the *work* you put on its shaft and its efficiency — never by the outlet pressure. That is the credo for rotating gear (Chapter 1): you furnish W_{shaft} and η, the pressure *emerges*. This chapter shows how the discharge state is found from an isentropic reference using the real-fluid entropy and enthalpy of Chapter 7's neighbourhood, why it needs a nested Newton for a compressible fluid, and why a pump — incompressible — needs none at all.

21.1 The isentropic reference and the efficiency

A reversible, adiabatic (hence *isentropic*) machine is the thermodynamic ideal. Real machines fall short by the isentropic efficiency η. The construction has three steps:

1. the ideal discharge has the *same entropy* as the inlet, $S(T_{\text{isen}}, P_{\text{out}}) = S(T_{\text{in}}, P_{\text{in}})$, which fixes T_{isen} once P_{out} is known;
2. the ideal work is the enthalpy change to that state, $w_{\text{isen}} = h(T_{\text{isen}}, P_{\text{out}}) - h_{\text{in}}$;
3. the real work relates to it by η — and here compressor and turbine differ:

$$\text{compressor: } w_{\text{real}} = \frac{w_{\text{isen}}}{\eta}, \qquad \text{turbine: } w_{\text{real}} = \eta\, w_{\text{isen}}, \tag{369}$$

because a compressor needs *more* work than ideal while a turbine delivers *less*.

The real discharge temperature then follows from the first law, $h(T_{\text{out}}, P_{\text{out}}) = h_{\text{in}} + w_{\text{real}}$ — the lost work reappears as heat, so $T_{\text{out}} > T_{\text{out,isen}}$.

21.2 Why a nested Newton (compressor / turbine)

The catch: P_{out} is unknown — it is what we furnished work to produce. We know $w_{\text{real}} = W_{\text{shaft}}/\dot{n}$, hence w_{isen}, and we search for the P_{out} whose isentropic enthalpy rise equals it. Choupo (`IsentropicCore`) runs an *outer* Newton-1D on P_{out},

$$F(P_{\text{out}}) = h\big(T_{\text{isen}}(P_{\text{out}}), P_{\text{out}}\big) - (h_{\text{in}} + w_{\text{isen}}) = 0, \tag{370}$$

wrapping an *inner* Newton that solves the entropy match for $T_{\text{isen}}(P_{\text{out}})$ at each trial pressure. H and S are the real-fluid $H = H^{\text{ig}} + H^{R}$, $S = S^{\text{ig}} + S^{R}$ with the residuals from the cubic EoS (SRK / PR). For an ideal gas the two levels decouple and it converges in one outer step; for a real gas they couple (4–10 steps). The expansion branch simply searches $P_{\text{out}} < P_{\text{in}}$.

Worked example 21.1. Air compressor, `compressor01_air`10 kW of shaft work, $\eta = 0.75$, on 1 mol/s of air: the nested solve returns $P_{\text{out}} = 8.90$ bar and $T_{\text{out}} = 633$ K, within 1 % of the hand calculation. A companion DesignSpec (`compressor02_designspec_pout`) manipulates W_{shaft} to *hit* a target $P_{\text{out}} = 8$ bar — the rotating-gear analogue of sizing an evaporator's area, the "specify the pressure" that is really a disguised DesignSpec.

21.3 The pump: incompressible, no EoS, no Newton

A liquid's density barely changes with pressure, so a pump needs none of the above. The work raises the pressure directly,

$$\Delta P = \frac{\eta\, W_{\text{shaft}}}{\dot{n}\,\bar{v}}, \qquad P_{\text{out}} = P_{\text{in}} + \Delta P, \tag{371}$$

with $\bar{v}$ the liquid molar volume; the small inefficiency dissipates as a tiny temperature rise $\Delta T = (1-\eta) w_{\text{real}}/C_{p,L}$. A closed form — the incompressible limit needs no equation of state and no iteration.

Intuition. Because that relation is *linear* in the work, it inverts in closed form: writing $Q_v = \dot{n}\,\bar{v}$ for the volumetric flow (which the feed already fixes),

$$\Delta P = \frac{\eta\, W_{\text{shaft}}}{Q_v} \quad \Longleftrightarrow \quad W_{\text{shaft}} = \frac{Q_v\, \Delta P}{\eta}.$$

So the pump can be specified *either* by the motor power (and the discharge pressure is the result) *or* by the discharge pressure P_{out} — or rise ΔP — it must reach (and the shaft power is the result). The second is the everyday engineering case: a pump feeds a header of *known* pressure, and you want to know the power it draws. Crucially this is **not** a DesignSpec — the target is local to the unit, so it is the same one-shot algebra solved the other way, not an outer Newton loop. A DesignSpec is only needed when the target lives *elsewhere* in the flowsheet. (Reaching for a DesignSpec on every one of a plant's dozens of pumps, just to set their discharge pressures, would be needless iteration.) The unit requires *exactly one* of $\{W_{\text{shaft}}, P_{\text{out}}, \Delta P\}$; a let-down with $P_{\text{out}} < P_{\text{in}}$ is a `valve`, not a pump.

Runnable case. `tutorials/steady/rotating/compressor01_air` — furnish work, the pressure emerges. `tutorials/steady/rotating/turbine01_air` — expansion, $T_{\text{out}} > T_{\text{out,isen}}$ (losses stay as heat). `tutorials/steady/rotating/pump01_water` — the incompressible closed form, head ≈ 736 m. `tutorials/steady/rotating/pump02_pressure_spec` — the SAME pump set by its discharge pressure ($P_{\text{out}} = 30$ bar); the shaft power is the result — the exact inverse of pump01.

22 Pipe pressure drop: Darcy–Weisbach and the friction factor

What you'll learn. A pump has a shaft and you furnish its work; a pipe has neither shaft nor knob. The pressure a pipe costs you is *never* a specification — it is a **computed result** of the geometry (diameter, length, roughness, elevation, fittings) and the flow (velocity, density, viscosity). This chapter derives the Darcy–Weisbach equation from the mechanical-energy balance, shows where the dimensionless friction factor f comes from in each flow regime — the exact laminar 64/Re, the implicit Colebrook–White reference, and the explicit Haaland and Churchill fits Choupo offers — and adds the static-head and minor-loss terms that close a real engineering line. The watchword is the valve's: *a pipe only dissipates, exactly as much as the physics says.*

22.1 The mechanical-energy balance for an incompressible liquid

Write the steady-flow mechanical-energy balance (the engineering Bernoulli equation) between the inlet (1) and outlet (2) of a constant-area pipe carrying an incompressible liquid of density ρ:

$$\frac{P_1}{\rho} + \tfrac{1}{2}u_1^2 + g z_1 \;=\; \frac{P_2}{\rho} + \tfrac{1}{2}u_2^2 + g z_2 \;+\; \underbrace{g\,h_f}_{\text{friction}} . \tag{372}$$

For a uniform bore the velocity is unchanged ($u_1 = u_2 = u$, since the volumetric flow $Q_v = uA$ is conserved and the area A is constant), so the kinetic terms cancel. Multiplying through by ρ and collecting,

$$\boxed{\Delta P \equiv P_1 - P_2 \;=\; \underbrace{\rho\, g\, h_f}_{\text{dissipation}} \;+\; \underbrace{\rho\, g\, \Delta z}_{\text{static head}} , \qquad \Delta z = z_2 - z_1 .} \tag{373}$$

The first term is the irreversible loss — mechanical energy that disappears into the fluid's internal energy through viscous shear at the wall. The second is the *reversible* cost of lifting the fluid a height Δz (a rise costs pressure; a drop returns it). Crucially z is a property of the *geometry*, not the flow, so the static head appears even at zero velocity.

Intuition. Where does the dissipated mechanical energy go? Into heat. But Choupo holds T across the pipe and does *not* re-inject the loss as a temperature rise — exactly as it does for the `valve`. The reason is honesty about scale: for a well-insulated short line the viscous heating raises T negligibly ($\Delta T \sim \Delta P/(\rho C_p) \sim 10^{-2}\,\mathrm{K}$ for a 0.4 bar drop in water), and the energy that *matters* — the irreversibility — is already accounted for, in the pressure. The drop in P *is* the lost work; re-injecting it as heat would double-count it.

22.2 Darcy–Weisbach: the friction head

The friction head h_f is not arbitrary. Dimensional analysis of fully-developed pipe flow shows the wall loss scales with the velocity head $\rho u^2/2$, the length-to-diameter ratio L/D, and a dimensionless *Darcy friction factor* f:

$$\rho\, g\, h_f \;=\; f\,\frac{L}{D}\,\frac{\rho u^2}{2}, \qquad \text{so} \qquad h_f \;=\; f\,\frac{L}{D}\,\frac{u^2}{2g}. \tag{374}$$

This is the **Darcy–Weisbach equation** [10]. The velocity, area and Reynolds number follow from the feed and the bore,

$$A = \frac{\pi D^2}{4}, \quad u = \frac{Q_v}{A}, \quad Q_v = \frac{\dot m}{\rho} = \frac{\dot n\, M_w}{\rho}, \quad \mathrm{Re} = \frac{\rho\, u\, D}{\mu}, \tag{375}$$

with $\dot n$ the molar flow, M_w the mixture molar mass, and μ the liquid viscosity (from the `thermoPackage` transport model of Chapter 2, or an explicit `operation` override). The only unknown left in Eq. (374) is f, and *all* of pipe hydraulics is the one question: *what is f?*

22.3 The friction factor in each regime

Laminar ($\mathrm{Re} < 2300$). Here the velocity profile is the exact Hagen–Poiseuille parabola, and integrating the wall shear gives a *closed form* with no adjustable constant and *no roughness dependence* (a smooth laminar sublayer buries the wall texture):

$$f_{\mathrm{lam}} = \frac{64}{\mathrm{Re}}. \tag{376}$$

Turbulent ($\mathrm{Re} > 4000$). Now the wall roughness matters, through the *relative roughness* ε/D (ε the absolute peak height, e.g. 4.6×10^{-5} m for commercial steel). The reference correlation is the implicit **Colebrook–White** equation [11], which interpolates Nikuradse's smooth- and rough-pipe data:

$$\frac{1}{\sqrt{f}} = -2\log_{10}\!\left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{\mathrm{Re}\,\sqrt{f}}\right). \tag{377}$$

It is transcendental in f (the unknown sits on both sides), so Choupo solves it by fixed-point iteration on $x = 1/\sqrt{f}$,

$$x_{k+1} = -2\log_{10}\left(\frac{\varepsilon/D}{3.7} + \frac{2.51\,x_k}{\mathrm{Re}}\right), \tag{378}$$

seeded with the Haaland value below so it converges in a handful of steps (this is the glass-box `f_colebrook` you can read in `Pipe.cpp`).

An explicit shortcut: Haaland. To avoid the iteration, Haaland [12] gives an explicit approximation within ∼2 % of Colebrook over the whole turbulent range:

$$\frac{1}{\sqrt{f}} = -1.8\log_{10}\left[\left(\frac{\varepsilon/D}{3.7}\right)^{1.11} + \frac{6.9}{\mathrm{Re}}\right]. \tag{379}$$

Choupo's `Haaland` and `Colebrook` models both fall back to the exact 64/Re of Eq. (376) below Re = 2300.

One expression for every regime: Churchill (the default). Churchill [13] stitched laminar, transition and turbulent flow into a *single* explicit formula valid for every Re and every ε/D — so it needs no laminar branch and is smooth through the awkward transition zone:

$$f = 8\left[\left(\frac{8}{\mathrm{Re}}\right)^{12} + \frac{1}{(A+B)^{3/2}}\right]^{1/12}, \tag{380}$$

$$A = \left\{-2.457\,\ln\left[\left(\frac{7}{\mathrm{Re}}\right)^{0.9} + 0.27\,\frac{\varepsilon}{D}\right]\right\}^{16}, \qquad B = \left(\frac{37530}{\mathrm{Re}}\right)^{16}.$$

As $\mathrm{Re} \to 0$ the $(8/\mathrm{Re})^{12}$ term dominates and Eq. (380) collapses to $f = 8(8/\mathrm{Re}) = 64/\mathrm{Re}$ — it reproduces the laminar law automatically. This is why Choupo defaults to `Churchill`; the other two are offered for pedagogy and for matching textbook hand calculations.

22.4 Minor losses and the assembled ΔP

Bends, valves, expansions and tees each dissipate a fixed multiple of the velocity head, captured by a *loss coefficient* K. Summing the fittings and substituting Eq. (374) into Eq. (373) gives the assembled pressure drop Choupo computes:

$$\boxed{\Delta P = \underbrace{f\,\frac{L}{D}\,\frac{\rho u^2}{2}}_{\text{distributed}} + \underbrace{\left(\sum_j K_j\right)\frac{\rho u^2}{2}}_{\text{minor (fittings)}} + \underbrace{\rho\, g\,\Delta z}_{\text{elevation}}, \qquad P_{\text{out}} = P_{\text{in}} - \Delta P.} \tag{381}$$

The friction head reported as a KPI is $h_f = (\Delta P_{\text{fric}} + \Delta P_{\text{fit}})/(\rho g)$ — the dissipative part only, excluding the reversible static head. If P_{out} comes out non-positive the solver warns aloud: the line is grossly undersized or too long for this flow, and you need a larger D or a pump — it never silently "fixes" the geometry.

Power. Three lines of physics — a velocity-head scaling, a friction-factor correlation, and a static head — predict the pressure a real piping run costs, with the factor traceable to Nikuradse's data through Colebrook. The model is closed-form (or one short fixed-point loop), so it costs almost nothing inside a flowsheet pass.

Limit. The equations above are for a *single-phase incompressible liquid*. For a single *gas*, ρ falls as P drops along the pipe, so one (ρ, u) evaluation is wrong — compressible flow needs the isothermal- or adiabatic-pipe integral, a documented future extension. *Gas-liquid two-phase* flow, by contrast, IS handled — it is the next subsection. There is no fitting database (you furnish each K), and the K-method itself is a lumped approximation to genuinely 3-D fitting flows.

Worked example 22.1. Water line, `pipe01_water_line`Water at 300 K, 5 bar, 2000 kmol/h flows through $D = 0.1$ m, $L = 50$ m commercial steel ($\varepsilon = 4.6 \times 10^{-5}$ m, so $\varepsilon/D = 4.6 \times 10^{-4}$), rising $\Delta z = 2$ m, with two elbows ($K = 0.9$ each) and an open globe valve ($K = 10$), giving $\sum K = 11.8$. The properties are $\rho = 875.5\,\mathrm{kg/m^3}$ and $\mu = 8.54 \times 10^{-4}$ Pa s. The kinematics:

$$\dot m = \dot n M_w = 0.5556\ \tfrac{\mathrm{kmol}}{\mathrm{s}} \times 18.02 = 10.0\ \mathrm{kg/s}, \quad Q_v = \frac{10.0}{875.5} = 1.14 \times 10^{-2}\ \tfrac{\mathrm{m^3}}{\mathrm{s}},$$

$$A = \frac{\pi(0.1)^2}{4} = 7.85 \times 10^{-3}\,\mathrm{m^2}, \quad u = 1.455\ \mathrm{m/s}, \quad \mathrm{Re} = \frac{875.5 \cdot 1.455 \cdot 0.1}{8.54 \times 10^{-4}} = 1.49 \times 10^5.$$

Turbulent, so Churchill (Eq. 380) returns $f = 0.01929$. The velocity head is $\rho u^2/2 = 927.3\,\text{Pa}$, and the three terms are

$$\Delta P_{\text{fric}} = 0.01929 \cdot \tfrac{50}{0.1} \cdot 927.3 = 8\,943\,\text{Pa}, \quad \Delta P_{\text{fit}} = 11.8 \cdot 927.3 = 10\,943\,\text{Pa},$$

$$\Delta P_{\text{elev}} = 875.5 \cdot 9.807 \cdot 2 = 17\,172\,\text{Pa}, \quad \Rightarrow \quad \Delta P = 37\,058\,\text{Pa} = 0.371\,\text{bar},$$

so $P_{\text{out}} = 4.63\,\text{bar}$ and $h_f = (8943 + 10943)/(875.5 \cdot 9.807) = 2.32\,\text{m}$. Note the elevation term *dominates* here — a useful reminder that for a short line the static head, not the friction, often sets the pressure budget. Switching the model to `Haaland` or `Colebrook` changes f (and hence ΔP) by under 2 %.

Runnable case. `tutorials/steady/hydraulics/pipe01_water_line` — a single commercial-steel water line; ΔP is the RESULT of geometry + flow, split into its friction, fittings and elevation parts (swap `model Churchill` for `Haaland` / `Colebrook` and watch f barely move).

22.5 Gas-liquid two-phase flow: Lockhart–Martinelli

When the feed flashes two-phase at (T, P) — the vapour fraction $0 < V/F < 1$ the flowsheet already resolves (§2.4) — the pipe switches AUTOMATICALLY from the single-phase law above to a gas-liquid model. Two-phase friction is dramatically higher than either phase alone: the phases jostle, the slower one is held back, and the interface adds drag. **Layer 1** offers two σ-free models (they need only the phase densities and viscosities the package already gives).

Homogeneous (no-slip). Treat the mixture as one pseudo-fluid with a quality-weighted density and a McAdams viscosity,

$$\frac{1}{\rho_H} = \frac{x}{\rho_G} + \frac{1-x}{\rho_L}, \qquad \frac{1}{\mu_H} = \frac{x}{\mu_G} + \frac{1-x}{\mu_L}, \tag{382}$$

(x the mass quality) and apply the single-phase Darcy law to it. Simple, and adequate for finely-dispersed bubbly/mist flow; it assumes the phases travel at the *same* velocity (no slip), so it under-predicts the liquid holdup.

Lockhart–Martinelli (the default). Compute the pressure gradient each phase *would* have flowing alone in the pipe at its superficial velocity $j_k = \dot{m}_k/(\rho_k A)$, $(\mathrm{d}P/\mathrm{d}z)_k = f_k\,\rho_k j_k^2/(2D)$, and form the **Martinelli parameter**

$$X^2 = \frac{(\mathrm{d}P/\mathrm{d}z)_L}{(\mathrm{d}P/\mathrm{d}z)_G}. \tag{383}$$

The two-phase gradient is the liquid-alone gradient times a **multiplier** (Chisholm's algebraic form of the original Lockhart–Martinelli chart):

$$\left(\frac{\mathrm{d}P}{\mathrm{d}z}\right)_{2\phi} = \phi_L^2 \left(\frac{\mathrm{d}P}{\mathrm{d}z}\right)_L, \qquad \phi_L^2 = 1 + \frac{C}{X} + \frac{1}{X^2}, \tag{384}$$

with C set by which phase is turbulent (each phase laminar if its alone-Re < 1000): $C = 20$ (both turbulent, *tt*), 12 (*vt*), 10 (*tv*), 5 (both viscous, *vv*). The liquid holdup uses Butterworth's [15] fit of the L–M void fraction, $\alpha = (1 + 0.28\,X^{0.71})^{-1}$, $\varepsilon_L = 1 - \alpha$, and the elevation term uses the holdup mixture density $\rho_{\text{mix}} = \varepsilon_L \rho_L + \alpha \rho_G$.

Intuition. For the air-water line of `pipe02_airwater_twophase` ($j_G \approx 4.3$, $j_L \approx 0.58\,\text{m/s}$, both phases turbulent), $X = 2.67$ and $\phi_L^2 = 8.6$ — the two-phase ΔP is **8.6 times** what the liquid alone would lose. And the holdup is $\varepsilon_L = 0.36$ against a no-slip $j_L/(j_L + j_G) = 0.12$: the slow heavy liquid is *held up* (it occupies three times the pipe fraction its flow rate suggests), because the light gas races past it. Switch to `homogeneous` and the holdup drops to the no-slip 0.12 — a one-line way to see why slip matters.

Layer 2: surface-tension models. Two further models add the interfacial physics (they need σ, supplied by `transport { surfaceTension { model BrockBird; } }`). **Friedel** [16] gives a general two-phase multiplier ϕ_{LO}^2 on the total mass flux flowing as liquid, built from density and viscosity ratios and the Froude and Weber numbers, $\phi_{LO}^2 = E + 3.24\,F\,H/(\text{Fr}^{0.045}\,\text{We}^{0.035})$ — the recommended general correlation. **Beggs–Brill** [17] goes further: from the no-slip holdup λ_L and the Froude number it classifies the *flow pattern* (segregated / intermittent / distributed), reads the holdup from a pattern-specific $H_L(0) = a\,\lambda_L^b/\text{Fr}^c$ (clamped $\geq \lambda_L$), corrects it for pipe *inclination* (the Payne et al. ψ factor; horizontal $\Rightarrow \psi = 1$) and scales the no-slip friction factor by a two-phase ratio e^S — so it is the one for inclined or hilly lines. On the `pipe02` air-water case the four models span $\Delta P = 6.0$–$9.0\,\text{kPa}$: that ∼50 % scatter between two-phase ΔP correlations is itself the lesson — they are engineering fits, not laws.

Scope and trust. The correlations are general ($\rho_L, \rho_G, \mu_L, \mu_G, \sigma$), but Lockhart & Martinelli [14] developed and validated them on *isothermal air-water* pipe data, so air-water is exactly where they are trusted. All four are **adiabatic** (the inlet V/F is held along the line — no flashing/condensing in the pipe). The liquid density still comes from the Rackett branch, ∼12 % low for water — it biases the *absolute* ΔP, not the model structure.

Runnable case. `tutorials/steady/hydraulics/pipe02_airwater_twophase` — air (N_2) + water; the two-phase path activates on its own, prints X, ϕ_L^2, the holdup and the regime, and lets you compare `LockhartMartinelli` against `homogeneous`.

23 Pneumatic conveying: carrying solids in a gas

What you'll learn. The pipe chapter above moves a single fluid. Now suspend SOLIDS in the gas and blow them down the line. The pressure drop is no longer a wall-friction story: most of it goes into the SOLIDS — accelerating them, dragging them against slip, lifting them, and (the punchline) re-accelerating them after every bend. This section derives the dilute-phase model Choupo uses: the additive pressure budget, the particle velocity from a force balance, the Yang solids-friction correlation, the saltation floor that decides whether the line PLUGS, and why a fitting can cost more than a hundred metres of pipe. Worked in `tutorials/steady/solids/pneumaticConveyor01_sand` and `pneumaticConveyor02_bends`.

23.1 The additive pressure budget

In developed dilute-phase flow the total drop is a SUM of independent momentum sinks — gas and solids treated separately, each contributing acceleration, friction and static (gravity) heads:

$$\Delta P = \underbrace{\tfrac{1}{2}\rho_g u_g^2}_{\text{gas accel}} + \underbrace{\rho_s^* u_p^2}_{\text{solids accel}} + \underbrace{2 f_g \rho_g u_g^2 \tfrac{L}{D}}_{\text{gas friction}} + \underbrace{f_p \tfrac{\rho_s^* u_p^2 L}{2D}}_{\text{solids friction}} + \underbrace{(\rho_g + \rho_s^*) g\, \Delta z}_{\text{static}} + \sum_{\text{bends}} \Delta P_{\text{bend}}. \tag{385}$$

Here u_g is the superficial gas velocity, u_p the particle velocity, and $\rho_s^* = \dot{m}_s/(u_p A)$ the *suspension density* — the mass of solids per unit pipe volume. The whole point of conveying is the solids terms: in a real line they dominate. Choupo PRINTS this breakdown line by line, so you SEE where the pressure goes.

23.2 Particle velocity: a force balance, not a fit

The particles do not travel at the gas velocity — they SLIP. In developed flow a particle no longer accelerates, so the gas DRAG balances the resistances it feels: the Coulomb friction against the pipe wall plus the gravity component along the pipe,

$$\tfrac{1}{2}\rho_g C_D \tfrac{\pi}{4} d_p^2 \,(u_g - u_p)^2 \;=\; \tfrac{\pi}{6} d_p^3 \rho_p \, g\,(\sin\theta + \mu_w \cos\theta), \tag{386}$$

solved for the slip $u_g - u_p$ (a scalar root-find; C_D depends on the slip Reynolds number). Two limits make it physical: for a VERTICAL line ($\theta = 90°$) the balance is drag = gravity, so the slip equals the terminal velocity u_t; for a HORIZONTAL line ($\theta = 0$) it is drag = wall friction, so the slip is $u_t\sqrt{\mu_w}$. This REPLACES a fitted slip correlation — there is no black formula and no fallback; μ_w, the particle–wall friction coefficient, is declared and announced.

23.3 Solids friction: Yang (1974)

The solids friction factor is the hard part — particles differ in size, shape and charge, and there is no universal Reynolds number. Choupo uses the correlation of Yang (1974), which ties f_p to the voidage ε ($1-\varepsilon = \rho_s^*/\rho_p$, the solids holdup) and the terminal-to- particle Reynolds ratio $(Re)_t/(Re)_p = u_t/u_p$:

$$\text{vertical:}\quad f_p \frac{\varepsilon^3}{1-\varepsilon} = 0.0206 \left[(1-\varepsilon)\frac{(Re)_t}{(Re)_p}\right]^{-0.869}, \tag{387}$$

$$\text{horizontal:}\quad f_p \frac{\varepsilon^3}{1-\varepsilon} = 0.117 \left[(1-\varepsilon)\frac{(Re)_t}{(Re)_p}\frac{u_g}{\sqrt{gD}}\right]^{-1.15}. \tag{388}$$

The correlations are validated for voidage 0.994–0.999; outside that window (a more dilute line) f_p is an EXTRAPOLATION and Choupo says so aloud. The horizontal form carries a pipe-diameter (Froude) dependence the vertical one does not, because in a horizontal pipe gravity segregates the particles radially.

23.4 The saltation floor — will the line plug?

Drop the gas velocity too far and the particles fall out of suspension and settle: the line SALTATES and eventually blocks. The saltation velocity u_{salt} (Rizk, 1973) is the floor you must stay above, and the single most important safety number in a conveying design is the MARGIN

$$\frac{u_g}{u_{\text{salt}}} \geq 1.5\text{–}2, \tag{389}$$

which Choupo reports as a headline KPI and flags in red when it approaches 1. Under-velocity plugging — not the pressure drop — is the number-one real failure of a conveying line; a beautiful ΔP on a line that will plug is worthless.

23.5 Bends: the re-acceleration that dominates

A straight pipe accelerates the solids ONCE, at the inlet. A bend undoes it: the particles crash the outer wall, decelerate to $u_{p,\mathrm{bend}} = r\,u_p$, and the gas must RE-ACCELERATE them over the straight run downstream — the same momentum work as the line entry, once per bend,

$$\Delta P_{\mathrm{bend}} = \frac{\dot{m}_s\,(u_p - u_{p,\mathrm{bend}})}{A} = \frac{\dot{m}_s\,u_p\,(1-r)}{A}. \tag{390}$$

The exit ratio r is set by the bend TYPE: a long-radius sweep keeps most of the velocity ($r \approx 0.75$), a blind (dead-leg) tee nearly stops the solids ($r \approx 0.25$) — three times the pressure cost, but almost no erosion, because the solids cushion on a stagnant pocket of their own kind rather than scouring the wall. In a bendy line the bends carry the majority of ΔP even though they add no length: *a fitting, not the pipe, sets the pressure budget.* (Choupo's term is the re-acceleration MOMENTUM, a mechanistic lower bound; the extra friction in the re-acceleration zone is not separately counted.)

23.6 Scope: dilute phase

This whole model is DILUTE phase — particles suspended and carried, loading up to ∼15 kg solids per kg gas. DENSE phase (plugs and slugs moving as a bed) is a different momentum regime with no honest general correlation; Choupo refuses to fake it and warns when the loading leaves the dilute range.

24 Gas-solid separation: cyclone, bag filter, ideal splitter

What you'll learn. Removing dust from a gas is a *size-classifying* operation, not a flow split: coarse particles are captured, fines escape, and the outlet streams carry different particle-size distributions than the feed. Choupo ships three models spanning the cost/accuracy spectrum — a spec'd ideal splitter, a cheap-physics cyclone, and a high-physics bag filter. This chapter gives the grade-efficiency curve that unifies them and the distinct physics of each.

24.1 Grade efficiency: the unifying idea

Every gas-solid separator is described by its *grade-efficiency* curve $\eta(d)$ — the fraction of particles of size d that are captured. Apply it bin-by-bin to the feed PSD and you get the captured and escaping streams, each with its own (different) distribution. This is why a separator needs a PSD, not just a total solids flow: it does not divide the feed proportionally, it *re-sorts* it by size.

24.2 Cyclone: cheap centrifugal classification

A cyclone spins the gas; inertia throws particles to the wall, where they fall out, while the cleaned gas leaves up the core. The characteristic number is the *cut diameter* d_{50} — the size captured with 50 % efficiency. The Lapple correlation gives

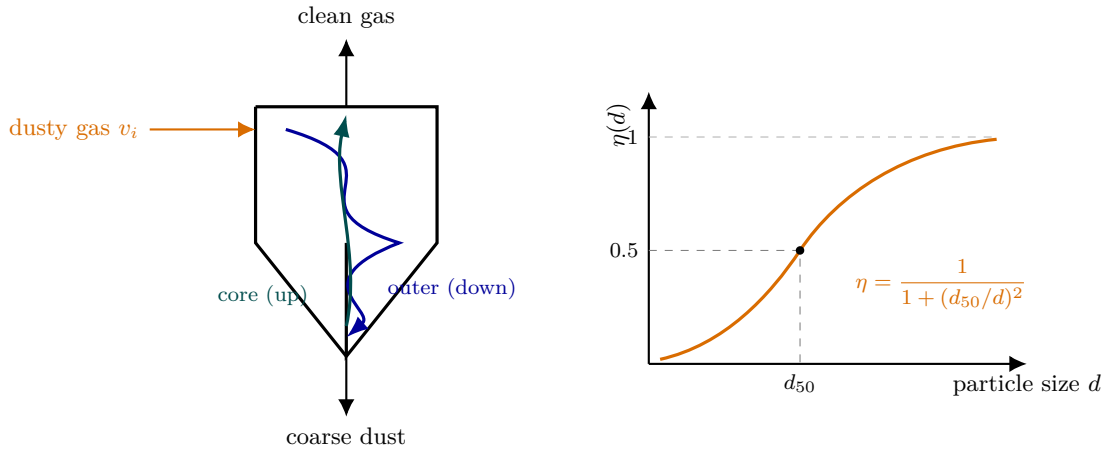

Figure 73: The cyclone classifies by inertia. Left: dusty gas enters tangentially and spins; the outer vortex carries particles down to the wall (blue), inertia throwing the coarse ones out, while the cleaned core swirls back up and leaves the top (teal). Right: the result is a *grade-efficiency* curve $\eta(d)$ — the fraction of size d captured. The characteristic number is the **cut diameter** d_{50}, the size caught with 50 % efficiency (Lapple's $d_{50} = \sqrt{9\mu b/(2\pi N_e v_i \Delta\rho)}$). A separator does not divide the feed proportionally — it *re-sorts* it by size, so the two outlets carry different PSDs. This same curve, with a different shape, describes the bag filter and the ideal splitter.

$$d_{50} = \sqrt{\frac{9\,\mu\,b}{2\pi N_e\, v_i\,(\rho_p - \rho_g)}}, \tag{391}$$

(μ the gas viscosity from the Chung model of Chapter 2, b the inlet width, N_e the effective turns, v_i the inlet velocity, ρ_p the particle density), and the grade-efficiency curve

$$\eta(d) = \frac{1}{1 + (d_{50}/d)^2}. \tag{392}$$

The collection efficiency follows by integrating $\eta(d)$ over the feed PSD. The pressure drop is a few velocity heads, $\Delta P = N_H \rho_g v_i^2/2$ (Shepherd–Lapple).

The cut is a *selectable sub-model* (the same explicit-factory pattern as the EoS and activity models — "all models are wrong, some useful"), chosen with `model <name>;`. There are **FIVE**, spanning the three classical approaches to the centrifugal/drag balance, and on one real cyclone they can disagree by ∼3× in d_{50} (`cyclone05_dirgo_leith_validation`) — which is why you validate against data:

- **Lapple** (1951) — *timed flight*: the residence-time d_{50} of Eq. (393)'s ancestor above, $d_{50} = \sqrt{9\mu b/(2\pi N_e v_i \rho_p)}$, with the S-curve $\eta = 1/(1 + (d_{50}/d)^2)$. The simple default.
- **Leith–Licht** (1972) — *back-mixing*: turbulence re-mixes uncollected dust across each plane, giving a closed-form efficiency $\eta(d) = 1 - \exp\big[-2\,(C\psi)^{1/(2n+2)}\big]$, where $\psi = \rho_p d^2 Q/(\dots)$ is the inertia parameter, C a geometric constant, and $n = 1 - (1 - 0.67\,D^{0.14})(T/283)^{0.3}$ the vortex exponent (weaker spin in a hotter, wider cyclone).
- **Iozia–Leith** (1989) — *flow pattern*: d_{50} from the core velocity, Eq. (393) (derived just below).
- **Barth** (1956) — *equilibrium orbit*: the critical particle orbits the vortex-finder edge where its outward drift balances the inward gas, $d_{50} = \sqrt{18\mu\, v_r r_{cs}/(\rho_p v_\theta^2)}$, with $v_r = Q/(\pi D_{cs} H_{cs})$ the radial and $v_\theta = \alpha v_i$ the tangential velocity on the control surface.
- **Muschelknautz** (MMD) — the gold standard, because it adds the *solids-loading effect* the other four miss: above a limit loading $c_{\lim}$ the surplus dust is mass-separated at the wall near 100 %, $\eta = \eta_L + (1 - \eta_L)\eta_I$ with $\eta_L = 1 - c_{\lim}/c$, so a heavily-loaded cyclone is far more efficient than Lapple predicts (`cyclone04_high_loading`: 98 % vs 82 %).

24.2.1 The Iozia–Leith flow-pattern cut

Lapple's d_{50} is a residence-time *surrogate*; Iozia & Leith (1989) instead read it off the gas FLOW PATTERN. The critical particle hangs at the core radius, where the tangential velocity peaks at $V_{t,\max}$; balancing its centrifugal throw against the inward drag of the gas draining into the vortex gives

$$d_{50} = \sqrt{\frac{9\,\mu\,Q}{\pi\, z_c\, \rho_p\, V_{t,\max}^2}}, \tag{393}$$

where z_c is the core length and Q the gas flow. The two flow-pattern quantities are regressed from the cyclone's OWN eight dimensions:

$$V_{t,\max} = 0.61\, V_i\, (ab/D^2)^{-0.61} (D_e/D)^{-0.74} (H/D)^{-0.33}, \tag{394}$$

$$d_c = 0.52\, D\, (ab/D^2)^{-0.25} (D_e/D)^{1.53}, \tag{395}$$

with $V_i = Q/(ab)$ the inlet velocity, a, b the inlet height and width, D_e the gas-outlet diameter, H the total height, and $z_c = H - S$ (or, if the core d_c exceeds the dust outlet B, the shorter distance where it meets the cone). Because it uses all eight dimensions it captures cut changes with geometry that Lapple's single d_{50} cannot. (Source note: the printed 1989 prefactor on $V_{t,\max}$ is 6.1, which yields a supersonic $V_{t,\max} \sim 400$ m/s and a ten-fold-too-fine cut — contradicting the paper's own Figure 4, where $V_{t,\max} \le 40$ m/s. The factor of exactly ten is a misplaced decimal; Choupo uses the Figure-4-consistent 0.61.) Validate it against the measured fractional-efficiency curves of Dirgo & Leith (1985) in `cyclone05_dirgo_leith_validation` — where the five sub-models spread over ∼3× in d_{50} on one real cyclone, the reason data, not a single correlation, has the last word.

24.3 Bag filter: the cake is the filter

A fabric filter is almost total: the dust cake that builds on the cloth does the filtering, so the grade efficiency is high and only mildly size-dependent,

$$\eta(d) = 1 - P_0\, e^{-d/d_c}, \tag{396}$$

(fines penetrate most). Here the headline result is not efficiency but the cake-dominated *pressure drop* — Darcy through clean cloth plus cake:

$$\Delta P = \mu\, V\, (K_1 + K_2 W), \tag{397}$$

with $V = Q/A$ the face velocity (the air-to-cloth ratio), W the areal dust load, K_1 the clean-cloth and K_2 the cake resistance. Same dust as the cyclone, the bag filter captures **99.93 %** (vs ∼82 %) at $\Delta P \approx 2.7$ kPa — which is why the classic train is cyclone (cheap pre-cleaner) then bag filter (polish).

24.4 Ideal splitter: when you just want a number

Sometimes a physics model is overkill — "assume 90 % capture". The `gasSolidSplitter` applies the split the user *states* (`solidsRecovery`, `gasCarryover`); the feed PSD passes *unchanged* into both outlets (a spec'd split does not classify by size — the very distinction from a cyclone). Use it as a placeholder, a bound, or when the physics is not worth the trouble.

Key idea. The three separators span a deliberate spectrum on the same PSD-aware interface: spec'd (`gasSolidSplitter`, you state the number), cheap-physics (`cyclone`, ∼82 % from a correlation), high-physics (`bagFilter`, ∼99.9 % from the cake). Pick the fidelity the problem deserves — the Pareto principle made into a menu.

Runnable case. `tutorials/steady/gas-solid/cyclone01_dust_removal` — Lapple, $d_{50} = 2.7\,\mu$m, 82 % collection (and cyclone02–04 for the other models + loading).
`tutorials/steady/gas-solid/bagFilter01_dust` — the same dust, 99.93 % + the cake ΔP.
`tutorials/steady/gas-solid/gasSolidSplitter01_ideal` — the spec'd split.

25 Drying: spray dryer and solid dryer

What you'll learn. Drying couples four physics at once — a mass/energy balance, droplet atomisation, the wet-bulb temperature, and the drying-rate kinetics — and ends at a residual moisture set by the air, not by wishful thinking. This chapter walks the spray dryer (atomise a liquid into a powder) and the solid dryer (finish an already-solid powder), and shows the recurring split the simulator enforces: the sorption *equilibrium* (thermodynamics, on the material's `.dat`) is kept separate from the drying *curve* (kinetics, in `constant/dryingKinetics`).

25.1 The four coupled pieces of a spray dryer

Mass and energy balance. Hot air meets atomised feed; the

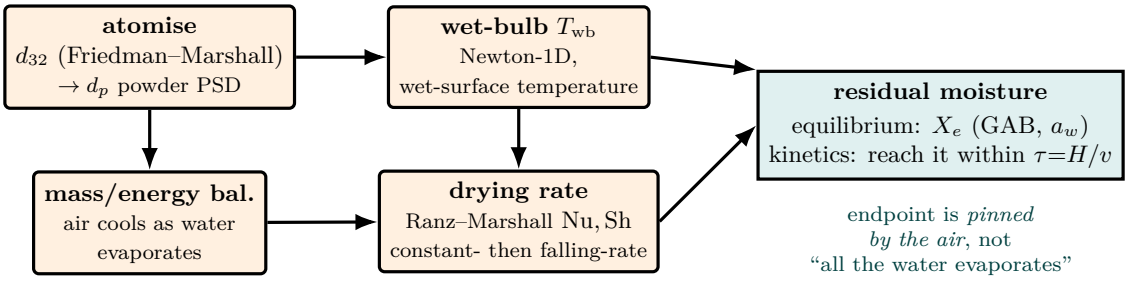

Figure 74: A spray dryer is four physics solved at once, ending at a moisture the air sets. **Atomisation** (a Friedman–Marshall d_{32} shrinking to the powder d_p) gives the particle PSD; the **mass/energy balance** ties the air's cooling to the water evaporated; the **wet-bulb** temperature (a Newton-1D) is the wet-surface temperature during constant-rate drying; and the **Ranz–Marshall** transfer coefficients set the drying-rate curve. They converge on the *residual moisture* — and here is the lesson: the endpoint is *not* "everything evaporates". The GAB sorption isotherm pins the equilibrium moisture X_e to the air's humidity (thermodynamics, kept on the `.dat`); the falling-rate curve says whether the powder reaches X_e within the residence time $\tau = H/v$ (kinetics, kept separate). Mixing the two is a category error the simulator refuses.

solvent evaporates and the air cools. An adiabatic balance ties the air outlet temperature to the water evaporated and the powder production.

Atomisation. A rotary wheel shears the liquid into droplets whose Sauter mean diameter follows a Friedman–Marshall correlation in the wheel speed, diameter and liquid properties; the droplet then shrinks to the powder particle by the solid fraction,

$$d_p = d_{32}\,(x_{\mathrm{solid}}\,\rho_L/\rho_{\mathrm{solid}})^{1/3}, \tag{398}$$

so the powder carries a PSD.

Wet-bulb temperature. While the surface is wet the droplet sits at the psychrometric wet-bulb temperature (a Lewis-number-≈ 1 balance of convective heat in against evaporative cooling out), found by a Newton-1D — the constant-rate surface temperature.

Drying-rate kinetics. The droplet's Reynolds number (from its terminal velocity) feeds the Ranz–Marshall correlation $\mathrm{Nu} = \mathrm{Sh} = 2 + 0.6\,\mathrm{Re}^{1/2}\mathrm{Pr}^{1/3}$, giving the heat- and mass-transfer coefficients h, k_c and hence the constant-rate drying time.

25.2 The residual moisture: equilibrium vs kinetics

The "all the water evaporates" assumption is wrong: a powder equilibrates with the air's humidity at a finite moisture. Two distinct things set the endpoint, and the simulator keeps them in separate files — keeping the sorption isotherm separate from the characteristic drying curve.

Equilibrium (on the component `.dat`, axiom 1): the GAB sorption isotherm gives the equilibrium moisture at the air's water activity a_w (= the relative humidity),

$$X_e = \frac{X_m\,C\,K\,a_w}{(1-Ka_w)(1-Ka_w+CKa_w)}. \tag{399}$$

Kinetics (in `constant/dryingKinetics`, axiom 3): below the critical moisture X_c the rate falls; the Lewis falling-rate model $dX/dt = -k(X - X_e)$, with k tied to the constant rate by continuity at X_c (no new parameter), carries X from X_c toward X_e. The final moisture is the drying curve evaluated at the *residence time*, itself a RESULT of the chamber geometry, $\tau = H/v_{\text{particle}}$ (credo: the chamber diameter and height are the hardware, residence is computed). The powder leaves carrying that residual water as bound liquid *alongside* the dry solid, so the mass balance closes.

> *Intuition.* Whether a powder comes out at 1 % or 5 % moisture is not a free choice — it is pinned by the air's humidity through the sorption isotherm, and reached (or not) within the residence time through the drying curve. Mixing the critical moisture X_c (a kinetics number) into the isotherm (thermodynamics) is a category error; the simulator refuses to, and so should the student.

25.3 The solid dryer: finishing an already-solid powder

A contact / through-circulation `solidDryer` takes an *already-solid* powder (crystals plus bound moisture) and polishes it down to the equilibrium moisture set by the air, $X_{\text{final}} = \min(X_{\text{in}}, X_e(\text{RH}))$, with the water removed and the duty as RESULTS. Hardware-only: air temperature and relative humidity. It complements the spray dryer (atomise a liquid → powder); the solid dryer finishes a solid — the two ends of a drying train.

Worked example 25.1. Sugar spray dryer, `sprayDryer01_sugar`A 40 wt% sugar solution dried by air at 200 °C, 0.1 m disc at 20000 rpm: $d_{32} \approx 58\,\mu$m, particle $\approx 37\,\mu$m, $T_{\text{wb}} \approx 45\,^\circ$C, $T_{\text{out}} \approx 98\,^\circ$C, Stokes regime; with the GAB isotherm + a ∼7 s residence the powder reaches $X_e \approx 0.014$ (1.3 wt% — a realistic sugar-powder moisture), and the mass balance closes 100 %.

Runnable case. `tutorials/steady/drying/sprayDryer01_sugar` — atomise → wet-bulb → constant-rate → falling-rate → equilibrium.
`tutorials/steady/drying/sprayDryer02_residence_sweep` — the chamber-height sweep, the drying curve appearing as the chamber grows.
`tutorials/steady/drying/solidDryer01_sugar` — finishing a solid to the air-set equilibrium moisture.

26 Two-stream heat exchanger: the ε-NTU rating method

What you'll learn. The single-stream `heater` (next section) takes a furnished duty Q; a two-stream `heatExchanger` takes only *hardware* — area A and overall coefficient U — and the duty, the outlet temperatures and the LMTD all emerge. That is the *rating* problem, and the effectiveness-NTU method solves it without iteration (unlike LMTD, which would need it when the outlets are unknown). The split between the two units is by *number of process streams* (1 vs 2), not by folklore.

With heat-capacity rates $C = \dot{n}\,C_p$ for each stream, $C_{\min}$, $C_{\max}$ and their ratio $C_r = C_{\min}/C_{\max}$, the number of transfer units and the effectiveness are

$$\mathrm{NTU} = \frac{UA}{C_{\min}}, \qquad \varepsilon_{\text{counter}} = \frac{1 - e^{-\mathrm{NTU}(1-C_r)}}{1 - C_r\, e^{-\mathrm{NTU}(1-C_r)}} \tag{400}$$

(and the co-current form $\varepsilon = [1 - e^{-\mathrm{NTU}(1+C_r)}]/(1 + C_r)$). The duty follows directly from the maximum possible temperature approach,

$$Q = \varepsilon\, C_{\min}\,(T_{h,\mathrm{in}} - T_{c,\mathrm{in}}), \tag{401}$$

and the outlet temperatures from each stream's energy balance. The LMTD is computed *a posteriori* and the identity $UA\,\mathrm{LMTD} = Q$ is a clean self-check. Tutorial `heatExchanger01_water_water`: $A = 10\,\mathrm{m}^2$, $U = 500\,\mathrm{W/m^2K}$ counter-current cool $360 \to 315\,\mathrm{K}$ against $300 \to 337\,\mathrm{K}$, $\varepsilon = 0.745$, $Q = 93.8\,\mathrm{kW}$.

27 Heat exchanger DESIGN: geometry, pressure drop, and the sizing workflow

What you'll learn. The ε–NTU section above RATES an exchanger whose U and area you already know. But where do U and the area come from? This section opens that box: the overall coefficient BUILT from the two film coefficients and the wall, the pressure drop both sides, the shell diameter implied by a tube count, and — the payoff — the DESIGN loop that inverts the problem: given a duty, SIZE the exchanger. This is the workflow a real project follows, and Choupo does it glass-box: every Nu, every resistance, every friction factor is printed and cited. Worked end-to-end in the two-part sequence `hxWorkflow1_design_from_duty` (size it) → `hxWorkflow2_rate_designed` (rate it).

27.1 Two questions: rating vs design

A shell-and-tube exchanger poses two inverse questions:

- **Rating** (`model geometry;`): the geometry is GIVEN; find U, the area, the pressure drop, and hence the duty. "I have this exchanger — what will it do?"
- **Design** (`model design;`): the DUTY is given; find a geometry that delivers it within a pressure budget. "I need this duty — size me an exchanger."

Design is built ON rating: you cannot size without being able to rate a trial.

27.2 The overall coefficient, built

Heat crosses three resistances in series — the tube-side film, the tube wall, the shell-side film — referred here to the OUTSIDE area $A_o = \pi\, d_o\, L\, N$ (the convention that makes $U_o A_o$ the group in $Q = U_o A_o\, \Delta T_{\mathrm{lm}}$):

$$\frac{1}{U_o} = \underbrace{\frac{1}{h_o}}_{\text{shell film}} + \underbrace{\frac{r_o \ln(r_o/r_i)}{k_{\mathrm{wall}}}}_{\text{cylindrical wall}} + \underbrace{\frac{r_o}{r_i\, h_i}}_{\text{tube film}} . \tag{402}$$

The engine prints the three terms and names the largest the *controlling resistance* — the one worth attacking (a better shell-side baffle cut buys nothing if the tube film controls).

Tube-side film (Gnielinski). For turbulent in-tube flow,

$$Nu = \frac{(f/8)(Re - 1000)\, Pr}{1 + 12.7\sqrt{f/8}\,(Pr^{2/3} - 1)}, \qquad f = (0.790 \ln Re - 1.64)^{-2}, \tag{403}$$

(Gnielinski; Incropera & DeWitt) with $h_i = Nu\, k/d_i$, evaluated at the stream's mean temperature with real (IF97 for water) ρ, μ, k, c_p.

Shell-side film (Kern). The shell side is a baffled crossflow; the Kern method lumps it into a bundle correlation on an EQUIVALENT diameter D_e and the crossflow mass flux G_s:

$$Nu = 0.36\, Re_s^{0.55}\, Pr^{1/3}, \quad Re_s = \frac{G_s D_e}{\mu}, \quad G_s = \frac{\dot m_s}{A_s}, \quad A_s = \frac{D_s\,(p - d_o)\,B}{p}, \tag{404}$$

with A_s the crossflow area at the shell centreline, B the baffle spacing, p the tube pitch, and $D_e = 4(p^2 - \pi d_o^2/4)/(\pi d_o)$ (square pitch). Kern is a ±25% method — honest for teaching and a first design; Bell–Delaware (leakage/bypass corrections) is the refinement.

27.3 Pressure drop: the other half of design

An exchanger is not sized for area alone — it lives within a PUMPING budget. The tube side (friction over all n_p passes plus the turnaround losses):

$$\Delta P_t = \left(4 f\, \frac{L\, n_p}{d_i} + 4 n_p\right) \frac{\rho u_t^2}{2}, \qquad f = 0.079\, Re^{-1/4} \text{ (Blasius)}. \tag{405}$$

The shell side (Kern), over the N_b baffles:

$$\Delta P_s = f_s\, \frac{G_s^2\, D_s\, (N_b + 1)}{2\, \rho\, D_e}, \qquad f_s = \exp(0.576 - 0.19 \ln Re_s). \tag{406}$$

The shell drop typically EXCEEDS the tube drop — baffled crossflow dissipates more — and the duty-vs-pumping trade-off is precisely the engineering the map of U against ΔP makes visible.

27.4 The design loop

Design inverts rating. Fix the tube choices (diameter, length, pitch, passes, material); everything else follows from ONE unknown, the tube count N:

1. From the duty target (an outlet T) and an energy balance, get Q_{req}, both outlet temperatures, and ΔT_{lm}; hence the required $UA_{\mathrm{req}} = Q_{\mathrm{req}}/\Delta T_{\mathrm{lm}}$.
2. The shell diameter follows from N by the Kern/Sinnott bundle correlation $D_b = d_o(N/K_1)^{1/n_1}$ + clearance (triangular pitch, one pass: $K_1 = 0.319$, $n_1 = 2.142$).
3. $U(N)\,A(N)$ increases monotonically with N (area grows linearly, U falls only slowly as the tube velocity drops), so a BISECTION on N finds the count that meets UA_{req}. Round UP: never undersize.
4. Rate the sized bundle (the same film coefficients) and read the pressure drop — if it busts the budget, fewer passes or a wider baffle spacing, and re-check.

27.5 The design workflow

The cardinal rule: *do not invent the geometry up front.* The standard practice splits the job into three:

1. a **Heater / shortcut** block establishes the DUTY while the flowsheet converges (no geometry);
2. a **detailed design** step sizes the exchanger from that duty (generates the geometry);
3. a **detailed rating** step checks the sized exchanger and probes off-design.

Choupo folds steps 1–2 into `model design;` — it computes the duty from your target, ANNOUNCES it, and sizes in one honest step — then `model geometry;` is the rating (step 3). The two-part tutorial sequence walks exactly this: Part 1 sizes 146 tubes / 0.344 m shell from a 330→315 K target; Part 2 rates that bundle (delivers 315.0 K, a touch over-surfaced from the round-up) and invites the off-design study (turn the coolant down, watch the outlet warm).

27.6 The deliverable: the datasheet

A design ends in a *datasheet* — the TEMA specification sheet an engineer signs off. Choupo's GUI builds one (the unit panel → "Open datasheet"): a formatted sheet with the service, the thermal and hydraulic results, the geometry, and a SCHEME of the shell-and-tube drawn from the solved numbers (the tube count, the baffles, the passes, the nozzle temperatures), printable to PDF; the raw numbers also export to a spreadsheet. Open, cited, and reproducible — the black box a commercial tool hands you, opened.

28 Shortcut distillation: Fenske–Underwood–Gilliland

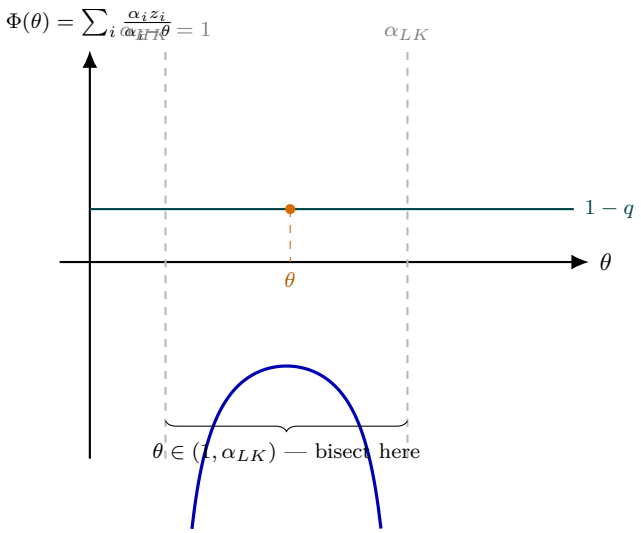

Figure 75: Locating the Underwood root. The function $\Phi(\theta) = \sum_i \alpha_i z_i/(\alpha_i - \theta)$ has a vertical asymptote at *every* component volatility α_i. The physically meaningful root — the one that fixes the minimum reflux — is the value of θ that satisfies $\Phi(\theta) = 1 - q$ on the single branch trapped *between* the two key volatilities, $\theta \in (1, \alpha_{LK})$. On that interval Φ runs monotonically from $-\infty$ to $+\infty$, so it crosses the $1 - q$ level exactly once and a simple bisection brackets it with no risk of jumping to a neighbouring branch. Feeding that θ into $R_{\min} + 1 = \sum_i \alpha_i x_{D,i}/(\alpha_i - \theta)$ then returns the minimum reflux that Gilliland needs.

> *What you'll learn.* Before the rigorous Wang–Henke or MESH column (which need a built column), the FUG shortcut sizes a column from the two key components, their recoveries and the reflux — $N_{\min}$, $R_{\min}$, the actual N and the feed stage all in closed form, at constant relative volatility. It is the textbook preliminary design, and the companion to the rigorous methods: FUG designs, Wang–Henke verifies.

Three classical correlations, in sequence:

- **Fenske** — minimum stages at total reflux and the non-key split: $N_{\min} = \ln\!\big[(x_{LK}/x_{HK})_D\,(x_{HK}/x_{LK})_B\big] / \ln \alpha_{LK}$.
- **Underwood** — minimum reflux from the root $\theta \in (1, \alpha_{LK})$ of $\sum_i \alpha_i z_i/(\alpha_i - \theta) = 1 - q$ (bisection), then $R_{\min} + 1 = \sum_i \alpha_i x_{D,i}/(\alpha_i - \theta)$.
- **Gilliland** (Molokanov form) — the actual stage count N from $N_{\min}$, the operating R and $R_{\min}$; **Kirkbride** then locates the feed stage.

The relative volatilities are taken at the feed bubble point (Chapter 33 below). Tutorial `shortcut01_benzene_toluene` (50/50, LK/HK 99 %/1 %, $R = 1.3R_{\min}$): $N_{\min} = 10.07$, $R_{\min} = 1.29$, $N = 21.6$, feed stage 10.8. The method assumes constant α — it does *not* handle azeotropes (use the MESH column for those).

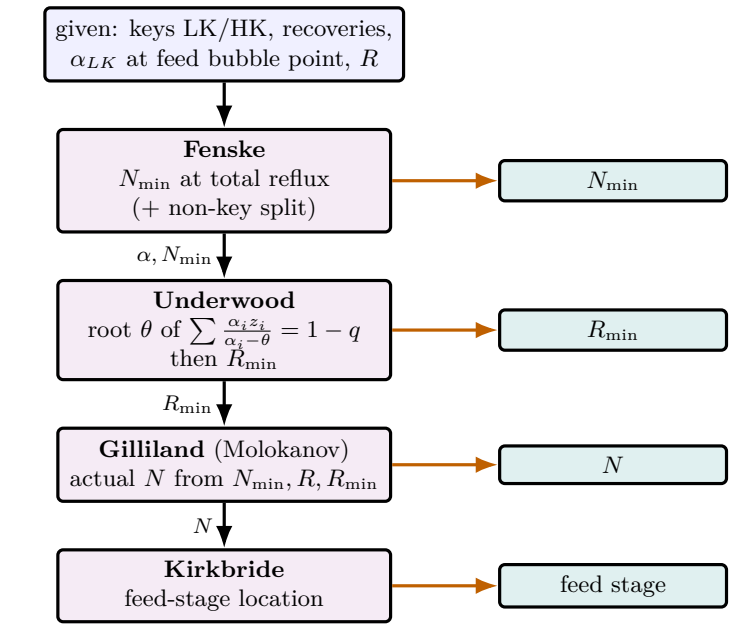

Figure 76: The Fenske–Underwood–Gilliland–Kirkbride shortcut as a one-pass pipeline — the textbook preliminary design, run before any rigorous column exists. **Fenske** gives the minimum stages $N_{\min}$ at total reflux (and the non-key split) from the two key recoveries and the relative volatility. **Underwood** finds the minimum reflux $R_{\min}$ via the root θ of its feed-quality equation. **Gilliland** (Molokanov form) then turns $N_{\min}$, the operating R and $R_{\min}$ into the actual stage count N, and **Kirkbride** locates the feed stage. Each box consumes the previous result; every box also emits a headline number (orange). The whole chain rests on *constant* relative volatility, so it sizes ideal columns in closed form but cannot cross an azeotrope — there the rigorous MESH column takes over. "FUG designs, Wang–Henke verifies."

29 Rigorous distillation by simultaneous MESH Newton

What you'll learn. The *ladder* of column models — bubble-point Wang–Henke, the constant-molal-overflow simultaneous Newton, and the full energy-balance Naphtali–Sandholm MESH — which one to reach for, how the cheap model warm-starts the rigorous one, and how multiple feeds, side draws and a stage reaction are layered on top.

29.1 The model ladder (*all models are wrong, some useful*)

One `distillationColumn` unit carries three solver models, selected by the `model` slot, in increasing order of rigour and cost:

`model`	method	best used when
`WangHenke` *(default)*	sequential bubble-point, CMO	ideal / near-ideal, ONE feed, quick screenir
`simultaneous`	simultaneous Newton, CMO	azeotropic / non-ideal; multi-feed, side drav
`fullMESH` *(= MESH = NaphtaliSandholm)*	full energy-balance MESH	wide-boiling / large ΔH_{vap} spread / when

A naming note: `simultaneous` is the *bubble-point-reduced* solve (constant molal overflow, NO per-stage energy balance); the genuine MESH (Mass, Equilibrium, Summation, *Heat*) is `fullMESH` — aliased `MESH` and `NaphtaliSandholm` so the name never means less than it says.

29.2 Wang–Henke vs the simultaneous CMO Newton

The Wang–Henke bubble-point method of Chapter 13 is robust for ideal systems but converges slowly ($\mathcal{O}(100)$ outer iterations) and can step *through* an azeotrope non-physically. Selecting `model simultaneous` solves all the stage equations at once by Newton (`solver::newtonND`): the variables are the per-stage $(x_{j,0..n-2}, T_j)$, the residuals the $n-1$ component balances plus the per-stage bubble-point $\sum_i K_i x_i - 1 = 0$ that fixes T_j, with $K_j(T_j)$ live. It converges *quadratically* (5–6 iterations vs ∼108) and, crucially, is stable on azeotropic systems: `column03_azeotrope_mesh` (ethanol/water, NRTL) finds a physical distillate *below* the azeotrope on the very case the bubble-point method cannot solve — and refuses to invent a non-physical "past the azeotrope" answer, because none exists.

Both Wang–Henke and `simultaneous` assume **constant molal overflow** (CMO): the internal flows L_j, V_j are fixed by a mass march down the column, and only x_j, T_j are solved. CMO is exact only when all components have equal molar latent heats and sensible/mixing effects are negligible.

29.3 The full MESH (Naphtali–Sandholm): the energy balance

`model fullMESH` drops the CMO assumption. It promotes V_j and L_j to *unknowns* (so the per-stage variable count rises from n to $n+2$: $x_{j,0..n-2}, T_j, V_j, L_j$) and adds, per interior stage, the **total-mass balance** and the **energy balance** — the enthalpy residual the CMO never carried:

$$L_{j-1}\,h_L(x_{j-1},T_{j-1}) + V_{j+1}\,h_V(y_{j+1},T_{j+1}) + F_j\,h_{F,j} \;-\; L_j\,h_L(x_j,T_j) \;-\; V_j\,h_V(y_j,T_j) \;=\; 0. \tag{407}$$

The internal flows now follow the *real* latent heats instead of being assumed equimolal. The end stages carry flow specifications instead of an energy residual: a total condenser fixes $V_1 = 0$ and $L_1 = R\,D$, the reboiler fixes $L_N = B$, and their energy balances yield Q_{cond} and Q_{reb} as *results*. Validated stage-by-stage against the original Naphtali & Sandholm (1971) Problem 2 in `column07_naphtaliSandholm` (T profile within ≤ 1.8 K over 20 stages); `column06_fullMESH` shows it agreeing with CMO on benzene/toluene (close latent heats), the small gap being exactly the energy-balance correction CMO drops.

29.4 The warm-started ladder (switching model mid-solve)

A rigorous Newton from a cold start can fail (the linearization is only valid near the solution). Choupo therefore *switches model mid-solve*: the `fullMESH` run first converges the cheap CMO problem, then hands the CMO's converged stage profile (x_j, T_j, V_j, L_j) to the full MESH as its *initial guess*. The intermediate profile is *kept*, not discarded — the rigorous model then converges in ∼3–4 iterations instead of from scratch. This homotopy/continuation pattern recurs throughout the simulator: non-reactive → reactive (the reaction is switched on, seeded from the non-reactive profile), and the catalyst-mass ramp in kinetic mode. It is the "cheap model bootstraps the rigorous one" principle, and it is why selecting a more rigorous model is cheap once a coarser one has run.

29.5 Multiple feeds and side draws

The `simultaneous` and `fullMESH` models generalise the single fixed feed into a per-stage *flow staircase*. Any stage j may receive a feed (a flowsheet stream mapped by `operation.feeds ({stream; stage;}...)`) and bleed a liquid (U_j) or vapour (W_j) *side draw* (`operation.sideDraws`). Each stage balance then carries an $F_j z_{F,j}$ source and

$(L_j{+}U_j), (V_j{+}W_j)$ outflows; the general form reduces *exactly* to the single-feed equations when there is one feed and no draw, so every legacy tutorial is unchanged. Feeds are FLOWSHEET STREAMS so the plant-boundary balance sees them (*information follows the streams*); a side draw is inherently less pure than an end product (it picks up the components boiling just above and below it). `column08_radfrac_multidraw` splits a C3–C6 mix four ways with two feeds and two side draws.

29.6 Reactive distillation

A `reaction{}` block makes named catalytic stages reactive. For a mole-conserving ($\sum_i \nu_i = 0$) reaction the CMO flows are undisturbed; the generation $\nu_i\,\xi_j$ enters each reactive stage's component balance with the molar extent ξ_j fixed either by chemical `equilibrium` (an activity-based $K_a(T)$ via van't Hoff, residual $\sum_i \nu_i \ln(\gamma_i x_i) = \ln K_a$ — one extra unknown ξ_j and one extra equation per stage) or by `kinetics` (a rate law × catalyst mass; ξ_j then a function of x_j, T_j, no extra unknown). Converged by the same homotopy: the column is solved non-reactive first, then the reaction is switched on from that profile. `column05_reactive_methylacetate` (methyl acetate, validated vs Pöpken 2001) reaches the equilibrium ceiling this way.

30 Tray hydraulics: can the column be built?

Everything up to here answers one question: *what separates?* The MESH equations know the stage temperatures, the compositions, the flows. They do not know the diameter of the column, and they never asked. A column of any diameter converges to the same distillate purity, because nothing in the stage equations depends on how wide the trays are.

That is a real hole. Vapour rising through a tray drags liquid with it, and above a certain velocity it carries the liquid to the tray above instead of letting it fall — *entrainment flooding*. Liquid crossing a tray must then descend through the downcomer against the tray's own pressure drop, and if the head it needs exceeds the space available it backs up to the tray above — *downcomer flooding*. Too little vapour and the liquid rains straight through the holes instead of bubbling — *weeping*. A flooded column separates nothing. It also converges.

So hydraulics is a **rating pass on the converged profile**, run after the stage equations, never inside them. That scope is a decision, not an oversight: a column whose pressure drop shifts its own stage temperatures needs the pressure profile carried back into the MESH, and Choupo does not pretend to do that. What it does do is refuse to let the student believe a stream table that no physical tray could produce.

30.1 First, the tray is not ideal: Murphree efficiency

Every column derived so far is built from *equilibrium stages*: the vapour leaving a tray is in equilibrium with the liquid leaving it. No real tray does that. The vapour bubbles through a froth of finite depth for a finite time and comes off somewhere between what entered it and what equilibrium would give. Murphree measured that shortfall on the vapour side:

$$y_j \;=\; E_{MV}\, y_j^{*} \;+\; (1 - E_{MV})\, y_{j+1}, \qquad y_j^{*} = K_j x_j, \tag{408}$$

so E_{MV} is the *fraction of the way* the vapour travels from what rose into the tray, y_{j+1}, towards equilibrium with the liquid on it. $E_{MV} = 1$ is the ideal stage; $E_{MV} = 0$ is a tray that does nothing.

Two consequences, and the second is the one that matters for the rest of this section. First, purity falls: fourteen real trays do less than fourteen ideal ones, which is why a column designed on ideal stages and built on real trays comes up short. Second, and more important, **once $E_{MV} < 1$ the stage count is a count of REAL TRAYS** — physical objects with a diameter, a weir and holes in them — and it is those the hydraulics below sizes. Without an efficiency we would be dimensioning to the millimetre a tray we then assumed was perfect.

> **Key idea.** E_{MV} is a *declared* number in Choupo, exactly like the orifice coefficient C_o and the weep constant K_2 that follow. It comes from a correlation (O'Connell's, from the liquid viscosity and the relative volatility), a chart, or plant data. The engine will not invent it, and it will not silently assume 1 when you meant something else.

Numerically, Eq. (408) couples tray j to the vapour rising from $j+1$, which is itself an unknown — and that would destroy the tridiagonal structure the bubble-point method depends on. Choupo folds the efficiency into an *effective* K,

$$K_{j,i}^{\mathrm{eff}} \;=\; E_{MV}\, K_{j,i} \;+\; (1 - E_{MV})\, \frac{y_{j+1,i}}{x_{j,i}}, \tag{409}$$

with y_{j+1} taken from the *previous* outer pass. Every equation in the tridiagonal sweep is untouched: $y_j = K_j^{\mathrm{eff}} x_j$ as before. The lag vanishes at convergence, where $y^{\mathrm{old}} = y$, so the converged profile satisfies Eq. (408) exactly. The outer loop pays for the coupling, and it is visible in the iteration count. The simultaneous MESH method has no outer pass to lag against — there the relation would have to enter the residual and the Jacobian — so it *refuses* the keyword rather than quietly handing back ideal stages.

30.2 Entrainment flooding: Souders–Brown and Fair

Suspend a droplet in the rising vapour. It falls under its own buoyant weight and is dragged up by the gas; at the flooding velocity the two balance. Writing the drag with a constant coefficient and clearing the algebra gives the *Souders–Brown* form [5]: the flooding velocity scales as the square root of the density difference over the vapour density,

$$u_{\mathrm{flood}} \;=\; C_{\mathrm{SB}} \left(\frac{\sigma}{20}\right)^{0.2} \sqrt{\frac{\rho_L - \rho_V}{\rho_V}}, \tag{410}$$

referred to the *net area* (the tower cross-section less one downcomer, which is the area the vapour actually has). Surface tension enters because it is what holds a droplet together against the shear that would atomise it; σ is in dyn/cm and the correlation is anchored at 20 dyn/cm.

The capacity parameter C_{SB} is not a constant. It falls as the tray carries more liquid relative to vapour, which is measured by the dimensionless *flow parameter*

$$F_{LV} \;=\; \frac{L}{V}\sqrt{\frac{\rho_V}{\rho_L}}, \qquad \text{(mass flows)} \tag{411}$$

and it rises with the tray spacing, because a droplet thrown off one tray has further to travel before it reaches the next. Fair [6] measured $C_{\mathrm{SB}}(F_{LV}, \mathrm{TS})$ and published it as a *chart*. A chart cannot be integrated into a solver, so Choupo uses the algebraic representation of Lygeros and Magoulas [7],

$$C_{\mathrm{SB}} \;=\; 0.0105 \;+\; 8.127\times10^{-4}\;\mathrm{TS}^{0.755}\;\exp\!\left(-1.463\,F_{LV}^{0.842}\right) \qquad [\mathrm{m/s}], \tag{412}$$

with the tray spacing TS in millimetres. This fit is *known* to misbehave below $F_{LV}\approx 0.03$: its slope turns the wrong way, where the real chart rises. Choupo says so out loud when a tray lands there rather than reporting a number it does not believe. This is the anti-black-box stance in its plainest form — a correlation is a fit to somebody's data, and when you leave the data behind, you are told.

Key idea. Kister and Haas (1990) published a physically grounded, fully algebraic entrainment-flood correlation, built on the clear-liquid height at the froth-to-spray transition. It is the better model, and it is *not* implemented: two of its constants could not be verified against a primary source. A teaching tool does not ship numbers it has not read. What you get instead is Fair's chart, an honest fit to it, and a warning where the fit is known to fail.

30.3 Pressure drop, as heads of liquid

A tray's pressure drop is most naturally written not in pascals but as the height of clear liquid that pressure would support, in millimetres. Three contributions add [9]:

$$h_d = 51\left(\frac{u_h}{C_o}\right)^2\frac{\rho_V}{\rho_L} \qquad \text{dry tray: the orifice equation,} \tag{413}$$

$$h_{ow} = 750\left(\frac{L_w}{\rho_L\, l_w}\right)^{2/3} \qquad \text{crest over the weir (Francis),} \tag{414}$$

$$h_r = \frac{12.5\times10^3}{\rho_L} \qquad \text{residual,} \tag{415}$$

so that the total head over a tray carrying a weir of height h_w is $h_t = h_d + h_w + h_{ow} + h_r$. Here u_h is the vapour velocity through the holes, L_w the liquid mass flow over the weir and l_w the weir length. The residual term h_r is Sinnott's deliberate simplification: the froth on a real tray is aerated, less dense than clear liquid, and rather than carry a charted aeration factor he absorbs the whole effect into a term that depends only on the liquid density. It is worth knowing this is a simplification and not a law.

The discharge coefficient C_o is itself a chart, against the hole-area ratio and the plate-thickness-to-hole-diameter ratio [8]. Choupo will not invent it: it is a *declared* input, printed back at you with its provenance, and it is yours to own.

30.4 Downcomer backup

The liquid leaving a tray must fall through the downcomer and squeeze out under the apron at the bottom. That last constriction costs head,

$$h_{dc} \;=\; 166\left(\frac{L_{wd}}{\rho_L\, A_m}\right)^2, \tag{416}$$

A_m being the smaller of the clearance area under the apron and the downcomer area itself. The liquid therefore stands in the downcomer at

$$h_b \;=\; (h_w + h_{ow}) \;+\; h_t \;+\; h_{dc}, \tag{417}$$

high enough to push itself through the gap *and* to overcome the pressure drop of the tray it is leaving. If h_b reaches the tray above, the column floods — from below this time, for an entirely different reason. The froth in the downcomer is roughly half the density of clear liquid, which gives the criterion in its familiar form:

$$h_b \;<\; \tfrac{1}{2}\,(\mathrm{TS} + h_w)\,. \tag{418}$$

30.5 Weeping

At the other end of the operating window, if the vapour is too slow it cannot hold the liquid up on the plate and the liquid drains through the holes. The weep point [9] is

$$u_{h,\min} \;=\; \frac{K_2 - 0.90\,(25.4 - d_h)}{\sqrt{\rho_V}}, \tag{419}$$

with d_h the hole diameter in millimetres. K_2 is read off a plot against $(h_w + h_{ow})$. It is optional in Choupo, and its absence is announced: give K_2 and the weep check runs, omit it and the column tells you the check was not performed. It never guesses.

Between the flood point above and the weep point below lies the whole operating window of a tray column, and *turndown* — the ratio of the two — is why a column designed for one throughput cannot always be run at half of it.

30.6 The two directions

Equation (410) contains the diameter nowhere; it is a velocity. The diameter enters only when the vapour volumetric flow Q_V is turned into a velocity through the net area, $u_V = Q_V/A_{\mathrm{net}}$. So the same equation runs both ways:

Rating	D given $\Rightarrow$ how close is each tray to flood?
Design	D absent $\Rightarrow$ what D keeps every tray at $\phi\,u_{\mathrm{flood}}$?

In design the widest tray sets the column, because a column has one diameter. Which tray that is, is not obvious in advance: below the feed the liquid traffic rises, which raises F_{LV} and *lowers* the capacity parameter, but the vapour volumetric flow falls too. The column is sized by whichever tray loses that argument, and you find out by looking.

`column09_tray_hydraulics` sizes the benzene–toluene column of §13 at $D = 1.322$ m, set by the last stripping tray at exactly the 80 % of flood it was asked for. `column10_flooding` builds the same column 1.10 m wide. The stream table is identical to the last digit, the MESH converges just as fast, the distillate is still 98.12 % benzene — and stage 14 runs at 115.6 % of its flooding velocity. On paper it is a perfectly good column. It would not work.

31 Multi-stream heat exchanger (MHeatX): a checker, not a solver

What you'll learn. The two-stream `heatExchanger` of Chapter 26 is a *rating* model: you give it hardware (U, A) and it *computes* the outlet temperatures. The multi-stream `multiStreamHX` (alias `MHeatX`) does the *opposite*. With N hot and M cold streams sharing a single adiabatic shell, the detailed conductances are not knowable without a full network model, so this unit asks the student to *furnish every outlet temperature* and then *verifies* two things the first and second laws demand: that $\sum_k Q_k = 0$ (the shell is adiabatic), and that the pooled hot and cold composite curves never cross ($\Delta T_{\min} > 0$). It is a *checker*, not a *solver* — the cleanest possible expression of the no-magic credo ("information follows the streams"): the unit never back-computes an outlet the topology cannot know.

31.1 The degrees of freedom: why verify, not solve

A single two-stream exchanger has a clean rating problem because two hardware numbers (U, A) plus the four inlet conditions close the degrees of freedom (Chapter 26). Put $N + M$ streams in one shell and the *internal* heat-transfer network — which stream sees which, over what area, with what local U — carries far more unknowns than the inlet/outlet temperatures alone can fix. Detailed-rating tools resolve this by demanding a fuller specification (zones, areas, or all-but-one outlet) and then closing the last balance. Choupo refuses to invent the missing information. Instead it adopts the spec that keeps the unit honest: *every* inlet stream carries its own outlet target temperature, and the unit's job is to *audit* the resulting energy balance, never to back-solve a duty the topology did not give it.

31.2 Per-stream duty and the first-law audit

For each stream k the duty is the enthalpy change between the furnished end states, evaluated with the *same* enthalpy machinery the rest of Choupo uses:

$$Q_k \;=\; \dot{n}_k \left[\, h_k(T_{k,\mathrm{out}}, P_k, v_{f,k}^{\mathrm{out}}) - h_k(T_{k,\mathrm{in}}, P_k, v_{f,k}^{\mathrm{in}}) \,\right], \tag{420}$$

where h_k is the formation-referenced molar enthalpy H_{stream} when every component carries a Gibbs-of-formation datum (so a stream that partly flashes inside the bundle is summed consistently), and the sensible H^L datum otherwise. No reaction happens inside an exchanger, so the per-component reference cancels in the *difference* (420): latent heat captured by the declared inlet→outlet path (a change in v_f) is the only non-trivial term beyond sensible heat. Streams sort themselves by sign:

$$Q_{\mathrm{hot}} = \sum_{Q_k<0} Q_k \;(\le 0,\ \text{releasing}), \qquad Q_{\mathrm{cold}} = \sum_{Q_k>0} Q_k \;(\ge 0,\ \text{absorbing}). \tag{421}$$

The first-law residual — the duty the bundle would have to import (>0) or export (<0) to satisfy the furnished outlets — is simply the sum over *all* streams,

$$\mathcal{I} \;=\; \sum_k Q_k \;=\; Q_{\mathrm{hot}} + Q_{\mathrm{cold}}. \tag{422}$$

A truly adiabatic shell has $\mathcal{I} = 0$. The unit reports $\mathcal{I}$ *honestly* as an external duty rather than silently nudging an outlet to force closure; the test is fractional,

$$\text{PASS} \iff \frac{|\mathcal{I}|}{\max\!\big(|Q_{\mathrm{hot}}|,\, |Q_{\mathrm{cold}}|\big)} \;\le\; \tau, \tag{423}$$

with $\tau =$ `operation.tolerance` (default 1 %). A failing balance is a loud warning, not a fixed-up answer.

31.3 The internal pinch: pooled composite curves

The second-law check reuses the composite-curve construction of pinch analysis, applied *inside* the one shell. All hot streams are pooled into a single hot composite and all cold streams into a single cold composite — each a piecewise-linear polyline in the (cumulative-duty, temperature) plane. Over each temperature band $[T_a, T_b]$ between consecutive breakpoints, a pool's duty is the sum of the *active* streams' contributions, each linearised as its share of its own ($T_{\mathrm{in}} \to T_{\mathrm{out}}$) span:

$$\Delta Q_{\mathrm{pool}}(T_a, T_b) \;=\; \sum_{k \in \mathrm{pool}} |Q_k| \, \frac{\min(T_b, T_k^{\mathrm{hi}}) - \max(T_a, T_k^{\mathrm{lo}})}{T_k^{\mathrm{hi}} - T_k^{\mathrm{lo}}}, \tag{424}$$

accumulated from $Q = 0$ at the pool's coldest end. Each pool is built over *its own* breakpoints, anchored at zero at its cold end — never padded with the other pool's range, which would corrupt the interpolation. Adopting the counter-current convention, the two composites are aligned at $q = 0$ and the approach is sampled over the shared duty span $Q_{\mathrm{span}} = \min(Q_{\mathrm{hot,tot}}, Q_{\mathrm{cold,tot}})$:

$$\Delta T_{\min} \;=\; \min_{0 \le q \le Q_{\mathrm{span}}} \left[\, T_{\mathrm{hot}}(q) - T_{\mathrm{cold}}(q) \,\right]. \tag{425}$$

A non-positive $\Delta T_{\min}$ is a *temperature cross*: the requested outlets would require heat to flow up a temperature gradient inside one adiabatic bundle, which the second law forbids. The unit flags it as a hard warning rather than silently accepting an infeasible design.

> *Intuition.* A single bundle "passes" only if the hot composite lies entirely above the cold composite over their shared duty range. $\mathcal{I} = 0$ (first law) says the two curves carry the *same total* duty; $\Delta T_{\min} > 0$ (second law) says they do not *touch or cross* while doing so. Both can fail independently: balanced-but-crossing (an infeasible internal arrangement), or feasible-but-unbalanced (the bundle needs an outside utility). The student sees, in one report, exactly which law is in trouble.

31.4 Worked mini-example

Three streams in one shell: a hot stream H1 releasing $Q_{\mathrm{H1}} = -100\,\mathrm{kW}$ cooling $400 \to 320\,\mathrm{K}$, and two cold streams absorbing it — C1 at $+60\,\mathrm{kW}$ ($300 \to 360\,\mathrm{K}$) and C2 at $+40\,\mathrm{kW}$ ($310 \to 350\,\mathrm{K}$). The first-law residual (422) is $\mathcal{I} = -100 + 60 + 40 = 0\,\mathrm{kW}$, so the shell is adiabatic and the balance PASSES. The hot composite runs $320 \to 400\,\mathrm{K}$ carrying $100\,\mathrm{kW}$; the pooled cold composite runs $300 \to 360\,\mathrm{K}$ carrying the same $100\,\mathrm{kW}$. Sampling (425) over the shared duty gives a positive minimum approach at the cold end ($320 - 300 = 20\,\mathrm{K}$), so $\Delta T_{\min} > 0$ and the design is thermodynamically feasible. Had the analyst instead asked C1 to leave at $410\,\mathrm{K}$ (above H1's inlet), the composites would cross, $\Delta T_{\min} < 0$, and the unit would refuse the design with a temperature-cross warning — even though the first law could still be made to close.

31.5 Power and limit

Power: the MHeatX is the honest multi-stream audit — it verifies first- and second-law feasibility of a heat-integration proposal *without* pretending to know an internal conductance network it cannot, and it surfaces the in-bundle pinch ($\Delta T_{\min}$, Q_{hot}, Q_{cold}, $\mathcal{I}$, plus a per-stream Q_k, all as KPIs) so the result feeds straight into the flowsheet-level pinch analysis and into sensitivity / sizing. **Limit:** precisely because it is a checker, it will *not* hand back an unknown outlet temperature — every inlet must carry a target; and the linearised two-point composites (425) assume each stream's Q–T relation is monotone and roughly straight between its declared end states (a stream crossing its dew or bubble dome inside the bundle should be split into segments so the latent kink is resolved). It models a single adiabatic shell — any genuine loss to the surroundings appears, correctly, as a non-zero $\mathcal{I}$ rather than being absorbed.

32 Drying of solids and spray drying

Drying removes a liquid (usually water) from a wet solid by evaporation. It competes with distillation as the most energy-intensive unit operation — the latent heat must be supplied and the (usually air) drying medium is thermodynamically inefficient — so the model must close both a mass and an energy balance. Choupo models drying at the "all models are wrong, but some are useful" altitude: measured drying curves and algebraic transfer correlations, no pore-scale or CFD detail.

32.1 Moisture content and the equilibrium

Moisture is reported on a *dry basis* X = kg water / kg dry solid (so $0 \le X < \infty$), related to the wet basis W by $X = W/(1-W)$. A wet solid held in air of a given relative humidity reaches an *equilibrium moisture* $X_e(a_w)$ — its *sorption isotherm* — below which that air cannot dry it. The *free moisture* $X - X_e$ is what can be removed; the bound fraction (held in capillaries and by sorption, its vapour pressure depressed) dries only under a lower humidity. Choupo reads X_e from a GAB isotherm on the solid component's `sorption` block; drying stops at $X_{\text{final}} = \max(X_e, \dots)$.

32.2 The drying-rate curve: constant and falling rate

Plot the drying rate $N = -\frac{m_s}{A}\frac{dX}{dt}$ against X and two regimes appear. In the **constant-rate period** ($X > X_c$) the surface stays wet: the particle surface sits at the *wet-bulb temperature* T_w and the rate is limited by external gas-film heat/mass transfer, so $N = N_c$ is flat. Below the **critical moisture** X_c the surface dries out and a receding front / crust governs; the **falling-rate period** ramps N down toward zero at X_e. The critical moisture X_c is a KINETIC datum (it depends on the drying intensity), so it lives in `constant/dryingKinetics`, not on the equilibrium isotherm. The drying *time* is the integral

$$t \;=\; \frac{m_s}{A}\int_{X_2}^{X_1}\frac{dX}{N(X)} \;=\; \underbrace{\frac{m_s(X_1 - X_c)}{A\,N_c}}_{\text{constant rate}} + \underbrace{\frac{m_s(X_c - X_e)}{A\,N_c}\,\ln\frac{X_c - X_e}{X_2 - X_e}}_{\text{falling rate (linear model)}},$$

the falling-rate branch using the linear (Lewis) closure $N \propto (X - X_e)$ matched to N_c at X_c — an honest first approximation that does *not* resolve crust diffusion; calibrate X_c/the curve against a measured drying curve of the real material.

32.3 Spray drying

A spray dryer atomises a solution into a cloud of droplets and dries them in hot gas to a powder of controlled particle size — seconds of residence, because the droplets are tiny. Three pieces compose.

32.4 Atomisation — the drop-size correlations

Atomisation sets the whole dryer. The spray is a *distribution* of drops, and its Sauter mean diameter $d_{32} = \sum n_i d_i^3 / \sum n_i d_i^2$ (the volume-to-surface mean) is the number that matters: the drying rate scales as $1/d$ (Ranz–Marshall, below), the powder inherits it ($d_p = d_{32}(x_{\text{solid}}\rho_L/\rho_{\text{solid}})^{1/3}$), and the chamber must be tall enough for the *largest* drops to dry. Choupo exposes the three families of Figure 77 as the selectable `atomiser` sub-model; each returns d_{32} and a Rosin–Rammler width n for the spread.

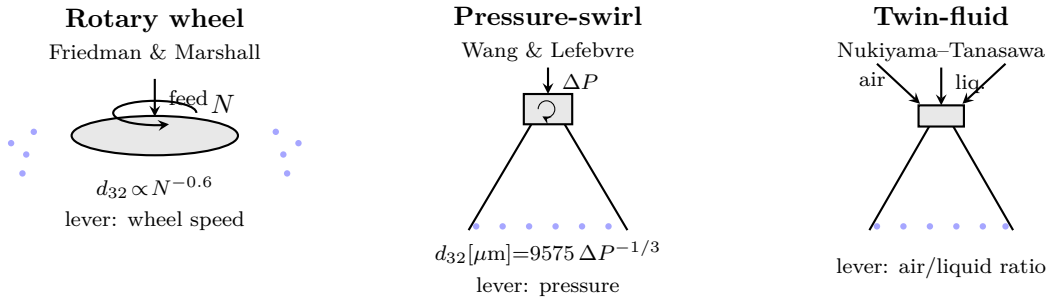

Figure 77: The three atomiser families. A drop is born when the atomising energy overcomes the surface tension holding the sheet together, against the liquid's viscous resistance: $d_{32} \sim (\text{cohesion } \sigma, \mu_L, \rho_L)/(\text{energy } N, \Delta P, U_{\text{air}})$. The wheel sizes by speed, the pressure nozzle by pressure, the twin-fluid by the air/liquid ratio; the twin-fluid gives the finest drops (best for viscous feeds).

The physics, in one line. A drop forms when the atomising energy overcomes the surface tension holding the liquid sheet/ligament together, against the viscosity resisting its deformation. So every correlation shares one skeleton — d_{32} grows with the cohesive properties (σ, μ_L, ρ_L) and shrinks as you spend energy (N, ΔP, air velocity). Because a concentrated, viscous feed therefore sprays *coarser*, σ and μ_L are drawn from the property layer — the *solution* viscosity via its mixing rule, not a fixed water value.

Rotary wheel — three regimes, one lever. Liquid fed to the centre of a disc (or cup) spinning at N rev/s is driven to the rim by centrifugal force. On a *flat* disc the liquid slips — it leaves below the rim speed, so the drop size is unpredictable — and industrial wheels therefore carry radial *vanes*: the liquid is trapped against a vane and discharged at the wheel's peripheral (tip) speed $U = \pi D N$, the true scaling velocity (100–200 m/s in practice). So the honest lever is not N alone but U: a small fast wheel and a large slow one that share U atomise alike.

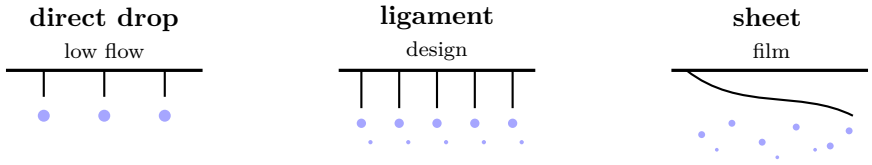

Figure 78: The three break-up regimes at a rotary wheel's rim as the feed rate rises (after Christensen & Steely; Lefebvre & McDonell, *Atomization and Sprays*). *Direct drop*: discrete, near-monodisperse drops shed one at a time. *Ligament*: radial jets break by Rayleigh instability into main drops + satellites — the industrial regime. *Sheet/film*: a continuous sheet overhangs the rim and breaks irregularly — a broad, coarse spray. Turning the feed up to raise throughput can silently cross ligament→sheet and *widen* the powder.

As the feed rate rises the rim passes through the three regimes of Figure 78; Tanasawa's critical flow rates (cgs: q [cm^3/s], D [cm], n [rpm]) fix the transitions,

$$q_{\text{direct}} = 2.8\left(\tfrac{D}{n}\right)^{2/3}\tfrac{\sigma}{\rho_L}\left[1+10\left(\tfrac{\mu_L}{(\rho_L\sigma D)^{1/2}}\right)^{1/3}\right]^{-1}, \qquad q_{\text{film}} \geq 5.3\left(\tfrac{D}{n}\right)^{2/3}\tfrac{\sigma}{\rho_L}\left(\tfrac{\rho_L}{\mu_L}\right)^{1/3},$$

the ligament (design) regime lying between. In it the Sauter mean follows the Friedman–Gluckert–Marshall correlation ($\Gamma = \dot{m}/\pi D$, the loading per wetted perimeter),

$$d_{32} = 0.4\,D\left(\frac{\Gamma}{\rho_L N D^2}\right)^{0.6}\left(\frac{\mu_L}{\Gamma}\right)^{0.2}\left(\frac{\sigma\rho_L D}{\Gamma^2}\right)^{0.1} \propto N^{-0.6},$$

a comparatively narrow spray ($n \approx 2.2$) — though the Rayleigh break-up leaves *satellite* drops, so the true distribution is mildly bimodal. Serrating the rim (a *toothed* wheel) delays the ligament→sheet transition; Christensen & Steely give the teeth needed to hold ligament formation, $z = 0.215(\rho_L\omega^2 D^3/\sigma)^{5/12}(\rho_L\sigma D/\mu_L^2)^{1/6}$ (e.g. ≈ 95 teeth on a 5 cm wheel at 3000 rpm). The wheel's asset is operational: since $d_{32} \propto \dot{m}^{0.2}$ only, drop size is *decoupled from throughput* — set size by speed, rate independently, with no small orifice to clog. This is why large food/dairy towers atomise by wheel, where a pressure nozzle would tie size and flow to one ΔP through one hole.

Pressure-swirl nozzle. Liquid forced through a swirl chamber leaves as a thin conical sheet that breaks up. The class-average design curve $d_{32}[\mu\text{m}] = 9575\,\Delta P[\text{Pa}]^{-1/3}$ is the quick estimate; the two-term SMD of Wang & Lefebvre resolves the properties and the sheet geometry,

$$d_{32} = 4.52\left(\frac{\sigma\mu_L^2}{\rho_G\Delta P^2}\right)^{0.25}(t\cos\theta)^{0.25} + 0.39\left(\frac{\sigma\rho_L}{\rho_G\Delta P}\right)^{0.25}(t\cos\theta)^{0.75},$$

t the film thickness and θ the half-cone angle. Pressure is the lever ($d_{32} \propto \Delta P^{-1/3}$); wider spray than a wheel ($n \approx 1.8$).

Twin-fluid (airblast) nozzle. A high-velocity air stream shreds the liquid — the finest drops of the three, the choice for viscous feeds. Nukiyama & Tanasawa (empirical, cgs; U_{rel} the air–liquid slip velocity, Q_L/Q_A the liquid-to-air volume ratio):

$$d_{32}\,[\mu\text{m}] = \frac{585}{U_{\text{rel}}}\sqrt{\frac{\sigma}{\rho_L}} + 597\left(\frac{\mu_L}{\sqrt{\sigma\rho_L}}\right)^{0.45}\left(1000\,\frac{Q_L}{Q_A}\right)^{1.5}.$$

A higher air/liquid ratio gives finer drops; broad spray ($n \approx 1.6$). This form is dimensional (cgs) — extrapolate with care.

The atomising energy — pressure follows the stream, power is a result. The atomising pressure of a pressure nozzle is *not* a free number: it is the feed's pressure above the chamber, $\Delta P = P_{\text{feed}} - P_{\text{chamber}}$, imposed by the feed pump on the *stream* (a bare-atomiser study may override it with `deltaP`). Each atomiser then carries a **shaft-power** estimate. For the pressure nozzle it is the hydraulic pumping power $\dot{W} = Q\,\Delta P/\eta_{\text{pump}}$. For the rotary wheel it splits in two: the kinetic energy imparted to the liquid, accelerated centre-to-rim to the tip speed, $\frac{1}{2}\dot{m}U^2$ (exact), *plus* **windage** — the wheel dragging the surrounding gas. The spray-drying literature gives windage only qualitatively ("rotary needs more power"), so Choupo anchors it on the classic rotating-disk drag (von Kármán 1921; Schlichting, *Boundary Layer Theory*),

$$\dot{W}_{\text{wind}} = \tfrac{1}{2}\,C_M\,\rho_g\,\omega^3 R^5, \qquad C_M = 0.146\,\mathrm{Re}_\omega^{-1/5}\ \text{(turbulent)}, \quad \mathrm{Re}_\omega = \frac{\omega R^2}{\nu_g},$$

flagged honestly as the *smooth free disk* — a vaned wheel pumps air like a fan and draws *more*, so this is a lower bound. The split (liquid KE, windage, drive efficiency) is printed in the atomiser note; the power feeds sizing and costing.

References. Marshall (1954); Masters (1991); Lefebvre & McDonell (2017, DOI 10.1201/9781315120911); Wang & Lefebvre, *J. Propulsion & Power* 3 (1987) 11 (DOI 10.2514/3.22946); Nukiyama & Tanasawa, *Trans. Soc. Mech. Eng. Japan* 5 (1939) 68.

Single-droplet drying. A droplet decelerates to its terminal velocity in milliseconds, after which the slip Reynolds number is small and the Ranz–Marshall correlation $\mathrm{Nu} = 2 + 0.6\,\mathrm{Re}^{1/2}\mathrm{Pr}^{1/3}$ collapses to its conduction limit $\mathrm{Nu} = 2$, i.e. $h = 2k_g/d$. The constant-rate time to reach the critical diameter D_c is then

$$t_c \;=\; \frac{\rho_L\,\Delta H_{\text{vap}}}{8\,k_g\,\Delta T_{\ln}}\left(D_0^2 - D_c^2\right),$$

with the log-mean driving force $\Delta T_{\ln}$ over the constant-rate span (the gas cools as it dries). The droplet then shrinks to the powder particle by mass, $d_f = d_0\left[(1+X_f)/(1+X_0)\right]^{1/3}$. The $\mathrm{Nu} = 2$ short-cut holds once the droplet has slowed; it *underpredicts* early-flight and large-droplet drying where the convective $\mathrm{Re}\,\mathrm{Pr}$ term matters.

Chamber: co- vs. counter-current. The chamber closes a lumped adiabatic mass + energy balance on the gas (fixing the dry-air rate G, the outlet humidity and temperature, and the air state at the critical point), with the residence time a RESULT of the hardware ($V \approx \dot{Q}_{\text{gas}}\,t$, $t \geq t_c$). *Co-current* (hot gas meets the wettest droplet) keeps the droplet near T_w — gentle, for heat-sensitive foods (milk, coffee); *counter-current* (hot gas meets the driest, crusted particle) is thermally efficient but harsh — for robust materials (detergents).

References. Marshall, *Atomization and Spray Drying*, AIChE Monograph 2 (1954); Masters, *Spray Drying Handbook*, 5th ed. (1991); Mujumdar (ed.), *Handbook of Industrial Drying*, 4th ed., CRC Press 2015 (DOI 10.1201/b17208). Atomisation: Lefebvre & McDonell, *Atomization and Sprays*, 2nd ed. 2017 (DOI 10.1201/9781315120911); Wang & Lefebvre, *J. Propulsion & Power* 3 (1987) 11 (DOI 10.2514/3.22946). Droplet drying and axial chamber profiles: Ranz & Marshall, *Chem. Eng. Prog.* 48 (1952) 141, 173; Langrish & Kockel, *Chem. Eng. J.* 84 (2001) 69 (DOI 10.1016/S1385-8947(00)00384-3, the characteristic drying curve); Mezhericher, Levy & Borde, *Chem. Eng. & Processing* 49 (2010) 1205 (DOI 10.1016/j.cep.2010.09.002). Falling-rate kinetics — the two non-CFD schools: the *characteristic drying curve* $N/N_c = f(\Phi)$ above (Langrish & Kockel), and the *Reaction Engineering Approach*, Chen & Xie, *Drying Technology* 15 (1997) 1063 (DOI 10.1080/07373939708917282) and Chen, *Drying Technology* 26 (2008) 627 (DOI 10.1080/07373930802045922), which replaces the critical-moisture break with a smooth apparent activation energy $\Delta E_v(X)$.

33 The simple unit operations

Not every unit needs a chapter. Four are one-liners, listed here for completeness — and because their very simplicity is the point: the flowsheet's plumbing (mixing, splitting, a furnished duty, a phase specification) is not where the modelling effort goes.

Mixer
Sums the component molar flows of its inlets and closes an adiabatic energy balance for the outlet temperature (a Newton-1D in T on $\sum H_{\text{in}} = H_{\text{out}}$).

Splitter
Divides one inlet into outlets by furnished split fractions — same T, P, composition in every branch (a flow split, *not* a separation: contrast the `gasSolidSplitter`, which carries solids but likewise does not classify).

Heater
A single process stream with a furnished duty Q (W, the only hardware); T_{out} is a RESULT from a Newton-1D on the enthalpy balance $H(T_{\text{out}}) = H_{\text{in}} + Q$. Phase-aware: a gas stream integrates H^{ig}, a liquid uses H^{L} — the same datum on both sides, so a combustor reaching $> 1000\,\text{K}$ is fine. A target outlet temperature is a DesignSpec on Q, not a spec.

Saturation & adiabatic flash
`bubbleT` solves $\sum_i K_i(T)\,x_i = 1$ for the bubble temperature; `dewT` solves $\sum_i y_i/K_i(T) = 1$ for the dew temperature (Newton-1D in T). The `adiabaticFlash` wraps the isothermal flash (Chapter 7's neighbourhood) in an outer Newton on T that drives the energy balance $H_{\text{out}} = H_{\text{in}}$ to zero — the Joule–Thomson / valve flash.

Valve
An isenthalpic pressure let-down: the outlet pressure is furnished (the only hardware) and the outlet state is the `adiabaticFlash` of the inlet at that lower P, holding $H_{\text{out}} = H_{\text{in}}$. A throttle that may flash part of the stream to vapour — the Joule–Thomson valve of the item above, exposed as a unit of its own.

ElectricLoad
A passive shaft sink: the electric generator (or any mechanical brake) wired to a turbine's shaft. It carries no process stream — the only thing crossing its boundary is the shaft work W delivered through an energy wire, which it absorbs. Its rating is not furnished; it is the W it actually receives, so it closes a turbine–generator shaft balance (Chapter 21) without itself setting a pressure.

Part VII — Equipment design and economics

After the simulator pass converges, the case may declare a `system/postDict` chain. Two post-processors are currently shipped: `sizing` turns each named unit op into a set of physical dimensions; `costing` turns those dimensions into a purchased + bare-module + total-module cost in EUR. Neither feeds back into the simulator — the contract is unidirectional, augmenting the converged `SimulationResult`.

> *Intuition.* Post-processing is what makes a process simulator more than a material-balance calculator. The same converged flowsheet, run with different `postDict` chains, can produce: a stream table for a PFD, a vessel list with weights and wall thicknesses, an EPC budget, a HAZOP-feed reactor inventory list, or a utility-summary report. The simulator does NOT know which use case; it just delivers a converged **F** and KPI table. The post-processor decides the rest.

1 Vessel sizing: the stirred tank

What you'll learn. How the simulator turns a CSTR's V_R KPI into a cylindrical drum with diameter D, length L, wall thickness t, weight, and a selected construction material.

1.1 Volume to geometry

For a vertical cylindrical drum with an L/D ratio specified by `designRules.L_over_D` (default 2.5), the diameter and length follow from the requested reactor volume:

$$V_R \;=\; \frac{\pi D^2}{4}\,L, \qquad L \;=\; (\mathrm{L/D})\,D \;\Longrightarrow\; D \;=\; \left(\frac{4V_R}{\pi\,(L/D)}\right)^{1/3}. \tag{426}$$

1.2 Wall thickness: ASME stress design

The vessel must withstand the design pressure $P_{\mathrm{design}} =$ `designRules.pressureDesign`. ASME Section VIII Div. 1 gives, for a thin cylindrical shell:

$$t \;=\; \frac{P_{\mathrm{design}}\,D/2}{S\,E - 0.6\,P_{\mathrm{design}}} \;+\; t_{\mathrm{corr}}, \tag{427}$$

where S is the allowable stress (taken as σ_y/F_S with $F_S = 4$), E is the joint efficiency (`jointEfficiency`, default 1.0), and t_{corr} is the corrosion allowance (`corrosionAllow`, default 3 mm). All material properties (σ_y, ρ, F_M, $T_{\max}$, $P_{\max}$) come from `data/standards/materials/<name>.dat` — carbon steel, SS304, SS316, aluminium are shipped.

1.3 Weight

With the wall thickness and an end-closure correction (the simulator uses $1.2\,V_{\mathrm{shell}}$ for an elliptical-head approximation), the shell metal mass is

$$m \;=\; \rho\,A_{\mathrm{shell}}\,t\cdot 1.2, \qquad A_{\mathrm{shell}} \;=\; \pi D L. \tag{428}$$

This m is the principal sizing output consumed by the Guthrie correlation downstream.

2 Heat-exchanger sizing: shell-and-tube

For a single-pass shell-and-tube HX the area follows from the

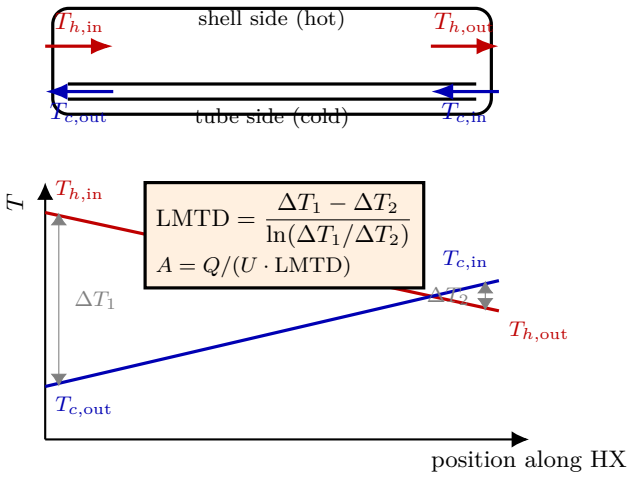

Figure 79: The shell-and-tube sizing picture. Top: a single-pass counter-current exchanger — hot stream in the shell, cold stream in the tubes, flowing in opposite directions. Bottom: the temperature profiles down the unit. The approach ΔT is *not* constant — it is ΔT_1 at one end and ΔT_2 at the other — so the correct mean driving force is the *log-mean* LMTD, not the arithmetic average. Once the duty Q is known (a unit-op KPI) and U is supplied, the area is simply $A = Q/(U \cdot \text{LMTD})$. Counter-current flow (cold rising against hot falling) keeps the approach open along the whole length — why it out-performs co-current for the same area.

duty Q (a unit-op KPI) and the user-supplied overall coefficient U and log-mean temperature difference LMTD:

$$A = \frac{Q}{U \cdot \text{LMTD}}. \tag{429}$$

Both U and LMTD are read from `designRules`; the simulator does not yet auto-compute LMTD from the streams (planned for). The pressure-design wall thickness uses Eq. (427) on the shell diameter.

3 Guthrie / Turton bare-module costing

What you'll learn. How the sized equipment list becomes a bare-module and total-module cost, anchored on Turton-style polynomial purchased-cost correlations, CEPCI year scaling, and a USD→EUR conversion.

3.1 Purchased cost from size

For each piece of equipment the model evaluates a polynomial fit of $\log_{10} C_p$ vs $\log_{10}$(size), with the size attribute equipment-specific (vessel → mass; HX → area):

$$\log_{10} C_p^{2001\,\mathrm{USD}} \;=\; K_1 \;+\; K_2\,\log_{10}(\mathrm{size}) \;+\; K_3\,\big[\log_{10}(\mathrm{size})\big]^2. \tag{430}$$

The coefficients (K_1, K_2, K_3) come from Turton 4th ed. [66] and live inside `src/postProcessing/costing/Guthrie.cpp` — one set per equipment type.

3.2 Bare-module factor

The base purchased cost assumes carbon steel and atmospheric operation. Corrections for actual material and design pressure are folded into the bare-module factor:

$$C_{\mathrm{BM}} \;=\; C_p\,\big(B_1 + B_2\,F_M\,F_P\big), \tag{431}$$

with B_1, B_2 from Turton (e.g. 1.49 and 1.52 for a vertical vessel), F_M the material factor read from `data/standards/materials/<name>` and F_P the pressure factor from a Turton polynomial in $\log_{10} P_{\mathrm{design}}$.

3.3 Total-module cost

Adding contingency and fees:

$$C_{\mathrm{TM}} \;=\; 1.18\,C_{\mathrm{BM}}. \tag{432}$$

3.4 Year and currency scaling

Costs are escalated from the Turton reference year (2001) to the user's target year via the Chemical Engineering Plant Cost Index (CEPCI), and converted from USD to EUR:

$$C(\mathrm{year},\mathrm{EUR}) \;=\; C_{\mathrm{TM}}^{2001\,\mathrm{USD}} \cdot \frac{\mathrm{CEPCI}_{\mathrm{year}}}{\mathrm{CEPCI}_{2001}} \cdot \texttt{usdToEur}. \tag{433}$$

The user supplies `year`, `cepci`, `cepci2001` (default 397), and `usdToEur` in `postDict`.

Common pitfall. Turton's correlations are accurate to about ±30% for order-of-magnitude work — they are *not* engineering-grade estimates. They are pedagogically valuable because the entire chain from V_R to EUR is exposed (no proprietary database, no black-box adjustment), so a student can perturb any link and see the downstream effect. They are *not* a substitute for a vendor quote.

Runnable case. `tutorials/process02_with_design` — the canonical end-to-end: two-unit flowsheet (→ CSTR + flash), `postDict` declares sizing for both, Guthrie costing in EUR_{2026} at the user-chosen CEPCI. The output table shows F_M, F_P, $C_{\mathrm{purchased}}$, C_{BM}, C_{TM} per unit and the grand total.

Appendix A — Symbol glossary

Symbol	Meaning
T	Temperature, K
P	Pressure, bar
F, L, V	Total molar flow: feed, liquid, vapour, kmol/h
z_i, x_i, y_i	Mole fraction of i in feed, liquid, vapour
K_i	Equilibrium K-value, $K_i = y_i/x_i$
β	Vapour fraction, V/F
γ_i	Activity coefficient of i in the liquid
φ_i^V	Fugacity coefficient of i in the vapour
$P_i^{\text{sat}}(T)$	Saturation pressure of pure i, bar
$\Delta H_{\text{vap}}(T)$	Molar heat of vaporisation, J/mol
R_{g}	Universal gas constant, $8.314\,\text{J/(mol K)}$
μ_i	Chemical potential of i
$\hat{f}_i$	Fugacity of i in the mixture
G^E	Excess Gibbs energy of the liquid mixture, J/mol
ξ	Reaction extent, mol/h or mol/s
$\boldsymbol{\nu}$	Stoichiometric vector of a reaction
$r(\mathbf{x}, T)$	Reaction rate, $\text{mol/(m}^3\,\text{s)}$
Da	Damköhler number, $k\,\tau$
$\mathbf{J}$	Jacobian matrix (residual w.r.t. unknowns or w.r.t. parameters)
q	Wegstein acceleration factor (Eq. 194)
λ	Damping factor: Newton line search or Marquardt damping (Eq. 184)
$\mathbf{p}$	Adjustable parameter vector in regression (Ch. 4)
$\mathbf{r}$	Residual vector, $r_i = T_i^{\text{pred}}(\mathbf{p}) - T_i^{\text{exp}}$
$\chi^2(\mathbf{p})$	Sum of squared residuals (Eq. 175)
τ, G, α	NRTL binary parameters
Λ	Wilson binary parameter

Appendix B — References

References

[1] Langmuir, I., "The adsorption of gases on plane surfaces of glass, mica and platinum", *J. Am. Chem. Soc.* **40** (1918) 1361. The isotherm Eq. (361) rests on.

[2] Hinshelwood, C.N., *The Kinetics of Chemical Change*, Oxford University Press, 1940. Surface reaction between adsorbed species as the rate-determining step.

[3] Hougen, O.A. and Watson, K.M., "Solid catalysts and reaction rates", *Ind. Eng. Chem.* **35** (1943) 529. The engineering form: kinetic factor, driving force, adsorption group.

[4] Pöpken, T., Götze, L. and Gmehling, J., "Reaction kinetics and chemical equilibrium of homogeneously and heterogeneously catalyzed acetic acid esterification with methanol and methyl acetate hydrolysis", *Ind. Eng. Chem. Res.* **39** (2000) 2601. Eq. 16 (adsorption-based rate law, Table 11), the adsorption constants (Table 10), the UNIQUAC parameters they were regressed with (Tables 2–3), and the independent K_a of Eq. 17.

[5] Souders, M. and Brown, G.G., "Design of fractionating columns. I. Entrainment and capacity", *Industrial & Engineering Chemistry* **26** (1934) 98. The droplet-suspension argument behind $u_{\mathrm{flood}} \propto \sqrt{(\rho_L - \rho_V)/\rho_V}$.

[6] Fair, J.R., "How to predict sieve tray entrainment and flooding", *Petro/Chem Engineer* **33**(10) (1961) 45. The capacity parameter $C_{\mathrm{SB}}(F_{LV}, \mathrm{TS})$, published as a chart.

[7] Lygeros, A.I. and Magoulas, K.G., *Hydrocarbon Processing* **65**(12) (1986) 43. The algebraic representation of Fair's chart used in Eq. (412); inaccurate below $F_{LV} \approx 0.03$, where its slope has the wrong sign.

[8] Liebson, I., Kelley, R.E. and Bullington, L.A., *Petroleum Refiner* **36** (1957). The sieve-tray discharge coefficient C_o, charted against A_h/A_p and plate thickness over hole diameter.

[9] Sinnott, R.K., *Coulson & Richardson's Chemical Engineering*, Vol. 6, *Chemical Engineering Design*, Butterworth-Heinemann. The clear-liquid-head lineage used here: dry-tray head, weir crest, residual head, downcomer backup, and the weep point.

[10] Weisbach, J., *Lehrbuch der Ingenieur- und Maschinen-Mechanik*, Vol. 1, Braunschweig, 1845; and Darcy, H., *Recherches expérimentales relatives au mouvement de l'eau dans les tuyaux*, Mallet-Bachelier, Paris, 1857. The velocity-head form of the pipe-friction law and the experiments behind the friction factor; the modern name credits both.

[11] Colebrook, C. F., "Turbulent Flow in Pipes, with Particular Reference to the Transition Region between the Smooth and Rough Pipe Laws," *J. Inst. Civ. Eng.* **11**(4), 133–156, 1939. The implicit friction-factor equation interpolating Nikuradse's smooth- and rough-pipe data; Choupo's `Colebrook` model.

[12] Haaland, S. E., "Simple and Explicit Formulas for the Friction Factor in Turbulent Pipe Flow," *J. Fluids Eng.* **105**(1), 89–90, 1983. The explicit approximation to Colebrook used by Choupo's `Haaland` model.

[13] Churchill, S. W., "Friction-Factor Equation Spans All Fluid-Flow Regimes," *Chem. Eng.* **84**(24), 91–92, 1977. The single explicit correlation valid across laminar, transition and turbulent flow; Choupo's default `Churchill` model.

[14] Lockhart, R. W., & Martinelli, R. C., "Proposed Correlation of Data for Isothermal Two-Phase, Two-Component Flow in Pipes," *Chem. Eng. Prog.* **45**(1), 39–48, 1949. The foundational gas–liquid pressure-drop correlation, developed and validated on air-water (and similar) pipe data; Choupo's default `twoPhase` model.

[15] Butterworth, D., "A Comparison of Some Void-Fraction Relationships for Co-Current Gas–Liquid Flow," *Int. J. Multiphase Flow* **1**(6), 845–850, 1975. The algebraic fit of the Lockhart–Martinelli void fraction Choupo uses for the holdup.

[16] Friedel, L., "Improved Friction Pressure Drop Correlations for Horizontal and Vertical Two-Phase Pipe Flow," European Two-Phase Flow Group Meeting, Ispra, paper E2, 1979. The general two-phase friction multiplier ϕ_{LO}^2; Choupo's `Friedel` model.

[17] Beggs, H. D., & Brill, J. P., "A Study of Two-Phase Flow in Inclined Pipes," *J. Petroleum Technology* **25**(5), 607–617, 1973. Flow-pattern map, holdup with inclination correction, and the two-phase friction-factor ratio; Choupo's `BeggsBrill` model.

[18] Poling, B. E., Prausnitz, J. M., & O'Connell, J. P., *The Properties of Gases and Liquids*, 5th ed., McGraw-Hill, 2001. The standard reference for pure-component property correlations and activity-coefficient models. Chapters 6 (vapour pressure), 8 (activity coefficients), 9 (mixture properties).

[19] Henley, E. J., Seader, J. D., & Roper, D. K., *Separation Process Principles*, 3rd ed., Wiley, 2011. Chapters 4–7 cover the equilibrium-stage methods used in this simulator (flash, distillation, absorption).

[20] Wijmans, J. G., & Baker, R. W., "The Solution-Diffusion Model: A Review", *J. Membr. Sci.*, 107, 1–21 (1995), DOI 10.1016/0376-7388(95)00102-I. The standard derivation of the pressure- and concentration-driven fluxes (Eqs. 325–326) used by the spiral-wound module.

[21] Bowen, W. R., Mohammad, A. W., & Hilal, N., "Characterisation of nanofiltration membranes for predictive purposes — use of salt, uncharged solutes and atomic force microscopy", *J. Membr. Sci.*, 126, 91–105 (1997). The Donnan-Steric Pore Model (DSPM): the extended Nernst–Planck pore flux (Eq. 338), the steric partition and Deen hindrance (Eqs. 335, 339–340), pore electroneutrality (Eq. 336) and the effective fixed charge density X_d fitted from salt rejection.

[22] Bowen, W. R., & Welfoot, J. S., "Modelling the performance of membrane nanofiltration — critical assessment and model development", *Chem. Eng. Sci.*, 57, 1121–1137 (2002). Adds dielectric exclusion to give DSPM-DE: the Born solvation partition (Eq. 337) with the reduced pore dielectric ε_p, the three-mechanism interfacial partition (Eq. 334) and the pore-transmission closure (Eq. 342); source of the NF270 archetype parameters (r_p, $A_k/\Delta x$, ε_p).

[23] Deen, W. M., "Hindered transport of large molecules in liquid-filled pores", *AIChE J.*, 33(9), 1409–1425 (1987). The hydrodynamic hindrance factors K_d (diffusive) and K_c (convective) as functions of $\lambda = r_i/r_p$ (Eqs. 339–340).

[24] Cowan, D. A., & Brown, J. H., "Effect of Turbulence on Limiting Current in Electrodialysis Cells", *Ind. Eng. Chem.*, 51(12), 1445–1448 (1959). The original limiting-current relation $i_{\lim} = zFk\,c/(t_{\mathrm{cu}} - t_{\mathrm{co}})$ (Eq. 350) and the conductance-vs-current/voltage diagnostic for the electrodialysis stack.

[25] Strathmann, H., *Ion-Exchange Membrane Separation Processes*, Membrane Science and Technology Series 9, Elsevier, 2004. The standard text on electrodialysis; source of the membrane (Donnan + diffusion) potential form of the Nernst equation (Eq. 344), the cell-pair resistance build-up (Eq. 348), and the representative Neosepta CMX/AMX area-resistance values in the IEM `.dat`.

[26] Davies, C. W., *Ion Association*, Butterworths, London, 1962. The extended Debye–Hückel ($\log_{10}\gamma_i = -Az_i^2[\sqrt{I}/(1+\sqrt{I}) - 0.3I]$, Eq. 345) used for the per-channel ionic activities, shared verbatim with the electrolyte speciation stack (Chapter 3).

[27] Malmberg, C. G., & Maryott, A. A., "Dielectric Constant of Water from 0° to 100°C," *J. Res. Natl. Bur. Stand.* **56**(1), 1–8, 1956. The static relative permittivity polynomial $\varepsilon_w(t)$ (US-government, public domain) anchoring Choupo's Debye–Hückel temperature factor (`SolventProperties::epsWater`).

[28] Kell, G. S., "Density, Thermal Expansivity, and Compressibility of Liquid Water from 0° to 150°C," *J. Chem. Eng. Data* **20**(1), 97–105, 1975. The standard liquid-water density polynomial $\rho_w(t)$ supplying the $(\rho_w/\rho_w^{25})^{1/2}$ part of the Debye–Hückel factor (`SolventProperties::rhoWaterKell`).

[29] Nightingale, E. R., "Phenomenological Theory of Ion Solvation: Effective Radii of Hydrated Ions", *J. Phys. Chem.*, 63(9), 1381–1387 (1959). Source of the per-ion infinite-dilution diffusivities D_i^0 (and hydrated radii) in `ions.dat`, from which the channel conductivity (Eq. 347) is built via Nernst–Einstein.

[30] Bard, A. J., & Faulkner, L. R., *Electrochemical Methods: Fundamentals and Applications*, 2nd ed., Wiley, 2001. The reference followed by Choupo's shared `electrochem` kernel for the Nernst equation, Faraday's law of electrolysis (Eq. 343), and the Butler–Volmer stub.

[31] Randolph, A. D., & Larson, M. A., *Theory of Particulate Processes*, 2nd ed., Academic Press, 1988. The standard text on the population balance and the MSMPR crystalliser; source of Eq. 307 and the $n(L) = n^0 e^{-L/G\tau}$ result.

[32] Hulburt, H. M., & Katz, S., "Some Problems in Particle Technology: A Statistical Mechanical Formulation", *Chem. Eng. Sci.*, 19, 555–574 (1964). Introduces the method of moments for the population balance (Eq. 310).

[33] Sandler, S. I., *Chemical, Biochemical, and Engineering Thermodynamics*, 5th ed., Wiley, 2017. The textbook the simulator's thermodynamic notation follows most closely; source of the cubic-EoS departure functions $H^{\mathrm{R}}, S^{\mathrm{R}}$ (eqs. 153–154).

[34] Soave, G., "Equilibrium Constants from a Modified Redlich–Kwong Equation of State", *Chem. Eng. Sci.*, 27(6), 1197–1203 (1972). Introduces the $\alpha(T;\omega)$ temperature function and the $m(\omega)$ correlation (eqs. 144–145) used by `SRK`.

[35] Peng, D.-Y., & Robinson, D. B., "A New Two-Constant Equation of State", *Ind. Eng. Chem. Fundam.*, 15(1), 59–64 (1976). The Peng–Robinson cubic ($\Omega_a = 0.45724$, $\Omega_b = 0.07780$, $\kappa(\omega)$) implemented in `PR`.

[36] Renon, H., & Prausnitz, J. M., "Local Compositions in Thermodynamic Excess Functions for Liquid Mixtures", *AIChE J.*, 14, 135–144 (1968). Original NRTL paper.

[37] Wilson, G. M., "A New Expression for the Excess Gibbs Energy of Mixing", *J. Am. Chem. Soc.*, 86, 127 (1964). Original Wilson paper.

[38] Fredenslund, A., Jones, R. L., & Prausnitz, J. M., "Group-Contribution Estimation of Activity Coefficients in Nonideal Liquid Mixtures", *AIChE J.*, 21, 1086–1099 (1975). The original UNIFAC paper.

[39] Hansen, H. K., Rasmussen, P., Fredenslund, A., Schiller, M., & Gmehling, J., "Vapor-Liquid Equilibria by UNIFAC Group Contribution. 5. Revision and Extension", *Ind. Eng. Chem. Res.*, 30, 2352–2355 (1991). The revised original-UNIFAC R_k/Q_k and main-group a_{mn} tables bundled in `data/standards/unifac/`.

[40] Abrams, D. S., & Prausnitz, J. M., "Statistical Thermodynamics of Liquid Mixtures: A New Expression for the Excess Gibbs Energy of Partly or Completely Miscible Systems", *AIChE J.*, 21(1), 116–128 (1975). Original UNIQUAC paper — the combinatorial (Staverman–Guggenheim) plus residual (quasi-chemical) split and the r, q structural constants.

[41] Watson, K. M., "Thermodynamics of the Liquid State", *Ind. Eng. Chem.*, 35, 398 (1943). The heat-of-vaporisation correlation.

[42] Parkhurst, D. L. and Appelo, C. A. J., "Description of Input and Examples for PHREEQC Version 3 — A Computer Program for Speciation, Batch-Reaction, One-Dimensional Transport, and Inverse Geochemical Calculations", U.S. Geological Survey Techniques and Methods, book 6, chap. A43 (2013). The public-domain speciation engine whose mass-action databases (`phreeqc.dat`) Choupo's `speciation.dat` catalogue is derived from, and whose charge-balance percent-error convention (Eq. 118) and charge-balance pH closure the speciation solver follows.

[43] Gaines, G. L., & Thomas, H. C., "Adsorption Studies on Clay Minerals. II. A Formulation of the Thermodynamics of Exchange Adsorption", *J. Chem. Phys.*, **21**(4), 714–718 (1953). The convention Choupo uses for the bound-species activity in (354): the activity of an exchanger species is its charge-weighted *equivalent fraction* on the resin, not its molality.

[44] Helfferich, F., *Ion Exchange*, McGraw-Hill, New York (1962; Dover reprint 1995). The classic monograph on ion-exchange equilibria, selectivity and the strong-acid cation resin used as the textbook capacity nameplate for `SAC_Na`.

[45] Wachinski, A. M., *Ion Exchange Treatment for Water*, American Water Works Association, Denver (2017). Practitioner reference for water-softening capacity bases (eq/L bed vs eq/kg dry), regeneration and the sodium "salt penalty" (359).

[46] Pitzer, K. S., "Thermodynamics of Electrolytes. I. Theoretical Basis and General Equations", *J. Phys. Chem.*, 77(2), 268–277 (1973). The ion-interaction model whose osmotic coefficient (Eq. 91) the simulator uses for the membrane osmotic pressure, via the Debye–Hückel limiting law plus short-range virial corrections.

[47] Pitzer, K. S., "Thermodynamics of Electrolytes. V. Effects of Higher-Order Electrostatic Terms", *J. Solution Chem.*, 4(3), 249–265 (1975). The unsymmetrical higher-order mixing term $E_\theta(I)$ and its integral $J(x)$ (Eqs. 107, 109) for like-sign ions of different charge, evaluated in `PitzerForm::Etheta`; the closed-form $J(x)$ approximation is the one tabulated in Pitzer (1991) "Activity Coefficients in Electrolyte Solutions", 2nd ed., CRC, ch. 3, and used by USGS PHRQPITZ (Plummer et al. 1988).

[48] Silvester, L. F., & Pitzer, K. S., "Thermodynamics of Electrolytes. 8. High-Temperature Properties, Including Enthalpy and Heat Capacity, with Application to Sodium Chloride", *J. Phys. Chem.*, 81(19), 1822–1828 (1977). The temperature derivatives of the Pitzer virial parameters ($\partial\beta^{(0)}/\partial T$, $\partial\beta^{(1)}/\partial T$, $\partial C^{\phi}/\partial T$, Eq. 306) from which the relative apparent molar enthalpy L_ϕ is obtained by Gibbs–Helmholtz; carried per pair in `pairs.dat` and fitted by `bin/curate/fit_lphi_pitzer.py`.

[49] Parker, V. B., *Thermal Properties of Aqueous Uni-univalent Electrolytes*, National Standard Reference Data Series NSRDS–NBS 2, U.S. National Bureau of Standards, Washington, D.C. (1965). The compiled relative apparent molar enthalpies $\Phi_L(m)$ at 25°C (e.g. NaOH) against which the Silvester–Pitzer T-slots are regressed and the `calorimetricFit` gate is evaluated.

[50] Harvie, C. E., Møller, N., & Weare, J. H., "The prediction of mineral solubilities in natural waters: The Na–K–Mg–Ca–H–Cl–SO_4–OH–HCO_3–CO_3–CO_2–H_2O system to high ionic strengths at 25 °C", *Geochim. Cosmochim. Acta*, 48(4), 723–751 (1984). The multi-ion assembly of the per-ion $\ln\gamma_i$ (Eqs. 102–104) implemented in `PitzerHMW`; the seawater/brine ternary parameter set (θ, ψ, λ, ζ) carried in `electrolyte/mixing.dat`.

[51] Chen, C.-C., & Evans, L. B., "A Local Composition Model for the Excess Gibbs Energy of Aqueous Electrolyte Systems", *AIChE J.*, 32(3), 444–454 (1986). The electrolyte-NRTL model (§4); source of the water–NaCl energy parameters $\tau = (8.885, -4.549)$ and $\alpha = 0.2$, and the unsymmetric-reference local-composition equations the `ENRTLSingleSalt` kernel implements.

[52] Chen, C.-C., Britt, H. I., Boston, J. F., & Evans, L. B., "Local Composition Model for Excess Gibbs Energy of Electrolyte Systems", *AIChE J.*, 28(4), 588–596 (1982). The original two-rule (like-ion repulsion + local electroneutrality) electrolyte-NRTL formulation underlying §4.

[53] Hamer, W. J., & Wu, Y.-C., "Osmotic Coefficients and Mean Activity Coefficients of Uni-univalent Electrolytes in Water at 25 °C", *J. Phys. Chem. Ref. Data*, 1(4), 1047–1100 (1972). The NaCl $\gamma_\pm(m)$ and $\phi(m)$ reference data the eNRTL kernel is validated against (AAD 1.8 % / 1.4 %, §4).

[54] Wegstein, J. H., "Accelerating Convergence of Iterative Processes", *Comm. ACM*, 1(6), 9–13 (1958). The two-page paper that powers the simulator's outer activity-coefficient loop.

[55] Wang, J. C., & Henke, G. E., "Tridiagonal Matrix for Distillation", *Hydrocarbon Processing*, 45(8), 155 (1966). The bubble-point method used in `DistillationColumn`.

[56] Naphtali, L. M., & Sandholm, D. P., "Multicomponent Separation Calculations by Linearization", *AIChE J.*, 17, 148–153 (1971). The equation-oriented Newton method on the roadmap.

[57] Michelsen, M. L., "The Isothermal Flash Problem. Part I — Stability" and "Part II — Phase-Split Calculation", *Fluid Phase Equilibria*, 9, 1–19, 21–40 (1982). The tangent-plane-distance criterion and the second-order Newton method on the roadmap.

[58] Golub, G. H., & Van Loan, C. F., *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. Reference for the Gauss elimination with partial pivoting used in `NewtonND`.

[59] Levenberg, K., "A Method for the Solution of Certain Non-Linear Problems in Least Squares", *Quarterly of Applied Mathematics*, 2(2), 164–168 (1944). The original damped Gauss-Newton (Eq. 183).

[60] Marquardt, D. W., "An Algorithm for Least-Squares Estimation of Nonlinear Parameters", *Journal of the Society for Industrial and Applied Mathematics*, 11(2), 431–441 (1963). The diagonal-scaling refinement (Eq. 184) that the simulator implements.

[61] Nelder, J. A., & Mead, R., "A Simplex Method for Function Minimization", *The Computer Journal*, 7(4), 308–313 (1965). The original direct-search simplex algorithm implemented in `solver::nelderMead` and used by the `OptimizationDriver`.

[62] Nocedal, J., & Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006. Primary reference for Choupo's line-search SQP (Ch. 18: damped-BFGS, ℓ_1 merit, merit directional derivative eq. 18.29) and for the inner active-set convex-QP solver (Sec. 16.5, Algorithm 16.3; the equality-constrained KKT system eq. 16.4). The method is implemented from scratch in `solver::sqp` and `solver::activeSetQP`; this is not a port of any code.

[63] Powell, M. J. D., "A fast algorithm for nonlinearly constrained optimization calculations", in *Numerical Analysis* (G. A. Watson, ed.), Lecture Notes in Mathematics, vol. 630, 144–157, Springer, 1978. Source of the *damped*-BFGS modification (the θ blend toward $\mathbf{Bs}$) that keeps the SQP Hessian approximation positive definite even when the Lagrangian curvature is indefinite.

[64] Hock, W., & Schittkowski, K., *Test Examples for Nonlinear Programming Codes*, Lecture Notes in Economics and Mathematical Systems, vol. 187, Springer, 1981. Source of the analytic problems HS21, HS35 and HS71 used by `verifySQP` as the merge-blocking self-check of the SQP algorithm against published optima.

[65] Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P., *Numerical Recipes: The Art of Scientific Computing*, 3rd ed., Cambridge University Press, 2007. Chapter 15.5 ("Nonlinear Models") treats Levenberg-Marquardt in the same notation used here; the 10-up/10-down λ heuristic of that text is gentled to 3-up/5-down in the simulator.

[66] Turton, R., Bailie, R. C., Whiting, W. B., & Shaeiwitz, J. A., *Analysis, Synthesis, and Design of Chemical Processes*, 4th ed., Prentice Hall, 2012. Source of the Guthrie-style cost correlations and the CEPCI scaling used by `CostingPass`.

[67] Rudd, D. F., & Powers, G. J., *Process Synthesis*, Prentice-Hall, 1973. The book that introduces the modern framing of a flowsheet as a directed graph of *physical equipment* connected by *physical streams*. The "unit op as equipment" stance in this chapter is, in spirit, the opening message of Chapters 1–3 of that text.

[68] Westerberg, A. W., Hutchison, H. P., Motard, R. L., & Winter, P., *Process Flowsheeting*, Cambridge University Press, 1979. The first textbook on sequential-modular flowsheeting. Chapters 2 and 4 articulate the "the user should not be required to know which variables the solver will iterate on" principle that Choupo inherits.

[69] Stephanopoulos, G., *Chemical Process Control: An Introduction to Theory and Practice*, Prentice-Hall, 1988. Chapters 1–4 distinguish carefully between *operating parameters* that live on a unit (and that an operator can change in real time) and *design constraints* that live on a higher control layer. Same conceptual split as Choupo's unit-op vs DesignSpec layering.

[70] Biegler, L. T., Grossmann, I. E., & Westerberg, A. W., *Systematic Methods of Chemical Process Design*, Prentice-Hall, 1997. Chapter 8 formalises the two-level inner/outer decomposition of design optimisation that Choupo mirrors with sequential-modular evaluation inside and DesignSpec / OptimizationDriver wraps outside.

[71] CAPE-OPEN Laboratories Network, *Unit Operation Interface Specification*, current version, `https://www.colan.org/specifications/unit-operation-interface-specification/`. The vendor-neutral standard (since 1999) that codifies the ports-vs-parameters separation, the material / energy / information stream taxonomy, and the contract that any third-party unit operation must satisfy to be droppable into a CAPE-OPEN-compliant flowsheeter. Choupo's `UnitOperation` interface is a deliberate subset of this standard; v1.0 will add adapter wrappers so Choupo modules can load into a CAPE-OPEN host and vice versa.

[72] Rosen, E. M., & Pauls, A. C., "Computer Aided Chemical Process Design: The FLOWTRAN System", *Computers & Chemical Engineering*, 1(1), 11–21 (1977). The canonical description of Monsanto's FLOWTRAN, the first commercially viable chemical process simulator and the architectural ancestor of every sequential-modular simulator since. Used here for the comparative context of Appendix C.

Appendix C — Historical context: Choupo and FLOWTRAN

Steady-state sequential-modular chemical process simulation as we know it begins with FLOWTRAN, developed at Monsanto by Robert Cavett, Edward Rosen, Allen Pauls and colleagues between 1964 and 1966 [72]. FLOWTRAN was the first system to package the abstractions used in this manual — streams as information records, unit operations as "blocks" acting on streams, an outer recycle convergence loop on tear streams — into a coherent, redistributable simulator. Every modern chemical-process simulator inherits this architecture. Choupo does too, by design: when you write a block that inherits from `UnitOperation` and registers itself with the factory, you are continuing a lineage that runs through Cavett's 1964 "building-block architecture" memo, the 1966 internal release, the 1969 commercial product, and the 1974 distribution through CACHE.

FLOWTRAN's scope, as documented by Rosen & Pauls (1977)

Component	What it was
FLOWTRAN simulator	A FORTRAN program translating a user's flow-sheet description into a series of `CALL` statements; compiled and run against a library of pre-built blocks. Sequential-modular, with recycle convergence by bounded Wegstein.
PROPTY	Companion program: fits Antoine, ideal-gas C_p and other property correlations from raw experimental data, writing parameters into the property file.
VLE	Companion program: fits liquid-phase activity-coefficient parameters (van Laar and others) from VLE data.
INF	Information-retrieval program; reads/writes the property file accessed by FLOWTRAN.

The unit-operation block library, as published in Table 3 of [72], contained approximately fifty-eight blocks across eleven categories:

Category	Blocks	Count
Flash	`IFLSH`, `AFLSH`, `BFLSH`, `KFLSH`, `FLSH3`	5
Absorption / stripping	`ABSBR` (rigorous)	1
Extraction	`EXTRC` (rigorous LLE)	1
Heat exchange	`EXCH1/2/3`, `HEATR`, `CLCN1`, `DESUP`, `HTR3`, `BOILR`	8
Miscellaneous unit ops	`ADD`, `MIX`, `SPLIT`	3
Distillation	`FRAKB` (matrix), `DISTL`, `DSTWU`, `SEPR`, `AFRAC`	5
Reaction	`REACT`, `AREAC`, `XTNT`	3
Pump / compressor	`PUMP`, `GCOMP`, `MULPY`	3
Convergence	Bounded Wegstein (tear streams)	1
Control	`CNTRL`, `PCVB`, `DSPLT`, `RCNTL`	4
Cost & sizing	`CAFLH`, `CIFLH`, `CKFLH`, `CFLH3`, `CFRKB`, `CAFRC`, `CDSTL`, `CABSB`, `CTABS`, `CEXC1/2/3`, `CHETR`, `CCLN1`, `CPUMP`, `CCOMP`	16
Report writer	`SUMRY` et al.	4
Profit analysis	`RAWMT`, `BPROD`, `PRODT`, `PRBFT`	4
Total		∼58

The public physical-property database carried data for 180 chemical species [72] — twenty times the Choupo inventory of nine. By 1975 FLOWTRAN ran on the CDC 6000 series, the GE/Honeywell 600/6080 and the IBM 360/370 series; approximately seventy companies held licences; twenty-one universities were using the CACHE distribution in coursework.

An honest LOC estimate

Rosen & Pauls (1977) does not publish a lines-of-code figure for FLOWTRAN, and no quantitative size measurement appears in any indexed source I could locate. The figure is presumably extractable from the FLOWTRAN System Collection at the Science History Institute (donated by Rosen in two accessions, 2006 and 2016), but that is a physical archive and not yet digitised in a form suitable for keyword search.

A defensible *estimate* can nevertheless be made from the information that Rosen & Pauls do publish. A typical FORTRAN-66 unit-operation subroutine of the era, with its property-table interface and convergence guards, runs to 200–400 lines. Taking the midpoint:

Component	LOC (estimated)
58 unit-operation blocks × 300 LOC mean	∼17 000
System shell (preprocessor, executor, recycle, I/O, error handling)	∼ 8 000
PROPTY + VLE + INF (three auxiliary FORTRAN programs)	∼10 000
Total (estimated)	**∼35 000 LOC FORTRAN-66**

The figure above is the author's calibrated guess from the published architecture, not a primary citation. If the Science History Institute archive eventually yields a primary number, this appendix should be updated.

Comparison with the current state

Metric	FLOWTRAN (1977)	Choupo (2026)
Unit-operation block types	∼58 (incl. costing/profit)	10
Components in the public database	180	9
Activity-coefficient models	van Laar, NRTL, others	ideal, NRTL, Wilson
Distillation methods	matrix (rigorous) + shortcuts	Wang-Henke bubble-point
Recycle convergence	bounded Wegstein	bounded Wegstein
Parameter estimation	companion program (VLE fitting)	in-simulator Levenberg-Marquardt outer driver
Equipment sizing + costing	20+ blocks built in	two equipment + Guthrie costing
LOC (source)	∼35 000 FORTRAN-66 (est.)	∼10 000 C++17
Equivalent C++ LOC (FORTRAN-66 → modern C++ compresses ∼ 3×) ⇒ ∼12 000		10 000 (direct)
External dependencies	none (FORTRAN library)	none (C++ 17 standard library)
Licence	commercial (Monsanto), then CACHE-academic	GPL-3.0-or-later (open source)

In feature density per source line, Choupo is in the same ballpark as FLOWTRAN of fifty years ago. What FLOWTRAN had and Choupo does not (yet): a rigorous matrix-method distillation solver (`FRAKB`); rigorous absorption (`ABSBR`) and extraction (`EXTRC`) blocks; dedicated pump and compressor sizing/cost blocks; a profit-

analysis pass. Each of these is a candidate item for the roadmap.

What Choupo has and FLOWTRAN did not: an embedded modern parameter-estimation engine (`fitParameters`) inside the simulator rather than as a companion program; a Web-based GUI with flowsheet canvas, plots and a WASM solver; an open-source licence that lets a student modify any layer; auditable transparency in the manuals and the source.

What survives

The Rosen donation (Science History Institute, accessions 2006 and 2016) preserves the FLOWTRAN source as paper listings — specifically Box 1 Folder 17 (PROPTY, 1974–78), Box 1 Folder 18 (simulator core, 1986 and 1993), Box 2 Folder 4 (VLE, undated) — and as physical FORTRAN punch cards in Series V. The collection is meticulously curated for research access but has not been digitised: there is no machine-readable copy of FLOWTRAN in circulation. Copyright passed from Monsanto to Solutia (1997), to Pharmacia/Pfizer (2002), to the new Monsanto Agriculture, and on to Bayer's acquisition in 2018. Reviving FLOWTRAN as runnable code would require systematic OCR of the listings, transcription against the punch-card decks, a port to modern GFortran, and a copyright clarification with Bayer. Choupo does not attempt this revival — it is an independent re-implementation of the same architectural pattern in modern C++.

The fate is not unique

FLOWTRAN's outcome is the rule, not the exception, for chemical process simulators of the mainframe era. PACER (P. T. Shannon, Dartmouth, mid-1960s), CHESS (R. L. Motard, 1968 — eventually re-implemented commercially), PROCESS (Simulation Sciences of Los Angeles, 1966), GEMCS and CONCEPT (both reviewed alongside FLOWTRAN by Peters & Barker, 1974), and HYSIM (Hyprotech, Calgary, 1991): every foundational chemical-process simulator of the mainframe era either disappeared or was absorbed into a commercial product whose source remained closed. None was open-sourced before the original mainframe was decommissioned. The one notable exception is ASCEND (Carnegie Mellon, 1978–), an equation-oriented modelling environment that was open source from inception and is still actively developed half a century later, although with limited adoption outside its research community.

The historical lesson is sobering and motivating in equal measure: the architectural primitives that make modern process simulation work — streams, blocks, the outer recycle loop, the physical property file — were already in place in 1966; the lineage spans the entire C++ era and the FORTRAN-66 era both; and yet the machine-readable code that implemented those primitives in production has been disappearing for fifty years, one decommissioned mainframe at a time. Choupo, by being on git under GPL-3.0-or-later from day one, takes the bet that machine-readable persistence is itself a pedagogical goal — not a side effect.

Colophon. This manual was typeset in Latin Modern with the Choupo shared LaTeX preamble. Sources live in `docs/`; rebuild with `cd docs && make`. The current edition covers, in eight parts, the foundations of vapour–liquid equilibrium, the activity-coefficient models (ideal, NRTL, Wilson), the numerical methods (Newton 1-D and n-D, Levenberg-Marquardt, Wegstein, Nelder-Mead, RK4), the selected unit operations (isothermal flash with VL / LL / VLLE branches, CSTR, PFR, Gibbs reactor by free-energy minimisation, Wang-Henke distillation), the equipment-sizing + Guthrie-costing chain that closes the loop from a converged flowsheet to a EUR figure, and — — the dynamic simulation classes: batch reactors (isothermal + adiabatic + multi-reaction), Rayleigh batch distillation, continuous dynamic CSTR, and the feedback control layer with the ideal PID controller. Four binaries (`choupoSolve`, `choupoBatch`, `choupoCtrl`, `choupoProps`) ship with this edition, one per problem class. Appendix C places Choupo in its historical lineage, comparing it block-by-block with Monsanto's FLOWTRAN (1966–1993) following the description by Rosen & Pauls [72].

Part VIII — Dynamic simulation and process control

Parts I–VII solved steady problems: find the state $\mathbf{x}^\star$ such that $\mathbf{F}(\mathbf{x}^\star) = \mathbf{0}$. A whole other class of chemical-engineering questions asks instead: starting from a known state $\mathbf{Y}(0)$, what is $\mathbf{Y}(t)$ for $t > 0$? Batch reactors, simple batch stills, continuous reactors with finite hold-up and control loops — none of these have a steady state to find; they tell a story over time. Choupo ships two extra binaries for this class, `choupoBatch` (closed vessels with recipe events) and `choupoCtrl` (continuous flow with controllers), both built on the same shared library used by `choupoSolve`.

The integration scheme is the same explicit RK4 derived in Section 9 — only the state vector and the right-hand side change.

1 Time-stepping with a packed state vector

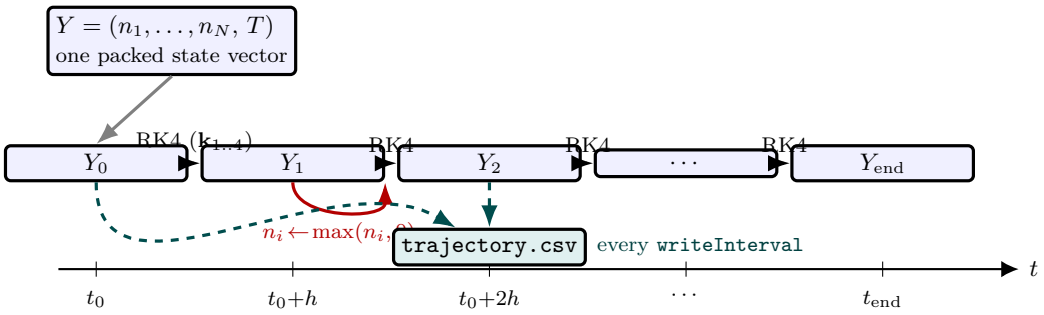

Figure 80: Time-marching with a packed state vector (Eq. 435). Every state variable of the vessel — the N component mole numbers *and* the temperature — is packed into one vector Y, and the generic RK4 stencil (Figure 45) advances it one fixed step $h =$ `deltaT` at a time along the instant ladder $Y_0 \to Y_1 \to \ldots$ After each step the mole numbers are projected onto the non-negative orthant ($n_i \leftarrow \max(n_i, 0)$, red) to absorb the small negative drifts RK4 produces near depletion. The solver does not store every instant: it *samples* the trajectory to `trajectory.csv` every `writeInterval` seconds (green). The same machinery serves the isothermal batch ($Y = \mathbf{n}$), the adiabatic batch ($Y = (\mathbf{n}, T)$, Section 2) and the dynamic CSTR (Section 4) — only $f(Y)$ changes.

> *What you'll learn.* The RK4 chapter wrote the PFR as $d\mathbf{x}/dV = f(\mathbf{x}, T)$ with V playing the role of the independent variable. In a closed batch vessel, the same RK4 integrates $dY/dt = f(Y, t)$ where Y packs *every state variable* of the vessel. The dimension of Y grows with the number of components and the energy balance, but the RK4 stencil is component-by-component identical.

For a single closed vessel with N components reacting under R reactions at isothermal conditions, the state is just the component mole numbers $\mathbf{n} = (n_1, \ldots, n_N)$ and the right-hand side is the standard rate-balance

$$\frac{dn_i}{dt} \;=\; V \sum_{r=1}^{R} \nu_{i,r}\, r_r(T, \mathbf{n}), \qquad i = 1, \ldots, N, \tag{434}$$

with r_r in kmol/(m$^3 \cdot$ s) and V the constant-density liquid volume. In adiabatic mode the temperature joins as state variable $N{+}1$ and the right-hand side gains an energy balance (Section 2). In a continuous dynamic CSTR (Section 4) the right-hand side gains inflow and outflow terms. In every case Choupo builds a single "packed state vector" $Y = (n_1, \ldots, n_N, T)$ (or just $\mathbf{n}$ in isothermal mode) and lets a generic RK4 step

$$\begin{aligned} \mathbf{k}_1 &= f(Y_n) \\ \mathbf{k}_2 &= f(Y_n + \tfrac{h}{2}\,\mathbf{k}_1) \\ \mathbf{k}_3 &= f(Y_n + \tfrac{h}{2}\,\mathbf{k}_2) \\ \mathbf{k}_4 &= f(Y_n + h\,\mathbf{k}_3) \\ Y_{n+1} &= Y_n + \tfrac{h}{6}\big(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\big) \end{aligned} \tag{435}$$

do the work. The step size h is `deltaT` in `controlDict`; output is sampled to `trajectory.csv` every `writeInterval` seconds.

The mole numbers are projected onto the non-negative orthant after each step ($n_i \leftarrow \max(n_i, 0)$) to absorb the small negative drifts that RK4 produces near depletion; an early-stop on $|\mathbf{n}| \to 0$ keeps the still chapter (Section 3) from dividing by zero in its bubble-T call.

2 Adiabatic batch reactor: coupled mass + energy balance

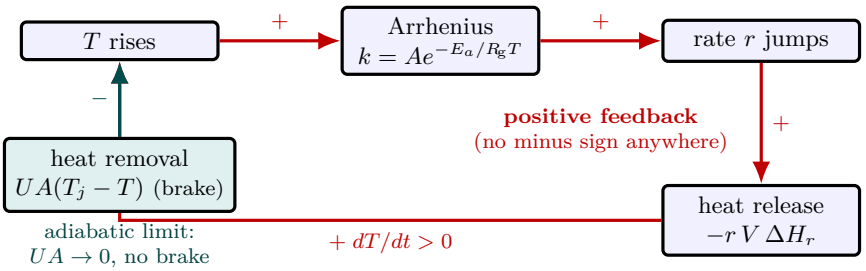

Figure 81: The thermal-runaway feedback loop of an adiabatic batch reactor. A rise in T lifts the Arrhenius rate constant k exponentially, which raises the reaction rate r, which releases more heat, which raises T further — every arrow carries a "+", so the loop is *positive* feedback with no restoring term. The only brake is heat removal $UA(T_j - T)$ (green); in the adiabatic limit $UA \to 0$ it vanishes and, once the "thermal Jacobian" $J_{T,T} \propto E_a/(R_g T^2 \sum_i n_i C_{p,i})$ exceeds the (here zero) loss term, the system has no stable steady state and *must* run away. The case `batch02_adiabatic` sits on this side: $\sim 200\,\mathrm{s}$ of quiescence while r is tiny, then the Arrhenius factor catches and T climbs $345 \to 482\,\mathrm{K}$ in $\sim 100\,\mathrm{s}$.

> *What you'll learn.* Adding the energy balance to a batch reactor is what gives the classical "thermal runaway" picture: a mild exotherm that lifts T at $\sim 1\,\mathrm{K/s}$ in the early phase, kinetics that double every $\sim 10\,\mathrm{K}$, and a self-amplifying ascent until the reactor empties of reactants or hits a heat-removal limit. Choupo integrates the balance trivially because RK4 already supports a packed state vector — only the right-hand side $f(Y)$ has to know about T.

The energy balance

For a perfectly mixed, constant-volume, well-insulated vessel (adiabatic limit, no jacket), the per-mole heat capacity of the liquid mixture is $\sum_i n_i\, C_{p,i}^{L}(T)$ in kJ/K (with n_i in kmol and C_p in J/(mol K) — the factor of 1000 cancels because of the kmol $\leftrightarrow$ mol ratio). An exothermic reaction releases $-\Delta H_r$ per mole of limiting reactant consumed. The energy balance becomes:

$$\Big(\sum_{i=1}^{N} n_i\, C_{p,i}^{L}(T)\Big)\,\frac{dT}{dt} \;=\; -\sum_{r=1}^{R} r_r(T,\mathbf{n})\,V\,\Delta H_r. \tag{436}$$

The sign convention is the standard one: $\Delta H_r < 0$ for an exothermic reaction, so the RHS is positive and $dT/dt > 0$. The heat-of-reaction is taken from the optional `dH_rxn` entry of each named reaction in `constant/reactions` (units J per mol of limiting reactant), and is assumed constant with T over the integration range — a Kirchhoff correction would couple ΔH_r to the Cp polynomials and is scheduled for a later version.

The full state derivative is, packed:

$$Y = (n_1, \ldots, n_N, T), \qquad f_i(Y) = V \sum_r \nu_{i,r}\, r_r(T,\mathbf{n}), \qquad f_{N+1}(Y) = \frac{-\sum_r r_r\, V\,\Delta H_r}{\sum_i n_i\, C_{p,i}^{L}(T)},$$

with C_p evaluated from the polynomial `liquidHeatCapacity` block in each component file.

The runaway signature

A linear stability analysis of (434) + (436) around a stationary point $(\mathbf{n}^*, T^*)$ gives a Jacobian with the block

$$J_{T,T} \;=\; \frac{\partial}{\partial T}\Big[\frac{-r(T,\mathbf{n}^*)\,V\,\Delta H}{\sum_i n_i^*\, C_{p,i}}\Big] \;\propto\; \frac{E_a/RT^{*2}}{\sum_i n_i^*\, C_{p,i}^{L}(T^*)}$$

for first-order Arrhenius kinetics. When this positive "thermal Jacobian" exceeds the heat-loss term, the system has no stable steady state and *must* runaway. The pedagogical case `tutorials/batch/reactor/batch02_adiabatic` sits on the runaway side: $T_0 = 320\,\mathrm{K}$, $k(T_0) \approx 3.7 \times 10^{-4}\,\mathrm{s}^{-1}$, $\tau_0 \approx 2700\,\mathrm{s}$. The first $\sim 200\,\mathrm{s}$ look quiescent

(the small r feeds back a $\sim 0.01\,\mathrm{K/s}$ heat rate), then the Arrhenius factor catches and the reactor runs from 345 to 482 K between $t = 230\,\mathrm{s}$ and $t = 330\,\mathrm{s}$.

The maximum adiabatic temperature rise from full conversion is $\Delta T_{\mathrm{ad}} = |\Delta H|/\bar{C}_p$ where $\bar{C}_p$ is a molar-weighted average; for the 15 kJ/mol exotherm of `batch02_adiabatic` and an average Cp of $\sim 92\,\mathrm{J/(mol\,K)}$ (the harmonic of pure ethanol's 112 and water's 75), this gives $\Delta T_{\mathrm{ad}} \approx 163\,\mathrm{K}$ — which is what the simulator reports to within 1 K.

Multi-reaction case

When $R > 1$ reactions share the vessel, both (434) and (436) simply sum over r. The matrix $(\nu_{i,r})$ of stoichiometric coefficients does the species bookkeeping; each reaction's own r_r, E_a, A and ΔH_r feed in independently. Choupo's `BatchReactor` holds a `std::vector<ReactionSpec>` and walks it inside `derivatives_`.

> **Runnable case.** batch/batch03_consecutive Consecutive A → B → C at 350 K with $k_1 = 10\,k_2$. The intermediate B accumulates and then decays as the second reaction overtakes the first; the maximum $n_B/n_{A,0} = 0.669$ at $t_{\max} = 200\,\mathrm{s}$ matches the closed-form $(k_1/k_2)^{k_2/(k_2-k_1)}$ and $\ln(k_1/k_2)/(k_1 - k_2)$ to 0.1 %.

3 Rayleigh batch distillation

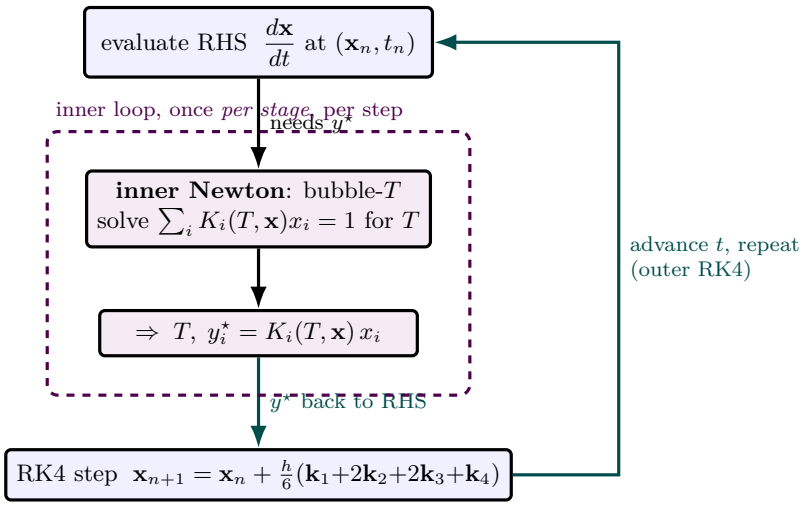

Figure 82: The nested-Newton pattern that recurs in every dynamic VLE problem, shown for Rayleigh batch distillation. The *outer* loop is RK4 marching the residue composition $\mathbf{x}$ in time (blue). But each RK4 stage needs the equilibrium vapour $y_i^\star = K_i(T,\mathbf{x})x_i$, and that requires the boiling temperature — so a full *inner* Newton iteration (violet) solves the bubble-point condition $\sum_i K_i(T,\mathbf{x})x_i = 1$ for T before the right-hand side can even be evaluated (`BubblePoint::compute` used as a black box). The inner solve runs once *per stage*, four times per RK4 step; its result feeds the outer step, time advances, and the whole thing repeats. The same Newton-inside-RK4 structure drives the dynamic flash and every moving-VLE unit op.

What you'll learn. The classical Rayleigh distillation strips a vessel of its more volatile components at a rate F_{vap} [kmol/s] of vapour removed; mass balance fixes the residue composition over time as a function of the vapour-liquid equilibrium $y^\star = K(T,\mathbf{x})\,\mathbf{x}$. Choupo integrates the corresponding ODE with RK4 using `BubblePoint::compute` as a black-box bubble-T solver inside the right-hand side — a nested-Newton pattern that recurs in every dynamic VLE problem.

The mass balance

For a closed pot at constant pressure P from which vapour is removed at molar rate F_{vap}, component i obeys

$$\frac{dn_i}{dt} \;=\; -\,F_{\text{vap}}\,y_i^\star(T,\mathbf{x}), \qquad \mathbf{x} = \mathbf{n}/\sum_j n_j, \qquad y_i^\star = K_i(T,\mathbf{x})\,x_i. \tag{437}$$

The temperature T is determined algebraically each instant by the bubble-point condition $\sum_i K_i(T,\mathbf{x})\,x_i = 1$; the RHS of (437) therefore depends on $\mathbf{x}$ through both the direct x_i in $y_i^\star$ and the implicit $T(\mathbf{x})$ in K_i. The classical *Rayleigh equation* is obtained by dividing through to eliminate t:

$$\frac{d\mathbf{x}}{dL} \;=\; \frac{\mathbf{x} - \mathbf{y}^\star(T,\mathbf{x})}{L}, \qquad L = \sum_i n_i,$$

which integrates by quadrature for ideal binaries (constant relative volatility α) to

$$\ln\frac{L}{L_0} \;=\; \frac{1}{\alpha-1}\left[\ln\frac{x}{x_0} \;+\; \alpha\,\ln\frac{1-x_0}{1-x}\right].$$

Choupo integrates (437) directly with RK4 rather than reaching for the closed form — the same code runs ideal and NRTL cases without change.

The warm-seeded bubble-T inside the integrator

Each RK4 stage evaluation has to call `BubblePoint::compute` on the current $\mathbf{x}$. Choupo caches the previous step's converged T_{bub} and feeds it back as the initial guess for the next call. Because $\mathbf{x}$ changes

slowly (the per-step composition jump is $O(F_{\text{vap}}\,h\,/\,L)$, typically $< 10^{-4}$) the warm-seed converges in 2–3 Newton iterations. This is the same warm-start pattern used by every multi-step ODE integrator that embeds an algebraic solver.

The pattern surfaced a long-standing bug in `solver::newton1D`: when the warm seed is already within tolerance the routine used to take one zero-length Newton step, find $\Delta x \approx 0$, and then *bisect* into the centre of the bracket because of an `x_new == x_hi` edge condition. The fix — check $|f| <$ tol before any state mutation — ships from.

Azeotropes from below: the Rayleigh barrier

The `still02_ethanol_water_azeo` tutorial starts at $x_{\text{eth}} = 0.40$ (below the 0.894 minimum-boiling azeotrope at 1 atm). In this regime $y^{\star}_{\text{eth}} > x_{\text{eth}}$, so ethanol leaves faster than water, and the pot composition drifts *monotonically toward pure water*, ending at $x_{\text{eth}} \approx 3 \times 10^{-11}$ with $T \to 372.4\,\text{K}$ (within 1 K of pure water's T_b). A Rayleigh still simply cannot cross the azeotrope from below; it can only walk further away from it. Crossing the azeotrope requires a column with reflux, an entrainer (extractive distillation), or a pressure swing.

> **Runnable case.** batch/still01_benzene_toluene 50/50 benzene/toluene at 1 atm, $F_{\text{vap}} = 1.5 \times 10^{-6}$ kmol/s for 600 s. Residue at 90 % distilled: $x_{\text{benzene}} = 0.082$. Closed-form Rayleigh with the mean Antoine-based $\alpha \approx 2.4$ predicts 0.080 — agreement at the 2 % level.

Batch rectification: the pot grows a column

The Rayleigh still wastes what the vapour knows: $y^{\star}$ is richer than the pot, but every mole of it leaves. The classical batch rectifier (`model rectifier;` on the same `batchStill`) inserts N ideal equilibrium stages between the pot and a *total* condenser, and returns a fraction of the condensate as reflux. With stage and condenser hold-up neglected — the standard textbook idealisation — the column is *quasi-steady*: at every instant it is an algebraic system riding on the slow pot ODE.

Let V be the boilup leaving the pot (the dict's `F_vap` in this model), R the reflux ratio. A total condenser splits V into distillate product $D = V/(R+1)$ and reflux $L = R\,D$. The pot then loses *product only*:

$$\frac{\mathrm{d}n_i}{\mathrm{d}t} \;=\; -\,D\,x_{D,i}(t), \tag{438}$$

with $x_D(t)$ solved at each instant from the cascade

$$\begin{aligned} y_0 &= K(T_{\text{bub}}(x_B))\;x_B && \text{(pot vapour)} \\ V\,y_{s-1} &= L\,x_s + D\,x_D, && s = 1\ldots N \quad \text{(section balance)} \\ y_s &= K(T_{\text{bub}}(x_s))\;x_s && \text{(stage equilibrium)} \\ x_D &= y_N. && \text{(total condenser)} \end{aligned} \tag{439}$$

The unknown is the *vector* x_D; Choupo iterates the closure $x_D \leftarrow y_N$ and declares the single residual $\max_i |y_{N,i} - x_{D,i}|$, with the section balances holding algebraically at every iterate (so $\sum_i x_{s,i} = 1$ follows from $\sum_i y_i = \sum_i x_{D,i} = 1$ — nothing is ever renormalised into looking converged). Two numerical honesty notes, both learnt the hard way: the closure tolerance sits at 10^{-7}, just *above* the 10^{-8} Newton floor of each stage's bubble-T (a 10^{-10} demand can never close — it chases the stages' round-off); and an infeasible iterate (a negative stage composition, which the natural first guess $x_D = y_0$ produces by over-enriching) is projected onto the cascade's own last vapour, which corrects toward the reachable set from either side.

Two limits anchor the model. With $R = 0$ or $N = 0$ no liquid runs down the column, $x_D = y_0$ and eq. (438) degenerates to the Rayleigh still exactly — the regression suite pins the equivalence to the last digit. As $R \to \infty$ at fixed N the cascade approaches total reflux, where the Fenske relation for an ideal binary gives

$$\frac{x_D}{1-x_D} \;=\; \alpha^{\,N+1}\,\frac{x_B}{1-x_B} \tag{440}$$

(N stages + the pot are $N{+}1$ equilibrium contacts). For benzene/toluene at $x_B = 0.5$ with $\alpha = 2.35$–2.6 the band is $x_D = 0.968$–0.979; the solver at $R = 200$, $N = 3$ lands at 0.9765 — inside, computed by a route (per-stage Antoine bubble-T) that shares nothing with eq. (440).

At *finite* reflux the constant-R policy trades purity for time: as the pot strips, y_0 leans and the whole cascade leans with it, so $x_D(t)$ drifts downward while the receiver accumulates the run average — two different

compositions, deliberately both reported (the trajectory carries the instantaneous $x_{D,i}$, R and D; the receiver holds the mean). The complementary policy (`refluxPolicy constantComposition`) *holds* x_D of a declared light key at a target by solving $R(t)$ each step: a *bracketed* solve of $f(R) = x_{D,\mathrm{LK}}(R) - x_D^{\mathrm{target}}$ over $[0, R_{\max}]$ on a feasibility map — never a bare Newton, because $x_D(R)$ can lose monotonicity (an azeotrope) and the solve must refuse an ambiguous branch rather than jump one silently. Three infeasibilities get three distinct diagnostics: a target *below* the $R = 0$ floor (even zero reflux over-purifies); a target unreachable from the charge at $R_{\max}$ (an azeotrope caps the cascade at *any* reflux — ethanol/water refuses a 0.95 target against its 0.894 barrier by computed VLE, not by a message); and the mid-campaign exhaustion, where the leaning pot drags the reachable set below the target and the run announces `onRefluxLimit stopDistillation`: $D = 0$, the pot inventory freezes, and the campaign idles *visibly* to `endTime` with the event in the trajectory and the KPIs — never a magic termination. $R(t)$ is a control input updated once per step at the committed state (first order in Δt, announced as such). No reboiler duty is reported by this model: V alone does not prove an energy balance, and a $V \cdot \lambda$ shortcut would be a decorative number.

> **Runnable case.** batch/still04_rectifier_benzene_toluene The still01 pot grown a column: $N = 3$, $R = 3$, $V = 1.5 \times 10^{-6}$ kmol/s at 1 atm. Initial distillate $x_D = 0.949$ (vs. 0.71 for the raw pot vapour) drifting to 0.906 over 600 s; the receiver accumulates 0.931; the pot loses exactly $D\,t = 0.225$ mmol — three quarters of the boilup refluxes.

> **Runnable case.** batch/still05_rectifier_constant_xD The same column under the second policy: x_D held at 0.9300 ± 10^{-4} while R climbs $2.0 \to 7.9$; at $t = 863$ s the ceiling $R_{\max} = 8$ can no longer reach the target and distillation stops — the receiver ends with the WHOLE cut on-spec at 0.930, the policy's selling point over still04's drifting purity.

Campaign accounting: the material ledger and the mass balance

Batch vessels are materially CONNECTED: a recipe `transfer` is a discrete material edge, a `dischargeTo` route a continuous one. Choupo records every such edge as a structured *ledger* entry — t_{start}, t_{end}, source, destination, the per-component amount moved, and the transported enthalpy

$$H_{\mathrm{moved}} \;=\; \sum_{\text{packages } p} n_p\, h(T_p, x_p)\,, \tag{441}$$

integrated *package by package at each package's own temperature* (elements datum). A single end-temperature stamped on an accumulated amount would destroy exactly the information an energy balance needs — a still's condensate leaves at a T that climbs all campaign. The timeline and the Gantt view are *projections* of this ledger, never independent records. A vessel that sheds material with no declared receiver is an EXTERNAL outlet — an unrouted product stream, recorded and named, never a hidden loss.

The transported enthalpy carries an explicit *validity*: a package containing a species with no formation datum cannot be priced, and the record then withholds H_{moved} entirely and names the species. Validity is *monotonic in time* — once one package was unpriceable the integral has lost a parcel forever, and a later priceable package must never resurrect it; an emitted enthalpy that silently dropped part of its own history would be worse than no number at all. The campaign reports the census (how many ledger records carry a valid H) as a first-class verdict.

> **Runnable case.** batch/still06_ledger_mixed_validity Two stills poison their distillate records in opposite temporal orders (unpriceable → priceable and the reverse); both records must end invalid, naming the datum-less species, with no partial integral emitted.

The material ledger has an energy twin. Each vessel contributes *energy records*: one per segment of constant physics — a recipe `setParameter` or a material charge closes the running segment, because the record's $h_i(T)$ belong to one temperature and the state difference it prices must be purely reactive. The isothermal reactor's jacket duty is the flagship: for a closed vessel at constant pressure the first law gives $Q = \Delta H$ over the segment, and on the elements datum Hess's law collapses the per-reaction story into a state difference,

$$Q \;=\; \sum_j \Delta H_{\mathrm{rxn},j}\, \Delta\xi_j \;=\; \sum_i \Delta n_i\, h_i(T), \tag{442}$$

so the integral is *exact* — computed from committed mole numbers, never a quadrature riding on the integrator's step size. When a lumped species carries no formation data but the reaction declares its

heat, the duty falls back to the announced-override branch: extents are solved from the stoichiometry ($N\boldsymbol{\xi} = \Delta\mathbf{n}$, refused aloud if the reaction set is linearly dependent) and priced per reaction. Neither datum nor declaration — the record is invalid and names the reaction.

Material transfers respect the same first law: a recipe charge solves its mix temperature from *enthalpy equality* on the elements datum,

$$H(\mathbf{n}_1 + \mathbf{n}_2,\, T_{\mathrm{mix}}) \;=\; H(\mathbf{n}_1, T_1) + H(\mathbf{n}_2, T_2), \tag{443}$$

by a 1-D Newton warm-started at the molar average — so charging is a material act, never a thermal one, and internal transfers cancel *exactly* in the campaign sum. Heat-capacity differences matter: a hot ethanol charge into cold water lands ∼4 K above the equal-C_p molar average. When a species cannot be priced the charge falls back to the molar average, *announced* — and that named occasion is enough to withhold the balance verdict below.

The campaign *energy balance* follows the same honesty rule as the mass balance, with one addition: it only claims a verdict when *every* piece is ledgered. A vessel type whose duty physics has not landed yet, an instantaneous `setParameter` temperature jump, a charge that fell back to the molar-T average, or an external outlet whose transported enthalpy could not be priced makes the balance report UNAVAILABLE, naming each missing piece — never a decorative closure. When it does close, it closes over everything that moved: $\sum_{\mathrm{vessels}} \Delta H = Q_{\mathrm{ledger}} - H_{\mathrm{external\,out}}$, with the closure scaled by the total enthalpy that MOVED ($\sum |\Delta H_{\mathrm{vessel}}|$), so a pure transfer campaign whose signed sum cancels to zero still reports its closure honestly instead of a 0/0.

> **Runnable case.** batch/batch08_isothermal_duty The override branch end-to-end: lumped $A \to B$ with a declared $\Delta H_{\mathrm{rxn}} = -50\,\mathrm{kJ/mol}$ prices the jacket duty $E = \Delta H\,\Delta\xi = -49.88\,\mathrm{kJ}$ against the analytic first-order decay, while the balance stays unavailable naming the datum-less species.

> **Runnable case.** batch/recipe04_charge_enthalpy The H-equality charge: hot ethanol into cold water mixes to 317.06 K (the enthalpy-true temperature, ∼4 K above the molar average), and the campaign balance CLOSES across the transfer at 2×10^{-12} — the first law surviving a recipe action.

The still's duties follow the same discipline: no invented $V\lambda$, only phase-enthalpy differences of the *actual* packages. For the Rayleigh pot the first law gives the reboiler directly,

$$Q_{\mathrm{reb}} \;=\; \Delta H_{\mathrm{pot}} + \sum_p n_p\, h^{\mathrm{vap}}(T_p, \mathbf{y}_p), \qquad Q_{\mathrm{cond}} \;=\; -\sum_p n_p \left[h^{\mathrm{vap}} - h^{\mathrm{liq}}\right](T_p), \tag{444}$$

each shed package priced at its own hand-off temperature — so the reboiler integral is a state difference plus a package sum, exact. The rectifier's condenser is the physical one — it condenses the whole boilup V, not the product D, at the top-stage temperature — and its reboiler closes the box first law as the exact residual. With that, a full react-then-distil campaign closes its energy balance end to end: reactor duty, charge, distillation and the unrouted product all on one elements-datum ledger.

> **Runnable case.** batch/recipe01_react_then_distill The showcase, now energy-closed: esterification duty, the reactor-to- still charge, reboiler/condenser duties and the external distillate balance to 6×10^{-16}.

An instantaneous `setParameter` temperature change is an energy *intervention*, and the ledger prices it as such: an `impulse` record with $E = H(\mathbf{n}, T_{\mathrm{new}}) - H(\mathbf{n}, T_{\mathrm{old}})$ on the same elements datum — a state difference including any phase leg, never a $\sum nC_p\Delta T$ shortcut when enthalpy data exist. A staged temperature programme thus reads off the ledger as its full story: reaction segments at each held temperature, with the jump energies in between, and the balance closes across all of them.

The ledger closes into *utilities*: every valid duty record carries the process temperature its service must beat — the segment's *worst-case* end (the hottest pot state for a reboiler, the coldest condensing temperature for a coolant) — and the campaign allocates each record to the lowest-grade catalogue utility that still serves it, the same tightest-margin rule the steady allocation uses, through one shared picker. Mass consumption uses the catalogue's phase-aware duty per kilogram (condensing steam is latent heat, never a $c_p\Delta T$), and the campaign cost is simply $\sum |E| \times$ price. A record no utility can serve is *listed*, never silently dropped.

The batch crystalliser's latent duty is another exact state difference: $Q = -\Delta H_{\mathrm{cryst}}\,\Delta n_{\mathrm{cryst}}$ with $n_{\mathrm{cryst}} = V\rho_c k_v \mu_3 / M$ read off the third moment, and $\Delta H_{\mathrm{cryst}}$ resolved through the *same* shared resolver the steady

crystalliser uses (ion-derived `dissolutionEnthalpy`, the molecular solution tier, or a declared constant that confesses itself) — the source travels in the record's basis, and steady and batch can never diverge on the number. The vessel-enthalpy half is an honestly *named* gap: a magma is dissolved solute plus crystal, and until inventories price on the solid rung the campaign balance reports it unavailable, verbatim.

The campaign closes on inventories. Each vessel reports its ALL-phase per-component inventory (a crystalliser adds its crystal magma $V\rho_c k_v \mu_3/M$ to the dissolved holdup — without it, crystallisation reads as mass loss), and

$$m_0 \;=\; m_{\text{final}} \;+\; m_{\text{external}}, \tag{445}$$

in kilograms via the component molar masses, because *reactions conserve mass, not moles.* For reacting campaigns the sharper invariant is the ELEMENTS: formulas are parsed against the periodic table (a lumped teaching species whose "formula" is `A` makes no elemental claim) and each element must close independently — a chemically wrong stoichiometry can hide inside a closed total mass, but not inside a closed oxygen balance. Two honest findings shipped with this machinery: catalogue molar masses from mixed sources drift from the stoichiometry at the 10^{-5} level ($\sum_j \nu_j M_j \neq 0$ exactly), reported as a curation note; and a genuine violation reports the run NON-CONVERGED — a leak is a failure, not a log line.

Runnable case. batch/recipe01_react_then_distill The react-then-still campaign: one discrete transfer at $t = 400\,\text{s}$, the still's unrouted distillate recorded as an external outlet (0.889 kg), and the campaign closing to 10^{-15} with every element independently balanced.

4 Continuous dynamic CSTR with finite hold-up

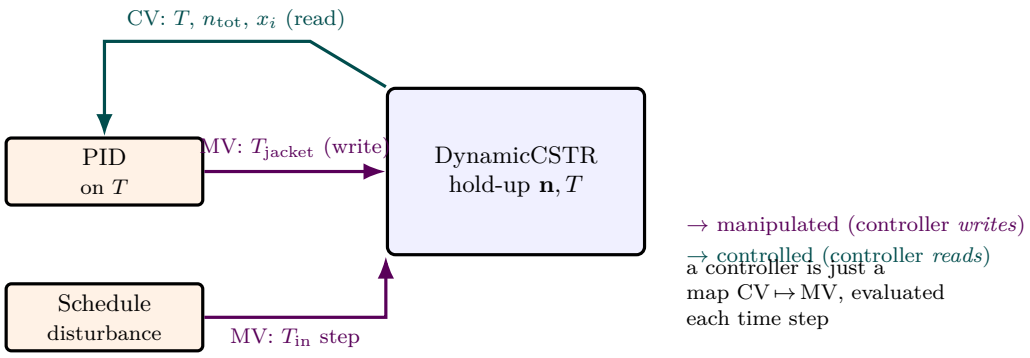

Figure 83: How a controller couples to a dynamic unit op, the Choupo `Controller` contract. The unit (`DynamicCSTR`) exposes a set of *controlled variables* the controller may read — here the reactor temperature T, total holdup, and composition x_i (green, "measurements") — and a set of *manipulated variables* a controller may write each step ($T_{\text{jacket}}, T_{\text{in}}, F_{\text{in}}$; violet). A `PIDController` reads T and writes T_{jacket} to hold setpoint; a degenerate `ScheduleController` ignores the measurement entirely and writes a piecewise-constant MV (e.g. a $\pm 15\,\text{K}$ step in T_{in}) to inject scripted disturbances. Both plug into the *same* infrastructure because a controller is nothing more than a map (controlled variables) ↦ (manipulated variables) evaluated once per time step, between RK4 steps.

> *What you'll learn.* A continuous stirred-tank reactor with finite liquid hold-up (*not* the steady CSTR of Section 5) has the same mass balance as a batch reactor plus convective in/out terms. Closed-form steady-state results recover for $dn_i/dt = 0$; the transient between two steady states is governed by the residence time $\tau = n_{\text{total}}/F_{\text{in}}$. This unit is the basic vehicle for studying control: it is non-trivial enough to have interesting closed-loop behaviour, simple enough that students can hand-derive the open-loop response.

Mass and energy balance

For a constant-volume liquid reactor with continuous inlet at $(F_{\text{in}}, T_{\text{in}}, \mathbf{z}_{\text{in}})$ and outlet at composition matching the tank ($x_i = n_i/\sum_j n_j$), with R reactions and a jacket exchanging heat at rate $Q_{\text{j}} = UA\,(T_{\text{jacket}} - T)$ in W (so $Q_{\text{j}}/1000$ in kJ/s for the units used in Section 2):

$$\frac{dn_i}{dt} = F_{\text{in}}\, z_{\text{in},\,i} \;-\; F_{\text{out}}\, x_i \;+\; V \sum_r \nu_{i,r}\, r_r(T, \mathbf{n}), \tag{446}$$

$$\Big(\sum_i n_i\, C^{L}_{p,i}\Big)\frac{dT}{dt} = F_{\text{in}}\, \bar{C}_{p,\text{in}}\,(T_{\text{in}} - T) \;+\; \frac{UA}{1000}\,(T_{\text{jacket}} - T) \;-\; \sum_r r_r\, V\, \Delta H_r. \tag{447}$$

With constant volume (incompressible liquid) the overall mass balance fixes $F_{\text{out}} = F_{\text{in}}$ — there is no accumulation of mass — but the *component* balance does accumulate because composition x_i tracks n_i.

The state is again packed as $Y = (n_1, \ldots, n_N, T)$ and RK4 (435) integrates it directly. Choupo's `DynamicCSTR` exposes T_{jacket}, T_{in} and F_{in} as *manipulated variables*: a controller or a recipe event can write to any of them at run time. The current T, n_{total} and x_i are exposed as *controlled variables*.

Open-loop steady state

Setting (446)–(447) to zero gives the steady state $(\mathbf{n}^*, T^*)$ at the operating point. For first-order kinetics in the limiting reactant A, the component balance simplifies to

$$F_{\text{in}}\,(z_{\text{in},A} - x_A^*) \;=\; k(T^*)\, x_A^*\, n_{\text{total}} \;\Longrightarrow\; x_A^* \;=\; \frac{z_{\text{in},A}}{1 + k(T^*)\,\tau}, \qquad \tau = n_{\text{total}}/F_{\text{in}}.$$

The classical CSTR formula recovers exactly — the dynamic simulator is the steady CSTR *plus* a transient envelope.

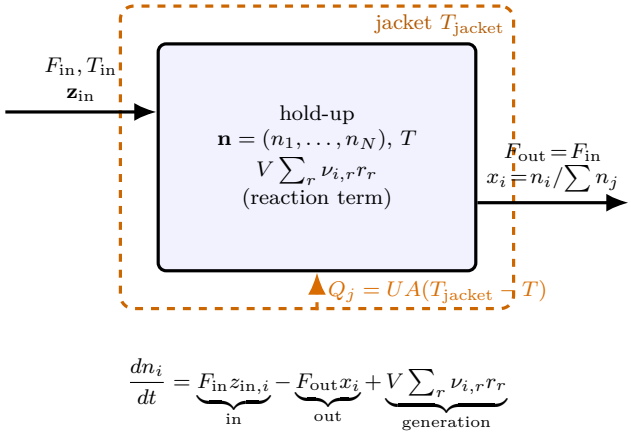

Figure 84: The continuous dynamic CSTR as a single 0-D well-mixed hold-up cell (`DynamicCSTR`). Unlike the *steady* CSTR of Section 5, the contents *accumulate*: the state $\mathbf{Y} = (\mathbf{n}, T)$ evolves in time. Three currents cross the boundary of each component balance — convective inflow $F_{\text{in}} z_{\text{in},i}$, convective outflow $F_{\text{out}} x_i$ at the *tank* composition (perfect mixing), and chemical generation $V \sum_r \nu_{i,r} r_r$ — and the jacket adds a heat current $Q_j = UA(T_{\text{jacket}} - T)$ to the energy balance. Constant liquid volume forces $F_{\text{out}} = F_{\text{in}}$, so total mass does not accumulate, yet *composition* does because x_i tracks n_i. The manipulated variables a controller may write $(T_{\text{jacket}}, T_{\text{in}}, F_{\text{in}})$ are exactly the labelled inlet and jacket quantities.

Time constants and what control can do

The residence-time constant of the mass balance is τ; the thermal time constant is $\tau_T \approx \sum_i n_i C_p / (F_{\text{in}} \bar{C}_p + UA/1000)$. When the jacket coupling UA is comparable to $F_{\text{in}} \bar{C}_p$, the thermal response is order-τ — fast enough that a well-tuned PID can hold T to within a few mK of setpoint over disturbances of a few K (see Section 5).

> **Runnable case.** ctrl/ctrl01_cstr_temp_control Mildly exothermic compA → compB at 350 K setpoint. Open-loop the reactor settles at the natural steady state 320.8 K (just above the cool inlet at 320 K because of the mild exotherm). A PID on T_{jacket} with $K_p = 4$, $K_i = 0.04$, $K_d = 0$ drives T to 350 K and holds it to within 10^{-4} K at $t = 1500$ s.

5 Feedback control: the ideal PID, derivative-on-PV, anti-windup

What you'll learn. The PID controller is one of the oldest objects in process control, the workhorse for over 90 % of single-input single-output loops in industry. The mathematics is two paragraphs. But the *implementation* choices — whether the derivative term sees the error or the process variable, what to do when the actuator saturates, how to discretise the integral — decide whether the closed loop behaves nicely or oscillates pathologically. This section derives the ideal form and explains the two implementation choices Choupo makes (`derivative-on-PV` and `clamping anti-windup`).

The ideal form

Let SP be the set point, $\mathrm{PV}(t)$ the measured process variable, $e(t) = \mathrm{SP} - \mathrm{PV}(t)$ the error, and $u(t)$ the controller output (the manipulated variable). The *ideal-form* PID is

$$u(t) \;=\; u_{\mathrm{bias}} \;+\; K_p\, e(t) \;+\; K_i \int_0^t e(\tau)\, d\tau \;-\; K_d\, \frac{d\,\mathrm{PV}(t)}{dt}. \tag{448}$$

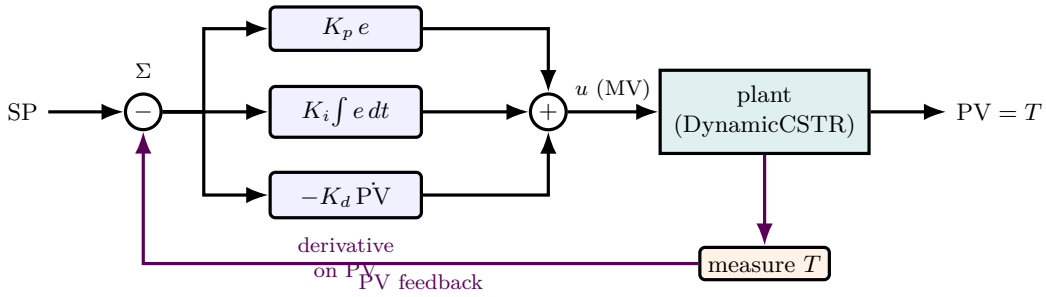

Figure 85: The ideal-form PID loop Choupo's `PIDController` implements (`src/control/PIDController`). The summing junction forms the error $e = \mathrm{SP} - \mathrm{PV}$; three parallel terms act on it — proportional ($K_p e$, the present), integral ($K_i \int e\, dt$, the accumulated past, killing offset) and derivative (the anticipated future) — and their sum is the manipulated variable u driving the plant (here T_{jacket} of the dynamic CSTR). The measured T is fed back (violet) to close the loop. Two Choupo choices are visible: the derivative tap reads PV directly, *not* e, so an operator's set-point step produces no derivative "kick"; and the integral term is governed by the anti-windup gate of Figure 86.

The *bias* u_{bias} is the controller's idle output when $e = 0$ — typically chosen to be the operating-point MV. Each of the three terms has a clear physical meaning:

- $K_p\, e$ — **proportional**: react to the present error. Larger K_p means faster response and tighter tracking but more aggressive transients and a tendency to overshoot.
- $K_i \int e\, dt$ — **integral**: eliminate any residual offset. In a P-only loop, a non-zero u_{bias} mismatch yields a permanent steady-state error (called *offset*); the integral keeps accumulating error until offset is removed. Larger K_i removes offset faster but adds phase lag, destabilising fast loops.
- $-K_d\, d\mathrm{PV}/dt$ — **derivative**: anticipate where the process is heading. Larger K_d damps oscillation but amplifies measurement noise.

Why derivative-on-PV, not derivative-on-error

A naive form puts $de/dt = d\mathrm{SP}/dt - d\mathrm{PV}/dt$ in the derivative term. When the operator changes SP, $d\mathrm{SP}/dt$ is a Dirac impulse, and the controller fires a derivative "kick" of the same shape onto u — a momentary spike that the actuator sees as a step change of theoretical magnitude $-K_d/0^+$. For a fast valve or heater this can be a real disturbance.

The *derivative-on-PV* form drops the SP contribution:

$$u_{\mathrm{D}}(t) \;=\; -\,K_d\, \frac{d\,\mathrm{PV}(t)}{dt},$$

which still damps the *response* to disturbances and load changes (because they all show up as PV changes) but makes set-point changes show up only through the proportional and integral terms. No spike, no SP-kick. Choupo's `PIDController` implements this form.

Anti-windup

The integral term is unbounded in (448). When the actuator saturates — the output u hits a physical limit, e.g. the cooling-jacket cannot go below $T_{\text{jacket,min}}$ — the controller *commands* an even more extreme u, but the process responds as if it were saturated. The error stays large, the integral keeps accumulating, and when the constraint clears the controller has wound up so much integral that the recovery overshoot is enormous.

Choupo uses the simplest robust scheme, *clamping anti-windup*. At each step, compute the unclamped output u_{unc} from (448), then clamp it:

$$u \;=\; \text{clip}\big(u_{\text{unc}},\; u_{\min},\; u_{\max}\big),$$

and **advance the integral only when the unclamped output is inside the bounds**:

$$I_{n+1} \;=\; \begin{cases} I_n + e_n\,h, & u_{\min} \le u_{\text{unc},n} \le u_{\max} \\ I_n, & \text{otherwise.} \end{cases} \tag{449}$$

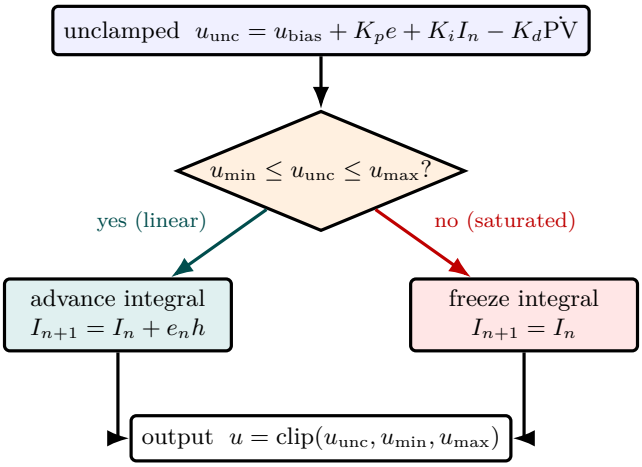

Figure 86: Clamping anti-windup, the integral gate inside Choupo's PID. Each step computes the *unclamped* output from the full ideal-form law. If it lands within the actuator limits the loop is operating linearly and the integral is advanced normally ($I_{n+1} = I_n + e_n h$, green). But if it has saturated — the cooling jacket cannot go below $T_{\text{jacket,min}}$, say — the integral is *frozen* (red) so it cannot "wind up" a huge reserve during the constrained phase that would then drive an enormous recovery overshoot when the constraint clears. Either way the delivered u is the clipped value. This is the simplest robust scheme; back-calculation is the more elaborate alternative deferred in the text.

This prevents the integral from growing during saturated phases. More sophisticated alternatives (back-calculation, conditional integration with a track-back constant) exist; for the tutorials (449) suffices.

Discretisation

For a fixed-step explicit integrator (Choupo's RK4 wrapper around `DynamicCSTR`) the simplest discretisation is rectangular for the integral and backward finite difference for the derivative:

$$I_{n+1} = I_n + e_n\,h, \qquad \dot{\text{PV}}_n \approx (\text{PV}_n - \text{PV}_{n-1})/h.$$

On the very first step the derivative cannot be evaluated — there is no previous PV — so Choupo zeros it (`primed` flag). This avoids an artificial spike at $t = 0$. The controller's output is held *constant* across the RK4 stages within the step (zero-order hold), which is acceptable when $h \ll \tau$.

Runnable case. ctrl/ctrl02_disturbance_rejection Same plant + PID as `ctrl01`, but a `Schedule` controller injects step disturbances of $\pm 15\,\mathrm{K}$ in T_{in} at $t = 700\,\mathrm{s}$ and $t = 1300\,\mathrm{s}$. The PID has no foreknowledge — it only sees T_{reactor} drift away from setpoint. Peak excursion after each step is $\approx 0.7\,\mathrm{K}$, settling to within $10^{-3}\,\mathrm{K}$ of setpoint in $\approx 200\,\mathrm{s}$.

Open-loop schedules: the `ScheduleController`

A degenerate "controller" that ignores the measurement and writes a piecewise-constant MV from a list of (time, value) pairs slots into the same `Controller` infrastructure. Choupo uses it for two purposes: (1) injecting scripted disturbances on T_{in} to test closed-loop disturbance rejection (above); (2) running open-loop ramps (e.g. linearly increasing feed rate over time) without writing a new unit op. The user-side dict syntax (`type Schedule;`, `schedule ( time; value; ...);`) is documented in the User Guide §8.2.

The forcing vocabulary culminates in the *PRBS* — the classic system-identification input. A maximal-length Fibonacci LFSR of n registers generates a binary sequence of period exactly $2^n - 1$ bits; its autocorrelation approximates an impulse, so one record excites the plant across the band and carries the information for a full low-order model fit, where a sinusoid probes a single frequency. Choupo's `type prbs;` is deterministic by construction — the seed and the feedback taps are *declared* in the dict (taps may come from the versioned canonical table of primitive polynomials), the initialiser *walks* the register and refuses any tap set whose cycle is not exactly $2^n - 1$, and the announce line prints the leading bits so the student can derive the same sequence by hand. Randomness at run time would make the experiment unrepeatable; a declared LFSR makes it exact.

Runnable case. ctrl/ctrl08_prbs_ident $T_{\mathrm{in}} = 320 \pm 10\,\mathrm{K}$ by an $n = 4$ LFSR (period 15 bits $\times$ 120 s), the hand-derived sequence `000100110101111` in the case header matching the announce; seed 0 and non-maximal taps refuse aloud.

6 Numerical integration of stiff systems

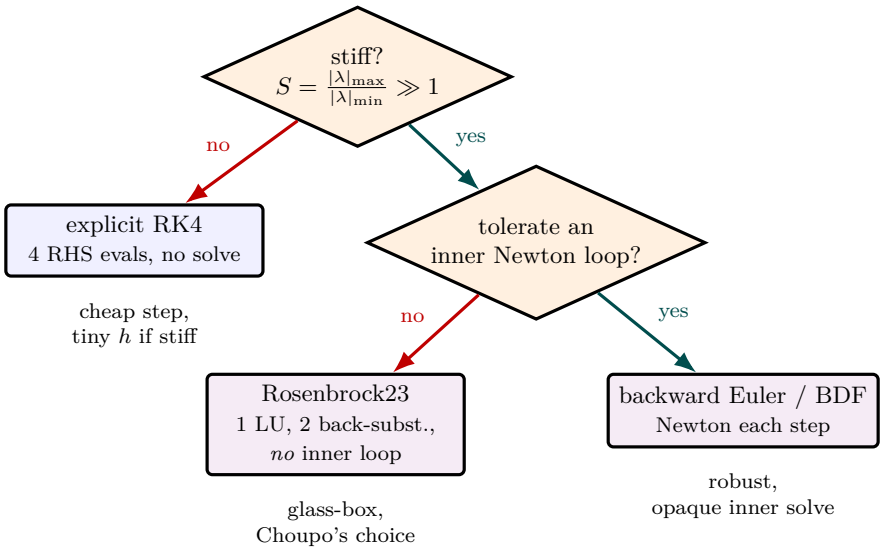

Figure 87: The integrator decision tree. If the eigenvalue spread S is $\mathcal{O}(1)$ the system is non-stiff and explicit RK4 wins — four right-hand-side evaluations, nothing solved. Once $S \gg 1$ an explicit method is hostage to the dead fast mode (Figure 89), so an implicit method is required, and the only remaining question is whether to pay for an inner Newton iteration. Choupo takes the left branch: the *linearly-implicit* Rosenbrock23 keeps full implicit stability but replaces the inner loop with one LU factorisation reused by two back-substitutions — the trait that keeps the stiff solver as readable as the non-stiff one.

> *What you'll learn.* The RK4 chapters (Sections 9 and 1) integrate $d\mathbf{Y}/dt = f(\mathbf{Y})$ with a fixed explicit step. That works while every state variable changes on a comparable timescale. A reacting mixture violates this: a radical can be born and consumed a million times faster than the bulk heats up. The system is *stiff*, and an explicit method must either crawl or explode. This chapter explains stiffness, why RK4 fails on it, and the linearly-implicit *Rosenbrock* cure Choupo uses.

6.1 The initial-value problem and explicit stepping

A closed reactor's state obeys an initial-value problem

$$\frac{d\mathbf{Y}}{dt} = f(\mathbf{Y}), \qquad \mathbf{Y}(0) = \mathbf{Y}_0, \tag{450}$$

with $\mathbf{Y}$ the packed mole numbers (and temperature) of Section 1. The simplest integrator is explicit (forward) Euler, $\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\, f(\mathbf{Y}_n)$; RK4 (435) is the same idea with four stages and a fourth-order local error. Both are *explicit*: the new state is an arithmetic combination of right-hand sides already evaluated at known states. Nothing is solved; the step is cheap.

The price of that cheapness is a stability limit. Linearise (450) about a point, $\delta\mathbf{Y}' = \mathbf{J}\,\delta\mathbf{Y}$ with $\mathbf{J} = \partial f/\partial\mathbf{Y}$ the Jacobian. An explicit step multiplies each eigenmode λ of $\mathbf{J}$ by an amplification factor; for forward Euler that factor is $1 + h\lambda$. Stability ($|1 + h\lambda| \le 1$) requires

$$h \lesssim \frac{2}{|\lambda|_{\max}}, \tag{451}$$

regardless of accuracy. The fastest-decaying mode dictates the step even after it has died out and no longer contributes anything to the answer.

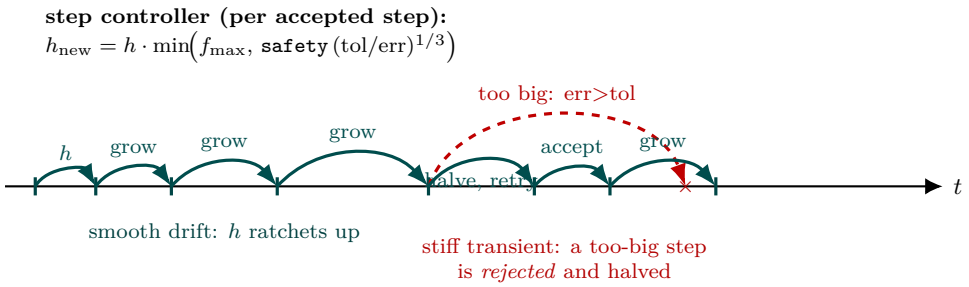

Figure 88: The adaptive step-size trajectory of Rosenbrock23 over several steps (the per-step internals are in Figure 91). On the smooth drift the embedded error stays well below tolerance, so after each accepted step the controller *grows* h (green arcs ratchet wider) — $h_{\text{new}} = h \cdot \min(f_{\max}, \texttt{safety}\,(\text{tol}/\text{err})^{1/3})$ for the order-2/3 pair. When the solution hits a stiff transient a trial step overshoots tolerance: it is *rejected* ($\times$), h is halved, the matrix $\mathbf{W}$ refactorised, and the step retried until it is accepted (red dashed $\rightarrow$ short green arcs). The integrator thus spends large steps where nothing happens and small steps exactly where the action is — automatically, from the embedded error alone.

6.2 What stiffness is

Stiffness is a separation of timescales. Define the stiffness ratio

$$S \;=\; \frac{|\lambda|_{\max}}{|\lambda|_{\min}}, \tag{452}$$

the spread of the Jacobian's eigenvalues (with negative real parts). When $S \sim 1$ the system is non-stiff and RK4 is ideal. When $S \gg 1$ it is stiff.

Chemical kinetics is the textbook offender. In a hydrogen flame the radicals H, O, OH reach a quasi-steady balance on a $\sim 10^{-8}$ s timescale, while the heat release that actually moves the temperature runs on $\sim 10^{-3}$ s. The ratio $S \sim 10^5$–10^6 is built into the physics, not an artefact. By (451) an explicit method must take steps of order 10^{-8} s to stay stable, yet the interesting evolution lasts milliseconds: tens of millions of steps, each limited by a mode that contributes nothing to the answer. Push the step any larger and the amplification factor exceeds one — the trace radical, driven negative, feeds a rate law $\prod c_j^{m_j}$ that overflows to a non-finite number within a few steps.

> *Intuition.* Stiffness is not "the problem is hard"; it is "the problem hides a fast spring inside a slow drift." An explicit integrator has to resolve the spring even when you only care about the drift. An implicit one lets the spring relax to its equilibrium each step and follows the drift.

6.3 The implicit cure

Backward (implicit) Euler, $\mathbf{Y}_{n+1} = \mathbf{Y}_n + h\, f(\mathbf{Y}_{n+1})$, has amplification factor $1/(1 - h\lambda)$, which is below one for *every* h when $\operatorname{Re}\lambda < 0$. It is unconditionally stable — the step is set by accuracy, not by the dead fast mode. The cost is that $\mathbf{Y}_{n+1}$ now appears on both sides: each step solves a nonlinear system, normally by an inner Newton iteration.

A *linearly-implicit* method keeps the stability and removes the inner loop by linearising f once. The semi-implicit Euler step,

$$(\,\mathbf{I} - h\,\mathbf{J}\,)\,\Delta\mathbf{Y} = h\, f(\mathbf{Y}_n), \qquad \mathbf{Y}_{n+1} = \mathbf{Y}_n + \Delta\mathbf{Y}, \tag{453}$$

is one matrix assembly and one linear solve — no Newton iteration. It is the readable "hello, stiffness" stepping-stone, first order and without error control.

6.4 Rosenbrock23: the scheme Choupo uses

Choupo's stiff integrator is a Rosenbrock–Wanner method of order two with an embedded third-order estimate (the "ode23s" / Rosenbrock23 family). It generalises (453) to several stages that all reuse the

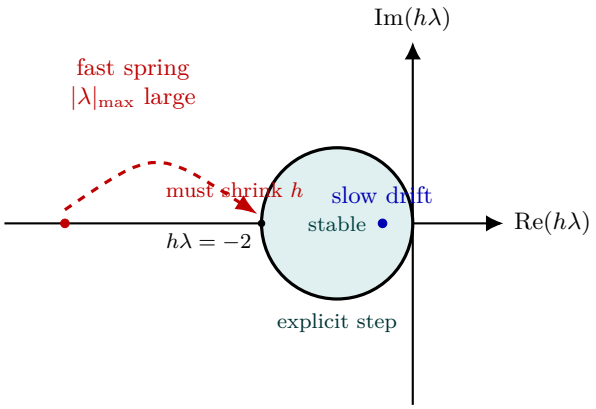

Figure 89: The stability region of an explicit step (forward Euler shown: the disc $|1 + h\lambda| \le 1$, centre -1, radius 1). Every eigenmode λ of the Jacobian, multiplied by the step h, must land *inside* the disc or that mode is amplified and the integration blows up. A slow drift mode (blue) sits comfortably inside for almost any h; but the fast spring (red, large $|\lambda|_{\max}$) forces $h \lesssim 2/|\lambda|_{\max}$ to drag it back across the boundary at $h\lambda = -2$ — even after that mode has died and stopped contributing to the answer. That is stiffness: the dead fast mode, not accuracy, sets the step. The implicit cure (Rosenbrock23, Figure 91) has *no* such boundary in the left half-plane.

same factorised matrix $\mathbf{W} = \mathbf{I} - h\gamma\mathbf{J}$:

$$\begin{aligned} \mathbf{W}\,\mathbf{k}_1 &= f(\mathbf{Y}_n), \\ \mathbf{W}\,\mathbf{k}_2 &= f(\mathbf{Y}_n + a\,h\,\mathbf{k}_1) - c\,\mathbf{k}_1, \\ \mathbf{Y}_{n+1} &= \mathbf{Y}_n + h\,(b_1\mathbf{k}_1 + b_2\mathbf{k}_2), \end{aligned} \tag{454}$$

with the scheme constants γ, a, c, b_i fixed by the order conditions. The properties that matter:

- **L-stable** — not merely stable but *strongly* damping of the fast modes, so a stiff transient is absorbed rather than rung.
- **One Jacobian, one LU factorisation per step.** The factorisation of $\mathbf{W}$ is reused across the stages; each stage is a cheap back-substitution. Choupo's hand-rolled `luFactor`/`luSolve` (split out of the Gauss solver) does this.
- **No inner Newton loop** — unlike backward Euler or a BDF method, the stages are linear in the unknowns. This is what keeps the code short and glass-box.
- **Embedded error estimate** — the difference between the order-2 and order-3 combinations gives a local error per step that drives an adaptive h: shrink when the error exceeds the tolerance, grow when it is comfortably below.

6.5 The Jacobian, and two correctness traps

The method needs $\mathbf{J} = \partial f/\partial \mathbf{Y}$. Choupo forms it by finite differences (a perturbation of each state in turn) with an analytic option; either is acceptable, but two details are not optional in a chemical system:

- **Per-component scaling.** The state mixes mole numbers that span orders of magnitude (a bath gas at $\mathcal{O}(1)$, a radical at $\mathcal{O}(10^{-10})$) with a temperature row in kelvin. A single scalar tolerance lets the abundant species drive the step controller and *under-resolve the trace radical that is the whole event.* The error norm $\|\mathrm{err}/(\mathrm{atol} + \mathrm{rtol}\,|\mathbf{Y}|)\|$ and the finite-difference perturbation must both be scaled component by component.
- **Positivity rejection.** A step that drives a concentration negative is rejected and halved, because the next rate evaluation would raise a negative number to a fractional order. The embedded controller does not give this for free.

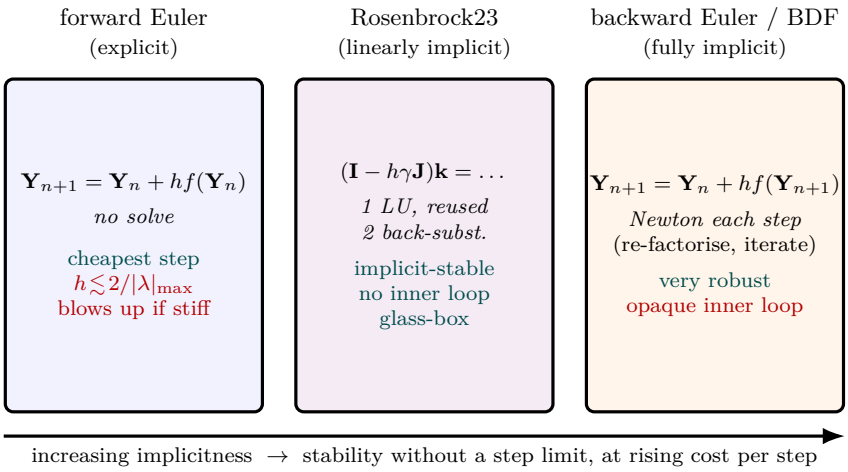

Figure 90: Three rungs of implicitness for the stiff IVP $d\mathbf{Y}/dt = f(\mathbf{Y})$. Forward Euler (left) solves nothing per step but is hostage to the stability limit $h \lesssim 2/|\lambda|_{\max}$ and overflows on a stiff system. Backward Euler/BDF (right) is unconditionally stable but pays an *inner Newton iteration* every step — robust, but the iteration is the opaque box Choupo wants to avoid. Rosenbrock23 (centre) is the *linearly-implicit* compromise: it linearises f once, assembles $\mathbf{W} = \mathbf{I} - h\gamma\mathbf{J}$, factorises it *once*, and reuses the factorisation across both stages — keeping full implicit stability *without* an inner loop. That is why Choupo's stiff solver (Figure 91) reads almost as plainly as its RK4.

Worked example 6.1. The Robertson problemThe standard stiff benchmark is Robertson's autocatalytic system

$$\dot{y}_1 = -0.04\,y_1 + 10^4\,y_2 y_3, \quad \dot{y}_2 = 0.04\,y_1 - 10^4\,y_2 y_3 - 3{\times}10^7 y_2^2, \quad \dot{y}_3 = 3{\times}10^7 y_2^2,$$

with $\mathbf{y}(0) = (1, 0, 0)$, integrated to $t = 10^5$. The intermediate y_2 shoots up on a microsecond scale then decays across five decades of time — stiffness ratio $\sim 10^6$. Rosenbrock23 integrates it in a couple of thousand adaptive steps, conserving $y_1 + y_2 + y_3 = 1$ to round-off and matching a reference solution to four figures. Explicit RK4 at any practical fixed step overflows to a non-finite value: the lesson that the implicit machinery earns its complexity.

Intuition. RK4 is still the right default for a mild, non-stiff trajectory: it is cheaper per step and needs no Jacobian. Choupo keeps it as a selectable integrator and announces, at run time, the stiffness it measures — so a student *sees* why a particular problem needed the Rosenbrock solver rather than being told.

The classic references are E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (Springer, 2nd ed. 1996), and A. Sandu et al., "Benchmarking stiff ODE solvers for atmospheric chemistry problems," *Atmos. Environ.* **31** (1997) 3151.

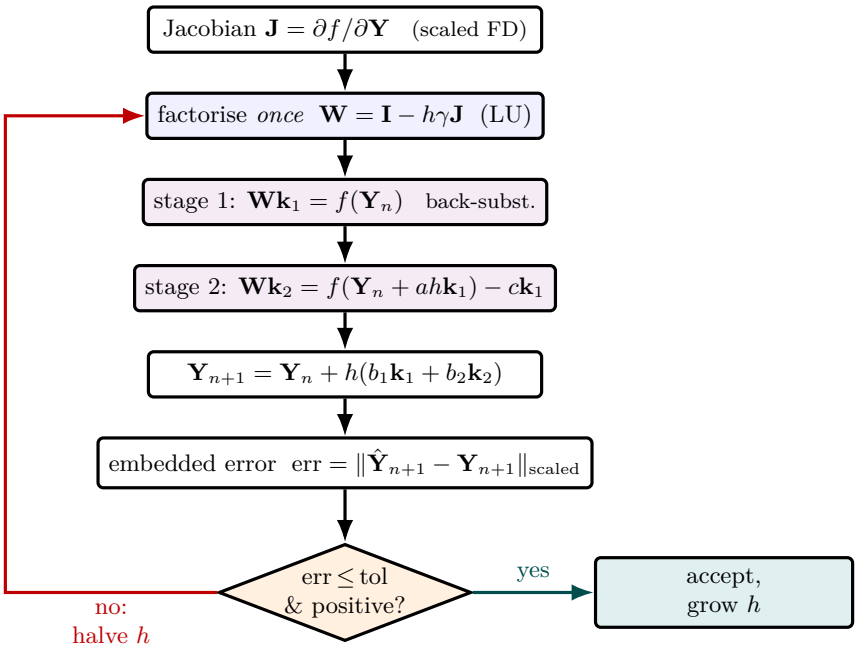

Figure 91: One adaptive step of Rosenbrock23, the stiff integrator (`src/solver/Rosenbrock23`). The key economy is the blue box: the matrix $\mathbf{W} = \mathbf{I} - h\gamma\mathbf{J}$ is LU-factorised *once* per step and then *reused* by both linear stages (violet) — each stage is just a back-substitution, and because the stages are *linear* in the unknowns there is no inner Newton loop (the trait that keeps the code glass-box, unlike backward-Euler/BDF). The order-2 update and an embedded order-3 estimate $\hat{\mathbf{Y}}$ differ by the local error, which drives the controller: if the (component-scaled) error is within tolerance *and* no concentration went negative, the step is accepted and h grows; otherwise h is halved, $\mathbf{W}$ refactorised, and the step retried (red). L-stability means a stiff transient is *absorbed*, not rung — where fixed-step RK4 would overflow on the Robertson benchmark.

7 Chemical kinetics in the gas phase

What you'll learn. The reactor chapters took a reaction rate as given. This chapter is the rate itself: the Arrhenius law and its modified form, the pressure-dependent corrections (third body, fall-off), and — the piece most often got wrong — how a reversible reaction's reverse rate is fixed by thermodynamics, with the concentration-basis factor that an isobaric mole-changing reaction must carry.

7.1 Mass action and the Arrhenius law

For an elementary reaction the rate is the law of mass action: a product of reactant concentrations raised to their stoichiometric orders, times a temperature-dependent rate coefficient,

$$r \;=\; k(T)\,\prod_{j} c_j^{\,m_j}. \tag{455}$$

The coefficient follows the Arrhenius law

$$k(T) \;=\; A\,\exp\!\Big(-\frac{E_a}{R_{\mathrm{g}}T}\Big), \tag{456}$$

with A the pre-exponential factor and E_a the activation energy. Collision theory and transition-state theory both predict a weak extra temperature dependence in the prefactor, captured by the *modified Arrhenius* form

$$k(T) \;=\; A\,T^{\,b}\,\exp\!\Big(-\frac{E_a}{R_{\mathrm{g}}T}\Big), \tag{457}$$

where the exponent b is typically between -1 and $+3$. Choupo reads A, b, E_a from the `kinetics` block; published mechanisms quote them in CHEMKIN units (cm–mol–s, cal/mol), which the parser converts to SI on load and announces, so the conversion is a visible step rather than a hidden one.

7.2 Third-body and pressure fall-off reactions

Some reactions need a collision partner M to carry away energy — a recombination such as $\mathrm{H} + \mathrm{O_2} + \mathrm{M} \rightarrow \mathrm{HO_2} + \mathrm{M}$. The rate scales with the total third-body concentration, weighted by per-species collision efficiencies ε_j (unlisted species take $\varepsilon = 1$):

$$[\mathrm{M}] \;=\; \sum_{j} \varepsilon_j\, c_j. \tag{458}$$

At high pressure the partner is always available and the reaction saturates to a pressure-independent (high-pressure-limit) rate; at low pressure it is starved and the rate is proportional to [M]. The transition between the two limits is the *fall-off* region. The simplest description (Lindemann) blends the low- and high-pressure coefficients k_0 and k_∞ through the reduced pressure $P_r = k_0[\mathrm{M}]/k_\infty$:

$$k \;=\; k_\infty\,\frac{P_r}{1+P_r}\,F, \tag{459}$$

with $F = 1$ for Lindemann. The Troe form supplies a more accurate broadening factor $F(P_r, T)$ from three or four tabulated parameters. This is why a gas-phase rate can depend on pressure even though (456) contains only temperature.

7.3 Reversible reactions: the reverse rate from thermodynamics

A reversible reaction must relax to chemical equilibrium, so its forward and reverse coefficients are not independent — their ratio is the equilibrium constant. Kinetics gives the forward k_f; thermodynamics gives the reverse,

$$k_r \;=\; \frac{k_f}{K_c}, \tag{460}$$

where K_c is the equilibrium constant *on a concentration basis*. The subtlety — and a standing trap — is that thermodynamics naturally delivers the constant on a *pressure* basis. From the standard-state Gibbs energies of reaction,

$$K_p \;=\; \exp\!\Big(-\frac{\sum_i \nu_i\, g_i^{\circ}}{R_{\mathrm{g}}T}\Big), \tag{461}$$

with ν_i the (signed) stoichiometric coefficients and g_i° the standard-state molar Gibbs energies the property layer already integrates from C_p, ΔH_f and S°. Converting to the concentration basis uses $c_i = P_i/(R_\mathrm{g}T)$ for an ideal gas, which introduces a factor for every net mole produced:

$$\boxed{K_c \;=\; K_p \left(\frac{P^\circ}{R_\mathrm{g}T}\right)^{\Delta\nu}, \qquad \Delta\nu = \sum_i \nu_i.} \tag{462}$$

Common pitfall. The $(P^\circ/R_\mathrm{g}T)^{\Delta\nu}$ factor is *mandatory* whenever $\Delta\nu \neq 0$. Applying K_p directly to a concentration product — omitting the factor — silently corrupts the reverse rate of every mole-changing reversible gas reaction (a recombination, $\Delta\nu = -1$; a dissociation, $\Delta\nu = +1$). It is invisible on a mole-conserving reaction such as the water–gas shift ($\mathrm{CO + H_2O \rightleftharpoons CO_2 + H_2}$, $\Delta\nu = 0$, factor $= 1$), which is exactly why the bug can hide for a long time. Choupo prints K_p, $\Delta\nu$ and K_c at verbosity 3 so the conversion is auditable.

Worked example 7.1. A chain-branching stepThe single most important reaction in hydrogen and hydrocarbon ignition is the chain-branching step $\mathrm{H + O_2 \rightleftharpoons O + OH}$. It is mole-conserving ($\Delta\nu = 0$), so $K_c = K_p$, and its forward coefficient has a strongly non-Arrhenius shape captured by the modified form (457) with a negative b. Its reverse, $\mathrm{O + OH \to H + O_2}$, is not transcribed independently: Choupo computes it from (460) using the cited thermodynamics of H, $\mathrm{O_2}$, O, OH. One forward triple plus the species' Gibbs data fixes both directions consistently — the reverse can never drift out of thermodynamic agreement.

A note on data and provenance. The numerical rate constants of a detailed mechanism are measured facts, each traceable to a primary kinetics study, and a value cited to its primary source is the honest unit of reuse. Choupo therefore authors a mechanism reaction by reaction from the primary literature rather than vendoring a compiled mechanism file, whose particular arrangement may carry its compiler's licence. Standard graduate texts for this chapter are C. K. Law, *Combustion Physics* (Cambridge, 2006), and S. R. Turns, *An Introduction to Combustion* (McGraw-Hill).

8 Polymerisation: chain statistics from conversion

What you'll learn. A polymerisation reactor's interesting outputs are not concentrations but *chain statistics*: how long the chains are on average, and how broad the spread is. For step-growth polymerisation these follow in closed form from a single number, the conversion p — a clean glass-box lesson in reaction engineering, where the whole molar-mass distribution is an analytic curve rather than the output of an engine.

8.1 Carothers: degree of polymerisation from conversion

In step-growth polymerisation (a diacid and a diol esterifying to a polyester, releasing water) any two functional groups can react. Let p be the fraction of functional groups that have reacted — the conversion. Carothers' counting argument gives the number-average degree of polymerisation, the mean number of repeat units per chain,

$$\bar{X}_n \;=\; \frac{1}{1-p}. \tag{463}$$

The divergence as $p \to 1$ is the central fact of step-growth chemistry: a high polymer demands near-complete conversion. At $p = 0.99$, $\bar{X}_n = 100$; reaching $\bar{X}_n = 200$ needs $p = 0.995$. The number-average molar mass follows from the repeat-unit mass M_0,

$$M_n \;=\; M_0\,\bar{X}_n \;=\; \frac{M_0}{1-p}. \tag{464}$$

8.2 The distribution and the polydispersity index

A mean is not the whole story — chains of every length coexist. For the equal-reactivity, uniform-time case the probability that a chain is exactly x units long is the *Flory–Schulz* (most-probable) distribution,

$$n(x) \;=\; (1-p)\,p^{\,x-1}, \tag{465}$$

the mole fraction, monotonically decaying in x; the corresponding weight fraction

$$w(x) \;=\; x\,(1-p)^2\,p^{\,x-1} \tag{466}$$

is peaked near $x \approx \bar{X}_n$ and broadens as p rises. The weight-average degree of polymerisation and molar mass are

$$\bar{X}_w \;=\; \frac{1+p}{1-p}, \qquad M_w \;=\; M_0\,\frac{1+p}{1-p}. \tag{467}$$

The ratio of the two averages is the polydispersity index, the single number that reports the breadth of the distribution,

$$\boxed{\mathrm{PDI} \;=\; \frac{M_w}{M_n} \;=\; 1+p \;\xrightarrow[p\to 1]{}\; 2.} \tag{468}$$

A step-growth polymer taken to high conversion always approaches a polydispersity of two. Watching $w(x)$ broaden while PDI climbs toward two is what makes the number mean something.

Common pitfall. Equations (465)–(468) assume every chain has seen the *same reaction time* — true in a batch vessel or an ideal plug-flow reactor. A continuous stirred tank imposes a residence-time distribution: some chains leave young, some old, and the molar-mass distribution is broader than Flory–Schulz, with $\mathrm{PDI} > 1 + p$. Printing $\mathrm{PDI} = 1 + p$ next to a CSTR result would teach a visibly wrong equation. Choupo therefore reports the bare conversion-only averages (p, $\bar{X}_n$, M_n, which are residence-time independent) on any reactor, but gates the distribution and $\mathrm{PDI} = 1 + p$ to the batch / plug-flow case.

Worked example 8.1. Polyesterification to high conversionA diacid and diol esterify in a plug-flow reactor to $p = 0.99$ with a repeat-unit mass $M_0 = 0.111\,\mathrm{kg/mol}$. Then $\bar{X}_n = 1/(1-0.99) = 100$, $M_n = 11.1\,\mathrm{kg/mol}$, $\bar{X}_w = 1.99/0.01 = 199$, $M_w = 22.1\,\mathrm{kg/mol}$, and $\mathrm{PDI} = 1.99$. The Flory–Schulz weight curve peaks near 100 repeat units with a long tail. Sweeping the reactor volume traces PDI

climbing from near 1 toward 2 as p rises — the entire Carothers/Flory story in one run.

These results are derived from a small set of closed-form algebraic relations layered on the existing reactor conversion, not from a new engine, which is why they cost almost nothing and stay transparent. The primary references are P. J. Flory, *Principles of Polymer Chemistry* (Cornell University Press, 1953), Chs. VIII–IX; G. Odian, *Principles of Polymerization* (Wiley, 4th ed.), Ch. 2; and H. S. Fogler, *Elements of Chemical Reaction Engineering* (Pearson, 5th ed.), §9.